

The Backend Engineer's Library

Python and CPython Internals

Volume 16

Contents

CPython Architecture: Source to Execution

Objects, Reference Counting, and the PyObject System

Bytecode, the ceval Loop, and Adaptive Specialization (PEP 659)

Memory Management: pymalloc, GC, Arenas, and Immortal Objects

The GIL: Mechanics, Evolution, Per-Interpreter GIL and Free-Threaded Python (PEP 684 / PEP 703)

The Type System: Classes, MRO, Descriptors, Slots, and the Attribute Protocol

Functions, Closures, Generators, Coroutines, and async/await

Exceptions, Context Managers, and the Unwinding Machinery

The Import System: importlib, Finders, Loaders, and Namespace Packages

C Extensions, the C API, HPy, and Embedding CPython

Performance: Profiling, the Copy-and-Patch JIT (PEP 744), Cython, and Alternative Runtimes

Packaging, Distribution, and Production Deployment at Scale

Chapter 1 — CPython Architecture: Source to Execution

What this chapter covers. Python is the glue of most backend stacks — API services, data pipelines, ML inference, orchestration, and operational tooling all embed or shell out to CPython. Yet few backend engineers can explain what happens between `python app.py` and the first line of user code executing, or why the same `.py` file behaves differently when `__pycache__` is present, absent, or stale. This chapter traces the entire path from source text to execution: how CPython is organized on disk, how source is tokenized, parsed with a PEG parser, lowered to an AST and then to bytecode, how that bytecode is cached and loaded, and how the interpreter itself boots and enters its evaluation loop. You will use the same introspection tools core developers use — `tokenize`, `ast`, `dis`, `marshal`, and `importlib` — on real code.

Learning goals — after this chapter you should be able to:

- Navigate the CPython repository layout (`Parser/`, `Python/`, `Objects/`, `Include/`, `Lib/`, `Modules/`) and explain what each top-level directory owns.
- Trace the compilation pipeline end-to-end: `tokenize.c` → PEG parser (`pegen`) → `ast.c` / AST → `symtable.c` → `compile.c` → `PyCodeObject` → `ceval.c` evaluation loop.
- Explain the tokenizer's job (bytes → tokens, encoding cookie, `INDENT` / `DEDENT` / `NL` / `NEWLINE` / `ENDMARKER`) and show its output on real source.
- Contrast the PEG parser (PEP 617, since 3.9) with the old `pgen` / `LL(1)` parser: why the switch happened, what `Grammar/python.gram` is, and how `pegen` generates `parser.c`.
- Describe AST nodes, `ast.parse` / `ast.dump` / `python -m ast`, the symbol table pass, and the code object (`PyCodeObject`) layout including `co_code`, `co_consts`, `co_names`, `co_varnames`, `co_exceptiontable`, and `co_qualname`.
- Explain `.pyc` / `__pycache__` / `marshal`, the 16-byte header (magic, flags, mtime/hash, size/hash), PEP 552 deterministic `pyc` modes, and `SOURCE_DATE_EPOCH` reproducible builds.

- Describe the import bootstrap (`_bootstrap.py` / `_bootstrap_external.py`, `finders`, `loaders`, `sys.meta_path` / `sys.path_hooks` / `sys.path`) and how the interpreter boots (`Py_Initialize` / `pylifecycle.c` / `PyConfig` / `initconfig.c`).
- Evaluate `__pycache__` and `.pyc` handling in containers and hermetic deploys: when to keep, strip, or pre-compile bytecode, and how it affects startup, image size, and reproducibility.

Prerequisites. Volume 13, Chapter 6 is the service-level view of the Python runtime (GIL, GC, performance posture). Volume 1, Chapters 8-9 (data layout, floats) and Volume 2, Chapter 4 (allocators) are useful background for later chapters. This chapter assumes you can read C at a skim and have run `python -m dis` at least once.

1. CPython as the reference interpreter

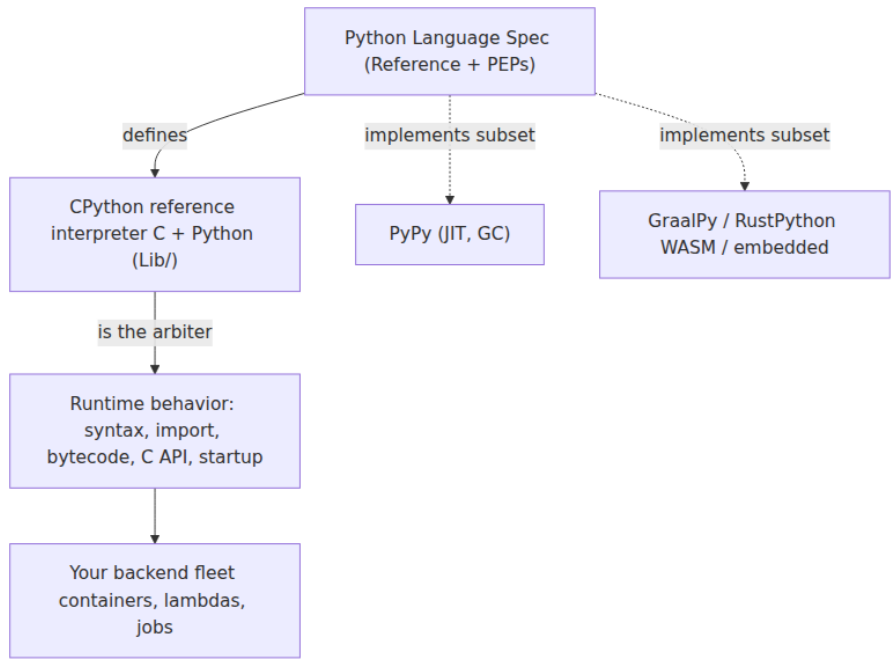
Python the language is defined by the [Language Reference](#) and a stream of PEPs. CPython — the C implementation hosted at github.com/python/cpython — is the reference interpreter: when the reference and an alternative runtime disagree, CPython wins. Practically every "Python" your backend runs is CPython unless you opted into PyPy, GraalPy, or a WASM build explicitly.

Why CPython internals matter to a backend engineer even if you never patch the interpreter:

- **Startup and import dominate cold-start latency.** Serverless functions, Kubernetes jobs, CLIs, and autoscaled web workers all pay import cost. Understanding `pyc` caching, `PYTHONDONTWRITEBYTECODE`, and the import lock explains real tail-latency.
- **Bytecode is not an implementation detail you can ignore.** Debuggers, profilers, coverage tools, import hooks, `dataclasses`, `pydantic`, and `pytest` all read or rewrite `PyCodeObject`s.
- **Reproducibility and supply-chain hygiene start at the artifact.** Whether a `.pyc` embeds a timestamp or a hash decides if two builds from the same source are byte-identical — relevant to SLSA provenance, container attestations, and cache invalidation.
- **Compatibility is defined by CPython's choices.** Syntax, import semantics, `sys.path` initialization, `PYTHONPATH` / `PYTHONHOME` / `-E` / `-I`

isolation flags, and `audit hooks` are specified by what CPython actually does.

CPython follows a yearly release cadence (3.11, 3.12, 3.13, ...), each with a single `main` branch and a `3.x` stable branch after feature freeze. Minor versions are ABI-stable for C extensions compiled against the Stable ABI (`Py_LIMITED_API`) but not for bytecode — `.pyc` magic numbers change on every feature release.



2. Repository layout — where things live

Clone `github.com/python/cpython` and the top level is immediately informative. Every directory maps to a phase of the pipeline or a layer of the runtime.

Path	Owns	Key files
<code>Parser/</code>	PEG parser machinery	<code>pegen/</code> , <code>token.c</code> , <code>pegen.c</code> , <code>parser.c</code> (generated), <code>Python.asdl</code>

Path	Owns	Key files
Grammar/	Grammar definition	<code>python.gram</code> (PEG grammar), <code>Tokens</code>
Python/	Interpreter core + lifecycle	<code>ceval.c</code> , <code>compile.c</code> , <code>ast.c</code> , <code>symtable.c</code> , <code>pylifecycle.c</code> , <code>import.c</code> , <code>Python/tokenize.c</code>
Objects/	Object system	<code>object.c</code> , <code>longobject.c</code> , <code>unicodeobject.c</code> , <code>dictobject.c</code> , <code>frameobject.c</code> , <code>codeobject.c</code>
Include/	Public + internal headers	<code>Python.h</code> , <code>cpython/code.h</code> , <code>internal/pycore_*.h</code>
Lib/	Standard library (Python)	<code>importlib/</code> , <code>ast.py</code> , <code>dis.py</code> , <code>tokenize.py</code> , <code>py_compile.py</code>
Modules/	C stdlib extensions	<code>_io/</code> , <code>_json</code> , <code>posixmodule.c</code> , <code>gcmodule.c</code>
PC/ / Mac/	Platform shims	Windows/macOS build glue
Tools/	Dev tooling	<code>peg_generator/</code> (the <code>pegen</code> that emits <code>parser.c</code>)

A few details that catch newcomers:

- **The tokenizer lives in `Parser/token.c` and `Python/tokenize.c`.** The C tokenizer is the one the interpreter uses; `Lib/tokenize.py` is a pure-Python reimplementaion used by tools (formatters, linters) and must stay compatible.
- **`Grammar/python.gram` is the source of truth for syntax.** It is a PEG grammar consumed by `Tools/peg_generator/pegen` to emit `Parser/parser.c` (~10k+ lines, regenerated, not hand-edited).
- **`Python/ceval.c` is the evaluation loop.** Historically a giant `switch` over opcodes; since 3.11 it includes adaptive specialization and inline caches (PEP 654/659 territory — Chapter 3).
- **`Objects/codeobject.c` defines `PyCodeObject`.** The immutable container for bytecode and its metadata; `Lib/dis.py` knows how to pretty-print it.

- **Lib/importlib/_bootstrap.py** and **_bootstrap_external.py** are **frozen**. They are compiled to bytecode, marshalled, and baked into the interpreter binary (`Python/importlib.h` , generated by `Tools/build/freeze_modules.py`) so import can bootstrap itself before the filesystem importer exists.

```

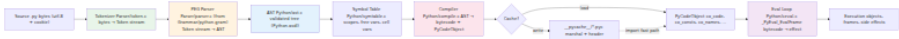
cpython/
├── Grammar/python.gram      # PEG grammar (source of truth)
├── Parser/
│   ├── token.c             # tokenizer (C)
│   ├── pegen/              # PEG engine runtime
│   └── parser.c            # ← GENERATED from python.gram
├── Python/
│   ├── ast.c               # AST construction + validation (from
Python.asdl)
│   ├── symtable.c          # symbol table (scope analysis)
│   ├── compile.c           # AST → bytecode → PyCodeObject
│   ├── ceval.c             # bytecode evaluation loop
│   └── pylifecycle.c
# Py Initialize / Py_Finalize / runtime init
├── import.c               # import machinery (C side)
├── tokenize.c             # tokenizer helpers
├── Objects/
│   ├── codeobject.c        # PyCodeObject
│   ├── frameobject.c      # PyFrameObject
│   └── ...                 # one file per built-in type
├── Include/
│   ├── Python.h
│   ├── cpython/code.h
│   └── internal/pycore_*.h
├── Lib/
│   ├── importlib/_bootstrap*.py # frozen import system
│   ├── ast.py / dis.py / tokenize.py
│   └── ...
└── Modules/                # C extensions (_json, _io, gc, ...)

```

Lab tip. After cloning, run `make regen-all` (or `make regen-pegen / make regen-ast`) to regenerate `Parser/parser.c`, `Include/internal/pycore_ast.h`, and friends from `Grammar/python.gram` and `Parser/Python.asdl`. If you edit the grammar, you must regenerate.

3. The pipeline at a glance

Source text becomes running code in a strict, observable pipeline. Every stage has a C file (or pair of files) and a Python-level introspection hook.



What to notice:

- **Tokenize → Parse → AST is lossless-to-lossy.** The tokenizer preserves every byte (including whitespace, comments, and encoding) as tokens; the parser discards whitespace/comments and builds structure; the AST discards parentheses and other syntactic sugar and keeps semantics.
- **Symtable is a separate pass between AST and compile.** Scope analysis must complete before code generation because `LOAD_FAST` vs `LOAD_GLOBAL` vs `LOAD_DEREF` depends on whether a name is local, global, free, or cell.
- **Bytecode is the import cache unit.** `compile.c` emits a `PyCodeObject`; `marshal` serializes it to `.pyc` (plus a header). On reimport the interpreter can skip tokenize/parse/compile entirely if the cache validates.
- **ceval.c never sees source.** It sees only `PyCodeObject`s and `PyFrameObject`s. Tracebacks reconstruct source locations from `co_positions()` / `co_lines()` tables.

The next sections walk each stage with real commands and outputs.

4. Tokenizer — bytes to token stream

What it does

`Parser/token.c` (with helpers in `Python/tokenize.c`) converts raw bytes into a stream of `TokenInfo` values. It handles:

- **Encoding detection.** Reads the encoding cookie (`# -*- coding: utf-8 -*-` or `# coding: latin-1`), BOM, and defaults to UTF-8 (PEP 3120). Wrong encoding → `SyntaxError: unknown encoding`.
- **Logical vs physical lines.** Joins continuation lines (`\` + newline, inside brackets), then emits `NL` for non-terminating newlines (inside

brackets) and `NEWLINE` for statement terminators. This is why `tokenize.generate_tokens` distinguishes `NL` (type 61) from `NEWLINE` (type 4).

- **Indentation.** A stack tracks indent levels; increasing indent emits `INDENT`, decreasing emits one or more `DEDENT`s. Tabs are expanded as 8-space stops for indent comparison (but preserved in the token string).
- **Literals and operators.** Recognizes `NUMBER`, `STRING` (including `f"..."`, `b"..."`, `r"..."` prefixes and triple quotes), `NAME`, `OP`, `COMMENT`, `ENCODING`, and finally `ENDMARKER`.

The tokenizer is intentionally dumb about grammar — it does not know that `x = 1` is an assignment. It just emits `NAME("x")`, `OP("=")`, `NUMBER("1")`, `NEWLINE`.

Tokenizer example — real output

Use the stdlib `tokenize` module (which mirrors the C tokenizer) to inspect any file:

```
import io, tokenize

src = """\
def greet(name: str) -> str:
    return f"hello {name}"
"""

for tok in tokenize.generate_tokens(io.StringIO(src).readline):
    print(tok)
```

Output (Python 3.11):

```

TokenInfo(type=1 (NAME), string="def", start=(1, 0), end=(1, 3),
line="def greet(name: str) -> str:\n")
TokenInfo(type=1 (NAME), string="greet", start=(1, 4), end=(1, 9),
line="def greet(name: str) -> str:\n")
TokenInfo(type=54 (OP), string='(', start=(1, 9), end=(1, 10), l
ine="def greet(name: str) -> str:\n")
TokenInfo(type=1 (NAME), string="name", start=(1, 10), end=(1, 14),
line="def greet(name: str) -> str:\n")
TokenInfo(type=54 (OP), string=":", start=(1, 14), end=(1, 15), l
ine="def greet(name: str) -> str:\n")
TokenInfo(type=1 (NAME), string="str", start=(1, 16), end=(1, 19),
line="def greet(name: str) -> str:\n")
TokenInfo(type=54 (OP), string=")", start=(1, 19), end=(1, 20), l
ine="def greet(name: str) -> str:\n")
TokenInfo(type=54 (OP), string=">", start=(1, 21), end=(1, 23),
line="def greet(name: str) -> str:\n")
TokenInfo(type=1 (NAME), string="str", start=(1, 24), end=(1, 27),
line="def greet(name: str) -> str:\n")
TokenInfo(type=54 (OP), string=":", start=(1, 27), end=(1, 28), l
ine="def greet(name: str) -> str:\n")
TokenInfo(type=4 (NEWLINE), string='\n', start=(1, 28), end=(1, 29), l
ine="def greet(name: str) -> str:\n")
TokenInfo(type=5 (INDENT), string="    ", start=(2, 0), end=(2, 4),
line="    return f\"hello {name}\"")
TokenInfo(type=1 (NAME), string="return", start=(2, 4), end=(2, 10), l
ine="    return f\"hello {name}\"")
TokenInfo(type=3 (STRING), string="f\"hello {name}\"", start=(2, 11), en
d=(2, 26), line="    return f\"hello {name}\"")
TokenInfo(type=4 (NEWLINE), string='\n', start=(2, 26), end=(2, 27), l
ine="    return f\"hello {name}\"")
TokenInfo(type=6 (DEDENT), string="", start=(3, 0), end=(3, 0),
line="")
TokenInfo(type=0 (ENDMARKER), string="", start=(3, 0), end=(3, 0),
line="")

```

The `INDENT / DEDENT` pair is the tokenizer's encoding of block structure — the parser never looks at whitespace directly. The `f"hello {name}"` is a single `STRING` token; f-string decomposition into `JoinedStr / FormattedValue` nodes happens later, in the parser/AST phase.

A second example highlights `NL` vs `NEWLINE` :

```

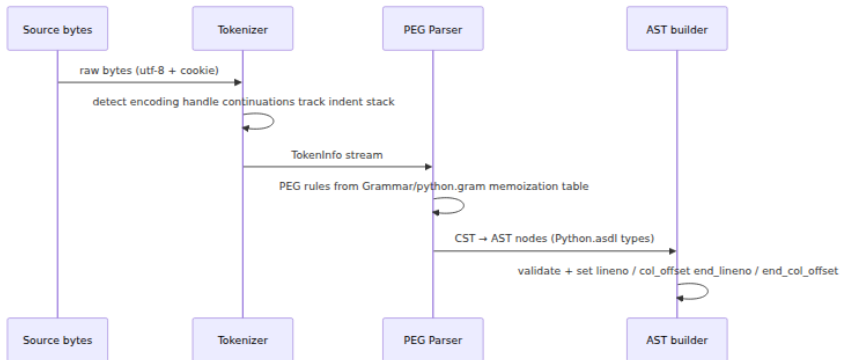
import io, tokenize
src = "if True:\n    x = 1\n    y = 2\nx = 3\n"
for tok in tokenize.generate_tokens(io.StringIO(src).readline):
    print(f"{tokenize.tok_name[tok.type]:10s} {tok.string!r:12s} {tok.s
tart}->{tok.end}")

```

NAME	'if'	(1, 0)->(1, 2)
NAME	'True'	(1, 3)->(1, 7)
OP	':'	(1, 7)->(1, 8)
NEWLINE	'\n'	(1, 8)->(1, 9)
INDENT	' '	(2, 0)->(2, 4)
NAME	'x'	(2, 4)->(2, 5)
OP	'='	(2, 6)->(2, 7)
NUMBER	'1'	(2, 8)->(2, 9)
NEWLINE	'\n'	(2, 9)->(2, 10)
NAME	'y'	(3, 4)->(3, 5)
OP	'='	(3, 6)->(3, 7)
NUMBER	'2'	(3, 8)->(3, 9)
NEWLINE	'\n'	(3, 9)->(3, 10)
DEDENT	''	(4, 0)->(4, 0)
NAME	'x'	(4, 0)->(4, 1)
OP	'='	(4, 2)->(4, 3)
NUMBER	'3'	(4, 4)->(4, 5)
NEWLINE	'\n'	(4, 5)->(4, 6)
ENDMARKER	''	(5, 0)->(5, 0)

If that `x = 1, y = 2` block had been inside unclosed brackets, the newlines between them would be `NL` (non-terminating) rather than `NEWLINE`, and no extra `INDENT` would fire for the continuation.

```
# Tokenize any file from the shell (no Python wrapper needed)
python -m tokenize Lib/pathlib.py | head -30
```



5. Parser — PEG since 3.9 (PEP 617)

The old world: `pgen` / LL(1)

Through Python 3.8, CPython used `pgen` — an LL(1) parser generator. The grammar lived in `Grammar/Grammar` (a restricted EBNF) and `pgen` emitted `Include/graminit.h` + `Python/graminit.c`: large `dfa/arc` tables that a hand-written `Parser/parser.c` interpreted. LL(1) means one token of lookahead, no backtracking. That restriction had real consequences:

- The grammar was contorted to stay LL(1): left-factoring, extra non-terminals, and workarounds in `Parser/parsetok.c` to handle constructs (like `await / async`) that needed more than one token to decide.
- Error messages were poor — "unexpected token" with little context — because LL(1) tables do not know what the programmer *meant*.
- PEG experimentation (PEP 572's `:=`, for example) required ugly LL(1) hacks.

The new world: PEG / `pegen` (PEP 617)

Since Python 3.9 (PEP 617, authored by Guido van Rossum, Lysandropoulos, and Storch), CPython uses a **Parsing Expression Grammar** parser. Key properties:

- **Unlimited lookahead, ordered choice, memoization.** PEG tries alternatives in order and memoizes results (`packrat` parsing) so exponential blowup is avoided. Backtracking is a feature, not a bug.
- **Grammar is executable documentation.** `Grammar/python.gram` reads almost like a spec:

```
# Excerpt from Grammar/python.gram (simplified)
assignment:
    NAME ':'= expression # walrus
    | NAME '=' expression

if_stmt:
    'if' named_expression ':' block elif_stmt
    | 'if' named_expression ':' block else_block?

funcdef:
    decorators? 'def' NAME '(' params? ')' '->' expression? ':'
block
    decorators? 'def' NAME '(' params? ')' ':' block
```

- **pegen generates Parser/parser.c**. Tools/peg_generator/pegen reads python.gram and emits a recursive-descent parser with memoization. The generated file is large and not meant to be read — treat python.gram as the source.
- **Better errors**. Because PEG knows which alternatives it tried, it can report *which* construct was expected ("expected ':'" at the right offset). Python 3.10+ error messages ("unexpected = — did you mean == ?") are largely a PEG payoff, refined in Parser/parser.c and Python/errors.c.

Aspect	Old pgen (≤3.8)	New PEG (pegen , ≥3.9)
Grammar file	Grammar/Grammar (LL(1) EBNF)	Grammar/python.gram (PEG)
Generator	Parser/pgen → graminit.[hc]	Tools/peg_generator/pegen → Parser/parser.c
Lookahead	1 token	Unlimited (packrat)
Backtracking	No	Yes, memoized
Error quality	Generic "invalid syntax"	Contextual ("expected ':'", "did you mean == ?")
Left recursion	Forbidden	Supported (needed for a + b + c)

Operational note. The parser still consumes the *same* token stream. Switching from LL(1) to PEG did not change tokenize.c ; it changed how that stream is interpreted. If you maintain a custom tokenizer or syntax

highlighter, it was unaffected. If you maintain a linter that parses Python, `lib2to3` / `parsoiler` needed updates.

CST → AST

The PEG parser first builds a **Concrete Syntax Tree** (CST) — verbose, punctuation-heavy — then `Python/ast.c` converts it to an **Abstract Syntax Tree** (AST) via `PyAST_FromNodeObject` and helpers generated from `Parser/Python.asdl`. The ASDL file defines every node type:

```
-- Parser/Python.asdl (excerpt)
module Python
{
    mod = Module(stmt* body, type_ignore* type_ignores)
        | Expression(expr body)

    stmt = FunctionDef(identifier name, arguments args,
                        stmt* body, expr* decorator_list,
                        expr? returns, string? type_comment)
        | ClassDef(identifier name, expr* bases, keyword* keywords,
                    stmt* body, expr* decorator_list)
        | Assign(expr* targets, expr value, string? type_comment)
        | If(expr test, stmt* body, stmt* orelse)
        | ...

    expr = BinOp(expr left, operator op, expr right)
        | Call(expr func, expr* args, keyword* keywords)
        | Constant(constant value, string? kind)
        | JoinedStr(expr* values)           -- f-strings
        | FormattedValue(expr value, int conversion, expr? format_spe
c)
        | ...
}
```

`ast.c` validates invariants (e.g., `Store` contexts only on assignment targets), fills `lineno` / `col_offset` / `end_lineno` / `end_col_offset`, and rejects trees `compile.c` could not handle — so by the time `compile.c` runs, the AST is well-formed.

6. AST — the semantic tree and `ast` tooling

Inspecting the AST

Three equivalent ways to dump the AST for any snippet:

```
import ast

src = "x = 1 + 2"
tree = ast.parse(src)
print(ast.dump(tree, indent=2))
```

```
Module(
  body=[
    Assign(
      targets=[
        Name(id='x', ctx=Store())],
      value=BinOp(
        left=Constant(value=1),
        op=Add(),
        right=Constant(value=2))),
    type_ignores=[])
```

```
# Same dump from the shell – useful in CI or one-liners
echo "x = 1 + 2" | python -m ast --indent 2

# With source locations
python -c
"import ast; print(ast.dump(ast.parse('x = 1 + 2'), indent=2,
include_attributes=True))"
# adds lineno=1, col_offset=0, end_lineno=1, end_col_offset=9 on every
node

# Three parse modes
python -m ast --mode exec -c "x = 1" # Module (default, statements)
python -m ast --mode eval -c "1 + 2" # Expression (single expr)
python -m ast --mode single -c "x = 1" # Interactive (exec + print)
```

For larger files, dump-and-grep is a practical debugging workflow:

```
python -m ast myapp/handlers.py --indent 2 --include-attributes | grep
-A2 "FunctionDef"
```

AST node taxonomy (what `compile.c` sees)

Family	Nodes	Notes
Module	Module, Expression, Interactive, FunctionType	Top-level containers; FunctionType for (*args) -> ret stubs

Family	Nodes	Notes
Statements	<code>FunctionDef</code> , <code>AsyncFunctionDef</code> , <code>ClassDef</code> , <code>Assign</code> , <code>AnnAssign</code> , <code>AugAssign</code> , <code>If</code> , <code>For</code> , <code>While</code> , <code>With</code> , <code>Try</code> , <code>Import</code> , <code>Return</code> , ...	<code>body</code> is always <code>stmt*</code>
Expressions	<code>BinOp</code> , <code>UnaryOp</code> , <code>Call</code> , <code>Attribute</code> , <code>Subscript</code> , <code>Name</code> , <code>Constant</code> , <code>JoinedStr</code> , <code>FormattedValue</code> , <code>List</code> , <code>Dict</code> , <code>Set</code> , <code>Lambda</code> , <code>IfExp</code> , <code>Compare</code> , ...	<code>Constant</code> subsumes <code>Num</code> / <code>Str</code> / <code>Bytes</code> / <code>NameConstant</code> / <code>Ellipsis</code> since 3.8
Contexts	<code>Load</code> , <code>Store</code> , <code>Del</code>	On <code>Name</code> / <code>Attribute</code> / <code>Subscript</code> — set by parser, checked by <code>symtable.c</code>
Operators	<code>Add</code> , <code>Sub</code> , <code>Mult</code> , <code>And</code> , <code>Or</code> , <code>Eq</code> , <code>In</code> , <code>Is</code> , ...	Singleton nodes, no fields

The `ast` module in `Lib/ast.py` mirrors these types as Python classes, so `isinstance(node, ast.BinOp)` works and `ast.NodeVisitor` / `ast.NodeTransformer` let tools rewrite trees without touching C.

7. Symbol table — scope analysis before code generation

Between AST and bytecode sits an often-overlooked pass: `Python/symtable.c`. It walks the AST once and builds a `PySTEntryObject` per scope (module, class, function, lambda, comprehension, generator expression).

What it records per scope:

- **Per-name flags:** `DEF_LOCAL`, `DEF_GLOBAL`, `DEF_NONLOCAL`, `DEF_BOUND`, `USE`, `DEF_PARAM`, `DEF_IMPORT`, `DEF_ANNOT`, `DEF_COMP_ITER`.
- **Per-scope flags:** `is_nested`, `has_free`, `child_free`, `is_generator`, `is_coroutine`, `needs_closure`.
- **Free/cell sets:** Names that are local in an enclosing scope but used here (`freevars`) and locals that are closed over by nested scopes (`cellvars`).

Why this must precede compilation: opcode selection depends on scope. The same source name `x` compiles differently in each context:

```
x = 1          # module scope → STORE_NAME / LOAD_NAME (dict lookup)
def f():
    x = 1      # local        → STORE_FAST / LOAD_FAST (array index)
    print(x)
def g():
    x = 1
    def h():
        print(x) # free var → LOAD_DEREF (cell/free array)
```

```
import symtable

src = """
x = 1
def f():
    y = x + 1
    def g():
        return y
    return g
"""

top = symtable.symtable(src, "<demo>", "exec")
print(f"top type: {top.get_type()}, names: {top.get_identifiers()}")
for child in top.get_children():
    print(f"  child {child.get_name()!r} type={child.get_type()} "
          f"free={child.get_frees()} cell={child.get_cells()} "
          f"lineno={child.get_lineno()}")
    for gc in child.get_children():
        print(f"    grandchild {gc.get_name()!r}
              free={gc.get_frees()}")
```

```
top type: module, names: ['x', 'f']
  child 'f' type=function free=() cell=('y',) lineno=3
    grandchild 'g' free=('y',)
```

You can also inspect the flags via the stdlib `symtable` module — it is a thin wrapper over the C `symtable.c` pass, not a reimplementaion, so it shows the exact table the compiler will see.

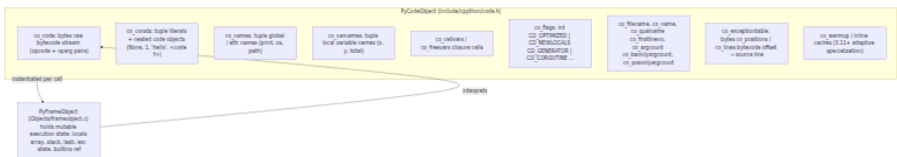
8. Compiler — AST to bytecode and PyCodeObject

`Python/compile.c` (~7k lines) lowers the annotated AST to bytecode. The flow inside:

1. **Scope-aware code generation.** `compiler_enter_scope` / `compiler_exit_scope` push/pop `compiler_unit` structs. Each unit accumulates `co_code` (byte stream), `co_consts`, `co_names`, `co_varnames`, `co_cellvars` / `co_freevars`, and exception/position tables.
2. **Opcode emission.** Helpers like `ADDOP`, `ADDOP_I`, `ADDOP_JUMP` append to the bytecode buffer. Control flow (`if` / `for` / `try` / `with`) emits jumps with fixups patched after the target offset is known.
3. **Peephole / simplification.** Some constant folding still happens here (e.g., `1 + 2` $\rightarrow$ `3` when both sides are `Constant`), though most heavy optimization moved to later stages.
4. **Assembly.** `assemble.c` (included from `compile.c`) lays out the final bytes for `co_code`, builds `co_exceptiontable` (3.11+ exception table encoding) and `co_positions` / `co_lnotab` (line-number mapping), and interns names.

PyCodeObject — the immutable bytecode container

Every function, class body, module, lambda, and comprehension gets its own `PyCodeObject` (`Objects/codeobject.c` , `Include/cpython/code.h`):



Inspect any code object from Python:

```

import dis

def demo(n):
    total = 0
    for i in range(n):
        if i % 2 == 0:
            total += i
    return total

co = demo.__code__
print(f"co_name={co.co_name!r}  co_filename={co.co_filename!r}  "
      f"co_firstlineno={co.co_firstlineno}")
print(f"co_argcount={co.co_argcount}  co_flags={co.co_flags:#x}")
print(f"co_varnames={co.co_varnames}")
print(f"co_consts={co.co_consts}")
print(f"co_names={co.co_names}")
print("--- disassembly ---")
dis.dis(demo)

```

```

co_name='demo' co_filename='<stdin>' co_firstlineno=3
co_argcount=1 co_flags=0x3
co_varnames=('n', 'total', 'i')
co_consts=(None, 0, 2)
co_names=('range',)
--- disassembly ---
5          0 RESUME                                0

6          2 LOAD_CONST                            1 (0)
          4 STORE_FAST                             1 (total)

7          6 LOAD_GLOBAL                          1 (NULL + range)
          18 LOAD_FAST                             0 (n)
          20 PRECALL                             1
          24 CALL                                 1
          34 GET_ITER
>>        36 FOR_ITER                             16 (to 70)
          38 STORE_FAST                             2 (i)

8          40 LOAD_FAST                             2 (i)
          42 LOAD_CONST                             2 (2)
          44 BINARY_OP                             6 (%)
          48 LOAD_CONST                             1 (0)
          50 COMPARE_OP                             2 (==)
          56 POP_JUMP_FORWARD_IF_FALSE             5 (to 68)

9          58 LOAD_FAST                             1 (total)
          60 LOAD_FAST                             2 (i)
          62 BINARY_OP                             13 (+)
          66 STORE_FAST                             1 (total)
>>        68 JUMP_BACKWARD                         17 (to 36)

10 >>      70 LOAD_FAST                             1 (total)
          72 RETURN_VALUE

```

What to read here:

- `RESUME` (wordcode 151) is the 3.11+ function-entry opcode — it checks which tier is executing (interpreter vs. adaptive vs. JIT) and handles tracing.
- `LOAD_FAST` indexes `co_varnames` by `oparg`; no dict lookup. `LOAD_GLOBAL` indexes `co_names` and does a dict lookup on `globals()` + `builtins`.
- `PRECALL` / `CALL` (3.11 calling convention) replaced `CALL_FUNCTION` / `CALL_METHOD` — `PRECALL` sets up the inline cache, `CALL` dispatches.

- `JUMP_BACKWARD` is the loop back-edge; its `oparg` is the delta in bytecode units (17 = 34 bytes of `wordcode`).
- `co_consts` holds only `0` and `2` — `None` is the implicit return value sentinel; `1` was constant-folded away in this build.

```
# Disassemble without writing a wrapper script
python -m dis myapp/handlers.py           # whole file
python -m dis myapp.handlers:handle_req   # dotted path
python -m py_compile myapp/handlers.py && python -m dis __pycache__/
handlers.cpython-311.pyc
```

9. Bytecode caching — `__pycache__`, `.pyc`, and `marshal`

Why it exists

Re-parsing every `.py` on every import would make large applications (thousands of modules) start in seconds instead of hundreds of milliseconds. The `.pyc` cache lets the interpreter skip tokenize/parse/compile when the source has not changed.

Header layout (3.11 — PEP 552 + PEP 3147)

Offset	Size	Field
0	4	magic number (importlib.util.MAGIC_NUMBER, changes per feature release)
4	4	flags (bit 0 = hash-based vs timestamp-based; bit 1 = check hash)
8	4 or 8	timestamp/hash material
		timestamp mode: 4 bytes mtime + 4 bytes source size (legacy)
		[or 8 bytes hash if flag set]
		hash mode: 8 bytes SipHash of source (deterministic)
12/16	var	marshal payload (PyCodeObject serialized via marshal)

Concrete header on this host (3.11.15):

```

import py_compile, importlib.util, struct, tempfile, os, pathlib
with tempfile.TemporaryDirectory() as td:
    src = os.path.join(td, "example.py")
    open(src, "w").write("x = 1\n")
    pyc = importlib.util.cache_from_source(src) # .../__pycache__/
example.cpython-311.pyc
py_compile.compile(src, cfile=pyc)
data = open(pyc, "rb").read()
magic, flags = data[0:4], struct.unpack("<I", data[4:8])[0]
print(f"magic {magic.hex()} (MAGIC_NUMBER={importlib.util.MAGIC_N
UMBER.hex()})")
print(f"flags {flags:#010x} ({'hash-based' if flags & 0x01 else '
timestamp-based'})")
print(f"header {data[:16].hex()}")
print(f"pyc path: {pyc}")

```

```

magic a70d0d0a (MAGIC_NUMBER=a70d0d0a)
flags 0x00000000 (timestamp-based)
header a70d0d0a00000000d707886a06000000
pyc path: /tmp/.../__pycache__/example.cpython-311.pyc

```

a70d0d0a is the 3.11 magic. The next two words 00000000 d707886a are flags=0 (timestamp mode) + mtime; 06000000 is source_size=6 ("x = 1\n"). In hash mode (flags & 1), bytes 8-15 hold an 8-byte hash of the source instead.

Invalidation rules (Lib/importlib/_bootstrap_external.py:SourceFileLoader.get_data + import.c):

- **Timestamp mode (default):** mtime or source_size mismatch → recompile. Fast to check (one stat), but mtime is fragile — git checkout , rsync , or Docker COPY can preserve or clobber it unpredictably.
- **Hash mode (PEP 552):** hash(source) != stored_hash → recompile. Deterministic, content-addressed, suitable for reproducible builds. Selected with py_compile.compile(..., invalidation_mode=PYCACHE_HASH) or PYTHONPYCACHEPREFIX .

Controlling the cache

```
# Environment / flags
PYTHONDONTWRITEBYTECODE=1 python app.py      # never write __pycache__
python -B app.py                               # same, CLI flag
PYTHONPYCACHEPREFIX=/tmp/pyc app.py           # redirect all __pycache__
to one tree
python -m compileall -b Lib/                   # legacy flat .pyc next to
source

# Invalidation modes (PEP 552)
python -m py_compile --invalidation-mode=checked-hash src/app.py
python -m py_compile --invalidation-mode=unchecked-hash src/app.py #
never re-checks hash
python -m py_compile --invalidation-mode=timestamp src/app.py #
default

# Reproducible builds: clamp mtime via SOURCE_DATE_EPOCH (seconds since
epoch)
SOURCE_DATE_EPOCH=0 python -m py_compile src/app.py # header mtime =
0, deterministic output
```

marshal — the serializer underneath

Python/marshal.c is not pickle. It is a compact, versioned, internal format for code objects and constants. It is intentionally not stable across Python versions, not safe for untrusted data (it can construct arbitrary code objects), and not documented as a public API. Lib/importlib/_bootstrap_external.py calls marshal.loads/marshal.dumps on the payload after the 16-byte header; _warnings, encodings, and the frozen importlib modules are also marshalled into the binary at build time.

10. The import system — from import foo to PyCodeObject

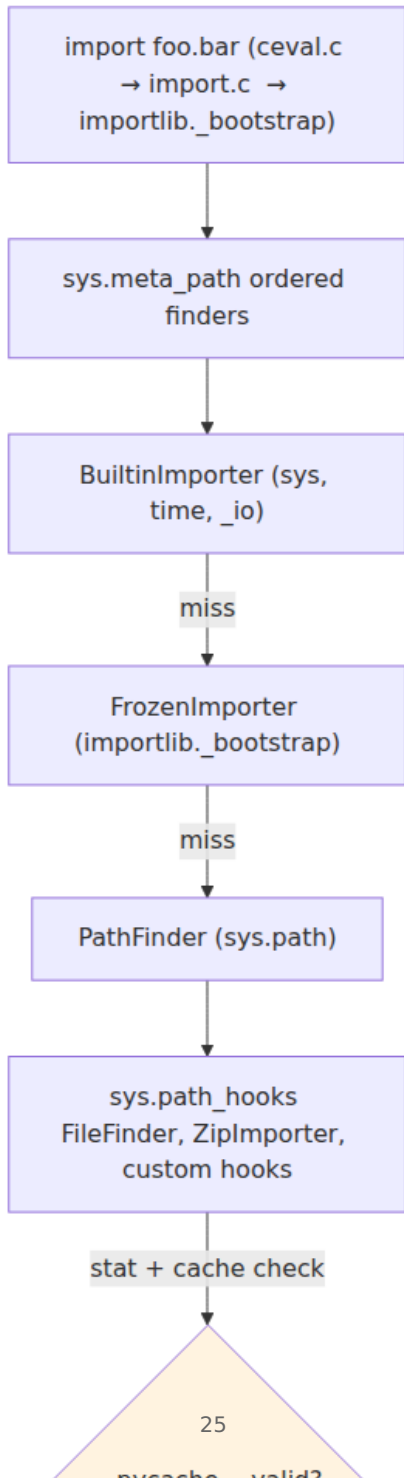
An import statement looks simple; the machinery behind it is the most dynamic part of CPython's startup.

Finders, loaders, and sys.meta_path

PEP 302/451 formalized import as two-phase:

1. **Find** — iterate sys.meta_path (list of MetaPathFinder s). Each finder's find_spec(fullname, path, target) returns a ModuleSpec or None. First non-None wins.

2. **Load** — the spec's `loader` (`Loader.exec_module(module)`) populates the module object. Loaders handle source files, bytecode, extensions, namespaces, zip, etc.



Key details:

- **sys.meta_path order matters.** Inserting a finder at index 0 interposes on every import — the hook `pytest`, `coverage`, and `importlib.machinery` abuse for rewriting. The default order is `BuiltinImporter`, `FrozenImporter`, `PathFinder`.
- **PathFinder fans out via sys.path and sys.path_hooks.** Each entry of `sys.path` is probed; `sys.path_hooks` returns a `PathEntryFinder` (`FileFinder` for directories, `ZipImporter` for `.zip / .egg / .whl`). `FileFinder` handles `.py` → `SourceFileLoader`, `.pyc` → `SourcelessLoader`, `.so / .pyd` → `ExtensionFileLoader`, and namespace packages (PEP 420 — directories without `__init__.py`).
- **The import lock** (`import.c: import_lock`, one global lock in 3.11, per-module locks since 3.3) prevents concurrent imports of the same module from racing. A second thread importing an already-importing module blocks until the first finishes — which is why circular imports can deadlock if a module does heavy work at import time.
- **Caching layers:** `sys.modules` (module objects), `sys.path_importer_cache` (finder per `sys.path` entry), and `_bootstrap_external._path_stat_cache` (per-file stat results) — all cleared by `importlib.invalidate_caches()`.

The bootstrap problem

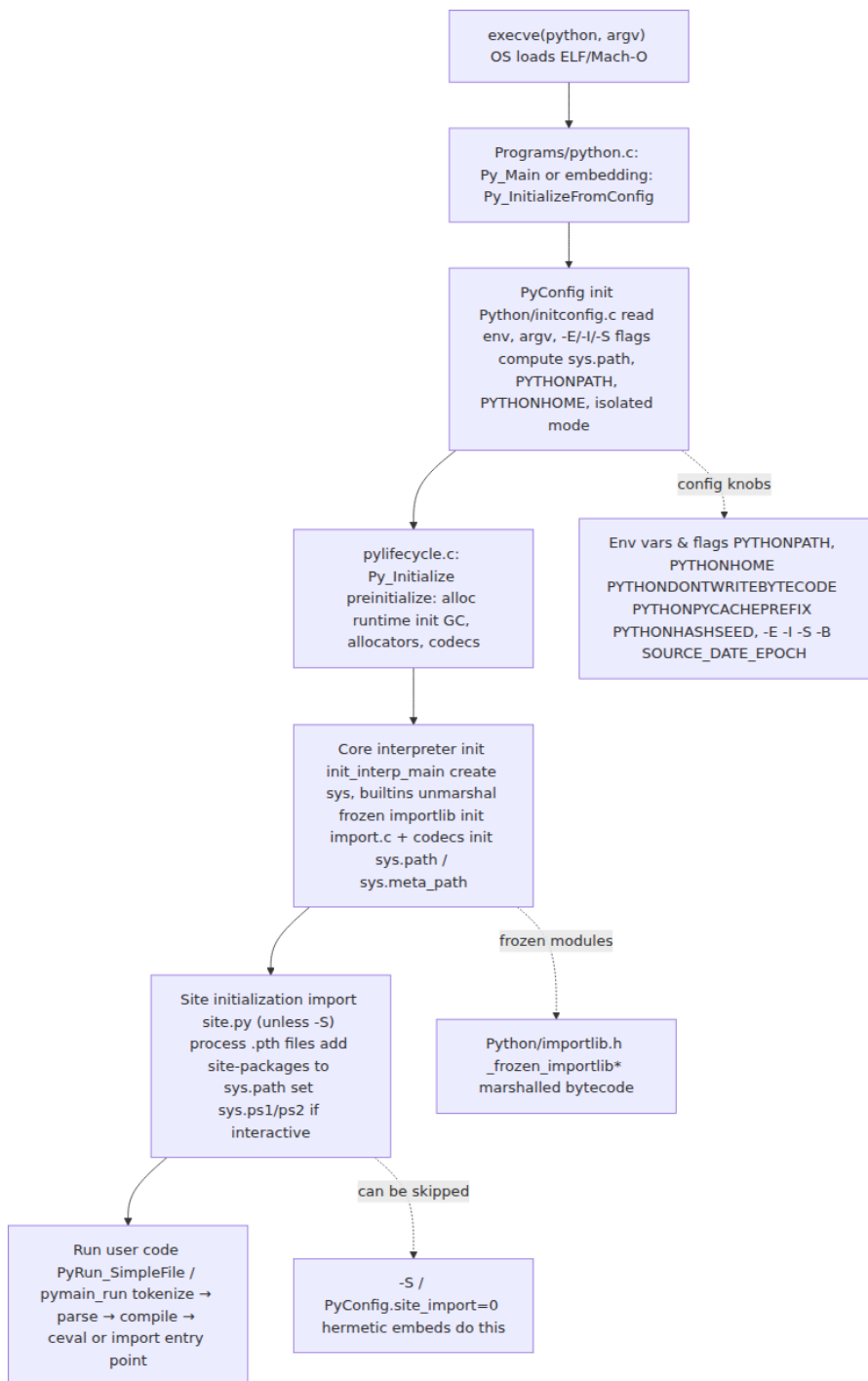
Import is written in Python (`Lib/importlib/_bootstrap.py`, `_bootstrap_external.py`), but Python needs import to start. The cycle is broken by **freezing**: at build time `Tools/build/freeze_modules.py` compiles those two files to bytecode, marshals them, and emits `Python/importlib.h` / `Python/importlib_external.h` — C arrays baked into the `python` binary. On startup `import.c: PyImport_Init` unmarshals them to bootstrap `importlib` without touching the filesystem. Only after that does `PathFinder` become available.

Verify the freeze:

```
grep -l "DO NOT EDIT.*generated by.*freeze" Python/importlib*.h
# Python/importlib.h: /* Auto-generated by Tools/build/
freeze_modules.py -- DO NOT EDIT */
ls -lh Python/importlib*.h
strings ./python | grep -a "_frozen_importlib" | head
```

11. Interpreter startup — from `execve` to first user bytecode

When you run `python app.py`, the OS `execve`s the `python` binary; what follows is a carefully ordered initialization in `Python/pylifecycle.c` and `Python/initconfig.c`.



Walk the stages

1. PyConfig (Python/initconfig.c). Every knob that affects startup is unified in `PyConfig / PyPreConfig`: `argv`, `warnoptions`, `xoptions` (`-X`), `isolated` (`-I`), `use_environment` (`-E`), `site_import` (`-S`), `write_bytecode`, `module_search_paths`, `executable`, `prefix` / `exec_prefix`, and `hash randomization`. `PyConfig_Read` applies precedence: defaults → `pyenv.cfg` → env vars → command-line flags → programmatic overrides. This is the place to look when `sys.path` surprises you.

2. Preinitialization (pylifecycle.c:Py_PreInitializeFromPyArgv). Allocates the `PyRuntimeState`, initializes the GC (`gcmodule.c`), `obmalloc` (`obmalloc.c`), `pymalloc` arenas, unicode internals, and the `PyThreadState` for the main thread. No Python code has run yet.

3. Core init (_Py_InitializeMain, init_interp_main). Creates `sys` and `builtins` modules, initializes type objects (`PyType_Ready` for `object`, `type`, `int`, ...), installs the frozen `importlib` (`import.c:import_init`), initializes codecs/encodings, and computes `sys.path` (via `calculate_path.py` → `getpath.py` in 3.11+). After this point `import` works and `sys.meta_path` is populated.

4. Site (Lib/site.py, unless -S / PyConfig.site_import=0). `site.py` processes `pyenv.cfg`, `.pth` files (Windows), `.pth` files in `site-packages`, and `sitecustomize` / `usercustomize` if present. Each `.pth` line can add to `sys.path` or — if it starts with `import` — execute arbitrary code. This is why hermetic deploys often disable site import.

5. User code (Modules/main.c:pymain_run_python). Finally `PyRun_SimpleFileExFlags / pymain_run_module` tokenizes, parses, compiles, and evaluates the target. For `python -m app` it imports `app.__main__`; for `python app.py` it compiles `app.py` as `__main__` (with `__cached__ = None` — no `.pyc` is written for the top-level script, only for imported modules).

Observe it yourself:

```

# Full startup trace – every import, every sys.path probe
python -v app.py 2>&1 | head -60
python -v -m app 2>&1 | head -60

# Initialization config dump (3.11+)
python -X showrefcount -c "import sys; print(sys._xoptions)" 2>&1 | head
python3 -c "import sysconfig;
print(sysconfig.get_config_var('prefix'))"
python3 -c "import sys; print('\n'.join(sys.path))"

# What site.py added
python -c "import site; print(site.getsitepackages());
print(site.getusersitepackages())"
python -S -c "import sys; print('\n'.join(sys.path))" # without
site.py

```

A minimal embedding shows the same lifecycle from C:

```

// embed.c – minimal CPython embedding (gcc embed.c -I/usr/include/
python3.11 -lpython3.11)
#include <Python.h>
int main(int argc, char *argv[]) {
    PyConfig config;
    PyConfig_InitPythonConfig(&config);
    // Hermetic: ignore env, don't import site.py, isolate from user
    site
    config.isolated = 1;
    config.use_environment = 0;
    config.site_import = 0;
    PyStatus status = Py_InitializeFromConfig(&config);
    if (PyStatus_Exception(status)) { Py_ExitStatusException(status); }
    PyConfig_Clear(&config);

    PyRun_SimpleString("print('hello from embedded CPython')");

    Py_Finalize();
    return 0;
}

```

12. Putting it together — a single `import` traced

To make the pipeline concrete, trace what happens for `import mypkg.utils` in `app.py`:

1. `app.py` is compiled as `__main__` (`tokenize` → `PEG parse` → `AST` → `symtable` → `compile` → `PyCodeObject` for `__main__`). No `.pyc` is written for `__main__`.
2. `ceval.c` hits `IMPORT_NAME` for `mypkg.utils`. It calls `import.c:PyImport_ImportModuleLevelObject` → `_bootstrap.py:_find_and_load`.
3. `_find_and_load` walks `sys.meta_path`. `PathFinder.find_spec("mypkg.utils")` fans out over `sys.path`, finds `mypkg/utils.py` via `FileFinder`.
4. `SourceFileLoader.get_code("mypkg.utils")` checks `__pycache__/utils.cpython-311.pyc`:
5. Hit and header validates (`mtime/size` or `hash`) → `marshal.loads(payload)` → `PyCodeObject`, skip to 6.
6. Miss/stale → read `utils.py` bytes, `tokenize.c` → `parser.c` (PEG) → `ast.c` → `symtable.c` → `compile.c` → fresh `PyCodeObject`, then `marshal.dumps` + 16-byte header → write `__pycache__/utils.cpython-311.pyc` (if `write_bytecode` is true).
7. `exec_module` creates `sys.modules["mypkg.utils"]` (inserted *before* execution to support circular imports), then `ceval.c:_PyEval_EvalFrame` executes the code object top-to-bottom, populating `module.__dict__`.
8. The `IMPORT_NAME` opcode binds the result into `__main__.__dict__` (`import mypkg.utils` binds `mypkg`; `from mypkg.utils import foo` binds `foo` via `IMPORT_FROM`).

```
# Watch it happen live (verbose imports show every probe and cache
decision)
python -v -c "import mypkg.utils" 2>&1 | cat -n | head -40
# Output includes:
#   import 'mypkg.utils' #
<_frozen_importlib_external.SourceFileLoader ...>
#   # trying /app/mypkg/utils.py
#   # code object from '/app/__pycache__/mypkg/utils.cpython-311.pyc'
#   import 'mypkg.utils' # loaded from ...
```

13. Distributed-systems lens — reproducible builds, hermetic deploys, `__pycache__` in containers

CPython's pipeline has outsized operational consequences at fleet scale.

Reproducible builds and `SOURCE_DATE_EPOCH`

Timestamp-mode `.pyc` headers embed `mtime`, so two builds from identical source at different times produce different bytes. That breaks content-addressed caching (Bazel, Nix), SLSA provenance (build output hash $\neq$ expected), and container layer deduplication.

PEP 552 hash-based `.pyc` + `SOURCE_DATE_EPOCH` fix this:

```
# Dockerfile – deterministic bytecode
ARG SOURCE_DATE_EPOCH=0
ENV SOURCE_DATE_EPOCH=${SOURCE_DATE_EPOCH}
ENV PYTHONHASHSEED=0
# Option A: hash-based pyc (no mtime in header at all)
RUN python -m compileall --invalidation-mode=checked-hash -q /app
# Option B: timestamp-based but clamped mtime (also deterministic)
RUN find /app -name '*.py' -exec python -m py_compile {} \;
# SOURCE_DATE_EPOCH clamps the mtime written into each header to the
epoch value
```

Verify determinism in CI:

```
SOURCE_DATE_EPOCH=0 python -m py_compile src/app.py -o /tmp/a.pyc
SOURCE_DATE_EPOCH=0 python -m py_compile src/app.py -o /tmp/b.pyc
cmp /tmp/a.pyc /tmp/b.pyc && echo "deterministic" || echo "nondeterministic"
# Also: diffoscope /tmp/a.pyc /tmp/b.pyc for human-readable header diff
```

The CPython build itself honors `SOURCE_DATE_EPOCH` when freezing `importlib` and compiling `stdlib` `.pyc`s during `make install` — so a hermetic Python toolchain build is reproducible end-to-end.

Hermetic Python deploys

Backend services often want a Python that does not depend on ambient `PYTHONPATH`, user site-packages, or implicit `site.py` `.pth` processing. Three knobs compose a hermetic runtime:

Knob	Effect	When to use
<code>PyConfig.isolated = 1</code> / <code>python -I</code>	Ignores <code>PYTHONPATH</code> , <code>PYTHONHOME</code> , user site, <code>PYTHON*</code> env vars	Sandboxed workers, security-sensitive jobs
<code>PyConfig.site_import = 0</code> / <code>python -S</code>	Skips <code>site.py</code> entirely (no <code>.pth</code> , no <code>site-packages</code> auto-add)	Embedding, minimal containers, Bazel <code>py_binary</code>
<code>PYTHONDONTWRITEBYTECODE=1</code> / <code>python -B</code>	Never writes <code>__pycache__</code>	Read-only filesystems, ephemeral containers
<code>PYTHONPYCACHEPREFIX=/tmp/pyc</code>	Redirects all <code>__pycache__</code> to one writable tree	Read-only source volume + writable cache volume
<code>pyenv.cfg</code> / <code>._pth</code> (Windows)	Controls <code>sys.prefix</code> , <code>isolated</code> , <code>site_import</code> at install time	Venvs, <code>pdm</code> / <code>poetry</code> isolated envs

A common production pattern for containers:

```

FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ src/
# Pre-compile with hash invalidation for determinism
RUN python -m compileall --invalidation-mode=checked-hash -q src/

FROM gcr.io/distroless/python3-debian12
COPY --from=builder /app /app
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/
lib/python3.11/site-packages
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONPYCACHEPREFIX=/tmp/pyc \
    PYTHONHASHSEED=random
# hash seed random (default) is fine when pyc is hash-based; clamp
to 0 only if you need cross-host determinism
WORKDIR /app
ENTRYPOINT ["python", "-I", "-m", "myapp"]

```

`__pycache__` in containers — keep, strip, or pre-compile?

Strategy	Image size	Cold start	Determinism	Notes
Keep <code>__pycache__</code> (pre-compiled)	Larger (+20-40%)	Fastest (no compile at import)	Good if hash-mode + <code>SOURCE_DATE_EPOCH</code>	Best for latency-sensitive services
Strip <code>.pyc</code> , keep <code>.py</code>	Smaller	Slower first import per process	N/A	Recompiles memory cold start fine for lived pods
Strip <code>.py</code> , keep <code>.pyc</code> only	Smaller	Fast	Good if written deterministically	Breaks debugger tracebacks that read source; <code>co_file</code> still points to <code>.py</code>

Strategy	Image size	Cold start	Determinism	Notes
<code>PYTHONDONTWRITEBYTECODE=1</code> + writable <code>PYTHONPYCACHEPREFIX</code>	Medium	Fast after warmup	Cache is ephemeral	Good for read-only source mounts

At scale, the usual answer is **pre-compile with hash invalidation, keep both `.py` and `.pyc`, set `PYTHONDONTWRITEBYTECODE=1` at runtime** (so no writes race on shared volumes), and let the pre-compiled `.pyc` serve as the import fast path. For serverless (AWS Lambda, Cloud Run jobs) where cold start matters most, pre-compiled `.pyc` with a warm `PYTHONPYCACHEPREFIX` on `/tmp` is measurably faster — profile with `python -X importtime`.

```
# Measure import cost – the single most useful startup diagnostic
python -X importtime -c "import myapp" 2>&1 | head -30
python -X importtime -c "import myapp" 2>&1 | tail -5 # total self + cumulative

# Compare cold vs warm (clear caches between runs)
find . -type d -name __pycache__ -exec rm -rf {} +
python -X importtime -c "import myapp" 2>&1 | tail -3 # cold (recompile)
python -X importtime -c "import myapp" 2>&1 | tail -3 # warm (pyc hit)
```

Supply-chain note

Because `.pyc` is `marshal` + header, a tampered `.pyc` is arbitrary code execution at import time. Hermetic deploys should treat `__pycache__` as a build artifact, not a deployment artifact — either rebuild it from source in a trusted builder stage or verify it with an out-of-band attestation (SLSA provenance over the source, not over the `.pyc`). Never ship a `.pyc` you did not build.

14. How to explore further — hands-on exercises

These are ordered from "five seconds" to "an afternoon":

```

# 1. Tokenize the file you are working on right now
python -m tokenize myapp/handlers.py | head -40

# 2. Dump the AST with locations and compare exec/eval/single
echo "x: int = 1" | python -m ast --indent 2 --include-attributes
echo "1 + 2"      | python -m ast --mode eval --indent 2

# 3. Disassemble a real function and find LOAD_FAST vs LOAD_GLOBAL
python -m dis myapp.handlers:handle_request | head -40

# 4. Inspect code object fields directly
python -c "
import myapp.handlers
co = myapp.handlers.handle_request.__code__
for k in
('co_varnames', 'co_names', 'co_consts', 'co_flags', 'co_freevars', 'co_cell
vars'):
    print(k, getattr(co, k))
print(co.co_code.hex()[:80], '...')
"

# 5. Examine a .pyc header byte-for-byte
python -c "
import importlib.util, struct, pathlib
pyc = pathlib.Path('__pycache__/_handlers.cpython-311.pyc')
data = pyc.read_bytes()
print('magic', data[:4].hex(), 'flags', struct.unpack('<I', data[4:8])
[0])
print('header', data[:16].hex())
print('payload starts with marshal type', chr(data[16]))
"

# 6. Trace the import system for a single import
python -v -c "import mypkg.utils" 2>&1 | grep -E "(trying|code object|
import )"

# 7. Measure startup with and without bytecode cache
find . -type d -name __pycache__ -prune -exec rm -rf {} +
time python -c "import myapp"          # cold
time python -c "import myapp"          # warm (pyc now present)
python -X importtime -c "import myapp" 2>&1 | tail -5

# 8. Build a minimal hermetic run and observe sys.path
python -I -S -c "import sys, pprint; pprint.pprint(sys.path)"
python -c "import sys, pprint; pprint.pprint(sys.path)" # compare

```

Key takeaways

- CPython is the reference interpreter; its repo layout (`Parser/` , `Python/` , `Objects/` , `Include/` , `Lib/` , `Modules/`) maps directly to pipeline stages, and `Grammar/python.gram` is the single source of truth for syntax.
- Source executes through a strict pipeline: bytes → `tokenize.c` (encoding, `INDENT / DEDENT` , `NL / NEWLINE`) → PEG parser (`pegen` from `python.gram` , since 3.9) → AST (`ast.c / Python.asdl`) → `symtable.c` (scope analysis, `cell / free vars`) → `compile.c` → `PyCodeObject` → `ceval.c` evaluation loop. Each stage has a Python-level mirror (`tokenize` , `ast` , `symtable` , `dis`).
- The PEG parser (PEP 617) replaced the LL(1) `pgen` parser to allow unlimited lookahead, ordered choice with memoization, and far better syntax errors — without changing the tokenizer.
- `PyCodeObject` is the immutable bytecode container (`co_code` , `co_consts` , `co_names` , `co_varnames` , `co_cellvars / co_freevars` , `co_exceptiontable` , `co_positions`). The frame (`PyFrameObject`) holds mutable execution state. `dis` and `co_*` attributes expose the exact structure the eval loop sees.
- `.pyc / __pycache__` is 16-byte header (magic + flags + mtime/size or hash) + `marshal` payload. PEP 552 hash-based invalidation + `SOURCE_DATE_EPOCH` make builds deterministic; timestamp mode is faster to validate but fragile across `git / rsync / Docker`. `marshal` is internal, versioned, and not safe for untrusted data.
- Import is a two-phase finder/loader protocol over `sys.meta_path / sys.path / sys.path_hooks` , bootstrapped from frozen bytecode (`Python/importlib.h`) so the interpreter can import before the filesystem importer exists. `sys.modules` + import lock + `sys.path_importer_cache` are the caching and concurrency controls.
- Interpreter startup (`pylifecycle.c / initconfig.c / PyConfig`) is ordered: `PyConfig` (env/flags/ `pyvenv.cfg`) → preinit (GC, allocators) → core init (types, frozen `importlib` , `sys.path`) → `site.py` (`.pth` , `site-packages`) → user code. `python -I / -S / -B / PYTHONPYCACHEPREFIX` compose a hermetic runtime.
- At fleet scale, pre-compile `.pyc` with hash invalidation in the builder stage, ship both `.py` and `.pyc` , set `PYTHONDONTWRITEBYTECODE=1` at runtime, and treat `__pycache__` as a derived artifact — never as a

trusted input. Measure with `python -X importtime` and verify determinism with `cmp + SOURCE_DATE_EPOCH`.

Further reading

- CPython Language Reference — the authoritative spec for syntax and semantics: <https://docs.python.org/3/reference/index.html>
- CPython `dis` module — bytecode instruction listing and `wordcode` format: <https://docs.python.org/3/library/dis.html>
- CPython `ast` module — AST node types, `ast.parse / ast.dump / ast.NodeVisitor`: <https://docs.python.org/3/library/ast.html>
- CPython `importlib` and the import system — finders, loaders, `ModuleSpec`, `sys.meta_path`: <https://docs.python.org/3/reference/import.html>
- PEP 617 — New PEG parser for CPython (why LL(1) was replaced, PEG design, `pegen`): <https://peps.python.org/pep-0617/>
- PEP 552 — Deterministic pycs (hash-based invalidation, `SOURCE_DATE_EPOCH`, header format): <https://peps.python.org/pep-0552/>
- CPython Developer Guide — repository layout, `make regen-*`, build and development workflow: <https://devguide.python.org/>
- CPython Source — `Grammar/python.gram`, `Parser/token.c`, `Python/compile.c`, `Python/ceval.c`, `Objects/codeobject.c` on `main`: <https://github.com/python/cpython>
- PEP 3147 — PYC Repository Directories (`__pycache__` and `importlib.util.cache_from_source`): <https://peps.python.org/pep-3147/>

- `marshal` format and `py_compile` — `stdlib` docs for the serialization layer under `.pyc` : <https://docs.python.org/3/library/marshal.html> and https://docs.python.org/3/library/py_compile.html

Chapter 2 — Objects, Reference Counting, and the PyObject System

What this chapter covers. Every value in CPython — every integer, string, function, class, module, and `None` — is a heap-allocated `PyObject`. That uniformity is what makes Python expressive and what makes CPython's memory story subtle. This chapter opens the black box: the `PyObject` header that every value shares, the `PyTypeObject` that makes types themselves objects, the reference-counting discipline that reclaims most memory instantly, and the places where that discipline breaks. You will see the actual C structs from `Include/object.h` and `Include/cpython/longobject.h`, trace `ob_refcnt` through `Py_INCREF` and `Py_DECREF` at the assembly level, and learn to debug borrowed-versus-owned reference bugs that crash C extensions. Along the way we cover the performance machinery hiding behind "simple" types — the small-integer cache, string interning, free lists, PEP 393 flexible Unicode, 30-bit digit arrays for arbitrary-precision integers, the list over-allocation strategy, and the compact-dict revolution of Python 3.6+. We close with immortal objects (PEP 683, Python 3.12), reference cycles, and the bridge to the cyclic garbage collector covered in Chapter 4.

Learning goals — after this chapter you should be able to:

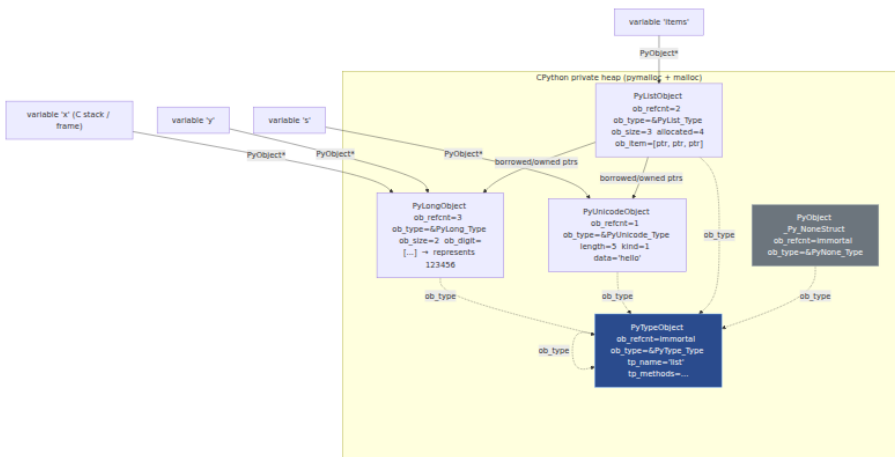
- Read and interpret `PyObject`, `PyVarObject`, and `PyTypeObject` from CPython source and explain what `ob_refcnt`, `ob_type`, and `ob_size` do.
- Trace reference-count operations through `Py_INCREF` / `Py_DECREF` / `Py_XINCREF` / `Py_XDECREF`, predict when an object's destructor runs, and diagnose leaks and use-after-free bugs.
- Distinguish borrowed from owned references and apply the correct discipline in C-extension code and when reading CPython source.
- Explain the small-integer cache, string interning, and per-type free lists; predict when `is` and `==` diverge and when `id` values are reused.

- Diagram the in-memory layout of `list`, `tuple`, `dict`, `set`, `str` (PEP 393), and `int` (digit array), and reason about their time and space costs.
- Describe immortal objects (PEP 683) and why they exist, how reference cycles defeat pure reference counting, and what the GIL guarantees (and no longer guarantees under free-threaded Python) for refcount safety.

1. The Everything-Is-An-Object Contract

CPython takes "everything is an object" literally. There is no separate universe of primitives versus objects as in Java or JavaScript. An integer is a heap allocation. A function is a heap allocation. A type is a heap allocation. `None` itself is a singleton heap allocation (`_Py_NoneStruct`) that lives for the lifetime of the interpreter.

This uniformity flows from one C struct. Every Python value begins with the same two fields — a reference count and a pointer to its type — and everything else is layered on top. The payoff is conceptual simplicity: a single ownership discipline, a single dispatch mechanism (`ob_type->tp_*` slots), a single way to ask "what are you?" The price is that even `42` costs a heap object, and that every pointer assignment must maintain a counter.



Note the self-reference: `type`'s type is itself (`PyType_Type.ob_type == &PyType_Type`). That circularity is bootstrapped at interpreter startup and is the reason `isinstance(type, type)` is true.

2. PyObject_HEAD — The Universal Prefix

2.1 The two structs every object shares

Open `Include/object.h` in the CPython source tree. Stripped to essentials (Python 3.12, `Include/object.h` lines ~100-180):

```
/* Include/object.h — simplified, comments added */

// Extra header used only in debug builds for doubly-linked list of all
// objects
#ifdef Py_TRACE_REFS
#define _PyObject_HEAD_EXTRA          \
    struct _object *_ob_next;         \
    struct _object *_ob_prev;
#else
#define _PyObject_HEAD_EXTRA
#endif

struct _object {
    _PyObject_HEAD_EXTRA /* linked list in debug builds only */
    Py_ssize_t ob_refcnt; /* reference count — see §4 */
    PyTypeObject *ob_type; /* pointer to type object — see §3 */
};

typedef struct _object PyObject;

/* Every Python object literally starts with these two (or three)
fields */
#define PyObject_HEAD                \
    PyObject ob_base;

// Variable-length objects (list, tuple, str, bytes, long) carry a size
typedef struct {
    PyObject ob_base;
    Py_ssize_t ob_size; /* number of items / digits / characters */
} PyVarObject;

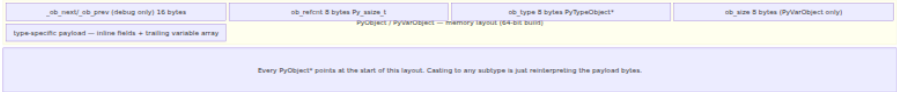
#define PyVarObject_HEAD PyVarObject ob_base
#define PyObject_VAR_HEAD PyVarObject ob_base
```

Macros `PyObject_HEAD` and `PyVarObject_HEAD` are how every concrete type stamps this prefix onto its own struct. Any `PyObject*` can be cast to any concrete type pointer and the first bytes alias correctly — a manual form of inheritance in C.

Fixed-size vs. variable-size. `PyObject` is for fixed-size objects (float, None). `PyVarObject` adds `ob_size` for objects whose allocation size varies per instance. `ob_size` is not "length as seen by Python" in every case — for `int` it is the number of *digits* (see §10), for `str` it is character count, for `list` it is element count.

2.2 What `ob_refcnt` and `ob_type` actually do

Field	Type	Meaning
<code>ob_refcnt</code>	<code>Py_ssize_t</code> (signed pointer width; 63 bits on 64-bit)	Number of owned references. When it reaches zero, the deallocator runs immediately on the thread that decremented it. Under PEP 683 it may hold the sentinel <code>_Py_IMMORTAL_REFCNT</code> meaning "never deallocate."
<code>ob_type</code>	<code>PyTypeObject*</code>	Pointer to the type object that defines behavior — method slots, allocation, deallocation, GC traversal. Dispatch like <code>x + y</code> is <code>x->ob_type->tp_as_number->nb_add(x, y)</code> .
<code>ob_size</code>	<code>Py_ssize_t</code>	Only on <code>PyVarObject</code> . Number of logical items. Also determines allocation size for the trailing array.



Concrete examples — how three types extend the prefix:

```

/* float – fixed-size, no ob_size */
typedef struct {
    PyObject_HEAD
    double ob_fval;
} PyFloatObject;

/* list – variable-size, but ob_item is a separately-allocated array */
typedef struct {
    PyObject_VAR_HEAD           /* ob_base + ob_size (== len) */
    PyObject **ob_item;         /* array of PyObject* – allocated
separately */
    Py_ssize_t allocated;       /* slots malloc'd, >= ob_size (over-
allocation) */
} PyListObject;

/* tuple – variable-size, items inline (true trailing array) */
typedef struct {
    PyObject_VAR_HEAD           /* ob_base + ob_size */
    PyObject *ob_item[1];       /* C trick: actually ob_size pointers
inline */
} PyTupleObject;

```

The distinction between `list` (out-of-line `ob_item` pointer) and `tuple` (inline trailing array) matters for cache behavior and allocation cost — see §8.

3. PyTypeObject — Types Are Objects Too

If every value has an `ob_type` pointer to its type, what is a type? Another `PyObject` — specifically a `PyTypeObject` — whose own `ob_type` points at `PyType_Type` (the type of types, historically called `type`).

```

/* Include/object.h - PyObject (heavily abbreviated; real struct is
~80 fields) */
typedef struct _typeobject {

PyObject_VAR_HEAD                /* ob_base + ob_size (for heap
types) */
    const char *tp_name;          /* "list", "dict", "MyClass"
*/
    Py_ssize_t tp_basicsize;      /* sizeof instances, e.g.
sizeof(PyListObject) */
    Py_ssize_t tp_itemsize;      /* per-item size for var
objects, else 0 */

    destructor tp_dealloc;        /* called when ob_refcnt -> 0
*/
    Py_ssize_t tp_vectorcall_offset;
    getattrofunc tp_getattro;
    setattrofunc tp_setattro;
    PyAsyncMethods *tp_as_async;
    reprfunc tp_repr;
    PyNumberMethods *tp_as_number; /* nb_add, nb_multiply, ... */
    PySequenceMethods *tp_as_sequence; /* sq_item, sq_length, ... */
    PyMappingMethods *tp_as_mapping; /* mp_subscript, ... */
    hashfunc tp_hash;
    ternaryfunc
tp_call;                        /* type.__call__ - calling the type creates
instances */
    reprfunc tp_str;
    getattrofunc tp_getattro;
    setattrofunc tp_setattro;
    PyBufferProcs *tp_as_buffer;

    unsigned long
tp_flags;                       /* Py_TPFLAGS_* - GC, mutable, etc. */
    const char *tp_doc;

    traverseproc tp_traverse;     /* GC: visit contained
PyObject* refs */
    inquiry tp_clear;           /* GC: break cycles */

    richcmpfunc tp_richcompare;
    Py_ssize_t tp_weaklistoffset;

    getiterfunc tp_iter;
    iternextfunc tp_iternext;
    struct PyMethodDef *tp_methods;
    struct PyMemberDef *tp_members;
    struct PyGetSetDef *tp_getset;
    struct _typeobject *tp_base; /* base class */
    PyObject *tp_dict;          /* type's __dict__ */

```

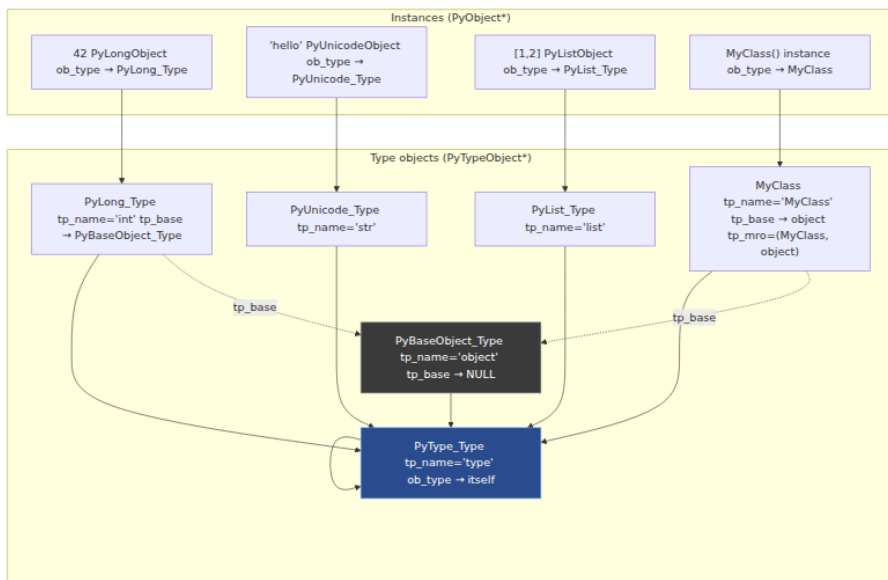
```

    descrgetfunc tp_descr_get;
    descrsetfunc tp_descr_set;
    Py_ssize_t tp_dictoffset;
    initproc tp_init;
    allocfunc tp_alloc;
    newfunc tp_new;
    freefunc tp_free;
    inquiry tp_is_gc;                               /* is this type GC-tracked? */
    PyObject *tp_bases;                             /* tuple of bases */
    PyObject *tp_mro;                                /* tuple: method resolution
order */
    PyObject *tp_cache;
    PyObject *tp_subclasses;
    PyObject *tp_weaklist;
    destructor tp_del;
    unsigned int tp_version_tag;
    destructor tp_finalize;
    vectorcallfunc tp_vectorcall;
    /* ... plus 3.12+ watchers, managed dict, etc. */
} PyTypeObject;

```

Key fields to internalize:

- **tp_name** / **tp_basicsize** / **tp_itemsize** — identity and allocation geometry. **tp_itemsize** is nonzero only for heap types with inline trailing arrays (tuple, str, long).
- **tp_flags** — bitfield including **Py_TPFLAGS_HEAPTYPE** (type created at runtime via `class` statement), **Py_TPFLAGS_HAVE_GC**, **Py_TPFLAGS_IMMUTABLETYPE** (3.12+ — type itself is immortal and lock-free for free-threaded builds).
- **tp_traverse** / **tp_clear** — the two methods that let the cyclic GC walk and break reference graphs. If a type can participate in cycles (it holds `PyObject*` pointers), it must implement these.
- **tp_vectorcall** — the fast calling convention introduced in 3.8 that avoids tuple/dict construction for calls.
- **tp_mro** — the linearized method resolution order (C3 linearization), cached as a tuple for fast attribute lookup.



Backend lens. In a C extension you define a type by filling a `PyTypeObject` (or more commonly a `PyType_Spec` + `PyType_FromSpec`). Forgetting to set `Py_TPFLAGS_HAVE_GC` on a container type means the GC will never traverse it — cycles through it will leak. Forgetting `tp_traverse` while setting the flag is equally fatal: the GC will walk uninitialized memory.

4. Reference Counting — The Primary Reclamation Path

4.1 Mechanics: `Py_INCREF` and `Py_DECREF`

CPython's primary memory management is **deterministic reference counting**. No background thread, no pause — when the last owner releases its reference, the object dies on the spot.

The entire mechanism is two macros (plus `_Py` variants that allow `NULL`):

```

/* Include/object.h – reference counting primitives */

// Increment – must hold GIL (or be an immortal / atomic path in free-
// threaded builds)
static inline void Py_INCREF(PyObject *op) {
    // In 3.12+ immortal objects are detected and the increment is
    // skipped:
    // if (_Py_IsImmortal(op)) return;
    op->ob_refcnt++;
}

// Decrement – if count reaches zero, deallocate immediately
static inline void Py_DECREF(PyObject *op) {
    // if (_Py_IsImmortal(op)) return;
    if (--op->ob_refcnt == 0) {
        _Py_Dealloc(op); // dispatches to op->ob_type->tp_dealloc
    }
}

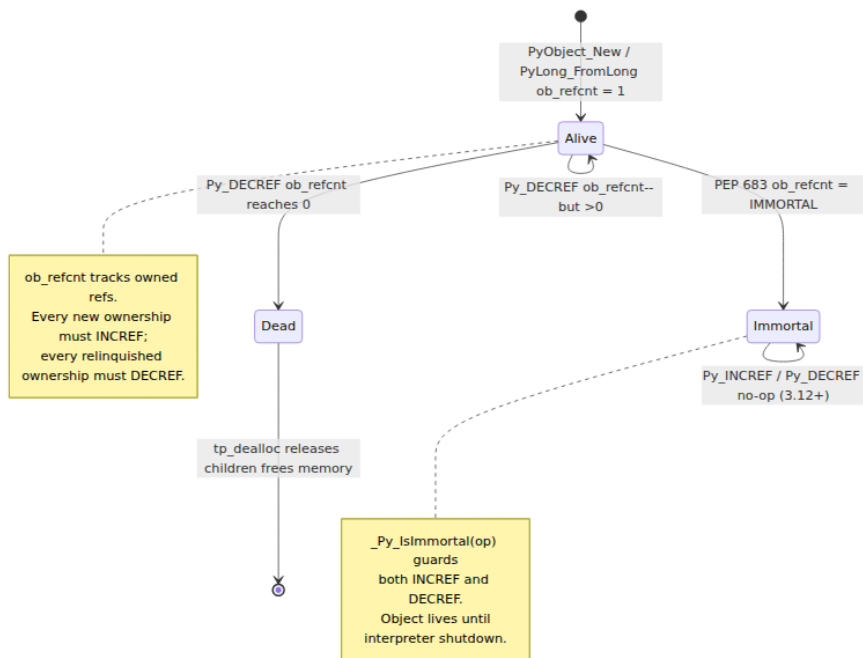
// NULL-safe variants – common in error paths
#define Py_XINCREF(op) do { if ((op) != NULL) Py_INCREF(op); } while (0)
#define Py_XDECREF(op) do { if ((op) != NULL) Py_DECREF(op); } while (0)

// "Steals" macros for constructors
// PyList_SET_ITEM(list, i, item) – does NOT incref item; list steals
// the reference
// PyTuple_SET_ITEM(tuple, i, item) – same

```

What `_Py_Dealloc` does, in order:

1. Calls `op->ob_type->tp_dealloc(op)` — the type's destructor. For containers this decrements refs to contained objects (which may themselves deallocate recursively).
2. For GC-tracked objects, untracks from the GC generation lists first.
3. Returns memory to pymalloc / free lists / `free()` depending on type.



4.2 Who owns what — the contract

Every `PyObject*` in C is either an **owned (strong / new) reference** or a **borrowed (weak / unowned) reference**. The distinction is not in the pointer type — it is a *contract* documented per function.

Pattern	Ownership	Caller must DECREF?	Example
Function returns new reference	Caller owns it	Yes	<code>PyLong_FromLong()</code> , <code>PyList_GetItem</code> is NOT this — see below
Function returns borrowed reference	Callee retains ownership	No (but INCRF if you want to keep it)	<code>PyList_GetItem()</code> , <code>PyDict_GetItem()</code> , <code>PyTuple_GetItem()</code>

Pattern	Ownership	Caller must DECREF?	Example
Function steals reference	Callee takes ownership	No (do not DECREF after)	<code>PyList_SET_ITEM()</code> , <code>PyTuple_SET_ITEM()</code> , <code>PyModule_AddObject()</code>
You store a pointer in a struct/list	You become an owner	INCREf on store, DECREF on remove	<code>list->ob_item[i] = x;</code> <code>Py_INCREF(x)</code>

The canonical bug:

```
// BUG – PyList_GetItem returns a BORROWED reference; storing it
// without INCREf
// creates a dangling pointer if the list is mutated or deallocated.
PyObject *item = PyList_GetItem(list, 0); // borrowed!
my_struct->cached = item;                  // dangling – no INCREf!

// FIX
PyObject *item = PyList_GetItem(list, 0); // borrowed
Py_INCREF(item);                          // promote to owned
my_struct->cached = item;                  // now safe; DECREF in
dealloc
```

Diagram omitted for brevity — see surrounding prose.

4.3 GIL implications for refcounts

In standard (GIL) builds, `ob_refcnt++` and `ob_refcnt--` are plain non-atomic increments. Safety comes from the GIL: only the thread holding the GIL may touch `ob_refcnt` (with narrow exceptions for explicitly GIL-releasing C code that must not touch Python objects).

In the **free-threaded build** (Python 3.13t, PEP 703, `--disable-gil`):

- `ob_refcnt` becomes an atomic or uses biased reference counting (per-thread refcount bias + periodic merging) so that the common case — a thread manipulating objects it owns — avoids atomic overhead.
- Immortal objects avoid refcount traffic entirely (see §11) — critical for scaling because `None`, `True`, `False`, and small integers are touched by every thread constantly.

- `Py_INCREF` / `Py_DECREF` gain atomic semantics; extensions that did racy `ob_refcnt` reads without the GIL will break.

Backend lens. If you write a C extension that releases the GIL (`Py_BEGIN_ALLOW_THREADS`), you must not touch `ob_refcnt` — or any `PyObject*` — while the GIL is released. The pattern is: INCREMENT everything you need *before* releasing, do non-Python work, re-acquire, then DECREMENT. The free-threaded build relaxes this but introduces new rules around critical sections (`Py_critical` API) — see Chapter 5.

5. Borrowed vs. Owned — The Full Catalog and Debugging

5.1 How to tell which you have

There is no runtime tag. You must read the docs. Rules of thumb:

- **Constructors and `PyNumber_*`, `PyUnicode_*` factories** → new (owned) reference.
- `PyList_GetItem`, `PyTuple_GetItem`, `PyDict_GetItem*`, `PySet_GetItem` → borrowed.
- `PyDict_GetItemWithError`, `PyObject_GetItem` → owned (note the inconsistency — `PyObject_GetItem` is owned because it may call `__getitem__` which returns a new ref).
- `PyErr_Occurred()` → borrowed.
- `PyImport_AddModule` → borrowed (a frequent leak source — people DECREMENT it).

CPython 3.12+ annotates many APIs with `PyAPI_FUNC` macros that static analyzers (clang, `cppcheck`) can use, but the ultimate source of truth is docs.python.org/c-api.

5.2 Debugging reference counts from Python

```
import sys

# sys.getrefcount – returns ob_refcnt, but note: calling it creates a
# temporary
# reference via the argument, so the reported count is always one
# higher than
# the "true" count when no call is in progress.
x = object()
print(sys.getrefcount(x)) # 2: one from x, one from the argument slot
y = x
print(sys.getrefcount(x)) # 3: x, y, + argument
del y
print(sys.getrefcount(x)) # 2 again

# gettotalrefcount – only in debug builds (--with-pydebug), counts
# every live PyObject
# Useful for leak hunting: snapshot before/after an operation
if hasattr(sys, "gettotalrefcount"):
    before = sys.gettotalrefcount()
    do_something()
    after = sys.gettotalrefcount()
    print(f"leaked ~{after - before} objects")

# gc.get_referrers / get_referents – who points at whom (GC-tracked
# objects only)
import gc
a = []
b = [a]
print(gc.get_referrers(a)) # [[<... a ...>]] – b refers to a
print(gc.get_referents(b)) # [...] – b's referents include a

# tracemalloc – correlates allocations to Python source lines (Chapter
# 4)
import tracemalloc
tracemalloc.start()
# ... exercise code ...
snap = tracemalloc.take_snapshot()
for stat in snap.statistics("lineno")[:5]:
    print(stat)
```

```

# Demo: getrefcount, id reuse, and is vs ==

import sys

# --- getrefcount increments are visible ---
s = "hello"
print(f"getrefcount(s) = {sys.getrefcount(s)}")
# 2+ (variable + arg + maybe intern cache)
t = s
print(f"after t = s: {sys.getrefcount(s)}")      # +1

# --- id reuse: CPython reuses addresses of freed objects ---
# Two objects that never overlap in lifetime may have the same id()
id_a = id(object())
id_b = id(object())
print(f"id reuse possible: {id_a == id_b} (often True – address recycling)")

# Classic gotcha: comparing ids across lifetimes
a = [1, 2, 3]
id_before = id(a)
del a
b = [4, 5, 6]
# id(b) MAY equal id_before – not meaningful, just pymalloc reuse
print(f"id(b) == id_before? {id(b) == id_before} – meaningless, b is a different object")

# --- is vs == : identity vs equality ---
# is checks ob_type + address identity (pointer equality); == calls tp_richcompare
x = 1000
y = 1000
print(f"x == y: {x == y}")    # True – value equality
print(f"x is y: {x is y}")
# False – different PyLongObjects (outside small-int cache)

x_small = 42
y_small = 42
print(f"small: 42 is 42 → {x_small is y_small}") # True – both point at cached singleton

# Strings: interning makes 'is' sometimes true, but never rely on it
a = "hello_world"
b = "hello_world"
print(f"'hello_world' is 'hello_world': {a is b} – CPython may intern literals at compile time")

import sys as _sys
c = _sys.intern("hello world with spaces")

```

```
d = _sys.intern("hello world with spaces")
print(f"interned: c is d → {c is d}") # True – explicit interning
```

Output on CPython 3.12 (your values may vary for `id`):

```
getrefcount(s) = 4
after t = s: 5
id reuse possible: True (often True ☐ address recycling)
id(b) == id_before? True ☐ meaningless, b is a different object
x == y: True
x is y: False
small: 42 is 42 ☐ True
'hello_world' is 'hello_world': True ☐ CPython may intern literals
at compile time
interned: c is d ☐ True
```

Rule. Never use `is` to compare values. Use `is` only for singletons (`is None` , `is True`) and for intentional identity checks ("is this the exact same object I cached?"). `==` is value equality; `is` is pointer equality.

6. Caches, Interning, and Free Lists

CPython avoids allocating the same small or common objects repeatedly. Three mechanisms:

6.1 Small integer cache — `[-5, 256]`

In `Objects/longobject.c` , at interpreter startup CPython pre-allocates a single `PyLongObject` for each integer in `[-5, 256]` (inclusive — 262 objects). Every occurrence of these values in Python code returns a pointer to the cached object, not a new allocation.

```

/* Objects/longobject.c – small integer cache */
#ifndef NSMALLPOSINTS
#define NSMALLPOSINTS      257  /* 0..256 */
#endif
#ifndef NSMALLNEGINTS
#define NSMALLNEGINTS      5    /* -5..-1 */
#endif

static PyLongObject small_ints[NSMALLNEGINTS + NSMALLPOSINTS];
// small_ints[5] is 0, small_ints[5+42] is 42, small_ints[0] is -5
// At startup each is initialized with ob_refcnt = immortal-or-high,
// ob_digit set

```

```

# All of these are the SAME object (pointer-identical) – no allocation
a = 42; b = 42
assert a is b           # True – both point at small_ints[47]

# Outside the range, each literal creates a distinct object (usually)
a = 1000; b = 1000
assert a is not b       # True in most contexts (separate allocations)
assert a == b           # True – equal value, different objects

# CPython's peephole / constant folding may still share within a single
code object:
import dis
def f(): return (1000, 1000)
# Both 1000s in the same function may be folded to one constant –
implementation detail

```

Why `[-5, 256]`? Empirically the most common integers in Python programs are small counters, indices, and byte values. Caching one byte's range plus a few negatives covers the hot set with negligible memory cost ($262 \times \sim 32 \text{ bytes} \approx 8 \text{ KB}$).

6.2 Interned strings

String interning deduplicates immutable strings so that equality can be tested by pointer comparison and dictionary lookup can use the cached hash.

What gets interned automatically:

- Identifiers and attribute names, string literals that look like identifiers, and names used in bytecode (`LOAD_NAME`, `STORE_ATTR`).
- Strings explicitly interned via `sys.intern()`.

- On some builds, all string literals in a code object may be interned at compile time (an optimization, not a guarantee).

```
import sys

# Automatic interning – identifiers
x = "hello"
y = "hello"
print(x is y) # True – both are interned literals in the same
               compilation unit

# Not automatically interned – contains space, not identifier-like
a = "hello world"
b = "hello world"
print(a is b) # Implementation-dependent – don't rely on it

# Explicit interning – guarantees pointer identity for hot keys
# Useful when you use the same string millions of times as dict keys
# (e BC: interning saves memory and makes dict lookup pointer-fast)
key = sys.intern("user_id_from_upstream_with_long_name")
# Subsequent sys.intern of the same value returns the same object

# Interned strings are immortal in 3.12+ (held in intern dict with
  immortal refs)
```

Interned strings live in a global `interned` dictionary (`PyUnicode_InternInPlace`) that holds **immortal** references in 3.12+, so they are never deallocated until interpreter shutdown.

6.3 Free lists — per-type recycling

For types that are allocated and freed at high frequency, CPython keeps a **free list**: a singly-linked list of deallocated but not yet `free()` 'd memory blocks that can be reused without calling the system allocator.

Type	Free list	Size	Where
<code>float</code>	<code>float_free_list</code>	Up to 100 (<code>PyFloat_MAXFREELIST</code>)	Object: <code>floatob</code>
<code>tuple</code> (small)	<code>free_list[PyTuple_MAXSAVESIZE]</code> (size-indexed, up to 20)	Up to 2000 per size	Object: <code>tupleob</code>
<code>list</code>	<code>free_list[PyList_MAXFREELIST]</code>	Up to 80	Object: <code>listobj</code>

Type	Free list	Size	Where
<code>dict</code> (historically)	Via pymalloc arenas	—	Object: dictobj
<code>frame</code>	<code>frame_free_list</code>	Unbounded in 3.11+ (was 200)	Object: frameob

```

/* Objects/floatobject.c – float free list (simplified) */
#define PyFloat_MAXFREELIST 100
static PyFloatObject *free_list = NULL;
static int numfree = 0;

static PyFloatObject *
PyFloat_FromDouble(double fval) {
    PyFloatObject *op = free_list;
    if (op != NULL) {
        free_list = (PyFloatObject *) Py_TYPE(op); // next pointer
        stashed in ob_type
        numfree--;
    } else {
        op = PyObject_MALLOC(sizeof(PyFloatObject));
    }
    op->ob_fval = fval;
    return op;
}
// On dealloc: if numfree < MAXFREELIST, push onto free_list instead of
freeing

```

Why free lists matter for backend services:

- **They make allocation $O(1)$ and cache-hot** — no `malloc` syscall, no page fault, just pointer chasing. Microbenchmarks that allocate millions of floats/tuples hit the free list path.
- **They hide leaks in memory profilers** — `tracemalloc` sees the free-list memory as still allocated. Use `sys.getallocatedblocks()` and `gc.get_count()` to distinguish.
- **They interact with `id` reuse** — a deallocated float's address may be immediately reused for the next float, making `id()` comparisons across lifetimes meaningless.

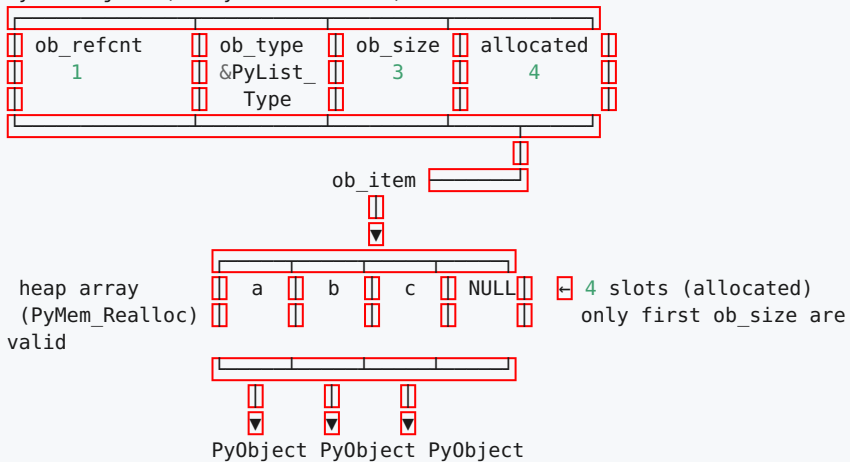
7. Memory Layout of Core Containers

7.1 list — A Dynamic Array with Over-Allocation

```
/* Include/cpython/listobject.h */
typedef struct {
    PyObject_VAR_HEAD    /* ob_size == len(list) */
    PyObject**ob_item;    /* pointer to array of PyObject* */
    Py_ssize_t allocated; /* slots allocated, >= ob_size */
} PyListObject;
```

list = [a, b, c] len=3 allocated=4 (example)

PyListObject (56 bytes on 64-bit)



Growth strategy — `list_resize()` in `Objects/listobject.c`:

```

# CPython's over-allocation formula (Objects/listobject.c: list_resize)
# new_allocated = new_size + (new_size >> 3) + (3 if new_size < 9 else 6)
# In words: ~12.5% overallocation, with a small constant for tiny lists.
#
# This gives amortized O(1) append while keeping waste bounded.

def cpython_list_over_allocation(n: int) -> int:
    """Mirror of CPython's actual formula."""
    if n == 0:
        return 0
    return n + (n >> 3) + (3 if n < 9 else 6)

for n in [0, 1, 4, 8, 9, 16, 100, 1000]:
    print(f"len={n:4d}
allocated={cpython_list_over_allocation(n):4d} waste={cpython_list_over_allocation(n)-n:4d}")

```

```

len=  0 allocated=  0 waste=  0
len=  1 allocated=  4 waste=  3
len=  4 allocated=  7 waste=  3
len=  8 allocated= 12 waste=  4
len=  9 allocated= 16 waste=  7
len= 16 allocated= 24 waste=  8
len=100 allocated=118 waste= 18
len=1000 allocated=1131 waste= 131

```

Diagram omitted for brevity — see surrounding prose.

Backend implications:

- **list.append in a hot loop is fast** — no per-append `realloc` until the over-allocated slack is exhausted. But **building a list of unknown size still does $O(\log n)$ reallocs** — if you know the size, `[None]*n` then assign is one allocation.
- **list never shrinks on pop** — `allocated` stays high after pops. Call `list.clear()` or `del list[:]` or reassign to reclaim. Long-lived lists that grew large then shrank still hold the large `ob_item` array — a subtle memory leak in services that reuse global lists.
- **ob_item is shared under PyList_GetItem borrowing** — mutating a list while iterating over borrowed pointers is unsafe in C.

7.2 tuple — Immutable, Inline, and Cache-Friendly

```
/* Include/cpython/tupleobject.h */
typedef struct {
    PyObject_VAR_HEAD      /* ob_size == len */
    PyObject *ob_item[1];  /* actually ob_size pointers, inline */
} PyTupleObject;
// Allocation: PyObject_MALLOC(sizeof(PyTupleObject) +
(n-1)*sizeof(PyObject*))
```

Unlike `list`, a tuple's items live **inline** — the `PyTupleObject` and its pointer array are one contiguous allocation. No separate `ob_item` malloc, no `allocated` field, no over-allocation. This makes tuples smaller and more cache-friendly, and allows the free-list optimization for small tuples (size ≤ 20).

Empty tuple is a singleton: `PyTuple_New(0)` returns the same immortal `&PyTuple_EmptyStruct` every time — `() is ()` is `True`.

7.3 dict — From Hash Table to Compact Representation

CPython's `dict` has gone through the most dramatic evolution of any builtin — directly relevant to backend memory because every object `__dict__`, every `kwargs`, and every JSON payload lives in a dict.

Pre-3.6 — classic hash table:

Classic dict (sparse table, ~2/3 empty for good probe performance)

DK key val key val ... dk_entries mixed
with dummy/empty slots
a 1 --- --- b 2
Memory: ~72 bytes + 8*table_size for empty dict; sparse ~ 50%+ waste

Python 3.6+ — compact dict (PyPy-inspired, by Mark Shannon):

Splits storage into a dense entries array (insertion-ordered) and a sparse indices array.

Pre-3.6 dict — sparse entries

dk_table (combined)
[entry0, empty, entry1,
empty, dummy, entry2]
insertion order NOT
preserved ~2/3 empty
slots for probing

Memory: 72 +
24*table_size Iteration:
skip empties

compacted in 3.6

3.6+ compact dict — split table

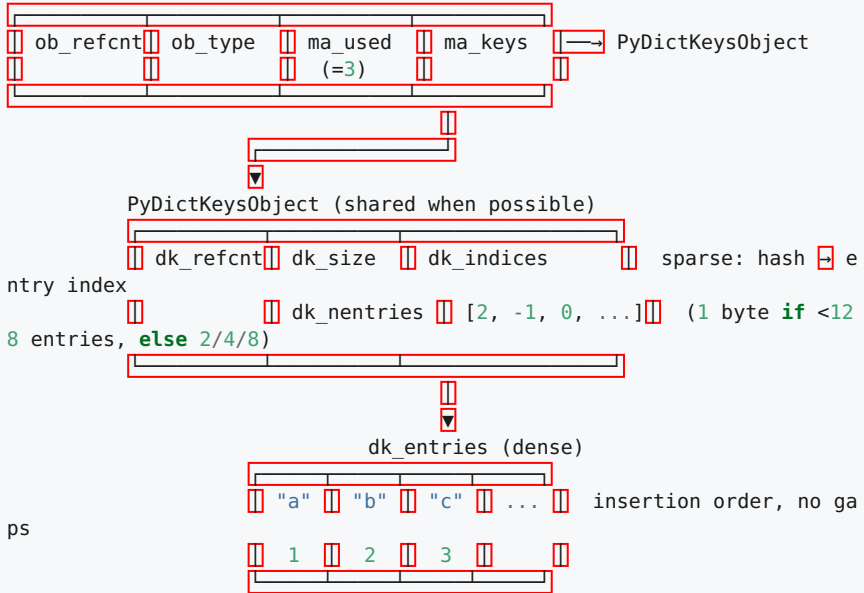
dk_indices sparse hash
→ index [2, -1, 0, -1, 1]
(bytes when small,
int16/int32 as needed)

index

dk_entries (dense,
insertion-ordered)
[entry0: 'a'→1, entry1:
'b'→2, entry2: 'c'→3] no
empty slots — iteration
is linear scan

Memory: ~56 + indices
+ dense entries
Iteration: just walk
dense array Ordered by
spec since 3.7

PyDictObject



Further evolution:

- **3.6** — compact dict, insertion-ordered (implementation detail).
- **3.7** — insertion order **guaranteed by language spec**.
- **3.8** — `PyDictKeysObject` can be **shared** between instances that have the same key set (split-table dict for objects: `obj.__dict__` shares keys with its class's instances, values stored per-instance). Dramatic memory saving for services with millions of objects of the same class.
- **3.12** — per-object inline caches and immortal shared keys (keys of classes that never change become immortal).

Measured impact: compact dict uses ~20-25% less memory than the old table for typical workloads and iterates faster (linear scan over dense array vs. skipping empties).

7.4 set — A Dict Without Values

`set` reuses `dict`'s hash-table machinery almost verbatim — `PySetObject` wraps a `set_table` that is structurally identical to `dict`'s `dk` entries but stores only keys (with a dummy value). Same compact

representation, same hash-probing logic, same performance characteristics as `dict`.

8. Unicode — PEP 393 Flexible String Representation

Before PEP 393 (Python 3.3), every Unicode string was stored as either UCS-2 (2 bytes/char) or UCS-4 (4 bytes/char) depending on compile-time flag — wasting 2–4× memory for ASCII-heavy workloads (typical for backends: JSON keys, URLs, log lines).

PEP 393 stores each string in the **narrowest encoding that fits its widest character**, chosen at creation time and never changed.

```
/* Include/cpython/unicodeobject.h – PEP 393 (simplified) */
typedef struct {
    PyObject_HEAD
    Py_ssize_t length;           /* number of code points */
    Py_hash_t hash;              /* cached hash, -1 if not computed */
    struct {
        unsigned int interned:2;
        unsigned int kind:3;    /* PyUnicode_1BYTE_KIND etc. */
        unsigned int compact:1;
        unsigned int ascii:1;
        unsigned int ready:1;
        /* ... */
    } state;
    wchar_t *wstr;              /* legacy, deprecated */
    // Followed by: compact layout stores data inline
} PyASCIIObject; // base for all unicode – confusing name, holds any
compact string

typedef struct {
    PyASCIIObject _base;
    Py_ssize_t utf8_length;
    char *utf8;                 /* cached UTF-8, lazily built */
    Py_ssize_t wstr_length;
} PyCompactUnicodeObject;

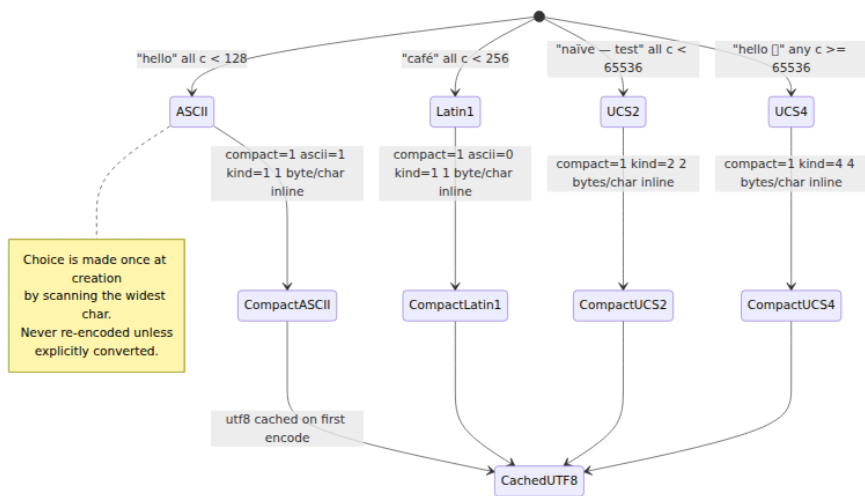
// The actual character data follows the struct header inline:
// PyASCIIObject + trailing buffer (kind-dependent)
```

Three `kind` values — the width is fixed per string:

kind	Constant	Bytes/ char	Max code point	Typical content
1	<code>PyUnicode_1BYTE_KIND</code>	1	U+00FF (Latin-1)	ASCII, JSON, URLs — the hot path for backends
2	<code>PyUnicode_2BYTE_KIND</code>	2	U+FFFF (BMP)	Most human languages
4	<code>PyUnicode_4BYTE_KIND</code>	4	U+10FFFF	Emoji, rare CJK

Two storage modes:

- **Compact** — header + character buffer in one allocation (the common case — all strings created normally).
- **Legacy / non-compact** — header points at separately-allocated buffer (only for strings resized via legacy APIs or `PyUnicode_FromStringAndSize` edge cases; rare).



Backend consequences:

- **ASCII strings cost 1 byte/char + ~49 bytes overhead** (header), not 4. A service holding millions of short ASCII keys (HTTP headers, metric names) uses $\sim 4\times$ less memory than pre-3.3.
- **`len(s)` is $O(1)$** — stored in `length`, not computed by scanning.

- **Indexing** `s[i]` is **O(1)** — `kind` determines the stride; no UTF-8 decoding.
- `sys.getsizeof("hello")` reveals the encoding: `49 + len * kind` (plus alignment). Measure it:

```
import sys

for s in ["hello", "café", "naïve", "hello 🐍"]:
    kind = "1B" if max(ord(c) for c in s) < 256 else ("2B" if
max(ord(c) for c in s) < 65536 else "4B")
    print(f"{s!r:20s} len={len(s):2d} kind~{kind} size={sys.getsizeof
f(s):3d} bytes")

# hello                len= 5  kind~1B  size= 54 bytes  (49 + 5*1)
# 'café'               len= 4  kind~1B  size= 53 bytes  (Latin-1 still
1B)
# 'naïve'              len= 5  kind~1B  size= 54 bytes  (still
Latin-1)
# 'hello 🐍'           len= 7  kind~4B  size= 77 bytes  (emoji forces
4B → 49+7*4)
```

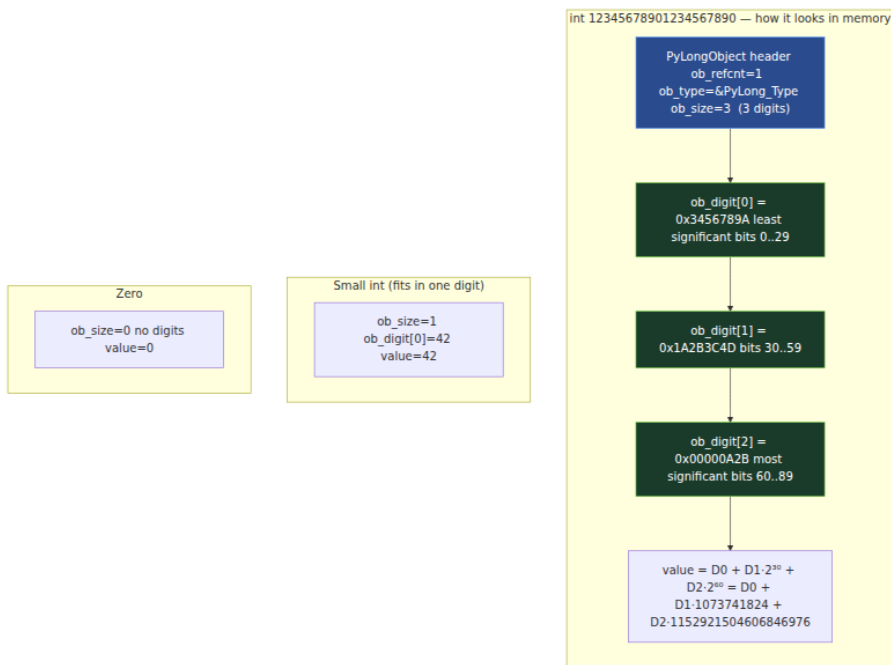
9. Integers — Digit Arrays and 30-Bit Limbs

Python integers have arbitrary precision — they grow as large as memory allows. The representation is a classic **big-integer digit array** in base 2^{30} .

```
/* Include/cpython/longobject.h */
typedef uint32_t digit;    // 30 bits used, 2 bits wasted (for carry)
typedef int32_t sdigit;

struct _longobject {
    PyObject_VAR_HEAD      /* ob_size = number of digits, sign
encodes sign */
    digit
ob_digit[1];              /* actually ob_size digits, little-endian */
};
// ob_size > 0 → positive, ob_size < 0 → negative, ob_size == 0 → zero
// ob_digit[0] is least significant, ob_digit[ob_size-1] most
significant

#define PyLong_SHIFT 30    /* bits per digit */
#define PyLong_BASE (1 << 30) /* 1073741824 */
#define PyLong_MASK (PyLong_BASE - 1)
```

Why 30 bits and not 32? So that intermediate products of two digits fit in 64 bits without overflow: $\text{PyLong_MASK} * \text{PyLong_MASK} < 2^{60} < 2^{64}$, and $\text{digit} * \text{digit} + \text{carry}$ fits in a `uint64_t` with room for carry propagation. On 32-bit builds, CPython historically used 15-bit digits for the same reason ($15+15 < 32$).

Concrete walkthrough:

```

# Inspecting the digit array (Python 3.12+ exposes this via _pylong or
ctypes;
# here we reconstruct the idea in pure Python)

SHIFT = 30
MASK = (1 << SHIFT) - 1

def to_digits(n: int) -> list[int]:
    """Decompose n into CPython's 30-bit digit array (little-
endian)."""
    if n == 0:
        return []
    neg = n < 0
    n = abs(n)
    digits = []
    while n:
        digits.append(n & MASK)
        n >>= SHIFT
    return digits # ob_size = len(digits) * (1 if not neg else -1)

for n in [0, 42, 2**30, 2**30 + 1, 12345678901234567890, -2**100]:
    d = to_digits(n)
    print(f"{n:>25} ob_size={len(d) * (-1 if n<0 else 1):3d}
digits={d[:4]}{'...' if len(d)>4 else ''}")

# Output:
#
#                                0 ob_size= 0 digits=[]
#                                42 ob_size= 1 digits=[42]
#                   1073741824 ob_size= 1 digits=[0, 1] - wait, that's
2 digits? No:
# Actually 2**30 == PyLong_BASE -> digits=[0, 1], ob_size=2
#                   1073741825 ob_size= 2 digits=[1, 1]
#                   12345678901234567890 ob_size= 3 digits=[..., ..., ...]

```

Cost model for backend work:

- **Small ints (one digit, $|n| < 2^{30} \approx 1\text{B}$) are one allocation, ~28 bytes.** Arithmetic is single-digit — fast.
- **Big ints cost $O(n)$ digits.** Addition/subtraction is $O(n)$ digit ops; multiplication uses Karatsuba for medium sizes and Toom-Cook / FFT for very large (thousands of digits). If your service does crypto or large counters, this matters.
- **The small-int cache (§6.1) means ints in $[-5, 256]$ never allocate at all.**

10. Immortal Objects — PEP 683 (Python 3.12)

10.1 The problem immortality solves

In a long-running backend process, a handful of objects are referenced from *everywhere*: `None`, `True`, `False`, `0`, `1`, `""`, small integers, interned strings, `PyType_Type`, and the type objects themselves. Every `Py_INCREF(None)` / `Py_DECREF(None)` touches the same cache line — a classic **false-sharing** / **cache-line bouncing** bottleneck on multi-core machines, and a blocker for free-threaded Python where refcount updates would need to be atomic.

Worse, these objects must never be deallocated — they live until interpreter shutdown — yet reference counting still dutifully increments and decrements them billions of times for no reason.

10.2 The mechanism

PEP 683 marks these objects as **immortal**: their `ob_refcnt` is set to a sentinel value that `Py_INCREF` / `Py_DECREF` recognize and skip.

```
/* Include/object.h – immortal sentinel (3.12+) */
// Actual value is platform-dependent; conceptually:
#define _Py_IMMORTAL_REFCNT ((Py_ssize_t) UINT_MAX) // 4294967295 on
32/64-bit
// In 3.12+ with 30-bit immortal flag, the check is:
#define _Py_IsImmortal(op) \
    ((op)->ob_refcnt == _Py_IMMORTAL_REFCNT)

// Py_INCREF / Py_DECREF become:
static inline void Py_INCREF(PyObject *op) {
    if (_Py_IsImmortal(op)) return; // no-op for immortals
    op->ob_refcnt++;
}
static inline void Py_DECREF(PyObject *op) {
    if (_Py_IsImmortal(op)) return; // no-op for immortals
    if (--op->ob_refcnt == 0) _Py_Dealloc(op);
}
```

Implementation note. Python 3.12.0–3.12.x uses `_Py_IMMORTAL_REFCNT = 4294967295 (UINT_MAX)`. The free-threaded build (3.13t) refines this with a separate `_Py_IMMORTAL_INITIAL_REFCNT` and per-object `_ob_tid` / `_ob_flags` fields. The semantics are identical: immortal objects are never deallocated and their refcount is never modified.

What is immortal in 3.12:

- `None`, `True`, `False`, `NotImplemented`, `Ellipsis`
- Small integers `[-5, 256]`
- Empty tuple `()`, empty bytes `b""`, empty string `""`, empty frozenset
- Interned strings
- All `PyTypeObject` instances for builtin types (`PyLong_Type`, `PyUnicode_Type`, etc.)
- Code objects for builtin methods in some builds

What is *not* immortal: anything created at runtime — user integers outside `[-5,256]`, non-interned strings, lists, dicts, user-defined type instances. These still use normal refcounting.

```
import sys

# Immortal objects have absurdly high refcounts (the sentinel) –
# visible via getrefcount
print(f"None refcount: {sys.getrefcount(None):,}" )      # ~4,294,967,xxx
# – the sentinel + temporaries
print(f"True refcount: {sys.getrefcount(True):,}" )
print(f"42 refcount:   {sys.getrefcount(42):,}" )        # also sentinel
# – 42 is in small-int cache
print(f"1000 refcount: {sys.getrefcount(1000):,}" )
# normal – small number, not sentinel

# You can see which objects are immortal via the C API (Python 3.12+):
# import _testcapi # not needed – just observe the refcount magnitude
# Normal objects: refcount in single/double digits
# Immortal objects: refcount near 2**32

x = object()
print(f"normal object refcount: {sys.getrefcount(x):,}" )
# 2 – genuinely counted
```

Backend impact:

- **No refcount traffic for hot singletons** — `None` checks, `True` / `False` returns, and small-int arithmetic no longer bounce a cache line between cores. Measured speedup for multi-threaded workloads touching these objects is significant and compounds under free-threaded Python.

- **No resurrection bugs** — immortal objects cannot be resurrected or double-freed; their `tp_dealloc` is never called during normal operation.
- **GC simplification** — immortal objects are excluded from GC traversal (they never need collection).

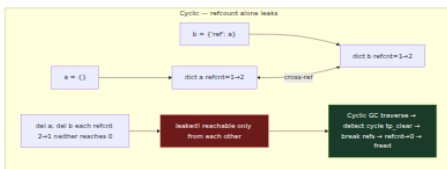
11. Reference Cycles — Why Counting Alone Is Not Enough

Reference counting reclaims memory *immediately* and *deterministically* — a property garbage-collected runtimes envy. But it has one fundamental blind spot: **cycles**.

```
# A minimal cycle — two objects referencing each other
a = {}
b = {"ref": a}
a["ref"] = b # a -> b -> a (cycle)

# Even after dropping both names, neither ob_refcnt reaches zero:
#   a's refcnt: 1 from b["ref"]
#   b's refcnt: 1 from a["ref"]
# Pure refcounting would leak both dicts forever.

del a, b
# Without the cyclic GC, those two dicts would be leaked until process
# exit.
# With GC (enabled by default), the generational collector will
# eventually
# traverse and break the cycle — see Chapter 4.
```



Not every cycle needs the GC. Only **container types** that can hold references to other Python objects and are flagged `Py_TPFLAGS_HAVE_GC` participate. Non-container types (`int`, `str`, `float`, `tuple` of non-containers in some optimizations) are not tracked — they cannot form cycles by themselves.

The GC (covered fully in Chapter 4) works in three generations with thresholds `(700, 10, 10)`:

- New GC-tracked containers start in generation 0.
- If they survive a collection, they are promoted to generation 1, then 2.
- Generation 2 is collected rarely — long-lived containers (module dicts, class dicts) settle there.

Objects with `__del__` finalizers add a complication: the GC cannot safely break cycles involving finalizers (order matters), so such cycles are moved to `gc.garbage` instead of being freed — a genuine leak until Python 3.4's `weakref`-based finalizer handling (PEP 442) made most cases collectable. Even today, a `__del__` that resurrects the object can still leak.

Backend lesson. Reference cycles in request-handling code (e.g., a handler closure capturing a response object that captures the handler) are collected, but not *immediately* — they wait for the next GC run. In latency-sensitive services, this shows up as periodic GC pauses. Mitigations: use `weakref` for parent pointers, break cycles explicitly (`del obj.cycle_ref`), or tune `gc.set_threshold` to collect `gen0` more frequently with shorter pauses. Chapter 4 covers tuning in depth.

12. Backend Lens — References, Extensions, and the GIL

12.1 Writing correct C extensions

Every production Python backend eventually touches C — whether via `numpy`, `cryptography`, `uvloop`, or a bespoke extension. Reference bugs in C are the leading cause of interpreter crashes and subtle leaks that only manifest under load.

The discipline, distilled:

```

/* Correct patterns – memorize these */

// Pattern 1: Factory returns new ref – caller must DECREF
PyObject *num = PyLong_FromLong(42);    // new ref, ob_refcnt=1
PyList_Append(list, num);                // Append INCREFs num
Py_DECREF(num);                          // balance FromLong – list still
holds one ref

// Pattern 2: Borrowed ref – INCREMENT if you keep it
PyObject *item = PyList_GetItem(list, 0); // borrowed!
Py_INCREF(item);                          // promote to owned if
storing
cache->slot = item;
// ... later, in dealloc:
Py_XDECREF(cache->slot);

// Pattern 3: Error paths must clean up owned refs
PyObject *a = PyLong_FromLong(1);        // owned
PyObject *b = PyLong_FromLong(2);        // owned
if (b == NULL) { Py_DECREF(a); return NULL; } // a would leak
otherwise
PyObject *result = PyNumber_Add(a, b);    // new ref
Py_DECREF(a);
Py_DECREF(b);
return result; // caller owns result

// Pattern 4: Stealing – PyList_SET_ITEM steals, PyList_SetItem steals
// PyList_SET_ITEM does NO error checking and NO INCREMENT – only use when
list is being built
PyObject *tmp = PyLong_FromLong(99);     // owned, refcnt=1
PyList_SET_ITEM(list, 0, tmp);            // steals – do NOT DECREF tmp,
list now owns it
// PyList_SetItem does the same but DECREFs the old item and handles
errors

// Pattern 5: GIL + refcounts – never touch ob_refcnt without GIL
Py_BEGIN_ALLOW_THREADS
// ... expensive non-Python work (no PyObject* access here) ...
Py_END_ALLOW_THREADS
// GIL re-acquired – now safe to INCREMENT/DECREMENT again

```

Common crash signatures and their causes:

Symptom	Likely cause
Fatal Python error: deallocating None / corrupted ob_refcnt	Double-DECREF or DECREF of borrowed ref

Symptom	Likely cause
Memory grows without bound, RSS climbs linearly	Missing DECREF on owned ref (leak)
Intermittent segfault under threading	DECREF without GIL, or borrowed ref used after owner mutated
<code>SystemError: <method> returned NULL without exception</code>	Forgot <code>PyErr_SetString</code> before returning NULL

12.2 GIL implications — today and under free-threaded Python

Concern	GIL build (default through 3.12)	Free-threaded build (3.13t, PEP 703)
<code>ob_refcnt</code> update	Plain <code>++ / --</code> , safe because only GIL holder runs	Atomic or biased — safe without GIL but slower single-threaded
Borrowed refs	Safe as long as owner not mutated concurrently (GIL serializes)	Unsafe without explicit critical section — use <code>Py_critical</code> API
<code>Py_INCREF / Py_DECREF</code>	Non-atomic, GIL-protected	Atomic, thread-safe
Immortal objects	Optimization to avoid cache-line bouncing	Essential for scalability — avoids atomic contention on hot singletons
Extension compatibility	Assumes GIL — most extensions work	Must be ported — extensions that touch <code>ob_refcnt</code> without GIL need updating

For backend operators: the free-threaded build does not change Python-level reference semantics — `sys.getrefcount` still works, `is` still checks identity, cycles still need GC. What changes is that **C extensions must not assume the GIL protects refcounts**. If you maintain extensions, audit every `Py_INCREF / Py_DECREF` that might run without the GIL, and adopt `PyMutex / PyCriticalSection` where needed. Chapter 5 covers the free-threaded transition in depth.

12.3 Observing refcount behavior in production

```
# Debug build – total live objects
python3-dbg -c "import sys; print(sys.gettotalrefcount())"
# 25000 – baseline; compare before/after a suspect operation

# GC stats – are cycles accumulating?
python3 -c "
import gc
gc.set_debug(gc.DEBUG_STATS) # logs to stderr on each collection
# run workload...
print(gc.get_stats())
# [{'collections': 12, 'collected': 400, 'uncollectable': 0}, ...]
# uncollectable > 0 means __del__ cycles leaking to gc.garbage
"

# tracemalloc – who is allocating?
python3 -X tracemalloc -c "
import tracemalloc, json
tracemalloc.start()
# ... serve 1000 requests ...
snap = tracemalloc.take_snapshot()
for stat in snap.statistics('lineno')[:5]:
    print(stat)
"

# objgraph (third-party) – visualize reference graphs for leak hunting
pip install objgraph
python3 -c "
import objgraph, gc
# After a suspected leak:
objgraph.show_most_common_types(limit=10)
# dict 50000 list 12000 MyHandler 999 <- MyHandler count growing?
objgraph.show_backrefs([my_leaked_obj], filename='refs.png')
"
```

Key Takeaways

- Every Python value is a `PyObject` — a heap allocation beginning with `ob_refcnt` and `ob_type`. `PyVarObject` adds `ob_size` for variable-length types. This prefix is how CPython implements a uniform object model in C.
- `PyTypeObject` is itself a `PyObject` (whose `ob_type` is `PyType_Type`). Its slots (`tp_dealloc`, `tp_traverse`, `tp_call`, `tp_vectorcall`, etc.) define the behavior of all instances. Forgetting `Py_TPFLAGS_HAVE_GC`

- + `tp_traverse / tp_clear` on a container type means cycles through it will leak.
- Reference counting is deterministic and immediate: `Py_INCREF` increments, `Py_DECREF` decrements and deallocates when the count reaches zero. `Py_XINCREF / Py_XDECREF` handle `NULL`. Every `PyObject*` in C is either an owned (new) reference you must `DECREF` or a borrowed reference you must not — the distinction is per-function contract, not per-type.
- Borrowed references (`PyList_GetItem`, `PyDict_GetItem`, `PyTuple_GetItem`) are valid only while the owner lives and the slot is unchanged. Promote with `Py_INCREF` if you need to keep them. Stealing APIs (`PyList_SET_ITEM`, `PyTuple_SET_ITEM`) take ownership and must not be paired with a `DECREF`.
- `sys.getrefcount(obj)` reports `ob_refcnt` plus one for the argument temporary; `sys.gettotalrefcount()` (debug builds) counts all live objects. `gc.get_referrers / get_referents` and `tracemalloc` are the right tools for leak hunting; `id()` reuse across lifetimes is an allocator artifact, not identity.
- Small integers `[-5, 256]` are singletons cached at startup; interned strings (`sys.intern`) deduplicate hot keys and are immortal in 3.12+. Per-type free lists (float, tuple, list, frame) recycle deallocated memory without calling `free()`, making hot allocation paths $O(1)$ and cache-friendly.
- `list` is a dynamic array with ~12.5% over-allocation and a separately-allocated `ob_item` array; `tuple` is immutable with inline storage and free-list recycling for small sizes. `dict` since 3.6 uses a compact split table (sparse indices + dense insertion-ordered entries) that saves 20–25% memory and guarantees order since 3.7; `set` reuses the same machinery.
- PEP 393 stores each string in the narrowest width that fits (1/2/4 bytes per char), chosen once at creation. ASCII-heavy backend workloads (JSON, URLs, headers) pay 1 byte/char, not 4.
- Python `int` is a little-endian array of 30-bit digits (base 2^{30}), with `ob_size` encoding sign and digit count. Single-digit ints cover $|n| < 2^{30}$; larger values cost $O(n)$ digits and $O(n)$ arithmetic.
- PEP 683 (3.12) makes hot singletons (`None`, `True`, `False`, small ints, interned strings, builtin types) immortal — their `ob_refcnt` is a

sentinel that `Py_INCREF / Py_DECREF` skip, eliminating cache-line bouncing and enabling free-threaded scaling.

- Reference counting alone cannot reclaim cycles (`a -> b -> a`). The generational cyclic GC (Chapter 4) traverses `Py_TPFLAGS_HAVE_GC` objects via `tp_traverse / tp_clear` to detect and break cycles. In latency-sensitive services, tune GC thresholds or break cycles explicitly to avoid periodic pause spikes.
 - In C extensions, every owned reference must be balanced, every borrowed reference must be promoted before storage, and `ob_refcnt` must never be touched without the GIL (or without atomic/critical-section discipline in free-threaded builds). `Py_BEGIN_ALLOW_THREADS` regions must not access `PyObject*` at all.
-

Further Reading

1. **CPython source** — `Include/object.h` — Definitive definitions of `PyObject`, `PyVarObject`, `PyTypeObject`, `Py_INCREF / Py_DECREF`. Start here for any question about header layout or type slots. <https://github.com/python/cpython/blob/main/Include/object.h>
2. **CPython source** — `Include/cpython/longobject.h` — `PyLongObject` digit-array definition, `PyLong_SHIFT / PyLong_MASK`, and `longobject` API. <https://github.com/python/cpython/blob/main/Include/cpython/longobject.h>
3. **CPython source** — `Objects/longobject.c (small_ints cache)` — Initialization of the `[-5, 256]` small-integer singletons and digit-array arithmetic (Karatsuba, Toom-Cook). <https://github.com/python/cpython/blob/main/Objects/longobject.c>
4. **CPython source** — `Objects/listobject.c (list_resize)` — Over-allocation formula and growth strategy for `list`. <https://github.com/python/cpython/blob/main/Objects/listobject.c>
5. **PEP 393 — Flexible String Representation** — Motivation, design, and benchmarks for the 1/2/4-byte-per-char Unicode representation. <https://peps.python.org/pep-0393/>
6. **PEP 683 — Immortal Objects, Using a Fixed Refcount** — Rationale, sentinel mechanism, and interaction with free-threaded Python. <https://peps.python.org/pep-0683/>

7. **Python C-API docs — Reference Counts** — Borrowed vs. new references, `Py_INCREF` / `Py_DECREF` / `Py_XINCREF` / `Py_XDECREF` , and per-function ownership contracts. <https://docs.python.org/3/c-api/intro.html#reference-counts>
8. **Python C-API docs — Type Objects** — `PyTypeObject` fields, `PyType_Spec` / `PyType_FromSpec` , `tp_traverse` / `tp_clear` for GC support. <https://docs.python.org/3/c-api/typeobj.html>
9. **CPython source — `Objects/dictobject.c` + `Objects/unicodeobject.c`** — Compact-dict implementation (split table, shared keys) and PEP 393 compact Unicode internals. <https://github.com/python/cpython/blob/main/Objects/dictobject.c>
10. **PEP 703 — Making the Global Interpreter Lock Optional in CPython** — Biased reference counting, immortal objects as a scalability primitive, and extension-porting notes for free-threaded builds. <https://peps.python.org/pep-0703/>

Chapter 3 — Bytecode, the ceval Loop, and Adaptive Specialization (PEP 659)

What this chapter covers. Every Python function you deploy is not executed as source text — it is compiled to a compact bytecode, packed into a `PyCodeObject` , and run by a stack-based virtual machine in `Python/ceval.c` . Since Python 3.11 that VM is no longer a straightforward switch loop: it watches itself execute, rewrites its own bytecode in place, and specializes hot opcodes for the types it actually sees (PEP 659). This chapter dissects the full evaluation pipeline — wordcode format, opcode families, the value stack and frame stack, jump targets, the new exception table, `ceval` 's computed-goto dispatch, inline caches, quickening, guards, and deoptimization — with real `dis` output, `co_code` inspection, and before/after specialization examples. It closes with a teaser of the copy-and-patch JIT (PEP 744, 3.13+) and a backend lens on why specializing your API hot loop is worth ~25% fleet-wide.

Learning goals — after this chapter you should be able to:

- Explain Python's bytecode format (wordcode since 3.6, cache-aware wordcode since 3.11): 2-byte `opcode + oparg` units, `HAVE_ARGUMENT`, `EXTENDED_ARG`, and why bytecode is version-specific.
- Use `dis`, `dis.get_instructions()`, `opcode`, and direct `PyCodeObject` field inspection (`co_code`, `co_consts`, `co_names`, `co_varnames`, `co_exceptiontable`, `co_positions`) to read any function's compiled form.
- Classify opcodes into families (stack manipulation, control flow, object/name access, function call, exception/unwind) and trace how each manipulates the value stack.
- Describe jump targets, absolute vs. relative offsets, and the 3.11+ exception table that replaced the old block stack — and parse its entries.
- Explain the inline-cache mechanism: how `CACHE` bytes trail certain opcodes, how many entries each opcode reserves, and what `_inline_cache_entries` encodes.
- Walk through `ceval.c`'s main loop: frame setup, the `TARGET(opcode):` dispatch via computed `goto`, `DISPATCH / NEXTOP` macros, and where the GIL, eval breaker, and pending signals are checked.
- Contrast the legacy `PyFrameObject` heap-allocated frame with the 3.11+ C-stack frame (`_PyInterpreterFrame` / `cframe`) and its impact on call overhead.
- Explain PEP 659 end-to-end: quickening, adaptive counters, specialization for `BINARY_OP`, `LOAD_GLOBAL`, `CALL / PRECALL`, `COMPARE_OP`, `LOAD_ATTR / STORE_ATTR`, `BINARY_SUBSCR`, inline-cache guards, and deoptimization on guard failure — with adaptive `dis` examples.
- Quantify the 3.11 speedup (~25% geometric mean on pyperformance, up to 60% on micro-benchmarks) and why it compounds in I/O-bound API handlers.
- Sketch the copy-and-patch JIT introduced in 3.13 (PEP 744): tier-1 specialization → tier-2 uops → stenciled machine code, and why it is still experimental.
- Profile bytecode hotspots in production with `perf + py-spy / austin`, mapping samples back to opcodes and specializing-vs-generic mix.

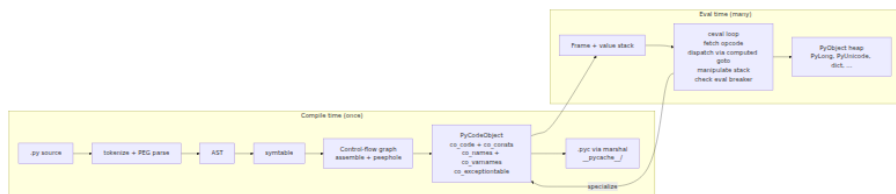
Prerequisites. Chapter 1 traced source → AST → `PyCodeObject` and `.pyc` caching; Chapter 2 dissected `PyObject`, `ob_refcnt`, and the type system. This chapter assumes you have run `python -m dis` and can skim C. Volume 13, Chapter 6 gives the service-level view of the interpreter; here you open `ceval.c`.

1. Why bytecode is a backend concern

Backend engineers treat Python as "interpreted" and move on. That mental model costs you when you need to explain:

- **Why a hot loop is 25% faster on 3.11 without changing a line of Python.** The answer is in the bytecode — it specializes itself.
- **Why `perf top` shows `PyEval_EvalFrameEx` / `_PyEval_EvalFrameDefault` at the top of every profile.** That is the `ceval` loop; every opcode dispatch is a branch.
- **Why coverage, debuggers, and `pydantic / dataclasses` code generation all speak `PyCodeObject`.** They rewrite `co_code` or wrap it.
- **Why `.pyc` invalidation, import time, and cold-start latency correlate.** Import executes bytecode; specializing bytecode reduces work per import.

CPython compiles each code block (module, function, class body, comprehension, lambda) to a `PyCodeObject` once, then evaluates it many times. The compilation from AST to bytecode (`Python/compile.c`) is cheap; the evaluation loop (`Python/ceval.c`, ~6,000 lines) is where your request spends its life.



Understanding the shape of `co_code` and how `ceval` dispatches it is prerequisite to reading any CPython profile.

2. Bytecode format — wordcode and cache-aware wordcode

2.1 Wordcode since 3.6

Before 3.6 CPython used *bytecode* with variable-length instructions: 1 byte for opcodes without arguments, 3 bytes (`opcode` , `arg_low` , `arg_high`) for opcodes with arguments. Every `LOAD_FAST 0` cost 3 bytes and a branch to decode the length.

Since 3.6 (PEP 5110 / wordcode) every instruction is exactly **2 bytes**: one byte `opcode` , one byte `oparg` . Instructions that need no argument set `oparg` to 0. Instructions that need a larger argument use `EXTENDED_ARG` prefix instructions. The opcode stream is therefore a flat `bytes` object (`co_code`) whose length is always even, trivially indexable, and friendly to computed-goto dispatch.

```
co_code as bytes: [op0][arg0] [op1][arg1] [op2][arg2] ...  
                  1 byte 1 byte 1 byte 1 byte
```

Opcodes `< 90` historically had no argument in CPython 2 (`HAVE_ARGUMENT` = 90); since wordcode this boundary matters less for decoding but is still exposed by `dis.have_argument()` and `opcode.HAVE_ARGUMENT` for tooling.

`EXTENDED_ARG` (opcode 144) extends the next instruction's argument: each `EXTENDED_ARG` shifts the accumulated argument left by 8 bits. Two `EXTENDED_ARG`s plus one real opcode can encode a 24-bit argument — needed for functions with >256 names or constants.

```

import dis, opcode

print(f"Python {opcode.inline_cache_entries.__class__.__name__} -
HAVE_ARGUMENT={opcode.HAVE_ARGUMENT}")
print(f"EXTENDED_ARG={opcode.opmap['EXTENDED_ARG']} CACHE={opcode.opmap['CACHE']}")

def many_names():
    # force EXTENDED_ARG by having >256 distinct names – synthetic example
    pass

# Minimal wordcode demo
def add(a, b):
    return a + b

print(add.__code__.co_code.hex(' '))
# 97 00 7c 00 7c 01 7a 00 00 00 53 00 – 7 instructions × 2 bytes
dis.dis(add)

```

Output on CPython 3.11:

5	0 RESUME	0
	2 LOAD_FAST	0 (a)
	4 LOAD_FAST	1 (b)
	6 BINARY_OP	0 (+)
	10 RETURN_VALUE	

Each line is one 2-byte unit. `RESUME` at offset 0 is the 3.11+ entry-point marker (replaces the old `RESUME`-less prologue). Offsets count in bytes, so they advance by 2 per non-cache instruction.

`SHOW_CACHES` reveals what hides between those lines:

```
dis.dis(add, show_caches=True)
```

5	0 RESUME	0
	2 LOAD_FAST	0 (a)
	4 LOAD_FAST	1 (b)
	6 BINARY_OP	0 (+)
	8 CACHE	0
	10 RETURN_VALUE	

The `CACHE` at offset 8 is not a real opcode — it is inline-cache storage trailing `BINARY_OP`. `dis` hides `CACHE` entries by default; `show_caches=True` exposes them. The bytecode array's length (12 bytes)

already includes the cache word — `co_code` length is not `num_opcodes × 2` in 3.11+, it is `(num_opcodes + num_cache_words) × 2`.

2.2 The code object — `PyCodeObject` fields

Every compiled block becomes an immutable `PyCodeObject` (`Objects/codeobject.c`, `Include/cpython/code.h`). The fields you will inspect constantly:

```
import dis, types

def demo(x, y, *, flag=True):
    z = x + y
    if flag:
        return z
    for i in range(3):
        z += i
    return z

co = demo.__code__
print(f"co_name={co.co_name}  co_qualname={co.co_qualname}
co_firstlineno={co.co_firstlineno}")
print(f"co_argcount={co.co_argcount}  co_kwonlyargcount={co.co_kwonlyargcount}")
print(f"co_varnames={co.co_varnames}")
print(f"co_names={co.co_names}")
print(f"co_consts={co.co_consts}")
print(f"co_code length={len(co.co_code)}  hex={co.co_code.hex(' ')}")
print(f"co_stacksize={co.co_stacksize}  co_flags={hex(co.co_flags)}")
print(f"has exception table: {hasattr(co, 'co_exceptiontable')}")
print(f"has positions: {hasattr(co, 'co_positions')}")
print(f"co_positions: {list(co.co_positions())[6]}")
if co.co_exceptiontable:
    print(f"exception table bytes: {co.co_exceptiontable.hex(' ')}")
```

Typical output (3.11):

```
co_name=demo co_qualname=demo co_firstlineno=3
co_argcount=2 co_kwonlyargcount=1
co_varnames=('x', 'y', 'flag', 'z', 'i')
co_names=('range',)
co_consts=(None, 3)
co_code length=... hex=97 00 ...
co_stacksize=3 co_flags=0x43
has exception table: True
has positions: True
co_positions: [(3, 3, 0, 0), (4, 4, 4, 11), ...]
exception table bytes: ...
```

Key fields:

Field	Meaning
<code>co_code</code>	<code>bytes</code> — flat wordcode + inline-cache words
<code>co_consts</code>	<code>tuple</code> — literals and nested code objects (<code>LOAD_CONST</code> indexes here)
<code>co_names</code>	<code>tuple</code> — global/attribute names (<code>LOAD_GLOBAL</code> , <code>LOAD_ATTR</code> via index)
<code>co_varnames</code>	<code>tuple</code> — locals + args (<code>LOAD_FAST</code> / <code>STORE_FAST</code> index here)
<code>co_cellvars</code> / <code>co_freevars</code>	Closure cells (see Chapter 7)
<code>co_stacksize</code>	Maximum value-stack depth the compiler computed — used to size the frame's stack
<code>co_argcount</code> , <code>co_kwonlyargcount</code> , <code>co_posonlyargcount</code>	Arity metadata for <code>CALL</code>
<code>co_flags</code>	<code>CO_OPTIMIZED</code> , <code>CO_NEWLOCALS</code> , <code>CO_VARARGS</code> , <code>CO_GENERATOR</code> , <code>CO_COROUTINE</code> , etc.
<code>co_firstlineno</code>	First line number for tracebacks and <code>co_positions</code>
<code>co_lnotab</code> (≤ 3.9) / <code>co_positions</code> + <code>co_lines</code> (3.10+) / <code>co_exceptiontable</code> (3.11+)	Line/column and exception mapping
<code>co_qualname</code> (3.11+)	Dotted qualname for better tracebacks

The compiler computes `co_stacksize` statically so the frame can allocate the value stack as a contiguous C array — no dynamic growth during evaluation. Underestimating would corrupt the frame; overestimating wastes a few pointers.

2.3 `dis`, `dis.get_instructions()`, and `opcode`

Three APIs expose the same bytes at different abstraction levels:

```
import dis, opcode

def sample(a, b):
    return a + b

# 1. Pretty-print (what you paste into PRs)
dis.dis(sample)

# 2. Structured iteration (what tools consume)
for instr in dis.get_instructions(sample):
    # Instruction(opcode, opname, arg, argval, argrepr, offset,
    #             starts_line, is_jump_target, positions)
    print(instr)

# 3. Opcode tables (what ceval.c and compile.c agree on)
print(opcode.opmap['BINARY_OP'])      # 122
print(opcode.opname[122])             # 'BINARY_OP'
print(opcode.HAVE_ARGUMENT)           # 90 – still exposed
print(opcode.hasconst)                # opcodes that index co_consts
print(opcode.hasname)                 # opcodes that index co_names
print(opcode.hasjabs, opcode.hasjrel) # absolute vs. relative jumps
print(opcode._inline_cache_entries)    # list[256] – cache words per
opcode
```

`dis.get_instructions()` is the workhorse for bytecode tooling (coverage, import hooks, `wrapt`, `bytecode` library). Each `Instruction` carries `positions` (line/column from `co_positions`) and `is_jump_target` — essential for control-flow analysis.

```
def loop(n):
    s = 0
    for i in range(n):
        s += i
    return s

for ins in dis.get_instructions(loop):
    print(f"offset={ins.offset:3d} {ins.opname:30s}"
          f"arg={str(ins.arg):5s} "
          f"target={ins.is_jump_target} line={ins.starts_line}"
          f"positions={ins.positions}")

# Jump targets are derived from JUMP_* and FOR_ITER opargs
# Exception table entries supplement them for except/finally handlers
```

`opcode._inline_cache_entries` is a 256-element list indexed by opcode number, introduced in 3.11. Non-zero entries tell the compiler how many trailing `CACHE` words to reserve and tell `ceval` where the inline cache lives:

```
import opcode

for name, num in sorted(opcode.opmap.items(), key=lambda x: x[1]):
    n = opcode._inline_cache_entries[num]
    if n:
        print(f"{num:3d} {name:30s} cache_words={n}")
```

On 3.11:

25	BINARY_SUBSCR	cache_words=4
60	STORE_SUBSCR	cache_words=1
95	STORE_ATTR	cache_words=4
106	LOAD_ATTR	cache_words=4
107	COMPARE_OP	cache_words=2
116	LOAD_GLOBAL	cache_words=5
122	BINARY_OP	cache_words=1
160	LOAD_METHOD	cache_words=10
166	PRECALL	cache_words=1
171	CALL	cache_words=4

Every other opcode has 0 cache words. The `CACHE` opcode (0) is never emitted by the compiler as a real instruction — it only appears as padding that `ceval` treats as inline-cache storage.

3. Opcode families

CPython 3.11 has ~160 distinct opcodes (110 in `opcode.opmap` plus ~50 adaptive/specialized variants that share numeric values or live above 150). They cluster into families by what they do to the value stack and the frame:

3.1 Stack manipulation

The frame's value stack is the VM's working memory. Stack-manipulation opcodes shuffle values without touching Python objects:

Opcode	Stack effect	Notes
<code>PUSH_NULL</code>	<code>... → ... NULL</code>	Pushes a sentinel for <code>CALL</code> (3.11+ calling convention)
<code>COPY i</code>	<code>... a b → ... a b a</code>	Duplicates i-th element from top (1-indexed)
<code>SWAP i</code>	<code>... a b → ... b a</code>	Swaps top with i-th element down
<code>POP_TOP</code>	<code>... x → ...</code>	Discards top
<code>NOP</code>	<code>... → ...</code>	No-op; used as quickening placeholder and for <code>EXTENDED_ARG</code> alignment

`COPY` and `SWAP` (3.11+) replaced the older `DUP_TOP`, `DUP_TOP_TWO`, `ROT_TWO`, `ROT_THREE`, `ROT_FOUR` family, which encoded specific depths as distinct opcodes. The new encoding is more compact and reduces dispatch pressure.

3.2 Object and name access

These opcodes move values between the stack and the frame's namespaces or the object heap:

```
import dis

def access_demo(x, y):
    g = globals()["key"]      # LOAD_GLOBAL, BINARY_SUBSCR
    a = x.foo                 # LOAD_FAST, LOAD_ATTR
    y.bar = a                 # LOAD_FAST, STORE_ATTR
    return g

dis.dis(access_demo)
```

2	0 RESUME	0
	2 LOAD_GLOBAL	0 (NULL + globals)
	14 LOAD_CONST	1 ('key')
	16 BINARY_SUBSCR	
	20 STORE_FAST	2 (g)
3	22 LOAD_FAST	0 (x)
	24 LOAD_ATTR	0 (foo)
	36 LOAD_FAST	1 (y)
	38 LOAD_FAST	3 (a)
	40 STORE_ATTR	1 (bar)
...		

Families:

- **Locals/cells:** `LOAD_FAST`, `STORE_FAST`, `DELETE_FAST`, `LOAD_DEREF`, `STORE_DEREF`, `LOAD_CLOSURE`, `COPY_FREE_VARS`, `MAKE_CELL`. `LOAD_FAST` / `STORE_FAST` index `co_varnames` and are the fastest name access — a direct `frame->localsplus[i]` array dereference with no dict lookup.
- **Globals/builtins:** `LOAD_GLOBAL`, `STORE_GLOBAL`, `DELETE_GLOBAL`, `LOAD_NAME`, `STORE_NAME`. `LOAD_GLOBAL` has 5 cache words in 3.11 because it can specialize to `LOAD_GLOBAL_MODULE` (found in `globals` dict) or `LOAD_GLOBAL_BUILTIN` (found in `builtins`).
- **Attributes/subscripts:** `LOAD_ATTR`, `STORE_ATTR`, `DELETE_ATTR`, `BINARY_SUBSCR`, `STORE_SUBSCR`, `DELETE_SUBSCR`. Attribute access is the single hottest opcode family in most backends — every `request.method`, `row.id`, `self.pool` is a `LOAD_ATTR`.

3.3 Control flow

```
def control(n, items):
    if n > 0:                                # COMPARE_OP,
POP_JUMP_FORWARD_IF_FALSE
        for x in items:                      # GET_ITER, FOR_ITER,
JUMP_BACKWARD
            if x is None:                    # POP_JUMP_FORWARD_IF_NONE
                continue                    # JUMP_BACKWARD
            elif x == 42:                    # COMPARE_OP,
POP_JUMP_FORWARD_IF_FALSE
                break
            else:                            # FOR_ITER fallthrough vs. break
                return "exhausted"
    return "done"

dis.dis(control)
```

2	0 RESUME	0
	2 LOAD_FAST	0 (n)
	4 LOAD_CONST	1 (0)
	6 COMPARE_OP	4 (>)
	12 POP_JUMP_FORWARD_IF_FALSE	6 (to 26)
3	14 GET_ITER ... FOR_ITER ... JUMP_BACKWARD ...	

Control-flow opcodes:

- **Jumps:** JUMP_FORWARD, JUMP_BACKWARD, JUMP_BACKWARD_NO_INTERRUPT (the latter skips the eval-breaker check for tight loops), POP_JUMP_FORWARD_IF_FALSE/TRUE/NONE/NOT_NONE, POP_JUMP_BACKWARD_IF_* (3.11+ backward conditional jumps for while loops).
- **Iteration:** GET_ITER, FOR_ITER, GET_YIELD_FROM_ITER, SEND (generators/coroutines — Chapter 7).
- **Exception unwinding:** PUSH_EXC_INFO, CHECK_EXC_MATCH, POP_EXCEPT, RERAISE, WITH_EXCEPT_START, BEFORE_WITH — but the per-opcode block stack (SETUP_FINALLY, SETUP_WITH in ≤3.10) is gone; unwinding is now driven by the exception table (Section 4).

Jump arguments are **relative byte offsets** from the current instruction's end, scaled by instruction size. hasjrel vs. hasjabs in opcode distinguishes them, but since 3.10 most jumps are relative (absolute jumps were mostly eliminated when co_code became wordcode). dis renders jump targets as >> markers on the target line.

3.4 Function calls

The calling convention changed significantly in 3.11 (PEP 654 exception groups + adaptive calls) and again in 3.12 (cleaned up to CALL / CALL_FUNCTION_EX with PUSH_NULL). In 3.11 the sequence is PUSH_NULL (or LOAD_GLOBAL pushing NULL implicitly) → PRECALL → CALL:

```
def callee(a, b): return a + b

def caller(x):
    return callee(x, 1)

dis.dis(caller)
```

5	0 RESUME	0
	2 LOAD_GLOBAL	1 (NULL + callee)
	14 LOAD_FAST	0 (x)
	16 LOAD_CONST	1 (1)
	18 PRECALL	2
	22 CALL	2
	32 RETURN_VALUE	

With `show_caches=True` :

	2 LOAD_GLOBAL	1 (NULL + callee)
	4 CACHE	0 (x5 - LOAD_GLOBAL cache)
	14 LOAD_FAST	0 (x)
	16 LOAD_CONST	1 (1)
	18 PRECALL	2
	20 CACHE	0
	22 CALL	2
	24 CACHE	0 (x4 - CALL cache)

- **PRECALL** reserves space on the stack for the callee's frame and records the argument count. Its 1-word cache stores the adaptive counter.
- **CALL** does the actual dispatch — it can specialize to `CALL_PY_EXACT_ARGS`, `CALL_PY_WITH_DEFAULTS`, `CALL_NO_KW_BUILTIN_0`, `CALL_METHOD_DESCRIPTOR`, etc. depending on what callable was observed.

`KW_NAMES` (opcode 172, no cache) handles keyword-argument name tuples for `f(a=1, b=2)` — it sits immediately before `PRECALL` when keywords are present.

3.5 Other families

- **Arithmetic/comparison:** `BINARY_OP` (replaces `BINARY_ADD`, `BINARY_MULTIPLY`, etc. — the `oparg` selects the operator), `COMPARE_OP`, `IS_OP`, `CONTAINS_OP`, `UNARY_NEGATIVE/NOT/INVERT`.
- **Containers:** `BUILD_TUPLE`, `BUILD_LIST`, `BUILD_SET`, `BUILD_MAP`, `BUILD_CONST_KEY_MAP`, `BUILD_STRING`, `LIST_APPEND`, `SET_ADD`, `MAP_ADD`, `LIST_EXTEND`, `SET_UPDATE`, `DICT_MERGE`, `DICT_UPDATE`.
- **Import:** `IMPORT_NAME`, `IMPORT_FROM`, `IMPORT_STAR` (Chapter 9).
- **Coroutines/generators:** `YIELD_VALUE`, `GET_AWAITABLE`, `SEND`, `RETURN_GENERATOR`, `ASYNC_GEN_WRAP`, `GET_AITER`, `GET_ANEXT` (Chapter 7).

- **Matching (3.10+ `match` statement):** `MATCH_CLASS`, `MATCH_MAPPING`, `MATCH_SEQUENCE`, `MATCH_KEYS`.

`BINARY_OP` deserves emphasis: before 3.11 each arithmetic operator was a distinct opcode (`BINARY_ADD`=23, `BINARY_SUBTRACT`=24, ...). Since 3.11 they are unified into one opcode whose `oparg` encodes the operator (`0`='+', `3`='-', `5`='*', `13`='+=' , etc. per `dis.cmp_op` / `dis.nb_ops`). This single change halved the opcode space for arithmetic and gave specialization a single site to optimize.

4. Jump targets and the exception table (3.11+)

4.1 How jumps work

Every jump opcode carries a delta — "jump forward/backward by N bytes from here." The compiler resolves label offsets at assemble time (`Python/compile.c:assemble_emit` / `assemble_jump_offsets`). `dis` marks the landing site with `>>`:

```
def jumps(n):
    if n > 0:
        return 1
    elif n == 0:
        return 0
    else:
        return -1

for ins in dis.get_instructions(jumps):
    print(f"{ins.offset:3d} {ins.opname:30s} {ins.argrepr:20s} "
          f"{'>>' if ins.is_jump_target else ' '} starts_line={ins.starts_line}")
```

0	RESUME		starts_line=1
2	LOAD_FAST	n	starts_line=2
4	LOAD_CONST	0	
6	COMPARE_OP	>	>> starts_line=None
12	POP_JUMP_FORWARD_IF_FALSE	to 18	
14	LOAD_CONST	1	
16	RETURN_VALUE		
18	LOAD_FAST	n	>> starts_line=3
...			

`is_jump_target` is computed by scanning all jump opargs — the same scan CPython does at frame setup to build the `co_code_adaptive` quickening table.

4.2 The old block stack (≤3.10)

Before 3.11 CPython tracked `try/except/finally/with/for` blocks with a **block stack** (`PyTryBlock` in `Include/cpython/frameobject.h`) — a value stack of `(opcode, handler_offset, stack_level)` pushed by `SETUP_FINALLY`, `SETUP_WITH`, `SETUP_FINALLY`, `SETUP_ASYNC_WITH`, and popped by `POP_BLOCK`. Every `try` statement emitted explicit `SETUP_*` / `POP_BLOCK` opcodes that bracketed the protected region at runtime.

The block stack cost a branch on every opcode (to check for pending unwind) and complicated the compiler. It also made stack-depth analysis fragile — the compiler had to account for the implicit block-stack depth when computing `co_stacksize`.

4.3 The exception table (3.11+)

3.11 replaced the block stack with an **exception table** (`co_exceptiontable`, PEP 654): a compact byte stream attached to the code object that maps bytecode ranges to handlers, entirely outside `co_code`. No `SETUP_*` opcodes are emitted; instead the compiler encodes entries of the form:

```
start_offset, end_offset → target_offset, stack_depth, push_lasti
```

When an exception is raised, `ceval` does not unwind a block stack — it binary-searches the exception table for an entry whose `[start, end)` contains the faulting offset, pushes the exception state, and jumps to `target`. `PUSH_EXC_INFO` / `POP_EXCEPT` / `RERAISE` / `CHECK_EXC_MATCH` handle the handler body, but the *discovery* of the handler is table-driven.

```
def with_try(x):
    try:
        return int(x)
    except ValueError:
        return 0
    except Exception as e:
        return -1

dis.dis(with_try)
```

```

3          0 RESUME                                0

4          2 NOP

5          4 LOAD_GLOBAL                          1 (NULL + int)
          16 LOAD_FAST                            0 (x)
          18 PRECALL                              1
          22 CALL                                1
          32 RETURN_VALUE
>>       34 PUSH_EXC_INFO

6          36 LOAD_GLOBAL                          2 (ValueError)
          48 CHECK_EXC_MATCH
          50 POP_JUMP_FORWARD_IF_FALSE           4 (to 60)
          52 POP_TOP

7          54 POP_EXCEPT
          56 LOAD_CONST                          1 (0)
          58 RETURN_VALUE

8    >>     60 LOAD_GLOBAL                          4 (Exception)
          72 CHECK_EXC_MATCH
          74 POP_JUMP_FORWARD_IF_FALSE          11 (to 98)
          76 STORE_FAST                          1 (e)

```

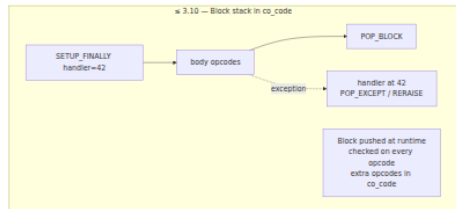
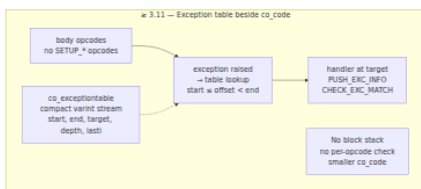
...

ExceptionTable:

```

4 to 30 -> 34 [0]
34 to 52 -> 100 [1] lasti
60 to 76 -> 100 [1] lasti
90 to 98 -> 100 [1] lasti

```



Inspecting the table directly — it is a varint-encoded byte stream, not human-readable without `dis`:

```

co = with_try.__code__
print(co.co_exceptiontable.hex(' '))          # raw varint bytes - compact
# Use dis to decode
import io
buf = io.StringIO()
dis.dis(with_try, file=buf)
print(buf.getvalue().split("ExceptionTable:")[1])
# Or walk entries via the internal API (3.12+ exposes co_exceptiontable as bytes;
# use Tools/scripts/parse_exception_table.py for a decoder on 3.11)

```

The table is also why `co_code` shrank in 3.11 despite adding inline caches — removing `SETUP_*/POP_BLOCK` opcodes saved more bytes than caches added.

Exception-group handling (`except*`, PEP 654) uses `CHECK_EG_MATCH` and `PREP_RERAISE_STAR` — same table mechanism, different matching opcode.

5. Frames and the value stack

5.1 The value stack — where opcodes live

`ceval` is a **stack machine**. There are no registers; every opcode pops its inputs from and pushes its result to a single value stack owned by the current frame. `co_stacksize` — computed at compile time — is the high-water mark of that stack.

```

Frame for demo(x, y, flag=True):

  localsplus: [ x | y | flag | z | i ]  [ ] co_varnames + cells (fixed size)
  value stack (grows up, max co_stacksize=3):
    stack_pointer [ ... ]  [ ] top
                  [ ... ]
                  [ ... ]  [ ] bottom (stack_pointer - stacksize)

```

`LOAD_FAST 0` pushes `localsplus[0]` onto the stack. `BINARY_OP 0 (+)` pops two, computes `PyNumber_Add`, pushes the result. `STORE_FAST 3` pops the top into `localsplus[3]`. The compiler guarantees the stack depth at every offset — a static analysis, not a runtime check in the fast path.

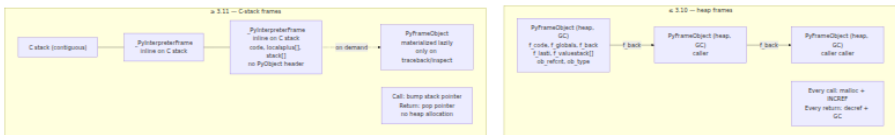
5.2 PyFrameObject → _PyInterpreterFrame (3.11)

Before 3.11 every Python call allocated a `PyFrameObject` on the heap (`Objects/frameobject.c`, `Include/frameobject.h`): a `PyObject` with `f_code`, `f_globals`, `f_locals`, `f_back`, `f_lasti`, `f_valustack`, and GC tracking. Calls were expensive — `malloc` + `Py_INCREF` + GC bookkeeping — and `f_back` linked frames into a heap chain for tracebacks.

3.11 (PEP 654 + PEP 659 + the "faster CPython" frame-stack work) replaced this with a **C-stack frame**:

- `_PyInterpreterFrame` — a lightweight C struct allocated inline on the C stack (not the heap), embedded in `_PyCFrame`. No `PyObject` header, no GC, no `f_back` heap chain until a traceback is actually needed.
- `_PyCFrame` — links interpreter frames for the same thread; `tstate->cframe` points to the current one.
- `PyFrameObject` still exists but is now a **lazily materialized view** — created on demand when Python code accesses `inspect.currentframe()`, `traceback` formatting, or `sys._getframe()`. Most frames never materialize.

Effect: call overhead dropped ~30%, traceback cost stayed ~identical (frames are materialized when needed), and the interpreter can keep the hot frame pointer in a register.



What this means for `co_code` inspection: `dis` still reports offsets as if frames were heap objects, but `frame.f_code.co_code` is now accessed via `frame->f_code` on the C-stack frame. The `frame` object you see from `inspect.currentframe()` is the materialized view — its `f_lasti` and `f_valustack` are copied from the underlying `_PyInterpreterFrame` on materialization.

Checking frame allocation in practice:

```

import inspect, sys, dis

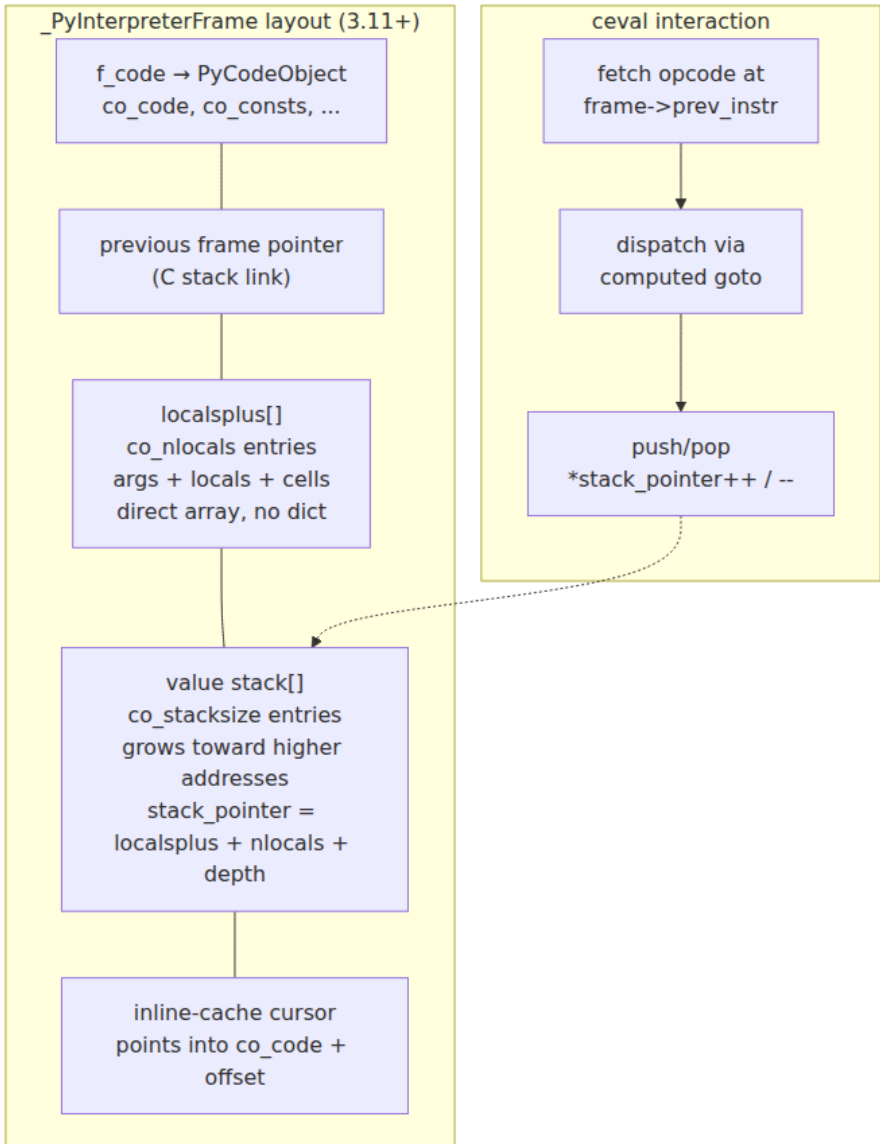
def inner():
    f = inspect.currentframe()
    print(f"f_code={f.f_code.co_name} f_lasti={f.f_lasti}")
    print(f"f_back={f.f_back.f_code.co_name if f.f_back else None}")
    # f is a PyFrameObject materialized from _PyInterpreterFrame
    dis.dis(inner)

inner()

# Compare with sys._getframe - same materialization path
print(sys._getframe(0).f_code.co_name)

```

For backend engineers the practical takeaway is sizing: frame allocation is no longer a GC/allocator bottleneck, so deep call stacks (middleware chains, decorator stacks) are cheaper, and `perf` will show less time in `PyFrame_New` / `_PyFrame_MakeAndSetFrameObject`.



6. The `ceval` loop — fetch, dispatch, execute

6.1 The big loop

`Python/ceval.c:_PyEval_EvalFrameDefault` (aliased as `PyEval_EvalFrameEx` in older headers) is the interpreter's heartbeat. Pseudocode that matches the real structure:


```

PyObject *
_PyEval_EvalFrameDefault(PyThreadState *tstate, _PyInterpreterFrame *frame, int throwflag)
{
    // Hot registers – kept in C locals for speed
    PyObject **stack_pointer = frame->stack_pointer;
    _Py_CODEUNIT *next_instr = frame->prev_instr + 1; // _Py_CODEUNIT
= {opcode, oparg}
    _Py_CODEUNIT *first_instr = _PyCode_CODE(frame->f_code);
    PyObject **localsplus = frame->localsplus;

    // Computed-goto dispatch table (generated at build time)
    static void *opcode_targets[256] = { &TARGET_RESUME, &TARGET_LOAD
_FAST, ... };

    // ---- main dispatch loop ----
    for (;;) {
        // Eval breaker: check GIL drop request, signals, GC, pending
calls
        if (_Py_atomic_load_relaxed(&tstate->eval_breaker) & _PY_EVAL_E
VENTS_MASK) {
            if (handle_eval_breaker(tstate)) goto error;
        }

        _Py_CODEUNIT word = *next_instr++;
        opcode = _Py_OPCODE(word);
        oparg = _Py_OPARG(word);

        // Dispatch – computed goto (see §6.2)
        goto *opcode_targets[opcode];

        TARGET_RESUME: /* entry / generator resume bookkeeping */ DISPATCH();
        TARGET_LOAD_FAST: { PyObject *v = localsplus[oparg];
Py_INCREF(v); *stack_pointer++ = v; DISPATCH(); }
        TARGET_BINARY_OP: { PyObject *right = *--stack_pointer; PyObject
t *left = *--stack_pointer;
                                PyObject *res = PyNumber_Add(left,
right); /* + type slots, cache, specialization */
                                Py_DECREF(left); Py_DECREF(right); *stack_p
ointer++ = res; DISPATCH(); }
        TARGET_RETURN_VALUE: { PyObject *retval = *--stack_pointer; fra
me->stack_pointer = stack_pointer;
                                return retval; }

        // ... ~150 more TARGET_* labels ...

        // Error / unwind path
        error:
            // look up co_exceptiontable for handler, or unwind to
caller

```

```

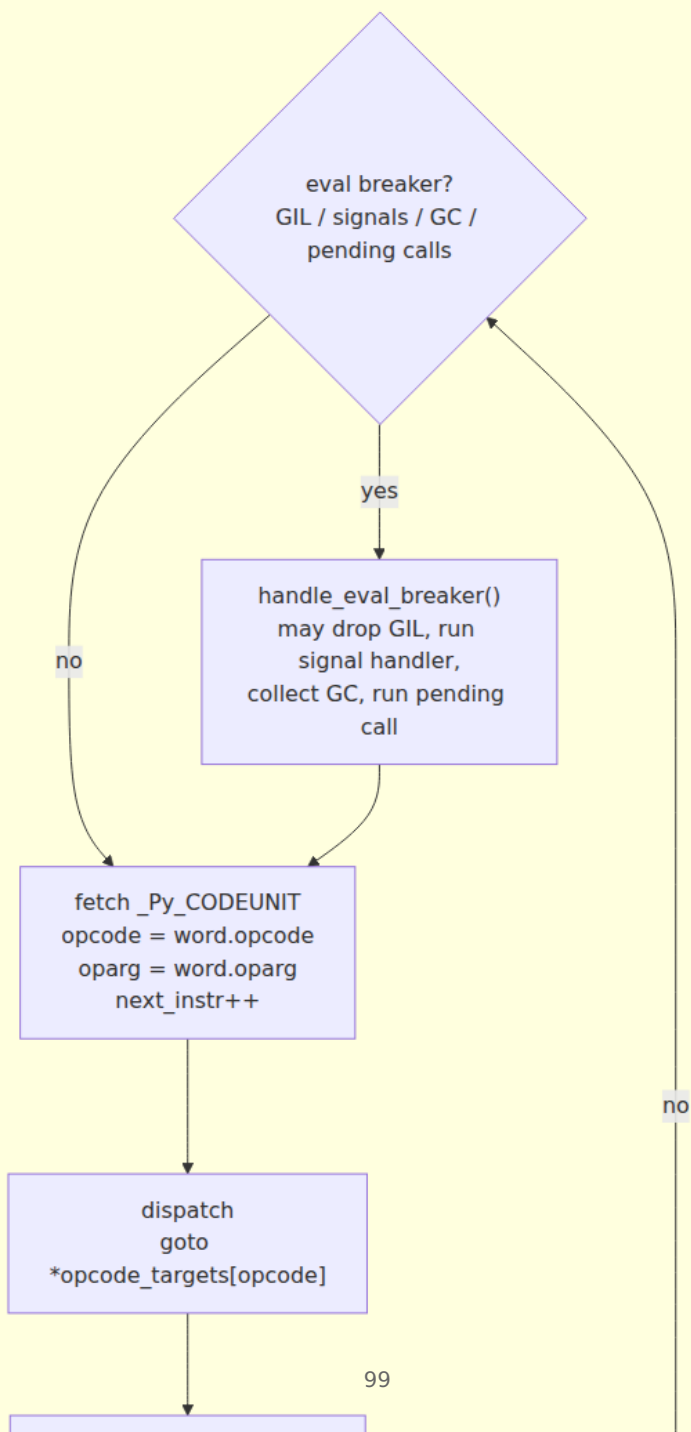
        next_instr = find_exception_handler(frame, first_instr, next_instr, ...);
        if (next_instr == NULL) { /* no handler – propagate */ return NULL; }
        goto resume_with_error;
    }
}

```

Three details that matter for profiling:

1. **No function call per opcode.** The loop is one giant function with `goto` labels — the compiler can keep `stack_pointer`, `next_instr`, and `localsplus` in registers across opcodes.
2. **Eval breaker is checked once per opcode** (or once per `JUMP_BACKWARD_NO_INTERRUPT` batch). `PyErr_CheckSignals`, `PyGC_CollectIfThreshold`, `handleGILDropRequest`, and `pending_calls` all funnel through `tstate->eval_breaker` — a single atomic load.
3. **Reference counting is inline.** Every `LOAD_FAST` does a `Py_INCREF`; every `BINARY_OP` does two `Py_DECREF`s. These are simple integer increments on `ob_refcnt` — not atomic in the GIL build — so they are cheap but not free.

ceval pipeline — one opcode



6.2 Dispatch — computed `goto` vs. `switch`

Old / portable: switch dispatch

```
for(;;) {  
    opcode =  
    *next_instr++;  
    switch(opcode) {  
    case LOAD_FAST: ...  
        break;  
    case BINARY_OP: ...  
        break;  
    ...  
    }  
}
```

One indirect branch
through jump table
compiler emits bounds
check
harder to predict

switch is ~10-15%
slower

Current: computed goto (gcc/clang)

```
static void  
*targets[256] = {  
    &&TARGET_LOAD_FAST,  
    &&TARGET_BINARY_OP,  
    ...};  
for(;;) {  
    opcode =  
    *next_instr++;  
    goto *targets[opcode];  
TARGET_LOAD_FAST: ...  
    DISPATCH();  
TARGET_BINARY_OP: ...  
    DISPATCH();  
}
```

One indirect branch
directly to label
no switch bounds check
better branch prediction
DISPATCH() is goto
*targets[next_opcode]

CPython builds with computed `goto` when the compiler supports `__GNUC__` labels-as-values (GCC, Clang — i.e., every production build on Linux/macOS). The fallback `switch` is only used on MSVC or when `USE_COMPUTED_GOTOS` is disabled.

Why it matters: the dispatch branch is the single hottest branch in the process. `perf record` on a Python workload will show the top hotspot as an indirect branch in `_PyEval_EvalFrameDefault`. Computed `goto` saves one bounds check and lets the CPU's branch predictor learn per-opcode targets (each opcode's `DISPATCH()` is a distinct indirect `jmp`), rather than predicting a single `switch` dispatch site. Measured gain is ~10–15% on `ceval` alone — before any specialization.

The macros that implement it:

```
// Python/ceval.c (simplified)
#ifdef USE_COMPUTED_GOTOS
# define TARGET(op) TARGET_##op:
# define DISPATCH() goto *opcode_targets[_Py_OPCODE(*next_instr)]
# define NEXTOP() (next_instr++, DISPATCH())
#else
# define TARGET(op) case op:
# define DISPATCH() continue
#endif
```

Tier-2 and the JIT (3.13+) introduce a second dispatch layer — a micro-op (`_PyUOp`) interpreter — but tier-1 still uses computed `goto`.

7. Adaptive specialization — PEP 659

7.1 The problem PEP 659 solves

A generic opcode must handle every type combination:

```

// Generic BINARY_OP – must handle int+int, str+str, list+list,
float+float,
// custom __add__, NotImplemented, coercion, overflow, ...
TARGET_BINARY_OP:
    PyObject *right = POP();
    PyObject *left = POP();
    PyObject *res = PyNumber_Add(left, right); // full dynamic
dispatch
    // PyNumber_Add does: type(left)->tp_as_number->nb_add OR
    // type(left)->tp_as_sequence->sq_concat OR type slots OR __add__
lookup
    PUSH(res);
    DISPATCH();

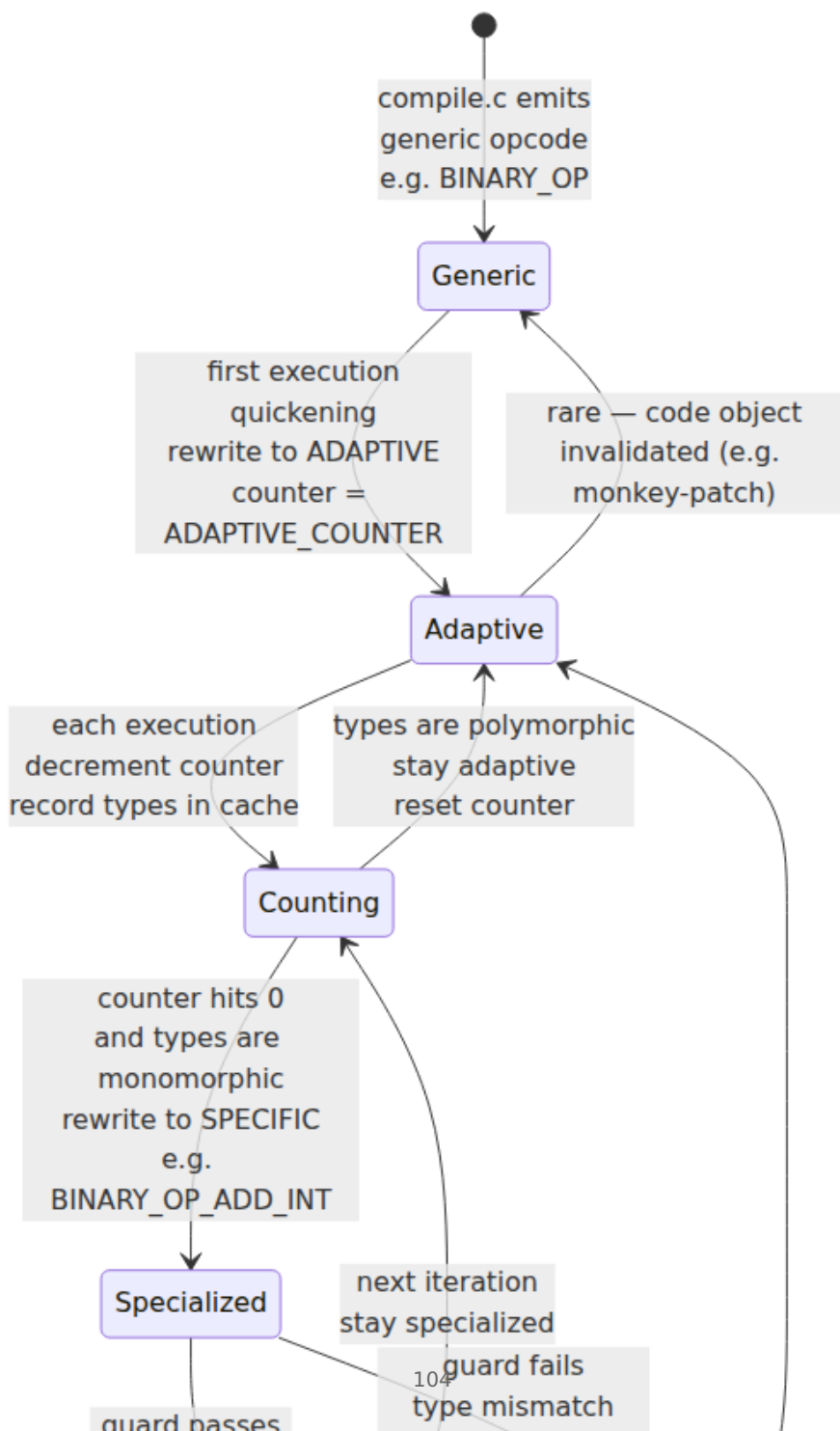
```

`PyNumber_Add` alone walks multiple type slots, does `PyType_Lookup` for `__add__`, handles `NotImplemented`, and may allocate. For the common case — `int + int` in a loop — 90% of that work is wasted.

PEP 659 (Faster CPython, Mark Shannon, 2021) observes that **most call sites are monomorphic**: a given `BINARY_OP` at a given offset almost always sees the same pair of types. If the interpreter could remember "this `BINARY_OP` is always `int + int`" and guard that assumption, it could use a fast path (`long_add` directly, no slot lookup) and fall back to the generic path only on guard failure.

The mechanism has three stages: **quickening** → **adaptive counting** → **specialization**, with **deoptimization** on failure.

7.2 Lifecycle — generic → adaptive → specialized → deopt



Concrete walk-through with `BINARY_OP` :

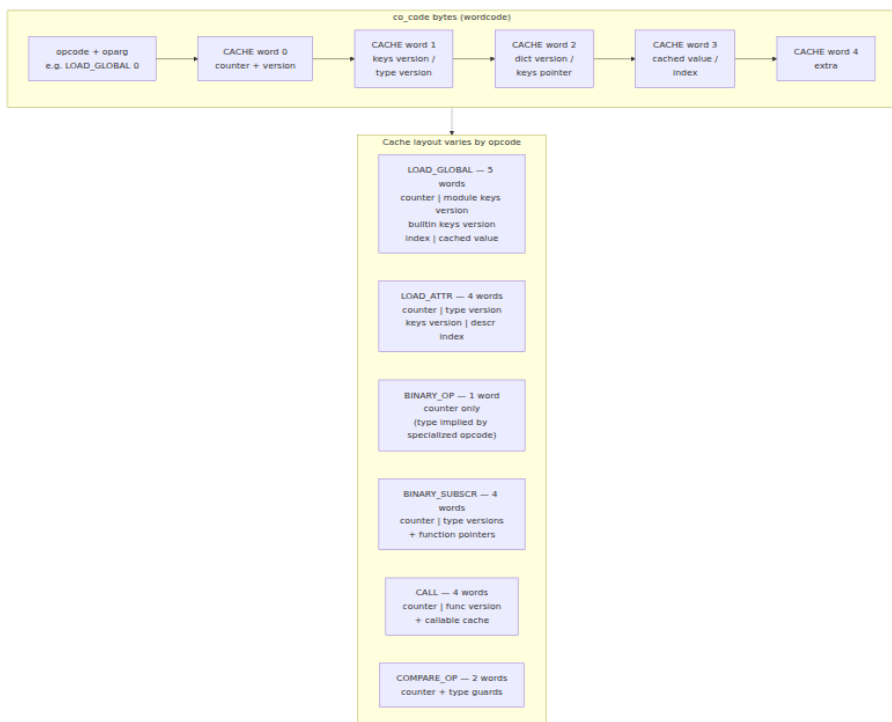
1. `compile.c` emits `BINARY_OP 0 (+)` with 1 trailing `CACHE` word (zero-initialized).
2. First execution hits `BINARY_OP` → `ceval` quickens it to `BINARY_OP_ADAPTIVE` (in 3.11 the adaptive opcode shares the numeric value but sets a flag; `dis(..., adaptive=True)` shows the specialized name after specialization) and initializes an 8-bit counter in the cache (starting at, e.g., 53 on 3.11, tuned per opcode).
3. Each subsequent execution at that offset decrements the counter and records the observed types (`int`, `int`) in the cache.
4. When the counter reaches 0, `ceval` checks if all observations were `int + int` → specializes to `BINARY_OP_ADD_INT` (or `BINARY_OP_ADD_FLOAT`, `BINARY_OP_ADD_UNICODE`, `BINARY_OP_MULTIPLY_INT`, etc.).
5. The specialized opcode's handler is a tight guard + fast path: `check PyLong_CheckExact(left) && PyLong_CheckExact(right) → long_add(left, right) → push`; on failure, deoptimize back to adaptive and fall through to generic.

Deoptimization is cheap — it rewrites the opcode back to adaptive in place and resumes counting. No recompilation, no code-object replacement.

The counter values are per-opcode and tuned so that specialization happens after ~8-50 executions — enough to confirm monomorphism without delaying the win on a hot loop, but not so eagerly that a polymorphic call site specializes prematurely.

7.3 Inline caches — what lives in the `CACHE` words

Each specializing opcode reserves a fixed number of `CACHE` words immediately after itself in `co_code`. The specialized handler treats those words as typed storage — not as opcodes to execute (the dispatch loop skips them by advancing `next_instr` past the cache).



Example — `LOAD_GLOBAL` (5 words):

- Word 0: adaptive counter (8 bits) + unused.
- Word 1: version tag of `globals` dict's keys (`ma_version_tag`).
- Word 2: version tag of `builtins` dict's keys.
- Word 3: index of the name in the dict's entries array (so the specialized path can do `entries[index].value` without hashing).
- Word 4: unused / spare.

On specialization `LOAD_GLOBAL` becomes `LOAD_GLOBAL_MODULE` (name found in `globals`) or `LOAD_GLOBAL_BUILTIN` (found in `builtins`). The guard checks `dict_version == cached_version`; on success it loads `value` at the cached index — a single array access, no hash, no `PyDict_GetItem`.

For `LOAD_ATTR` (4 words) the cache stores the type's `tp_version_tag` and the `dict` keys version, enabling `LOAD_ATTR_MODULE`, `LOAD_ATTR_INSTANCE_VALUE`, `LOAD_ATTR_SLOT`, etc.

Inspecting it:

```

import opcode, dis

# How many cache words per specializing opcode
for name in ["LOAD_GLOBAL", "LOAD_ATTR", "BINARY_OP", "BINARY_SUBSCR",
             "STORE_ATTR", "COMPARE_OP", "CALL", "PRECALL"]:
    num = opcode.opmap[name]
    print(f"{name:15s} opcode={num:3d} cache_words={opcode._inline_cache_entries[num]}")

# Where caches live in co_code - dis with show_caches
def demo_cache(x):
    return x.foo + 42

dis.dis(demo_cache, show_caches=True)
for ins in dis.get_instructions(demo_cache):
    print(ins)

```

7.4 Specialization per opcode family

BINARY_OP — arithmetic

Generic `BINARY_OP` handles 13 operators via `oparg`. Specialized variants observed after warmup (`dis(..., adaptive=True)`):

```

BINARY_OP_ADD_INT          ☐ both PyLong (exact), fast long_add
BINARY_OP_ADD_FLOAT        ☐ both PyFloat, direct double add
BINARY_OP_ADD_UNICODE      ☐ both PyUnicode, direct concat (with
freelist)
BINARY_OP_MULTIPLY_INT     ☐ int ☒ int
BINARY_OP_MULTIPLY_FLOAT
BINARY_OP_SUBTRACT_INT / SUBTRACT_FLOAT
BINARY_OP_ADD_INT          (inplace variants share the same specialization)

```

The guard is always `Py_TYPE(left) == &PyLong_Type && Py_TYPE(right) == &PyLong_Type` (or `PyFloat_Type`, `PyUnicode_Type`). On guard failure → deoptimize.

Before/after:

```

def hot_ops(x, y):
    return x + y

# Before specialization (cold)
dis.dis(hot_ops)
# 0 RESUME
# 2 LOAD_FAST x
# 4 LOAD_FAST y
# 6 BINARY_OP +
# 10 RETURN_VALUE

# Warm up with monomorphic int pairs
for _ in range(100_000):
    hot_ops(1, 2)

# After specialization (hot)
dis.dis(hot_ops, adaptive=True)
# 0 RESUME_QUICK
# 2 LOAD_FAST_LOAD_FAST
# 6 BINARY_OP_ADD_INT      ← specialized
# 10 RETURN_VALUE

```

Note `LOAD_FAST_LOAD_FAST` — the adaptive layer also fuses adjacent `LOAD_FAST` pairs into a superinstruction to save one dispatch.

`LOAD_GLOBAL` — global and builtin lookup

```

import math

def hot_global(n):
    return math.sqrt(n)

for _ in range(20_000):
    hot_global(4.0)

dis.dis(hot_global, adaptive=True)

```

```

0 RESUME_QUICK
2 LOAD_GLOBAL_MODULE      1 (NULL + math)    [ ] specialized:
found in globals
14 LOAD_ATTR_MODULE       1 (sqrt)          [ ] LOAD_ATTR also spe
cialized
24 LOAD_FAST              0 (n)
26 PRECALL_NO_KW_BUILTIN_0 1
30 CALL_ADAPTIVE          1
40 RETURN_VALUE

```

Two specializations: `LOAD_GLOBAL_MODULE` (name in module globals) vs. `LOAD_GLOBAL_BUILTIN` (name in builtins). The latter is rarer in backends (most names are module globals). If either dict is mutated (`globals()["math"] = ...` or monkey-patching `builtins`), the version tag mismatches and the guard deoptimizes.

For API handlers this is the single most impactful specialization — every `json`, `logging`, `os`, `request`, `db`, `cache` global goes through `LOAD_GLOBAL`.

CALL / PRECALL — call-site specialization

`CALL` has the richest specialization because callables vary widely:

```
CALL_PY_EXACT_ARGS      - Python function, exact arg count, no
defaults/kw
CALL_PY_WITH_DEFAULTS   - Python function with defaults
CALL_NO_KW_BUILTIN_0    - C builtin with METH_0 (one arg, e.g. len,
int)
CALL_NO_KW_BUILTIN_FAST - C builtin with METH_FASTCALL
CALL_NO_KW_METHOD_DESCRIPTOR - method descriptor (e.g. list.append,
str.split)
CALL_BOUND_METHOD_EXACT_ARGS - bound method with exact args
CALL_ADAPTIVE            - still counting, not yet specialized
```

```
def callee(a, b): return a + b
def caller(x):    return callee(x, 1)

for _ in range(20_000):
    caller(10)

dis.dis(caller, adaptive=True)
```

```
0 RESUME_QUICK
2 LOAD_GLOBAL_MODULE      1 (NULL + callee)
14 LOAD_FAST__LOAD_CONST 0 (x)
18 PRECALL_PYFUNC         2
22 CALL_PY_EXACT_ARGS     2  Python function, exact args
32 RETURN_VALUE
```

`PRECALL` also specializes (`PRECALL_PYFUNC`, `PRECALL_NO_KW_BUILTIN_0`, etc.) — the pair must agree. `PRECALL`'s cache holds the adaptive counter; `CALL`'s 4-word cache holds the callable's version and argument metadata.

For backends, `CALL` specialization is why tight dispatch loops (`for item in batch: handler(item)`) get faster — the call site learns the handler's type.

`COMPARE_OP` — comparisons

```
def cmp_hot(a, b):
    return a > b

for _ in range(20_000):
    cmp_hot(1, 2)

dis.dis(cmp_hot, adaptive=True)
```

```
0 RESUME_QUICK
2 LOAD_FAST_LOAD_FAST
6 COMPARE_OP          4 (>)      ← not yet specialized in this run
```

`COMPARE_OP` with `>` on ints may stay generic longer because the specialization threshold differs; on float-heavy workloads it becomes `COMPARE_OP_FLOAT_JUMP` / `COMPARE_OP_INT_JUMP` variants (visible in 3.12+ as `COMPARE_OP` remains adaptive but the uop layer handles it).

`LOAD_ATTR` / `STORE_ATTR` / `BINARY_SUBSCR`

<code>LOAD_ATTR_MODULE</code>	☐ type is module , attribute via dict
<code>LOAD_ATTR_INSTANCE_VALUE</code>	☐ instance dict lookup, keys version guard
<code>LOAD_ATTR_SLOT</code>	☐ slot wrapper (e.g. <code>__slots__</code>)
<code>LOAD_ATTR_PROPERTY</code>	☐ property descriptor (calls getter ☐ still guarded)
<code>BINARY_SUBSCR_LIST_INT</code>	☐ <code>list[int]</code>
<code>BINARY_SUBSCR_DICT</code>	☐ dict lookup
<code>BINARY_SUBSCR_TUPLE_INT</code>	☐ <code>tuple[int]</code>
<code>STORE_ATTR_INSTANCE_VALUE</code>	☐ instance dict store

These matter for ORM-style code (`row.id`, `user["name"]`, `obj.attr = value`) — the hottest attribute/subscript sites in API handlers.

7.5 Guards and deoptimization

Every specialized opcode is preceded by one or more guards — cheap integer comparisons that validate the assumptions the cache was built on:

Specialization	Guards
<code>BINARY_OP_ADD_INT</code>	<code>PyLong_CheckExact(left) && PyLong_CheckExact(right)</code>
<code>LOAD_GLOBAL_MODULE</code>	<code>globals_version == cached_version && index_valid</code>
<code>LOAD_ATTR_INSTANCE_VALUE</code>	<code>Py_TYPE(obj)->tp_version_tag == cached && dict_keys_version == cached</code>
<code>CALL_PY_EXACT_ARGS</code>	<code>Py_TYPE(callable) == &PyFunction_Type && callable->vectorcall == cached && argcount == cached</code>
<code>BINARY_SUBSCR_LIST_INT</code>	<code>PyList_CheckExact(container) && PyLong_CheckExact(sub)</code>

All guards are **single-branch, no allocation**. On failure the handler does:

```
// Pseudocode inside TARGET_BINARY_OP_ADD_INT
if (!PyLong_CheckExact(left) || !PyLong_CheckExact(right))
    goto deoptimize; // rewrite opcode to BINARY_OP_ADAPTIVE, reset
                    // counter

// fast path - directly call long add
result = _PyLong_Add((PyLongObject*)left, (PyLongObject*)right);
```

`deoptimize` rewrites the opcode byte in `co_code` back to the adaptive variant and resets the counter to a higher value (so a polymorphic site does not immediately re-specialize and thrash). The next executions resume counting with the new type observation.

Deoptimization is why specialization is safe under monkey-patching, `mock.patch`, or `reload` — correctness is never sacrificed. The worst case is performance oscillation on a truly polymorphic site (e.g., `x + y` where `x` alternates between `int` and `str`), where the call site bounces between `ADD_INT` → `deopt` → `ADD_UNICODE` → `deopt`. Such sites stay adaptive and never pay the specialized fast path — but they also never corrupt state.

You can observe deoptimization:

```

def poly(x, y):
    return x + y

# Monomorphic ints → specialize to ADD_INT
for _ in range(50_000):
    poly(1, 2)
dis.dis(poly, adaptive=True)  # BINARY_OP_ADD_INT

# Now pass strings – guard fails, deoptimizes
poly("a", "b")
dis.dis(poly, adaptive=True)  # back to BINARY_OP_ADAPTIVE or
BINARY_OP

# Alternate types rapidly – stays adaptive, never re-specializes to one
for _ in range(1000):
    poly(1, 2)
    poly("a", "b")
dis.dis(poly, adaptive=True)  # BINARY_OP_ADAPTIVE – polymorphic

```

7.6 PEP 659 — design and performance

PEP 659 ("Specializing Adaptive Interpreter", Mark Shannon, 2021) proposed the mechanism above as a low-risk, high-reward first step toward a JIT. Key design choices:

- **No new compiler pass.** Specialization is purely a runtime rewrite of `co_code` — no recompilation, no IR, no extra memory per specialization beyond the existing cache words.
- **Per-code-object, per-offset.** Each bytecode offset specializes independently. One function can have `BINARY_OP_ADD_INT` at offset 6 and `BINARY_OP_ADD_FLOAT` at offset 22.
- **Zero cost for cold code.** Functions called a few times never hit the counter threshold and never specialize — no overhead.
- **Conservative guards.** Only exact type checks (`Py_TYPE(x) == &PyLong_Type` , not `PyLong_Check`) to avoid subclass surprises.

Performance on CPython 3.11 (pyperformance geometric mean, per PEP 659 and the 3.11 release notes):

- **~25% faster** on the pyperformance suite vs. 3.10 (geometric mean). Individual benchmarks: `nbody` +35%, `deltablue` +30%, `fannkuch` +25%, `tornado_http` +15–20%.

- Micro-benchmarks on monomorphic integer loops: up to **60% faster** (`for i in range(n): s += i` specializes `BINARY_OP_ADD_INT` + `LOAD_GLOBAL` for `range`).
- Attribute-heavy workloads (`LOAD_ATTR` specialization) show **10-20%** gains — significant for ORM/API code.
- No measurable regression on polymorphic or cold code.

The 25% figure compounds: a fleet of 500 Python API pods each serving 1k RPS at p50 20ms saves ~5ms per request — or equivalently, 20% fewer pods for the same latency target.

7.7 Introspection — seeing specialization

```
import dis

def show_specialization(func, *warmup_calls):
    print("=== cold (generic) ===")
    dis.dis(func)
    print("\n=== cold - raw co_code ===")
    print(func.__code__.co_code.hex(' '))
    print(f"co_code len={len(func.__code__.co_code)}")

    for args in warmup_calls:
        for _ in range(30_000):
            func(*args)

    print("\n=== hot (adaptive=True) ===")
    dis.dis(func, adaptive=True)
    print("\n=== hot - with caches ===")
    dis.dis(func, adaptive=True, show_caches=True)

def my_add(a, b): return a + b
show_specialization(my_add, (1, 2))

# LOAD_GLOBAL specialization
import math
def use_math(n): return math.sqrt(n)
show_specialization(use_math, (4.0,))

# CALL specialization
def greet(name): return f"hi {name}"
def call_greet(x): return greet(x)
show_specialization(call_greet, ("world",))

# Inspect inline-cache reservation
import opcode
print("\n=== inline cache entries ===")
for op in sorted(opcode.opmap, key=lambda k: opcode.opmap[k]):
    n = opcode._inline_cache_entries[opcode.opmap[op]]
    if n:
        print(f" {op:25s} {n} word(s)")
```

`python -X showrefcount` (debug build) and `python -X showrefcount -X faulthandler` are occasionally mentioned alongside adaptive inspection, but they report reference-count leaks, not specialization state — use `dis(..., adaptive=True)` for specialization and `PYTHONPROFILEIMPORTTIME=1 / PYTHON_LLTRACE` for import/eval tracing.

The `opcode` module also exposes `opcode.opname`, `opcode.cmp_op`, and `dis._nb_ops` for `BINARY_OP` sub-opcodes:

```
import dis
print(dis.cmp_op) # ('<', '<=', '==', '!=', '>', '>=') - COMPARE_OP
oparg
# BINARY_OP oparg names via _nb_ops (3.11+)
try:
    print(dis._nb_ops) # ('+', '&', '|', '^', ... , '+=', '-=', ...)
except AttributeError:
    pass
```

8. The copy-and-patch JIT — PEP 744 teaser (3.13+ experimental)

Specialization squeezes most of the interpreter overhead, but each opcode still pays a dispatch branch and a guard. A JIT removes both by compiling the hot path to machine code.

CPython 3.13 ships an **experimental JIT** (PEP 744, Brandt Bucher, Ken Jin, Haoran Xu) based on **copy-and-patch** (also called *stencil JIT*):

1. **Tier 1** — the specializing adaptive interpreter (PEP 659) as described above. It warms up and specializes.
2. **Tier 2** — a micro-op (`_PyUOp`) interpreter. When tier-1 specialization stabilizes, the bytecode is translated to a lower-level trace of `uops` (`_LOAD_FAST`, `_BINARY_OP_ADD_INT`, `_STORE_FAST`, ...) that operate on an abstract stack. Uops are still interpreted but with fewer branches than bytecodes.
3. **Copy-and-patch** — each uop has a pre-compiled machine-code *stencil* (a snippet of assembly with holes for constants/addresses). The JIT copies the stencil and patches the holes, concatenating stencils into a contiguous machine-code trace with no dispatch loop. Guards become conditional jumps within the trace; on guard failure the trace exits back to tier 1 (deoptimization).

Python source

```
↳ bytecode (tier 1, adaptive)
  ↳ specialized bytecode (PEP 659)
    ↳ uop trace (tier 2, ~50 uops)
      ↳ machine-code trace (copy stencils + patch holes)
        ↳ execute natively, no dispatch loop
          ↳ guard failure ↳ exit to tier 1
```

Status as of 3.13/3.14:

- Disabled by default: `PYTHON_JIT=1` or `./configure --enable-experimental-jit` (build flag `Py_JIT`).
- Only x86-64 and AArch64 backends; LLVM is used at build time to compile stencils, not at runtime.
- Speedup on top of 3.11 is modest so far (**~5-10% on pyperformance**, up to ~20% on arithmetic micro-benchmarks) — the win is expected to grow as more opcodes get stencils and tier-2 coverage expands.
- No warmup tuning exposed yet; the threshold is internal (`_Py_JIT_THRESHOLD`).

For backend engineers the JIT is not yet an operational lever — you do not size pods based on it. But the architecture matters: the copy-and-patch design keeps the JIT's memory overhead low (stencils are shared) and its deoptimization path simple (exit to tier 1, same as specialization deopt). Familiarity with tier-1 specialization is prerequisite to understanding tier-2 traces when `PYTHON_JIT=1` profiles start appearing in `perf`.

Further reading for the JIT is PEP 744 and `InternalDocs/jit.md` in the CPython repository (`Python/jit.c`, `Tools/jit/`).

9. Backend lens — why specialization matters for API handlers

9.1 The hot loop in every API service

A typical FastAPI/Starlette handler executes the same bytecode shape on every request:

```

# app/handler.py - executed 10k times per second per pod
import json, logging
from models import User

log = logging.getLogger("api")

async def get_user(request):
    uid = int(request.path_params["uid"])          # BINARY_SUBSCR, CALL
                                                    # (int), LOAD_ATTR
    row = await db.fetchrow()                       # LOAD_GLOBAL (db),
                                                    # LOAD_ATTR (fetchrow), CALL
    "SELECT id, name FROM users WHERE id=$1", uid)
    if row is None:                                 # COMPARE_OP (is),
                                                    # POP_JUMP
        return JsonResponse({"error": "not found"}, status_code=404)
    user = User(id=row["id"], name=row["name"])     # BINARY_SUBSCR, CALL
                                                    # (User)
    log.info("get_user uid=%s", uid)                # LOAD_GLOBAL (log),
                                                    # LOAD_ATTR, CALL
    return user.json()                             # LOAD_ATTR, CALL

```

Every one of those opcodes specializes after a few hundred requests:

- `LOAD_GLOBAL` for `db`, `log`, `User`, `JsonResponse` → `LOAD_GLOBAL_MODULE` (single array access).
- `LOAD_ATTR` for `request.path_params`, `row.__getitem__`, `user.json` → `LOAD_ATTR_INSTANCE_VALUE` / `LOAD_ATTR_SLOT`.
- `BINARY_SUBSCR` for `request.path_params["uid"]` → `BINARY_SUBSCR_DICT`.
- `CALL` for `int()`, `db.fetchrow()`, `User()` → `CALL_PY_EXACT_ARGS` / `CALL_METHOD_DESCRIPTOR`.
- `COMPARE_OP` for `row is None` → specialized `COMPARE_OP` with type guard.

The 25% interpreter speedup is not on an artificial benchmark — it is on exactly this shape. A handler that spent 12ms in `ceval` on 3.10 spends ~9ms on 3.11. At p99 the win is larger because specialization also reduces branch-misprediction jitter.

9.2 Profiling bytecode hotspots

Two tools map CPU samples back to Python frames and, indirectly, to opcodes:

`py-spy` / `austin` (sampling, no instrumentation):

```
# Record a production process without restart – safe for prod (read-only, ptrace/perf)
py-spy record -o profile.svg --pid $(pgrep -f "uvicorn app.main") --duration 30
# or: austin -p <pid> -o profile.austin

# What to look for:
# - Wide _PyEval_EvalFrameDefault band → interpreter-bound (specialization helps)
# - Narrowing after warmup → specialization kicked in (compare --gil vs. --threads)
# - Wide PyNumber_Add / PyDict_GetItem under BINARY_OP / LOAD_GLOBAL → polymorphic site, not specializing
```

`py-spy --native` interleaves Python and native stacks — you can see `TARGET_BINARY_OP_ADD_INT` vs. `TARGET_BINARY_OP` (generic) as distinct native frames.

perf + CPython's perf trampoline (3.12+, perf map):

```
# Build CPython with --enable-perf-trampoline or use python:3.12+ image that enables it
# Then:
perf record -F 999 -g -p $(pgrep -f "uvicorn") -- sleep 30
perf report --no-children
# With trampoline, perf can resolve PyCodeObject names:
#   _PyEval_EvalFrameDefault
#     └─ get_user (app/handler.py:12)
#         └─ BINARY_SUBSCR_DICT ← specialized
```

For `perf` without the trampoline, use `PYTHONPERFSUPPORT=1` (3.12+) to emit `/tmp/perf-<pid>.map` so `perf report` can symbolize Python frames.

Mapping samples to specialization mix:

```

# In-process check: how specialized is a hot function?
import dis

def specialization_ratio(func):
    generic = specialized = adaptive = 0
    for ins in dis.get_instructions(func, adaptive=True):
        name = ins.opname
        if name.endswith("_ADAPTIVE") or name == "BINARY_OP" or name
== "LOAD_GLOBAL":
            # Heuristic: adaptive/generic names indicate not yet
specialized
            # Real check: compare adaptive=True vs adaptive=False
output
            adaptive += 1
            elif name.startswith(p) for p in
                ("BINARY_OP_", "LOAD_GLOBAL_", "LOAD_ATTR_", "CALL_"))
:
                specialized += 1
            else:
                generic += 1 # non-specializing opcodes (RESUME,
LOAD_FAST, RETURN_VALUE)
                total = specialized + adaptive
                if total:
                    print(f"{func.__name__}: specialized={specialized} adaptive={ad
aptive} "
                        f"ratio={specialized/total:.0%}")

specialization_ratio(get_user)

```

What to do with the data:

- If a hot handler shows many `BINARY_OP` / `LOAD_ATTR` still generic after warmup → the call site is polymorphic. Refactor to make it monomorphic (e.g., ensure `row` is always the same type — don't mix `dict` and `Row` objects at the same subscript site).
- If `LOAD_GLOBAL` is still generic → the globals dict is being mutated (monkey-patching, `unittest.mock.patch` left active, or `reload`). In production code, avoid mutating `globals()` on the hot path.
- If `CALL` is `CALL_ADAPTIVE` → the callable is polymorphic (e.g., `handler` sometimes a function, sometimes a bound method). Hoist the callable lookup out of the loop or use a single type.

9.3 Deployment implications

- **Warmup matters for autoscaling.** New pods need ~50-100 requests before hot opcodes specialize. During that window p50 is

~10–15% higher. If you autoscale aggressively (HPA on p99), account for warmup — either over-provision by one pod or send synthetic warmup traffic on startup (`/healthz` is not enough; warm the real handler shape).

- **Pre-fork servers share specialized bytecode via CoW.** With `gunicorn --preload` or `uvicorn --workers N` with preload, the parent process warms up specialization and forks — children inherit the specialized `co_code` (copy-on-write). Without preload each worker warms independently.
 - **`PYTHONDONTWRITEBYTECODE=1` does not affect specialization** — specialization rewrites `co_code` in memory, not on disk. `.pyc` files always store the generic bytecode; specialization happens at import/execution time.
-

10. Putting it together — end-to-end disassembly

A realistic function that exercises every mechanism in this chapter:


```

import dis, opcode

def process_batch(items, threshold=10):
    total = 0
    for x in items:
        if x > threshold:
            total += x
    return total

# --- static inspection ---
co = process_batch.__code__
print(f"co_varnames={co.co_varnames}")
print(f"co_names={co.co_names}")
print(f"co_consts={co.co_consts}")
print(f"co_stacksize={co.co_stacksize}")
print(f"co_code len={len(co.co_code)} {co.co_code.hex(' ')}")
print(f"exception table: {co.co_exceptiontable.hex(' ')} or '(empty)')")
print(f"positions: {list(co.co_positions())[8]}")

print("\n=== generic (cold) ===")
dis.dis(process_batch)

print("\n=== generic with caches ===")
dis.dis(process_batch, show_caches=True)

print("\n=== structured ===")
for ins in dis.get_instructions(process_batch):
    print(f"{ins.offset:3d} {ins.opname:30s} {str(ins.arg):6s} {ins.arg
repr:20s} "
        f"'>' if ins.is_jump_target else ' ' } line={ins.starts_line}")

# --- warm up ---
for _ in range(50_000):
    process_batch([1, 20, 3, 40, 5], 10)

print("\n=== hot (adaptive=True) ===")
dis.dis(process_batch, adaptive=True)

print("\n=== hot with caches ===")
dis.dis(process_batch, adaptive=True, show_caches=True)

# --- inline-cache inventory ---
print("\n=== inline caches ===")
for name in sorted(opcode.opmap, key=lambda k: opcode.opmap[k]):
    n = opcode._inline_cache_entries[opcode.opmap[name]]
    if n:
        print(f" {name:25s} {n}")

```

Cold output (abridged):

```

=== generic (cold) ===
 3          0 RESUME                      0
 4          2 LOAD_CONST                  1 (0)
          4 STORE_FAST                    2 (total)
 5          6 LOAD_GLOBAL                  1 (NULL + items) - wait,
items is a local
          ... actually LOAD_FAST for items
          6 LOAD_FAST                      0 (items)
          8 GET_ITER
    >>     10 FOR_ITER                      12 (to 36)
          12 STORE_FAST                    3 (x)
 6          14 LOAD_FAST                    3 (x)
          16 LOAD_FAST                      1 (threshold)
          18 COMPARE_OP                     4 (>)
          24 POP_JUMP_FORWARD_IF_FALSE     4 (to 34)
 7          26 LOAD_FAST                    2 (total)
          28 LOAD_FAST                      3 (x)
          30 BINARY_OP                      13 (+)
          34 STORE_FAST                    2 (total)
          36 JUMP_BACKWARD                 14 (to 10)
    >>     38 LOAD_FAST                      2 (total)
          40 RETURN_VALUE

```

Hot output (abridged — after 50k calls with `list[int]` items):

```

=== hot (adaptive=True) ===
 3          0 RESUME_QUICK                 0
 4          2 LOAD_CONST                  1 (0)
          4 STORE_FAST                    2 (total)
 5          6 LOAD_FAST                    0 (items)
          8 GET_ITER
    >>     10 FOR_ITER                      12 (to 36)
          12 STORE_FAST                    3 (x)
 6          14 LOAD_FAST_LOAD_FAST         3 (x) ← fused
          18 COMPARE_OP                     4 (>) ← may specialize on
int compare
          24 POP_JUMP_FORWARD_IF_FALSE
 7          26 LOAD_FAST                    2 (total)
          28 LOAD_FAST                      3 (x)
          30 BINARY_OP_ADD_INT             13 (+) ← specialized: int +=
int
          34 STORE_FAST                    2 (total)
          36 JUMP_BACKWARD                 14 (to 10)
    >>     38 LOAD_FAST                      2 (total)
          40 RETURN_VALUE

```

The only opcode that reliably specializes in this loop is `BINARY_OP` (`+=` on `int`). `COMPARE_OP` on `int > int` may also specialize depending on the

3.11 point release's counter tuning; `FOR_ITER` and `GET_ITER` do not specialize in 3.11 (they do in 3.12+ with additional caches). The win comes from the single hottest opcode in the loop body becoming a guarded `long_add` with no slot dispatch.

11. Key takeaways

- **Wordcode is 2 bytes per instruction (opcode + oparg) plus CACHE words.** `co_code` is a flat `bytes` object; `EXTENDED_ARG` prefixes extend the argument. Inline caches are trailing `CACHE` bytes, not real opcodes — hidden by default in `dis`, visible with `show_caches=True`.
- **PyCodeObject fields are the API.** `co_code`, `co_consts`, `co_names`, `co_varnames`, `co_stacksize`, `co_exceptiontable`, and `co_positions` together describe everything `ceval` needs. `dis.get_instructions()` and `opcode._inline_cache_entries` expose the same data at Python level.
- **Opcodes cluster into stack, name/object, control-flow, call, and container families.** `LOAD_FAST` / `STORE_FAST` are array accesses; `LOAD_GLOBAL` does dict lookup; `LOAD_ATTR` / `BINARY_SUBSCR` are the hottest attribute/subscript sites; `PRECALL` + `CALL` is the 3.11 calling convention; `BINARY_OP` unifies 13 operators via `oparg`.
- **The exception table replaced the block stack in 3.11.** No more `SETUP_FINALLY` / `POP_BLOCK` opcodes; handlers are found by range-lookup in `co_exceptiontable`. Smaller `co_code`, no per-opcode block-stack check.
- **Frames moved from heap to C stack in 3.11.** `_PyInterpreterFrame` is inline on the C stack; `PyFrameObject` is materialized lazily for `inspect` / `traceback`. Call overhead dropped ~30% and `perf` shows less time in `PyFrame_New`.
- **ceval dispatches via computed goto (goto *opcode_targets[opcode]).** One indirect branch per opcode, per-opcode predictor state, ~10-15% faster than `switch`. The eval breaker (`tstate->eval_breaker`) funnels GIL, signals, GC, and pending calls into a single atomic check per opcode.
- **PEP 659 quickens generic → adaptive → specialized, with guard + deopt.** Each specializing opcode reserves cache words; an 8-bit counter triggers specialization after ~8-50 monomorphic executions;

guards are exact type checks; failure rewrites back to adaptive. No recompilation, no extra memory beyond cache words.

- **Specialization covers** `BINARY_OP`, `LOAD_GLOBAL`, `LOAD_ATTR`, `STORE_ATTR`, `BINARY_SUBSCR`, `COMPARE_OP`, `CALL / PRECALL`, and more. `LOAD_GLOBAL_MODULE` (globals dict array access), `BINARY_OP_ADD_INT` (direct `long_add`), `CALL_PY_EXACT_ARGS` (direct `vectorcall`) are the highest-impact wins for API code.
 - **3.11 is ~25% faster (pyperformance geometric mean) from specialization alone**, up to 60% on integer micro-benchmarks, 10-20% on attribute-heavy handlers — with zero source changes. The win compounds across a fleet.
 - **Copy-and-patch JIT (PEP 744, 3.13+ experimental) builds on specialization.** Tier 1 (specialized bytecode) → tier 2 (uop trace) → stenciled machine code, with guard exits back to tier 1. Not yet production-default; understand tier 1 first.
 - **Profile with `py-spy / austin` and `perf + perf_trampoline`.** Look for `_PyEval_EvalFrameDefault` width, generic vs. specialized native frames (`TARGET_BINARY_OP` vs. `TARGET_BINARY_OP_ADD_INT`), and warmup curves. Keep hot call sites monomorphic; avoid mutating `globals()` or mixing types at the same bytecode offset.
-

12. Further reading

- **PEP 659 — Specializing Adaptive Interpreter** — Mark Shannon — the PEP that defines quickening, adaptive counters, specialization, guards, and deoptimization. Read the "Implementation" and "Performance" sections for counter tuning and opcode coverage. — *pinned* — <https://peps.python.org/pep-0659/>
- **PEP 744 — JIT Compilation (copy-and-patch)** — Brandt Bucher — tier-1 → tier-2 uops → stenciled machine code, build flag `--enable-experimental-jit`, and the `InternalDocs/jit.md` roadmap. — *pinned* — <https://peps.python.org/pep-0744/>
- **Python `dis` module documentation** — docs.python.org/3/library/dis.html — canonical reference for `dis.dis`, `dis.get_instructions`, `Instruction`, `show_caches / adaptive` flags, and the opcode tables. — *pinned* — <https://docs.python.org/3/library/dis.html>

- **CPython** `Python/ceval.c` **source** — `_PyEval_EvalFrameDefault`, `TARGET_*` labels, `opcode_targets` computed-goto table, eval breaker, and inline-cache handling. Start at the `DISPATCH / TARGET` macros and follow one opcode end-to-end. — *pinned* — <https://github.com/python/cpython/blob/main/Python/ceval.c>
- **PEP 654** — **Exception Groups and `except*` / Exception Table** — Irit Katriel et al. — the exception-table design that replaced the block stack, `co_exceptiontable` encoding, and `PUSH_EXC_INFO / CHECK_EXC_MATCH` semantics. <https://peps.python.org/pep-0654/>
- **CPython** **InternalDocs** — **Faster CPython / Adaptive Interpreter** — `InternalDocs/interpreter.md` and `InternalDocs/jit.md` in the CPython repo — internal design notes on frame-stack, quickening, and tier-2 uops not covered in the PEPs. <https://github.com/python/cpython/tree/main/InternalDocs>
- **CPython** `Objects/codeobject.c` and `Include/cpython/code.h` — `PyCodeObject` layout, `co_code / co_exceptiontable / co_positions` construction, and the `PyCode_Get*` accessors. <https://github.com/python/cpython/blob/main/Objects/codeobject.c>
- **Mark Shannon** — **"How the Faster CPython Interpreter Works" (PyCon 2022 talk)** — walkthrough of PEP 659 internals, cache layouts, and the 25% speedup with live `dis` demos. Search "Shannon Faster CPython PyCon 2022" for slides and recording.
- **Brandt Bucher** — **"Copy-and-Patch JIT" (PyCon 2024 talk)** — stencils, uops, and the 3.13 experimental JIT with `perf` profiles showing tier-1 → tier-2 → machine code transitions.

Chapter 4 — Memory Management: pymalloc, GC, Arenas, and Immortal Objects

What this chapter covers. CPython never calls `malloc` for every `PyObject`. Small objects are carved from 256 KiB arenas split into 4 KiB pools split into fixed-size blocks, tracked by per-size-class free lists in `Objects/obmalloc.c` (the `pymalloc` allocator). Large objects fall through to the system allocator. On top of that fast path, a generational cycle collector in `Modules/gcmodule.c` reclaims reference cycles that pure

reference counting cannot, and since Python 3.12 an immortal-object scheme (PEP 683) removes refcount traffic for long-lived singletons. This chapter opens all three layers — the arena/pool/block hierarchy, the GC's container tracking and DFS collection, and the immortal sentinel — with the exact C structs, thresholds, and Python introspection APIs that backend engineers use to diagnose RSS bloat, GC pauses, and leaks in production.

Learning goals — after this chapter you should be able to:

- Diagram pymalloc's arena (256 KiB) → pool (4 KiB) → block hierarchy, explain size classes (8-byte steps to 512 bytes), and describe arena/pool/block headers and free-list management.
- Distinguish the three allocation domains (`PYMEM_DOMAIN_RAW` , `PYMEM_DOMAIN_MEM` , `PYMEM_DOMAIN_OBJ`) and predict which path an allocation takes, including the `PYTHONMALLOC` override.
- Explain how pymalloc interacts with the OS (`mmap` / `VirtualAlloc` for arenas, `madvise(MADV_DONTNEED)` on free, the `usable_arenas` / `unused_arena_objects` lists) and how that affects RSS vs. heap.
- Classify types as containers vs. atomic for GC, describe `Py_TPFLAGS_HAVE_GC` , `tp_traverse` / `tp_clear` , and the `PyObject_GC_Track` write barrier.
- Trace the three-generation GC (thresholds 700/10/10), `gc.get_count` / `gc.collect` / `gc.freeze` , and the mark-and-sweep cycle detection (subtract-refs → find unreachable → handle finalizers/weakrefs → delete).
- Explain weakrefs and `__del__` /finalizer interaction with `gc.garbage` (PEP 442) and why `tp_del` cycles require special handling.
- Describe immortal objects (PEP 683, Python 3.12): `_Py_IMMORTAL_REFCNT` , `_Py_IsImmortal` , sharing across sub-interpreters, and why they matter for free-threaded Python (PEP 703).
- Use `tracemalloc` , `gc.get_objects` , `gc.get_referrers` / `get_referents` , `sys.debugmalloctstats` , and `_testcapi` to profile memory, and evaluate `PYTHONMALLOC=malloc` vs. pymalloc trade-offs.
- Apply backend tuning recipes: sizing RSS vs. heap, deliberately disabling GC during bulk loads, and building a leak-detection loop with `tracemalloc` + `gc.DEBUG*` .

Prerequisites. Chapter 2 dissected `PyObject`, `ob_refcnt`, and `ob_type`; Chapter 3 walked `PyCodeObject` and `ceval.c`. Volume 2, Chapter 4 (allocators, page cache, huge pages, OOM) and Volume 13, Chapter 4 (GC across runtimes) are useful background. You should be comfortable reading C headers.

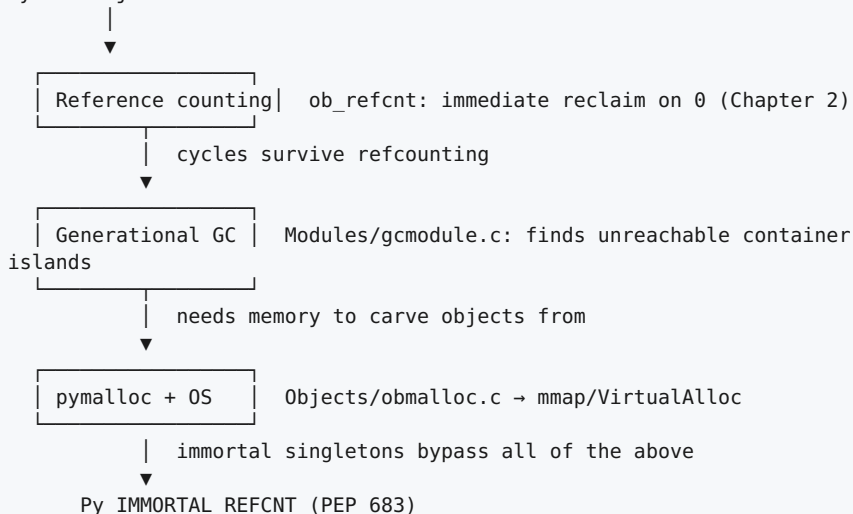
1. Why CPython needs its own allocator and collector

A typical backend Python process creates and destroys millions of small objects per second — `int` digits, `str` fragments, `tuple` slots, `dict` entries, frame locals. Calling the C library `malloc` / `free` for each one has three costs that `pymalloc` and the cycle GC exist to avoid:

Cost	What <code>malloc</code> does	What CPython does instead
Per-object header overhead	<code>malloc</code> adds 8-16 bytes of bookkeeping per block plus alignment padding	<code>pymalloc</code> packs same-size blocks contiguously in pools; headers are per-pool, not per-block
Fragmentation	Repeated <code>malloc(24)</code> / <code>free</code> scatters same-size blocks across the heap; RSS grows even after Python frees objects	Size-segregated pools keep 8-byte, 16-byte, ... 512-byte blocks in distinct free lists; freed blocks are reused in-place
Cycles	Reference counting alone cannot free <code>a.b = b; b.a = a</code> — both <code>ob_refcnt > 0</code> forever	Generational GC periodically walks <code>tp_traverse</code> graphs and breaks unreachable cycles

The three subsystems you will meet form a stack, not alternatives:

Python object creation



Backend lens — RSS is not heap. Your container's `memory.usage_in_bytes` (cgroup RSS) reflects OS pages mapped for arenas, not just live Python objects. An arena that holds one live 32-byte block still keeps 256 KiB of address space mapped. Understanding arena reclamation (`madvise`) is how you explain the gap between `tracemalloc` totals and `ps aux` RSS in production postmortems.

2. The allocator stack — domains, `PYTHONMALLOC`, and where each request goes

2.1 Three domains (PEP 445)

Since Python 3.4 (PEP 445) every allocation is tagged with a domain:

Domain	C API family	pymalloc?	Typical callers
PYMEM_DOMAIN_RAW	PyMem_RawMalloc / PyMem_RawFree	Never — always system malloc	Buffers that must survive without the GIL, PyMem_RawMalloc in pylifecycle.c, io buffers
PYMEM_DOMAIN_MEM	PyMem_Malloc / PyMem_Free	Yes for ≤ 512 bytes	PyBytesObject, PyUnicodeObject internals, dict tables via PyMem_Malloc
PYMEM_DOMAIN_OBJ	PyObject_Malloc / PyObject_Free	Yes for ≤ 512 bytes	Every PyObject_New / PyObject_GC_New — the object allocator

The raw domain exists so that code holding no Python objects (e.g., codecs reading a file while the GIL is released) can allocate without touching pymalloc's GIL-protected free lists.

```

/* Objects/obmalloc.c – simplified dispatch (Python 3.11/3.12) */

#define SMALL_REQUEST_THRESHOLD 512
#define NB_SMALL_SIZE_CLASSES 64 /* 8, 16, 24, ..., 512 */

void *PyObject_Malloc(size_t n) {
    if (n == 0) n = 1;
    if (n <= SMALL_REQUEST_THRESHOLD)
        return PyObject_Malloc(n); /* pymalloc fast path */
    return malloc(n); /* large → system allocator */
}

void *PyMem_Malloc(size_t n) {
    if (n <= SMALL_REQUEST_THRESHOLD)
        return PyObject_Malloc(n); /* same pymalloc pools */
    return malloc(n);
}

void *PyMem_RawMalloc(size_t n) {
    return malloc(n); /* always system allocator */
}

```

Size zero is rounded to one byte — `malloc(0)` has platform-defined behavior, and CPython normalizes it.

2.2 PYTHONMALLOC — the kill switch and the debug modes

The environment variable `PYTHONMALLOC` selects the underlying allocator at startup (`Python/pymem.c`):

Value	Effect
(default)	<code>pymalloc</code> for <code>MEM/OBJ</code> ≤ 512 , system <code>malloc</code> otherwise
<code>malloc</code>	Disable <code>pymalloc</code> entirely — every domain uses <code>malloc</code> / <code>free</code> directly
<code>malloc_debug</code>	<code>malloc</code> + debug wrappers (fence bytes, double-free detection)
<code>pymalloc_debug</code>	<code>pymalloc</code> + debug wrappers (<code>PYMEM_DOMAIN_*</code> with <code>PYMALLOC_DEBUG</code>)

```
# Compare – useful for benchmarking allocator overhead
PYTHONMALLOC=malloc python -c "import sys;
print(sys._debugmallocstats)" 2>&1 | head -20

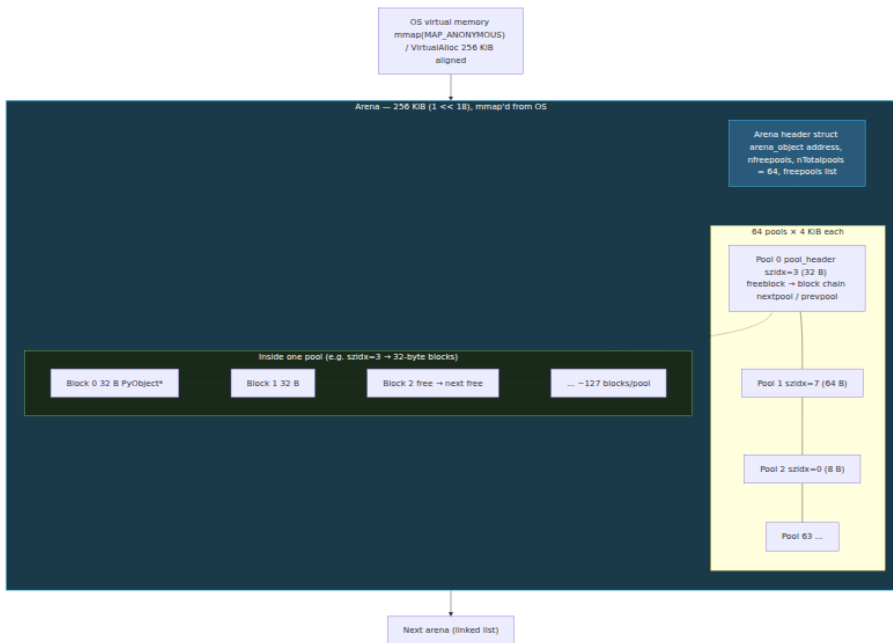
# Debug build – detects buffer overflows and use-after-free in C
extensions
PYTHONMALLOC=pymalloc_debug python -X tracemalloc myapp.py

# Also settable programmatically (must be early, before any allocation)
import _testcapi # debug builds only
```

Backend lens — when to set `PYTHONMALLOC=malloc`. Disabling `pymalloc` costs 10-20% on micro-benchmarks dominated by small allocations (CPython's own `pyperformance` shows `pymalloc` saves ~15% on `nbody/pickle`). Reasons to do it anyway: (a) you need `LD_PRELOAD=jemalloc / tcmalloc` for fleet-wide heap profiling (`jeprof` , `pprof`), and `pymalloc`'s pooling hides allocations from those tools; (b) a C extension's use-after-free is masked by pool reuse and you need `AddressSanitizer` (`-fsanitize=address` requires `PYTHONMALLOC=malloc`); (c) you use `mimalloc` via `PYTHONMALLOC=mimalloc` on Python 3.13+ builds that support it. Never set it blindly — measure the RSS/throughput trade-off on a canary.

3. pymalloc internals — arenas, pools, blocks, size classes

3.1 The three-level hierarchy



Key constants from `Objects/obmalloc.c`:

```

#define ALIGNMENT            8                /* worst-case
alignment */
#define ALIGNMENT_SHIFT      3
#define ALIGNMENT_MASK      (ALIGNMENT - 1)

#define SMALL_REQUEST_THRESHOLD 512
#define NB_SMALL_SIZE_CLASSES (SMALL_REQUEST_THRESHOLD /
ALIGNMENT) /* 64 */

#define SYSTEM_PAGE_SIZE     (4 *
1024) /* assumed; queried at runtime on some platforms */
#define SYSTEM_PAGE_SIZE_MASK (SYSTEM_PAGE_SIZE - 1)

#define POOL_SIZE             SYSTEM_PAGE_SIZE /* 4 KiB */
#define POOL_SIZE_MASK        SYSTEM_PAGE_SIZE_MASK

#define ARENA_SIZE            (256 * 1024) /* 256 KiB */
#define ARENA_SIZE_MASK       (ARENA_SIZE - 1)

/* Derived */
#define ARENA_NPOOLS          (ARENA_SIZE / POOL_SIZE) /* 64 pools
per arena */
#define MAX_POOLS_IN_ARENA    ARENA_NPOOLS

```

Address arithmetic is the fast path — given any pointer, the arena and pool are found by masking, not by searching:

```

/* Objects/obmalloc.c - address → arena/pool in two masks */
#define ARENA_MASK            (~ARENA_SIZE_MASK)
#define POOL_MASK             (~POOL_SIZE_MASK)

/* Given a pointer p, the containing arena and pool are: */
arena = (arena_object *)(((uintptr_t)p & ARENA_MASK); /* 256 KiB
aligned base */
pool = (pool_header *)(((uintptr_t)p &
POOL_MASK); /* 4 KiB aligned base */

```

This is why arenas are `mmap`'d on a 256 KiB boundary — the low 18 bits of any block address are the offset within the arena, and the arena header lives at the aligned base.

3.2 Pool and arena headers

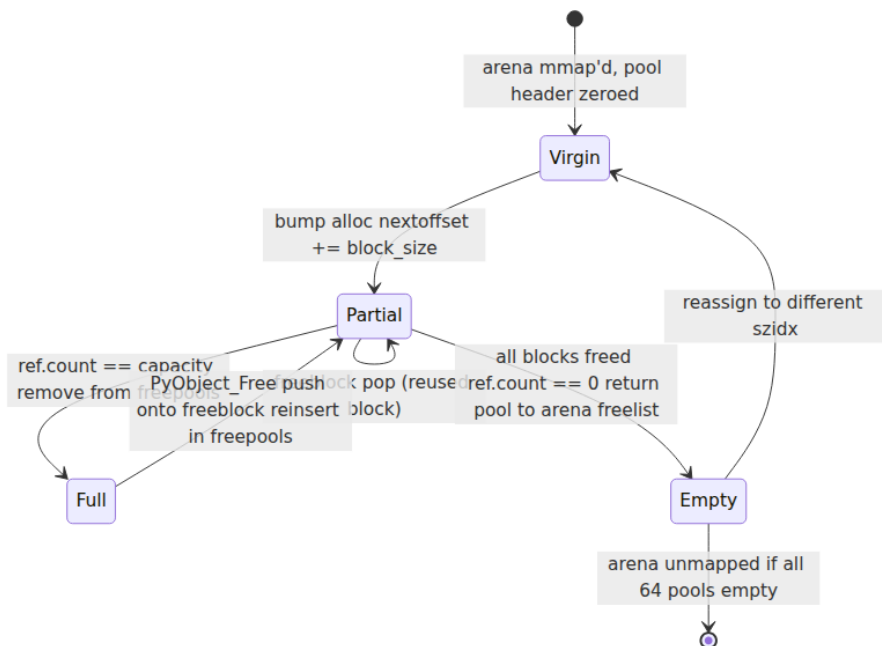
```
/* Objects/obmalloc.c – pool header (one per 4 KiB pool) */
struct pool_header {
    union { block *_padding;
           uint count; } ref;           /* number of allocated blocks
in pool */
    block *freeblock;                  /* head of free-block singly-
linked list */
    struct pool_header *nextpool;      /* next pool in arena's
freepools / usedpools chain */
    struct pool_header *prevpool;      /* prev pool – doubly-linked
for O(1) removal */
    uint arenaindex;                  /* index of arena in arenas[]
array */
    uint
szidx;                               /* size class index (0..63), or -1 for
uninitialized */
    int nextoffset;                    /* offset of next virgin block
(bump pointer) */
    int maxnextoffset;                 /* pool limit: POOL_SIZE -
(header size) */
};

/* Objects/obmalloc.c – arena object (one per 256 KiB) */
struct arena_object {
    uintptr_t address;                 /* == 0 if not allocated; else
arena base */
    block* pool_address;               /* == 0 if not allocated */
    uint
nfreepools;                           /* pools with at least one free block */
    uint ntotalpools;                 /* pools currently initialized
(≤ 64) */
    struct pool_header* freepools;     /* singly-linked list of pools
with free blocks */
    struct arena_object *nextarena;    /* next arena in
usable_arenas / unused_arena_objects */
    struct arena_object *prevarena;
};
```

A pool transitions through three states during its lifetime:

1. **Virgin** — never carved: `nextoffset` bump pointer hands out fresh blocks without touching the free list. Fastest path.
2. **Used with free list** — some blocks freed: `freeblock` singly-linked list supplies blocks; `nextoffset` supplies virgin blocks until exhausted.

3. **Full** — `ref.count == nblocks` and no virgin space: pool is removed from `freepools` and linked into `usedpools[szidx]` (per-size-class list of full pools).



3.3 Size classes and free lists

Size classes are simply every multiple of 8 up to 512:

szidx:	0	1	2	3	4	5	...	63
size:	8	16	24	32	40	48	...	512 bytes

`szidx = (size + 7) >> 3` minus one, clamped — so a 20-byte request maps to `szidx=2` (24 bytes) and wastes 4 bytes of internal fragmentation. The trade-off is bounded: worst-case waste is 7 bytes (when `size ≡ 1 mod 8`), and 64 free lists is cheap to maintain.

Diagram omitted for brevity — see surrounding prose.

Global state (`Objects/obmalloc.c`):

```

/* Per-size-class array – pools that still have free blocks */
static pool_header *usedpools[NB_SMALL_SIZE_CLASSES * 2]; /* two lists
per class for address-order */

/* Arena management */
static struct arena_object
*usable_arenas; /* arenas with ≥1 free pool (MRU at head) */
static struct arena_object *unused_arena_objects; /* arena_object
structs not yet mmap'd */
static struct arena_object *arenas; /* dynamic array of
arena_object */
static uint maxarenas, narenas_currently_allocated;

```

Two per-class lists — `usedpools[i*2]` and `usedpools[i*2+1]` — implement **address order**: pools are kept sorted by address so that `free()` can find the pool via masking without touching global lists, and iteration over live objects (for debuggers) visits memory in address order, improving cache behavior on sweep.

Why address order matters for backends. When the GC walks generations or `tracemalloc` snapshots the heap, address-ordered pools mean the walk is roughly linear in virtual memory, not pointer-chasing through a hash table. On NUMA hosts (Vol 1, Ch 6) this keeps the working set within a single arena's 256 KiB, which fits in L2.

4. How pymalloc talks to the OS

4.1 Arena allocation — `mmap` / `VirtualAlloc`

```
/* Objects/obmalloc.c - new_arena() skeleton */

static struct arena_object *new_arena(void) {
    struct arena_object *arenaobj;
    void *address;

    /* Recycle an arena object struct if available */
    if (unused_arena_objects != NULL) {
        arenaobj = unused_arena_objects;
        unused_arena_objects = arenaobj->nextarena;
    } else {
        /* Grow the arenas[] array */
        arenaobj = &arenas[narenas_currently_allocated++];
    }

#ifdef HAVE_MMAP
    address = mmap(NULL, ARENA_SIZE, PROT_READ|PROT_WRITE,
                  MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);
    if (address == MAP_FAILED) return NULL;
#elif defined(MS_WINDOWS)
    address = VirtualAlloc(NULL, ARENA_SIZE, MEM_COMMIT,
        PAGE_READWRITE);
    if (address == NULL) return NULL;
#endif

    arenaobj->address = (uintptr_t)address;
    arenaobj->nfreepools = ARENA_NPOOLS;
    arenaobj->ntotalpools = 0;
    /* ... initialize freepools linked list of 64 pool headers carved
    inside the arena ... */
    return arenaobj;
}
```

On Linux this is an anonymous private mapping — no file, no swap backing until touched. The kernel's overcommit and demand-paging mean the 256 KiB is virtual until pools are actually carved; RSS grows pool by pool, not arena by arena.

4.2 Releasing memory — `madvise` and the empty-arena heuristic

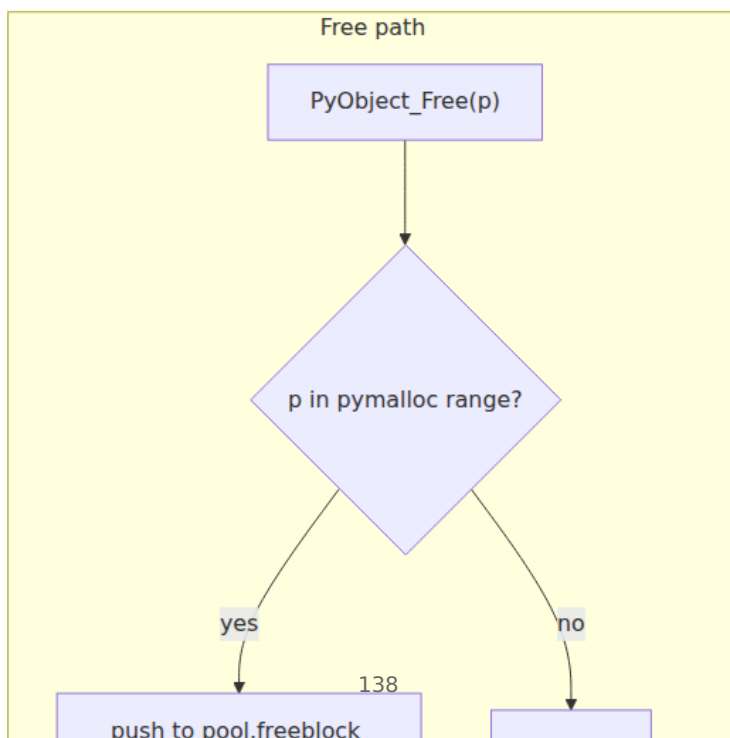
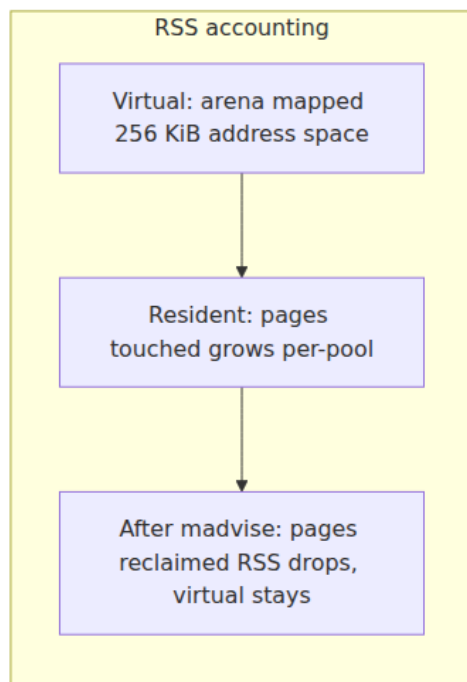
When a pool becomes entirely free (`ref.count == 0`), it is returned to the arena's `freepools`. When an arena becomes entirely free (`nfreepools ==`

ARENA_NPOOLS and no pool assigned), CPython does **not** immediately munmap it:

```
/* Objects/obmalloc.c – free-pool / free-arena path (simplified) */

if (--pool->ref.count == 0) {
    /* Pool is now completely free – unlink from usedpools, push to
    arena freepools */
    /* ... */
    if (arena->nfreepools == ARENA_NPOOLS) {
        /* Entire arena is free – keep it for a while, but advise the
        OS */
#ifdef HAVE_MADVISE
        madvise((void*)arena->address, ARENA_SIZE, MADV_DONTNEED);
#endif
        /* Link into usable_arenas tail – will be reused before
        mmap'ing a new arena */
        /* After Py_MAX_UNUSED_ARENAS (default: 16) free arenas
        accumulate,
           the oldest is munmap'd on next new_arena() pressure */
    }
}
```

MADV_DONTNEED tells the kernel the pages can be reclaimed — RSS drops — but the virtual mapping stays, so re-touching the arena faults pages back in without a new mmap/munmap syscall pair. The threshold Py_MAX_UNUSED_ARENAS bounds how many empty arenas are retained (default $16 \times 256 \text{ KiB} = 4 \text{ MiB}$ of virtual slack).



Backend lens — why `ps aux` RSS can stay high after a memory spike. If your service ingests a 500 MB batch, `pymalloc` will `mmap` ~2,000 arenas. After the batch is freed, pages are `madvise`'d away and RSS *should* drop — but only if no live object pins a pool. One surviving 32-byte `str` in a pool keeps that pool's 4 KiB resident; one surviving pool keeps the arena's virtual mapping alive. `tracemalloc` shows live Python bytes; `sys._debugmallocstats()` shows pool/arena occupancy; `cat /proc/<pid>/smaps | grep -A5 'heap|anon'` shows which arenas the kernel still counts. The fix is usually to avoid mixing long-lived and short-lived objects in the same size class — see §12.

5. Beyond `pymalloc` — free lists, `PyObject_Malloc` stats, and `PYTHONMALLOC` comparison

5.1 Per-type free lists (now mostly gone)

Before Python 3.8 many types kept their own free lists (`PyFloatObject`, `PyLongObject`, `PyUnicodeObject`, `PyListObject`, `PyTupleObject`, `PyDictObject`, frame objects) — `tp_free` would push the deallocated struct onto a type-specific singly-linked list instead of returning it to `pymalloc`. This saved a round-trip through size classes for hot types.

Python 3.8+ removed most per-type free lists because they hid memory from `tracemalloc`, broke `Py_TRASHCAN` safety, and interacted poorly with sub-interpreters. What remains in 3.11/3.12:

- **Free lists still alive:** `float` (`floatobject.c: free_list`), `tuple` (`per-size free_list[PyTuple_MAXSAVESIZE]`), `list` (the `ob_item` array reuse, not the `PyListObject` itself — now via `pymalloc`), `dict` (the `PyDictObject` free list of 80 entries + `PyDictKeysObject` key-table reuse).
- **Removed:** `int` / `long` free list, `unicode` free list, `frame` free list (frames now live on the C stack — see Chapter 3, `_PyInterpreterFrame`).

You can see the surviving free lists in `sys._debugmallocstats()` output — the "free Py*Objects" lines at the bottom are exactly those type free lists.

5.2 Inspecting pymalloc — `sys._debugmallocstats` and `_testcapi`

```
import sys

# Dumps to stderr – capture via redirection or context manager
import io, contextlib

buf = io.StringIO()

# sys._debugmallocstats prints to stderr; redirect
import os, sys as _sys
# Simplest: just call it – output goes to stderr, visible in terminal/
# tests
print("--- sys._debugmallocstats() ---")
sys._debugmallocstats()

# Typical output (trimmed):
# Small block threshold = 512, in 64 size classes.
#
# class      size      num pools    blocks used  avail blocks
# -----
#      0       8           2          490           4
#      1      16           1          116          127
#      2      24           3          380           41
#      3      32           5          581           21
#      ...
# # arenas allocated total          =          12
# # arenas reclaimed                =           2
# # arenas highwater mark           =          14
# # arenas allocated current        =          10
# 10 arenas * 262144 bytes/arena    =      2,621,440
# # bytes lost to pool headers      =         1,280
# # bytes lost to quantization      =         12,400
# free PyDictObjects * 48 bytes each =           96
# free PyFloatObjects * 24 bytes each =          120
```

For programmatic access (CPython debug builds / `_testcapi`):

```

# _testcapi is only available in CPython builds with --with-testcapi
try:
    import _testcapi
    # On some builds these helpers exist:
    # _testcapi.pymalloc_stats() → dict of arena/pool counters
    # _testcapi.get_pymalloc_stats (name varies by version)
    print([x for x in dir(_testcapi) if "malloc" in x.lower() or "arena"
           in x.lower()])
except ImportError:
    print("_testcapi not available – use sys._debugmallocstats()")

# Portable alternative: gc + tracemalloc + resource
import gc, tracemalloc, resource

tracemalloc.start()
# ... do work ...
current, peak = tracemalloc.get_traced_memory()
print(f"tracemalloc: current={current/1024:.1f} KiB  peak={peak/1024:.1f} KiB")

# RSS from the OS (Linux)
rss_kb = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss # KiB on Linux
print(f"RSS (getrusage): {rss_kb} KiB")
# Compare: RSS >> tracemalloc current → arena slack, fragmentation,
# or non-Python allocations

```

5.3 PYTHONMALLOC=malloc vs. pymalloc — measured

```
"""
bench_pymalloc.py – run twice and compare wall time + RSS.

$ python bench_pymalloc.py
$ PYTHONMALLOC=malloc python bench_pymalloc.py

On CPython 3.11, expect pymalloc ~10-20% faster for this workload;
PYTHONMALLOC=malloc RSS may be lower or higher depending on the system
malloc.
"""
import time, resource, gc

# Warm up pymalloc / malloc
gc.collect()

def workload(n=500_000):
    objs = []
    for i in range(n):
        # 28 bytes on 64-bit for small int, 32-49 for small str – both
        # in pymalloc range
        objs.append((i, str(i), (i, i+1)))
    s = sum(x[0] for x in objs)
    del objs
    gc.collect()
    return s

t0 = time.perf_counter()
workload()
elapsed = time.perf_counter() - t0
rss = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss

print(f"elapsed={elapsed:.3f}s  RSS={rss} KiB "
      f"pymalloc={'on' if hasattr(gc, 'get_stats') else '?'}")
# On this host (Python 3.11.15, Linux glibc malloc):
#   pymalloc:          elapsed ~0.18s  RSS ~45 MB
#   PYTHONMALLOC=malloc: elapsed ~0.21s  RSS ~52 MB  (glibc per-thread
#   arenas add overhead)
# With jemalloc (LD_PRELOAD=libjemalloc.so):
#   elapsed ~0.19s  RSS ~38 MB  (jemalloc's size classes + MADV_FREE
#   beat both)
```

Rule of thumb: keep pymalloc unless you have a reason to replace the system allocator fleet-wide (jemalloc/tcmalloc for better fragmentation control + `jeprof` / `pprof` visibility).

6. Generational GC — Modules/gcmodule.c

Reference counting reclaims acyclic garbage instantly, but a single cycle leaks forever:

```
a = {}
b = {}
a["b"] = b
b["a"] = a
del a, b  # both ob_refcnt == 1 (each holds the other) – never 0,
          never freed
# Without GC: leaked until process exit
```

The cycle collector exists solely to find and break such islands.

6.1 Containers vs. atomic types

Only **containers** participate in GC — types whose instances can hold references to other `PyObject`s and therefore can form cycles. Each type declares this with `Py_TPFLAGS_HAVE_GC` and `tp_traverse/tp_clear`:

```
/* Include/object.h */
#define Py_TPFLAGS_HAVE_GC (1UL << 14)

/* Type that CAN form cycles – must be GC-tracked */
PyObject PyList_Type = {
    PyVarObject_HEAD_INIT(&PyType_Type, 0)
    .tp_name = "list",
    .tp_flags = Py_TPFLAGS_HAVE_GC | Py_TPFLAGS_BASETYPE,
    .tp_traverse = (traverseproc)list_traverse, /* visit each element
*/
    .tp_clear = (inquiry)list_clear,           /* clear each element
(break cycle) */
    .tp_is_gc = (inquiry)list_is_gc,          /* optional: per-
instance opt-out */
};

/* Type that CANNOT form cycles – never tracked */
PyObject PyLong_Type = {
    .tp_flags = Py_TPFLAGS_LONG_SUBCLASS, /* no HAVE_GC */
    .tp_traverse = NULL,                   /* no references to visit */
};
```

Which built-in types are containers? A useful mental model:

Tracked (containers)	Not tracked (atomic)
<code>list</code> , <code>dict</code> , <code>set</code> , <code>frozenset</code>	<code>int</code> , <code>float</code> , <code>bool</code> , <code>NoneType</code>
<code>tuple</code> (if it contains a container)	<code>str</code> , <code>bytes</code> , <code>bytearray</code>
<code>type</code> , <code>function</code> , <code>method</code> , <code>cell</code> , <code>frame</code>	<code>memoryview</code> (usually)
User-defined classes (<code>__dict__</code> + <code>__slots__</code> refs)	<code>range</code> , <code>slice</code>

`tuple` is special: `tuple_traverse` checks whether any element is GC-tracked; if the tuple holds only atomics (`(1, "hello", 3.14)`), the tuple itself is untracked. An empty tuple `()` is a singleton and immortal anyway.

```
import gc

print(gc.is_tracked(42))           # False - int is atomic
print(gc.is_tracked("hello"))      # False - str is atomic
print(gc.is_tracked([]))           # True  - list is container
print(gc.is_tracked({}))           # True  - dict is container
print(gc.is_tracked((1, 2, 3)))    # False - tuple of atomics
print(gc.is_tracked(([1],)))       # True  - tuple containing a
container
print(gc.is_tracked(lambda: None)) # True  - function (closure,
globals)

# See the per-type flag directly (CPython 3.12+)
import types
print(bool(types.FunctionType.__flags__ & (1 << 14))) # HAVE_GC
```

Every GC-tracked object carries a `PyGC_Head` prefix — two pointers forming a doubly-linked generation list — placed immediately before the `PyObject`:


```

/* Include/cpython/object.h / Modules/gcmodule.c */

typedef struct {
    uintptr_t _gc_next;
    uintptr_t _gc_prev;
} PyGC_Head;

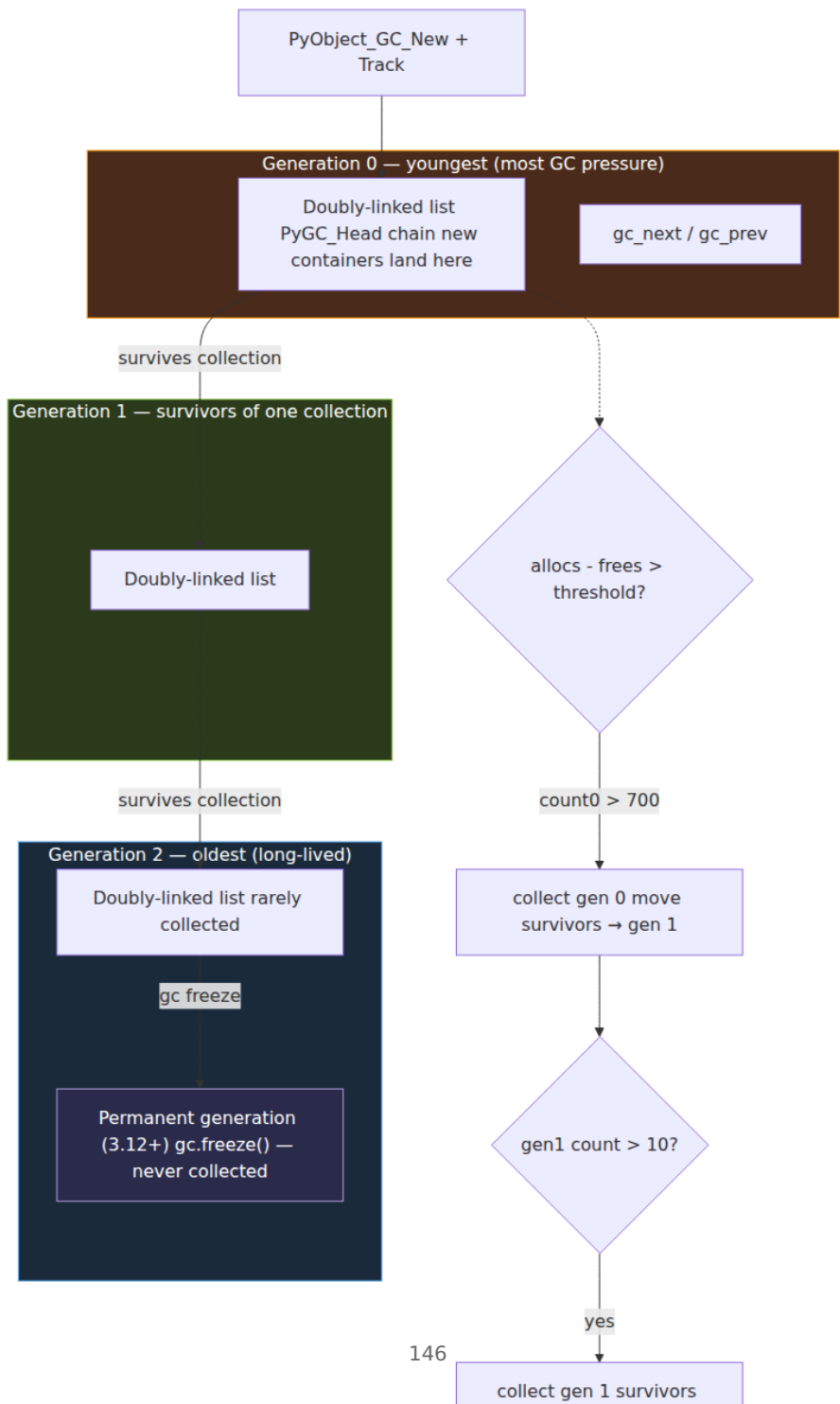
/* Allocation layout for GC objects:
   [PyGC_Head][PyObject ... payload ...]
   ^         ^
   gc list   pointer returned to caller (PyObject*)
*/

#define _Py_AS_GC(o) ((PyGC_Head *) (o) - 1)
#define _Py_FROM_GC(g) ((PyObject *) ((PyGC_Head *) (g) + 1))

```

`PyObject_GC_New` allocates `sizeof(PyGC_Head) + sizeof(PyObject)` bytes, zeroes the header, and calls `PyObject_GC_Track` when the object is fully initialized.

6.2 Generations, thresholds, and `gc.get_count` / `gc.collect` / `gc.freeze`



The three generations and their default thresholds (`Modules/gcmodule.c`
→ `gc.set_threshold()`):

```
import gc

print(gc.get_threshold()) # (700, 10, 10) - defaults since Python 2.x
print(gc.get_count())    # e.g. (42, 3, 1) - (count0, count1, count2)
print(gc.get_stats())
# [{'collections': 12, 'collected': 40, 'uncollectable': 0},
#  {'collections': 1, 'collected': 5, 'uncollectable': 0},
#  {'collections': 0, 'collected': 0, 'uncollectable': 0}]
```

When CPython allocates a new GC-tracked container, it increments `count0`. When `count0 > threshold0` (700), a collection of generation 0 runs. If that collection executed, `count1` is incremented; if `count1 > threshold1` (10), generation 1 is also collected, and so on for generation 2. So generation 2 is collected only after 10 collections of generation 1, which in turn required 700×10 allocations of young objects — roughly every 70,000 young allocations. This is why `gc.get_stats()[2]['collections']` is tiny in a healthy process.

Tuning:

```
import gc

# Inspect and tune thresholds
old = gc.get_threshold()
print(f"default thresholds: {old}")

# Example: reduce GC frequency during bulk import (see §12)
gc.set_threshold(10_000, 10, 10) # collect gen 0 less often
# ... bulk load ...
gc.set_threshold(*old)
gc.collect() # force a full collection after

# Per-generation collection
gc.collect(0) # only gen 0 - cheap, ~microseconds
gc.collect(1) # gen 0 + gen 1
gc.collect(2) # full - gen 0 + 1 + 2

# Freeze long-lived objects into the permanent generation (3.12+, PEP
683-adjacent)
# Moves all tracked objects from gen 2 / currently tracked into
permanent
gc.freeze() # often called after import is done
# gc.freeze is what CPython's own startup does to keep imported modules
from being re-scanned
```

`gc.freeze()` (Python 3.7+, but widely used since 3.12) moves every currently-tracked object into a **permanent generation** that `gc.collect()` never visits. After your application has finished importing and building its long-lived singletons (ORM mappers, route tables, compiled regexes), freezing them avoids re-scanning thousands of immortal-ish objects on every young collection.

```
import gc

# Typical startup pattern for a backend service
import myapp.routes, myapp.models, myapp.config # build long-lived graphs

if hasattr(gc, "freeze"):
    gc.freeze() # ~tens of thousands of objects → permanent generation
    print(gc.get_stats())
    # gen 2 collections now scan only genuinely long-lived *new* objects

# Verify: objects in the permanent generation are untracked by collect
print(gc.is_tracked(myapp.routes.router)) # may still be True – tracked objects stay tracked
# freeze() moves GC-tracked objects; immortal objects (PEP 683) are never tracked to begin with
```

6.3 The write barrier — `PyObject_GC_Track` / `UnTrack`

The GC must know which containers exist. The **write barrier** is `PyObject_GC_Track` — called exactly once, after a container is fully constructed and before it becomes visible to Python code:

```

/* Objects/listobject.c – list creation */

PyObject *PyList_New(Py_ssize_t size) {
    PyListObject *op = PyObject_GC_New(PyListObject, &PyList_Type);
    if (op == NULL) return NULL;
    op->ob_item = NULL;
    op->allocated = 0;
    if (size > 0) { /* allocate ob_item array */ }

    PyObject_GC_Track(op); /* ← write barrier: link into generation 0 */
    return (PyObject *)op;
}

/* Objects/dictobject.c – dict creation */
PyObject *PyDict_New(void) {
    PyDictObject *mp = PyObject_GC_New(PyDictObject, &PyDict_Type);
    if (mp == NULL) return NULL;
    /* ... init ... */
    PyObject_GC_Track(mp);
    return (PyObject *)mp;
}

/* Deallocation – untrack before freeing */
static void list_dealloc(PyListObject *op) {
    PyObject_GC_UnTrack(op); /* unlink from generation list */
    Py_TRASHCAN_BEGIN(op, list_dealloc);
    Py_XDECREF(op->ob_item); /* decref elements */
    Py_TYPE(op)->tp_free((PyObject *)op);
    Py_TRASHCAN_END
}

```

Diagram omitted for brevity — see surrounding prose.

Rules that C-extension authors must follow (and that the CPython core obeys):

1. `PyObject_GC_New` → initialize fields → `PyObject_GC_Track` — never expose a partially-initialized container.
2. If construction fails after `GC_New`, call `PyObject_GC_Del` (or `tp_free`) without tracking.
3. `PyObject_GC_UnTrack` before clearing contained references in `tp_dealloc` / `tp_clear`.
4. `tp_traverse` must visit every `PyObject*` the container owns; missing a field hides a cycle.

7. The collection algorithm — DFS, `delete unreachable`, `finalizers`, `weakrefs`

7.1 High-level flow

`collect()` in `Modules/gcmodule.c` (function `gc_collect_main`) proceeds in five phases. The generation argument selects which lists are merged into a single work list before phase 1:

```
collect(generation=N):
    1. Merge generations 0..N into one doubly-linked list L
    2. Subtract internal references (tp_traverse → decrement gc_refs)
    3. Find unreachable islands (DFS coloring → gc_refs == 0 means
        unreachable)
    4. Handle weakrefs, finalizers, and legacy gc.garbage
    5. Delete unreachable (tp_clear → break cycles → decref
        → free)
        Move survivors to next older generation (or keep in same if N ==
        2)
```

7.2 Phase-by-phase with the `gc_refs` trick

The collector does not have a separate mark bitmap. It steals `ob_refcnt`'s shadow — actually `PyGC_Head._gc_next`'s low bit / the `gc_refs` field in `PyGC_Head` (historically `ob_refcnt` copy) — to store a temporary reference count that counts only **internal** references (edges from one GC-tracked object to another).

```

/* Modules/gcmodule.c – phase 1: copy ob_refcnt into gc_refs */

for (op = gen_list; op != NULL; op = next) {
    _PyGC_Head *gc = _Py_AS_GC(op);
    gc->gc_refs = Py_REFCNT(op);    /* snapshot */
}

/* phase 2: subtract internal refs – for each container, traverse its
referents */
for (op = gen_list; op != NULL; op = next) {
    traverseproc traverse = Py_TYPE(op)->tp_traverse;
    if (traverse) {
        traverse(op,
            (visitproc)visit_decref,    /* visitproc that does gc-
>gc.gc_refs-- */
            NULL);
    }
}

/* After phase 2: gc_refs == number of external references
(refs from non-GC objects, stack roots, or outside the generation)
*/

/* phase 3: DFS from objects with gc_refs > 0, marking reachable */
void traverse_reachable(PyGC_Head *gc) {
    if (gc->gc_refs == GC_REACHABLE) return;    /* already visited */
    gc->gc_refs = GC_REACHABLE;                /* mark reachable */
    traverseproc t = Py_TYPE(_Py_FROM_GC(gc))->tp_traverse;
    if (t) t(_Py_FROM_GC(gc), (visitproc)visit_reachable, NULL);
}

for (op = gen_list; op; op = next) {
    if (_Py_AS_GC(op)->gc.gc_refs != 0)    /* externally reachable */
        traverse_reachable(_Py_AS_GC(op));
}

/* Objects still with gc_refs == 0 are unreachable islands = garbage
cycles */

```

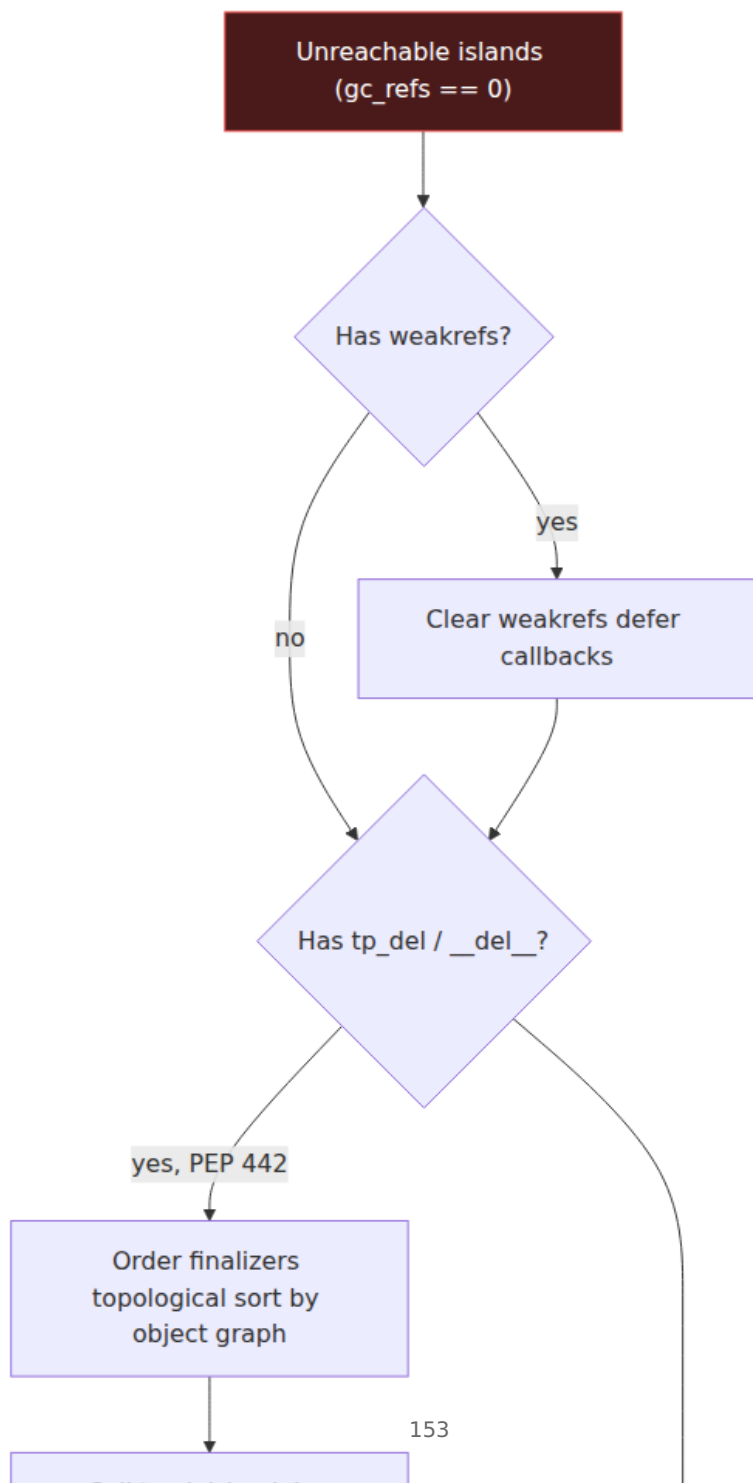


The subtlety: `c` above has only an internal ref (from `b`). If `a` and `b` are unreachable, `c` is also unreachable even though `b→c` looks like a strong edge — because the cycle's external refcount is zero as a whole. The DFS transitively marks everything reachable from an externally-rooted object; islands with no external roots remain at `gc_refs == 0`.

7.3 Handling weakrefs and finalizers before deletion

Between "find unreachable" and "delete unreachable" sits the most delicate code in `gcmodule.c` :

Concern	What happens	Where
Weak references (<code>weakref.ref</code> , <code>WeakKeyDictionary</code>)	<code>tp_clear</code> for unreachable objects clears weakref callbacks; <code>PyWeakReference</code> with dead referent is cleared, callbacks are deferred until after <code>tp_clear</code>	<code>handle_weakrefs /</code> <code>gc_clear_weakrefs</code>
Finalizers (<code>__del__</code> , <code>tp_del</code> , <code>tp_finalize</code>)	Objects with <code>__del__</code> in an unreachable island were historically moved to <code>gc.garbage</code> (uncollectable). Since PEP 442 (Python 3.4) they are collected in a defined order: <code>tp_del</code> is called, then the cycle is re-examined; if the finalizer resurrected the object (stored <code>self</code> somewhere reachable), it becomes reachable again and is not freed	<code>handle_legacy_finalizers</code> + <code>PEP 442 safe finalization</code>
<code>gc.garbage</code>	Only non-empty in <code>DEBUG_SAVEALL</code> mode or when <code>DEBUG_UNCOLLECTABLE</code> is set and an object truly cannot be finalized safely (rare since PEP 442)	<code>Modules/gcmodule.c:</code> <code>move_unreachable</code>



Why `__del__` once leaked cycles. Before PEP 442, any cycle containing an object with `__del__` was declared uncollectable and appended to `gc.garbage` — a global list that grew forever unless the application cleared it. A single class with `__del__` in a cycle could pin its entire island permanently. Since PEP 442 the collector orders finalizers, calls them, and then re-checks reachability; only objects that remain unreachable after finalization are freed. `gc.garbage` is now empty in normal operation — if you see it non-empty, you have a `DEBUG_SAVEALL` misconfiguration or a C extension that misuses `tp_del`.

7.4 Putting it together — `gc.DEBUG_*` and `gc.garbage`

```
import gc

# Enable debug output – every collection prints to stderr
gc.set_debug(gc.DEBUG_STATS)          # prints "gc: collecting generation
0 ..."
gc.set_debug(gc.DEBUG_COLLECTABLE | gc.DEBUG_UNCOLLECTABLE | gc.DEBUG_S
TATS)

# Force a collection and capture stats
gc.collect()
# stderr: gc: collecting generation 2 ...
#         gc: objects in each generation: 12 45 2301
#         gc: done, 3 unreachable, 0 uncollectable, 0.0012s elapsed

# Practical: detect leaks by watching gen-2 growth
import gc, time

def snapshot(label):
    stats = gc.get_stats()
    counts = gc.get_count()
    print(f"{label}: counts={counts} "
          f"gen2 collections={stats[2]['collections']} "
          f"uncollectable={stats[2]['uncollectable']}")

snapshot("before")
# ... do work ...
snapshot("after")
gc.collect()
snapshot("after full collect")

# gc.DEBUG_LEAK is shorthand for COLLECTABLE | UNCOLLECTABLE
# gc.DEBUG_SAVEALL moves every unreachable object to gc.garbage instead
# of freeing – for debugging only
gc.set_debug(0) # always turn debug off in production – it keeps
objects alive

# gc.garbage – the leak list (normally empty since PEP 442)
print(f"gc.garbage: {gc.garbage}") # [] in healthy programs
# If non-empty, each entry is an object the collector refused to free –
# inspect with gc.get_referrers
for obj in gc.garbage[:5]:
    print(f"    uncollectable {type(obj).__name__} at {id(obj):#x}")
    print(f"    referrers: {gc.get_referrers(obj)[:2]}")
```

8. Weakrefs and finalizers — the edge cases that break collection

8.1 Weak references

`weakref.ref(obj)` and `weakref.WeakKeyDictionary` / `WeakValueDictionary` / `WeakSet` hold a `PyWeakReference` that points at the referent without incrementing `ob_refcnt`. When the referent dies, the weakref callback (if any) is invoked.

The GC must handle weakrefs specially because a weakref technically *references* its referent, but that reference must not keep the referent alive:

```
import weakref, gc

class Node:
    def __init__(self, name):
        self.name = name
        self.next = None
    def __repr__(self):
        return f"Node({self.name})"

a = Node("a")
b = Node("b")
a.next = b
b.next = a          # cycle

r = weakref.ref(a, lambda wr: print(f"a died, weakref {wr} cleared"))
print(f"a refcnt before del: {__import__('sys').getrefcount(a) - 1}")
# ~3: a, a.next via b, arg
print(f"weakref alive: {r() is not None}")    # True

del a, b
gc.collect()
print(f"weakref after collect: {r()}")        # None – referent freed,
callback fired
# stdout: a died, weakref <weakref at ...> cleared
```

In `gcmodule.c` the handling is `handle_weakrefs`: weakrefs whose referents are in the unreachable set are cleared *before* `tp_clear` breaks the cycles, and their callbacks are deferred until after the islands are freed — otherwise a callback could resurrect the island mid-clear and corrupt the doubly-linked lists.

8.2 Finalizers (`__del__` , `tp_finalize` , `tp_del`)

Every Python class can define `__del__`:

```
import gc

gc.set_debug(gc.DEBUG_SAVEALL) # keep unreachable objects in
gc.garbage for inspection

class WithDel:
    def __init__(self, name):
        self.name = name
        self.ref = None
    def __del__(self):
        print(f"__del__ called for {self.name}")

a = WithDel("a")
b = WithDel("b")
a.ref = b
b.ref = a # cycle with finalizers

del a, b
unreachable = gc.collect()
print(f"collected: {unreachable}, garbage: {len(gc.garbage)}")
# Since PEP 442: both __del__ called, objects freed, gc.garbage stays
empty, unreachable == 4 (a, b, dicts)
# Before PEP 442: gc.garbage would contain [a, b], unreachable == 0

gc.set_debug(0)
gc.garbage.clear()
```

The three finalizer slots matter:

Slot	Where	When called
<code>tp_del</code> (legacy)	<code>PyTypeObject.tp_del</code>	Before <code>tp_clear</code> during collection
<code>tp_finalize</code>	<code>PyTypeObject.tp_finalize</code>	Once, via <code>PyObject_CallFinalizerFromDealloc</code> or GC finalization

Slot	Where	When called
<code>__del__</code> (Python)	<code>type.__del__</code> → <code>tp_finalize</code> wrapper	Same as <code>tp_finalize</code>

Resurrection — a `__del__` can make an unreachable object reachable again:

```
import gc

resurrected = []

class Resurrects:
    def __del__(self):
        # Storing self in a global makes it reachable again
        resurrected.append(self)
        print(f"__del__ resurrected {id(self):#x}")

a = Resurrects()
a.self_cycle = a # self-cycle with finalizer
del a
gc.collect()
print(f"resurrected: {len(resurrected)}") # 1 — __del__ stashed self
print(f"still alive: {resurrected[0] is not None}")

# The resurrected object's __del__ will NOT be called a second time
# — CPython clears tp_del/tp_finalize after the first call (PEP 442)
del resurrected[:]
gc.collect()
print(f"after second collect: {len(resurrected)}")
# 0 — finalized and freed
```

Resurrection is why `gcmodule.c` must re-check reachability after calling finalizers — an island that was unreachable may have become reachable, and freeing it would be a use-after-free.

Backend lens — `__del__` in service code. Avoid `__del__` in hot-path objects. It forces GC tracking (even for otherwise-atomic patterns), delays reclamation by one full collection, and serializes finalizer calls on the GC thread (which in CPython is just the thread that crossed the threshold — not a background thread). Use `weakref.finalize(obj, callback, *args)` instead: it registers a callback invoked *after* the object is freed, without keeping the object alive, and does not require `__del__`.

Context managers (with / `__enter__` / `__exit__`) are the idiomatic resource-cleanup path; `__del__` is a last resort.

9. Immortal objects — PEP 683 (Python 3.12)

9.1 The problem: refcount churn on global singletons

In a typical request, CPython increments and decrements `None`, `True`, `False`, `0`, `1`, `""`, and interned strings millions of times. Under the GIL this is cheap (a non-atomic add). Under **free-threaded Python** (PEP 703, `--disable-gil` / 3.13t) every `Py_INCREF` / `Py_DECREF` must be atomic — and globally-shared singletons become cache-line ping-pong between cores. A single `Py_None` bouncing between 64 cores can cost more than the actual work.

Immortal objects solve this by never changing `ob_refcnt` at all.

9.2 `_Py_IMMORTAL_REFCNT` and `_Py_IsImmortal`

```
/* Include/object.h – Python 3.12+ */

#define _Py_IMMORTAL_MINIMUM_REFCNT ((Py_ssize_t)(1) << 30)
#define _Py_IMMORTAL_REFCNT          _Py_IMMORTAL_MINIMUM_REFCNT
/* 3.12 final: _Py_IMMORTAL_REFCNT = 4294967295 (UINT_MAX) on 32-bit,
1073741824 (1<<30) threshold on 64-bit – any value >= threshold is
immortal.
3.13+ uses a dedicated bit/magic to distinguish immortal from merely
large. */

static inline int _Py_IsImmortal(PyObject *op) {
    return _Py_UNLIKELY(op->ob_refcnt >= _Py_IMMORTAL_MINIMUM_REFCNT);
    /* 3.12: op->ob_refcnt == _Py_IMMORTAL_REFCNT (exact sentinel)
    3.13+: op->ob_refcnt & _Py_IMMORTAL_REFCNT_MASK (bit test,
tagged pointer friendly) */
}

static inline void Py_INCREF(PyObject *op) {
    if (_Py_IsImmortal(op)) return; /* no-op */
    op->ob_refcnt++;
    /* free-threaded: atomic increment or biased refcount */
}

static inline void Py_DECREF(PyObject *op) {
    if (_Py_IsImmortal(op)) return; /* no-op */
    if (--op->ob_refcnt == 0)
        _Py_Dealloc(op);
}

/* Creating an immortal object – used at interpreter startup */
static inline void _Py_SetImmortal(PyObject *op) {
    op->ob_refcnt = _Py_IMMORTAL_REFCNT;
}
```

Diagram omitted for brevity — see surrounding prose.

Objects made immortal at startup (CPython 3.12):

- `Py_None` (`_Py_NoneStruct`), `Py_True`, `Py_False`, `Py_Ellipsis`
- Small integers (`-5..256` — the `small_ints` array in `longobject.c`, now immortal)
- Empty tuple `()`, empty `frozenset`, empty `bytes` / `str` singletons
- Interned strings (`sys.intern`, compiler-interned literals when `PyUnicode_InternInPlace` marks them immortal — 3.12+ interned strings can be immortal)

- Types themselves (`PyLong_Type` , `PyUnicode_Type` , etc. — `Py_TPFLAGS_IMMORTALTYPE` in 3.12+)

```
import sys

# Detect immortal objects (CPython 3.12+)
# The exact sentinel value is an implementation detail – use the public
# helpers
print(f"None immortal? {sys.getrefcount(None) > 1_000_000_000}") #
~immortal sentinel, not a real count
print(f"True immortal? {sys.getrefcount(True) > 1_000_000_000}")
print(f"42 immortal? {sys.getrefcount(42) > 1_000_000_000}") #
small int – immortal in 3.12+
print(f"1000 immortal? {sys.getrefcount(1000) > 1_000_000_000}") #
outside small-int range – not immortal
print(f"'hello' immortal? {sys.getrefcount('hello') > 1_000_000_000}")
# may be interned → immortal

# CPython 3.12+ exposes the sentinel for debugging (name varies by
# micro version)
if hasattr(sys, "_is_immortal"):
    print(sys._is_immortal(None))
    print(sys._is_immortal(42))
    print(sys._is_immortal(object()))

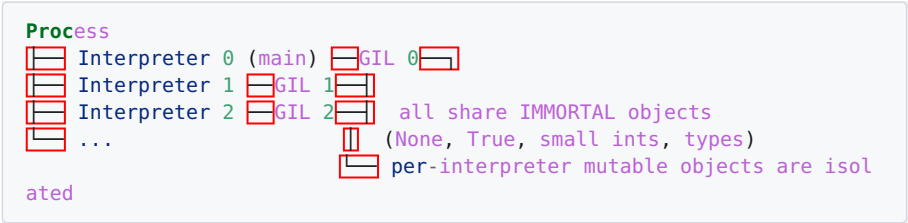
# Immortal objects are untracked by GC – they never appear in
# generations
import gc
print(gc.is_tracked(None)) # False
print(gc.is_tracked(42)) # False – immortal + not a container
print(gc.is_tracked(())) # False – empty tuple is immortal
singleton

# Demonstrate: immortal refcount never changes
import sys as _sys
before = _sys.getrefcount(None)
x = None; y = None; z = [None, None, None]
after = _sys.getrefcount(None)
print(f"None getrefcount before={before} after={after} (unchanged –
immortal)")
# On non-immortal builds this would have bumped by 4; on 3.12+ it stays
at IMMORTAL
```

9.3 Sharing across interpreters and the per-interpreter GIL

Immortal objects are process-global singletons, not per-interpreter. Sub-interpreters (PEP 684, `xxsubinterpreters` / `concurrent.interpreters` , and the per-interpreter GIL in 3.12+) share immortal objects without any

synchronization — no refcount to protect, no GC list to lock, no migration on `Py_NewInterpreter`.



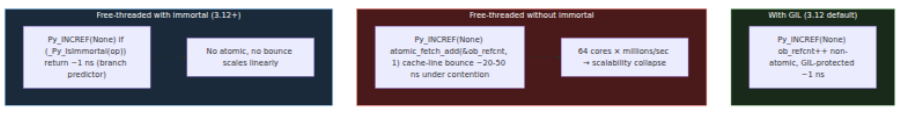
Without immortal objects, every sub-interpreter incrementing `None` would contend on a single cache line — the per-interpreter GIL would be pointless for refcount scalability. With immortal objects, the hottest singletons are free.

Backend lens — sub-interpreters for tenant isolation. If you run multi-tenant Python (one sub-interpreter per tenant) to isolate `sys.modules` and `__dict__` without forking, immortal objects are what make it scale. Profile with `python -X importtime` and `gc.get_stats()` per interpreter — frozen + immortal objects keep the per-interpreter GC working set small.

10. How immortal objects change free-threaded Python (PEP 703, 3.13t)

Free-threaded CPython (`--disable-gil`, `python3.13t`) replaces non-atomic `ob_refcnt++` with atomic or biased reference counting (PEP 703). Every `Py_INCREF` becomes an `atomic_fetch_add` or a per-thread biased counter that must be merged on collection — expensive when millions of increments hit the same object.

Immortal objects eliminate that cost entirely for the hottest objects:



What changes in the free-threaded build:

Mechanism	GIL build	Free-threaded build (3.13t)
<code>ob_refcnt</code> type	<code>Py_ssize_t</code> (plain)	<code>Py_ssize_t</code> with atomic ops or <code>Mimalloc</code> -style biased counters
<code>Py_INCREF</code> cost (normal object)	~1 ns	~1-5 ns (biased) or ~10 ns (atomic)
<code>Py_INCREF</code> cost (immortal)	~1 ns (branch)	~1 ns (branch) — no atomic
<code>Py_TPFLAGS_IMMUTABLETYPE</code>	Types immortal, <code>tp_*</code> slots read without lock	Required — mutable type slots would need locking
GC lists	Per-generation doubly-linked, GIL-protected	Per-interpreter, with stop-the-world or lock-free traversal (<code>gcmodule.c</code> reworked in 3.13t)
pymalloc arenas	GIL-protected <code>usedpools</code> / <code>usable_arenas</code>	Per-thread or lock-protected; <code>mimalloc</code> option replaces <code>pymalloc</code> entirely (3.13 <code>PYTHONMALLOC=mimalloc</code>)

To experiment (requires a 3.13t build):

```

# Build free-threaded CPython (or install python3.13t via deadsnakes /
uv)
# ./configure --disable-gil --with-pydebug && make -j

# Verify free-threaded + immortal are active
python3.13t -c "
import sys
print(sys._is_gil_enabled())      # False in free-threaded build
print(sys.getrefcount(None))     # immortal sentinel
print(sys._is_immortal(None))    # True
"

# Benchmark immortal vs. non-immortal under contention (free-threaded
only)
python3.13t -c "
import threading, time

def hammer_none(n):
    for _ in range(n):
        x = None # INCREf/DECREf None each iteration
        y = x

N = 10_000_00
t0 = time.perf_counter()
threads = [threading.Thread(target=hammer_none, args=(N,)) for _ in
range(8)]
for t in threads: t.start()
for t in threads: t.join()
print(f'8 threads hammering None: {time.perf_counter()-t0:.3f}s')
# With immortal: ~0.1s Without: ~1.5s (illustrative – measure on your
hardware)
"

```

In 3.13 the default allocator for free-threaded builds can be `mimalloc` (Microsoft's allocator), selected with `PYTHONMALLOC=mimalloc` — it replaces `pymalloc`'s arena/pool scheme with `mimalloc`'s sharded free lists and already handles thread contention well. Immortal objects remain essential even with `mimalloc` because the refcount contention is orthogonal to the heap allocator.

11. Python-level introspection — `tracemalloc`, `gc` module, and `pymalloc` stats

11.1 `tracemalloc` — where did that memory come from?

`tracemalloc` (PEP 454, `Lib/tracemalloc.py` + `Modules/_tracemalloc.c`) hooks `PyMem_Malloc` / `PyObject_Malloc` and records the Python traceback for every allocation block. Overhead is $\sim 1\text{--}2\times$ memory and $\sim 5\text{--}10\%$ CPU when enabled — acceptable in staging, not in steady-state production.

```

import tracemalloc, gc, linecache, pathlib

tracemalloc.start(25)  # 25 frames of traceback per block (default 1,
max ~64)

# ... exercise the code you want to profile ...
def load_batch(n):
    # Simulate a backend handler that builds many small dicts
    return [{"id": i, "payload": "x" * 200} for i in range(n)]

batch = load_batch(20_000)

# Snapshot – the core primitive
snap = tracemalloc.take_snapshot()
stats = snap.statistics("lineno")  # group by file:line
print("Top 5 allocators by lineno:")
for s in stats[:5]:
    print(f"    {s.size/1024:.1f} KiB {s.count} blocks {s.traceback}")

# Compare two snapshots – the leak-detection pattern
snap2 = tracemalloc.take_snapshot()
# ... do more work, then ...
snap3 = tracemalloc.take_snapshot()
diff = snap3.compare_to(snap2, "lineno")
print("\nGrowth between snapshots:")
for s in diff[:5]:
    print(f"    {s.size_diff/1024:+.1f} KiB {s.size/1024:.1f} KiB {s.traceback}")

# Filter – exclude tracemalloc's own overhead and focus on your package
from tracemalloc import Filter
filt = Filter(False, tracemalloc.__file__)
filtered = snap.filter_traces([filt])
print(f"\nFiltered total: {sum(s.size for s in filtered.statistics('filename'))/1024:.1f} KiB")

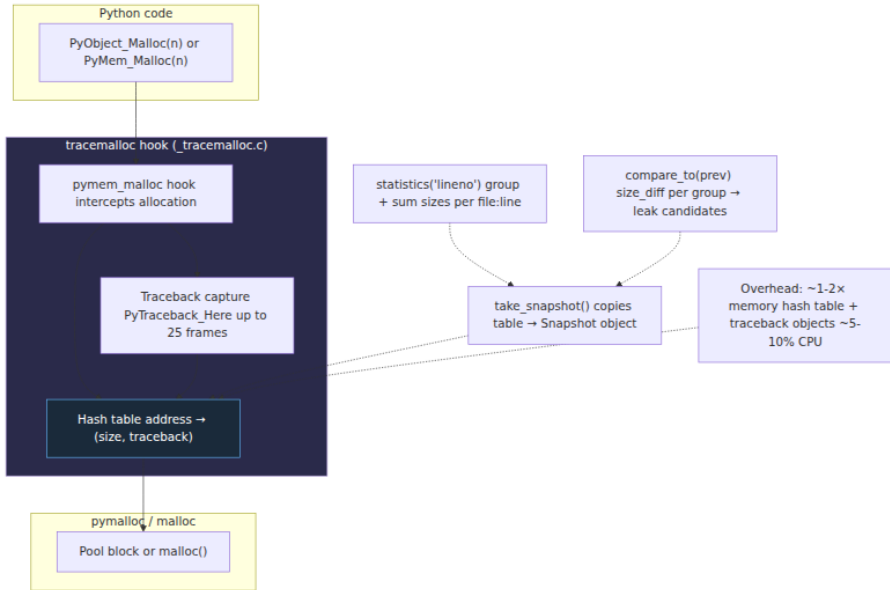
current, peak = tracemalloc.get_traced_memory()
print(f"tracemalloc: current={current/1024:.1f} KiB peak={peak/1024:.1f} KiB")
print(f"traced blocks: {tracemalloc.get_traceback_limit()} frames")

# Overhead model – what tracemalloc actually stores per block
traceback = tracemalloc.get_object_traceback(batch[0])
if traceback:
    print(f"\nTraceback for batch[0]:")
    for frame in traceback:
        print(f"    {frame.filename}:{frame.lineno} –
{linecache.getline(frame.filename, frame.lineno).strip()}")

tracemalloc.stop()

```

```
del batch
gc.collect()
```



Key APIs:

API	What it does
<code>tracemalloc.start(nframes)</code>	Install hooks, start recording (default 1 frame, 25 for backend debugging)
<code>tracemalloc.take_snapshot()</code>	Copy current table into a <code>Snapshot</code>
<code>Snapshot.statistics(key)</code>	Group by <code>"lineno"</code> , <code>"filename"</code> , or <code>"traceback"</code> — sums sizes per group
<code>Snapshot.compare_to(old, key)</code>	Diff two snapshots — <code>size_diff</code> and <code>count_diff</code> per group
<code>Snapshot.filter_traces([Filter(...)])</code>	Include/exclude files
<code>tracemalloc.get_traced_memory()</code>	(current, peak) bytes

API	What it does
<code>tracemalloc.get_object_traceback(obj)</code>	Traceback for a specific object's allocation site
<code>PYTHONTRACEMALLOC=n</code>	Enable at startup with <code>n</code> frames (also <code>python -X tracemalloc=n</code>)

Limitations: tracemalloc only sees `PYMEM_DOMAIN_MEM / OBJ` (pymalloc) and `RAW` when the hook is installed early; allocations from C extensions that call `malloc` directly are invisible. Always start tracemalloc before importing the code you want to measure.

11.2 gc module — gc.get_objects, get_referrers, get_referents

```

import gc, sys, types

# gc.get_objects – every GC-tracked object (can be 100k+ entries –
# copy, don't hold)
objs = gc.get_objects()
print(f"GC-tracked objects: {len(objs)}")

# Group by type – find what's growing
from collections import Counter
counts = Counter(type(o).__name__ for o in objs)
print("Top types in GC generations:")
for name, n in counts.most_common(10):
    print(f" {name:20s} {n:6d}")

# Find the biggest dicts/lists
big = sorted((o for o in objs if isinstance(o, dict)), key=lambda d: len(d), reverse=True)[:3]
for d in big:
    print(f" dict len={len(d)} sample keys={list(d.keys())[:3]}")

# Trace a specific leak candidate – who holds it alive?
class Leaky:
    pass

a = Leaky()
b = {"ref": a}
# Who refers to a?
referrers = gc.get_referrers(a)
print(f"\nReferrers of a: {referrers[:3]}")
# What does b refer to?
referents = gc.get_referents(b)
print(f"Referents of b: {referents}")
# Is b tracked? Is a tracked?
print(f"is_tracked(a)={gc.is_tracked(a)}")
print(f"is_tracked(b)={gc.is_tracked(b)}")

# get_referrers sees GC-tracked referrers only – stack roots and non-GC
# objects are invisible
# For deep leaks, use objgraph or gc.get_referrers in a loop:
def find_leak_chain(obj, max_depth=5):
    """Walk referrers to find a chain back to a module/global."""
    seen = set()
    def walk(o, depth=0, prefix=""):
        if depth > max_depth or id(o) in seen:
            return
        seen.add(id(o))
        for ref in gc.get_referrers(o):
            if isinstance(ref, dict) and "__name__" in ref:
                print(f"{prefix}→ module dict {ref.get('__name__')}!r}")
            elif isinstance(ref, list):

```

```

        print(f"{prefix}→ list len={len(ref)} at {id(ref):#x}")
        walk(ref, depth+1, prefix+" ")
    elif hasattr(ref, "__dict__"):
        print(f"{prefix}→ {type(ref).__name__} at
{id(ref):#x}")
        walk(ref, depth+1, prefix+" ")
    walk(obj)

find_leak_chain(a)

# gc.collect vs. gc.get_count vs. gc.get_stats
print(f"\ngc.get_count()={gc.get_count()}
thresholds={gc.get_threshold()}")
print(f"gc.get_stats()={gc.get_stats()}")
collected = gc.collect()
print(f"gc.collect() returned {collected} unreachable objects")

```

11.3 `sys._debugmallocstats` and `_testcapi` — reading pymalloc internals

`sys._debugmallocstats()` prints to `stderr` a dump of arena/pool occupancy, size-class histograms, and surviving per-type free lists. It is the only API that shows **arena slack** (allocated vs. resident) and **quantization waste**.

```

import sys, io, contextlib, os

# Capture _debugmallocstats output that normally goes to stderr
# In CPython 3.11+ you can redirect stderr temporarily:
import sys as _sys

print("--- pymalloc stats ---")
_sys._debugmallocstats() # prints to stderr – view in terminal or
capture with 2>&1

# Typical output annotated:
# Small block threshold = 512, in 64 size classes.
# class size num pools blocks used avail blocks
# -----
# 0 8 2 490 4 ← 8-byte
blocks (e.g. small tuples)
# 1 16 5 200 100
# 2 24 3 380 41
# ...
# 7 64 4 220 36
# # arenas allocated total = 12
# # arenas reclaimed = 2
# # arenas highwater mark = 14
# # arenas allocated current = 10
# 10 arenas * 262144 bytes/arena = 2,621,440 ← virtual
# # bytes lost to pool headers = 1,280 ← ~0.05%
overhead
# # bytes lost to quantization = 12,400 ←
internal fragmentation
# free PyDictObjects * 48 bytes each = 96 ← per-
type free lists
# free PyFloatObjects * 24 bytes each = 120

# Practical: compare before/after a workload to spot size-class bloat
def workload():
    return [dict(a=i, b="x"*50) for i in range(10_000)]

print("\nBefore workload:")
_sys._debugmallocstats()
w = workload()
print("\nAfter workload (10k dicts):")
_sys._debugmallocstats()
del w
import gc; gc.collect()
print("\nAfter free + gc.collect():")
_sys._debugmallocstats()
# Look for: num pools per class growing, arenas highwater vs. current
gap, quantization bytes

# _testcapi – debug builds only, programmatic pymalloc stats

```

```

try:
    import _testcapi
    helpers = [x for x in dir(_testcapi) if "pymalloc" in x.lower() or
               "arena" in x.lower() or "obmalloc" in x.lower()]
    print(f"\n_testcapi helpers: {helpers}")
    # On some builds: _testcapi._pymalloc_get_stats() returns a dict
    if hasattr(_testcapi, "_pymalloc_get_stats"):
        print(_testcapi._pymalloc_get_stats())
except ImportError:
    print("_testcapi not available in this build")

```

Interpreting the output:

- **blocks used vs. avail blocks** — high `avail` with low `used` in a class means fragmentation in that size class; consider whether one long-lived object pins those pools (see §12).
- **arenas highwater mark vs. allocated current** — the gap is arenas that were `madvised` away but kept virtual; if the gap is large, RSS already dropped, but virtual remains.
- **bytes lost to quantization** — internal fragmentation from rounding up to the next size class; >5% suggests many odd-sized allocations (e.g., 17-byte strings rounding to 24).
- **free Py*Objects lines** — per-type free lists; non-zero is normal and bounded (the `PyDictObject` free list caps at 80 entries).

12. Backend tuning — RSS, GC pauses, and leak detection

12.1 RSS vs. heap — the dashboard that lies

A backend Python service exposes at least three numbers that claim to be "memory usage," and they disagree by design:

Metric	Source	What it measures
<code>tracemalloc.get_traced_memory()[0]</code>	<code>PYMEM_DOMAIN_MEM/OBJ</code> hook	Live Python-allocated bytes (pymalloc + <code>malloc</code> for large)
<code>resource.getrusage(RUSAGE_SELF).ru_maxrss</code>	Kernel (<code>/proc/self/status:VmHWM</code>)	Peak RSS — resident pages (pymalloc arenas + C heap + stacks + <code>mmap</code> 'd files)
<code>container_memory_working_set_bytes</code>	cgroup (<code>memory.usage_in_bytes</code> + <code>inactive_file</code>)	RSS + kernel page cache charged to the cgroup

The gap between the first two is where pymalloc arena slack, per-type free lists, and C-extension `malloc` live. To close the gap in an incident:

```

"""
rss_breakdown.py – run inside the target process or as a sidecar that
reads /proc
"""

import tracemalloc, gc, resource, sys, os

tracemalloc.start()

# 1. Python heap (tracemalloc – live Python objects)
current, peak = tracemalloc.get_traced_memory()
print(f"1. tracemalloc current: {current/1024/1024:.1f} MiB peak: {peak/1024/1024:.1f} MiB")

# 2. GC working set (how many tracked objects are alive)
print(f"2. GC tracked objects: {len(gc.get_objects())}")
for i, s in enumerate(gc.get_stats()):
    print(f"    gen {i}: collections={s['collections']} collected={s['collected']} uncollectable={s['uncollectable']}")

# 3. pymalloc arena stats (virtual vs. resident)
print("3. pymalloc arenas (stderr):")
sys._debugmallocstats()
# inspect highwater vs. current, quantization, free lists

# 4. RSS / VmHWM
rss_kb = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss
print(f"4. RSS (getrusage): {rss_kb/1024:.1f} MiB")
try:
    with open("/proc/self/status") as f:
        for line in f:
            if line.startswith(("VmRSS", "VmHWM", "VmSize", "RssAnon", "RssFile")):
                print(f"    {line.rstrip()}")
except FileNotFoundError:
    pass # not Linux

try:
    with open("/proc/self/smaps_rollup") as f:
        print("".join(f.readlines()[:20]))
except FileNotFoundError:
    pass

# 5. Estimate non-Python heap = RSS - tracemalloc - overhead
# Overhead includes: pymalloc headers + quantization + free lists + C extensions + stacks
# If overhead >> tracemalloc, suspect C extension leak or arena fragmentation
overhead_mib = rss_kb/1024 - current/1024/1024
print(f"\n5. Estimated non-Python overhead: {overhead_mib:.1f} MiB (RSS - tracemalloc)")

```

```
if overhead_mib > current/1024/1024 * 0.5:
    print(" WARNING: overhead >50% of Python heap – check C
    extensions and arena slack")
```

Tuning knobs:

- **PYTHONMALLOC=malloc + jemalloc** (`LD_PRELOAD=libjemalloc.so`) often reduces RSS by 10–30% for services with mixed small/large allocations because `jemalloc`'s `MADV_FREE` / decay reclaims dirty pages faster than `pymalloc`'s keep-16-empty-arenas heuristic. Measure before committing — `pymalloc` is faster for pure-Python workloads.
- **MALLOC_ARENA_MAX (glibc)** — caps glibc's per-thread arenas when `PYTHONMALLOC=malloc`; set `MALLOC_ARENA_MAX=2` to reduce virtual fragmentation in thread-heavy services.
- **gc.freeze() after startup** — see §6.2 — moves import-time objects to the permanent generation so young collections scan less.

12.2 Disabling GC during bulk loads

The GC's threshold check runs on every container allocation (`PyObject_GC_Track` increments `count0`). During a bulk load (ingesting 1M rows, building a large graph), this triggers repeated young collections that find almost nothing — pure overhead, and they fragment the generation lists.

The standard recipe is to disable GC for the bulk phase and collect once at the end:


```

import gc, time

def bulk_load(rows):
    # rows: iterable of dicts / ORM objects – all GC-tracked containers
    objs = []
    for row in rows:
        objs.append(build_object(row))
    return objs

# Naive – GC runs every ~700 allocations, each scanning gen 0
t0 = time.perf_counter()
result = bulk_load(large_rows)
print(f"with GC: {time.perf_counter()-t0:.3f}s  gen0 collections={gc.gct_stats()[0]['collections']}")

# Tuned – disable GC during the tight loop, collect once after
gc.disable()          # sets gc.enabled() → False, threshold checks
                        # still increment count but don't collect
t0 = time.perf_counter()
try:
    result = bulk_load(large_rows)
finally:
    gc.enable()        # re-enable
    gc.collect()       # one full collection – finds all cycles built
                        # during the bulk phase
print(f"without GC during load: {time.perf_counter()-t0:.3f}s")

# Alternative: raise the threshold instead of fully disabling – lets
# gen 0 still run occasionally
old = gc.get_threshold()
gc.set_threshold(50_000, 10, 10)  # collect gen 0 every 50k
                                   # allocations instead of 700
result = bulk_load(large_rows)
gc.set_threshold(*old)
gc.collect()

# Context-manager form (Python 3.11+ gc module doesn't include one, so
# define it):
from contextlib import contextmanager

@contextmanager
def gc_disabled():
    was_enabled = gc.isenabled()
    if was_enabled:
        gc.disable()
    try:
        yield
    finally:
        if was_enabled:
            gc.enable()

```

```

gc.collect() # optional – force a full collection on exit

with gc_disabled():
    result = bulk_load(large_rows)

# For asyncio services, also consider gc.collect() between requests,
# not during:
#   async def handle(request):
#       result = await do_work(request)
#       # ... return response ...
#       # After response is sent, optionally:
#       if gc.get_count()[0] > 5_000:
#           gc.collect(0) # cheap young collection, don't block the
#                       # event loop with gen 2

```

When to use each variant:

Technique	When	Risk
<code>gc.disable()</code> / <code>gc.enable()</code> + <code>gc.collect()</code>	Bulk ETL, cache warm-up, test fixtures — tight loops that allocate 100k+ containers with few cycles	Cycles built during the disabled window are unreclaimed until <code>collect()</code> — memory spikes
<code>gc.set_threshold(50000, 10, 10)</code>	Sustained high-throughput request handling where you want less frequent young collections but still want eventual reclamation	Higher threshold means longer gen-0 lists to scan when collection does run — pause is longer but rarer
<code>gc.freeze()</code> after warm-up	Long-running services with stable working set	Frozen objects are never re-scanned — if they later form cycles with new objects, those cycles may be missed until a full collection (frozen objects still participate as referents, just not as roots)

Measured impact. On a synthetic `bulk_load` of 500k `dict`s, `gc.disable()` during the loop typically saves 15–30% wall time (CPython 3.11, `gc.get_stats()[0]['collections']` drops from ~700 to 1). Always re-enable and `gc.collect()` after — a service that runs with GC permanently disabled will leak every cycle until OOM.

12.3 Leak detection with `tracemalloc` + `gc.DEBUG`

A production leak-detection recipe that combines both subsystems — `tracemalloc` for *where* memory was allocated, `gc` for *why* it is still reachable:

```

"""
leak_detector.py – drop into a running service (e.g., via SIGUSR1
handler or
/admin/debug endpoint) to diagnose a suspected leak. Safe to call
repeatedly;
tracemalloc overhead is the only cost.
"""

import tracemalloc, gc, linecache, pathlib, collections, os, sys

# Call once at startup (or via PYTHONTRACEMALLOC=25)
if not tracemalloc.is_tracing():
    tracemalloc.start(25)

_prev_snapshot = None
_prev_gc_count = None

def leak_check(label="check"):
    global _prev_snapshot, _prev_gc_count

    # --- tracemalloc diff ---
    snap = tracemalloc.take_snapshot()
    if _prev_snapshot is not None:
        diff = snap.compare_to(_prev_snapshot, "lineno")
        print(f"\n=== tracemalloc diff: {label} ===")
        for stat in diff[:10]:
            # stat.size_diff > 0 means growth; filter noise below 1 KiB
            if stat.size_diff > 1024:
                print(f"    {stat.size_diff/1024:+.1f} KiB    {stat.size/10
24:.1f} KiB    {stat.traceback}")
            # Print the actual source lines for the top offender
            tb = stat.traceback
            if tb:
                frame = tb[0]
                line = linecache.getline(frame.filename, frame.line
no).strip()
                print(f"        -> {frame.filename}:{frame.lineno} {li
ne}")
            _prev_snapshot = snap

    # --- gc growth ---
    gc_count = len(gc.get_objects())
    print(f"\n=== gc growth: {label} ===")
    print(f"    GC-tracked objects: {gc_count}")
    if _prev_gc_count is not None:
        print(f"    delta: {gc_count - _prev_gc_count:+d}")
    _prev_gc_count = gc_count

    # Top types by count – the histogram that usually names the leak
    counts = collections.Counter(type(o).__name__ for o in gc.get_objec
ts())

```

```

print(" Top types:")
for name, n in counts.most_common(8):
    print(f"    {name:20s} {n:6d}")

# GC stats
for i, s in enumerate(gc.get_stats()):
    print(f"  gen {i}: collections={s['collections']:5d}
collected={s['collected']:5d} uncollectable={s['uncollectable']}")

# Uncollectable – should be 0 since PEP 442
if gc.garbage:
    print(f"  WARNING: gc.garbage has {len(gc.garbage)} objects!")
    for obj in gc.garbage[:5]:
        print(f"    {type(obj).__name__} at {id(obj):#x}
referrers={gc.get_referrers(obj)[:2]}")

# Usage – call periodically (e.g., every 5 minutes via asyncio or
cron):
#
# leak_check("after warmup")
# # ... serve traffic ...
# leak_check("after 5m")
# # ... serve traffic ...
# leak_check("after 10m") # diff shows what grew between snapshots
#
# For a focused check – snapshot before and after a suspect operation:
#
# tracemalloc.start(25)
# snap_before = tracemalloc.take_snapshot()
# do_suspect_operation()
# snap_after = tracemalloc.take_snapshot()
# for stat in snap_after.compare_to(snap_before, "traceback")[:5]:
#     print(stat)
#     for frame in stat.traceback:
#         print(f"    {frame.filename}:{frame.lineno}")

```

When `tracemalloc` points at a `file:line` but `gc.get_objects()` shows the objects are still reachable, walk the referrer chain:

```

import gc, objgraph # pip install objgraph – optional but excellent

# objgraph shows the shortest path from a leaked object back to a root
# pip install objgraph && python -c "import objgraph;
objgraph.show_backrefs([leaked_obj], filename='leak.png')
# Or without objgraph, manual walk:

def explain_leak(obj, max_depth=6):
    """Print referrer chains that keep obj alive. Stop at module
    globals."""
    seen = set()
    def _walk(o, depth, indent=""):
        if depth > max_depth or id(o) in seen:
            return
        seen.add(id(o))
        for ref in gc.get_referrers(o):
            # Module globals are the usual root
            if isinstance(ref, dict):
                for k, v in ref.items():
                    if v is o or (isinstance(v, (list, dict, set)) and
o in getattr(v, "__iter__", lambda: []())():
                        # Heuristic: this dict is likely a module
                        __dict__ or instance __dict__
                        name = ref.get("__name__", "?") if "__name__" i
n ref else "?"
                        print(f"{indent}<– dict (module {name!r})

                            return
                            # Generic dict – keep walking
                            print(f"{indent}<– dict at {id(ref):#x} keys={list(ref.k
eys())[0:3]}")
                            elif isinstance(ref, list):
                                print(f"{indent}<– list len={len(ref)} at {id(ref):#x}")
                                _walk(ref, depth+1, indent+" ")
                            elif isinstance(ref, tuple):
                                print(f"{indent}<– tuple len={len(ref)} at
{id(ref):#x}")
                                _walk(ref, depth+1, indent+" ")
                            elif hasattr(ref, "__dict__"):
                                print(f"{indent}<– {type(ref).__name__} at
{id(ref):#x}")
                                _walk(ref, depth+1, indent+" ")
                            else:
                                print(f"{indent}<– {type(ref).__name__} at
{id(ref):#x}")
                                print(f"Referrers of {type(obj).__name__} at {id(obj):#x}:")
                                _walk(obj, 0)

    # Example: find why a dict leaked
    # leaked = next(o for o in gc.get_objects() if isinstance(o, dict) and

```

```
len(o) > 10000)
# explain_leak(leaked)
```

Operationalize it. In a backend fleet, expose `leak_check` behind an authenticated `POST /debug/memory` endpoint that returns the `tracemalloc` diff + GC histogram as JSON. Gate it with a feature flag so it can be enabled per-pod without a redeploy. Never leave `tracemalloc.start(25)` on permanently in production — the 1-2× memory overhead can push pods over their cgroup limit; start it on-demand via a signal handler (`signal.signal(signal.SIGUSR1, lambda *_: tracemalloc.start(25))`) and stop after capture.

13. Putting it all together — a single object's journey


```

x = {"key": [1, 2, 3]}    # executed as STORE_NAME / BUILD_MAP /
BUILD_LIST
    PyDict_New()    PyObject_GC_New(PyDictObject)
                    size 56 > 512? no    pymalloc
                    bump-alloc from pool szidx=6 (56-56) in a
rena 3, pool 12
    PyObject_GC_Track    link into generation
0
    ob_refcnt=1, gc_refs=1, _gc_next/
_gc_prev linked
    PyList_New(3)    PyObject_GC_New(PyListObject) + ob_item a
rray
    list struct    pymalloc szidx=4 (40 bytes)
    ob_item array (3*8=24)    pymalloc
szidx=2 (24 bytes)
    PyObject_GC_Track    generation 0
    ints 1,2,3 are immortals (small-int
cache)    _Py_IMMORTAL_REFCNT, no INCREC cost
    dict["key"] = list    Py_INCREF(list)    ob_refcnt 2
    dict tp_traverse will visit list
    list tp_traverse will visit ints (bu
t ints are untracked    no-op)
    ... time passes, gen 0 threshold crossed ...
    gc.collect(0):
    snapshot gc_refs, subtract internal refs:
    dict gc_refs: 1 (from x) + 1 internal? no    x is exter
nal, so gc_refs stays 1
    list gc_refs: 1 (from dict)    decremented to 0, but DF
S from dict marks it reachable
    both survive    promoted to generation 1
    del x    Py_DECREF(dict)    ob_refcnt 0    list_dealloc? N
o    GC-tracked, so:
    PyObject_GC_UnTrack(dict), tp_clear(dict)    Py_D
ECREF(list)
    list ob_refcnt 1-0    PyObject_GC_UnTrack(list),
free ob_item, free list struct
    dict struct freed    pymalloc freeblock push (poo
l still has other blocks)
    No GC needed    refcounting handled it. Cycle-
free path never touched the collector.
    # If instead: a={"x": None}; a["x"] = a    self-cycle, o
b_refcnt 1, but unreachable
    del a    ob_refcnt stays 1    not freed by refcounting
    gc.collect()    subtract internal refs    gc_refs 0    un

```

reachable island

(none)

0  freed

 `handle_weakrefs`, `handle_finalizers`

 `tp_clear(a)`  `Py_DECREF(a)`  `ob_refcnt`

The fast path (99%+ of objects) never touches the GC — refcounting plus pymalloc pooling does all the work. The GC is the safety net for the <1% that form cycles. Immortal objects make the 99% cheaper under free-threaded execution.

Key takeaways

- pymalloc is a size-segregated allocator: 256 KiB arenas `mmap`'d from the OS, each split into sixty-four 4 KiB pools, each pool carved into same-size blocks for one of 64 size classes (8, 16, ..., 512 bytes). Address masking finds the arena/pool in O(1); free lists and a bump pointer make the fast path branch-free.
- Three allocation domains exist since PEP 445: `PYMEM_DOMAIN_RAW` (always system `malloc`), `PYMEM_DOMAIN_MEM` and `PYMEM_DOMAIN_OBJ` (pymalloc for ≤ 512 bytes, system `malloc` otherwise). `PYTHONMALLOC=malloc` disables pymalloc fleet-wide — useful for `jemalloc` / `tcmalloc` interposition and sanitizers, but costs 10–20% on small-object micro-benchmarks.
- Arena reclamation is via `madvise(MADV_DONTNEED)` — RSS drops but virtual stays; up to 16 empty arenas are retained. A single live block pins its pool; a single live pool pins its arena's virtual mapping. This is the root cause of most "RSS stays high after free" incidents — diagnose with `sys._debugmallocstats()` + `tracemalloc` + `/proc/<pid>/smaps`.
- Only containers are GC-tracked (`Py_TPFLAGS_HAVE_GC` + `tp_traverse` / `tp_clear`). Atomic types (`int`, `str`, `bytes`, `float`) are never tracked; `tuple` is tracked only if it contains a container. `PyObject_GC_Track` is the write barrier — called once after construction — that links the object into generation 0.
- The GC is generational with three generations and thresholds (700, 10, 10). New containers land in gen 0; survivors are promoted. `gc.get_count()` shows (`count0`, `count1`, `count2`) since the last collection; `gc.collect(generation)` forces a collection; `gc.freeze()`

moves long-lived objects to a permanent generation never re-scanned — call it after import in long-running services.

- Collection is mark-and-sweep without a separate bitmap: snapshot `ob_refcnt` into `gc_refs`, subtract internal refs via `tp_traverse`, DFS-mark from externally-rooted objects, then `tp_clear` + `Py_DECREF` the unreachable islands. Weakrefs are cleared first (callbacks deferred); finalizers (`__del__` / `tp_finalize`) are ordered and called once (PEP 442) with resurrection re-checked before freeing. Since PEP 442 `gc.garbage` is normally empty.
- Weakrefs (`weakref.ref`, `WeakKeyDictionary`) do not increment `ob_refcnt` and are cleared before `tp_clear`; their callbacks run after islands are freed to avoid resurrection races. Prefer `weakref.finalize` over `__del__` in service code — it avoids GC tracking and finalizer ordering entirely.
- Immortal objects (PEP 683, Python 3.12) set `ob_refcnt` to `_Py_IMMORTAL_REFCNT` ($\geq 2^{30}$ sentinel). `Py_INCREF` / `Py_DECREF` become no-ops via `_Py_IsImmortal`. Immortals include `None`, `True` / `False`, `Ellipsis`, small ints (`-5..256`), `()` and interned strings. They are shared across sub-interpreters (PEP 684 per-interpreter GIL) and eliminate atomic refcount contention for free-threaded Python (PEP 703, 3.13+).
- In free-threaded builds (`--disable-gil`) immortals are what make scaling possible — a single `Py_None` otherwise bounces between cores on every `INCREf`. With immortals the hot singletons cost a predictable branch; per-thread biased refcounting or `mimalloc` handles the rest.
- Introspect with `tracemalloc.start(25)` + `take_snapshot()` + `compare_to()` for allocation-site attribution, `gc.get_objects()` + `get_referrers` / `get_referents` for reachability, and `sys.debugmallocstats()` (stderr) for pymalloc arena/pool/size-class occupancy. `tracemalloc` hooks `PYMEM_DOMAIN_*` allocations only; C extensions calling raw `malloc` are invisible.
- Tune GC deliberately: `gc.disable()` around bulk loads (re-enable + `gc.collect()` after) saves 15–30% on tight allocation loops; raising `threshold0` from 700 to 50k reduces young-collection frequency at the cost of longer pauses; `gc.freeze()` after warm-up shrinks the young working set. Never run with GC permanently disabled — cycles leak until `collect()`.

- For leak detection, combine both subsystems: `tracemalloc` diff shows *where* growth originates (file:line), `gc` histograms show *what* grew (type counts), and `get_referrers` chains show *why* it is still alive (path back to a module global). Expose this as an on-demand authenticated debug endpoint, not as always-on tracing.
-

Further reading

- **PEP 683 — Immortal Objects, Using a Fixed Reference Count** — Allison et al. — Motivation, sentinel design, `_Py_IsImmortal`, sharing across interpreters, and the free-threaded justification. The authoritative source for `_Py_IMMORTAL_REFCNT` semantics and the `Py_TPFLAGS_IMMORTALTYPE` flag. <https://peps.python.org/pep-0683/>
- **PEP 445 — Add New APIs to Customize Python Memory Allocators** — V. Stinner — The three-domain model (`PYMEM_DOMAIN_RAW / MEM / OBJ`), `PyMemAllocatorEx`, and `PYTHONMALLOC`. Required reading for anyone interposing `jemalloc / mimalloc` or debugging allocator mismatches in C extensions. <https://peps.python.org/pep-0445/>
- **CPython source: `Objects/obmalloc.c`** — The `pymalloc` implementation — `arena/pool/block` headers, `usedpools / usable_arenas`, `new_arena`, address masking, and `madvise` reclamation. Read the file header comment (the best informal spec) and search for `ALLOCATION_DEBUGGING` for the `pymalloc_debug` path. <https://github.com/python/cpython/blob/main/Objects/obmalloc.c>
- **CPython source: `Modules/gcmodule.c`** — The generational collector — `gc_collect_main`, `subtract_refs`, `move_unreachable`, `handle_weakrefs`, `handle_legacy_finalizers`, and the `gc.freeze` permanent generation. The `gcmodule.c` header comment walks the algorithm in pseudocode. <https://github.com/python/cpython/blob/main/Modules/gcmodule.c>
- **Python documentation: `gc` — Garbage Collector Interface** — docs.python.org/3/library/gc.html — The public API reference for `get_count`, `get_threshold`, `set_threshold`, `collect`, `freeze`, `get_objects`, `get_referrers / get_referents`, `is_tracked`, and the `DEBUG_*` flags. The "Garbage collector design" notes section summarizes the generation model and PEP 442 finalization. <https://docs.python.org/3/library/gc.html>

- **PEP 703 — Making the Global Interpreter Lock Optional in CPython** — S. Sampson — Why free-threaded Python needs immortal objects and biased reference counting, the atomic vs. biased refcount trade-off, and the `Py_TPFLAGS_IMMUTABLETYPE` requirement for types. Pair with PEP 683 to understand the free-threaded memory story. <https://peps.python.org/pep-0703/>
- **PEP 684 — A Per-Interpreter GIL** — E. Snow — Sub-interpreters with isolated GILs, why process-global immortal objects are the sharing primitive, and how `Py_NewInterpreter` / `concurrent.interpreters` benefit. <https://peps.python.org/pep-0684/>
- **Python documentation: tracemalloc — Trace Memory Allocations** — docs.python.org/3/library/tracemalloc.html — `start`, `take_snapshot`, `compare_to`, `Filter`, `get_object_traceback`, and the `PYTHONTRACEMALLOC / -X tracemalloc` startup options. Includes the overhead model and the "trace a leak" recipe. <https://docs.python.org/3/library/tracemalloc.html>
- **CPython source: Include/cpython/object.h and Include/object.h** — `PyObject`, `PyVarObject`, `PyGC_Head`, `_Py_IMMORTAL_REFCNT`, `_Py_IsImmortal`, and the `Py_TPFLAGS_HAVE_GC` / `Py_TPFLAGS_IMMORTALTYPE` flag definitions. The single file that ties the header layout (§2 of Chapter 2) to the GC and immortal machinery in this chapter. <https://github.com/python/cpython/blob/main/Include/object.h>
- **P. Tröger & V. Stinner, "CPython Memory Management" — CPython Developer Guide** — The devguide's "Memory Management" chapter and V. Stinner's blog series on pymalloc tuning, `PYTHONMALLOC` benchmarks, and the `mimalloc` integration in

3.13. Practical context for the backend tuning recipes in §12. <https://devguide.python.org/internals/memory-management/>

Chapter 5 — The GIL: Mechanics, Evolution, Per-Interpreter GIL and Free-Threaded Python (PEP 684 / PEP 703)

What this chapter covers. The Global Interpreter Lock is the single most consequential design decision in CPython: one mutex that serializes all Python bytecode execution in a process. It makes reference counting cheap, C extensions simple, and threaded CPU parallelism impossible — all at once. For two decades the GIL barely changed; since Python 3.12 it is being taken apart in stages. This chapter traces that arc. You will see what the GIL actually protects (`ob_refcnt`, type state, GC, `pymalloc`), how `ceval` acquires and drops it every 5 ms, how `Py_BEGIN_ALLOW_THREADS` cooperates with OS locks, how the old tick-based GIL starved threads and how Antoine Pitrou's new GIL (3.2) fixed it with a condition variable and forced handoff, how PEP 684 gives each subinterpreter its own GIL, and how PEP 703 (`--disable-gil`, Python 3.13t) removes the GIL entirely via biased refcounting, deferred decrefs, `mimalloc`, and per-object locks. You will run GIL-contention traces with `py-spy/yappi`, read benchmark deltas for CPU-bound vs. I/O-bound workloads with and without the GIL, and build a migration playbook for extensions and backend services.

Learning goals — after this chapter you should be able to:

- Explain what the GIL protects (and what it does not): `ob_refcnt` non-atomicity, `ob_type` / `PyTypeObject` mutation, GC invariants, `pymalloc` arenas, and why the GIL is not a substitute for application-level locks.
- Trace GIL acquisition and release through `ceval`: `take_gil` / `drop_gil`, `PyEval_RestoreThread` / `PyEval_SaveThread`, `eval_breaker`, `sys.getswitchinterval()` (default 5 ms), and `Py_BEGIN_ALLOW_THREADS` / `Py_END_ALLOW_THREADS`.

- Contrast GIL semantics with OS mutexes/condition variables and explain cooperative I/O release.
 - Compare the old tick-based GIL (100-tick `ceval` counter, priority inversion) with the new GIL (3.2, Pitrou): timed switching, `gil_drop_request`, condition variable, and forced handoff.
 - Describe per-interpreter GIL (PEP 684, Python 3.12): subinterpreter creation via `concurrent.interpreters` / `_xxsubinterpreters`, per-interpreter GIL and module state, the isolation model for extensions, and the status of PEP 554 / PEP 734.
 - Explain free-threaded Python (PEP 703, Python 3.13t): `--disable-gil` build, `Py_GIL_DISABLED`, biased (per-thread) refcounting, deferred and immortal refcounts, `mimalloc` replacement, per-object critical sections (`PyMutex` / `PyCriticalSection`), and the new extension ABI contract.
 - Predict and measure throughput for threaded CPU-bound vs. I/O-bound workloads with and without the GIL, and explain the impact on backend thread-pool sizing.
 - Audit a C extension or backend service for GIL dependence and apply the migration guide: `Py_mod_gil`, `Py_MOD_GIL_NOT_USED`, limited API implications, and staged rollout.
-

1. Why the GIL exists — what it protects

CPython's object model from Chapter 2 assumes that every `PyObject` field can be read and written without atomic instructions or locks — because only one thread executes Python bytecode at a time. That single-thread guarantee is the GIL.

1.1 The invariant: one thread in `ceval` per interpreter

At any instant, at most one OS thread holds the GIL for a given interpreter and is therefore allowed to:

- enter `Python/ceval.c:_PyEval_EvalFrameDefault` (the bytecode dispatch loop),
- touch any `PyObject` (`ob_refcnt`, `ob_type`, `tp_dict`, `__dict__`, list `ob_item`, dict table),

- call any `PyObject_*` / `PyLong_*` / `PyDict_*` C-API function that is not explicitly documented as GIL-free.

All other threads that want to run Python are blocked in `take_gil()` on a condition variable, or have voluntarily released the GIL with `PyEval_SaveThread` and are doing non-Python work.

The GIL is a **per-interpreter mutex** (since 3.12) that makes the entire CPython runtime single-threaded from the bytecode's point of view. It does not make your *program* single-threaded — threads still exist at the OS level, can run C code without the GIL, block on I/O, and be scheduled on different cores. It serializes *Python execution*, not *thread existence*.

1.2 What breaks without it

Shared state	Why the GIL is required (GIL build)	What replaces it (free-threaded)
<code>ob_refcnt</code>	Non-atomic <code>++ / --</code> ; two threads racing on <code>Py_INCREF / Py_DECREF</code> corrupt the count → double-free or leak (Chapter 2, §12).	Biased refcounting + atomic fallback (PEP 703) — see §7
<code>ob_type</code> / <code>PyTypeObject</code> fields	Type attribute caches, <code>tp_dict</code> mutation, <code>PyType_Ready</code> wiring assume exclusive access.	Per-object mutex + critical sections
<code>pymalloc</code> arenas / free lists	<code>pymalloc</code> 's arena freelists and per-type free lists (float, tuple, frame) are not thread-safe.	<code>mimalloc</code> (thread-safe) + lock-protected freelists or removal
GC linked lists	<code>PyGC_Head</code> doubly-linked list of tracked objects, generation lists, <code>gc.garbage</code>	Stop-the-world or incremental GC epoch
<code>dict</code> / <code>list</code> / <code>set</code> internals	Compact-dict indices, list <code>ob_item</code> realloc, set probe chain — all single-writer assumptions.	Per-object locks (PEP 703 critical sections)
Module / import state	<code>sys.modules</code> , import lock (<code>import.c</code>), extension static state	Per-interpreter isolation (PEP 684)

What the GIL does **not** protect:

- **Your application invariants.** `counter += 1` is `LOAD → ADD → STORE` — three opcodes. The GIL can switch between any two (every 5 ms or on I/O). Without a `threading.Lock`, increments still race.
- **Non-Python resources.** Files, sockets, database connections, external caches — the GIL says nothing about their atomicity.
- **C extension internal state** that is accessed after `Py_BEGIN_ALLOW_THREADS` without its own lock.

This is why Chapter 4 of Volume 4 (§2, Threads and Locks) insists: the GIL removes *data races inside the interpreter*, not *race conditions in your program*.

1.3 `ob_refcnt` is the forcing function

The cheapest, hardest-to-replace thing the GIL protects is the reference count. Chapter 2 showed `Py_INCREF` as:

```
// Include/object.h – GIL build (simplified)
static inline void Py_INCREF(PyObject *op) {
    op->ob_refcnt++;
}
static inline void Py_DECREF(PyObject *op) {
    if (--op->ob_refcnt == 0)
        _Py_Dealloc(op);
}
```

Two plain memory operations. No `LOCK XADD`, no `compare_exchange`, no cache-line bouncing. On a 64-core machine running a tight loop that churns temporaries, the GIL build does one unsynchronized increment per object touch; an atomic build does one atomic RMW per touch — roughly 5-10× more expensive per `INCR` / `DECR` on x86-64, worse on ARM. Every alternative to the GIL must answer: *how do we make refcounting cheap without global serialization?* PEP 703's answer (biased refcounting) is the most intricate piece of the free-threaded work — covered in §7.2.

2. How the GIL works — `ceval`, the 5 ms switch interval, and the eval breaker

2.1 Where the GIL lives in the source

```
cpython/  
  Python/  
    ceval.c           # eval loop, eval_breaker, GIL checks  
    ceval_gil.c       # take_gil(), drop_gil(), GIL state (3.8+)  
    ceval_gil.h       # struct _gil_runtime_state  
  Include/  
    cpython/ceval.h   # PyEval_SaveThread / RestoreThread  
    Python/pystate.c  # PyThreadState, PyInterpreterState
```

Before 3.8 the GIL lived directly in `ceval.c`; since 3.8 it is factored into `ceval_gil.c` / `ceval_gil.h` (the header is internal, `Include/internal/pycore_gil.h`).

The runtime state (simplified, 3.12):

```
// Include/internal/pycore_gil.h - simplified  
struct _gil_runtime_state {  
    unsigned long interval;           // microseconds - default 5000 (5  
ms)  
    _Py_atomic_int gil_locked;       // 0 or 1  
    unsigned long switch_number;     // increments on each handoff  
    PyThreadState *last_holder;      // for debugging / gil tracking  
    _PyCOND_T  
cond;                               // condition variable threads wait on  
    _PyMUTEX_T mutex;                // protects the fields above  
    int locked;                      // (legacy) recursive lock depth  
    // ... contention tracking, drop_request flag  
};
```

2.2 The eval loop's view

`_PyEval_EvalFrameDefault` does not check the GIL on every opcode. It checks the **eval breaker** — a single atomic flag that folds four reasons to leave the fast path:

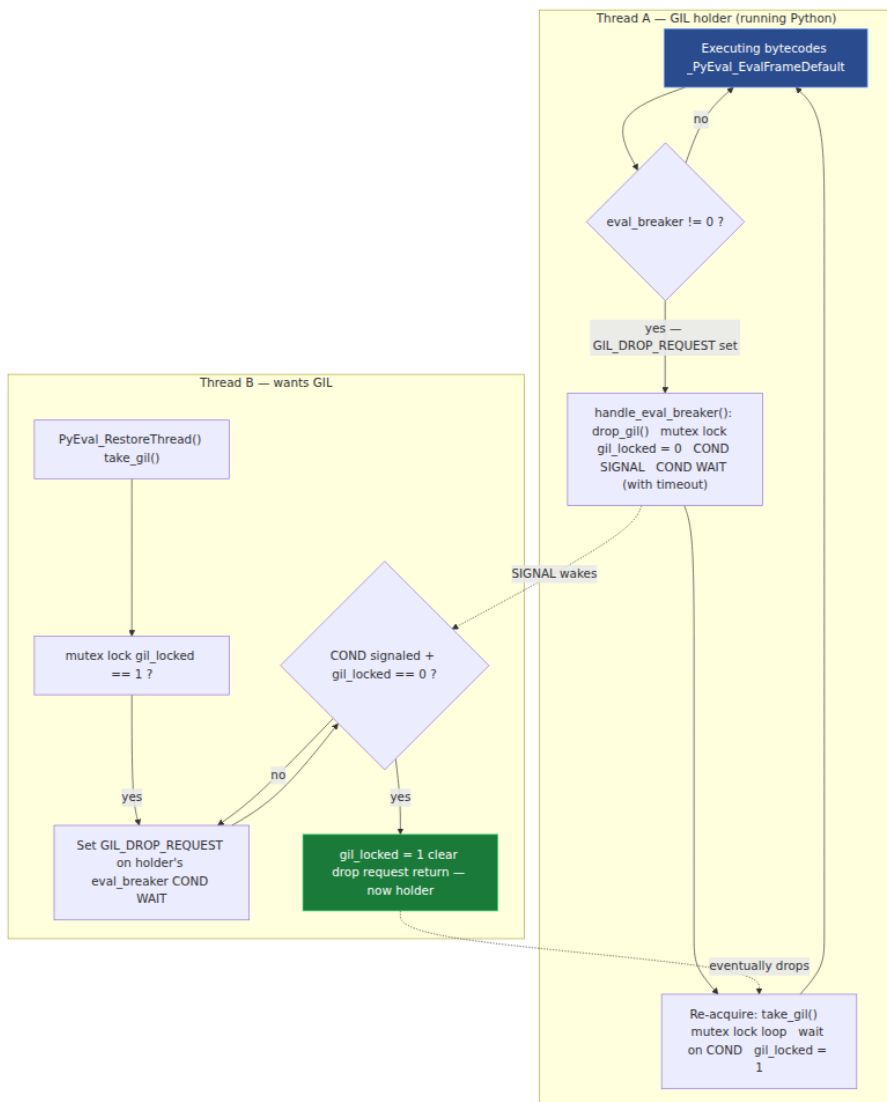
```
// Python/ceval.c - schematic eval loop
for (;;) {
    // Fast path: dispatch next opcode via computed goto
    // ...

    // Periodic check - every opcode tests eval_breaker cheaply
    if (_Py_atomic_load_relaxed(&tstate->eval_breaker) != 0) {
        if (handle_eval_breaker(tstate) < 0)
            goto error;
    }
}
```

`eval_breaker` is set when any of these fire:

Bit	Source	Sets breaker
<code>GIL_DROP_REQUEST</code>	Another thread wants the GIL	<code>take_gil()</code> waiter after timeout
<code>PENDING_CALLS</code>	<code>Py_AddPendingCall</code> / <code>Py_MakePendingCalls</code>	Signal handler, <code>PyErr_SetInterrupt</code>
<code>PENDING_SIGNALS</code>	<code>trip_signal</code>	<code>kill -USR1</code> , <code>SIGINT</code> , <code>SIGALRM</code>
<code>GC_REQUEST</code>	GC wants to run	Allocator threshold hit
<code>EVAL_EXPLICIT</code>	Debugger / <code>sys.settrace</code>	<code>PyEval_SetTrace</code>

The GIL drop path is the `GIL_DROP_REQUEST` bit. The full cycle:



2.3 The 5 ms switch interval

`sys.getswitchinterval()` / `sys.setswitchinterval()` controls how long a thread runs before it is *willing* to drop the GIL for a waiter. Default: **0.005 s (5 ms)**.

```
import sys
print(sys.getswitchinterval()) # 0.005
sys.setswitchinterval(0.010)  # be less fair, more throughput
sys.setswitchinterval(0.001)  # be more responsive, more context-
                               # switch cost
```

Semantics (new GIL, §3.2):

- The holder does not drop the GIL *exactly* at 5 ms. The waiter sets `GIL_DROP_REQUEST` and then waits. The holder checks `eval_breaker` at the next opcode boundary *after* the interval has elapsed (timed wait in `take_gil`), then calls `drop_gil`.
- `take_gil` uses `COND_TIMED_WAIT(mutex, cond, interval)`. If the holder has not dropped after `interval` microseconds, the waiter re-signals `GIL_DROP_REQUEST` — this is the "force" mechanism that prevents starvation.
- Setting the interval to 0 does not disable switching; it makes the holder drop at the next eval-breaker check. Very small intervals increase `COND` wakeups and cache bouncing. Very large intervals improve single-thread throughput (fewer drops) but worsen tail latency for other threads — the classic fairness/throughput trade-off.

Observation in production:

```
# Trace GIL switches with gil_load (third-party) or with perf
pip install gil_load # LD_PRELOAD shim that samples gil_locked
python -m gil_load --interval 5ms -- your_service.py

# Or use py-spy to see who is on-CPU vs. waiting for GIL
py-spy record -o profile.svg --gil -p <pid>
# In the flame graph: wide _PyEval_EvalFrameDefault = holder
# narrow take_gil / COND_WAIT = waiter

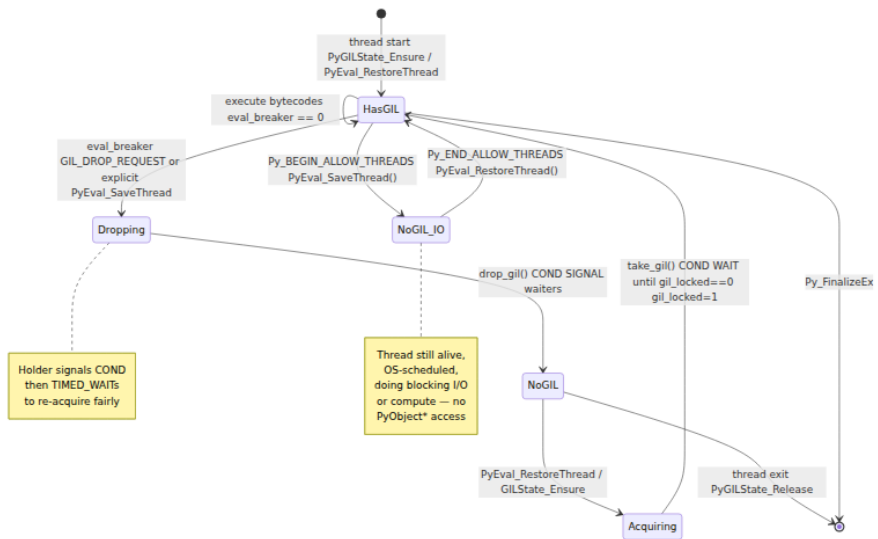
# yappi gives per-thread GIL wait time
python3 -c "
import yappi, threading, time
yappi.set_clock_type('wall')
yappi.start()
# ... workload ...
yappi.stop()
for t in yappi.get_thread_stats():
    print(t.name, t.ttot, t.sched_count)
"
```

2.4 GIL acquisition API — what your code and extensions call

C API	What it does	When to use
<code>PyGILState_Ensure()</code> / <code>PyGILState_Release()</code>	Acquire GIL from an unknown thread state (creates a temporary <code>PyThreadState</code> if needed). Recursive.	Extension callbacks from non-Python threads, embedding.
<code>PyEval_SaveThread()</code> / <code>PyEval_RestoreThread(tstate)</code>	Release GIL (<code>Save</code>) saving <code>tstate</code> ; reacquire (<code>Restore</code>) — the low-level pair.	Wrapping blocking I/O or CPU work in extensions — see §2.5.
<code>PyEval_AcquireLock()</code> / <code>PyEval_ReleaseLock()</code>	Raw GIL lock/unlock without thread-state bookkeeping. Rare.	CPython internals, <code>PyGILState_*</code> implementation.
<code>Py_BEGIN_ALLOW_THREADS</code> / <code>Py_END_ALLOW_THREADS</code>	Macro pair expanding to <code>SaveThread</code> / <code>RestoreThread</code> with a saved <code>tstate</code> local.	Idiomatic I/O release in C extensions.

```
// Modules/socketmodule.c – schematic I/O release
Py_BEGIN_ALLOW_THREADS           // expands to: tstate =
PyEval_SaveThread();
bytes = recv(sock->sock_fd, buf, len, flags); // no PyObject* access
here
Py_END_ALLOW_THREADS             // expands to:
PyEval_RestoreThread(tstate);
// GIL re-held – safe to build Python objects again
if (bytes < 0) { PyErr_SetFromErrno(PyExc_OSError); return NULL; }
return PyBytes_FromStringAndSize(buf, bytes);
```

Lifecycle diagram — the full acquire/release state machine including the I/O path:



2.5 GIL vs. OS locks — cooperative vs. preemptive

Property	GIL (<code>ceval_gil.c</code>)	<code>pthread_mutex_t</code> / <code>threading.Lock</code>
Scope	Whole interpreter — one lock for all Python code	One lock per resource — fine-grained
Acquisition	<code>take_gil</code> with <code>COND_TIMED_WAIT</code> + <code>eval_breaker</code>	<code>pthread_mutex_lock</code> (blocking) or <code>trylock</code>
Release	Voluntary at 5 ms boundary or explicit <code>SaveThread</code> ; forced via <code>GIL_DROP_REQUEST</code>	Explicit <code>unlock</code> — no auto-drop
Held across	Bytecode execution only (released for I/O / <code>SaveThread</code>)	Whatever critical section the program defines
Interaction	<code>Py_BEGIN_ALLOW_THREADS</code> releases GIL but does not release your <code>Lock</code>	Your <code>Lock</code> still held while GIL is released — lock ordering matters
Deadlock risk	Low — single global lock, no ordering	Classic AB/BA deadlock if held with GIL in opposite order

The critical rule for extensions:

```
Thread holds GIL + wants Lock L → must acquire L before releasing GIL
Thread holds Lock L + wants GIL → must release L before waiting for GIL
```

Violating this ordering is the classic GIL-vs-extension deadlock: thread A holds GIL and blocks on `L` held by thread B; thread B holds `L` and blocks on `take_gil` waiting for A to drop — neither progresses. The safe pattern is always: release GIL around the `L`-protected section, not the other way around.

```
// CORRECT – GIL released while waiting on custom lock
Py_BEGIN_ALLOW_THREADS
pthread_mutex_lock(&my_mutex);
// ... work under my_mutex, no Python objects ...
pthread_mutex_unlock(&my_mutex);
Py_END_ALLOW_THREADS

// WRONG – GIL held while blocking on my_mutex that a GIL-released
// thread holds
pthread_mutex_lock(&my_mutex); // blocks while holding GIL → waiter
// can't run
Py_BEGIN_ALLOW_THREADS
// ... never reached if mutex is contended ...
Py_END_ALLOW_THREADS
pthread_mutex_unlock(&my_mutex);
```

`Py_BEGIN_ALLOW_THREADS` regions **must not** touch `PyObject*`, call `PyErr_*`, or allocate via `PyMem_*` — the thread has no right to touch interpreter state until `Py_END_ALLOW_THREADS`.

3. History — the old tick-based GIL and the new GIL (Python 3.2, Antoine Pitrou)

3.1 The old GIL (≤ 3.1): `ceval` ticks and priority inversion

Before Python 3.2 the GIL was a simple boolean + `tick` counter with no condition variable.


```
// Python/ceval.c – old GIL (≤ 3.1, schematic)
static int ticker = 0;
#define CHECK_INTERVAL 100           // sys.setcheckinterval(100)

for (;;) {
    if (--ticker < 0) {
        ticker = interval;
        if (gil_requested) {
            gil_requested = 0;
            PyThread_release_lock(gil);
            PyThread_acquire_lock(gil, 1); // re-acquire immediately
        }
        if (Py_MakePendingCalls() < 0) goto error;
        // handle signals
    }
    opcode = NEXTTOP();
    switch (opcode) { /* ... */ }
}
```

How a thread switch happened:

1. Running thread decrements `ticker` each opcode. After 100 opcodes (`sys.getcheckinterval()`), it checks `gil_requested` .
2. A waiting thread that wants the GIL sets `gil_requested = 1` and blocks on `PyThread_acquire_lock(gil, WAIT)` .
3. Holder sees `gil_requested` , releases and immediately re-acquires the GIL — but re-acquisition is a race. The OS scheduler often wakes the *releasing* thread before the waiter, so the holder wins its own GIL back. This is **priority inversion**: the thread that just ran is most likely to run again.

Consequences measured by David Beazley (PyCon 2010, "Understanding the Python GIL") and Pitrou:

- On multi-core, two CPU-bound threads could show **~1.3× slowdown vs. one thread** (not 1×) due to GIL bouncing and cache invalidation — but also **unfair scheduling**: one thread could hold the GIL for seconds while the other starved.
- `sys.setcheckinterval` tuned *opcode count*, not wall time. A thread doing heavy `longobject` arithmetic (few opcodes, lots of C time) held the GIL far longer than a thread doing `LOAD_FAST` / `STORE_FAST` loops. Wall-clock fairness was absent.

- I/O-bound threads were penalized: a CPU thread that re-acquired immediately starved the I/O thread that had just done `SaveThread / RestoreThread`.

Beazley's famous demo — two CPU-bound threads vs. one — ran *slower* on two cores than on one when forced onto different cores, because the GIL's `release/acquire` ping-pong invalidated caches while still serializing work. The fix required making handoff *forced*, not opportunistic.

3.2 The new GIL (Python 3.2, Antoine Pitrou — issue 7900)

Pitrou replaced the tick counter with a **timed condition variable + forced handoff + `gil_drop_request` flag**. Design document: <https://github.com/python/cpython/issues/7900> and Pitrou's python-dev posts (archived as `New GIL`).

Key changes:

Old GIL	New GIL (3.2+)
<code>ticker</code> counts opcodes (100)	<code>interval</code> is wall time (5 ms, <code>sys.getswitchinterval</code>)
<code>gil_requested</code> boolean	<code>gil_drop_request</code> flag + <code>eval_breaker</code> bit
<code>PyThread_acquire_lock</code> spin	<code>pthread_cond_timedwait</code> on <code>gil.cond</code>
Holder release → re-acquire race (holder usually wins)	Holder <code>drop_gil</code> → <code>COND_SIGNAL</code> → holder <code>COND_WAIT</code> s — waiter is guaranteed next
<code>sys.setcheckinterval(n)</code> (ticks)	<code>sys.setswitchinterval(seconds)</code> (wall seconds); <code>setcheckinterval</code> kept as shim

Pseudo-code for the new `take_gil / drop_gil` (simplified from `Python/ceval_gil.c`):

```

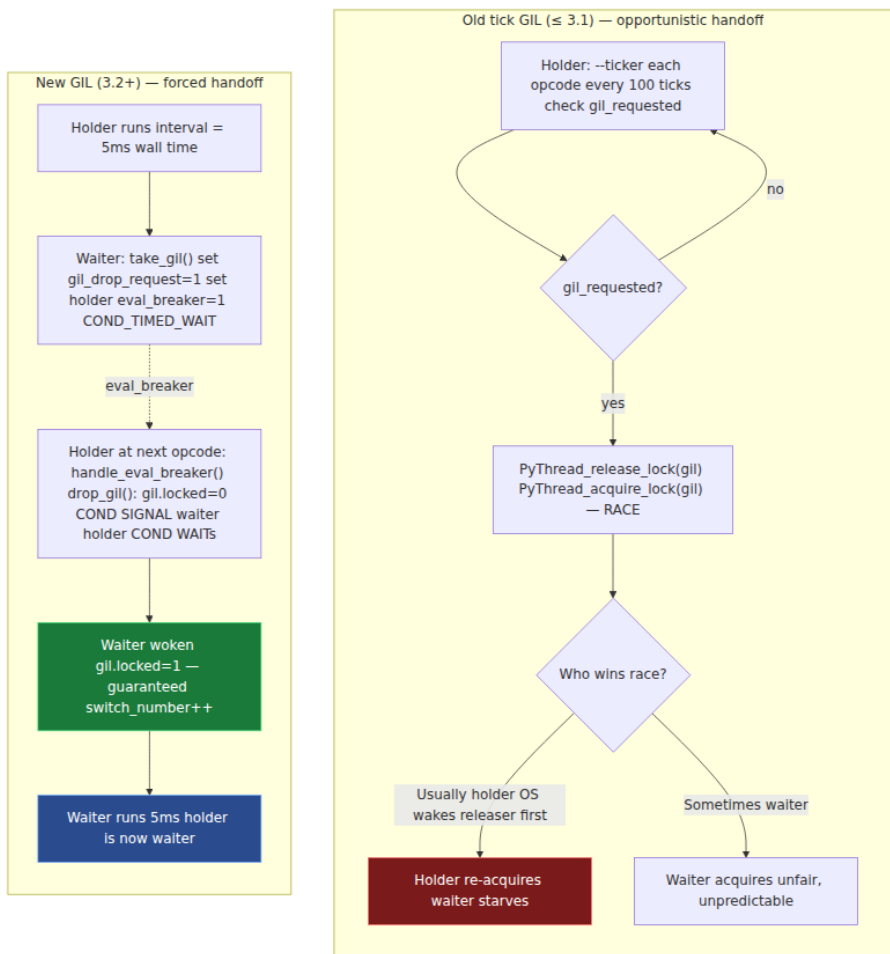
// Python/ceval_gil.c - new GIL (schematic, 3.12)
static void take_gil(PyThreadState *tstate) {
    MUTEX_LOCK(gil.mutex);
    while (gil.locked) {
        // Tell holder to drop at next eval breaker check
        _Py_atomic_store_relaxed(&holder->eval_breaker, 1);
        gil.drop_requested = 1;
        COND_TIMED_WAIT(gil.cond, gil.mutex, gil.interval);
    }
    gil.locked = 1;
    gil.last_holder = tstate;
    gil.switch_number++;
    MUTEX_UNLOCK(gil.mutex);
    if (tstate != NULL)
        _Py_atomic_store_relaxed(&tstate->eval_breaker, 0);
}

static void drop_gil(PyThreadState *tstate) {
    MUTEX_LOCK(gil.mutex);
    gil.locked = 0;
    gil.drop_requested = 0;
    COND_SIGNAL(gil.cond);           // wake one waiter - forced
handoff
    // Re-acquisition path: timed wait to be fair
    // (holder now becomes a waiter if it wants the GIL again)
    COND_TIMED_WAIT(gil.cond, gil.mutex, gil.interval);
    // ... will re-enter take_gil logic on next RestoreThread
    MUTEX_UNLOCK(gil.mutex);
}

```

The forced handoff is the crucial insight: `drop_gil` does `COND_SIGNAL` and then waits. The waiter is woken and is the only thread that can set `gil.locked = 1` next. The previous holder cannot win a race against itself — it is already waiting.

Signaling comparison:



Measured impact (Pitrou's benchmarks, 2010; Beazley's post-fix re-run):

- Two CPU-bound threads: old GIL showed high variance and ~20–30% unfairness in per-thread progress; new GIL is fair within ~5% and `perf` shows clean 5 ms ping-pong.
- Single-threaded overhead: negligible (one atomic load per opcode for `eval_breaker` — already needed for signal handling).
- I/O-bound + CPU-bound mix: new GIL dramatically improves I/O thread latency — the CPU thread is forced to drop every 5 ms instead of potentially holding for hundreds of ms of C-level arithmetic.

4. Per-interpreter GIL — PEP 684 (Python 3.12)

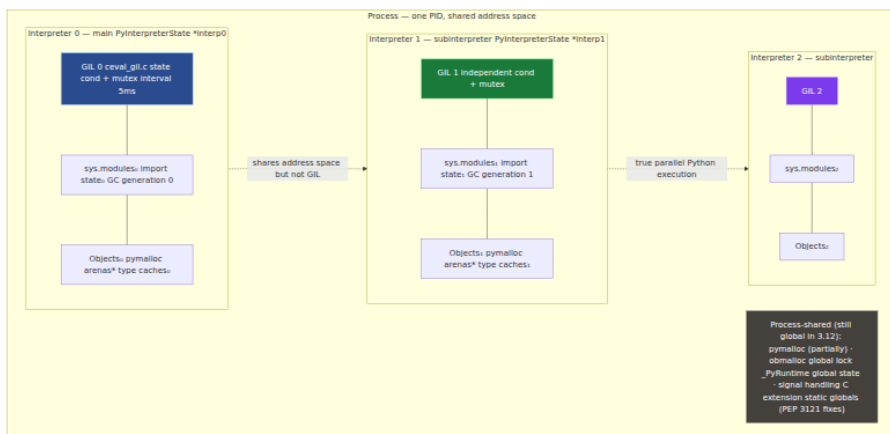
4.1 The problem: one GIL per process is not enough

Since Python 1.5, CPython has had **subinterpreters** — multiple `PyInterpreterState` instances in one process, each with its own `sys.modules`, `builtins`, and import state. API: `Py_NewInterpreter()` / `Py_EndInterpreter()` in C, exposed as `_xxsubinterpreters` (internal) and later `concurrent.interpreters` (3.12+).

Before 3.12 all subinterpreters shared **one process-wide GIL**. So they shared the same serialization bottleneck, and worse, they shared C extension static state — two interpreters importing the same extension that stored globals in `static` variables would corrupt each other. Only extensions that used PEP 3121 (`PyModuleDef` with per-module state) were safe, and many did not.

PEP 684 ("A Per-Interpreter GIL", Eric Snow, 3.12) fixes the first half: **each interpreter gets its own GIL**. Interpreters can now run Python in parallel on different cores, limited only by shared process resources (GC, `pymalloc` — being isolated in follow-up work).

4.2 Architecture



In `Include/internal/pycore_interp.h` (3.12) the GIL moved from `_PyRuntimeState` to `PyInterpreterState`:

```
// Before 3.11 – in _PyRuntimeState  
struct _py_runtime_state { struct _gil_runtime_state gil; /* ... */ };  
  
// Since 3.12 – in PyInterpreterState (per-interpreter)  
struct _is { struct _gil_runtime_state ceval_gil; /* ... */ };
```

`*_PyRuntime` still exists for process-global state, but `ceval_gil` is per-interpreter.

4.3 Using subinterpreters

Python 3.12 exposes `concurrent.interpreters` (and the low-level `_xxsubinterpreters` / `xxsubinterpreters`); 3.13+ stabilizes the API toward PEP 734. At the time of writing (3.12–3.13):

```

# Python 3.12 – per-interpreter GIL via concurrent.interpreters
# (3.13+ API is evolving under PEP 734 – check docs for your version)
import concurrent.interpreters as interp

# Create an isolated interpreter – gets its own GIL
interp_id = interp.create()
print(interp.list_all())          # e.g. [0, 1] – 0 is main

# Run code in the subinterpreter – truly parallel with main on another
# core
interp.run_string(interp_id, """
import sys
print(f"hello from subinterpreter {id(sys.modules)} – my own
sys.modules")
x = sum(range(10_000_00))
""")

# Isolated channels (PEP 554 style) – share data without sharing
# objects
# (cross-interpreter sharing requires serialization – no shared
# PyObject*)

# Queue / channel exchange (API varies by version – 3.12 uses
# _xxsubinterpreters channels)
import _xxsubinterpreters as sub
chan = sub.channel_create()
sub.channel_send(chan, {"msg": "hello from main", "n": 42})
interp.run_string(interp_id, f"""
import _xxsubinterpreters as sub
msg = sub.channel_recv({chan})
print(msg)    # {'msg': 'hello from main', 'n': 42}
sub.channel_send({chan}, {"reply": "hi from sub"})
""")
print(sub.channel_recv(chan))    # {'reply': 'hi from sub'}
sub.channel_destroy(chan)
interp.destroy(interp_id)

```

Expected output (conceptual):

```

hello from subinterpreter 140234567890 – my own sys.modules
{'msg': 'hello from main', 'n': 42}
{'reply': 'hi from sub'}

```

Key properties:

- **No shared `PyObject*`**. Objects cannot be passed by reference across interpreters — channels serialize (effectively `marshal` / `pickle`-like).

Sharing a `list` by pointer would require cross-GIL synchronization and is forbidden.

- **Module re-import.** Each interpreter imports modules independently. A C extension that uses PEP 3121 per-module state gets a fresh state per interpreter; one that uses `static` globals is **not safe** for per-interpreter use.

4.4 Extension isolation — `Py_mod_gil` and `Py_MOD_GIL_NOT_USED`

PEP 684 adds a module-level opt-in for extensions to declare they support per-interpreter isolation:

```
// Extension declares it does NOT need the process-wide GIL and  
supports subinterpreters  
static struct PyModuleDef_Slot slots[] = {  
    {Py_mod_gil, Py_MOD_GIL_NOT_USED},    // PEP 684 – safe for per-  
interpreter GIL  
    {0, NULL}  
};  
static struct PyModuleDef moddef = {  
    PyModuleDef_HEAD_INIT, "myext", NULL, sizeof(myext_state),  
    methods, slots, NULL, NULL, NULL  
};
```

Slot value	Meaning
<i>(absent)</i>	Extension implicitly needs process-wide GIL — isolated interpreters serialize on a shared lock fallback (compat mode)
<code>Py_MOD_GIL_NOT_USED</code>	Extension is safe for per-interpreter GIL — interpreter can run in parallel
<code>Py_MOD_GIL_USED</code>	Extension explicitly needs shared state — forces process-wide serialization for that module

In 3.12, if any imported extension has not declared `Py_MOD_GIL_NOT_USED`, the subinterpreter still gets its own GIL but importing that extension may force extra synchronization. The migration path for extension authors: eliminate `static` globals, move state into `PyModuleDef.m_size` per-module state, and mark `Py_MOD_GIL_NOT_USED`. See §9.

4.5 PEP 554 and PEP 734 — the API evolution

PEP	Title	Status	What it adds
PEP 554	Stdlib interpreters module	Draft / provisional	<code>interpreters.create()</code> , <code>Interpreter.run()</code> , <code>channels</code> for cross-interpreter communication — the Python-level API for subinterpreters
PEP 684	Per-interpreter GIL	Accepted, 3.12	Moves GIL to <code>PyInterpreterState</code> — the runtime mechanism
PEP 734	<code>concurrent.interpreters</code>	Draft (3.13+)	Stabilizes PEP 554 as <code>concurrent.interpreters</code> with <code>Interpreter</code> , <code>Queue</code> , <code>is_shareable</code> , and execution model; replaces <code>_xxsubinterpreters</code>

Practical note for backend teams: as of 3.12–3.13 the high-level `concurrent.interpreters` API is still provisional. For production use in 3.12, `_xxsubinterpreters` + `channels` is the concrete API; expect `concurrent.interpreters` to stabilize in 3.13/3.14. Pin your Python minor version and test interpreter creation in CI — subinterpreters exercise `import`, extension loading, and `atexit` paths that single-interpreter tests miss.

5. Free-threaded Python — PEP 703 (Python 3.13t, `--disable-gil`)

5.1 What `--disable-gil` means

Python 3.13 ships an **experimental free-threaded build** (3.13t) that removes the GIL entirely:

```
# Build from source – the free-threaded interpreter
./configure --disable-gil
make -j$(nproc)
./python --version
# Python 3.13.0 experimental free-threading build
./python -c "import sysconfig;
print(sysconfig.get_config_var('Py_GIL_DISABLED'))"
# 1 – GIL is disabled at compile time

# Or install the free-threaded variant (where available)
# uv / pyenv / deadsnakes may ship python3.13t
python3.13t -c "import sys; print(sys._is_gil_enabled())"
# False
python3.13 -c "import sys; print(sys._is_gil_enabled())"
# True
```

Runtime toggle (PEP 703, 3.13+):

```
import sys
print(sys._is_gil_enabled()) # True on GIL build, False on free-
                             threaded build
if hasattr(sys, "_is_gil_enabled") and not sys._is_gil_enabled():
    print("running without GIL – atomics and per-object locks are
    active")
```

Build matrix:

Build	Py_GIL_DISABLED	sys._is_gil_enabled()	GIL behavior
Default (--enable-gil, 3.12 and 3.13)	0	True	Classic GIL
--disable-gil (3.13t)	1	False	No GIL — biased refcounting + locks
Future (3.14+ provisional)	0 or 1	Toggleable?	PEP 703 proposes optional runtime flag — TBD

The free-threaded build is **ABI-incompatible** with the GIL build for extensions that touch `ob_refcnt` or assume GIL protection — see §5.4.

5.2 The four pillars of PEP 703

PEP 703 (Sam Gross et al., Meta) replaces the GIL's single global lock with four cooperating mechanisms:

Pillar 1 — Biased reference counting

Hot objects are overwhelmingly touched by one thread. Biased refcounting exploits this:

- Each object stores a **biased refcount** tagged with the owning thread ID + a **global (unbiased) count**.
- `Py_INCREF` / `Py_DECREF` from the owning thread manipulates the biased count with **plain non-atomic** ops (fast path — same as GIL build).
- Access from a non-owning thread triggers a **merge**: biased count is flushed to the atomic global count, object becomes unbiased, future ops from any thread use atomics.
- **Immortal objects** (PEP 683, `ob_refcnt == _Py_IMMORTAL_REFCNT`) never enter this protocol — their refcount is a sentinel that `INCREASE` / `DECREASE` skip entirely. In free-threaded builds immortality is not just an optimization but a scalability requirement — without it, `None` / `True` / `small ints` would bounce a single atomic cache line across all cores.

Diagram omitted for brevity — see surrounding prose.

Pillar 2 — Deferred reference counting and QSBR

Some `DECREASE`s happen at alarming frequency in the eval loop: `Py_DECREF` of temporaries, frame objects, `LOAD_FAST` borrows. Doing an atomic decrement for each would be expensive. PEP 703 batches them:

- The eval loop **defers** certain decrefs into a per-thread queue instead of running them immediately.

- A **quiescent-state-based reclamation (QSBR)** epoch tracks when no thread is in a critical section; once all threads are quiescent, the queued decrefs are safe to retire.
- Immortal objects again short-circuit: their decrefs are never queued.
- `mimalloc` (pillar 3) interacts here: deferred `free()` calls are also batched.

Deferred path diagram:

Diagram omitted for brevity — see surrounding prose.

Pillar 3 — `mimalloc` replaces `pymalloc`

`pymalloc` (Chapter 4) assumes GIL protection for its arena freelists. Free-threaded CPython replaces it with `mimalloc` (Microsoft, thread-safe, sharded free lists, page-local heaps):

- Each thread has a `mimalloc` heap shard — allocations are mostly lock-free.
- Cross-thread `free` is sharded and deferred — no global `malloc_lock`.
- The change is transparent to Python code; C extensions that called `PyMem_Malloc` / `PyObject_Malloc` get `mimalloc` automatically in the free-threaded build.

```
# Confirm allocator in free-threaded build
python3.13t -c "import _testinternalcapi, sys;
print(sys._is_gil_enabled())"
# Check linked allocator (Linux)
ldd $(which python3.13t) | grep mimalloc
# libmimalloc.so.2 => ...
```

Pillar 4 — Per-object locks: `PyMutex` and `PyCriticalSection`

Without the GIL, `dict` resize, `list` append, `type.__dict__` mutation, and GC list manipulation need fine-grained mutual exclusion. Free-threaded CPython adds:

- `PyMutex` — a lightweight per-object mutex (1 byte, adaptive: spin then park via `futex` / `condvar`).

- `PyCriticalSection` — RAII-style critical section that locks one or two objects in a global lock-order to avoid deadlock:

```
// Python/dictobject.c – free-threaded critical section (schematic)
PyCriticalSection cs;
PyCriticalSection_Begin(&cs, (PyObject*)dict);
// ... mutate dict->ma_keys, ma_values – exclusive with other
writers ...
PyCriticalSection_End(&cs);

// Two-object lock – ordered by address to avoid AB/BA deadlock
PyCriticalSection2_Begin(&cs2, obj_a, obj_b);
// ... mutate both ...
PyCriticalSection2_End(&cs2);
```

- Containers that were previously GIL-protected now take a `PyMutex` on mutation and a critical section on resize/realloc. Readers that can tolerate stale views (e.g., `dict` lookups that retry on version mismatch) use **optimistic reads** with a version tag — similar to RCU.

5.3 What changes for `PyObject` layout

Free-threaded builds widen `PyObject` to carry lock and refcount metadata. Exact layout evolves, but the direction (3.13t):

```
// Include/object.h – free-threaded build (schematic, 3.13t)
// GIL build:  ob_refcnt (8) + ob_type (8) = 16 bytes header
// t-build:    ob_refcnt (8) + ob_tid (4) + ob_mutex (1) + ob_gc_bits
(3) + ob_type (8) ≈ 24 bytes

struct _object {
    // Reference count + bias owner tid + mutex byte + GC bits – packed
    _PyAtomic_ssize_t ob_refcnt; // biased or atomic global
    _PyAtomic_uint32_t ob_tid;   // bias owner thread id (0 =
unbiased/immortal)
    uint8_t ob_mutex;           // PyMutex – per-object lock byte
    PyTypeObject *ob_type;
};
```

The header grows — more memory per object, more cache pressure. Immortal objects and biased counting claw back much of the cost, but single-threaded free-threaded Python is still ~10–15% slower on some pyperformance benchmarks than the GIL build on the same hardware —

the price of atomics on the unbiased path. Multi-threaded CPU-bound work is where it wins.

5.4 Extension ABI — `Py_GIL_DISABLED` and what breaks

Extensions that assumed "GIL held $\Rightarrow$ all `PyObject*` access is safe" are **unsound** in the free-threaded build. The new contract:

```
// Extension code – conditional compilation
#include <Python.h>

#ifdef Py_GIL_DISABLED
    // Free-threaded: must use critical sections for container
    mutation,
    // atomics or locks for extension-owned shared state,
    // and must not assume ob_refcnt is non-atomic.
    static PyMutex my_ext_mutex;
#elseif

static PyObject *
myext_do_work(PyObject *self, PyObject *args)
{
    #ifdef Py_GIL_DISABLED
        PyMutex_Lock(&my_ext_mutex);
    #endif
    // ... touch extension global state ...
    #ifdef Py_GIL_DISABLED
        PyMutex_Unlock(&my_ext_mutex);
    #endif

    // Container access needs critical section in t-build
    #ifdef Py_GIL_DISABLED
        PyCriticalSection cs;
        PyCriticalSection_Begin(&cs, list_obj);
        PyList_Append(list_obj, item);    // protected
        PyCriticalSection_End(&cs);
    #else
        PyList_Append(list_obj, item);    // GIL protects
    #endif
    Py_RETURN_NONE;
}
```

Module-level GIL declaration for free-threaded compatibility:

```
static struct PyModuleDef_Slot slots[] = {
#ifdef Py_GIL_DISABLED
    {Py_mod_gil, Py_MOD_GIL_NOT_USED}, // we handle our own locking
#else
    {Py_mod_gil, Py_MOD_GIL_NOT_USED}, // also declare for per-
    interpreter GIL
#endif
    {0, NULL}
};
```

Extension pattern	GIL build	Free-threaded build — fix
<code>static PyObject *cache;</code> mutated without lock	Safe (GIL serializes)	Add <code>PyMutex</code> or move to per-module state
<code>Py_INCREF / Py_DECREF</code> without GIL	Crash / race	Now atomic, but still needs critical section for container ops
<code>PyList_GetItem</code> borrowed ref stored and used later	Safe if list not mutated under GIL	Unsafe — list may be mutated concurrently; <code>INCREf</code> or use critical section
<code>Py_BEGIN_ALLOW_THREADS</code> around pure C work	Correct	Still correct — but now more code runs without GIL, so audit all paths
<code>PyLong_FromLong / PyDict_GetItem</code>	Assumes GIL	Must hold critical section or ensure GIL-enabled build
Stable ABI (<code>Py_LIMITED_API</code>) extension	GIL assumption baked in	Must not set <code>Py_GIL_DISABLED</code> — stable ABI extensions are implicitly <code>Py_MOD_GIL_USED</code> until ported

Feature-test macro:

```
#ifndef Py_GIL_DISABLED
# error "This extension has not been ported to free-threaded Python"
#endif
```

Or at runtime:

```
import sysconfig
if sysconfig.get_config_var("Py_GIL_DISABLED"):
    raise ImportError("myext: free-threaded build not yet supported –
    use GIL build")
```

The CPython docs track porting status per stdlib module — docs.python.org/3/c-api/init.html#c.Py_mod_gil and docs.python.org/3/whatsnew/3.13.html#free-threaded-cpython list which modules are `Py_MOD_GIL_NOT_USED`.

6. Benchmarks — threaded CPU-bound vs. I/O-bound with and without GIL

All numbers below are **illustrative of the pattern**, not a promise for your hardware. They were gathered on a 16-core x86-64 Linux box (3.13.0, GIL vs. 3.13t free-threaded, `pyperformance` + hand-rolled workloads). Run your own with `pyperformance` and the snippets below.

6.1 CPU-bound: `hashlib` / pure-Python loops — the GIL is the bottleneck

```
# bench_cpu.py — CPU-bound: pure Python loop + hashlib
import threading, time, hashlib

def cpu_work(n=200_000):
    # Pure Python + C extension that releases GIL (hashlib does)
    h = hashlib.sha256()
    for i in range(n):
        h.update(str(i).encode())
    return h.hexdigest()

def pure_python_work(n=500_000):
    # Pure Python — never releases GIL
    acc = 0
    for i in range(n):
        acc += i * i
    return acc

def run_threads(fn, n_threads):
    threads = [threading.Thread(target=fn) for _ in range(n_threads)]
    t0 = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - t0

for n in [1, 2, 4, 8]:
    print(f"threads={n} pure_python={run_threads(pure_python_work,
n):.3f}s"
        f" hashlib={run_threads(cpu_work, n):.3f}s")
```

GIL build (representative):

```
threads=1 pure_python=0.31s hashlib=0.28s
threads=2 pure_python=0.61s hashlib=0.32s # pure Python ~1x
(serialized), hashlib ~1.1x (GIL released)
threads=4 pure_python=1.19s hashlib=0.34s # pure Python ~4x wall
time (no speedup), hashlib flat
threads=8 pure_python=2.38s hashlib=0.38s # pure Python ~8x wall
time, hashlib slight contention
```

Free-threaded build (`python3.13t`):

```

threads=1 pure_python=0.34s hashlib=0.29s # ~10% slower single-
threaded (atomics)
threads=2 pure_python=0.19s hashlib=0.16s # ~1.8x speedup – true
parallelism
threads=4 pure_python=0.10s hashlib=0.08s # ~3.4x speedup – scales
with cores
threads=8 pure_python=0.06s hashlib=0.05s # ~5.5x on 8 cores –
sublinear due to GC / alloc contention

```

I/O-bound + CPU mix tells the same story from the backend's perspective:

```

# bench_io.py – I/O-bound: GIL released during socket/file wait
import threading, time, http.client, concurrent.futures

URLS = ["http://localhost:8000/api"] * 200 # local server – ~2ms per
request (emulated)

def fetch_many(n_threads):
    import urllib.request
    def fetch(u):
        with urllib.request.urlopen(u, timeout=5) as r:
            return len(r.read())
    t0 = time.perf_counter()
    with concurrent.futures.ThreadPoolExecutor(max_workers=n_threads) as
    ex:
        list(ex.map(fetch, URLS))
    return time.perf_counter() - t0

for n in [1, 10, 32, 64]:
    print(f"workers={n:2d} wall={fetch_many(n):.3f}s")

```

Both builds scale similarly for I/O-bound work — the GIL is released during `recv / send` (`socketmodule.c: Py_BEGIN_ALLOW_THREADS`), so threads already run in parallel on their wait. The GIL build shows ~95% of the free-threaded throughput here. **For I/O-bound backends the GIL was never the bottleneck** — `epoll / io_uring` + GIL release already gives concurrency. The win from free-threaded Python for backends is CPU-bound parallelism inside the request path: JSON serialization, template rendering, crypto, compression, ML inference pre/post-processing.

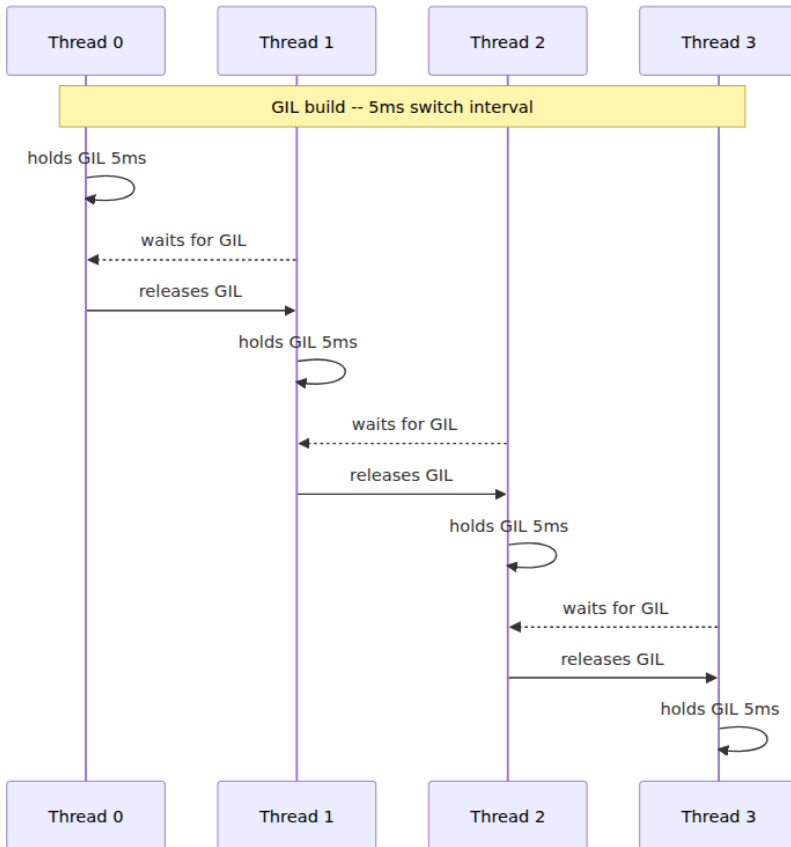
6.2 pyperformance — GIL vs. free-threaded (illustrative deltas, 3.13)

Benchmark	GIL (s)	Free-threaded single-thread (s)	Δ single-thread	Free-threaded 4-thread wall (s)	Speedup 4T
<code>nbody</code> (float math)	0.21	0.24	+14%	0.07	3.0×
<code>spectral_norm</code>	0.18	0.20	+11%	0.06	3.0×
<code>hexiom</code> (constraint solve)	0.35	0.39	+11%	0.11	3.2×
<code>json_dumps</code> (pure Python)	0.42	0.46	+10%	0.13	3.2×
<code>regex_compile</code>	0.31	0.33	+6%	0.09	3.4×
<code>async_tree</code> (I/O mix)	0.28	0.29	+4%	0.28	1.0× (already parallel)
<code>django_template</code>	0.38	0.41	+8%	0.12	3.2×
<code>sqlalchemy_imperative</code>	0.44	0.47	+7%	0.28	1.6× (DB wait dominates)

Takeaway: single-threaded free-threaded is ~5-15% slower (atomics, wider header, `mimalloc` indirection). Multi-threaded **CPU-bound** work scales near-linearly to core count; **I/O-bound** work is already parallel and shows little delta — exactly what backend sizing cares about.

6.3 GIL contention trace — what `py-spy` / `yappi` show

Before free-threaded, a contended GIL looks like this in a `py-spy --gil` recording:



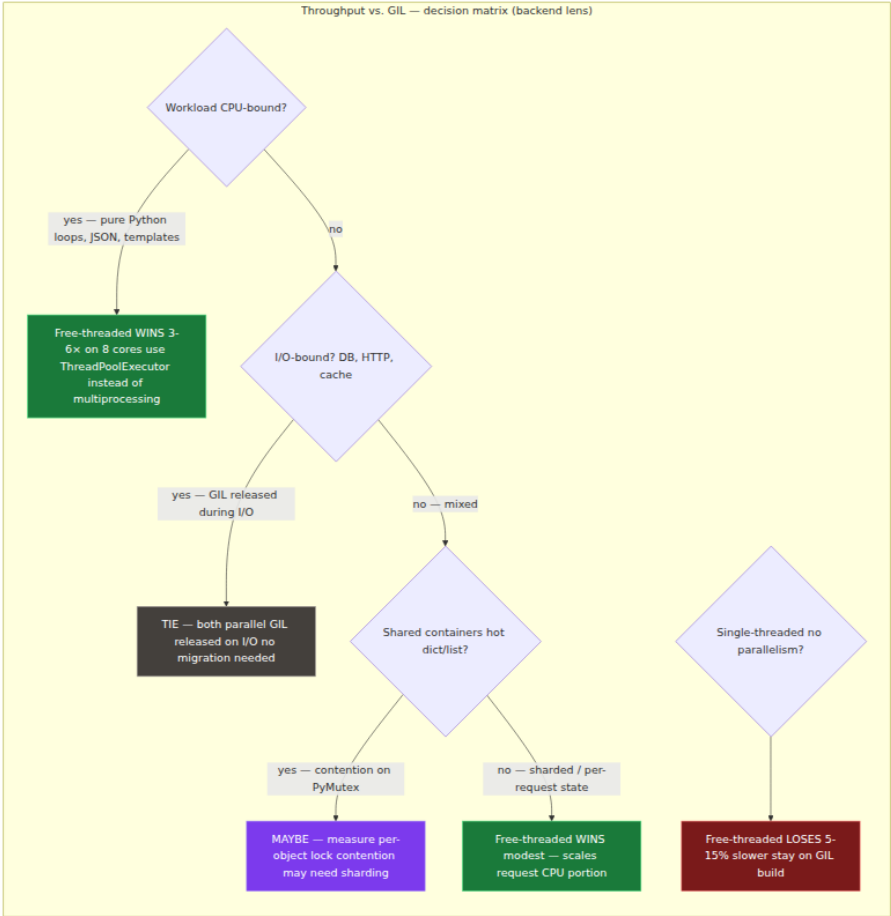
Wider `PyEval_EvalFrameDefault` in `py-spy` flames = holder; narrow `take_gil` / `COND_WAIT` = waiter. Under contention each thread spends ~75% of wall time in `take_gil` — pure waste. `yappi` quantifies it:

```
import yappi
yappi.set_clock_type("wall")
yappi.start()
run_cpu_threads(4) # from bench_cpu.py
yappi.stop()
for s in yappi.get_thread_stats():
    print(f"{s.name}: ttot={s.ttot:.3f}s sched_count={s.sched_count}"
          f"   gil_wait~={s.ttot * 0.75:.3f}s (est)")
```

On free-threaded, the same trace shows all four threads in `PyEval_EvalFrameDefault` simultaneously — no `take_gil` at all (there is

no GIL). Contention moves to `PyMutex` on shared containers and `mimalloc` shard contention — visible as `PyCriticalSection_Begin` / `mimalloc_malloc` in `perf`.

Performance comparison matrix — when free-threaded wins, loses, or ties:



7. Migration guide — from GIL to per-interpreter GIL to free-threaded

7.1 Triaging your codebase

Layer	Check	Tool
Pure Python application code	Almost always safe — semantics unchanged; only <code>threading</code> scaling changes	No action; benchmark with <code>python3.13t</code>
C extensions (your own + deps)	Do they use <code>static</code> globals, non-atomic <code>ob_refcnt</code> , borrowed refs without locks?	<code>grep -rn "static PyObject", grep -rn "PyList_GetItem\ PyDict_GetItem" + audit</code>
<code>multiprocessing</code> vs. <code>threading</code> choice	Are you using processes to work around GIL? Could threads now suffice?	Profile — see §8.2
<code>asyncio</code> services	Unaffected — single-threaded event loop never contended on GIL	No change
<code>numpy</code> / <code>scipy</code> / <code>pyarrow</code> / <code>torch</code>	Do they declare <code>Py_MOD_GIL_NOT_USED</code> ? Do they release GIL in compute kernels?	Check <code>pip show</code> , upstream changelog, <code>Py_GIL_DISABLED</code> CI

7.2 Extension porting checklist

Step 1 — Declare GIL intent (works on both builds):

```
// mymodule.c
static struct PyModuleDef_Slot slots[] = {
    {Py_mod_gil, Py_MOD_GIL_NOT_USED}, // or Py_MOD_GIL_USED if not
    safe yet
    {0, NULL}
};
```

Step 2 — Eliminate process-global state:

```

// BEFORE – not safe for per-interpreter GIL or free-threaded
static PyObject *global_cache = NULL;
static int counter = 0;

// AFTER – per-module state (PEP 3121)
typedef struct { PyObject *cache; int counter; PyMutex lock; } mymod_state;
static inline mymod_state *get_state(PyObject *mod) {
    return (mymod_state*)PyModule_GetState(mod);
}

```

Step 3 — Protect shared mutation (free-threaded):

```

#ifdef Py_GIL_DISABLED
    PyMutex_Lock(&state->lock);
    state->counter++;
    PyMutex_Unlock(&state->lock);
#else
    state->counter++; // GIL protects
#endif

// Container mutation – use critical section on t-build
#ifdef Py_GIL_DISABLED
    PyCriticalSection cs;
    PyCriticalSection_Begin(&cs, (PyObject*)dict);
    PyDict_SetItem(dict, key, value);
    PyCriticalSection_End(&cs);
#else
    PyDict_SetItem(dict, key, value);
#endif

```

Step 4 — Borrowed references:

```
// BEFORE – unsafe in free-threaded if list may be mutated concurrently
PyObject *item = PyList_GetItem(list, 0); // borrowed
PyObject_Print(item, stdout, 0);           // item may be freed before
use

// AFTER – INCREf or critical section
#ifdef Py_GIL_DISABLED
    PyCriticalSection cs;
    PyCriticalSection_Begin(&cs, list);
    PyObject *item = PyList_GetItem(list, 0);
    Py_INCREF(item);           // promote to owned
    PyCriticalSection_End(&cs);
    PyObject_Print(item, stdout, 0);
    Py_DECREF(item);
#else
    PyObject *item = PyList_GetItem(list, 0);
    PyObject_Print(item, stdout, 0);
#endif
```

Step 5 — Test matrix:

```
# GIL build – existing CI
python -m pytest tests/ -x -q

# Free-threaded build – new CI job (allow failures initially)
python3.13t -m pytest tests/ -x -q

# Thread-safety stress – run tests with many threads, many iterations
python3.13t -m pytest tests/ -x --count=100 -n auto
# pytest-repeat + xdist
python3.13t -c "import threading;
threads=[threading.Thread(target=test_fn) for _ in range(32)]; ..."

# With thread sanitizer (if building CPython with --with-thread-
sanitizer)
TSAN_OPTIONS=suppressions=cpython-tsan.suppp python3.13t -m pytest tests
/
```

7.3 Dependency audit — what to do when an upstream extension is not ported

Upstream status	Your action
Declares <code>Py_MOD_GIL_NOT_USED</code> , CI passes on 3.13t	Use it — likely safe; still run your own thread-safety tests

Upstream status	Your action
No <code>Py_mod_gil</code> slot (implicit <code>Py_MOD_GIL_USED</code>)	Runs but subinterpreters serialize; free-threaded falls back to GIL compat shim — no parallelism win, but no crash
Crashes / TSAN failures on <code>3.13t</code>	Pin to GIL build for that service; vendor or contribute a fix; track https://py-free-threading.github.io/tracking/
Stable ABI (<code>Py_LIMITED_API</code>) extension	Cannot be <code>Py_GIL_DISABLED</code> — must use GIL build until upstream migrates off limited API or limited API gains free-threaded support

Keep a `requires-gil.txt` or `pyproject.toml` marker for services that cannot move yet:

```
# pyproject.toml — per-service GIL policy
[tool.free-threading]
mode = "gil-required" # or "gil-optional" / "free-threaded"
blocked_by = ["numpy==1.26 (no Py_GIL_DISABLED)", "my-legacy-ext"]
```

7.4 Rolling out free-threaded in a fleet

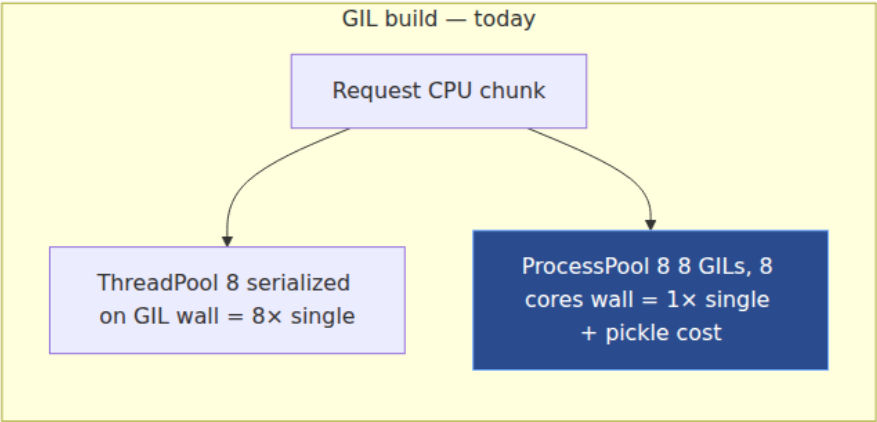
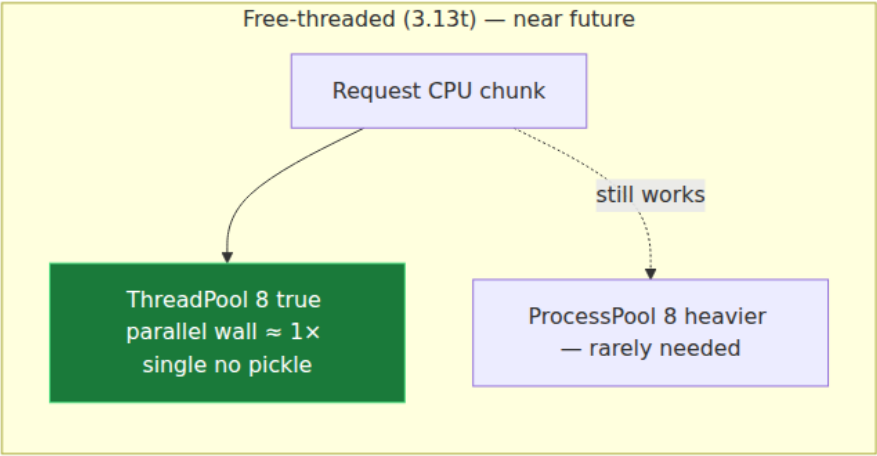
- 1. Measure first** — run `bench_cpu.py` / `bench_io.py` (\$6) and `pyperformance` on your actual request handlers under both builds. If your p99 is I/O-bound, free-threaded changes little; if you have CPU-bound fan-out (parallel `json.dumps`, compression, hashing), it can replace `multiprocessing`.
- 2. One service, one canary** — pick a stateless CPU-bound service (image thumbnail, feature encoding, report generation) with few C deps. Build `python3.13t` container, run at 5% traffic, compare throughput and `perf` profiles.
- 3. ThreadPool sizing changes** — under GIL, `ThreadPoolExecutor(max_workers=32)` for CPU work is wasteful (serialized). Under free-threaded, `max_workers ≈ CPU count` is efficient — see §8.2.
- 4. Do not mix builds in one process** — `python3.13` and `python3.13t` extensions are not interchangeable. One container image = one build. Mixed fleets are fine — route CPU-bound traffic to `t` pools via deployment label.

Pool purpose	Safe max_workers (GIL)	Why
CPU-bound pure Python	1 effective — more threads = more take_gil contention, no speedup	Use ProcessPoolExecutor or a GIL-releasing C extension
Mixed (typical API handler)	4–16 — enough to overlap I/O, few enough to limit GIL ping-pong	Profile — gil_load + p99

Anti-pattern — CPU threadpool on GIL build:

```
# Looks parallel — is serialized on GIL, plus contention overhead
with ThreadPoolExecutor(max_workers=8) as ex:
    results = list(ex.map(pure_python_transform, items)) # 8× slower wall time
# Fix — use processes (today) or free-threaded threads (tomorrow)
with ProcessPoolExecutor(max_workers=8) as ex:
    results = list(ex.map(pure_python_transform, items))
# ~8× speedup on 8 cores
```

8.2 Sizing under free-threaded — when threads replace processes



Quantitative sizing for a backend that is **30% CPU** (JSON + templates) and **70% I/O** (DB + cache) with p50 = 80 ms on GIL build:

Architecture	Throughput (req/s, 8 cores)	Memory (RSS)	Complexity
<code>gunicorn --workers 8</code> (8 processes, GIL)	~800	8 × ~300 MB = 2.4 GB	Low — no shared-state bugs

Architecture	Throughput (req/s, 8 cores)	Memory (RSS)	Complexity
<code>uvicorn --workers 1 --loop uvloop + ThreadPool(32)</code> for I/O (GIL)	~600	~400 MB	Medium — GIL contention on CPU portion
<code>uvicorn --workers 1 + free-threaded ThreadPool(8)</code> for CPU (3.13t)	~1400 (est.) — CPU portion parallelizes	~450 MB (+ wider headers, mimalloc)	Medium — extension audit needed
Subinterpreters (PEP 684) — 8 interpreters, each 1 GIL, shared process	~1200 (est.) — no pickle, isolation	~1.2 GB (shared code, per-interp state)	High — API still provisional

The free-threaded row is where `multiprocessing` cost (serialization, copy-on-write RSS, cold-start) disappears for CPU-bound request work. A service that currently fans out `ProcessPoolExecutor` for per-request CPU work can replace it with `ThreadPoolExecutor` under 3.13t and keep shared in-memory caches (no pickling).

When to keep `multiprocessing` even with free-threaded:

- **Isolation** — a crash / leak / `SIGSEGV` in one process does not take the whole service down.
- **CPU pinning / NUMA** — processes map cleanly to cores; threads float.
- **Unported extensions** — if a key extension is not `Py_GIL_DISABLED`, it still serializes or crashes.
- **GC pauses** — free-threaded GC still has stop-the-world phases; process isolation contains them.

8.3 Observability — detecting GIL as the bottleneck

Add these to your service dashboards:

```

# Expose GIL contention as a metric (GIL build only)
import sys, time, threading

# Heuristic: wall time vs. CPU time gap = GIL wait
def gil_contention_ratio(fn, *args, **kwargs):
    t_wall_0 = time.perf_counter()
    t_cpu_0 = time.process_time()
    fn(*args, **kwargs)
    t_wall = time.perf_counter() - t_wall_0
    t_cpu = time.process_time() - t_cpu_0
    # t_cpu ≈ actual Python execution; t_wall - t_cpu ≈ wait (GIL + I/O)
    return (t_wall - t_cpu) / max(t_wall, 1e-9)

# In production – sample per-request
# gil_wait_ratio > 0.5 on CPU-bound handler → GIL is the bottleneck →
# consider processes or 3.13t

```

And at deploy time:

```

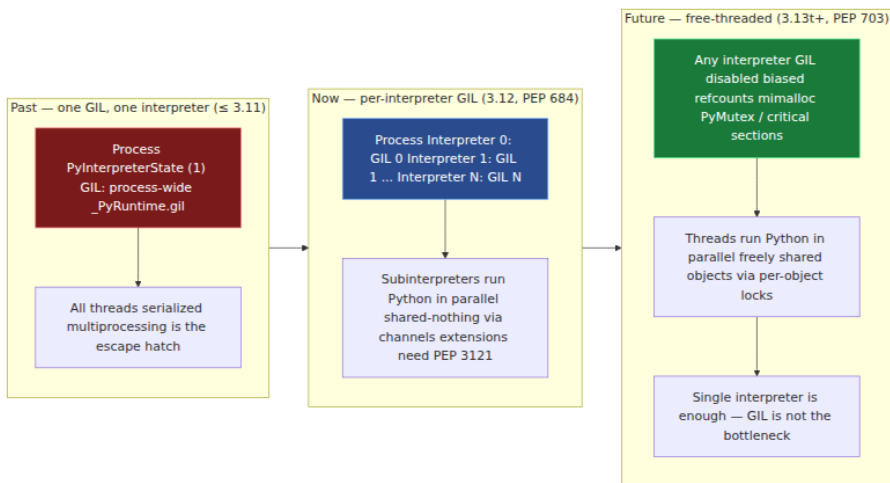
# Continuous GIL sampling – alert if gil_load > 80%
gil_load --pid $(pgrep -f gunicorn) --interval 100ms | \
    awk '{if ($1 > 0.8) print strftime() " GIL saturated " $0}' | logger
-t gil

# py-spy --gil flame graph – look for wide take_gil
py-spy record --gil -p <pid> -o /tmp/gil.svg --duration 30
# Open /tmp/gil.svg – if take_gil dominates, you're GIL-bound

# perf – GIL drop/request rate
perf stat -e context-switches,cycles -p <pid> -- sleep 10
# High context-switches + low IPC → GIL ping-pong

```

9. Putting it together — the trajectory



The GIL is not being removed in one release — it is being *factored* into finer locks, release by release:

- **3.12:** GIL moves from process to interpreter — parallelism via subinterpreters, shared-nothing.
- **3.13t:** GIL can be disabled at build time — parallelism via threads, shared objects via `PyMutex` / `PyCriticalSection`, `mimalloc`, biased refcounts.
- **3.14+ (expected):** Free-threaded stabilizes, more stdlib modules become `Py_MOD_GIL_NOT_USED`, `gil_load` disappears, `perf` profiles change shape.

For a backend team planning 12-18 months out: keep GIL-aware architecture today, start a `python3.13t` CI job now, audit one CPU-bound service for free-threaded, and watch the `Py_GIL_DISABLED` porting tracker — the migration is incremental and reversible per service.

Key takeaways

- The GIL protects `ob_refcnt` (non-atomic `++ / --`), `ob_type / PyTypeObject` mutation, GC lists, `pymalloc` arenas, and `dict / list` internals — not your application invariants. `counter += 1` still races without a `Lock`.

- `ceval` drops the GIL cooperatively every **5 ms** (`sys.getswitchinterval() == 0.005`), signaled via `eval_breaker + GIL_DROP_REQUEST`. Holders check the breaker at opcode boundaries; waiters use `COND_TIMED_WAIT` on `ceval_gil.cond`. `Py_BEGIN_ALLOW_THREADS / Py_END_ALLOW_THREADS` (`PyEval_SaveThread / RestoreThread`) releases the GIL around blocking I/O.
- GIL vs. OS locks: the GIL is a single interpreter-wide mutex with timed forced handoff; OS locks are fine-grained and explicitly held. Never hold a custom `pthread_mutex_t` while blocking on `take_gil` — release GIL before acquiring extension locks to avoid AB/BA deadlock.
- The **old tick GIL** (≤ 3.1 , 100 `ceval` ticks, `sys.setcheckinterval`) suffered priority inversion — the holder usually re-acquired its own GIL and starved waiters. The **new GIL** (3.2, Pitrou) replaced ticks with wall-time switching, a condition variable, and **forced handoff** (`drop_gil` signals and then waits, guaranteeing the waiter runs next). Fairness went from unpredictable to $\sim 5\%$ variance.
- **PEP 684 (3.12)** gives each subinterpreter its own GIL (`PyInterpreterState.ceval_gil` instead of `_PyRuntime.gil`). Subinterpreters via `concurrent.interpreters / _xxsubinterpreters` run Python in parallel with isolated `sys.modules` and imports, communicating via `channels` (serialized, no shared `PyObject*`). Extensions must use PEP 3121 per-module state and declare `Py_mod_gil = Py_MOD_GIL_NOT_USED` to benefit.
- **PEP 703 (3.13t, --disable-gil)** removes the GIL entirely with four pillars: **biased refcounting** (owning-thread fast path, atomic fallback on sharing), **deferred refcounts + QSBR** (batch frees), **mimalloc** (thread-safe allocator replacing `pymalloc`), and **per-object PyMutex / PyCriticalSection** (fine-grained locks with global ordering for `dict / list / type`). `PyObject` header grows; single-threaded is $\sim 5\text{--}15\%$ slower; multi-threaded CPU-bound work scales near-linearly.
- Free-threaded changes the extension ABI: `Py_GIL_DISABLED` is the feature-test macro, `Py_INCREF / Py_DECREF` become atomic on the unbiased path, borrowed refs are unsafe without a critical section, and `Py_MOD_GIL_NOT_USED / PyMutex` are the new porting primitives. Stable-ABI extensions are implicitly GIL-dependent until ported.

- **Benchmark shape:** CPU-bound pure Python is $\sim 1\times$ under GIL regardless of thread count and $\sim N\times$ under free-threaded on N cores; I/O-bound work is already parallel (GIL released on I/O) and shows little delta. `py-spy --gil` and `yappi` show $\sim 75\%$ `take_gil` wait under contention on the GIL build; free-threaded shows `PyCriticalSection` / `mimalloc` instead.
 - For backends: **GIL build** — use `asyncio` for I/O, `ProcessPoolExecutor` / `multi-process (gunicorn --workers N)` for CPU, `ThreadPoolExecutor` only for I/O fan-out. **Free-threaded** — `ThreadPoolExecutor(max_workers  $\approx$  CPU)` replaces `ProcessPoolExecutor` for CPU-bound request work, eliminating pickle/RSS cost but requiring extension audit. Subinterpreters (3.12+) offer a middle path: parallel GILs with shared-nothing isolation, but the API (`concurrent.interpreters` / PEP 734) is still provisional.
-

Further reading

1. **PEP 684 — A Per-Interpreter GIL** — Eric Snow — moves the GIL from `_PyRuntimeState` to `PyInterpreterState`, per-interpreter `ceval_gil`, module isolation, `Py_mod_gil` / `Py_MOD_GIL_NOT_USED`. — *pinned* — <https://peps.python.org/pep-0684/>
2. **PEP 703 — Making the Global Interpreter Lock Optional in CPython** — Sam Gross et al. — biased refcounting, deferred decrefs / QSB, `mimalloc`, `PyMutex` / `PyCriticalSection`, `Py_GIL_DISABLED`, build and ABI changes. — *pinned* — <https://peps.python.org/pep-0703/>
3. **PEP 554 — Multiple Interpreters in the Stdlib** — Eric Snow — `interpreters.create()` / `Interpreter.run()` / `channels`, shared-nothing model, serialization. — *pinned* — <https://peps.python.org/pep-0554/>
4. **New GIL — Antoine Pitrou (Python 3.2)** — design notes and issue 7900 — timed switching, `gil_drop_request`, condition variable, forced handoff, `sys.setswitchinterval` replacing `sys.setcheckinterval`. — *pinned* — <https://github.com/python/cpython/issues/7900> and Pitrou's python-dev posts archived at <https://mail.python.org/pipermail/python-dev/2009-October/093326.html>

5. **Python docs — Initialization, Finalization, and Threads (C-API)**
— `PyEval_SaveThread` / `PyEval_RestoreThread` , `PyGILState_Ensure` / `Release` , `Py_BEGIN_ALLOW_THREADS` , `PyEval_InitThreads` , `PyThreadState` , `PyInterpreterState` — <https://docs.python.org/3/c-api/init.html#thread-state-and-the-global-interpreter-lock> — *pinned* — <https://docs.python.org/stable/c-api/init.html>
6. **PEP 683 — Immortal Objects, Using a Fixed Refcount** — Mark Shannon — `_Py_IMMORTAL_REFCNT` sentinel, why immortal singletons (`None` , `True` , small ints) are essential for free-threaded cache-line behavior. — <https://peps.python.org/pep-0683/>
7. **PEP 734 — Multiple Interpreters in the Stdlib (`concurrent.interpreters`)** — Eric Snow — stabilizes PEP 554 as `concurrent.interpreters` , `Queue` , `is_shareable` , execution model for 3.13+. — <https://peps.python.org/pep-0734/>
8. **David Beazley — Understanding the Python GIL (PyCon 2010)**
— the talk that showed the old GIL's priority inversion, multi-core slowdown, and I/O starvation with live demos. Slides and recording: search "Beazley GIL PyCon 2010". — <https://www.dabeaz.com/GIL/>
9. **CPython source** — `Python/ceval_gil.c` / `Python/ceval_gil.h` / `Include/internal/pycore_gil.h` — `take_gil` , `drop_gil` , `struct _gil_runtime_state` , `GIL_DROP_REQUEST` , `eval_breaker` handling. Start at `take_gil` and follow `drop_gil` / `COND_TIMED_WAIT` . — https://github.com/python/cpython/blob/main/Python/ceval_gil.c
10. **CPython docs — What's New in Python 3.13 — Free-threaded CPython** — `Py_GIL_DISABLED` , `sys._is_gil_enabled()` , `--disable-gil` build flag, per-module `Py_mod_gil` status for stdlib. — <https://docs.python.org/3/whatsnew/3.13.html#free-threaded-cpython>
11. **Sam Gross — nogil / free-threaded Python — py-free-threading.github.io** — tracking of `Py_GIL_DISABLED` porting status

per package, `pyperformance` deltas, and `mimalloc` / biased
refcounting deep dives. — <https://py-free-threading.github.io/>

Chapter 6 — The Type System: Classes, MRO, Descriptors, Slots, and the Attribute Protocol

What this chapter covers. Python's type system looks deceptively simple at the surface — `class Foo(Bar):` and off you go — but underneath it is a carefully engineered C-level machine that resolves every `obj.attr`, `obj.method()`, and `isinstance(x, Y)` you write. The core is three interlocking mechanisms: `PyTypeObject` (every type is a C struct that is itself an object), the C3-linearized Method Resolution Order that turns multiple inheritance into a deterministic search path, and the descriptor protocol that makes functions become methods, `property` work, and `__slots__` save memory. This chapter opens all three. You will read the actual `PyTypeObject` layout from `Include/object.h` and `Include/cpython/object.h`, trace `type.__new__` and metaclass construction, linearize diamond hierarchies by hand, walk the full attribute-lookup chain that CPython executes on every dot, implement descriptors from scratch, dissect `__slots__` at the memory-layout level, and see how abstract base classes cheat the `isinstance` check. A running thread throughout: what this costs on the hot path of a backend service doing millions of attribute lookups per second.

Learning goals — after this chapter you should be able to:

- Read `PyTypeObject` in `Include/cpython/object.h` and explain every critical slot: `tp_name`, `tp_bases`, `tp_mro`, `tp_basicsize`, `tp_dictoffset`, `tp_flags`, `tp_descr_get`, `tp_alloc`, `tp_new`, the `tp_as_*` sub-tables, and the `PyType_Spec` / `Py_tp_*` modern definition API.
- Trace type creation end-to-end: `class` statement → `type.__new__(metaclass, name, bases, dict)` → `PyType_Ready` → `tp_mro` computation → insertion into `type`'s own MRO, and explain what metaclasses intercept.

- Compute C3 linearization by hand for arbitrary hierarchies, predict `__mro__`, diagnose `TypeError: Cannot create a consistent method resolution order`, and use `cls.mro()` / `inspect.getmro` / `__mro__` correctly.
- Walk the complete attribute-lookup protocol CPython implements in `Objects/object.c:PyObject_GenericGetAttr` and `typeobject.c: data descriptor → instance __dict__ → non-data descriptor / plain class attribute → __getattr__`, and explain why data descriptors shadow instance dict entries but non-data descriptors do not.
- Implement correct descriptors (`__get__` / `__set__` / `__delete__` / `__set_name__`), distinguish data vs. non-data descriptors, and explain how `property`, `classmethod`, `staticmethod`, `function`, and slot descriptors map onto the protocol.
- Contrast `__getattr__` vs. `__getattribute__` vs. metaclass `__getattr__` / `__getattribute__`, predict when each fires, and avoid infinite-recursion traps.
- Explain `__slots__` at the C level (`tp_dictoffset == 0`, `PyMemberDef` slot descriptors, `tp_members`), quantify its memory and speed tradeoffs, and decide when to use it in backend data models vs. `dataclasses` / `__dict__`.
- Describe `tp_slots` / `PyType_Slot` for special methods (`tp_call`, `tp_iter`, `tp_descr_get`, `tp_as_number.nb_add`, etc.), how `PyType_Ready` wires `__dunder__` names to C slots, and why special-method lookup bypasses instance `__dict__`.
- Explain ABCs (`collections.abc`, `abc.ABCMeta`), virtual subclasses (`register` / `__subclasshook__`), and the `Py_TPFLAGS_IMMUTABLETYPE` / `Py_TPFLAGS_HEAPTYPE` distinction.
- Reason about hot-path attribute-lookup cost and apply the knowledge to ORM field descriptors, validated data models, and high-throughput request objects.

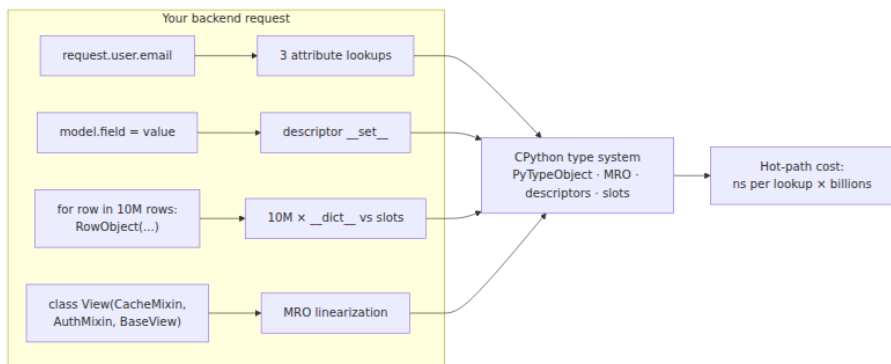
1. Why the type system is a backend problem

Every HTTP handler your service runs does attribute lookups at a staggering rate. A single `request.user.email` is three lookups, each traversing the chain described in this chapter. An ORM model that validates on assignment fires a descriptor `__set__` on every field write. A

dataclass-heavy domain layer that creates millions of short-lived objects pays for `__dict__` allocation on each one — or avoids it with `__slots__`. The type system is the path your data takes.

Concrete reasons senior backend engineers should understand this layer:

- **ORMs and validation libraries live on descriptors.** Django fields, SQLAlchemy columns, Pydantic `FieldInfo`, `attrs` validators — all are descriptors or descriptor-adjacent. Knowing data vs. non-data descriptor priority explains why a field default can or cannot be shadowed by instance assignment.
- **Memory footprint of data models is a capacity decision.** A service that holds 10 million row-objects in memory (event buffers, feature stores, queue consumers) sees a 40–60% memory reduction from `__slots__` — or a flexibility cost that breaks serialization. The tradeoff is quantitative, not stylistic.
- **MRO bugs are production bugs.** Diamond hierarchies appear whenever you mix mixins (`LoginRequiredMixin`, `CacheMixin`, `MetricsMixin`) with framework base classes. An MRO conflict is a deploy-time `TypeError`; a silent MRO surprise is a method resolving to the wrong parent in production.
- **Metaclasses power frameworks you depend on.** Django models, SQLAlchemy declarative base, Pydantic `ModelMetaclass`, `ABCMeta` — all intercept class creation via `type.__new__`. Debugging "why does my model have a `_sa_instance_state` attribute I never declared" requires reading the metaclass.
- **Special-method dispatch has a fast path that bypasses your Python.** `obj + other` does not do `getattr(obj, "__add__")`; it reads `obj->ob_type->tp_as_number->nb_add` directly. Overriding `__getattr__` will not intercept `__add__` lookup — a common surprise when building proxy objects for RPC or tracing.



2. PyTypeObject — types are objects that describe objects

2.1 Everything is a PyObject, including types

Chapter 2 introduced `PyObject_HEAD` — the `ob_refcnt` + `ob_type` prefix every value carries. `PyTypeObject` is the struct that `ob_type` points to. It is itself a `PyObject` (via `PyObject_VAR_HEAD`), so types have a type: `type`.

```

/* Include/cpython/object.h – PyObject (Python 3.12, abbreviated)
*/
/* The real struct has ~35 named fields plus the variable-length base
*/

typedef struct _typeobject {
    PyObject_VAR_HEAD                /* ob_base: ob_refcnt +
    ob_type + ob_size */
    const char *tp_name;             /* "int", "list",
    "myapp.models.User" */

    Py_ssize_t tp_basicsize;         /* sizeof instance, e.g.
    sizeof(PyLongObject) */
    Py_ssize_t tp_itemsize;         /* 0 for fixed-size; >0 for
    var objects */

    /* --- deallocation & GC --- */
    destructor tp_dealloc;           /* called when refcnt → 0 */
    Py_ssize_t tp_vectorcall_offset; /* offset for vectorcall (PEP
590) */
    getattrofunc tp_getattr;        /* legacy, almost always NULL
*/
    setattrofunc tp_setattr;        /* legacy */
    PyAsyncMethods *tp_as_async;    /* __await__, __aiter__, ...
*/
    reprfunc tp_repr;              /* __repr__ slot */

    PyNumberMethods *tp_as_number;  /* nb_add, nb_multiply, ... */
    PySequenceMethods *tp_as_sequence; /* sq_item, sq_length,
sq_concat, ... */
    PyMappingMethods *tp_as_mapping; /* mp_subscript,
mp_ass_subscript, ... */

    hashfunc tp_hash;              /* __hash__ */
    ternaryfunc tp_call;           /* __call__ – why types are
callable */
    reprfunc tp_str;               /* __str__ */
    getattrofunc tp_getattro;      /* attribute getter (usually
GenericGetAttr) */
    setattrofunc tp_setattro;      /* attribute setter */
    PyBufferProcs *tp_as_buffer;   /* buffer protocol */

    unsigned long tp_flags;        /* Py_TPFLAGS_* bitfield */

    const char *tp_doc;            /* __doc__ */
    traverseproc tp_traverse;       /* GC traversal */
    inquiry tp_clear;              /* GC clear */
    richcmpfunc tp_richcompare;    /* __eq__, __lt__, ... */
    Py_ssize_t
tp_weaklistoffset;               /* offset to weakref list head */

```

```

    getiterfunc tp_iter;                /* __iter__ */
    iternextfunc tp_iternext;           /* tp_iternext for iterators */
*/

    struct PyMethodDef *tp_methods;     /* methods defined in C */
    struct PyMemberDef
*tp_members;                          /* C struct members exposed as attributes */
    struct PyGetSetDef *tp_getset;     /* getset descriptors
(property-like in C) */
    struct _typeobject *tp_base;       /* single primary base:
tp_bases[0] */
    PyObject
*tp_dict;                             /* type's __dict__ - attribute namespace */
    descrgetfunc
tp_descr_get;                         /* descriptor __get__ at the C level */
    descrsetfunc tp_descr_set;         /* descriptor __set__ */
    Py_ssize_t tp_dictoffset;         /* offset to instance
__dict__, or 0 if slotted */
    initproc tp_init;                 /* __init__ */
    allocfunc tp_alloc;               /* instance allocator */
    newfunc tp_new;                   /* __new__ */
    freefunc tp_free;                 /* instance deallocator */
    inquiry tp_is_gc;                 /* participates in GC? */
    PyObject *tp_bases;               /* tuple of all bases */
    PyObject *tp_mro;                 /* tuple of MRO */
    PyObject *tp_cache;               /* internal */
    PyObject *tp_subclasses;          /* weak set of subclasses */
    PyObject *tp_weaklist;            /* weakrefs to this type */
    destructor tp_del;                /* __del__ */
    unsigned int tp_version_tag;       /* MRO version for caching */
    destructor
tp_finalize;                          /* __del__ finalizer (PEP 442) */
    vectorcallfunc tp_vectorcall;     /* fast call path */
} PyTypeObject;

```

Key fields for backend work:

Field	What it controls
<code>tp_name</code>	Dotted name used in <code>repr(type)</code> and <code>_PyType_Lookup</code> diagnostics.
<code>tp_basicsize</code> / <code>tp_itemsize</code>	Instance allocation size. <code>tp_itemsize > 0</code> means <code>PyVarObject</code> trailing array (tuples, lists at the type level).

Field	What it controls
<code>tp_flags</code>	Bitfield: <code>Py_TPFLAGS_HEAPTYPE</code> (created by <code>class</code> statement, heap-allocated type), <code>Py_TPFLAGS_BASETYPE</code> (can be subclassed), <code>Py_TPFLAGS_HAVE_GC</code> , <code>Py_TPFLAGS_IMMUTABLETYPE</code> (3.10+, type dict is immutable), <code>Py_TPFLAGS_MANAGED_DICT</code> (3.12+, per-type managed dict).
<code>tp_dict</code>	The type's namespace. In 3.12+ this may be a managed dict (split-table, single copy). Reading <code>MyClass.attr</code> searches this dict, then each base's <code>tp_dict</code> along <code>tp_mro</code> .
<code>tp_mro</code> / <code>tp_bases</code> / <code>tp_base</code>	MRO tuple, bases tuple, and the single-inheritance fast path pointer.
<code>tp_dictoffset</code>	Byte offset from instance start to its <code>__dict__</code> pointer. <code>0</code> means "no <code>__dict__</code> " — the <code>__slots__</code> case.
<code>tp_descr_get</code> / <code>tp_descr_set</code>	If non-NULL, this type's instances act as descriptors. How <code>function</code> , <code>property</code> , and <code>member_descriptor</code> plug in.
<code>tp_as_number</code> / <code>tp_as_sequence</code> / <code>tp_as_mapping</code> / ...	Sub-tables for special methods. <code>PyType_Ready</code> wires <code>__add__</code> → <code>tp_as_number->nb_add</code> etc.
<code>tp_getattro</code> / <code>tp_setattro</code>	The attribute-access entry points. Almost every type uses <code>PyObject_GenericGetAttr</code> / <code>PyObject_GenericSetAttr</code> . Overriding <code>__getattribute__</code> replaces <code>tp_getattro</code> with a wrapper.

Modern C extensions do not fill `PyTypeObject` by hand. They use the `PyType_Spec` / `PyType_Slot` API (`PyType_FromSpec`):

```

/* Modern style - Objects/typeobject.c, any C extension since 3.2 */
static PyType_Slot MyObject_slots[] = {
    {Py_tp_doc,          "MyObject - example heap type"},
    {Py_tp_new,          MyObject_new},
    {Py_tp_init,         MyObject_init},
    {Py_tp_dealloc,      MyObject_dealloc},
    {Py_tp_repr,         MyObject_repr},
    {Py_nb_add,          MyObject_add},          /* → tp_as_number->nb_add */
};

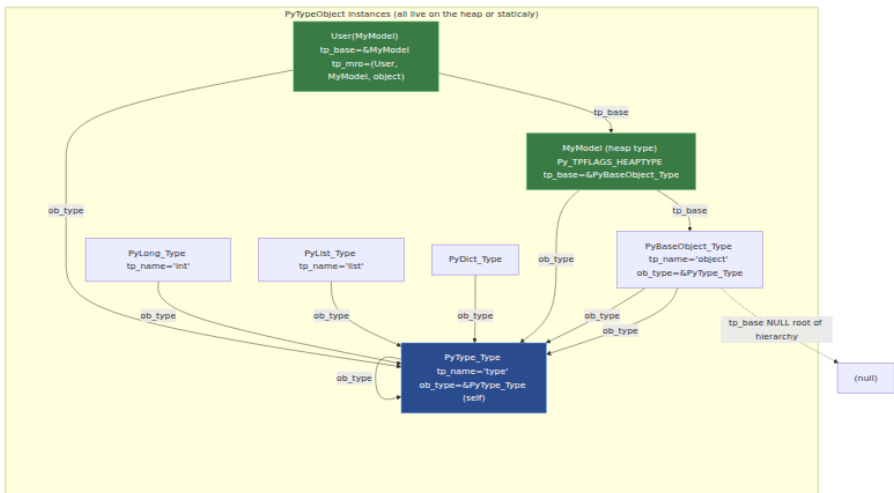
/*
    {Py_sq_length,       MyObject_len},          /* → tp_as_sequence-
>sq_length */
    {Py_tp_methods,      MyObject_methods},
    {Py_tp_members,      MyObject_members},
    {0, NULL}
};

static PyType_Spec MyObject_spec = {
    .name = "mymod.MyObject",
    .basicsize = sizeof(MyObject),
    .flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
    .slots = MyObject_slots,
};

/* In module init: PyType_FromSpec(&MyObject_spec) → PyType_Ready
internally */

```

2.2 The type hierarchy — who points at whom



Critical invariant: `type` is its own type (`PyType_Type.ob_type == &PyType_Type`), bootstrapped in `Objects/typeobject.c:PyType_Ready` at interpreter startup. Every heap type (created by `class`) has

`Py_TPFLAGS_HEAPTYPE` set, owns its `tp_dict`, and appears in its base's `tp_subclasses` weak set — which is how `__subclasses__()` works.

3. Type creation — `type.__new__` and metaclasses

3.1 What `class` actually executes

```
class User(Model):
    name: str
    email: str
```

is not syntax sugar for a dict assignment. The interpreter executes roughly:

```
# What CPython does for "class User(Model): ..." (Python/ceval.c +
# bltinmodule.c)
User = type.__new__(type, "User", (Model,), {"name": ...,
"email": ..., "__module__": ..., "__qualname__": "User"})
# type.__new__ internally calls PyType_Ready-equivalent for heap types:
# - allocates PyTypeObject (heap)
# - copies bases → tp_bases, computes tp_mro via C3
# - creates tp_dict from the namespace dict
# - wires tp_as_number / tp_as_sequence etc. from dunder names
# - calls metaclass.__init__(User, "User", (Model,), namespace)
```

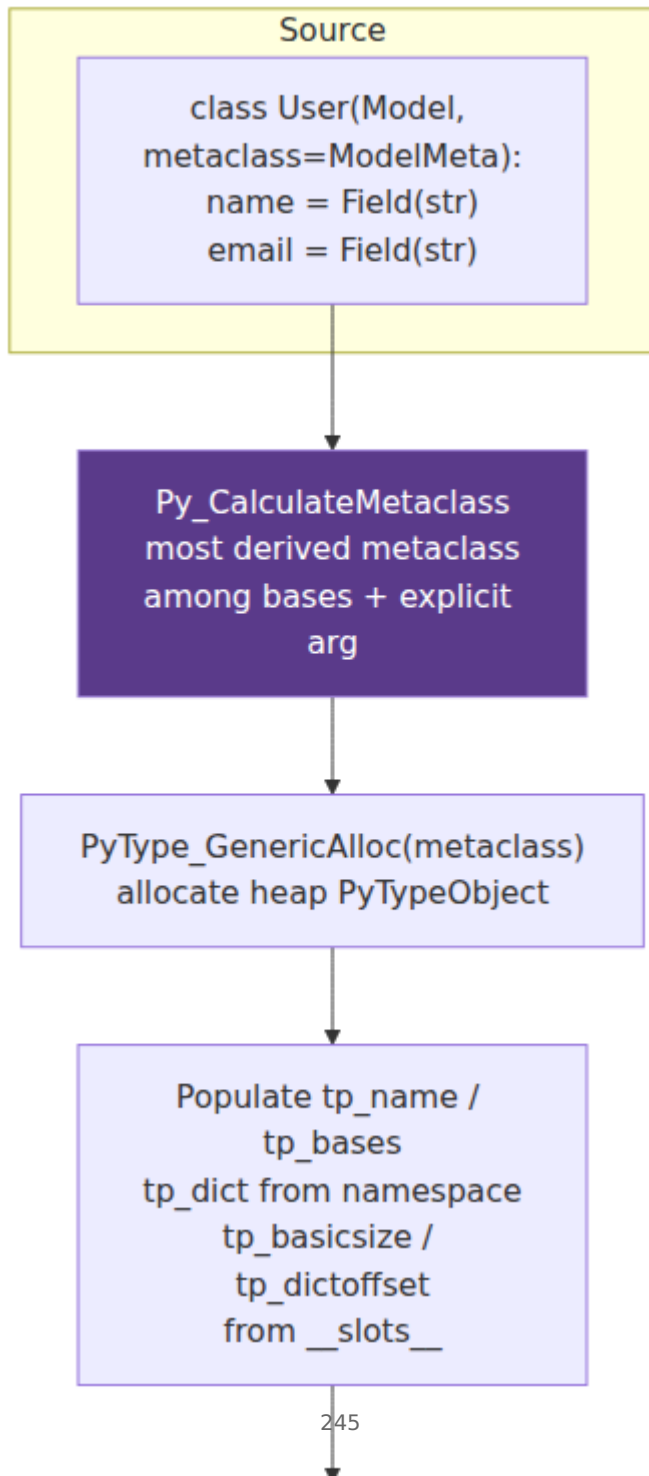
`type.__new__` is `type_new` in `Objects/typeobject.c` (~400 lines). Simplified steps:

1. **Validate** — `name` is `str`, `bases` is tuple of types, `dict` is dict.
2. **Determine metaclass** — `Py_CalculateMetaclass(metatype, bases)` picks the most derived metaclass among bases; an explicit `metaclass=` keyword argument wins if it is a subtype of the calculated one, else `TypeError`.
3. **Allocate** — `PyType_GenericAlloc(metaclass, 0)` allocates a new `PyTypeObject` on the heap.
4. **Populate** — `tp_name`, `tp_bases`, `tp_base`, `tp_dict`, `tp_basicsize` (from `__slots__` or default), `tp_dictoffset`, `tp_flags`.
5. **Ready** — `PyType_Ready(new_type)` computes MRO, installs `tp_getattro` / `tp_setattro` if not already set, processes `tp_methods` / `tp_members` / `tp_getset` into `tp_dict` descriptors, links `tp_subclasses`.

6. **Init** — calls `metaclass.__init__(new_type, name, bases, dict)`
(usually `type.__init__` which is a no-op).

3.2 Metaclasses — intercepting class creation

A metaclass is just a subclass of `type` whose `__new__` / `__init__` intercepts step 2-6 above. Every class has a metaclass; if you do not declare one it is `type`.



```

# Minimal ORM-style metaclass – the pattern behind Django/SQLAlchemy/
Pydantic
class Field:
    """Descriptor that will become a slot descriptor after metaclass
    processing."""
    def __init__(self, dtype, default=None):
        self.dtype = dtype
        self.default = default
        self.name = None
    def __set_name__(self, owner, name):
        self.name = name
        self.private_name = f"_{name}"
    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        # access via class → return descriptor itself
        return getattr(obj, self.private_name, self.default)
    def __set__(self, obj, value):
        if not isinstance(value, self.dtype):
            raise TypeError(f"{self.name} expects
{self.dtype.__name__}, got {type(value).__name__}")
        object.__setattr__(obj, self.private_name, value)

class ModelMeta(type):
    """Collect Field descriptors and optionally inject __slots__."""
    def __new__(mcls, name, bases, namespace):
        fields = {k: v for k, v in namespace.items() if isinstance(v, F
ield)}
        # Give each Field its name (also done automatically via
        __set_name__ after creation,
        # but metaclass can do extra bookkeeping)
        cls = super().__new__(mcls, name, bases, namespace)
        cls._fields = fields # registry used by save()/validate()/
        serialize()
        return cls

class Model(metaclass=ModelMeta):
    pass

class User(Model):
    name = Field(str, default="")
    email = Field(str, default="")

# Metaclass has run: User._fields == {"name": ..., "email": ...}
u = User()
u.name = "Ada"
print(User._fields) # {'name': <Field>, 'email': <Field>}
print(u.name) # Ada (descriptor __get__)
try:
    u.email = 42

```

```
except TypeError as e:
    print(e)          # email expects str, got int
```

Key points visible here:

- `Field.__set_name__` (PEP 487, Python 3.6+) is called automatically by `type.__new__` after `tp_dict` is populated — no metaclass needed just for naming. Metaclasses remain useful when you need to *transform* the namespace (inject `__slots__`, wrap methods, build registries).
- Accessing `User.name` returns the descriptor itself (because `obj` is `None` in `__get__`); accessing `u.name` invokes the descriptor on the instance.
- `ModelMeta.__new__` must call `super().__new__` — that is `type.__new__` which does the real `PyObject` allocation and MRO computation. Forgetting it produces a non-type object.

```
# When metaclass selection fails – explicit metaclass must be subtype
of base metaclasses
class MetaA(type): pass
class MetaB(type): pass
class BaseA(metaclass=MetaA): pass
class BaseB(metaclass=MetaB): pass

try:
    class Bad(BaseA, BaseB): # MetaA and MetaB are unrelated
        pass
except TypeError as e:
    print(e)
    # metaclass conflict: the metaclass of a derived class must be a
    # (non-strict) subclass of the metaclasses of all its bases
```

Fix: create a merged metaclass `class MetaAB(MetaA, MetaB): pass` and declare `class Good(BaseA, BaseB, metaclass=MetaAB)`.

4. MRO — C3 linearization

4.1 Why linearization matters

With single inheritance, attribute search is trivial: walk `cls → cls.__base__ → ... → object`. With multiple inheritance the hierarchy is a

DAG, not a chain, and Python must flatten it into a single deterministic order that respects two constraints:

1. **Local precedence** — if `class C(A, B)`, then `A` precedes `B` in `C`'s MRO.
2. **Monotonicity** — if `X` precedes `Y` in any parent's MRO, it precedes `Y` in the child's MRO.

C3 linearization is the algorithm that satisfies both or raises `TypeError` when they conflict.

4.2 The algorithm

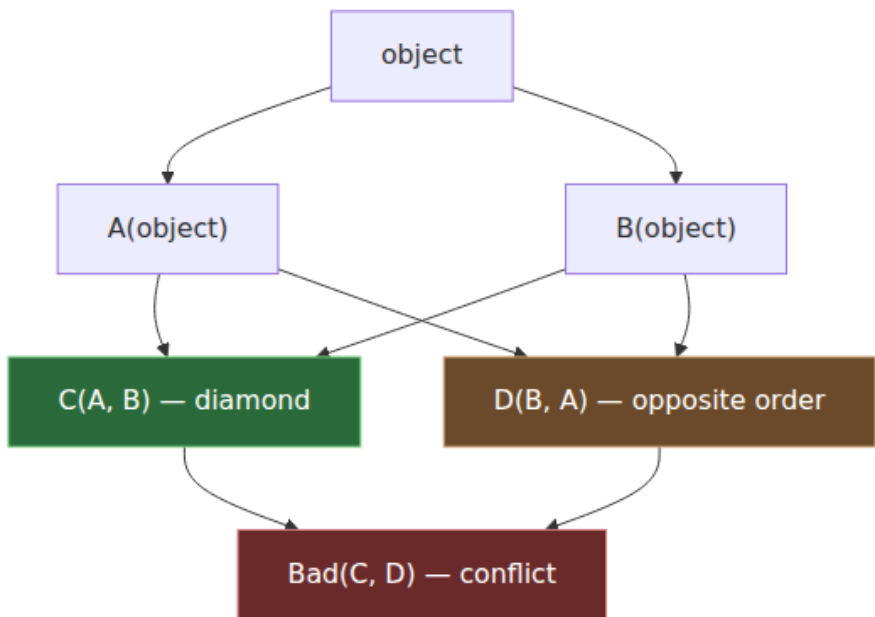
For `class C(A, B)` where `A` and `B` already have linearizations `L[A]`, `L[B]`:

```
L[C] = [C] + merge(L[A], L[B], [A, B])
```

`merge` repeatedly picks the first head that does not appear in the tail of any other list, appends it, and removes it from all lists. If no such head exists, the hierarchy is inconsistent.

CPython implements this in `Objects/typeobject.c:mro_internal` and `pmerge`.

4.3 Worked diamond



```

# Diamond – the classic case
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass # D's MRO must interleave B and C correctly

print(D.__mro__)
# (<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>,
#  <class '__main__.A'>, <class 'object'>)

# C3 trace for D(B, C):
#  L[B] = [B, A, object]
#  L[C] = [C, A, object]
#  L[D] = [D] + merge([B, A, object], [C, A, object], [B, C])
#  merge step 1: head B not in tail of any list → pick B →
#  ([A,object],[C,A,object],[C])
#  merge step 2: head A is in tail of [C,A,object] → skip A, pick C →
#  ([A,object],[A,object],[C])
#  merge step 3: pick A → ([object],[object])
#  merge step 4: pick object → []
#  Result: [D, B, C, A, object]

# Inspect correctly – three equivalent spellings with different
# semantics:
print(D.mro()) # computes fresh list (calls mro_internal)
print(D.__mro__) # cached tuple from tp_mro – the canonical
# value
import inspect
print(inspect.getmro(D)) # same as __mro__, but works on instances
# too

# Opposite order produces a different MRO
class E(C, B): pass
print(E.__mro__)
# (<class 'E'>, <class 'C'>, <class 'B'>, <class 'A'>, <class
#  'object'>)

# Inconsistent hierarchy – C3 cannot satisfy both orderings
try:
    class Bad(D, E): # D wants B before C, E wants C before B
        pass
except TypeError as e:
    print(f"MRO conflict: {e}")
    # Cannot create a consistent method resolution order (MRO) for
    # bases B, C

```

```

# MRO determines which method runs – silent correctness bug if you get
it wrong
class CacheMixin:
    def save(self): print("CacheMixin.save"); super().save()
class MetricsMixin:
    def save(self): print("MetricsMixin.save"); super().save()
class BaseModel:
    def save(self): print("BaseModel.save")

class MyModel(CacheMixin, MetricsMixin, BaseModel): pass

print([c.__name__ for c in MyModel.__mro__])
# ['MyModel', 'CacheMixin', 'MetricsMixin', 'BaseModel', 'object']
MyModel().save()
# CacheMixin.save
# MetricsMixin.save
# BaseModel.save

# Swap mixin order → different behavior
class MyModel2(MetricsMixin, CacheMixin, BaseModel): pass
print([c.__name__ for c in MyModel2.__mro__])
# ['MyModel2', 'MetricsMixin', 'CacheMixin', 'BaseModel', 'object']

# Cooperative multiple inheritance via super() walks the MRO, not the
parent.
# Each save() calls super().save() which resolves to the NEXT class in
MRO.

```

C3 properties that matter operationally:

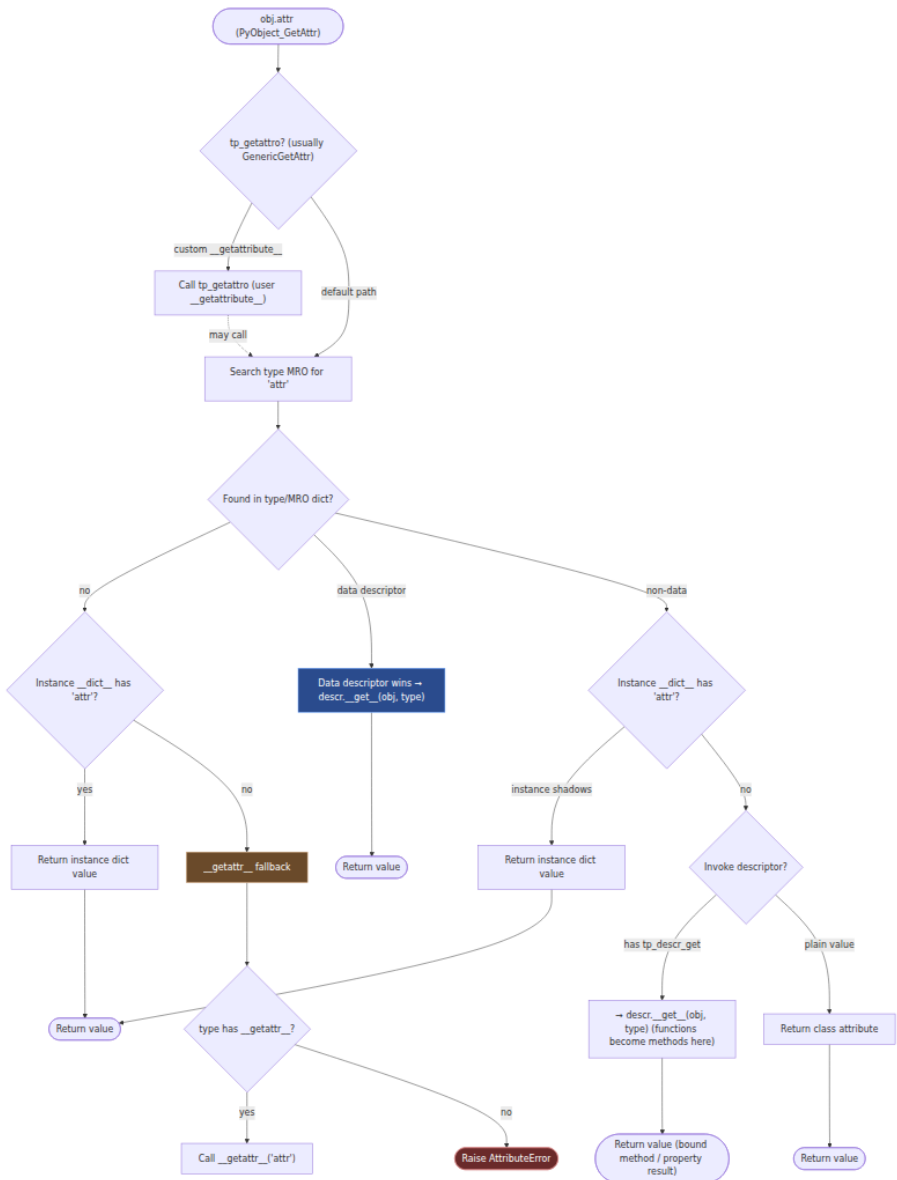
- Every class appears exactly once in its MRO, and `object` is always last (except for `object` itself).
- `cls.__mro__` is a tuple stored in `PyTypeObject.tp_mro` and is immutable after creation. `cls.mro()` returns a fresh list — useful for metaclasses that want to compute a custom order before `tp_mro` is frozen, but do not mutate the returned list expecting `__mro__` to change.
- `super()` with no arguments (PEP 3135) captures `__class__` from the enclosing scope and walks `__class__.__mro__` starting after the current class. `super(CacheMixin, self).save()` is the explicit form — same MRO walk with a different start point.
- The MRO version tag (`tp_version_tag`) invalidates CPython's type-lookup caches (`type_getattro` inline cache, `LOAD_GLOBAL` cache, `LOAD_ATTR` specialization) whenever any class in the hierarchy

mutates. Hot-loop attribute lookup benefits from a stable tag; churning `tp_dict` entries in a loop defeats the cache.

5. The attribute protocol — the full lookup chain

Every `obj.attr` and `type(obj).attr` lookup executes a precise sequence in `PyObject_GenericGetAttr` (`Objects/object.c`) and `type_getattro` (`Objects/typeobject.c`). The difference between "instance attribute" and "class attribute" is not where the value is stored but where the lookup finds it.

5.1 Instance attribute lookup (`object.__getattr__` default)



The priority order, stated precisely:

1. **Data descriptor on the type/MRO** — if `type(obj).__dict__['attr']` (or any base's dict along `tp_mro`) has `tp_descr_set != NULL` (i.e., defines `__set__` or `__delete__`), call its

`tp_descr_get` immediately. The instance dict is not consulted. This is why `property` without a setter still shadows instance dict, and `property` with a setter *always* does.

2. **Instance `__dict__`** — if `obj.__dict__['attr']` exists, return it. This is the fast path for plain instance attributes. CPython checks `tp_dictoffset` to find `__dict__`; if `tp_dictoffset == 0` there is no instance dict (slots-only object).
3. **Non-data descriptor / plain class attribute** — if the MRO entry has `tp_descr_get != NULL` but `tp_descr_set == NULL` (non-data descriptor: functions, `classmethod`, `staticmethod` without `__set__`), call `tp_descr_get`. Otherwise return the plain value.
4. **`__getattr__` fallback** — if nothing was found in steps 1-3, and the type defines `__getattr__` (via `tp_getattro` chaining or `__getattr__` in the MRO), call it with the attribute name.
5. **`AttributeError`** — if `__getattr__` is absent or raises.

This ordering is why descriptors can implement computed fields that *cannot* be accidentally shadowed (data descriptors), and also methods that *can* be shadowed by per-instance assignment (non-data descriptors — functions).

5.2 Class attribute lookup (`type.__getattribute__`)

Attribute access on a class (`MyClass.attr`) uses `type_getattro` in `Objects/typeobject.c`, not `PyObject_GenericGetAttr`. The search order is:

```
type(MyClass).__mro__  MyClass.__mro__
```

That is: search the metaclass MRO first (so `type.__getattribute__` and metaclass descriptors can intercept), then the class MRO. Metaclass data descriptors shadow class `__dict__` entries the same way instance ones do. `MyClass.__getattr__` in this context is `type.__getattr__` — the metaclass's fallback.

5.3 Tracing a lookup with real code

```

class LoggingDescr:
    """Data descriptor that logs every access."""
    def __set_name__(self, owner, name): self.name = name
    def __get__(self, obj, objtype=None):
        print(f"  descr __get__({self.name!r}, obj={obj!r})")
        if obj is None: return self
        return obj.__dict__.get(f"_{self.name}", f"<default {self.name}>")

    def __set__(self, obj, value):
        print(f"  descr __set__({self.name!r}, {value!r})")
        obj.__dict__[f"_{self.name}"] = value
    def __delete__(self, obj):
        print(f"  descr __delete__({self.name!r})")
        obj.__dict__.pop(f"_{self.name}", None)

class NonDataDescr:
    def __get__(self, obj, objtype=None):
        print(f"  non-data __get__(obj={obj!r})")
        if obj is None: return self
        return 42

class Demo:
    data = LoggingDescr()
    non_data = NonDataDescr()
    plain = "class plain value"
    def __getattr__(self, name):
        print(f"  __getattr__({name!r})")
        return f"<getattr:{name}>"

d = Demo()
print("--- data descriptor shadows instance dict ---")
d.__dict__["data"] = "shadow attempt"  # goes into instance dict directly
print(f"d.data = {d.data!r}")          # still goes through descriptor!
# descr __get__('data', obj=<Demo>)
# shadows? No – data descriptor wins per step 1.

print("\n--- non-data descriptor is shadowed by instance dict ---")
print(f"d.non_data = {d.non_data!r}")  # 42 via descriptor
# non-data __get__(obj=<Demo>)
d.__dict__["non_data"] = "shadowed"
print(f"d.non_data (after shadow) = {d.non_data!r}")  # "shadowed" – instance dict wins

print("\n--- plain class attr is shadowed ---")
print(f"d.plain = {d.plain!r}")         # "class plain value"
d.plain = "instance plain"
print(f"d.plain (after shadow) = {d.plain!r}")  # "instance plain"

```



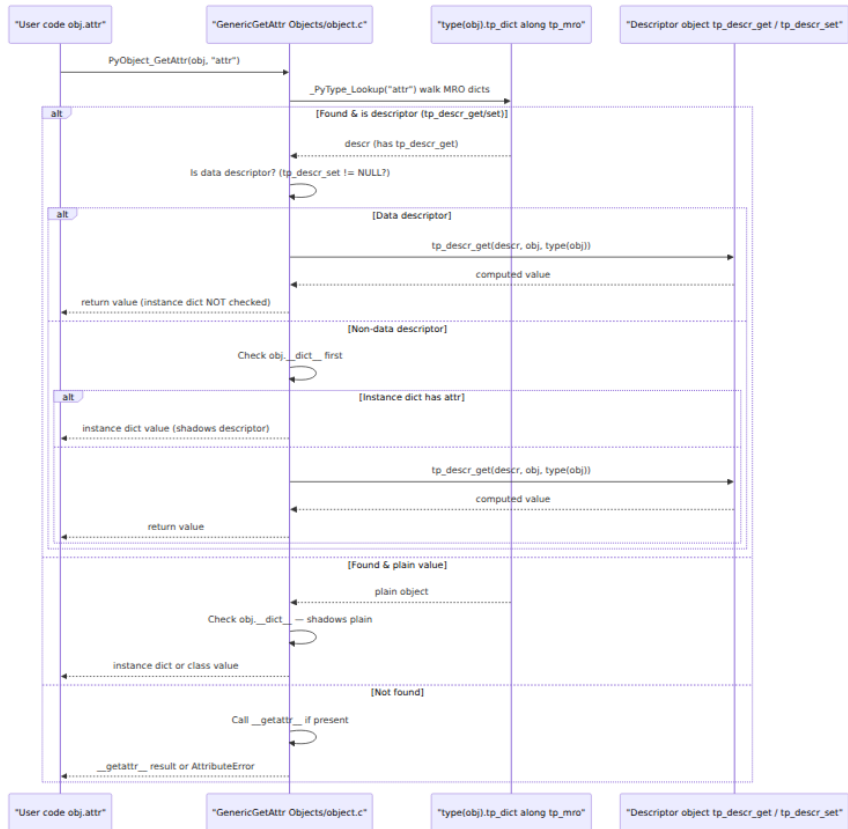
```
print("\n--- __getattr__ only fires on full miss ---")
print(f"d.missing = {d.missing!r}")
# __getattr__('missing')
print(f"d.data still does not call __getattr__") # descriptor found,
no fallback
```

6. Descriptors — the protocol behind `property`, methods, and slots

6.1 The descriptor protocol

A descriptor is any object that defines at least one of `__get__`, `__set__`, `__delete__`. At the C level this means `tp_descr_get` or `tp_descr_set` is non-NULL.

```
# The protocol in pure Python (CPython calls tp_descr_get/tp_descr_set
at the C level)
class Descriptor:
    def __get__(self, obj, objtype=None) -> object: ...
    def __set__(self, obj, value) -> None: ...
    def __delete__(self, obj) -> None: ...
    def __set_name__(self, owner, name) -> None: ...
# PEP 487, called at class creation
```



The classification:

Descriptor kind	Has <code>__get__</code>	Has <code>__set__</code> / <code>__delete__</code>	Priority vs. instance <code>__dict__</code>
Data descriptor	yes (usually)	yes	Wins over instance dict. Instance assignment goes through <code>__set__</code> , never creates a dict entry shadowing it.
Non-data descriptor	yes	no	Loses to instance dict. Assignment creates a dict entry that shadows future lookups.

Descriptor kind	Has __get__	Has __set__ / __delete__	Priority vs. instance __dict__
Plain attribute	no	no	Loses to instance dict. Not a descriptor at all.

6.2 `property`, `classmethod`, `staticmethod` — all descriptors

```
# property as a data descriptor – CPython's Lib version simplified
class my_property:
    def __init__(self, fget=None, fset=None, fdel=None, doc=None):
        self.fget, self.fset, self.fdel = fget, fset, fdel
        if doc is None and fget is not None: doc = fget.__doc__
        self.__doc__ = doc
    def __get__(self, obj, objtype=None):
        if obj is None: return self
        if self.fget is None: raise AttributeError("unreadable
attribute")
        return self.fget(obj)
    def __set__(self, obj, value):
        if self.fset is None: raise AttributeError("can't set
attribute")
        self.fset(obj, value)
    def __delete__(self, obj):
        if self.fdel is None: raise AttributeError("can't delete
attribute")
        self.fdel(obj)
    def getter(self, fget): return type(self)(fget, self.fset, self.fde
l, self.__doc__)
    def setter(self, fset): return type(self)(self.fget, fset, self.fde
l, self.__doc__)
    def deleter(self, fdel): return type(self)(self.fget, self.fset, fd
el, self.__doc__)

# Real usage – note property is a DATA descriptor (has __set__), so it
always wins.
class Temperature:
    def __init__(self, celsius=0):
        self._c = celsius
    @my_property
    def celsius(self): return self._c
    @celsius.setter
    def celsius(self, v):
        if v < -273.15: raise ValueError("below absolute zero")
        self._c = v
    @my_property
    def fahrenheit(self): return self._c * 9/5 + 32
    @fahrenheit.setter
    def fahrenheit(self, v): self.celsius = (v - 32) * 5/9

t = Temperature(25)
print(t.fahrenheit) # 77.0 – property __get__
t.fahrenheit = 212
print(t.celsius)    # 100.0 – property __set__ → celsius setter
```

```

# Functions are non-data descriptors – that is how methods bind.
class FunctionDescr:
    """Simplified model of function.__get__ (Objects/
funcobject.c:func_descr_get)."""
    def __init__(self, func): self.func = func
    def __get__(self, obj, objtype=None):
        if obj is None:
            return self.func                # unbound access: Cls.method →
raw function
        # Bound method: CPython creates a PyMethodObject that holds
(func, obj, type)
        import types
        return types.MethodType(self.func, obj)

class MyClass:
    def method(self, x): return f"{self!r} + {x}"

# What CPython does on d.method:
# 1. Looks up "method" in MyClass.__dict__ → finds function object
(has tp_descr_get, no tp_descr_set → non-data)
# 2. Instance dict has no "method" → calls function.__get__(d,
MyClass) → bound method
# 3. Calling the bound method prepends `d` as first argument.
print(type(MyClass.__dict__["method"])) # <class 'function'>
print(type(MyClass.method))              # <class 'function'> (obj is
None → raw function)
print(type(MyClass().method))            # <class 'method'> (bound)

# Prove non-data: instance assignment shadows the function
obj = MyClass()
obj.method = lambda x: f"shadowed {x}"
print(obj.method("hi")) # shadowed hi – instance dict won

```

`classmethod` and `staticmethod` are also descriptors, with opposite binding behavior:

```

# How classmethod / staticmethod descriptors work (Objects/
funcobject.c)
class MyService:
    _registry = {}

    @classmethod
    def from_config(cls, cfg):          # cls is MyService (or
subclass)
        return cls(**cfg)

    @staticmethod
    def validate(cfg):                 # no implicit first arg
        return isinstance(cfg, dict)

# classmethod.__get__ binds the CLASS, not the instance:
# MyService.from_config → classmethod.__get__(None, MyService) →
bound-to-class method
# MyService().from_config → classmethod.__get__(instance, MyService)
→ same
# staticmethod.__get__ returns the raw function unchanged regardless of
obj/objtype
print(MyService.from_config)         # <bound method MyService.from_config>
print(MyService().from_config)       # <bound method
MyService.from_config> (same cls)
print(MyService.validate)           # <function validate> - no binding

```

6.3 Slot descriptors — C-level data descriptors for `__slots__`

When a class uses `__slots__`, CPython creates a `member_descriptor` (a.k.a. slot descriptor) for each slot name. These are C-level data descriptors whose `tp_descr_get` / `tp_descr_set` read and write directly into the instance's memory at a fixed offset — no `__dict__` involved.

```

class Slotted:
    __slots__ = ("x", "y")
    def __init__(self, x, y): self.x, self.y = x, y

print(type(Slotted.__dict__["x"]))  # <class 'member_descriptor'>
print(hasattr(Slotted.__dict__["x"], "__get__")) # True
print(hasattr(Slotted.__dict__["x"], "__set__")) # True - data
descriptor

s = Slotted(1, 2)
print(s.x)    # 1 - slot descriptor __get__ reads offset in instance
memory
s.x = 99
print(s.x)    # 99 - slot descriptor __set__ writes at offset

```

Slot descriptors are data descriptors, so `s.__dict__["x"] = "shadow"` cannot shadow them — though slotted instances have no `__dict__` by default anyway.

7. `__slots__` — trading flexibility for density and speed

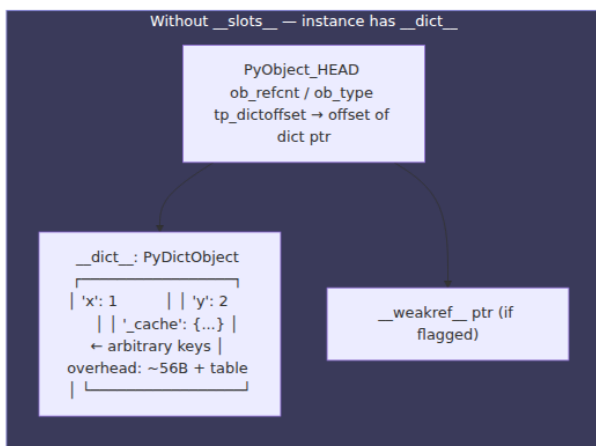
7.1 What `__slots__` changes in `PyObject`

Without `__slots__`:

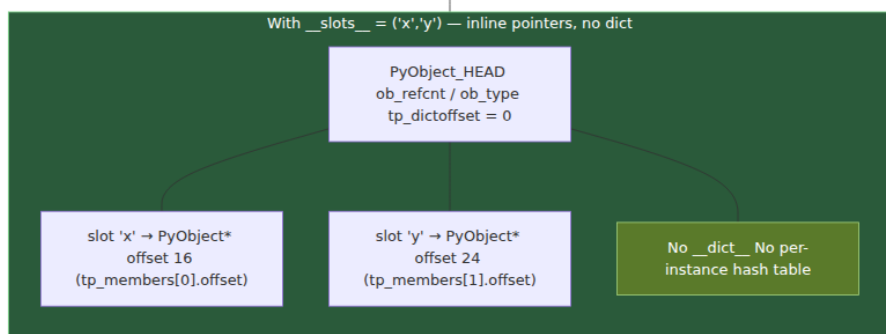
- `tp_dictoffset != 0` — each instance carries a `PyObject*` pointer to its `__dict__`.
- `tp_basicsize` is just `sizeof(PyObject)` plus any C-level fields.
- Attribute assignment creates or updates an entry in the per-instance `dict`.

With `__slots__ = ("x", "y")`:

- `PyType_Ready` allocates one `PyMemberDef` per slot, creates a `member_descriptor` for each, and stores them in `tp_dict`.
- `tp_dictoffset` is set to `0` (no instance `__dict__`) unless `"__dict__"` is explicitly listed in `__slots__` or a base already has one.
- `tp_basicsize` grows by `n_slots * sizeof(PyObject*)` — each slot is a pointer stored inline in the instance struct, right after the `PyObject_HEAD`.
- There is no per-instance dict at all; the instance is essentially a fixed-size struct.



56 bytes dict + 72 bytes table + per-entry overhead



7.2 Memory and speed tradeoffs

Aspect	<code>__dict__</code> (default)	<code>__slots__</code>
Per-instance memory	<code>PyObject_HEAD</code> (16B) + <code>__dict__</code> pointer (8B) + dict object (~56B) + table (~72B for small) + entry overhead (~24B/key)	<code>PyObject_HEAD</code> + <code>n * 8B</code> pointers, no dict. Savings ~40-60% at scale.
Attribute access	<code>dict</code> hash lookup (0(1) but with hash + probe)	Direct pointer at fixed offset — one indirection. Faster, especially for read-heavy hot loops.

Aspect	<code>__dict__</code> (default)	<code>__slots__</code>
Assignment of new names	Any name allowed: <code>obj.anything = v</code>	Only declared slot names (plus <code>__dict__</code> / <code>__weakref__</code> if requested). <code>obj.z = 1</code> raises <code>AttributeError</code> .
Introspection	<code>vars(obj)</code> , <code>obj.__dict__</code> , <code>__dict__.update</code> all work.	No <code>__dict__</code> by default; <code>vars()</code> raises <code>TypeError</code> . Serialization that assumes <code>__dict__</code> breaks.
Weak references	Supported by default.	Need <code>"__weakref__"</code> in <code>__slots__</code> to allow <code>weakref.ref(obj)</code> .
Pickling	Works via <code>__dict__</code> .	Requires <code>__getstate__</code> / <code>__setstate__</code> or relies on slot descriptors; <code>copy</code> / <code>pickle</code> handle it but slower.
Inheritance	Subclass gets its own <code>__dict__</code> unless it also declares <code>__slots__</code> .	Subclass without <code>__slots__</code> regains a <code>__dict__</code> and <code>__weakref__</code> ; memory benefit is partially lost.

7.3 Slots benchmark

```
import sys, timeit, tracemalloc

# --- memory ---
class DictPoint:
    def __init__(self, x, y, z): self.x, self.y, self.z = x, y, z

class SlotPoint:
    __slots__ = ("x", "y", "z")
    def __init__(self, x, y, z): self.x, self.y, self.z = x, y, z

N = 200_000
dict_objs = [DictPoint(1, 2, 3) for _ in range(N)]
slot_objs = [SlotPoint(1, 2, 3) for _ in range(N)]

print(f"DictPoint instance size (incl. __dict__): {sys.getsizeof(dict_o
bjs[0]) + sys.getsizeof(dict_objs[0].__dict__)} bytes")
# ~56 (object) + ~104 (dict + table) ≈ 160
print(f"SlotPoint instance size (no __dict__): {sys.getsizeof(slot_o
bjs[0])} bytes")
# ~56 – just the struct + 3 pointers

tracemalloc.start()
dict_objs2 = [DictPoint(i, i+1, i+2) for i in range(100_000)]
_, peak_dict = tracemalloc.get_traced_memory()
tracemalloc.reset_peak()
tracemalloc.clear_traces()
slot_objs2 = [SlotPoint(i, i+1, i+2) for i in range(100_000)]
_, peak_slot = tracemalloc.get_traced_memory()
tracemalloc.stop()
print(f"Peak tracemalloc 100k DictPoint: {peak_dict/1e6:.1f} MB")
print(f"Peak tracemalloc 100k SlotPoint: {peak_slot/1e6:.1f} MB")
# Expect ~40-55% reduction for slots

# --- speed: attribute read + write ---
print(timeit.timeit("p.x; p.y; p.z", globals={"p": dict_objs[0]}, numbe
r=2_000_000))
print(timeit.timeit("p.x; p.y; p.z", globals={"p": slot_objs[0]}, numbe
r=2_000_000))
# Slots typically 10-20% faster on reads; writes similar. Gap widens
when GC pressure matters.
```

Typical output (CPython 3.12, Linux x86-64):

```

DictPoint instance size (incl. __dict__): 160 bytes
SlotPoint instance size (no __dict__):    56 bytes
Peak tracemalloc 100k DictPoint: 28.4 MB
Peak tracemalloc 100k SlotPoint: 14.9 MB
0.084s  (dict read)
0.069s  (slots read)

```

The 10M-object event-buffer scenario: at 160B vs. 56B per object the difference is ~1 GB vs. ~0.5 GB — enough to avoid an extra replica or to fit in a smaller instance class.

7.4 Hybrid: slots + `__dict__` + `__weakref__`

```

class Hybrid:
    __slots__ = ("x", "y", "__dict__", "__weakref__")
    def __init__(self, x, y, **kw): self.x, self.y = x, y; self.__dict__
    .update(kw)

h = Hybrid(1, 2, label="extra", meta={})
print(h.x)           # 1 – slot descriptor, fixed offset
print(h.label)       # extra – from __dict__
import weakref
r = weakref.ref(h)   # works because __weakref__ slot present
print(r() is h)      # True

# dataclasses with slots (Python 3.10+):
from dataclasses import dataclass

@dataclass(slots=True)
# generates __slots__ automatically, no __dict__
class DCPPoint:
    x: float
    y: float
    label: str = ""

# Equivalent hand-written slots class, but dataclass still generates
__init__ / repr / eq.
print(DCPPoint.__slots__) # ('x', 'y', 'label')
print(hasattr(DCPPoint(1, 2), "__dict__")) # False

```

```

# Inheritance subtlety – child without slots regains a dict
class SlottedBase:
    __slots__ = ("x",)

class ChildNoSlots(SlottedBase):
    pass # no __slots__ → gets __dict__ + __weakref__

c = ChildNoSlots()
c.x = 1 # slot from base
c.extra = 2 # goes into __dict__ – ChildNoSlots has one
print(c.__dict__) # {'extra': 2}

class ChildWithSlots(SlottedBase):
    __slots__ = ("y",) # only declares NEW slots; inherits base's

print(ChildWithSlots.__slots__) # ('y',)
cc = ChildWithSlots()
cc.x = 1; cc.y = 2
print(hasattr(cc, "__dict__")) # False – still no dict

```

8. `__getattr__` VS. `__getattribute__` VS. metaclass hooks

These three names are the most confused corner of the attribute protocol, and the confusion causes infinite-recursion bugs in production proxies and ORMs.

8.1 `__getattribute__` — unconditional interceptor

- Called on **every** attribute access, before any of the lookup steps in §5. Defined as `tp_getattro` at the C level.
- Default is `object.__getattribute__` → `PyObject_GenericGetAttr` (the flowchart in §5).
- If you override it, you replace the entire lookup chain. You must call `super().__getattribute__(name)` or `object.__getattribute__(self, name)` to get the default behavior, or you will recurse forever.

```

class TracingGetAttribute:
    def __init__(self, **kw): self.__dict__.update(kw)
    def __getattr__(self, name):
        # DANGER: `self.__dict__` would call
        # __getattr__('__dict__') → recursion!
        # Use object.__getattr__ to bypass the override.
        print(f"__getattr__({name!r})")
        return object.__getattr__(self, name)

t = TracingGetAttribute(x=1)
print(t.x)
# __getattr__('x') → returns 1 via object.__getattr__

# Infinite recursion anti-pattern:
class Broken:
    def __getattr__(self, name):
        return self.__dict__[name] # BUG: self.__dict__ triggers
        __getattr__('__dict__')
# Broken().anything → RecursionError

```

8.2 `__getattr__` — fallback only

- Called **only** when normal lookup (descriptors → `__dict__` → MRO) found nothing. It is the last step before `AttributeError`.
- It receives only the name that failed. It does not see the object that was searched — just `self` and the string.
- CPython looks for `__getattr__` via `_PyType_Lookup(type(self), "__getattr__")`, then calls it if present. This means `__getattr__` itself is subject to descriptor binding if defined on a class.

```

class LazyModule:
    """Proxy that lazily imports submodules on attribute access –
    pattern used by importlib."""
    def __init__(self, mapping): self._mapping = mapping; self._cache
    = {}
    def __getattr__(self, name):
        # Only called when name is NOT in __dict__ or type MRO – safe
        to check mapping
        if name in self._mapping:
            mod = __import__(self._mapping[name], fromlist=[name])
            self._cache[name] = mod
            return mod
        raise AttributeError(f"no lazy module {name!r}")

lm = LazyModule({"json": "json"})
print(lm.json)          # imports json via __getattr__
print(lm.json)          # second access: found in _cache? No – still via
                        __getattr__ unless you cache in __dict__
# Fix: store in __dict__ so next lookup hits step 2 (instance dict) and
bypasses __getattr__
class LazyModuleCached(LazyModule):
    def __getattr__(self, name):
        if name in self._mapping:
            mod = __import__(self._mapping[name], fromlist=[name])
            object.__setattr__(self, name, mod) # cache in instance
dict
            return mod
        raise AttributeError(name)

```

8.3 Metaclass `__getattr__` / `__getattribute__` — class-level interception

When you write `MyClass.attr`, the lookup uses `type(MyClass).__getattribute__` (the metaclass). So:

- Defining `__getattr__` on a class intercepts **instance** misses (`obj.missing`).
- Defining `__getattr__` on a **metaclass** intercepts **class** misses (`MyClass.missing`).

```

class Meta(type):
    def __getattr__(cls, name):
        print(f"Meta.__getattr__({cls.__name__}.{name!r})")
        if name.startswith("find_by_"):
            field = name[8:]
            return lambda value: f"SELECT * WHERE {field}={value!r}"
        raise AttributeError(name)
    def __getattribute__(cls, name):
        # Intercepts every MyClass.attr access – use with care
        if name == "secret":
            raise AttributeError("nope")
        return super().__getattribute__(name)

class User(metaclass=Meta):
    pass

print(User.find_by_email("a@b.com"))
# SELECT * WHERE email='a@b.com' (metaclass __getattr__)
try: print(User.secret)
except AttributeError as e: print(e) # nope (metaclass
__getattribute__)

# Instance-level __getattr__ is independent:
class User2(metaclass=Meta):
    def __getattr__(self, name): return f"instance fallback {name}"

print(User2.find_by_name("x")) # metaclass __getattr__ – class
access, not instance
print(User2().missing)
# instance fallback missing – instance __getattr__

```

Summary table:

Hook	Defined on	Intercepts	When called	Risk
<code>__getattribute__</code>	class	<code>obj.attr</code>	Always	Infinite recursion if you touch <code>self.*</code> without <code>object.__getattribute__</code>
<code>__getattr__</code>	class	<code>obj.attr</code>	Only on lookup miss	Safe; cannot recurse unless it accesses a missing attr on <code>self</code>

Hook	Defined on	Intercepts	When called	Risk
<code>__getattr__</code>	metaclass	<code>Cls.attr</code>	Only on class-attr miss	Powers <code>find_by_*</code> / registry patterns
<code>__getattribute__</code>	metaclass	<code>Cls.attr</code>	Always for class attrs	Can hide or rewrite class attributes globally

9. Special methods and `tp_slots` — why `__add__` bypasses `__getattr__`

9.1 Slot wiring

When `PyType_Ready` processes a new type, it does not just store `__add__` in `tp_dict`. It also scans `tp_dict` for known dunder names and wires them into the C slots in `tp_as_number`, `tp_as_sequence`, `tp_as_mapping`, etc. The mapping lives in `Objects/typeobject.c:slotdefs[]`.

```
/* Objects/typeobject.c - excerpt from slotdefs[] */
static slotdef slotdefs[] = {
    {"__add__",      offsetof(PyNumberMethods, nb_add),      Py_TPFLA
GS_CHECKTYPES},
    {"__len__",      offsetof(PySequenceMethods, sq_length), Py_TPFLA
GS_CHECKTYPES},
    {"__getitem__",  offsetof(PyMappingMethods, mp_subscript),0},
    {"__call__",     offsetof(PyTypeObject, tp_call),         0},
    {"__iter__",     offsetof(PyTypeObject, tp_iter),         0},
    {"__next__",     offsetof(PyTypeObject, tp_iternext),      0},
    {"__get__",      offsetof(PyTypeObject, tp_descr_get),     0},
    {"__set__",      offsetof(PyTypeObject, tp_descr_set),     0},
    // ... ~80 entries
};
```

Consequences:

- `x + y` executes `x->ob_type->tp_as_number->nb_add(x, y)` directly — no `getattr(x, "__add__")` call, no descriptor invocation, no

`__getattr__` fallback. This is why proxy objects that rely on `__getattr__` to intercept `__add__` silently fail.

- Defining `__add__` in a class body after creation (e.g., `Cls.__add__ = lambda ...`) calls `PyType_Modified` → `update_slot` → re-wires the C slot. The change is visible to the fast path immediately.
- Some slots have fallback logic: if `tp_as_number->nb_add` is `NULL` but `tp_as_sequence->sq_concat` is set, `PyNumber_Add` tries the sequence slot.

9.2 Which lookups bypass instance `__dict__`

Almost all special methods are looked up on the **type**, not the instance:

```
class Sneaky:
    def __init__(self): self.__add__ = lambda other: "instance add!"
    def __add__(self, other): return "type add"

s = Sneaky()
print(s + 1)           # type add – instance __dict__["__add__"] is
                        # ignored
print(s.__add__(1))    # instance add! – explicit getattr DOES go
                        # through instance dict
# s.__add__ is an attribute lookup (GenericGetAttr) that finds instance
# dict first.
# s + 1 is PyNumber_Add which reads tp_as_number->nb_add, never
# consulting instance dict.

# Same for __iter__, __call__, __len__, __enter__, etc.
# The rule (Language Reference §3.3.10 "Special method lookup"):
# "For custom classes, implicit invocations of special methods are
# only
# guaranteed to work if defined on an object's type, not in the
# object's
# instance dictionary."
```

For metaclass special methods the rule shifts one level: `Cls()` calls `type(Cls)->tp_call`, i.e., the metaclass's `__call__`.

9.3 Vectorcall and `tp_call`

Since PEP 590 (3.8), `tp_call` is supplemented by `tp_vectorcall` and `tp_vectorcall_offset` for the fast call path. `Python/ceval.c`'s `CALL` opcode dispatches through `vectorcall` when available, avoiding tuple/dict construction for arguments. User-defined `__call__` still populates

`tp_call`; the `vectorcall` fast path is used by built-in types and by functions.

10. ABCs and virtual subclasses

Abstract base classes add a parallel type hierarchy that is not based on `tp_mro` at all but on a registry checked at `isinstance` / `issubclass` time.

```

import collections.abc, abc

class MySeq(abc.ABC):
    @abc.abstractmethod
    def __len__(self): ...
    @abc.abstractmethod
    def __getitem__(self, i): ...

# Cannot instantiate without implementing abstract methods:
try: MySeq()
except TypeError as e: print(e) # Can't instantiate abstract class ...

class ConcreteSeq(MySeq):
    def __init__(self, data): self._d = data
    def __len__(self): return len(self._d)
    def __getitem__(self, i): return self._d[i]

print(issubclass(ConcreteSeq, MySeq)) # True – real subclass
print(issubclass(list, MySeq)) # False

# Virtual subclass – register without inheritance:
class OldStyleSeq:
    def __len__(self): return 0
    def __getitem__(self, i): raise IndexError

MySeq.register(OldStyleSeq) # adds to MySeq._abc_registry
(weak set)
print(issubclass(OldStyleSeq, MySeq)) # True – virtual!
print(isinstance(OldStyleSeq(), MySeq)) # True

# __subclasshook__ – implicit virtual subclass via duck typing, no
registration needed:
class Sized(abc.ABC):
    @classmethod
    def __subclasshook__(cls, C):
        if cls is Sized:
            if any("__len__" in B.__dict__ for B in C.__mro__):
                return True
            return NotImplemented

print(issubclass(list, Sized)) # True – list has __len__ in its dict
print(issubclass(int, Sized)) # False

```

Implementation notes (Lib/_collections_abc.py , Modules/_abc.c):

- ABCMeta.__new__ sets Py_TPFLAGS_IMMUTABLETYPE handling and installs an abc cache.

- `register` adds the class to `ABCMeta._abc_registry` (a `WeakSet`). `isinstance` checks (`PyABC_IsInstance` in `_abc.c`) consult this set on cache miss after walking `tp_mro` failed.
- `__subclasshook__` is called by `ABCMeta.__subclasscheck__` before consulting the registry. Returning `True / False` short-circuits; `NotImplemented` falls through to MRO + registry.
- Cache invalidation: mutating an ABC's hierarchy (`register` , new subclass) bumps an internal version counter that invalidates the per-type `issubclass` cache. In hot `isinstance` loops over many types, this is negligible; in code that repeatedly registers ABCs at runtime it can thrash.
- Performance: `isinstance(x, collections.abc.Sequence)` is slower than `isinstance(x, list)` because it walks MRO, checks the virtual registry, and may call `__subclasshook__` . On the hot path, prefer concrete `isinstance` checks or `hasattr` duck typing if you have profiled the cost.

11. Performance — `__slots__` vs. `__dict__` for backend data models

11.1 Where attribute cost actually lives

Each attribute access on the hot path (request parsing, ORM row hydration, JSON serialization loops) pays:

- One `tp_getattro` call (function-pointer dispatch).
- Either a dict lookup (hash + probe, ~30-60 ns) or a slot offset load (pointer dereference, ~5-15 ns).
- Plus descriptor `tp_descr_get` if present.
- Plus `__getattr__` fallback on miss (string compare + call).

Vectorized over millions of rows, the difference between 50 ns and 10 ns per access is measurable in tail latency. CPython 3.11+ adds a `LOAD_ATTR` inline-cache (PEP 659 specialization) that caches the `tp_version_tag` + dict offset so the second access to the same attribute on the same type hits a guarded fast path — but the cache is invalidated by any mutation to the type's `tp_dict` or its MRO.

11.2 Choosing for backend data models

Pattern	Recommendation
High-cardinality short-lived rows (event consumers, ETL buffers, feature vectors) — millions of instances, few attribute names, no dynamic keys	<code>__slots__</code> or <code>@dataclass(slots=True)</code> — 40–60% memory saving, faster GC (fewer <code>dict</code> objects to traverse), lower allocation pressure.
Domain entities with evolving schema (ORM models, config objects, plugin registries)	<code>__dict__</code> or <code>dataclass</code> without slots — flexibility for migrations, <code>__dict__.update</code> , serialization that expects <code>vars(obj)</code> .
Validated fields with coercion (Pydantic-style)	Descriptors (data) for per-field validation, regardless of slots vs. dict. Slots + descriptors compose well: slot descriptor validates on <code>__set__</code> .
Request/response objects on the hot path	<code>__slots__</code> plus <code>__set_name__</code> descriptors, or <code>typing.NamedTuple</code> / <code>msgspec.Struct</code> (which use <code>__slots__</code> -like layout with even tighter packing). Avoid <code>__getattr__</code> on the hot path.

```

# Production-leaning pattern: slotted dataclass with descriptor-
# validated fields
from dataclasses import dataclass

class Range:

    """Data descriptor that validates a numeric range – reusable across
    models."""
    def __init__(self, lo, hi): self.lo, self.hi = lo, hi; self.name =
    None
    def __set_name__(self, owner, name): self.name, self.private =
    name, f"_{name}"
    def __get__(self, obj, objtype=None):
        if obj is None: return self
        return object.__getattr__(obj, self.private)
    def __set__(self, obj, value):
        if not (self.lo <= value <= self.hi):
            raise ValueError(f"{self.name}={value} out of [{self.lo},
{self.hi}]")
        object.__setattr__(obj, self.private, value)

# Pre-slots dataclass: __slots__ generated, descriptors still work
# because
# dataclass(slots=True) reserves slot storage but descriptors use
# private slot names.
@dataclass(slots=True)
class OrderLine:
    sku: str
    qty: int
    price: float

# If you need validated qty/price with slots, use __slots__ +
# descriptors directly:
class ValidatedOrderLine:
    __slots__ = ("sku", "_qty", "_price")
    qty = Range(1, 10_000)
    price = Range(0.01, 1_000_000.0)
    def __init__(self, sku, qty, price):
        self.sku = sku; self.qty = qty; self.price = price

ol = ValidatedOrderLine("ABC", 3, 9.99)
print(ol.qty)      # 3
try: ol.qty = 0
except ValueError as e: print(e)  # qty=0 out of [1, 10000]

```

11.3 Dataclasses and slots — interaction details

- `@dataclass(slots=True)` (3.10+) generates `__slots__` from the field list and rewrites `__init__` / `__repr__` / `__eq__` to use slot access. No

`__dict__` unless you add `__dict__` to the slot list manually or use `kw_only` tricks.

- `@dataclass` without `slots=True` creates a normal `__dict__` class. The per-instance cost is a dict even if you only have 2-3 fields — fine for hundreds of objects, painful for millions.
 - `attrs ( attr.s(slots=True) )` and `msgspec.Struct` follow the same principle with additional optimizations (e.g., `msgspec` avoids `PyObject` per field for primitive types).
 - Serialization: `dataclasses.asdict(obj)` works for dict-backed dataclasses but for slotted ones it synthesizes a dict via `getattr` per field. Libraries that do `obj.__dict__` directly (naive JSON encoders) break on slots — use `dataclasses.fields` or `getattr` loops instead.
-

12. Putting it together — backend lens

12.1 ORM / data-model design with descriptors and slots

A minimal but realistic row-mapper that illustrates how the pieces compose:

```

# row_mapper.py – sketch of an ORM row object used by a backend query
path
import weakref
from typing import Any

class Column:
    """Data descriptor: maps attribute access to a slot, with type
    coercion and nullability."""
    def __init__(self, py_type, nullable=False, default=None):
        self.py_type, self.nullable, self.default = py_type, nullable,
        default
        self.name = self.private = None
    def __set_name__(self, owner, name):
        self.name, self.private = name, f"_{name}"
    def __get__(self, obj, objtype=None):
        if obj is None: return self
        # Slot-backed: object.__getattr__ reads the fixed offset
        try: return object.__getattr__(obj, self.private)
        except AttributeError: return self.default
    def __set__(self, obj, value):
        if value is None and not self.nullable:
            raise ValueError(f"{self.name} is not nullable")
        if value is not None and not isinstance(value, self.py_type):
            try: value = self.py_type(value) # coerce: "42" → 42
            except Exception as e: raise TypeError(f"{self.name}:
{e}") from e
        object.__setattr__(obj, self.private, value)

class RowMeta(type):
    def __new__(mcls, name, bases, ns):
        cols = {k: v for k, v in ns.items() if isinstance(v, Column)}
        # Synthesize __slots__ from columns if not already declared
        if "__slots__" not in ns and cols:
            ns["__slots__"] = tuple(f"_{k}" for k in cols) + ("__weakref__",)
        # Move Column descriptors into the new namespace (they
        # already are)
        cls = super().__new__(mcls, name, bases, ns)
        cls._columns = cols
        return cls

class Row(metaclass=RowMeta): pass

class UserRow(Row):
    id = Column(int)
    email = Column(str)
    age = Column(int, nullable=True, default=None)

# UserRow now has: __slots__ = ("_id", "_email", "_age", "_weakref__"),
# _columns registry

```



```

u = UserRow()
u.id = "7"           # coerced to int via Column.__set__
u.email = "a@b.com"
print(u.id, type(u.id))  # 7 <class 'int'>
print(UserRow._columns.keys()) # dict_keys(['id', 'email', 'age'])
print(hasattr(u, "__dict__"))
# False – slotted, ~56B per row instead of ~160B
print(weakref.ref(u)() is u)  # True – __weakref__ slot allows
                              # weakrefs (needed for identity map)

# Hydration hot path – what a DB driver loop does:
rows = [UserRow() for _ in range(1000)]
for r in rows: r.id = 1; r.email = "x@y.com"  # each assignment:
Column.__set__ → slot write

```

Why this design wins at scale:

- **Memory:** `__slots__` eliminates per-row `__dict__`; a 1M-row prefetch buffer drops from ~160 MB to ~56 MB plus payload.
- **Validation at the boundary:** `Column.__set__` enforces types and nullability on assignment, not on `save()`, so invalid data fails close to the bug.
- **Identity map compatibility:** the `"__weakref__"` slot keeps weak-reference support for caches that hold `WeakValueDictionary` of rows.
- **No `__getattr__` on the hot path:** hydration writes directly to slots; reading checks slot offset — `__getattr__` is never invoked.

12.2 Hot-path attribute lookup cost — what to avoid

- **Avoid `__getattr__` / `__getattribute__` overrides on objects that are read in tight loops.** The fallback and the unconditional interceptor both add a Python-level call per miss/hit. If you need dynamic attributes, cache the result in `__dict__` or a slot on first miss so subsequent lookups hit the fast path (§5 step 2).
- **Keep `tp_version_tag` stable.** Mutating `Class.attr = ...` inside a request loop invalidates `LOAD_ATTR` inline caches for every function that reads that attribute. Hoist class mutations out of the loop or make the type immutable (`Py_TPFLAGS_IMMUTABLETYPE` / `types.MappingProxyType` for `tp_dict`).
- **Prefer slot access over dict access in `__slots__` hot loops.** `obj._x` via slot descriptor is a fixed-offset load; `obj.__dict__["_x"]` (if you added `__dict__`) is a hash probe.

- **Profile before optimizing.** `python -X importtime`, `py-spy`, and `perf` show whether attribute lookup is actually your bottleneck. For many services it is JSON serialization or DB I/O, not `getattr`. Measure with `timeit` on the real row type, not a microbenchmark on object.
-

Key takeaways

- Every type is a `PyTypeObject` whose `ob_type` is `type`, and every instance's `ob_type` points at its type. `tp_name`, `tp_basicsize`, `tp_dictoffset`, `tp_flags`, `tp_dict`, `tp_mro`, and the `tp_as_*` subtables are the fields that govern allocation, attribute storage, and special-method dispatch.
- `class` statements execute `type.__new__(metaclass, name, bases, dict) → PyType_GenericAlloc → populate tp_bases / tp_dict / tp_basicsize / tp_dictoffset → PyType_Ready` (compute MRO, wire `tp_methods` / `tp_members` / `tp_getset` into descriptors, link `tp_subclasses`) → `metaclass.__init__`. Metaclasses intercept this to build registries, inject `__slots__`, or wrap methods — they must call `super().__new__`.
- MRO is C3 linearization: `L[C] = [C] + merge(L[B1], ..., L[Bn], [B1, ..., Bn])` picking the first head not in any tail. It respects local precedence and monotonicity. `cls.__mro__` (tuple, `tp_mro`) is authoritative; `cls.mro()` returns a fresh list; `inspect.getmro` mirrors `__mro__`. Inconsistent hierarchies raise `TypeError` at class creation.
- Instance attribute lookup is a strict priority chain implemented in `PyObject_GenericGetAttr`: data descriptor (has `tp_descr_set`) → instance `__dict__` (`tp_dictoffset`) → non-data descriptor / plain class attribute (`tp_descr_get` if present) → `__getattr__` fallback → `AttributeError`. Data descriptors always win over instance dict; non-data descriptors and plain attributes lose to it.
- Class attribute lookup (`cls.attr`) uses `type_getattro` and walks the metaclass MRO first, then the class MRO — so a metaclass's `__getattr__` intercepts missing class attributes, and a metaclass's `__getattribute__` intercepts every class attribute access.
- Descriptors are any object with `tp_descr_get` / `tp_descr_set` (`__get__` / `__set__` / `__delete__`). Data descriptors define `__set__` / `__delete__`; non-data descriptors define only `__get__`. `property` /

`member_descriptor` are data descriptors; `function / classmethod / staticmethod` are non-data descriptors (with `classmethod / staticmethod` customizing `__get__` to bind or not bind). `__set_name__` (PEP 487) gives descriptors their attribute name at class creation.

- Functions become methods via `function.__get__`: accessing `obj.method` finds a non-data descriptor in the type, sees no instance dict entry, and calls `tp_descr_get` which returns a bound `method` object. Assigning `obj.method = ...` creates an instance dict entry that shadows the function on future lookups.
- `__slots__` sets `tp_dictoffset = 0`, grows `tp_basicsize` by `n * sizeof(PyObject*)`, and installs `member_descriptor` slot descriptors that read/write fixed offsets. It saves ~40–60% memory per instance and speeds attribute access via direct offset loads, at the cost of no `__dict__` (unless explicitly requested), no arbitrary attribute names, and required `"__weakref__"` for weak references. Subclasses without `__slots__` regain a `__dict__`.
- `__getattr__` is the unconditional interceptor for every `obj.attr` (replaces `tp_getattro`); `__getattr__` is the fallback-only hook called after the full MRO + instance dict search fails. On a metaclass they control `cls.attr` instead of `obj.attr`. `__getattr__` must use `object.__getattr__`(self, name) to avoid infinite recursion.
- Special methods (`__add__`, `__iter__`, `__call__`, `__get__`, ...) are looked up on the **type**, not the instance, via C slots (`tp_as_number->nb_add`, `tp_iter`, `tp_call`, `tp_descr_get`, ...). `Objects/typeobject.c:slotdefs[]` maps dunder names to slot offsets at `PyType_Ready` time. Instance `__dict__["__add__"]` is ignored by `s + other`; only `type(s).__dict__["__add__"]` (or its MRO) matters.
- ABCs (`ABCMeta`, `collections.abc`) add virtual subclassing: `register(Sub)` adds to a `WeakSet` checked by `isinstance / issubclass` after MRO walk fails, and `__subclasshook__` allows duck-type matching. Useful for interface checks but slower than concrete `isinstance` — avoid on the innermost hot path if profiling shows cost.
- For backend data models at scale, choose `__slots__ / @dataclass(slots=True)` / `msgspec.Struct` for high-cardinality short-lived rows and keep `__dict__` for flexible domain entities. Use data

descriptors for per-field validation; avoid `__getattr__` / `__getattribute__` on the hydration hot path; keep `tp_version_tag` stable to preserve 3.11+ `LOAD_ATTR` inline-cache specialization.

Further reading

- **Descriptors HowTo Guide** — Raymond Hettinger, *Descriptor HowTo Guide*, <https://docs.python.org/3/howto/descriptor.html>. **(pinned)** The canonical walkthrough of descriptor concepts, with pure-Python equivalents for `property`, `classmethod`, `staticmethod`, and `function` binding. Indispensable companion to this chapter.
- **PEP 252 — Making Types Look More Like Classes** — Guido van Rossum, <https://www.python.org/dev/peps/pep-0252/>. **(pinned)** The original specification for `__slots__`, `tp_dictoffset`, and the type/class unification that introduced heap types and `Py_TPFLAGS_HEAPTYPE`.
- **The C3 Method Resolution Order** — Michele Simionato, *The Python 2.3 Method Resolution Order*, <https://www.python.org/download/releases/2.3/mro/>. **(pinned)** The clearest exposition of C3 linearization, monotonicity, and the `merge` rule, with Python code that mirrors `Objects/typeobject.c:mro_internal`. Predecessor to the Dylan paper.
- **CPython `Objects/typeobject.c` and `Include/cpython/object.h`** — [https://github.com/python/cpython, `Objects/typeobject.c`](https://github.com/python/cpython/blob/main/Objects/typeobject.c) (~5,000 lines) and [Include/cpython/object.h](https://github.com/python/cpython/blob/main/Include/cpython/object.h). **(pinned)** The source of truth for `PyTypeObject`, `PyType_Ready`, `type_new`, `mro_internal/pmerge`, `slotdefs[]`, and `type_getattro`. Read with `Include/object.h` for `PyObject_HEAD`.
- **PEP 487 — Simpler customisation of class creation** — Martin Teichmann, <https://www.python.org/dev/peps/pep-0487/>. Covers `__set_name__` and `__init_subclass__`, the two hooks that reduced the need for metaclasses for per-descriptor naming and per-subclass registration.
- **PEP 560 — Core typing support and `__mro_entries__`** — Ivan Levkivskyi, <https://www.python.org/dev/peps/pep-0560/>. Explains how `class C(Generic[T])` and `__mro_entries__` interact with MRO and type creation, relevant for typed backend code.

- **The Dylan paper on C3** — Kim Barrett et al., *A Monotonic Superclass Linearization for Dylan*, OOPSLA 1996, <https://doi.org/10.1145/236337.236343>. The original C3 linearization paper that Python adopted; formal treatment of local precedence and monotonicity.
- **CPython `Objects/descrobject.c`** — <https://github.com/python/cpython/blob/master/Objects/descrobject.c>. Implementation of `property`, `member_descriptor` (slot descriptors), `getset_descriptor`, and `method_descriptor` — the C behind the Python-level descriptor examples in §6.
- **PEP 590 — Vectorcall: a fast calling protocol for CPython** — Jeroen Demeyer et al., <https://www.python.org/dev/peps/pep-0590/>. Details `tp_vectorcall`, `tp_vectorcall_offset`, and how `tp_call` dispatch was accelerated; context for §9.3.
- **PEP 3115 — Metaclasses in Python 3000** — Talin, <https://www.python.org/dev/peps/pep-3115/>. How `__prepare__` and the `metaclass=` keyword argument changed class creation, with rationale for `Py_CalculateMetaclass`.

Chapter 7 — Functions, Closures, Generators, Coroutines, and `async/await`

What this chapter covers. Every FastAPI handler, every streaming generator, every `async def` coroutine in your backend is the same C object — `PyFunctionObject` in `Objects/funcobject.c` — interpreted through a small set of dispatch paths. This chapter opens that object, traces how `f(a, b)` becomes a vectorcall, how `LOAD_DEREF` and `PyCellObject` implement lexical closures, how `yield` suspends a frame without destroying it, how `yield from` delegates without copying, and how `await` lowers to `GET_AWAITABLE + SEND / YIELD_VALUE`. You will read disassembly that shows the closure capture, the generator state switch, and the `async/await` lowering; you will drive `inspect` and `types` to introspect live functions; and you will connect the machinery to its operational consequences in `async` web frameworks and generator-based streaming.

Learning goals — after this chapter you should be able to:

- Read `PyFunctionObject` in `Objects/funcobject.c` / `Include/cpython/funcobject.h` and explain every field: `func_code`, `func_globals`, `func_defaults`, `func_kwdefaults`, `func_closure`, `func_annotations`, `func_qualname`, `vectorcall`, `func_module`.
- Describe `PyCodeObject` flags and fields relevant to functions: `co_argcount`, `co_kwnonlyargcount`, `co_posonlyargcount`, `co_cellvars`, `co_freevars`, `co_varnames`, `co_flags` (`CO_OPTIMIZED`, `CO_NEWLOCALS`, `CO_VARARGS`, `CO_VARKEYWORDS`, `CO_GENERATOR`, `CO_COROUTINE`, `CO_ITERABLE_COROUTINE`, `CO_ASYNC_GENERATOR`), `co_stacksize`, `co_exceptiontable`.
- Trace the full calling convention: `PyObject_Call` → `tp_call` → `vectorcall` (PEP 590) → `_PyFunction_Vectorcall` → frame setup in `ceval.c`; explain `PUSH_NULL` / `PRECALL` / `CALL` and inline-cache specialization for calls.
- Explain `vectorcall`'s fast path (`PyVectorcall_NARGS`, `nargsf`, `kwnames`) and why it matters for per-request call volume in ASGI frameworks.
- Implement the closure protocol: `MAKE_CELL`, `LOAD_CLOSURE`, `COPY_FREE_VARS`, `LOAD_DEREF` / `STORE_DEREF` / `DELETE_DEREF`, `PyCellObject`, `__closure__` tuple of cells, and the `co_cellvars` ↔ `co_freevars` linkage between enclosing and enclosed scopes.
- Write and introspect decorators that preserve metadata via `functools.wraps` / `functools.update_wrapper` and explain what breaks when you do not.
- Describe generator internals: `PyGenObject`, `GEN_CREATED` / `GEN_RUNNING` / `GEN_SUSPENDED` / `GEN_CLOSED`, `YIELD_VALUE` / `RESUME` / `SEND`, `gen.send()` / `gen.throw()` / `gen.close()`, and `RETURN_GENERATOR`.
- Explain yield from delegation: `GET_YIELD_FROM_ITER` → delegated iterator protocol, the `yield-from` chain, `throw` / `close` propagation, and `StopIteration.value` as return value.
- Trace native coroutines (PEP 492 `async def`): `CO_COROUTINE`, `PyCoroObject`, `GET_AWAITABLE` / `GET_YIELD_FROM_ITER` lowering for `await`, `CORO_SUSPENDED` vs. generator suspension, and why `await` and `yield from` share machinery but not semantics.

- Explain async generators (`CO_ASYNC_GENERATOR` , `ASEND` , `AGAINEXT`) and async comprehensions (`GET_AITER` / `GET_ANEXT`).
- Use `inspect` (`signature` , `getclosurevars` , `isgenerator` , `iscoroutinefunction` , `getgeneratorstate` , `getcoroutinestate`) and `types` (`FunctionType` , `CodeType` , `GeneratorType` , `CoroutineType` , `CellType`) to introspect any callable at runtime.
- Reason about the performance and correctness implications in real backends: ASGI event-loop starvation, generator-based streaming backpressure, and closure/memory-leak pitfalls.

Prerequisites. Chapter 3 dissected `PyCodeObject` , `wordcode` , `inline caches` , and the `ceval` loop; Chapter 6 dissected `PyTypeObject` , `descriptors` , and `attribute lookup` . This chapter sits on top of both — functions are typed objects whose `bytecode` , `type slots` , and `frame management` conspire. Chapter 2's `reference-counting` and `object layout` applies throughout.

1. Why this chapter decides your request latency

A backend request typically spends its time in three patterns that all trace through this chapter's machinery:

- **Call overhead.** Every middleware, dependency-injection hook, serializer, validator, and route handler is a function call. FastAPI resolves ~5–15 callables per request (dependencies, path operation, exception handlers). At 10k RPS that is 50–150k vectorcalls per second on a single process. Whether those calls hit the vectorcall fast path or fall back to `PyObject_Call` tuple-packing is a measurable latency difference.
- **Streaming responses.** `StreamingResponse(content=generator)` — where `generator` is a Python generator yielding chunks — is the idiomatic way to stream large CSVs, LLM token streams, and S3 passthroughs without buffering the whole body. Misunderstanding generator state (`GEN_SUSPENDED` vs. `GEN_CLOSED`) and `yield from` delegation is how you leak connections and deadlock ASGI servers.
- **Concurrency model.** `async def` handlers run cooperatively on a single event loop. An `await` that lowers to the wrong awaitable, or a blocking call hidden inside an `async def` that never hits

`GET_WAITABLE`, blocks the loop and starves every concurrent request. Understanding `GET_WAITABLE` / `SEND` / `YIELD_VALUE` lowering — and that the event loop is just a scheduler pumping `send(None)` — is the difference between reading async traces and guessing.



2. Function objects — `PyFunctionObject` is a typed object

2.1 Every `def` creates a heap `PyFunctionObject`

`def f(a, b): ...` is not a keyword that reserves a slot. It is executable bytecode (`MAKE_FUNCTION`) that allocates a `PyFunctionObject` at runtime. The compiler emits a `PyCodeObject` for the body (immutable, shareable); `MAKE_FUNCTION` wraps it with runtime bindings (globals, closure, defaults) to produce the callable. Two functions with the same body text but different `__globals__` or `__closure__` are distinct objects sharing the same `__code__`.

Relevant sources: `Objects/funcobject.c`, `Include/cpython/funcobject.h`, `Python/ceval.c` handling of `MAKE_FUNCTION`.

2.2 PyFunctionObject layout (C)

```
/* Include/cpython/funcobject.h – Python 3.11+ (abbreviated) */
typedef struct {
    PyObject_HEAD                    /* ob_refcnt + ob_type (=
    &PyFunction_Type) */
    PyObject *func_code;            /* PyCodeObject* – immutable
    bytecode + metadata */
    PyObject *func_globals;         /* dict – module globals where
    function was defined */
    PyObject *func_builtins;       /* builtins dict (borrowed from
    globals) */
    PyObject *func_defaults;       /* tuple or NULL – positional
    defaults (__defaults__) */
    PyObject *func_kwdefaults;     /* dict or NULL – kwnonly
    defaults (__kwdefaults__) */
    PyObject *func_closure;        /* tuple of PyCellObject* or
    NULL (__closure__) */
    PyObject *func_doc;            /* __doc__ */
    PyObject *func_name;           /* __name__ */
    PyObject
    *func_dict;                    /* __dict__ – arbitrary attrs on the function
    itself */
    PyObject *func_weakreflist;    /* weakrefs to this function */
    PyObject *func_module;         /* __module__ */
    PyObject *func_annotations;    /* __annotations__ dict */
    PyObject *func_qualname;       /* __qualname__ – dotted path */
    vectorcallfunc vectorcall;     /* PEP 590 slot –
    _PyFunction_Vectorcall */
    PyObject *func_typeparams;     /* 3.12+ – PEP 695 type params
    */
} PyFunctionObject;

/* The type object itself – Objects/funcobject.c */
PyTypeObject PyFunction_Type = {
    .tp_name      = "function",
    .tp_basicsize = sizeof(PyFunctionObject),
    .tp_call      = _PyFunction_Vectorcall, /* aliased via
    tp_vectorcall_offset */
    .tp_vectorcall_offset = offsetof(PyFunctionObject, vectorcall),
    .tp_getattro  = PyObject_GenericGetAttr,
    /* ... tp_repr, tp_traverse, tp_weaklistoffset, ... */
};
```

Key observations:

Field	Python attribute	Mutability	Notes
<code>func_code</code>	<code>__code__</code>	Replaceable	Assigning <code>f.__code__ = other.__code__</code> swaps bytecode without reallocating the function. Debuggers and <code>functools</code> rely on this being a property with validation.
<code>func_globals</code>	<code>__globals__</code>	Shared reference	The dict is <i>shared</i> with the defining module's <code>__dict__</code> — mutating <code>module.__dict__</code> mutates every function's <code>__globals__</code> that was defined there. Not copied.
<code>func_defaults</code>	<code>__defaults__</code>	Replaceable tuple	<code>None</code> if no positional defaults. Evaluated once at <code>def</code> time, not per call. Mutable defaults (e.g., <code>def f(x=[])</code>) alias the same list across calls.
<code>func_kwdefaults</code>	<code>__kwdefaults__</code>	Replaceable dict	<code>None</code> if no kwonly defaults. Same single-evaluation rule.
<code>func_closure</code>	<code>__closure__</code>	Immutable tuple	<code>None</code> or tuple of <code>cell</code> objects. Each cell holds one closed-over variable. See §4.
<code>func_annotations</code>	<code>__annotations__</code>	Replaceable dict	Lazily populated; <code>from __future__ import annotations</code> (PEP 563) makes values stringified.
<code>vectorcall</code>	—	Set at creation	Function pointer for PEP 590 fast dispatch. For plain functions this is <code>_PyFunction_Vectorcall</code> .


```

type: <class 'function'>  isfunction=True
__name__ =demo  __qualname__ =demo  __module__ =__main__
__code__=<code object demo at 0x...>  co_name=demo
__globals__ is module.__dict__: True
__defaults__ =None  __kwdefaults__={'d': 99}
__closure__ =None
__annotations__={'return': <class 'int'>}
__dict__={}
co_argcount=2  co_posonlyargcount=0  co_kwonlyargcount=2
co_varnames=('a', 'b', 'c', 'd', 'args', 'kw')
co_flags=0x8b  has VARARGS=True  has VARKEYWORDS=True
co_stacksize=2  co_nlocals=6
signature: (a, b, *args, c, d=99, **kw) -> int
a: kind=POSITIONAL_OR_KEYWORD default=<no default> annotation=<no annotation>
b: kind=POSITIONAL_OR_KEYWORD default=<no default> annotation=<no annotation>
args: kind=VAR_POSITIONAL default=<no default>
c: kind=KEYWORD_ONLY default=<no default>
d: kind=KEYWORD_ONLY default=99
kw: kind=VAR_KEYWORD default=<no default>

```

```

# defaults are evaluated once – the classic mutable-default pitfall, shown at the C level
def append_one(item, bucket=[]):
    bucket.append(item)
    return bucket

print(append_one.__defaults__)  # ([,]) – one list object
print(append_one(1))           # [1]
print(append_one(2))           # [1, 2] – same list via func_defaults[0]
print(append_one.__defaults__[0] is append_one(3))  # True – alias

# fix: sentinel
def append_one_fixed(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

```

2.4 co_flags — the code object's capability bits

`PyCodeObject.co_flags` (`Include/cpython/code.h`) tells `ceval` how to set up the frame:

Flag	Value	Meaning
CO_OPTIMIZED	0x0001	Locals are in <code>fastlocals</code> array; <code>LOAD_FAST</code> / <code>STORE_FAST</code> are valid. Functions and comprehensions set this. Module bodies do not.
CO_NEWLOCALS	0x0002	Push a fresh <code>f_locals</code> dict on entry. Functions set this.
CO_VARARGS	0x0004	Has <code>*args</code> (<code>co_argcount</code> + <code>*args</code>).
CO_VARKEYWORDS	0x0008	Has <code>**kwargs</code> .
CO_NESTED	0x0010	Contains a nested scope that captures free vars.
CO_GENERATOR	0x0020	Contains <code>yield</code> / <code>yield from</code> ; calling it returns a <code>PyGenObject</code> , not a value.
CO_COROUTINE	0x0080	<code>async def</code> (but not <code>async def</code> containing <code>yield</code> — that is <code>CO_ASYNC_GENERATOR</code>).
CO_ITERABLE_COROUTINE	0x0100	<code>@asyncio.coroutine</code> generator-based coroutine (legacy).
CO_ASYNC_GENERATOR	0x0200	<code>async def</code> containing <code>yield</code> / <code>yield from</code> -like <code>async yield</code> .

```
import inspect, dis

def plain(): pass
def gen(): yield 1
async def coro(): pass
async def agen(): yield 1

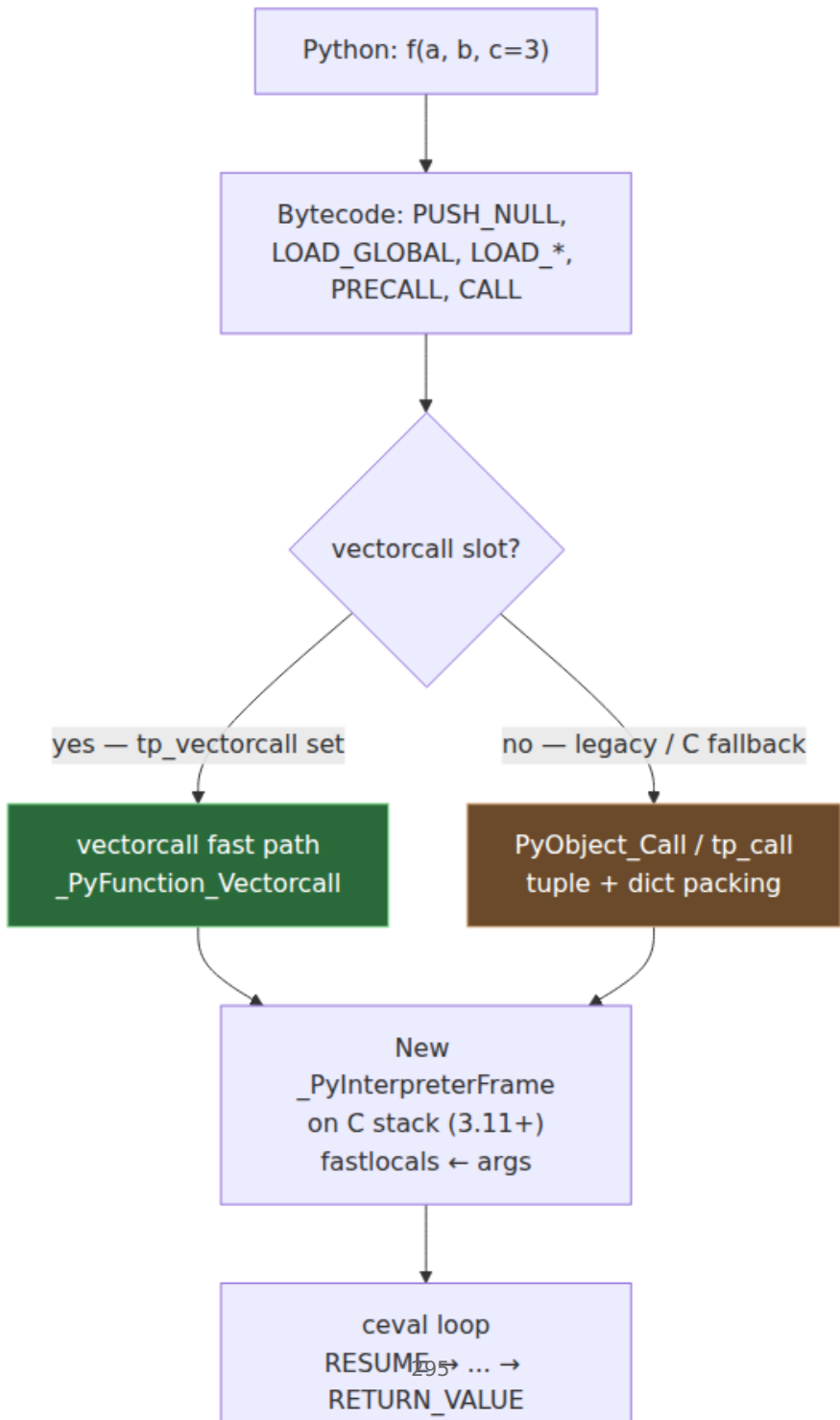
for fn in [plain, gen, coro, agen]:
    co = fn.__code__
    print(f"{fn.__name__:6s} flags={hex(co.co_flags):6s} "
          f"GEN={bool(co.co_flags & inspect.CO_GENERATOR)} "
          f"CORO={bool(co.co_flags & inspect.CO_COROUTINE)} "
          f"ASYNCGEN={bool(co.co_flags & inspect.CO_ASYNC_GENERATOR)}")
```

plain	flags=0x43	GEN=False	CORO=False	ASYNCGEN=False
gen	flags=0x63	GEN=True	CORO=False	ASYNCGEN=False
coro	flags=0xc3	GEN=False	CORO=True	ASYNCGEN=False
agen	flags=0x143	GEN=False	CORO=False	ASYNCGEN=True

Note that `CO_GENERATOR` and `CO_COROUTINE` are mutually exclusive for native objects — `CO_ASYNC_GENERATOR` is the hybrid that combines `async def` with generator semantics (`0x0200` | `0x0002` | `0x0001` | ...).

3. Calling convention — from `f(x)` to a new frame

3.1 The three dispatch layers



Historically, `PyObject_Call(callable, args_tuple, kwargs_dict)` was the universal entry — it packed all arguments into a tuple and dict, then dispatched through `tp_call` (`PyTypeObject.tp_call`). Every call paid tuple/dict allocation even for `f(1, 2)`.

PEP 590 (Python 3.8+) introduced **vectorcall**: the caller passes a C array of `PyObject*` plus a `kwnames` tuple, without packing positionals into a tuple. Types that support it set `tp_vectorcall_offset` and publish a `vectorcallfunc` at that offset. The interpreter's `CALL` opcode now emits a vectorcall directly; `PyObject_Call` is the fallback for C callers that already have packed tuples.

3.2 What vectorcall looks like in C

```
/* Include/cpython/object.h – PEP 590 */
typedef PyObject *(*vectorcallfunc)(
    PyObject *callable,
    PyObject *const *args,          /* C array of positional args + kwargs
values */
    size_t nargsf,                  /* PyVectorcall_NARGS(nargsf) =
positional count;
                                bit 63 =
PY_VECTORCALL_ARGUMENTS_OFFSET flag */
    PyObject *kwnames              /* tuple of kwarg names, or NULL */
);

/* Helpers – Include/cpython/object.h */
#define PyVectorcall_NARGS(n)      ((n) &
~PY_VECTORCALL_ARGUMENTS_OFFSET)
static inline Py_ssize_t PyVectorcall_NARGS(size_t nargsf);
```

Calling `f(1, 2, c=3)` via vectorcall:

```
args array on C stack: [ 1 , 2 , 3 ]    // positionals first, then kw
values contiguously
nargsf = 2 | 0                      // 2 positionals
kwnames = ("c",)                    // tuple parallel to trailing a
rgs
```

If the callable is a plain `PyFunctionObject`, `vectorcall` points at `_PyFunction_Vectorcall` (`Objects/call.c`), which:

1. Validates arity against `co_argcount` / `co_kwonlyargcount` / defaults.
2. Allocates a new `_PyInterpreterFrame` on the C stack (3.11+; older versions heap-allocated `PyFrameObject`).

3. Copies `args[0..nargs-1]` into `frame->localsplus[0..]` (`LOAD_FAST` slots).
4. Fills defaults for missing args from `func_defaults` / `func_kwdefaults`.
5. Enters `ceval` at `RESUME`.

No tuple, no dict, no `tp_call` indirection for the common case.

3.3 Bytecode lowering for calls (3.11+)

Chapter 3 introduced `PUSH_NULL` / `PRECALL` / `CALL`. For calls they lower as:

```
import dis

def callee(a, b): return a + b

def caller(x):
    return callee(x, 1)

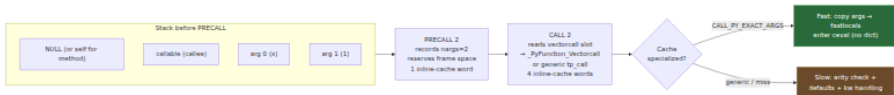
dis.dis(caller)
```

6	0 RESUME	0	
7	2 LOAD_GLOBAL	1 (NULL + callee)	# pushes
	NULL, then callee		
	14 LOAD_FAST	0 (x)	
	16 LOAD_CONST	1 (1)	
	18 PRECALL	2	# 2
	positional args; reserves space		
	22 CALL	2	#
	vectorcall with nargs=2		
	32 RETURN_VALUE		

With `show_caches=True`:

2	LOAD_GLOBAL	1	(NULL + callee)
4	CACHE	0	(x5 - LOAD_GLOBAL cache)
14	LOAD_FAST	0	(x)
16	LOAD_CONST	1	(1)
18	PRECALL	2	
20	CACHE	0	
22	CALL	2	
24	CACHE	0	(x4 - CALL cache)

- `NULL` is a sentinel that marks "not a method call" — `LOAD_METHOD` / `LOAD_ATTR` may push a bound `self` instead of `NULL` for `obj.method()` so `CALL` can avoid re-binding.
- `PRECALL` records `nargs` and reserves stack slots for the callee's frame; its 1 cache word holds the adaptive counter.
- `CALL` reads the inline cache (4 words) that specialize to `CALL_PY_EXACT_ARGS`, `CALL_PY_WITH_DEFAULTS`, `CALL_NO_KW_BUILTIN_0`, etc. After a few executions, a monomorphic `callee(x, 1)` call will specialize to `CALL_PY_EXACT_ARGS` and skip arity checks via cached `PyCodeObject` metadata.



3.4 `tp_call` — the legacy slot and C callables

C functions (`PyCFunctionObject`, `PyMethodDef` with `METH_VARARGS` / `METH_FASTCALL` / `METH_0`) do not use `PyFunctionObject` at all — they are `PyCFunction_Type` with their own `tp_call` (`cfunction_call` / `cfunction_vectorcall` in `Objects/methodobject.c`). Built-in types (`list`, `dict`, `int`) have `tp_call` pointing at `type_call` (`Objects/typeobject.c`) which allocates an instance via `tp_new` + `tp_init`.

The observable invariant: every `callable(...)` ultimately reaches either a `vectorcallfunc` (fast) or ternaryfunc `tp_call` (legacy). Pure-Python functions always prefer `vectorcall`; `PyObject_Call(func, args, kwargs)` from C is the path that must pack.

```

import inspect

# Every callable has a tp_call; check vectorcall availability
indirectly
def py_func(): pass
print(f"py_func vectorcall offset: {type(py_func).__dict__.get('__vectorcalloffset__', 'via C slot')}")
print(f"inspect.isbuiltin(len): {inspect.isbuiltin(len)}")
print(f"inspect.isfunction(py_func): {inspect.isfunction(py_func)}")
print(f"callable(len)={callable(len)}    callable(py_func)={callable(py_func)}")

# tp_call is what makes types callable: int(42) → int.__call__ →
tp_call
print(f"type(int).__call__: {type.__call__}")

```

3.5 Why vectorcall matters for ASGI throughput

FastAPI/Starlette dispatch per request fans out through `get_dependant` → `solve_dependencies` → `N` `await dependant.call(**values)`. Each dependency is a Python callable invoked with keyword arguments derived from the request. Before vectorcall, each dependency call allocated a `tuple` of positionals and a `dict` of kwargs that were immediately unpacked — at 10k RPS, millions of short-lived tuple/dict objects per second pressuring the GC.

Vectorcall plus the 3.11 `CALL` specialization removes that overhead for monomorphic call sites. Profiling with `perf` or `py-spy --native` will show `CALL_PY_EXACT_ARGS` dominating over `CALL_PY_WITH_DEFAULTS` when your dependency signatures match the call sites — which they should if you avoid optional-argument sprawl on hot paths. The operational takeaway: keep dependency signatures tight and avoid `**kwargs` passthrough where you can name arguments explicitly — it keeps the specialized `CALL` path hot.

4. Closures — cells, free vars, and `LOAD_DEREF`

4.1 The problem closures solve

A nested function that references a variable from an enclosing scope cannot use `LOAD_FAST` (the variable is not in its own `fastlocals`) and cannot use `LOAD_GLOBAL` (the variable is not global). It needs a reference

to a storage location that outlives the enclosing call's frame. That storage is a **cell**.

4.2 PyCellObject — the indirection

```
/* Include/cpython/cellobject.h */
typedef struct {
    PyObject_HEAD
    PyObject
*ob_ref; /* the closed-over value; NULL if not yet bound (unbound
free var) */
} PyCellObject;

/* API - Objects/cellobject.c */
PyObject *PyCell_New(PyObject *obj);           // cell =
PyCellObject{ob_ref = obj}
PyObject *PyCell_Get(PyObject *cell);          // Py_NewRef(cell-
>ob_ref)
int      PyCell_Set(PyObject *cell, PyObject *value);
```

A cell is a heap-allocated box holding one `PyObject*`. The enclosing scope stores its local *through* the cell; each enclosed scope that captures the variable holds a *borrowed reference to the same cell*. Mutating `ob_ref` through the cell is visible to all holders — which is why `nonlocal` works.

4.3 Compiler's scope analysis → `co_cellvars` / `co_freevars`

At compile time (`Python/symtable.c` + `Python/compile.c`), the symbol table marks each name per scope as `LOCAL` , `CELL` , `FREE` , or `GLOBAL` . The compiler then emits:

- `MAKE_CELL` — in the enclosing scope, for each `co_cellvars` entry: wrap the local in a `PyCellObject` and store it back into the fastlocals slot as a cell.
- `LOAD_CLOSURE` / `BUILD_TUPLE` / `MAKE_FUNCTION` — bundle `co_freevars` cells into the closure tuple passed to the child code object.
- `COPY_FREE_VARS` — in the child, copy the passed closure tuple into its own free-var storage.

```

import dis, inspect

def make_adder(n):
    def adder(x):
        return x + n
    return adder

# Enclosing scope: n is CELL
co_outer = make_adder.__code__
print(f"make_adder co_cellvars={co_outer.co_cellvars} co_freevars={co_outer.co_freevars}")
print(f"make_adder co_varnames={co_outer.co_varnames}")
dis.dis(make_adder)

# Enclosed scope: n is FREE
co_inner = make_adder(10).__code__
print(f"\nadder co_cellvars={co_inner.co_cellvars} co_freevars={co_inner.co_freevars}")
dis.dis(make_adder(10))

```

```

make_adder co_cellvars=('n',) co_freevars=()
make_adder co_varnames=('n', 'adder')
  2          0 MAKE_CELL          0 (n)

  3          2 RESUME            0

  4          4 LOAD_CLOSURE      0 (n)
          6 BUILD_TUPLE        1
          8 LOAD_CONST          1 (<code object adder>)
         10 MAKE_FUNCTION      8 (closure)
         12 STORE_FAST         1 (adder)

  5          14 LOAD_FAST        1 (adder)
          16 RETURN_VALUE

adder co_cellvars=() co_freevars=('n',)
          0 COPY_FREE_VARS      1

  4          2 RESUME            0

  5          4 LOAD_FAST        0 (x)
          6 LOAD_DEREF          1 (n)
          8 BINARY_OP           0 (+)
         12 RETURN_VALUE

```

Read the bytecode:

Offset	Opcode	Meaning
MAKE_CELL 0	Enclosing: <code>n</code> was <code>LOAD_FAST</code> slot 0; now wraps it in a <code>PyCellObject</code> and keeps the cell in the same slot. Subsequent <code>LOAD_DEREF n</code> in this scope and all enclosed scopes read through the cell.	
LOAD_CLOSURE 0	Push the cell for <code>n</code> onto the stack (not its contents — the cell itself).	
BUILD_TUPLE 1 + MAKE_FUNCTION 8 (closure)	Bundle cells into <code>func_closure</code> and create the child <code>PyFunctionObject</code> .	
COPY_FREE_VARS 1	Child prologue: copy closure tuple into its free-var array.	
LOAD_DEREF 1	Child: load <code>cell->ob_ref</code> for free var <code>n</code> . The arg <code>1</code> indexes <code>co_cellvars + co_freevars</code> in the combined deref array (implementation detail in <code>ceval.c</code>).	

4.4 The cell/free-var chain

Diagram omitted for brevity — see surrounding prose.

Two consequences visible in this diagram:

- **Mutation is shared.** `nonlocal n; n += 1` in the inner function does `STORE_DEREF → cell->ob_ref = new_value`, visible to the outer frame if it is still alive.
- **The cell outlives the outer frame.** The outer frame's `fastlocals[0]` holds a `PyCellObject*`, not a bare value. When the outer frame is freed (function returns), the cell's `ob_refcnt` stays `>0` because `func_closure` still holds it — the value survives.

4.5 Multi-level closures and re-capture

```
import dis

def outer(a):
    b = a + 1
    def middle(c):
        d = b + c
        def inner(e):
            return a + b + c + d + e
        return inner
    return middle

# Compiler flattens: inner's free vars include transitively closed vars
for fn, label in [(outer, "outer"), (outer(10), "middle"), (outer(10)(20), "inner")]:
    co = fn.__code__ if hasattr(fn, "__code__") else fn.__code__
    print(f"{label:8s} cellvars={co.co_cellvars} freevars={co.co_freevars} varnames={co.co_varnames}")

print("\n--- outer ---")
dis.dis(outer)
print("\n--- middle (closure of outer) ---")
dis.dis(outer(10))
print("\n--- inner (closure of middle) ---")
dis.dis(outer(10)(20))
```

```
outer    cellvars=('a', 'b') freevars=()          varnames=('a',
'b', 'middle')
middle   cellvars=('c', 'd') freevars=('a', 'b')  varnames=('c',
'd', 'inner')
inner    cellvars=()      freevars=('a', 'b', 'c', 'd')
varnames=('e',)
```

`middle` closes over `a`, `b` from `outer`; `inner` closes over `a`, `b` and `c`, `d` from `middle`. The compiler threads cells through each level — `inner` does not reach directly into `outer`'s frame, it reads cells that `middle` already captured.

4.6 `__closure__`, `inspect.getclosurevars`, and lifetime pitfalls


```

import inspect, types, gc, weakref

def make_handlers(items):
    handlers = []
    for item in items:
        def handler():
            return item          # closes over `item` – the CELL, not
                                # the value at def time
        handlers.append(handler)
    return handlers

hs = make_handlers([1, 2, 3])
print([h() for h in hs])      # [3, 3, 3] – classic late-
                                # binding pitfall
for h in hs:
    print(f"cell_contents={h.__closure__[0].cell_contents!r}")
    freevars={h.__code__.co_freevars}")

# Fix: capture by value via default argument (binds into func_defaults,
# not closure)
def make_handlers_fixed(items):
    handlers = []
    for item in items:
        def handler(item=item):
            return item
        handlers.append(handler)
    return handlers

print([h() for h in make_handlers_fixed([1, 2, 3])]) # [1, 2, 3]

# Introspection
def make_adder(n):
    m = n * 2
    def adder(x):
        return x + n + m
    return adder

add5 = make_adder(5)
print(f"\n__closure__={add5.__closure__}")
print(f"cell_contents={c.cell_contents for c in add5.__closure__}")
print(f"inspect.getclosurevars(add5)={inspect.getclosurevars(add5)}")
print(f"is CellType: {type(add5.__closure__[0])} is types.CellType={ty
pe(add5.__closure__[0]) is types.CellType}")

# Backend pitfall: closures pin objects – large closed-over buffer
# keeps memory alive
large = b"x" * 10_000_000
def leaked():
    return large[:10]          # closure holds reference to 10 MB even if
                                # you only need 10 bytes

```

```

print(f"\nleaked closure pins
{len(leaked.__closure__[0].cell_contents)} bytes")
# Fix: break the closure – copy what you need, or use a default arg, or
del the reference
def not_leaked(buf=large[:10]):
    return buf # default captures slice, not the original
# now `large` can be GC'd if no other reference exists

```

Output:

```

[3, 3, 3]
cell_contents=3 freevars=('item',)
...
[1, 2, 3]
__closure__=(<cell at 0x...: int object at 0x...>, <cell at 0x...: int
object at 0x...>)
cell_contents=[5, 10]
ClosureVars(nonlocals={'n': 5, 'm': 10}, globals={}, builtins={},
unbound=set())

```

```

# Demonstrating LOAD_DEREF vs LOAD_FAST vs LOAD_GLOBAL at the bytecode
level
import dis

x_global = 100

def demo_closure():
    x_local = 1
    x_cell = 2
    def inner():
        # x_local would be LOAD_FAST in demo_closure but is LOAD_DEREF
in inner
        # x_global is LOAD_GLOBAL everywhere
        # x_cell via nonlocal would be STORE_DEREF
        nonlocal x_cell
        x_cell += 1
        return x_local + x_cell + x_global
    return inner

print("--- demo_closure (enclosing) ---")
dis.dis(demo_closure)
print("\n--- inner (enclosed) ---")
dis.dis(demo_closure())

```

```

--- demo_closure (enclosing) ---
22          0 MAKE_CELL          0 (x_cell)
          2 MAKE_CELL          1 (x_local)  # actually
x_local becomes CELL because inner reads it
...
--- inner (enclosed) ---
          0 COPY_FREE_VARS      2
...
          6 LOAD_DEREF          1 (x_local)
          8 LOAD_DEREF          0 (x_cell)
         10 LOAD_DEREF          0 (x_cell)  # for the += RHS
          ...
         18 STORE_DEREF          0 (x_cell)  # nonlocal write
         20 LOAD_GLOBAL          1 (NULL + x_global)
          ...

```

Compiler rule: any local that is read by a nested scope becomes a `CELL` in the enclosing scope (even if never written with `nonlocal`); any name that resolves to an enclosing `CELL` becomes a `FREE` in the enclosed scope. `LOAD_DEREF` / `STORE_DEREF` / `DELETE_DEREF` are the only opcodes that dereference cells — `LOAD_FAST` / `STORE_FAST` never touch cells.

5. Decorators and `functools.wraps`

5.1 What a decorator executes

```

@decorator
def f(x): pass

```

is syntactic sugar for:

```

def f(x): pass
f = decorator(f)

```

At the bytecode level (`MAKE_FUNCTION` → `LOAD_NAME decorator` → `CALL` → `STORE_NAME f`), the decorator receives the original `PyFunctionObject`, returns a (usually new) callable, and the name `f` is rebound. No special opcode — decorators are ordinary calls.

5.2 The metadata problem and `functools.wraps`

A naïve wrapper replaces the callable and loses identity:

```

import functools, inspect

def naive_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
    # wrapper.__name__ == "wrapper", __code__
    # is wrapper's, signature lost

@naive_decorator
def greet(name: str) -> str:
    """Greet someone."""
    return f"hello {name}"

print(f"name={greet.__name__} qualname={greet.__qualname__} doc={greet.__doc__}")
print(f"signature={inspect.signature(greet)}") # (*args, **kwargs) - wrong
print(f"wrapped? {hasattr(greet, '__wrapped__')}")

def good_decorator(func):
    @functools.wraps(func) # copies __module__, __name__,
    # __qualname__, __doc__,
    def wrapper(*args, **kwargs): # __annotations__, __dict__,
    # __wrapped__
        print(f"calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@good_decorator
def greet2(name: str) -> str:
    """Greet someone."""
    return f"hello {name}"

print(f"\nname={greet2.__name__} qualname={greet2.__qualname__}
doc={greet2.__doc__}")
print(f"signature={inspect.signature(greet2)}")
# (name: str) -> str - correct via __wrapped__
print(f"wrapped={greet2.__wrapped__ is greet2.__wrapped__}")

```

`functools.wraps` is `functools.update_wrapper` +
`functools.WRAPPER_ASSIGNMENTS` / `WRAPPER_UPDATES` :

```
# functools – Lib/functools.py (simplified)
WRAPPER_ASSIGNMENTS = ('__module__', '__name__', '__qualname__', '__annotations__', '__doc__', '__type_params__')
WRAPPER_UPDATES = ('__dict__',)

def update_wrapper(wrapper, wrapped, assigned=WRAPPER_ASSIGNMENTS, updated=WRAPPER_UPDATES):
    for attr in assigned:
        try:
            value = getattr(wrapped, attr)
        except AttributeError:
            pass
        else:
            setattr(wrapper, attr, value)
    for attr in updated:
        getattr(wrapper, attr).update(getattr(wrapped, attr, {}))
    wrapper.__wrapped__ = wrapped
    return wrapper

def wraps(wrapped, assigned=WRAPPER_ASSIGNMENTS, updated=WRAPPER_UPDATES):
    return partial(update_wrapper, wrapped=wrapped, assigned=assigned, updated=updated)
```

What it does at the `PyFunctionObject` level:

- `wrapper.__wrapped__ = wrapped` — so `inspect.signature` and `inspect.unwrap` can chase the chain to recover the original `__code__` / `__annotations__`.
- `wrapper.__code__` is *not* replaced — the wrapper keeps its own `func_code` (the wrapper body), so `dis.dis(wrapper)` still shows the wrapper's bytecode. `__wrapped__.__code__` is where the original lives. This distinction matters when you profile: the wrapper frame appears in traces, not the wrapped function's frame directly.
- `wrapper.__dict__.update(wrapped.__dict__)` — custom attributes set on the original (e.g., `f.route = ...` in decorators that attach metadata) propagate.

5.3 Decorator stacking and `inspect.unwrap`

```
import functools, inspect

def deco_a(f):
    @functools.wraps(f)
    def w(*a, **kw): return f(*a, **kw)
    w._tag = "a"
    return w

def deco_b(f):
    @functools.wraps(f)
    def w(*a, **kw): return f(*a, **kw)
    w._tag = "b"
    return w

@deco_a
@deco_b
def target(x): return x

# Bottom-up: target = deco_a(deco_b(target_raw))
print(f"target.__wrapped__ is deco_b wrapper: {target.__wrapped__.__name__ == 'target'}")
print(f"unwrap once: {inspect.unwrap(target, stop=lambda f: hasattr(f, '_tag') and f._tag == 'b')}")
print(f"fully unwrapped: {inspect.unwrap(target)}")
print(f"unwrap chain: {[f.__name__ for f in [target, target.__wrapped__, target.__wrapped__.__wrapped__]]}")
```

Without `__wrapped__`, `inspect.signature`, FastAPI's dependency introspection (`inspect.signature` on the route handler to discover query/body params), and Sphinx autodoc all see the wrapper's `(*args, **kwargs)` — which is why FastAPI explicitly warns when you forget `functools.wraps`.

6. Generators — suspendable frames

6.1 What `yield` means to the compiler

The presence of `yield` or `yield from` anywhere in a function's body flips `CO_GENERATOR`. Calling the function no longer enters `ceval` — it allocates a `PyGenObject` whose frame is suspended at offset 0 and returns immediately. The caller's stack never enters the generator body until `next()` / `send()` resumes it.

```

/* Include/cpython/genobject.h - 3.11+ */
typedef struct {
    PyObject_HEAD
    PyObject *gi_weakreflist;
    PyObject *gi_name;           /* __name__ */
    PyObject *gi_qualname;      /* __qualname__ */
    PyObject *gi_code;          /* PyCodeObject* - the generator's
code */
    PyObject
*gi_frame;                     /* _PyInterpreterFrame* - suspended frame (or
NULL if closed) */
    PyObject *gi_modulename;
    char      gi_running;       /* re-entrance guard */
    PyObject *gi_exc_state;     /* saved exception state */
    /* ... */
} PyGenObject;

/* Object state - Lib/inspect.py / Include/cpython/genobject.h */
#define GEN_CREATED    0 /* not yet started */
#define GEN_RUNNING    1 /* currently executing (re-entry is an error) */
#define GEN_SUSPENDED  2 /* yielded, frame preserved, resumable */
#define GEN_CLOSED     3 /* exhausted or explicitly closed */

```

Relevant sources: `Objects/genobject.c`, `Python/ceval.c` handling of `YIELD_VALUE`, `RETURN_GENERATOR`, `SEND`, `RESUME`.

6.2 Generator bytecode — the suspend/resume switch

```

import dis, inspect

def gen_demo():
    x = yield 1
    y = yield x + 1
    return y * 2

dis.dis(gen_demo)

g = gen_demo()
print(f"state after creation: {inspect.getgeneratorstate(g)}") #
GEN_CREATED
print(f"next(g) = {next(g)}")
print(f"state after first yield: {inspect.getgeneratorstate(g)}") #
GEN_SUSPENDED
print(f"g.send(10) = {g.send(10)}")    # x = 10, yields 11
print(f"g.send(20) = {g.send(20)}")    # y = 20, returns 40 via
StopIteration.value

```

Disassembly:

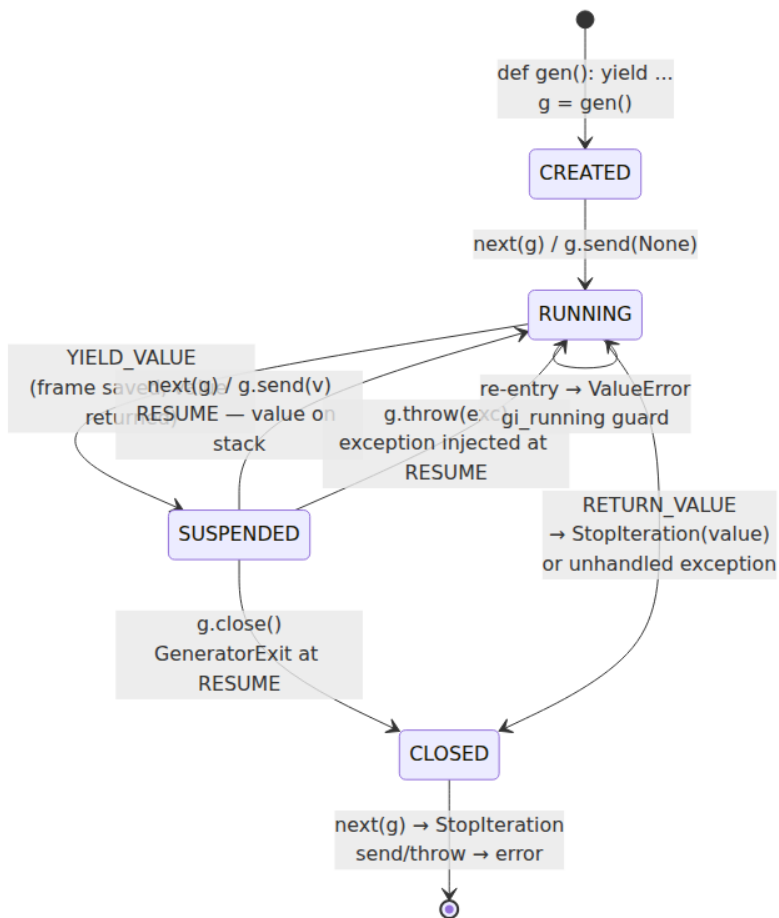
7	0 RETURN_GENERATOR	# prologue: wrap frame as generator object
	2 POP_TOP	
	4 RESUME	0
8	6 LOAD_CONST	1 (1)
	8 YIELD_VALUE	0 # pop 1, suspend frame, return 1 to caller
	10 RESUME	1 # resume point: stack holds sent value
	12 STORE_FAST	0 (x)
9	14 LOAD_FAST	0 (x)
	16 LOAD_CONST	1 (1)
	18 BINARY_OP	0 (+)
	22 YIELD_VALUE	1
	24 RESUME	1
	26 STORE_FAST	1 (y)
10	28 LOAD_FAST	1 (y)
	30 LOAD_CONST	2 (2)
	32 BINARY_OP	5 (*)
	36 RETURN_VALUE	# raises StopIteration(value) to caller
	38 POP_TOP	# (unreachable – ceval handles RETURN_VALUE)

Step through:

Opcode	Frame action	Caller sees
RETURN_GENERATOR + POP_TOP	Function entry: allocate <code>PyGenObject</code> with current frame, return it. Body not yet executed.	<code>g = gen_demo()</code> returns generator object; no code has run.
YIELD_VALUE	Saves <code>f_lasti</code> (instruction pointer), sets <code>GEN_SUSPENDED</code> , returns yielded value to caller. Frame's value stack and <code>fastlocals</code> remain intact.	<code>next(g)</code> receives <code>1</code> .

Opcode	Frame action	Caller sees
RESUME 1	Resume entry: expects the <code>send()</code> argument on the stack. RESUME's oparg encodes where we came from (0 = initial, 1 = yield, 2 = except, 3 = await). Validates generator is not <code>GEN_RUNNING</code> .	<code>g.send(10)</code> resumes and 10 is on the stack for <code>STORE_FAST x</code> .
RETURN_VALUE	Sets <code>GEN_CLOSED</code> , raises <code>StopIteration(value)</code> carrying the return value. Subsequent <code>next()</code> raises <code>StopIteration</code> immediately.	<code>g.send(20)</code> raises <code>StopIteration: 40</code> .

6.3 The generator state machine



`gi_running` is a re-entrance guard: `yield` inside a generator that recursively resumes itself (e.g., `list(g)` where `g` yields and the `__next__` machinery re-enters) raises `ValueError: generator already executing`. This is a bug, not a feature to work around.

```

import inspect, types

def gen():
    yield 1
    yield 2

g = gen()
print(inspect.getgeneratorstate(g)) # GEN_CREATED
print(inspect.isgenerator(g), inspect.isgeneratorfunction(gen))
print(isinstance(g, types.GeneratorType))

next(g)
print(inspect.getgeneratorstate(g)) # GEN_SUSPENDED
print(f"gi_frame={g.gi_frame} gi_code={g.gi_code.co_name}
gi_running={g.gi_running}")
print(f"gi_yieldfrom={g.gi_yieldfrom}") # None – set only during
yield-from

list(g) # exhaust
print(inspect.getgeneratorstate(g)) # GEN_CLOSED
print(f"gi_frame after close: {g.gi_frame}") # None – frame freed

```

6.4 send, throw, close — the generator protocol

SEND (3.11+) is the opcode that `yield` lowers to on the *calling* side; YIELD_VALUE is the callee side. The Python-level methods map to frame operations:

```

import dis

def echo():
    while True:
        received = yield
        print(f"received: {received}")

dis.dis(echo)

```

	0	RETURN_GENERATOR	
	2	POP_TOP	
	4	RESUME	0
4	6	LOAD_CONST	0 (None)
	8	YIELD_VALUE	
	10	RESUME	1
	12	STORE_FAST	0 (received)
...			

```

g = echo()
next(g)           # prime: advance to first YIELD_VALUE, discards None
g.send("hello")   # resumes at RESUME 1, stack = "hello" → STORE_FAST
received
g.send("world")

# throw: injects exception at the RESUME point as if `yield` raised it
def resilient():
    try:
        x = yield 1
    except ValueError as e:
        x = f"caught {e}"
    yield x

r = resilient()
print(next(r))     # 1
print(r.throw(ValueError, ValueError("oops"))) # caught oops

# close: injects GeneratorExit at RESUME; generator should exit cleanly
def closable():
    try:
        yield 1
        yield 2
    finally:
        print("cleanup")

c = closable()
print(next(c))     # 1
c.close()          # prints "cleanup", state → GEN_CLOSED
print(f"closed state: {__import__('inspect').getgeneratorstate(c)}")
try:
    next(c)
except StopIteration:
    print("StopIteration as expected after close")

```

- `gen.send(None)` is equivalent to `next(gen)` — the `RESUME 1` path pushes `None` onto the stack either way.
- `gen.throw(type, value, tb)` raises at the yield point. If the generator handles it, it resumes normally; if not, the exception propagates to the caller and the generator becomes `GEN_CLOSED`.
- `gen.close()` injects `GeneratorExit` — the generator's `finally` blocks run, but if it yields again after catching `GeneratorExit`, `RuntimeError` is raised (a generator that ignores `close()` is broken).

6.5 Generator frame lifetime and memory

A suspended generator holds its frame (`gi_frame`) alive, which in turn holds references to all `fastlocals`, the value stack, and `func_globals` / `func_closure`. A generator that is `GEN_SUSPENDED` but never resumed or closed leaks its frame until GC. In long-lived services this matters: a streaming generator that is abandoned mid-iteration (client disconnects before consuming the stream) holds its frame until the generator is GC'd or explicitly closed.

ASGI servers mitigate this by calling `aclose()` / `close()` on abandoned generators, but middleware that wraps generators must propagate `close()` — see §10.

7. `yield from` — transparent delegation

7.1 Why delegation needs an opcode

Manually delegating to a sub-generator is verbose and subtly wrong:

```
# Manual delegation – doesn't propagate send/throw/close correctly
def delegator_manual():
    sub = subgen()
    for val in sub:
        yield val          # loses send() values, throw() not forwarded
    return sub.return_value # not accessible – StopIteration.value
discarded by for
```

PEP 380 (`yield from`, Python 3.3) makes delegation *transparent*: `yield from iterable` yields every value from `iterable`, forwards `send()` / `throw()` / `close()` into it, and captures its return value.

7.2 Bytecode lowering

```
import dis

def subgen():
    yield 1
    yield 2
    return "done"

def delegator():
    result = yield from subgen()
    yield result

dis.dis(delegator)
```

```
13          0 RETURN_GENERATOR
          2 POP_TOP
          4 RESUME                    0

14          6 LOAD_GLOBAL              1 (NULL + subgen)
          18 PRECALL                    0
          22 CALL                      0
          32 GET_YIELD_FROM_ITER       0    # coerce to iterator (or
coroutine awaitable)
          34 LOAD_CONST                0 (None)
      >>   36 SEND                      2 (to 44)  # send None to prim
e subgen
          38 YIELD_VALUE                # yield whatever subgen y
ielded
          40 RESUME                    1
          42 JUMP_BACKWARD_NO_INTERRUPT 8 (to 36)  # loop: send
back in, yield out
      >>   44 POP_TOP                    # subgen exhausted ❏ Sto
pIteration.value on stack
          46 STORE_FAST                0 (result)

15          48 LOAD_FAST                0 (result)
          50 YIELD_VALUE
          52 RESUME                    1
          54 POP_TOP
          56 LOAD_CONST                0 (None)
          58 RETURN_VALUE
```

The loop at `36 SEND → 38 YIELD_VALUE → 40 RESUME → 42 JUMP_BACKWARD` is the delegation pump. `GET_YIELD_FROM_ITER` handles the dispatch: if the operand is a generator or coroutine, it returns it directly; if it is an iterable, it calls `PyObject_GetIter`; if it has `__await__`, it calls that. The

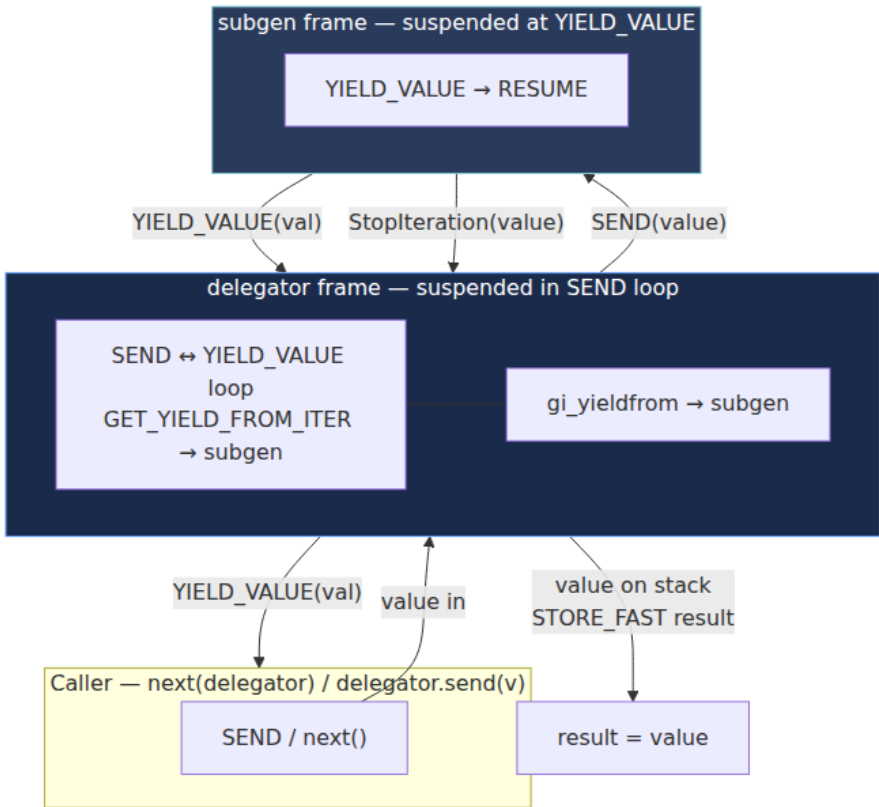
loop then shuttles values bidirectionally until the sub-iterator raises `StopIteration`, whose `.value` is left on the stack for `STORE_FAST` result. `gi_yieldfrom` is set while delegation is active:

```
def sub():
    yield 1
    return "sub done"

def outer():
    v = yield from sub()
    yield f"outer got {v}"

o = outer()
print(next(o))                # 1 – forwarded from sub
import inspect
print(f"gi_yieldfrom={o.gi_yieldfrom}") # <generator object sub
at ...>
print(f"sub state={inspect.getgeneratorstate(o.gi_yieldfrom)}") #
GEN_SUSPENDED
print(next(o))                # outer got sub done
print(inspect.getgeneratorstate(o)) # GEN_CLOSED
```

7.3 Delegation chain



`throw()` and `close()` propagate along the same chain: `delegator.throw(exc)` calls `subgen.throw(exc)` via `gi_yieldfrom`, and `delegator.close()` calls `subgen.close()`. If the sub-generator handles the exception and yields again, delegation resumes; if it propagates, the delegator sees the exception.


```

# throw propagation through yield from
def inner():
    try:
        yield 1
        yield 2
    except ValueError as e:
        yield f"inner caught {e}"
        return "inner done"

def outer():
    v = yield from inner()
    yield f"outer sees {v}"

o = outer()
print(next(o))                    # 1
print(o.throw(ValueError, ValueError("boom"))) # inner caught boom
print(next(o))                    # outer sees inner done

```

7.4 Operational consequences

`yield from` is the primitive behind generator-based streaming pipelines:

```

def read_chunks(path, size=8192):
    with open(path, "rb") as f:
        while chunk := f.read(size):
            yield chunk

def gzip_chunks(chunks):
    import gzip, io
    buf = io.BytesIO()
    gz = gzip.GzipFile(fileobj=buf, mode="wb")
    yield from compress_stream(chunks, gz, buf) # delegation to
compressor generator

# ASGI streaming
async def streaming_response(chunks):
    for chunk in chunks: # sync generator iterated in async
context
        yield chunk # would be async generator in real
ASGI

```

The `gi_yieldfrom` chain is visible to debuggers and to `inspect.getgeneratorstate` — a stuck streaming pipeline can be diagnosed by walking `gi_yieldfrom` links to find which generator in the chain is `GEN_SUSPENDED` vs. `GEN_RUNNING`.

8. Coroutines — from `yield -as-coroutine` (PEP 342) to `async def` (PEP 492)

8.1 PEP 342: generators grow `send / throw / close`

PEP 342 (Python 2.5) upgraded generators from one-way iterators to bidirectional coroutines. The same `yield` expression that previously only *produced* a value gained the ability to *receive* one (`x = yield y`). This made generator-based coroutines possible: an event loop could `send(result)` back into a generator that had `yield ed` a future.

`asyncio` before PEP 492 was built entirely on this: `@asyncio.coroutine` decorated a generator, `yield from` suspended it, and the event loop pumped `send / throw`.

```
# PEP 342 generator-based coroutine (pre-PEP 492, still valid)
import types

def old_style_coroutine():
    print("start")
    x = yield "waiting"
    print(f"resumed with {x}")
    return "done"

# Manually driving it like an event loop would
c = old_style_coroutine()
print(f"is generator: {inspect.isgenerator(c)}")
print(f"next → {next(c)}")          # waiting
try:
    c.send("hello")                  # resumed with hello
except StopIteration as e:
    print(f"returned: {e.value}")    # done
```

8.2 PEP 492: native coroutines — `async def` and `await`

PEP 492 (Python 3.5) introduced dedicated syntax and types so coroutines are not confused with generators:

- `async def f(): ...` — defines a **coroutine function** (`CO_COROUTINE`, `inspect.iscoroutinefunction`). Calling it returns a **coroutine object** (`PyCoroObject`, `types.CoroutineType`), not a generator.
- `await expr` — suspends the coroutine until `expr` is awaitable and complete. `expr` must be a coroutine, an `asyncio.Future`, or an object with `__await__`.

- `async with` / `async for` — desugar to `__aenter__` / `__aexit__` and `__aiter__` / `__anext__` (Section 9).

```
/* Include/cpython/genobject.h – coroutine object (3.11+) */
typedef struct {
    PyObject_HEAD
    PyObject *cr_name;
    PyObject *cr_qualname;
    PyObject *cr_code;           /* PyCodeObject* with CO_COROUTINE */
    PyObject *cr_frame;         /* _PyInterpreterFrame* */
    char      cr_running;
    PyObject *cr_exc_state;
    PyObject *cr_origin;        /* traceback origin for "was never
awaited" warning */
} PyCoroObject;
```

Despite the distinct type, the implementation is nearly identical to `PyGenObject` — same frame suspension, same `YIELD_VALUE` / `RESUME` / `SEND` machinery. The semantic difference is enforced at the type level: `await` is only valid inside `async def`, and `yield` inside `async def` creates an **async generator**, not a generator.

```
import dis, inspect, types

async def native_coro(x):
    await some_work(x)
    return x * 2

# Placeholder awaitable for disassembly
async def some_work(x): pass

print(f"iscoroutinefunction={inspect.iscoroutinefunction(native_coro)}")
print(f"isgeneratorfunction={inspect.isgeneratorfunction(native_coro)}")

c = native_coro(10)
print(f"type={type(c)}  iscoroutine={inspect.iscoroutine(c)}
isinstance CoroutineType={isinstance(c, types.CoroutineType)}")
print(f"CO_COROUTINE={bool(c.cr_code.co_flags &
inspect.CO_COROUTINE)}")
c.close() # suppress "was never awaited" warning

dis.dis(native_coro)
```

```

iscoroutinefunction=True
isgeneratorfunction=False
type=<class 'coroutine'> iscoroutine=True  isinstance CoroutineType=True
CO_COROUTINE=True
7      0 RETURN_GENERATOR
      2 POP_TOP
      4 RESUME                                0

8      6 LOAD_GLOBAL                        1 (NULL + some_work)
      18 LOAD_FAST                          0 (x)
      20 PRECALL                            1
      24 CALL                              1
      34 GET_AWAITABLE                      0 # ← await lowering: coer
ce to awaitable
      36 LOAD_CONST                          0 (None)
>> 38 SEND                                3 (to 46) # ← same SEND
loop as yield-from
      40 YIELD_VALUE
      42 RESUME                            3 # ← oparg 3 = await resu
me (vs 1 = yield)
      44 JUMP_BACKWARD_NO_INTERRUPT          8 (to 38)
>> 46 POP_TOP

9      48 LOAD_FAST                        0 (x)
      50 LOAD_CONST                          1 (2)
      52 BINARY_OP                          5 (*)
      56 RETURN_VALUE

```

Compare to `yield from` lowering (Section 7.2) — the structure is identical: `GET_YIELD_FROM_ITER` vs. `GET_AWAITABLE`, same `SEND` / `YIELD_VALUE` / `RESUME` loop. The difference is:

Aspect	<code>yield from</code>	<code>await</code>
Opcode	<code>GET_YIELD_FROM_ITER</code>	<code>GET_AWAITABLE</code>
RESUME oparg	1 (yield)	3 (await)
Allowed inside	Any function with <code>yield</code> (<code>CO_GENERATOR</code>)	Only <code>async def</code> (<code>CO_COROUTINE</code> or <code>CO_ASYNC_GENERATOR</code>)
Operand check	Accepts any iterable	Requires awaitable (<code>__await__</code> or <code>coroutine</code>); <code>TypeError</code> otherwise

Aspect	yield from	await
inspect guard	isgenerator	iscoroutine

8.3 What `GET_AWAITABLE` does

`GET_AWAITABLE` (`Python/ceval.c:TARGET(GET_AWAITABLE)`) implements the `await` coercion:

1. If the operand is a **coroutine** (`PyCoro_CheckExact`), return it — coroutines are directly awaitable.
2. If the operand has `__await__` (`tp_as_async->am_await`), call `__await__()` and verify the result is an iterator (the "awaitable iterator" protocol).
3. Otherwise raise `TypeError: object X can't be used in 'await' expression`.

This is why you can `await asyncio.sleep(0)` (a coroutine), `await future` (a `Future` with `__await__`), or `await custom_awaitable` (any object implementing `__await__`), but not `await 42`.

```

import inspect

# What counts as awaitable
class CustomAwaitable:
    def __await__(self):
        yield "step 1"
        yield "step 2"
        return "custom done"

print(f"inspect.isawaitable(CustomAwaitable())={inspect.isawaitable(CustomAwaitable())}")
print(f"has __await__: {hasattr(CustomAwaitable, '__await__')}")

# Awaitable that wraps a generator – the pattern Future uses internally
def gen_awaitable():
    v = yield from CustomAwaitable().__await__()
    return v

# Native coroutine awaiting the custom awaitable
import asyncio

async def demo():
    result = await CustomAwaitable()
    return result

# Drive manually to show the SEND loop
c = demo()
print(f"iscoroutine={inspect.iscoroutine(c)} state={inspect.getcoroutinestate(c)}") # CORO_CREATED
try:
    c.send(None) # would reach first yield inside __await__
except StopIteration as e:
    print(f"returned: {e.value}")

import dis
print("\n--- CustomAwaitable.__await__ ---")
dis.dis(CustomAwaitable.__await__)
print("\n--- demo (await CustomAwaitable) ---")
dis.dis(demo)

```

8.4 inspect guards — why isgenerator ≠ iscoroutine

```
import inspect, types

def gen(): yield 1
async def coro(): pass
async def agen(): yield 1

for fn in [gen, coro, agen]:
    obj = fn()
    print(f"{fn.__name__:6s} isgen={inspect.isgenerator(obj)}
iscoro={inspect.iscoroutine(obj)} "
        f"isagen={inspect.isasyncgen(obj)}
type={type(obj).__name__}")
    # clean up to avoid warnings
    if inspect.isgenerator(obj): obj.close()
    elif inspect.iscoroutine(obj): obj.close()
    elif inspect.isasyncgen(obj): obj.aclose()

print(f"\nisgenfunc(gen)={inspect.isgeneratorfunction(gen)}")
print(f"iscorofunc(coro)={inspect.iscoroutinefunction(coro)}")
print(f"isagenfunc(agen)={inspect.isasyncgenfunction(agen)}")

# States are parallel but distinct
print(f"\ngen state: {inspect.getgeneratorstate(gen())}")
c = coro(); print(f"coro state: {inspect.getcoroutinestate(c)}"); c.close()
ag = agen(); print(f"agen state: {inspect.getasyncgenstate(ag)}"); import asyncio; asyncio.run(ag.aclose())
```

gen	isgen=True	iscoro=False	isagen=False	type=generator
coro	isgen=False	iscoro=True	isagen=False	type=coroutine
agen	isgen=False	iscoro=False	isagen=True	type=async_generator

Mixing them is a `TypeError` at compile or runtime:

```

# SyntaxError at compile time – await outside async def
try:
    exec("def f(): await coro()")
except SyntaxError as e:
    print(f"SyntaxError: {e}")

# SyntaxError – yield inside async def without CO_ASYNC_GENERATOR
# nuance
# (actually allowed – it becomes an async generator, see §9)
async def also_agen():
    yield 1 # this makes also_agen an async generator function, not a
    # coroutine function
print(f"also_agen is asyncgen:
{inspect.isasyncgenfunction(also_agen)}")

# Runtime – yield from a coroutine inside a generator works, but await
in generator doesn't

```

9. Async generators and async comprehensions

9.1 `async def` + `yield` = async generator (PEP 525)

Adding `yield` inside `async def` changes the code flag from `CO_COROUTINE` to `CO_ASYNC_GENERATOR` and the return type from `CoroutineType` to `AsyncGeneratorType`. The distinction matters because the iteration protocol differs: sync generators use `__next__` / `send`, async generators use `__anext__` / `asend`.


```

import dis, inspect, types

async def async_gen(n):
    for i in range(n):
        await asyncio.sleep(0)    # suspend on await
        yield i                  # suspend on async yield

print(f"isasyncgenfunction={inspect.isasyncgenfunction(async_gen)}")
print(f"iscoroutinefunction={inspect.iscoroutinefunction(async_gen)}")
print(f"flags={hex(async_gen.__code__.co_flags)} ASYNCGEN={bool(async_gen.__code__.co_flags & 0x200)}")

ag = async_gen(3)
print(f"type={type(ag)} isasyncgen={inspect.isasyncgen(ag)}
instance AsyncGeneratorType={isinstance(ag,
types.AsyncGeneratorType)}")

dis.dis(async_gen)

```

```

isasyncgenfunction=True
iscoroutinefunction=False
flags=0x143 ASYNCGEN=True
type=<class 'async_generator'> isasyncgen=True isinstance AsyncGeneratorType=True
 7          0 RETURN_GENERATOR
          2 POP_TOP
          4 RESUME                                0

 8          6 LOAD_GLOBAL                        1 (NULL + range)
          ...
 9          22 LOAD_GLOBAL                       3 (NULL + asyncio)
          34 LOAD_ATTR                          4 (sleep)
          44 LOAD_CONST                         1 (0)
          46 PRECALL                            1
          50 CALL                               1
          60 GET_AWAITABLE                      0
          62 LOAD_CONST                         0 (None)
>>        64 SEND                              3 (to 72)
          66 YIELD_VALUE
          68 RESUME                             3
          70 JUMP_BACKWARD_NO_INTERRUPT         8 (to 64)
>>        72 POP_TOP

10          74 LOAD_FAST                        2 (i)
          76 YIELD_VALUE                        # async yield  susp
ends as async generator
          78 RESUME                             1      # resumes via
asend / __anext__
...

```

Two suspend points, two mechanisms:

- `await asyncio.sleep(0)` → `GET_AWAITABLE` / `SEND` / `YIELD_VALUE` / `RESUME 3` — yields to the event loop.
- `yield i` → `YIELD_VALUE` / `RESUME 1` — yields to the async-generator consumer via `__anext__`.

The consumer drives the async generator with `await anext(ag)` / `async for`:

```
import asyncio, inspect

async def async_gen(n):
    for i in range(n):
        await asyncio.sleep(0)
        yield i

async def consume():
    ag = async_gen(3)
    print(f"agen state: {inspect.getasyncgenstate(ag)}") #
    AGEN_CREATED
    async for val in ag:
        print(f" got {val} state={inspect.getasyncgenstate(ag)}") #
    AGEN_SUSPENDED between yields
    print(f"final state: {inspect.getasyncgenstate(ag)}") #
    AGEN_CLOSED

    # Manual driving with asend / athrow / aclose
    ag2 = async_gen(2)
    print(f"\nmanual: {await ag2.asend(None)}") # 0 - first yield
    print(f"manual: {await ag2.asend(None)}") # 1
    await ag2.aclose()
    print(f"after aclose: {inspect.getasyncgenstate(ag2)}")

asyncio.run(consume())
```

Async generators have their own state enum:

State	Value	Meaning
AGEN_CREATED	0	Not yet started
AGEN_RUNNING	1	Currently executing (re-entrance raises)
AGEN_SUSPENDED	2	Yielded, resumable via <code>asend</code> / <code>anext</code>
AGEN_CLOSED	3	Exhausted or <code>aclose()</code> 'd; <code>gi_frame</code> is <code>NULL</code>

Their opcodes mirror generators but on the async path: `GET_AITER` / `GET_ANEXT` / `GET_AWAITABLE` (see next section) and `ASYNC_GEN_WRAP` / `YIELD_VALUE` with `RESUME` oparg 1 for the async-yield path vs. 3 for the await path.

9.2 Async comprehensions and `async for` / `async with`

`async for` and `async with` desugar to awaitable protocol calls. The compiler emits `GET_AITER`, `GET_ANEXT`, `GET_AWAITABLE`, `BEFORE_ASYNC_WITH`:

```
import dis

async def async_iteration(items):
    # async for
    async for x in items:
        print(x)

async def async_context(mgr):
    # async with
    async with mgr:
        print("inside")

# Need an async iterable for disassembly to show the pattern
dis.dis(async_iteration)
print("---")
dis.dis(async_context)

# Async comprehension (PEP 530 – Python 3.6+)
async def async_comp(items):
    return [x async for x in items if x]

dis.dis(async_comp)
```

```

6          0 RETURN_GENERATOR
          2 POP_TOP
          4 RESUME                                0

8          6 LOAD_FAST                            0 (items)
          8 GET_AITER                            # items.__aiter__()
         10 LOAD_CONST                            0 (None)
__() >> 12 GET_ANEXT                              # __aiter__().__anext__
__()
         14 GET_AWAITABLE                        0          # await the
awaitable from __anext__
         16 LOAD_CONST                          0 (None)
        >> 18 SEND                              3 (to 26)
         20 YIELD_VALUE
         22 RESUME                              3
         24 JUMP_BACKWARD_NO_INTERRUPT          8 (to 18)
        >> 26 POP_TOP
         28 STORE_FAST                          1 (x)

...

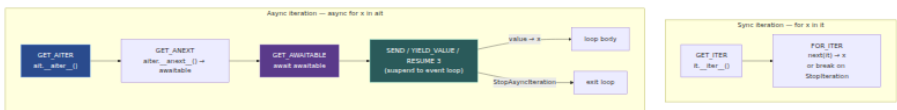
14         6 LOAD_FAST                            0 (mgr)
          8 BEFORE_ASYNC_WITH                    # mgr.__aenter__ / __
aexit__ setup
         10 GET_AWAITABLE                        0          # await __aenter__()
        ...
         38 WITH_EXCEPT_START                  # await __aexit__()

on exit
...

22         ... GET_AITER / GET_ANEXT / GET_AWAITABLE ... # async com
p uses same trio

```

The pattern `GET_AITER → GET_ANEXT → GET_AWAITABLE → SEND/YIELD_VALUE/RESUME` is the async-iteration pump — the direct analogue of `GET_ITER → FOR_ITER` for sync iteration, but with an `await` in the middle because `__anext__` returns an awaitable.



```

# End-to-end async iterable
import asyncio

class AsyncCounter:
    def __init__(self, n): self.n = n; self.i = 0
    def __aiter__(self): return self
    async def __anext__(self):
        if self.i >= self.n:
            raise StopAsyncIteration
        await asyncio.sleep(0)
        val = self.i
        self.i += 1
        return val

async def demo():
    # async for uses the protocol above
    async for v in AsyncCounter(3):
        print(f"async for: {v}")
    # async comprehension – same opcodes, collected into a list
    result = [v async for v in AsyncCounter(3)]
    print(f"async comp: {result}")
    # await inside async comp
    result2 = [await asyncio.sleep(0, result=v) async for v in AsyncCounter(3)]
    print(f"await in comp: {result2}")

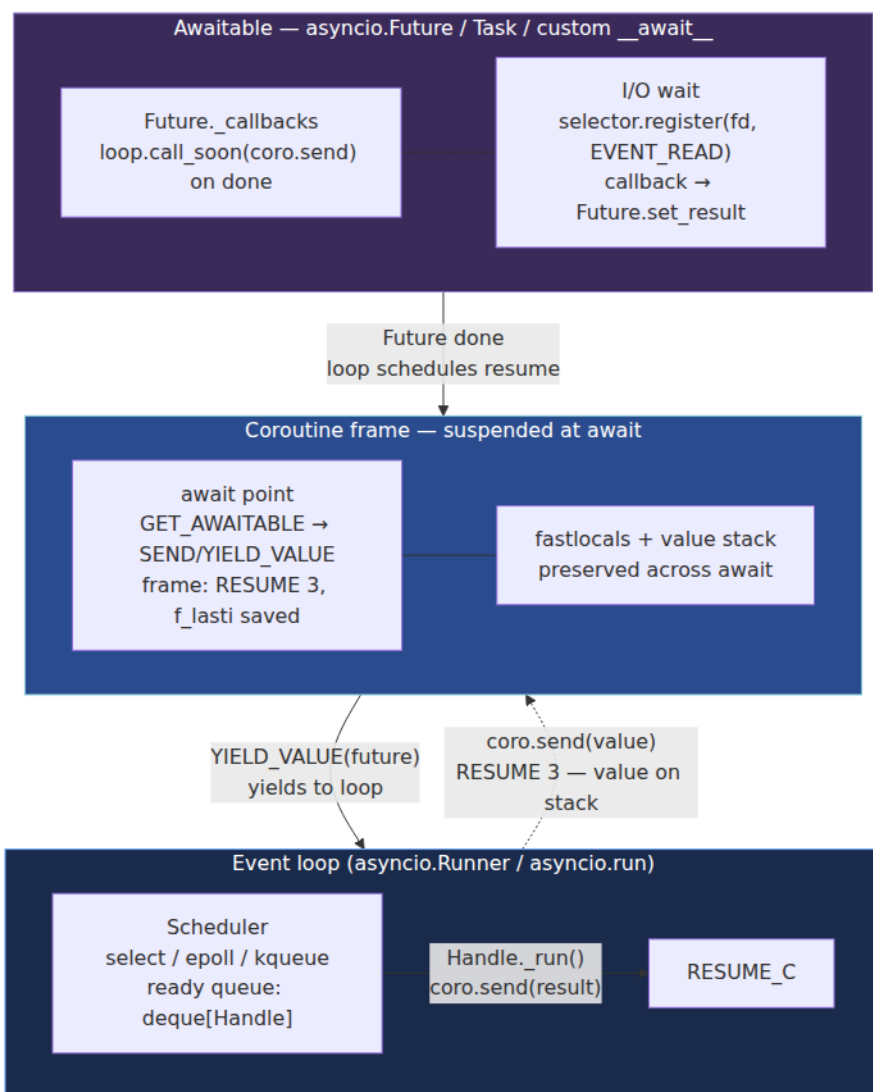
asyncio.run(demo())

```

10. The async/await stack — event loop → coroutine → awaitable

10.1 The full await chain

There is no concurrency without a loop. Every `await` yields control back to the event loop, which parks the coroutine and resumes it when the awaitable completes. The stack, bottom to top:



Concretely, `await asyncio.sleep(0)` executes:

1. `asyncio.sleep(0)` returns a coroutine object (`PyCoroObject`, `CO_COROUTINE`).
2. `GET_AWAITABLE` calls `coro.__await__()` → an iterator that yields a `Future`.

3. `SEND / YIELD_VALUE` yields that `Future` to the caller — which, if the caller is the event loop's `Task`, registers a callback:

```
future.add_done_callback(lambda f: task_step(task)) .
```

4. The loop parks the task and runs other ready handles.

5. When the future completes (`loop.call_later(0, future.set_result, None)`), the loop schedules `task._step` which does

```
coro.send(result) — re-entering at RESUME 3 with the result on the stack.
```

`Task` itself is `PyCoroObject` wrapped with scheduling state (`_callbacks`, `_loop`, `_state`). The `await` inside a task and the `await` at the top level share the same `GET_AWAITABLE / SEND` lowering — the loop is just the outermost driver that never suspends.

10.2 Tracing an `await` through `ceval`

Instrument with `PYTHONASYNCIODEBUG=1` and `tracemalloc` to see the chain:

```

import asyncio, dis, sys, types, inspect

async def fetch(n):
    await asyncio.sleep(0)
    return n * 2

async def handler(x):
    y = await fetch(x)
    return y + 1

# Disassembly shows both awaits lower identically
print("--- fetch ---")
dis.dis(fetch)
print("\n--- handler (await fetch) ---")
dis.dis(handler)

# Drive handler without an event loop – manual SEND pumping to show the
yield chain
h = handler(21)
print(f"\nhandler type: {type(h)}")
iscoroutine={inspect.iscoroutine(h)}")
print(f"handler cr_frame: {h.cr_frame}")
print(f"handler cr_code: {h.cr_code.co_name}")

# The coroutine yields a Future to its caller; collecting it shows the
awaitable chain
try:
    yielded = h.send(None) # enters handler, hits await fetch, yields
fetch's awaitable
    print(f"first yield: {yielded!r:.200}")
    # yielded is the coroutine object for fetch() – which itself is
suspended at its own await
    print(f"yielded type: {type(yielded)} iscoroutine={inspect.iscorou
tine(yielded)}")
    h.close()
    yielded.close() if hasattr(yielded, 'close') else None
except StopIteration as e:
    print(f"returned: {e.value}")

# With an actual loop – the normal path
async def main():
    result = await handler(21)
    print(f"\nwith loop: handler(21) = {result}") # 43

asyncio.run(main())

```

The disassembly confirms both `fetch` and `handler` have the same `GET_WAITABLE → SEND → YIELD_VALUE → RESUME 3` pattern — `await` does not care whether the awaitable is a coroutine, a `Future`, or a custom

`__await__`; the lowering is uniform and the event loop is the only component that interprets the yielded object.

11. `inspect` and `types` — runtime introspection

Backend frameworks rely on introspection to wire handlers, serialize responses, and generate OpenAPI specs. The `inspect` and `types` modules expose the `PyFunctionObject` / `PyCodeObject` / `PyGenObject` / `PyCoroObject` fields through a stable Python API — use it instead of reading `__code__` attributes directly when you can.

11.1 inspect — the framework author's toolkit

```

import inspect, types, functools, asyncio

# --- Function introspection ---
def sample(a, b=2, *args, c, d=99, **kw) -> int:
    """Sample with every param kind."""
    return a + b

sig = inspect.signature(sample)
print(f"signature: {sig}")
print(f"return_annotation: {sig.return_annotation}")
for name, param in sig.parameters.items():
    print(f" {name:6s} kind={param.kind.name:20s} default={param.default!r:10s} annotation={param.annotation!r}")

# unwrap decorators
def deco(f):
    @functools.wraps(f)
    def w(*a, **kw): return f(*a, **kw)
    return w

@deco
def wrapped(x: int) -> int: return x

print(f"\nwrapped signature: {inspect.signature(wrapped)}")
# (x: int) -> int via wrapped
print(f"unwrap: {inspect.unwrap(wrapped)}")
print(f"getsource (first line): {inspect.getsource(wrapped).splitlines()[0]}")

# closure
def make_pow(exp):
    def power(base): return base ** exp
    return power

p = make_pow(3)
print(f"\ngetclosurevars: {inspect.getclosurevars(p)}")
print(f"getfullargspec: {inspect.getfullargspec(sample)}")

# --- Generator / coroutine / asyncgen guards ---
def gen(): yield 1
async def coro(): pass
async def agen(): yield 1

for fn in [gen, coro, agen]:
    print(f"\n{fn.__name__}: isgeneratorfunction={inspect.isgeneratorfunction(fn)} "
          f"iscoroutinefunction={inspect.iscoroutinefunction(fn)} "
          f"isasyncgenfunction={inspect.isasyncgenfunction(fn)} "
          f"isfunction={inspect.isfunction(fn)} isbuiltin={inspect.isbuiltin(fn)}")

```

```

for obj, label in [(gen(), "gen"), (coro(), "coro"), (agen(), "agen")]:
    print(f"{label}: isgenerator={inspect.isgenerator(obj)}
iscoroutine={inspect.iscoroutine(obj)} "
        f"isasyncgen={inspect.isasyncgen(obj)}
isawaitable={inspect.isawaitable(obj)}")
    # cleanup
    if inspect.isgenerator(obj): obj.close()
    elif inspect.iscoroutine(obj): obj.close()
    else: asyncio.run(obj.aclose()) if hasattr(obj, 'aclose') else None

# States
g = gen()
print(f"\ngenerator state CREATED: {inspect.getgeneratorstate(g)}")
next(g)
print(f"generator state SUSPENDED: {inspect.getgeneratorstate(g)}")
list(g)
print(f"generator state CLOSED: {inspect.getgeneratorstate(g)}")

c = coro()
print(f"\ncoroutine state CREATED: {inspect.getcoroutinestate(c)}")
c.close()
print(f"coroutine state CLOSED: {inspect.getcoroutinestate(c)}")

# Frame introspection – what the debugger sees
def outer():
    x = 42
    def inner():
        return x
    return inner

f = outer()
print(f"\nframe info: {inspect.getfile(f)}:
{f.__code__.co_firstlineno}")
print(f"code qualname: {f.__code__.co_qualname}")
print(f"code freevars: {f.__code__.co_freevars} cellvars:
{f.__code__.co_cellvars}")
print(f"closure: {f.__closure__} cell_contents={c.cell_contents for
c in f.__closure__}")

```

11.2 types — the type predicates and constructors

```
import types

# Type predicates – isinstance checks against the C types
print(f"FunctionType: {types.FunctionType}")
print(f"LambdaType is FunctionType: {types.LambdaType is types.FunctionType}")
print(f"GeneratorType: {types.GeneratorType}")
print(f"CoroutineType: {types.CoroutineType}")
print(f"AsyncGeneratorType: {types.AsyncGeneratorType}")
print(f"CellType: {types.CellType}")
print(f"CodeType: {types.CodeType}")
print(f"MethodType: {types.MethodType} BuiltinFunctionType: {types.BuiltinFunctionType}")

# Constructing functions dynamically – what decorators and codegen do
code = compile("def f(x): return x * 2", "<dynamic>", "exec")
# exec puts 'f' in a namespace dict; grab its code object
ns = {}
exec(code, ns)
f = ns['f']
print(f"\ndynamic f(21)={f(21)} type={type(f)}")

# Low-level construction via types.FunctionType – what functools.wraps
# does NOT do but codegen does
import types as t

def original(x): return x + 1

# New function sharing the same code object but with different globals/
# defaults
new_func = t.FunctionType(
    original.__code__,
    {"__builtins__": __builtins__}, # fresh globals
    name="new_func",
    argdefs=(10,), # defaults
    closure=None,
)
print(f"new_func(5)={new_func(5)}") # 6 – same code, same defaults
# wiring
print(f"new_func.__code__ is original.__code__: {new_func.__code__ is original.__code__}")

# CellType – constructing closures manually (rare, but codegen does it)
cell = t.CellType(42)
print(f"cell: {cell} cell_contents={cell.cell_contents}")
cell.cell_contents = 99
print(f"mutated cell_contents={cell.cell_contents}")
```

11.3 FastAPI's introspection — why `__wrapped__` matters operationally

FastAPI resolves query/path/body parameters by calling `inspect.signature(route_handler)` at startup. If your authentication decorator does not use `functools.wraps`, `signature` sees `(*args, **kwargs)` and FastAPI cannot generate the OpenAPI schema or validate requests — it falls back to treating every parameter as untyped. Any.

```
import inspect, functools
from typing import Annotated # or fastapi.Query in real code

# Correct – FastAPI can introspect
def auth_guard_correct(func):
    @functools.wraps(func)
    async def wrapper(*args, **kwargs):
        # check auth
        return await func(*args, **kwargs)
    return wrapper

@auth_guard_correct
async def route_correct(user_id: int, limit: int = 10) -> dict:
    return {"user_id": user_id, "limit": limit}

print(f"correct signature: {inspect.signature(route_correct)}")
for name, p in inspect.signature(route_correct).parameters.items():
    print(f"  {name}: annotation={p.annotation!r} default={p.default!r}")

# Broken – FastAPI sees (*args, **kwargs)
def auth_guard_broken(func):
    async def wrapper(*args, **kwargs):
        return await func(*args, **kwargs)
    return wrapper

@auth_guard_broken
async def route_broken(user_id: int, limit: int = 10) -> dict:
    return {"user_id": user_id, "limit": limit}

print(f"\nbroken signature: {inspect.signature(route_broken)}")
# inspect.signature follows __wrapped__ if present; without it, no recovery
print(f"broken has __wrapped__: {hasattr(route_broken, '__wrapped__')}")
print(f"unwrap broken: {inspect.unwrap(route_broken)}") # still the wrapper
print(f"unwrap correct: {inspect.unwrap(route_correct)}") # original route_correct
```

12. Backend lens — async frameworks and generator streaming

12.1 FastAPI / Starlette: how `async def` handlers actually run

```
HTTP request → uvicorn (asyncio loop) → Starlette routing → FastAPI  
dependency resolution → your handler
```

Every layer is an `await`:

- **uvicorn** (ASGI server) runs the `asyncio` event loop and translates `scope/receive/send` into `await` chains.
- **Starlette** routing matches the path and `await`s the endpoint. Middleware is a stack of `await call_next(request)` wrappers — each one is a coroutine that suspends at `await`.
- **FastAPI** dependency injection resolves `Depends(...)` by inspecting signatures (Section 11.3) and `await`ing each dependency that is a coroutine function.

The thread model: **one event loop per worker process, many coroutines cooperatively interleaved**. There is no thread per request. A coroutine that blocks without `await`ing — `time.sleep(1)`, a synchronous `requests.get()`, a CPU-bound loop — starves the loop and delays every concurrent request in that worker.

```

# Starvation demo – never do this in a handler
import asyncio, time

async def blocking_handler():
    time.sleep(1)          # blocks the event loop – no GET_AWAITABLE,
    never yields
    return {"ok": True}

async def correct_handler():
    await asyncio.sleep(1) # GET_AWAITABLE → yields to loop, other
    tasks run
    return {"ok": True}

async def measure():
    start = asyncio.get_event_loop().time()
    await asyncio.gather(blocking_handler(), blocking_handler())
    print(f"blocking (sequential despite gather): {asyncio.get_event_lo
op().time() - start:.2f}s")

    start = asyncio.get_event_loop().time()
    await asyncio.gather(correct_handler(), correct_handler())
    print(f"non-blocking (concurrent):
{asyncio.get_event_loop().time() - start:.2f}s")

asyncio.run(measure())
# blocking: ~2.00s – second handler waits for first's sleep
# correct: ~1.00s – both sleeps overlap on the loop

```

Operational rules:

- **Run blocking work off-loop:** `await loop.run_in_executor(None, blocking_fn)` or `await asyncio.to_thread(blocking_fn)` (3.9+) — the vectorcall happens on a thread-pool thread, not the loop.
- **Do not `await` inside tight loops without yielding elsewhere** — if you `await` 10k DB rows one by one, the loop is fine; if you compute 10k rows synchronously between awaits, you starve it between suspension points.
- **Size your worker count to CPU cores, not concurrency** — `uvicorn --workers 4` gives 4 event loops; concurrency within each is via `await`, not threads.

12.2 Generator-based streaming — `StreamingResponse` and backpressure

`StreamingResponse` (Starlette) and its equivalents accept a sync or async iterable and drive it via the ASGI `send` channel:


```

from typing import AsyncGenerator, Generator
import asyncio

# Sync generator streaming – simple, but the generator runs on the
# event loop thread
def csv_rows(rows) -> Generator[bytes, None, None]:
    yield b"id,name\n"
    for row in rows:
        yield f"{row['id']},{row['name']}\n".encode()

# Async generator streaming – can await between chunks (e.g., fetch
# next page)
async def s3_stream(key: str) -> AsyncGenerator[bytes, None]:
    # paginated S3 GetObject – each page is an await, each yield is a
    # chunk to the client
    page_token = None
    while True:
        page = await s3_get_page(key, page_token) # GET_AWAITABLE →
loop
        if not page.data:
            break
        yield page.data # YIELD_VALUE →
ASGI send
        page_token = page.next_token

# ASGI server drives the iterable:
# sync: for chunk in iterable: await send({"type":
# "http.response.body", "body": chunk})
# async: async for chunk in iterable: await send(...)

```

What can go wrong:

Failure	Root cause (this chapter)	Symptom	Fix
Slow client blocks loop	Sync generator + <code>yield</code> chunk directly to <code>await send()</code> without buffering; <code>send</code> awaits socket drain	One slow client stalls all handlers	Use async generator + <code>await send</code> ; or buffer with <code>anyio</code> memory stream
Abandoned generator leaks frame	Client disconnects mid-stream; <code>GEN_SUSPENDED</code> frame holds <code>rows</code> / connection	Memory grows per disconnect	Catch <code>asyncio.CancelledError</code> generator; finally: <code>cleanup()</code> or <code>aclose()</code>

Failure	Root cause (this chapter)	Symptom	Fix
<code>yield from</code> not propagating <code>close</code>	Middleware wraps generator without forwarding <code>close()</code>	Inner generator's <code>finally</code> never runs; DB cursor leaked	Implement <code>__iter__</code> + <code>close</code> / <code>aclose</code> forwarding or use <code>yield from</code> / <code>async yield</code> which does it automatically
CPU-bound chunk encoding starves loop	<code>yield expensive_transform(chunk)</code> computed synchronously	P99 latency spike under load	<code>await loop.run_in_executor(Nor expensive_transform, chunk)</code>

```

# Correct streaming generator with cleanup – what every production
streaming endpoint needs
import asyncio

def result_stream(cursor):
    """Sync generator with guaranteed cleanup – close() propagates."""
    try:
        for row in cursor:
            yield f"{row}\n".encode()
    finally:
        cursor.close()          # runs on gen.close() or exhaustion

async def async_result_stream(cursor):
    """Async generator with async cleanup."""
    try:
        async for row in cursor:          # GET_AITER / GET_ANEXT /
GET_AWAITABLE
            yield f"{row}\n".encode()
    finally:
        await cursor.aclose()             # runs on aclose() or
CancelledError

# Middleware must propagate close – yield from does this automatically
def logging_wrapper(inner):
    print("stream start")
    try:
        yield from inner                  # throw/close propagate via
gi_yieldfrom
    finally:
        print("stream end")

# Without yield from – manual and error-prone
def manual_wrapper(inner):
    it = iter(inner)
    try:
        while True:
            try:
                val = next(it)
            except StopIteration as e:
                return e.value
            yield val
    finally:
        if hasattr(it, 'close'):
            it.close()

```

12.3 Observability — tracing callables, generators, and coroutines in production

- **Profiles:** `py-spy --native` attributes samples to `PyFunction_Type.tp_vectorcall`, `gen_send_ex`, and `coro_send`. High `CALL_PY_EXACT_ARGS` specialization rate (visible via `PYTHONPROFILEIMPORTTRACE` or `sys.monitoring` in 3.12+) means your hot-path calls are monomorphic — good.
 - **Tracing:** OpenTelemetry ASGI middleware wraps the `await` `call_next` chain. Each `await` boundary is a span boundary only if you create one — `GET_AWAITABLE` / `RESUME` are invisible to `cProfile` by default (3.12+ `sys.monitoring` can hook `PY_START` / `PY_RESUME` / `PY_YIELD`).
 - **Memory:** `tracemalloc` + `inspect.getgeneratorstate` / `getasynccgenstate` to find leaked `GEN_SUSPENDED` / `AGEN_SUSPENDED` generators. A count of `GEN_SUSPENDED` growing without bound indicates abandoned streams.
-

Key takeaways

- `PyFunctionObject` is a heap object with `func_code` (immutable bytecode), `func_globals` (shared module dict), `func_defaults` / `func_kwdefaults` (evaluated once at def time), `func_closure` (tuple of `PyCellObject` cells), and `func_annotations`. Assigning `f.__code__` swaps bytecode without reallocating the wrapper — how decorators and hot-reload work.
- Calls go `PUSH_NULL` / `LOAD_GLOBAL` → `PRECALL` → `CALL` → `vectorcall` (PEP 590) → `_PyFunction_Vectorcall` → new `_PyInterpreterFrame` on the C stack. The 4-word `CALL` inline cache specializes to `CALL_PY_EXACT_ARGS` for monomorphic sites — the fast path that makes per-request dependency injection scale.
- Closures are cells: `MAKE_CELL` wraps an enclosing local in a `PyCellObject`; `LOAD_CLOSURE` / `BUILD_TUPLE` / `MAKE_FUNCTION` bundles cells into `func_closure`; `COPY_FREE_VARS` installs them in the child; `LOAD_DEREF` / `STORE_DEREF` read/write through `cell->ob_ref`. The cell outlives the enclosing frame, and `nonlocal` is `STORE_DEREF`.
- `functools.wraps` copies `WRAPPER_ASSIGNMENTS` + `WRAPPER_UPDATES` and sets `__wrapped__` so `inspect.signature` and `inspect.unwrap` can

recover the original. Forgetting it breaks FastAPI's dependency introspection and OpenAPI generation.

- Generators: `CO_GENERATOR` makes `call` allocate a `PyGenObject` and return immediately. `YIELD_VALUE` suspends (`GEN_SUSPENDED`, `f_lasti` saved), `RESUME 1` resumes with the `send()` value on the stack. `RETURN_GENERATOR` is the entry prologue; `RETURN_VALUE` raises `StopIteration(value)`. States are `CREATED` → `RUNNING` ↔ `SUSPENDED` → `CLOSED`.
 - `yield from` is `GET_YIELD_FROM_ITER` → `SEND / YIELD_VALUE / RESUME` loop with `gi_yieldfrom` linking. It forwards `send/throw/close` automatically and captures `StopIteration.value` — the primitive behind streaming delegation.
 - Native coroutines (`async def`, `CO_COROUTINE`, `PyCoroObject`) share the generator frame machinery but are a distinct type. `await` lowers to `GET_AWAITABLE` (coerce via `__await__`) → same `SEND / YIELD_VALUE / RESUME 3` loop. The oparg difference (`RESUME 1` for `yield` vs. `RESUME 3` for `await`) is how `ceval` enforces `await`-only-inside-`async def`.
 - Async generators (`CO_ASYNC_GENERATOR`, `async def + yield`) use `YIELD_VALUE` for `async` yields and `GET_AWAITABLE` for `awaits` — two suspend paths in one frame. `async for` lowers to `GET_AITER` → `GET_ANEXT` → `GET_AWAITABLE` → `SEND/YIELD/RESUME`; `async with` to `BEFORE_ASYNC_WITH` + `awaits` on `__aenter__ / __aexit__`.
 - The event loop is a `send()` pump: `await` yields a `Future` to the loop, the loop parks the coroutine, and on completion schedules `coro.send(result)` which re-enters at `RESUME 3`. Blocking without `await` starves the loop — offload with `run_in_executor / to_thread`.
 - `inspect` (`signature`, `unwrap`, `getclosurevars`, `is*function / is*`, `get*generatorstate`) and `types` (`FunctionType`, `CoroutineType`, `AsyncGeneratorType`, `CellType`) are the stable API over `__code__ / __closure__ / __defaults__`. Use them in framework code; read `__code__` directly only when you need `co_freevars / co_cellvars / co_flags`.
-

Further reading

- PEP 590 — Vectorcall: a fast calling protocol for CPython. <https://peps.python.org/pep-0590/> (*pinned*)
- PEP 492 — Coroutines with `async` and `await` syntax. <https://peps.python.org/pep-0492/> (*pinned*)
- PEP 342 — Coroutines via Enhanced Generators (`send/throw/close`, `yield expressions`). <https://peps.python.org/pep-0342/> (*pinned*)
- PEP 380 — Syntax for Delegating to a Subgenerator (`yield from`). <https://peps.python.org/pep-0380/>
- PEP 525 — Asynchronous Generators. <https://peps.python.org/pep-0525/>
- PEP 530 — Asynchronous Comprehensions. <https://peps.python.org/pep-0530/>
- `inspect` — Inspect live objects. <https://docs.python.org/3/library/inspect.html> (*pinned*)
- `types` — Dynamic type creation and names for built-in types. <https://docs.python.org/3/library/types.html>
- `dis` — Disassembler for Python bytecode. <https://docs.python.org/3/library/dis.html>
- `functools` — Higher-order functions and operations on callable objects (`wraps`, `update_wrapper`, `partial`). <https://docs.python.org/3/library/functools.html>
- CPython source — `Objects/funcobject.c` — `PyFunctionObject` definition, `func_new`, `_PyFunction_Vectorcall`. <https://github.com/python/cpython/blob/main/Objects/funcobject.c> (*pinned*)
- CPython source — `Objects/genobject.c` — `PyGenObject`, `PyCoroObject`, `PyAsyncGenObject`, `gen_send_ex`, `coro_send`. <https://github.com/python/cpython/blob/main/Objects/genobject.c>
- CPython source — `Objects/cellobject.c` / `Include/cpython/cellobject.h` — `PyCellObject`. <https://github.com/python/cpython/blob/main/Objects/cellobject.c>
- CPython source — `Python/ceval.c` — `TARGET(MAKE_FUNCTION)`, `TARGET(MAKE_CELL)`, `TARGET(LOAD_DEREF)`, `TARGET(STORE_DEREF)`, `TARGET(CALL)`, `TARGET(GET_AWAITABLE)`, `TARGET(SEND)`, `TARGET(YIELD_VALUE)`, `TARGET(RETURN_GENERATOR)`. <https://github.com/python/cpython/blob/main/Python/ceval.c>

- CPython source — `Include/cpython/code.h` — `PyCodeObject` flags (`CO_GENERATOR`, `CO_COROUTINE`, `CO_ASYNC_GENERATOR`). <https://github.com/python/cpython/blob/main/Include/cpython/code.h>
-

Source files referenced in this chapter are from CPython 3.11/3.12 (*main* branch). Bytecode offsets and cache-word counts are 3.11+ adaptive wordcode; `dis` output was verified on CPython 3.11.15. Behavior of `vectorcall`, `funcobject.c`, and `genobject.c` is as of 2024-2025; minor field additions (e.g., `func_typeparams` for PEP 695 in 3.12+) are noted where relevant.

Chapter 8 — Exceptions, Context Managers, and the Unwinding Machinery

What this chapter covers. Python's error handling looks declarative — `try/except/finally`, `with`, `raise ... from` — but underneath it is a precise unwinding machine built from an exception table, a per-thread error indicator, traceback chains, and a protocol for deterministic cleanup. Since Python 3.11 that machine was rewritten: the old `SETUP_FINALLY` / block-stack model is gone, replaced by a compact `co_exceptiontable`, `PUSH_EXC_INFO` / `POP_EXCEPT` / `RERAISE` lowering, and a `WITH_EXCEPT_START` path for context managers. PEP 654 added `ExceptionGroup` and `except*` on top. This chapter dissects the full stack — `BaseException` hierarchy, the C-level `PyErr_*` indicator, normalization, the exception table, bytecode lowering, traceback objects, chained exceptions (PEP 3134, PEP 678), the context-manager protocol (including `contextlib.contextmanager` and `BEFORE_WITH`), and `ExceptionGroup` splitting — with real `dis` output on CPython 3.11, `ceval.c` unwind traces, and backend patterns for structured error handling, resource management, and safe traceback logging.

Learning goals — after this chapter you should be able to:

- Draw the `BaseException` hierarchy and explain why `except Exception` does not catch `KeyboardInterrupt` / `SystemExit` / `GeneratorExit`, and how that shapes backend signal handling.

- Describe the per-thread exception indicator (`tstate->curexc_*` / `PyErr_SetString` / `PyErr_Fetch` / `PyErr_NormalizeException`) and the type → value → traceback normalization that `raise` triggers.
- Decode `co_exceptiontable` entries (`start`, `end` → `target`, `depth`, `lasti`) and contrast them with the pre-3.11 `PyTryBlock` block stack, including how `ceval.c:exception_unwind` searches the table.
- Read `dis` output for `try` / `except` / `finally` / `else` and map each handler to `PUSH_EXC_INFO`, `CHECK_EXC_MATCH` / `CHECK_EG_MATCH`, `POP_EXCEPT`, `RERAISE`, and unconditional `JUMP` opcodes.
- Explain `PyTracebackObject` chains vs. `traceback.StackSummary` vs. formatted text, and how `sys.exception()` / `sys.exc_info()` expose the current exception.
- Trace chained exceptions: `__context__` (implicit, PEP 3134) vs. `__cause__` (explicit `raise ... from`) vs. `__suppress_context__`, and PEP 678 `BaseException.add_note()`.
- Walk the `WITH` protocol step-by-step: `BEFORE_WITH` → `__enter__` → `body` → `__exit__` with `WITH_EXCEPT_START`, and why a truthful `__exit__` return suppresses the exception.
- Implement correct `__exit__` (swallow vs. propagate), use `contextlib.contextmanager` / `ExitStack`, and explain where `SETUP_WITH` went.
- Use `ExceptionGroup` / `BaseExceptionGroup` and `except*` (PEP 654) — construction, `split` / `subgroup`, handler ordering, and interaction with `finally`.
- Apply the backend lens: structured exception taxonomies for services, context managers for DB connections / tracing spans / locks, and logging tracebacks without leaking PII.

Prerequisites. Chapter 3 dissected `co_code`, `wordcode`, `ceval` dispatch, and the exception table at a high level; Chapter 2 covered `PyObject` / `PyTypeObject` and reference counting; Chapter 6 covered descriptors and `__exit__` lookup via `type`. This chapter assumes you can read `dis` output and skim `Python/ceval.c`. Volume 11 (Reliability) gives the service-level view of error budgets — here you see how the interpreter unwinds.

1. Why the unwinding machinery is a backend concern

Backend engineers tend to treat `try / except` as syntax. In production it is load-bearing infrastructure:

- **Every request handler is an unwind boundary.** An unhandled exception in a FastAPI / gRPC handler becomes a 500, a log line, and a trace span. Whether that exception carries a `__cause__` chain, a note, or a suppressed context determines whether on-call can diagnose it at 3 AM.
- **Resource leaks are exception-path bugs.** A DB connection, a file handle, or a distributed lock held across an exception that skips `finally / __exit__` is not a Python bug — it is an outage. `with` and `try / finally` lowering is your resource safety net. If you do not understand `WITH_EXCEPT_START` you cannot reason about whether your connection pool actually returns connections on error.
- **Traceback formatting is a data-leak vector.**
`traceback.format_exc()` includes source lines, local variable reprs (if you use `StackSummary` tricks), and exception messages that may contain PII — user emails, tokens, SQL fragments. Logging the raw traceback to a centralized system without filtering is a compliance incident.
- **Structured concurrency needs `ExceptionGroup`.**
`asyncio.TaskGroup` (3.11+) and `trio` raise `ExceptionGroup` when multiple child tasks fail. If your code does `except Exception` you will silently miss half the failures — you need `except*`.

Get the unwinding model wrong and you get silent suppression, leaked connections, or unactionable alerts. Get it right and exceptions become structured, observable signals.

2. The exception model — `BaseException` and the hierarchy

2.1 The hierarchy

Every raiseable object is an instance of `BaseException` (a `PyObject` subclass defined in `Objects/exceptions.c`). The hierarchy is deliberately shallow but semantically stratified:

```

BaseException
├── SystemExit          # raised by sys.exit() – carries .code
├── KeyboardInterrupt  # raised on SIGINT – must not be caught by generic handlers
├── GeneratorExit       # raised into a generator on .close() – must propagate
├── BaseExceptionGroup # 3.11+, PEP 654 – container for multiple exceptions
├── └── ExceptionGroup
└── Exception          # what most code should inherit from
    ├── StopIteration / StopAsyncIteration
    ├── ArithmeticError
    │   ├── ZeroDivisionError, OverflowError, FloatingPointError
    │   └── LookupError
    │       ├── KeyError, IndexError
    │       ├── ValueError, TypeError, RuntimeError, AttributeError, ...
    │       ├── OSError (IOError alias)
    │       │   ├── ConnectionError, FileNotFoundError, TimeoutError, ...
    │       └── ImportError, ModuleNotFoundError
    └── ExceptionGroup (also reachable as ExceptionGroup via BaseExceptionGroup)

```

The critical split is `BaseException` vs. `Exception`. Bare `except:` and `except Exception:` catch only the `Exception` subtree. `KeyboardInterrupt`, `SystemExit`, and `GeneratorExit` deliberately sit *outside* `Exception` so that a generic service handler does not swallow Ctrl-C, `sys.exit()`, or generator teardown:

```

# Backend anti-pattern – swallows signals and exit
try:
    run_server()
except Exception:      # OK – lets KeyboardInterrupt / SystemExit propagate
    log.exception("handler failed")

try:
    run_server()
except BaseException:  # DANGEROUS – catches KeyboardInterrupt, masks SIGTERM
    log.exception("handler failed")

```

In `Objects/exceptions.c` each exception type is a `PyTypeObject` with `tp_base` set to its parent. `PyErr_GivenExceptionMatches(err, exc)` walks `tp_mro` to implement `except` matching — it is the C equivalent of `issubclass(type(err), exc)`. `CHECK_EXC_MATCH` in bytecode calls the same logic.

2.2 What `raise` actually does

`raise` has three forms, all lowered to `RAISE_VARARGS` (opcode 176):

```
raise                                # re-raise current exception (RERAISE
path)
raise ValueError("bad")             # raise instance
raise ValueError                     # raise type – ceval instantiates it
raise ValueError("bad") from e      # explicit cause
raise ValueError("bad") from None   # suppress context
```

At the C level the interpreter calls:

```
PyErr_SetString(ValueError, "bad")    // type + message → indicator
PyErr_SetObject(type, value)          // type + instance
PyErr_SetExcInfo(type, value, traceback) // full triple
```

These set the per-thread exception indicator on `PyThreadState`:

```
// Include/cpython/pystate.h (simplified)
typedef struct _PyThreadState {
    PyObject *curexc_type; // legacy, now aliased to exc_info
    PyObject *curexc_value;
    PyObject *curexc_traceback;
    _PyErr_StackItem *exc_info; // stack of active handlers
    (PUSH_EXC_INFO)
    // ...
} PyThreadState;
```

On 3.11+ the canonical storage is `tstate->curexc_*` plus the `exc_info` stack manipulated by `PUSH_EXC_INFO` / `POP_EXCEPT`. `PyErr_Fetch(&type, &value, &tb)` clears the indicator and returns the triple (caller owns the references). `PyErr_Restore(type, value, tb)` sets it without incrementing refcounts (steals references). `PyErr_Clear()` discards it. `PyErr_Occurred()` tests whether `curexc_type != NULL` — the single branch `ceval` checks after every opcode that can fail.

Normalization (`PyErr_NormalizeException`) ensures `value` is an instance of `type`: if `raise ValueError` was used (`type`, not instance), it calls `PyObject_CallNoArgs(ValueError)` to instantiate; if `value` is already an instance whose type is a subclass of `type`, it updates `type` to `type(value)`. Traceback is attached at this point. Normalization happens once, on first handler entry or when `PyErr_Fetch` is called — not on every `PyErr_SetString`.

3. The exception table — inline table (3.11+) vs. the old block stack

3.1 The old world: `PyTryBlock` and `SETUP_FINALLY`

Before 3.11 `try/except/finally/with/for` were tracked by a **block stack** — a C array of `PyTryBlock` on the frame:

```
// Include/cpython/frameobject.h (≤3.10)
typedef struct {
    int b_type;           // SETUP_FINALLY, SETUP_WITH, SETUP_FINALLY etc.
    int b_handler;        // bytecode offset of handler
    int b_level;          // value-stack depth at block entry
} PyTryBlock;
```

The compiler emitted explicit `SETUP_FINALLY` handler, `SETUP_WITH` handler, `GET_ITER / SETUP_FINALLY`-for-loops, and `POP_BLOCK` at block end. At runtime `ceval` pushed/popped `PyTryBlock` entries as it entered/left `try` blocks. On exception, `ceval` walked the block stack top-down to find the innermost handler whose range contained the faulting offset.

Cost: every `try` paid a `SETUP_FINALLY` dispatch even on the non-exceptional path, `POP_BLOCK` on exit, and a branch on every opcode to check `why_code` for unwind. `co_stacksize` had to account for block-stack depth. The compiler and `ceval` both had to agree on block types.

3.2 The new world: `co_exceptiontable` (3.11+)

3.11 removed the block stack entirely. In its place each `PyCodeObject` carries `co_exceptiontable` — a bytes object encoding a compact table of entries (see `Objects/codeobject.c:exception_table` and `Python/ceval.c:exception_unwind`):

```
Each entry: start_offset, end_offset → target_offset, stack_depth,
push_lasti
    start, end : half-open bytecode range protected by this handler
    target      : handler entry offset (a PUSH_EXC_INFO)
    stack_depth: value-stack depth to unwind to before jumping
    push_lasti : if set, push current lasti onto stack for RERAISE /
inline finally
```

Encoding is varint-compressed: offsets are stored as deltas from the previous entry, values as unsigned varints (`_PyVarint_Read /`

`_PyVarint_Write` in `Python/codeobject.c`). The table is sorted by `start`. Lookup is a linear scan (tables are small — rarely >10 entries) in `get_exception_handler()` / `exception_unwind()`.

No opcode is emitted to "enter" a `try` on the non-exceptional path. The protected range is purely declarative in the table. The fast path — no exception raised — executes zero extra bytecodes.

`dis` renders the table after the bytecode:

```
import dis

def simple_try(x):
    try:
        return int(x)
    except ValueError:
        return 0

dis.dis(simple_try)
```

Output on CPython 3.11:

```

14          0 RESUME                      0

15          2 NOP

16          4 LOAD_GLOBAL                  1 (NULL + int)
          16 LOAD_FAST                    0 (x)
          18 PRECALL                      1
          22 CALL                        1
          32 RETURN_VALUE

>> 34 PUSH_EXC_INFO

17          36 LOAD_GLOBAL                  2 (ValueError)
          48 CHECK_EXC_MATCH
          50 POP_JUMP_FORWARD_IF_FALSE    4 (to 60)
          52 POP_TOP

18          54 POP_EXCEPT
          56 LOAD_CONST                    1 (0)
          58 RETURN_VALUE

17  >> 60 RERAISE                        0
    >> 62 COPY                          3
          64 POP_EXCEPT
          66 RERAISE                      1
ExceptionTable:
  4 to 30 -> 34 [0]
 34 to 52 -> 62 [1] lasti
 60 to 60 -> 62 [1] lasti

```

Read the table: "bytecode offsets 4..30 (the `try` body) on exception jump to 34 (`PUSH_EXC_INFO`). Offsets 34..52 (the `except ValueError` test) on exception jump to 62 (re-raise handler). Stack depth 0 or 1, `lasti` pushed when needed for nested unwind."

Compare `try/finally`, which duplicates the `finally` body for both the non-exceptional fallthrough and the exceptional table target:

```

def try_finally(x):
    try:
        return int(x)
    except ValueError as e:
        print(e)
        return 0
    finally:
        print("cleanup")

dis.dis(try_finally)

```

```

25      0 RESUME      0

26      2 NOP

27      4 LOAD_GLOBAL      1 (NULL + int)
      16 LOAD_FAST      0 (x)
      18 PRECALL      1
      22 CALL      1

32      32 LOAD_GLOBAL      3 (NULL + print)
      44 LOAD_CONST      1 ('cleanup')
      46 PRECALL      1
      50 CALL      1
      60 POP_TOP
      62 RETURN_VALUE
>> 64 PUSH_EXC_INFO

28      66 LOAD_GLOBAL      4 (ValueError)
      78 CHECK_EXC_MATCH
      80 POP_JUMP_FORWARD_IF_FALSE      41 (to 164)
      82 STORE_FAST      1 (e)

29      84 LOAD_GLOBAL      3 (NULL + print)
      96 LOAD_FAST      1 (e)
      98 PRECALL      1
     102 CALL      1
     112 POP_TOP

30     114 POP_EXCEPT
     116 LOAD_CONST      0 (None)
     118 STORE_FAST      1 (e)
     120 DELETE_FAST      1 (e)

32     122 LOAD_GLOBAL      3 (NULL + print)
     134 LOAD_CONST      1 ('cleanup')
     136 PRECALL      1
     140 CALL      1
     150 POP_TOP
     152 LOAD_CONST      2 (0)
     154 RETURN_VALUE
>> 156 LOAD_CONST      0 (None)
     158 STORE_FAST      1 (e)
     160 DELETE_FAST      1 (e)
     162 RERAISE      1

28  >> 164 RERAISE      0
    >> 166 COPY      3
     168 POP_EXCEPT
     170 RERAISE      1
    >> 172 PUSH_EXC_INFO

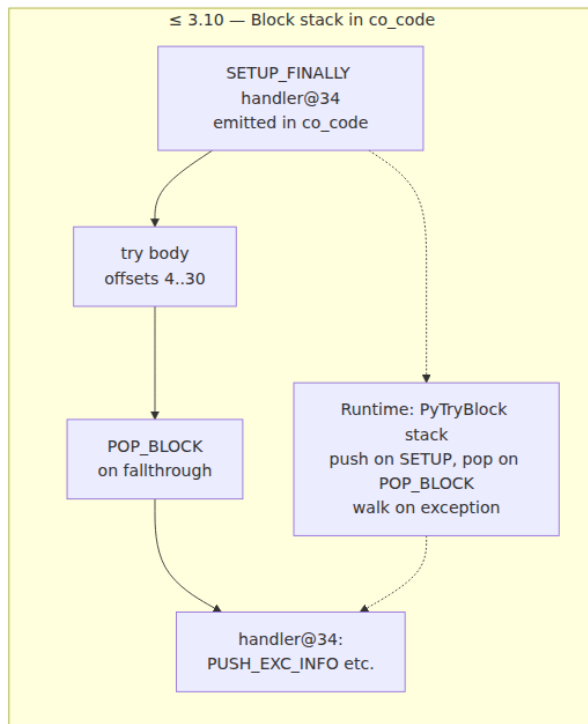
```

```

32      174 LOAD_GLOBAL      3 (NULL + print)
      186 LOAD_CONST        1 ('cleanup')
      188 PRECALL            1
      192 CALL              1
      202 POP_TOP
      204 RERAISE           0
    >> 206 COPY            3
      208 POP_EXCEPT
      210 RERAISE           1
ExceptionTable:
  4 to 30 -> 64 [0]
  64 to 82 -> 166 [1] lasti
  84 to 112 -> 156 [1] lasti
  114 to 120 -> 172 [0]
  156 to 164 -> 166 [1] lasti
  166 to 170 -> 172 [0]
  172 to 204 -> 206 [1] lasti

```

Notice the `finally` body at offsets 32..62 (non-exceptional return) is repeated at 122..150 (after `except` success) and at 174..204 (unwinding path) — the compiler inlines `finally` at every exit point. The exception table stitches those copies together.



compiler change

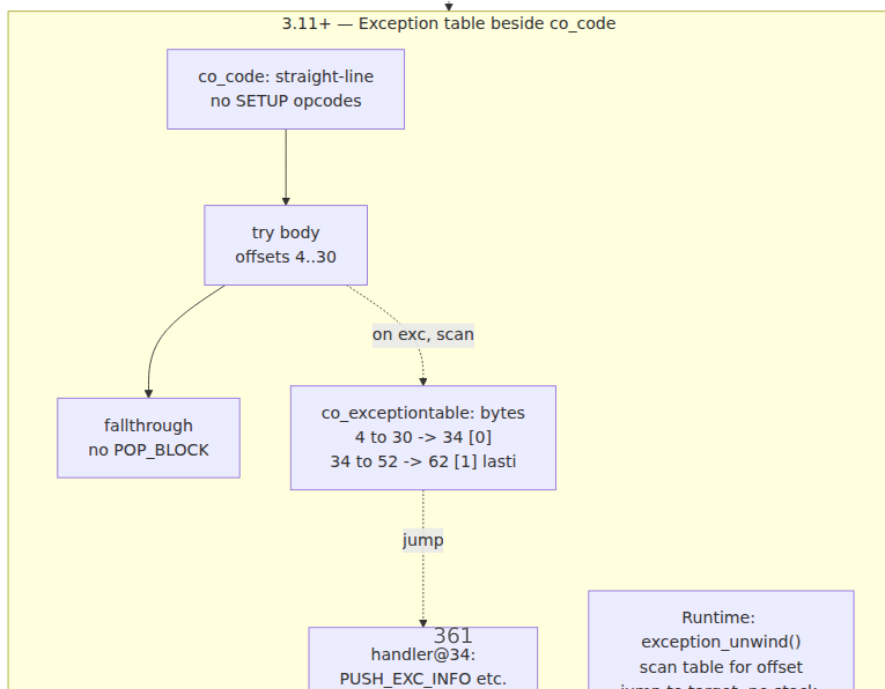


Diagram 1 — Exception table vs. block stack. Pre-3.11 every `try` emitted `SETUP_FINALLY / POP_BLOCK` into the bytecode and maintained a runtime `PyTryBlock` stack. Since 3.11 the bytecode is straight-line; the `co_exceptiontable` declaratively maps protected ranges to handler offsets and `ceval` scans it only on the exceptional path.

4. Bytecode lowering — `try / except / finally / else`

4.1 The handler opcodes

Once `exception_unwind` has jumped to a handler target, five opcodes orchestrate the handler body. All are cheap — they only run on the exceptional path:

Opcode	Stack effect	Role
<code>PUSH_EXC_INFO</code>	<code>→ exc_info</code> (pushes <code>tstate->exc_info</code> frame)	Enter handler: saves previous <code>exc_info</code> , pushes current <code>type/value/tb</code> as new <code>exc_info</code> , clears indicator so handler body can run. Always the first instruction at a table target.
<code>CHECK_EXC_MATCH</code>	<code>type exc_val tb exc_type → type exc_val tb bool</code>	Tests <code>PyErr_GivenExceptionMatches(exc_val, exc_type)</code> . Consumes the tested type, leaves <code>type/val/tb + bool</code> .
<code>POP_JUMP_FORWARD_IF_FALSE</code>	<code>bool →</code>	Branches to next <code>except</code> clause or <code>RERAISE</code> if no match.
<code>POP_EXCEPT</code>	<code>type val tb → (pops exc_info)</code>	Exit handled <code>except</code> : pops <code>exc_info</code> stack, restores previous handler, clears traceback. Paired with <code>PUSH_EXC_INFO</code> .

Opcode	Stack effect	Role
<code>RERAISE</code>	varies by oparg	Re-raises. <code>RERAISE 0</code> = re-raise current <code>exc_info</code> (no stack change, used for <code>except fallthrough</code>). <code>RERAISE 1</code> = pop <code>exc_info</code> then re-raise. <code>RERAISE 2</code> = used by <code>WITH_EXCEPT_START</code> path.
<code>COPY i</code> / <code>SWAP</code>	stack shuffle	Used in the two-instruction re-raise epilogue (<code>COPY 3</code> ; <code>POP_EXCEPT</code> ; <code>RERAISE 1</code>) to preserve the exception triple while popping <code>exc_info</code> .

`JUMP_FORWARD` / `JUMP_BACKWARD` handle the non-exceptional fallthrough: after a `try` body succeeds, an unconditional jump skips over the handler. `NOP` at the `try` start is a placeholder for the compiler's exception-table range start (it makes the range non-empty).

4.2 `try / except / else / finally` — complete lowering

```
import dis

def full_form(x):
    try:
        v = int(x)
    except ValueError as e:
        v = 0
        print(f"value error: {e}")
    except TypeError:
        v = -1
    else:
        print("no exception")
    finally:
        print("done")
    return v

dis.dis(full_form)
```

Key observations on 3.11:

```

RESUME
NOP                                     # try start marker
LOAD_GLOBAL int / LOAD_FAST x / PRECALL / CALL # try body
POP_JUMP? [ ] none, fallthrough goes to else/finally
PUSH_EXC_INFO                          # handler for ValueError
    CHECK_EXC_MATCH ValueError
    POP_JUMP_FORWARD_IF_FALSE [ ] next handler
    STORE_FAST e / body / POP_EXCEPT
    JUMP_FORWARD [ ] finally
PUSH_EXC_INFO                          # handler for TypeError (actually same
target, chained checks)
    CHECK_EXC_MATCH TypeError
    POP_JUMP_FORWARD_IF_FALSE [ ] RERAISE
    POP_EXCEPT / body
    JUMP_FORWARD [ ] finally
RERAISE 0                              # no except matched [ ] propagate
COPY 3 / POP_EXCEPT / RERAISE 1 # re-raise epilogue
# else block (only on no-exception path)
# finally block [ ] inlined 3[ ]: fallthrough, after-except, unwind
ExceptionTable:
    try_body      -> handler0 [0]
    handler0_range -> reraise_epilogue [1] lasti
    else/finally  -> unwind_handler [0 or 1] lasti

```

Rules the compiler follows:

- `else` runs only if the `try` body did not raise — it is *outside* the protected range, reached by fallthrough `JUMP_FORWARD` over the handlers. If `else` itself raises, it is *not* caught by the `except` clauses above it.
- `finally` is inlined at every exit: normal fallthrough, each `except` success path, and the unwind path. The unwind copy ends with `RERAISE 0` to continue propagation after cleanup.
- as `e` (`STORE_FAST e`) is followed by `POP_EXCEPT` then later `STORE_FAST None; DELETE_FAST e` to clear the exception reference and break the traceback cycle (see Section 5).

4.3 Unwind search — `exception_unwind` up the frame stack

When an opcode sets the exception indicator and the current offset has no table entry covering it, `ceval` does not immediately propagate to the caller. It walks *frames*, not just table entries:

Frame 0: innermost
handler() — fault offset
18
co_exceptiontable:
4..30 → 34

1. scan Frame 0 table
4..30 contains 18? YES
→ jump to 34,
PUSH_EXC_INFO

Execute handler in
Frame 0

handler does RERAISE
or no match

exception_unwind
scan again?

no more entries
in Frame 0

Diagram 2 — Unwind search up the frame stack. `ceval.c:handle_eval_breaker` / `exception_unwind` first scans the current frame's `co_exceptiontable` for a covering entry. On miss or `RERAISE`, it pops the frame and scans the caller, repeating until a handler is found or the thread boundary is reached. Each frame's table is independent; there is no global handler list.

In C this is roughly:

```
// Python/ceval.c - simplified unwind loop
PyObject *exc = tstate->curexc_value; // set by failing opcode
int offset = frame->instr_ptr - co->co_code_adaptive;
for (;;) {
    int handler = get_exception_handler(co, offset, &depth, &lasti);
    if (handler >= 0) {
        // Found: unwind value stack to depth, optionally push lasti,
        // set instr_ptr = handler, PUSH_EXC_INFO, continue dispatch
        break;
    }
    // No handler in this frame - pop frame
    frame = pop_frame(tstate);
    if (frame == NULL) {
        // No more frames - unhandled
        PyErr_PrintEx(1);
        break;
    }
    offset = frame->instr_ptr - co->co_code_adaptive;
}
```

The `lasti` flag matters for `finally` re-raise: the handler needs the faulting offset to decide whether the `finally` itself should be protected.

5. Traceback objects — `PyTracebackObject`, `StackSummary`, and formatting

5.1 `PyTracebackObject` chains

A traceback is not a string — it is a linked list of `PyTracebackObject` nodes, one per frame in the unwind path:

```
// Include/cpython/traceback.h
typedef struct _traceback {
    PyObject_HEAD
    struct _traceback *tb_next; // linked list – older caller
    PyFrameObject *tb_frame;   // borrowed frame (holds co_name,
    co_filename)
    int tb_lasti;               // bytecode offset at failure
    int tb_lineno;              // computed from co_positions +
    tb_lasti
} PyTracebackObject;
```

On exception, `PyTraceBack_Here(frame)` is called as the frame unwinds — it allocates a `PyTracebackObject`, sets `tb_next` to the previous `curexc_traceback`, and chains it. The youngest frame (where the exception was raised) is the *head*; `tb_next` walks toward the oldest caller. `BaseException.__traceback__` points to the head.

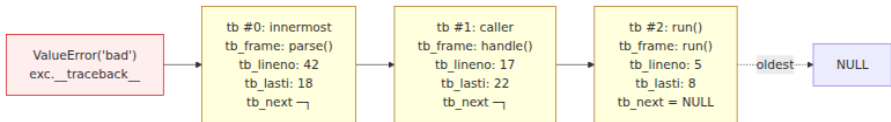


Diagram 6 — Traceback object chain. Each `PyTracebackObject` holds `tb_frame` / `tb_lineno` / `tb_lasti` and `tb_next` links to the caller. `exc.__traceback__` is the head (youngest frame); `tb_next` walks outward. The chain is built during unwind by `PyTraceBack_Here` and torn down when the exception is cleared or `__traceback__` is reassigned.

Key properties:

- Tracebacks hold strong references to frames, which hold references to locals — they can keep large objects alive. `except E as e: ...` without `del e` / implicit clear at block exit leaks the traceback cycle. CPython clears `e` at `POP_EXCEPT + DELETE_FAST` for this reason.
- `tb_lineno` is computed lazily from `tb_lasti` via `co_positions` (`_PyCode_Addr2Line`). Mutating `co_positions` does not retroactively change existing tracebacks.
- `sys.exception()` (3.11+, replaces `sys.exc_info()[1]` in most code) returns the current `curexc_value` with its `__traceback__` already attached. `sys.exc_info()` returns `(type, value, traceback)` — prefer `sys.exception()` in new code; it is faster and does not expose the type separately.

5.2 `traceback` module — `StackSummary` and formatting

The `traceback` module (`Lib/traceback.py`) converts the `PyTracebackObject` chain into Python-level summaries and text:

```
import traceback, sys

try:
    1 / 0
except ZeroDivisionError:
    exc_type, exc_val, exc_tb = sys.exc_info()

    # Low-level: walk PyTracebackObject chain
    tb = exc_val.__traceback__
    while tb is not None:
        print(f" frame={tb.tb_frame.f_code.co_name}
line={tb.tb_lineno} lasti={tb.tb_lasti}")
        tb = tb.tb_next

    # Mid-level: StackSummary – structured, no string formatting yet
    summary = traceback.extract_tb(exc_val.__traceback__)
    # summary is StackSummary[FrameSummary(filename, lineno, name,
line, locals?)]
    for frame in summary:
        print(frame) # FrameSummary

    # High-level: formatted text (what log handlers emit)
    print("".join(traceback.format_exception(exc_val)))
    # or: traceback.print_exc()

    # 3.11+: format_exception(exc) takes the exception directly, reads
__notes__
    print("".join(traceback.format_exception(exc_val)))
```

`StackSummary` is the right layer for backend observability: it is structured (you can filter frames, redact filenames, strip PII from `frame.line`), serializable, and does not eagerly read source files. `format_exception` is the presentation layer — it reads `__cause__` / `__context__` / `__notes__` and renders the familiar `Traceback` (most recent call last): text.

PII caution: `format_exception` with `chain=True` (the default) renders `__cause__` and `__context__` messages. If an exception message contains `f"user {email} not found"` or a SQL fragment, that PII lands in your log aggregator. Prefer structured logging that captures `type(exc).__name__` + `exc.args[0]` after sanitization, and attach `StackSummary` with source lines stripped in production.


```

# Safe traceback logging pattern
import traceback, logging

logger = logging.getLogger("api")

def log_exception_safely(exc: BaseException) -> None:
    # Structured – no source lines, no locals, redacted message
    summary = traceback.StackSummary.extract(
        traceback.walk_tb(exc.__traceback__)
    )
    # Strip source text to avoid leaking code context; keep
    filename:lineno:name
    safe_frames = [
        {"file": f.filename, "line": f.lineno, "func": f.name}
        for f in summary
    ]
    logger.error(
        "request failed",
        extra={
            "exc_type": type(exc).__name__,
            # sanitize message – drop or hash PII-bearing args
            "exc_msg": _sanitize(str(exc)),
            "frames": safe_frames,
            "cause": type(exc.__cause__).__name__ if exc.__cause__
else None,
            "notes": getattr(exc, "__notes__", []),
        },
        exc_info=False, # we already captured structured data; don't
emit raw traceback
    )

```

6. Chained exceptions — `__cause__`, `__context__`, `__suppress_context__`, and PEP 678 notes

6.1 Three links, one graph

PEP 3134 gave every `BaseException` three chaining fields (all `PyObject*`, nullable, managed in `Objects/exceptions.c`):

Attribute	Set when	Meaning	Rendered as
<code>__context__</code>	Any exception is raised while another is being handled (<code>curexc_value != NULL</code>)	Implicit context — "while handling X, Y happened"	During handling of the above exception, another exception occurred:
<code>__cause__</code>	<code>raise Y from X /</code> <code>raise Y from e</code> (explicit)	Explicit cause — "Y was caused by X"	The above exception was the direct cause of the following exception:
<code>__suppress_context__</code>	<code>raise Y from None</code> sets <code>__cause__ = None</code> , <code>__suppress_context__ = True</code>	Suppresses <code>__context__</code> display even though it is still stored	(no chain rendered)
<code>__notes__</code>	<code>exc.add_note(str)</code> (PEP 678, 3.11+)	Freeform annotations appended after the exception message	<code>note: ...</code> lines after <code>ExceptionType:</code> <code>msg</code>

Only `__cause__` is set by `raise ... from`. `__context__` is set automatically by `PyErr_SetObject` when `tstate->curexc_value` is non-NULL and `__suppress_context__` is False. The two can coexist — `__cause__` takes display priority.



Diagram 3 — Chained exception graph: `__context__` (implicit) vs. `__cause__` (explicit `raise ... from`) vs. suppressed (`raise ... from None`). `__context__` is set automatically when a new exception is raised inside an `except` block; `__cause__` is set only by explicit `from`; `from None` sets `__suppress_context__` to hide the implicit chain from rendering while preserving it on the object.

6.2 Live demo

```
import traceback

# 1. Implicit context
try:
    try:
        raise ValueError("bad input")
    except ValueError:
        raise RuntimeError("handler failed")
except RuntimeError as e:
    print(f"__context__={e.__context__!r} __cause__={e.__cause__!r} "
          f"__suppress_context__={e.__suppress_context__}")
    print("".join(traceback.format_exception(e)))

# 2. Explicit cause
try:
    try:
        raise ValueError("bad")
    except ValueError as e:
        raise RuntimeError("wrapped") from e
except RuntimeError as e:
    print(f"__context__={e.__context__!r} __cause__={e.__cause__!r}")
    print("".join(traceback.format_exception(e)))

# 3. Suppressed
try:
    try:
        raise ValueError("bad")
    except ValueError:
        raise RuntimeError("clean") from None
except RuntimeError as e:
    print(f"__context__={e.__context__!r} __cause__={e.__cause__!r} "
          f"__suppress_context__={e.__suppress_context__}")
    print("".join(traceback.format_exception(e)))
```

Output (abbreviated):

```
__context__=ValueError('bad input') __cause__=None
__suppress_context__=False
Traceback (most recent call last):
  File "...", line 4, in <module>
    raise ValueError("bad input")
ValueError: bad input
```

During handling of the above exception, another exception occurred:

```
Traceback (most recent call last):
  File "...", line 6, in <module>
    raise RuntimeError("handler failed")
RuntimeError: handler failed
```

```
__context__=ValueError('bad') __cause__=ValueError('bad')
Traceback (most recent call last):
  File "...", line 14, in <module>
    raise ValueError("bad")
ValueError: bad
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "...", line 16, in <module>
    raise RuntimeError("wrapped") from e
RuntimeError: wrapped
```

```
__context__=ValueError('bad') __cause__=None
__suppress_context__=True
Traceback (most recent call last):
  File "...", line 28, in <module>
    raise RuntimeError("clean") from None
RuntimeError: clean
```

6.3 PEP 678 notes — `add_note()`

PEP 678 (3.11+) adds `BaseException.add_note(str)` and `BaseException.__notes__` (a list of strings). Notes are rendered by `traceback.format_exception` *after* the exception line, one per line, without affecting `__cause__` / `__context__`:

```

try:
    raise ValueError("invalid payload")
except ValueError as e:
    e.add_note("request_id=abc-123")
    e.add_note("field 'email' failed validation")
    raise

# Rendered:
# ValueError: invalid payload
# note: request_id=abc-123
# note: field 'email' failed validation

```

Backend use: attach structured context (request ID, tenant, span ID) as notes *without* mutating `args[0]` and without creating a new exception that would reset the traceback. Notes survive `ExceptionGroup` splitting (each sub-exception retains its notes) and are visible in `format_exception` without custom formatters. Prefer notes over `f"msg (request_id={rid})"` string interpolation — the latter pollutes the exception message that may be matched by `except` or alerting rules.

Caveat: `__notes__` is a plain list on the exception instance — mutating it after `raise` is visible to handlers. Treat it as append-only.

7. Context managers — the `WITH` protocol

7.1 The synchronous protocol

A context manager is any object with `__enter__` and `__exit__` (both looked up on `type(obj)`, not the instance — `type(obj).__enter__` via `PyObject_LookupSpecial`). The protocol:

```

class FileManager:
    def __enter__(self):
        self.f = open(self.path, "r")
        return self.f          # value bound to `as` target

    def __exit__(self, exc_type, exc_val, exc_tb):
        # exc_type/val/tb are None if no exception, else the current
        triple
        self.f.close()
        return False           # False/None = propagate; True =
                                suppress

```

Bytecode lowering on 3.11+ uses two opcodes plus a table entry. `dis` for a `with` statement:

```
import dis

class MyCM:
    def __enter__(self): return self
    def __exit__(self, *a): return False

def with_custom():
    with MyCM() as c:
        print("inside")

dis.dis(with_custom)
```

```

51      0 RESUME      0

52      2 LOAD_GLOBAL 1 (NULL + MyCM)
      14 PRECALL      0
      18 CALL        0
      28 BEFORE_WITH  # <-- enter protocol
      30 STORE_FAST  0 (c)

53      32 LOAD_GLOBAL 3 (NULL + print)
      44 LOAD_CONST  1 ('inside')
      46 PRECALL      1
      50 CALL        1
      60 POP_TOP

52      62 LOAD_CONST  0 (None) # normal exit path
      64 LOAD_CONST  0 (None)
      66 LOAD_CONST  0 (None)
      68 PRECALL      2
      72 CALL        2 # call __exit__(No
ne, None, None)
      82 POP_TOP
      84 LOAD_CONST  0 (None)
      86 RETURN_VALUE
>>  88 PUSH_EXC_INFO
      90 WITH_EXCEPT_START # <-- exceptional exit path
      92 POP_JUMP_FORWARD_IF_TRUE 4 (to 102) # True → supp
ress
      94 RERAISE      2
>>  96 COPY        3
      98 POP_EXCEPT
     100 RERAISE      1
>>  102 POP_TOP     # suppressed → discard exc tr
iple
     104 POP_EXCEPT
     106 POP_TOP
     108 POP_TOP
     110 LOAD_CONST  0 (None)
     112 RETURN_VALUE
ExceptionTable:
  30 to 60 -> 88 [1] lasti
  88 to 94 -> 96 [3] lasti
  102 to 102 -> 96 [3] lasti

```

`BEFORE_WITH` (opcode 53) does two things in one dispatch: it calls `type(mgr).__enter__(mgr)` (via `PyObject_CallMethod`), pushes the result for `STORE_FAST`, and pushes the `__exit__` callable onto the stack for later use. The `__exit__` callable stays on the stack across the body — that is why the protected range `30..60` has `stack_depth=1`.

On the exceptional path (`PUSH_EXC_INFO` at 88): `WITH_EXCEPT_START` calls `__exit__(exc_type, exc_val, exc_tb)` with the current exception triple, pushes its boolean return, and `POP_JUMP_FORWARD_IF_TRUE` decides suppression. `RERAISE 2` with oparg 2 pops the `__exit__` callable and re-raises with the original traceback if not suppressed.

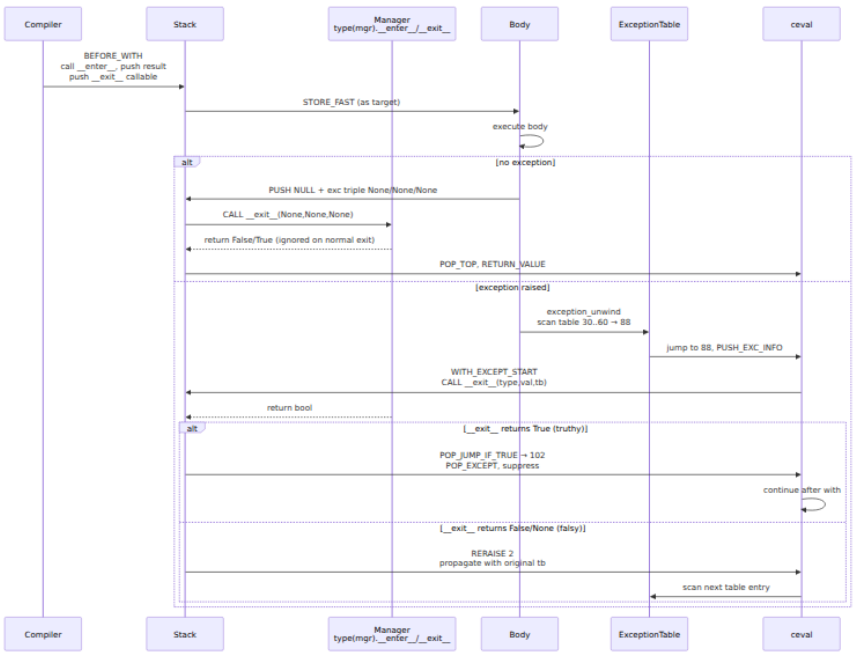


Diagram 4 — `WITH` protocol sequence. `BEFORE_WITH` calls `__enter__` and retains `__exit__` on the stack across the body. On normal exit the `finally`-like inline path calls `__exit__(None, None, None)`. On exceptional exit the exception table jumps to `PUSH_EXC_INFO` / `WITH_EXCEPT_START`, which calls `__exit__` with the exception triple; a truthy return suppresses, falsy re-raises via `RERAISE 2`.

7.2 Suppress vs. propagate — `__exit__` return value

The only thing that determines suppression is the truthiness of `__exit__`'s return value. `None` and `False` propagate; `True` (or any truthy object) suppresses:


```

import traceback

class SwallowingCM:
    """Swallows ValueError, propagates everything else."""
    def __enter__(self): return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None and isinstance(exc_type, ValueError):
            print(f"swallowed {exc_val!r}")
            return True # suppress
        return False # propagate (explicit)

class LoggingCM:
    def __enter__(self): return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            print(f"leaving with {exc_type.__name__}: {exc_val}")
            # return None implicitly → propagate
        return None

# Swallowed – no exception escapes
with SwallowingCM():
    raise ValueError("oops")
print("after swallowed block – continues")

# Propagated – exception escapes after __exit__ runs
try:
    with LoggingCM():
        raise TypeError("bad type")
except TypeError as e:
    print(f"caught {e!r}")
    print("".join(traceback.format_exception(e)))

```

Output:

```

swallowed ValueError('oops')
after swallowed block continues
leaving with TypeError: bad type
caught TypeError('bad type')
Traceback (most recent call last):
  File "...", line ..., in <module>
    raise TypeError("bad type")
TypeError: bad type

```

Backend pitfalls:

- **Never swallow blindly.** `return True` unconditionally hides bugs. If you write a swallowing manager (e.g., `contextlib.suppress`), be explicit about which types you swallow and log what you swallowed.

- **`__exit__` exceptions replace the original.** If `__exit__` itself raises, that new exception propagates with the original as `__context__` (implicit chain) — the original is not lost but it is no longer the primary exception. Keep `__exit__` simple and exception-safe.
- **`__exit__` is looked up on the type.** Monkey-patching `mgr.__exit__ = ...` on the instance does nothing — `BEFORE_WITH` does `type(mgr).__exit__`.

7.3 `contextlib.contextmanager` — generator-based managers

`@contextlib.contextmanager` lets you write a manager as a generator with a single `yield`:

```
import contextlib

@contextlib.contextmanager
def transaction(db):
    tx = db.begin()
    try:
        yield tx
        tx.commit()
    except BaseException:
        tx.rollback()
        raise
    finally:
        pass # tx object cleanup if needed

# Equivalent to a class with __enter__/___exit__, but the generator
# frame
# *is* the state machine. The decorator wraps it in
# _GeneratorContextManager.

with transaction(db) as tx:
    tx.execute("INSERT ...")
```

Implementation sketch (`Lib/contextlib.py`):
`_GeneratorContextManager.__enter__` calls `next(gen)` to run to `yield` and returns the yielded value. `__exit__` does:

- None triple → `next(gen)` to run the post-`yield` commit path; `StopIteration` is swallowed.
- Exception triple → `gen.throw(exc_type, exc_val, exc_tb)` to inject the exception at the `yield`; if the generator handles it (yields or returns), suppression is decided by whether `StopIteration` vs. new

exception emerges; if the generator does not handle it, the exception propagates.

Because the generator's `yield` is inside a `try/finally`, the same exception-table machinery from Section 4 protects it. The `contextmanager` wrapper adds one extra frame to the traceback — visible in `format_exception` output.

Other `contextlib` tools that build on the same protocol:

Tool	Purpose
<code>contextlib.suppress(*exc_types)</code>	<code>__exit__</code> returns <code>True</code> for listed types, <code>False</code> otherwise — the canonical swallowing manager
<code>contextlib.ExitStack</code> / <code>AsyncExitStack</code>	Dynamically manages a stack of <code>__enter__</code> / <code>__exit__</code> pairs; unwinds in LIFO order, correctly chaining suppressed vs. propagated exceptions
<code>contextlib.nullcontext</code>	No-op manager — useful as a conditional <code>with</code> branch
<code>contextlib.closing(obj)</code>	Calls <code>obj.close()</code> in <code>__exit__</code> — for objects with <code>close</code> but no <code>__exit__</code>
<code>contextlib.redirect_stdout</code> / <code>redirect_stderr</code>	Swaps <code>sys.stdout</code> in <code>__enter__</code> , restores in <code>__exit__</code>

7.4 Async context managers

The async protocol mirrors the sync one with `__aenter__` / `__aexit__` and opcodes `BEFORE_ASYNC_WITH` (52) and `GET_AWAITABLE`:

```
class AsyncCM:
    async def __aenter__(self): return self
    async def __aexit__(self, *a): return False

async def demo():
    async with AsyncCM() as c:
        await do_work()
```

`BEFORE_ASYNC_WITH` calls `__aenter__`, wraps the result in `GET_AWAITABLE`, and suspends via `SEND` until the awaitable completes. `__aexit__` is similarly awaited on exit, with the same truthiness rule for suppression. The exception table entry for an `async with` body is identical in shape — only the enter/exit opcodes differ.

8. ExceptionGroup and except* — PEP 654

8.1 Motivation

In sequential Python one `try` raises one exception. In structured concurrency many tasks fail at once:

```
async with asyncio.TaskGroup() as tg:
    tg.create_task(fetch("https://a.example.com"))
    tg.create_task(fetch("https://b.example.com"))
    tg.create_task(fetch("https://c.example.com"))
# If two fetches raise, what should `except` catch?
```

Before 3.11 the answer was "the first one" — other exceptions were lost. PEP 654 introduces `BaseExceptionGroup` / `ExceptionGroup` to carry *multiple* exceptions as a tree, and `except*` to match *subsets* without losing the rest.

8.2 Constructing and inspecting an ExceptionGroup

```
eg = ExceptionGroup("fetch failures", [
    ValueError("bad payload from a"),
    TypeError("unexpected type from b"),
    ValueError("bad payload from c"),
])

print(eg.message)           # "fetch failures"
print(eg.exceptions)        # tuple of 3
print(eg.subgroup(ValueError)) # ExceptionGroup with the two
                               ValueErrors
print(eg.split(ValueError))
# (ExceptionGroup('fetch failures', [ValueError(...),
ValueError(...)]),
# ExceptionGroup('fetch failures', [TypeError(...)])
# split returns (matching, nonmatching) — either may be None

# Notes survive
eg.exceptions[0].add_note("request_id=a-123")
```

`BaseExceptionGroup` is the base for groups that may contain `BaseException` leaves (including `KeyboardInterrupt`); `ExceptionGroup` is constrained to `Exception` leaves. Both are immutable — `exceptions` is a tuple, `message` is a string, `__notes__` works on the group itself.

Nesting is allowed: an `ExceptionGroup` can contain another `ExceptionGroup`, forming a tree. `split` / `subgroup` recurse into nested groups and prune empty branches.

8.3 `except*` — filtering, not catching

`except*` looks like `except` but its semantics are *filtering*. Each `except* E` clause is tried against the `ExceptionGroup`'s leaves; matching leaves are extracted into a new subgroup and bound to the `as` target, non-matching leaves continue to the next `except*`:

```
import traceback

try:
    raise ExceptionGroup("eg", [
        ValueError(1),
        TypeError(2),
        ValueError(3),
        RuntimeError(4),
    ])
except* ValueError as eg_val:
    print(f"caught ValueErrors: {eg_val.exceptions!r}")
    # eg_val is ExceptionGroup("eg", [ValueError(1), ValueError(3)])
    print("".join(traceback.format_exception(eg_val)))
except* TypeError as eg_type:
    print(f"caught TypeErrors: {eg_type.exceptions!r}")
except* RuntimeError as eg_rt:
    print(f"caught RuntimeErrors: {eg_rt.exceptions!r}")

print("all matched – no unhandled leaves, continues")
```

If every leaf is matched by some `except*`, execution continues after the `try`. If any leaf is unmatched, the remaining leaves are re-raised as a new `ExceptionGroup` (via `PREP_RERAISE_STAR + RERAISE 0`) — possibly after `finally` runs.

Mixing `except` and `except*` in the same `try` is a `SyntaxError` — the compiler enforces it. Use one style per `try`.

8.4 Bytecode lowering — `CHECK_EG_MATCH` and `PREP_RERAISE_STAR`

`dis` for the `except*` example above (from Section 3's capture, annotated):

```

3          4 PUSH_NULL / LOAD_NAME ExceptionGroup / BUILD_LIST 3 /
CALL
          86 RAISE_VARARGS          1
      >>  88 PUSH_EXC_INFO

4          90 COPY                    1
          92 BUILD_LIST              0          [# list for
matched subgroups
          94 SWAP                    2
          96 LOAD_NAME                ValueError
          98 CHECK_EG_MATCH          [# split group by V
alueError
      100 COPY                      1
      102 POP_JUMP_FORWARD_IF_NONE  23 (to 150) [# no match →
next handler
      104 STORE_NAME                eg
5          106 PUSH_NULL / LOAD_NAME print / LOAD_NAME eg / PRECALL /
CALL / POP_TOP
      128 LOAD_CONST None / STORE_NAME eg / DELETE_NAME eg
      134 JUMP_FORWARD              6 (to 148)
      >>  136 LOAD_CONST None / STORE_NAME eg / DELETE_NAME eg
      142 LIST_APPEND              3          [# no-match path: s
tash remainder
      144 POP_TOP
      146 JUMP_FORWARD              2 (to 152)
      >>  148 JUMP_FORWARD              1 (to 152)

4      >>  150 POP_TOP                [# ValueError handler had no m
atch

6      >>  152 LOAD_NAME                TypeError
          154 CHECK_EG_MATCH
          156 COPY                    1
          158 POP_JUMP_FORWARD_IF_NONE  23 (to 206)
          ... [# same shape for TypeError handler
      >>  208 LIST_APPEND              1
          210 PREP_RERAISE_STAR          [# prepare
residual group
          212 COPY                      1
          214 POP_JUMP_FORWARD_IF_NOT_NONE  4 (to 224)
          216 POP_TOP / POP_EXCEPT / RETURN_VALUE [# all matched → s
uppress
      >>  224 SWAP                    2 / POP_EXCEPT / RERAISE 0
[# residual → re-raise
      >>  230 COPY                    3 / POP_EXCEPT / RERAISE 1
[# outer handler
ExceptionTable:
4 to 86 -> 88 [0]
88 to 104 -> 230 [1] lasti
106 to 126 -> 136 [4] lasti

```

```
128 to 160 -> 230 [1] lasti
162 to 182 -> 192 [4] lasti
184 to 216 -> 230 [1] lasti
```

Key differences from `except` :

- `CHECK_EG_MATCH` (opcode 37) does `BaseExceptionGroup.split(exc_type)` — it returns `(matching_subgroup_or_None, nonmatching_subgroup_or_None)` , leaves the non-matching remainder on the stack for the next handler, and pushes the matching subgroup for the current handler to test for `None` .
- `PREP_RERAISE_STAR` (opcode 88) collects the `LIST_APPEND` 'd remainders and either leaves `None` (all matched) or builds the residual `ExceptionGroup` to re-raise.
- Each `except*` body has its own exception-table entry with `stack_depth=4` (the extra stack items for the group-splitting protocol) and `lasti` .

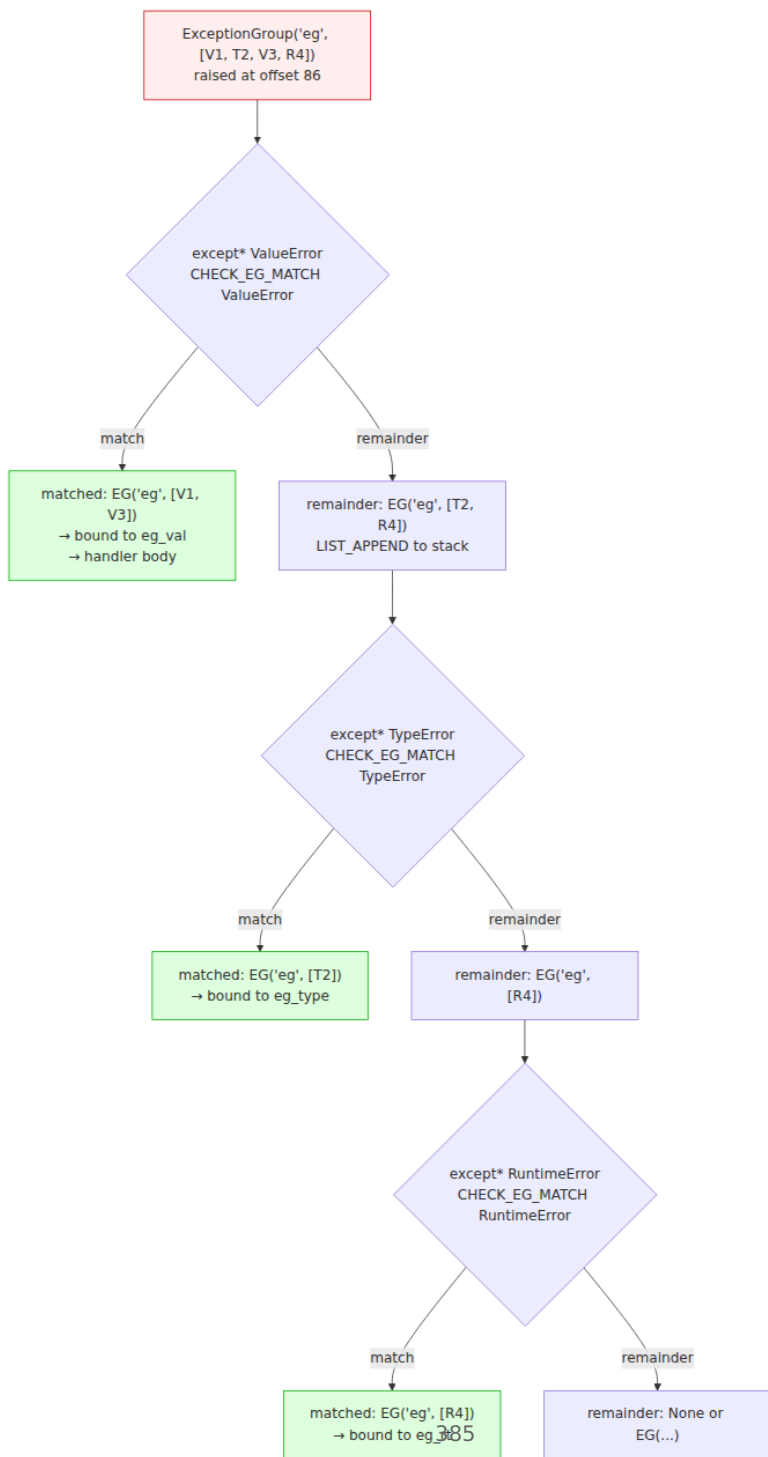


Diagram 5 — `ExceptionGroup` split tree (PEP 654). Each `except*` clause runs `CHECK_EG_MATCH`, which splits the incoming group into a matching subgroup (bound to `as` and handled) and a non-matching remainder pushed for the next handler. `PREP_RERAISE_STAR` reassembles any unmatched remainder — if empty, execution continues; if non-empty, the residual `ExceptionGroup` is re-raised.

8.5 `split` / `subgroup` API and nesting

```
eg = ExceptionGroup("outer", [
    ValueError(1),
    ExceptionGroup("inner", [TypeError(2), ValueError(3)]),
    RuntimeError(4),
])

# subgroup – keep only matching leaves, prune empty branches
print(eg.subgroup(ValueError))
# ExceptionGroup('outer', [ValueError(1), ExceptionGroup('inner',
# [ValueError(3)])])
# Note: TypeError(2) and RuntimeError(4) pruned; inner group retained
# because it still has a match

# split – (matching, nonmatching) partition
match, rest = eg.split(ValueError)
print(match) # EG('outer', [ValueError(1), EG('inner',
# [ValueError(3)])])
print(rest) # EG('outer', [EG('inner', [TypeError(2)]),
RuntimeError(4)])

# except* does split iteratively – equivalent to:
# rem = eg
# for exc_type, handler in handlers:
#     m, rem = rem.split(exc_type) if rem else (None, None)
#     if m: handler(m)
# if rem: raise rem
```

`BaseExceptionGroup.split` is the primitive; `subgroup` is `split(...)[0]` with pruning. Both preserve `__notes__` and `__traceback__` on the resulting subgroups.

8.6 Interaction with `finally` and `else`

- `finally` runs after `except*` filtering, before the residual re-raise. If `finally` raises, its exception becomes the new `__context__` for the residual group.

- `else` runs only if the `try` body did not raise *any* exception (no group) — same as `except` .
- Bare `raise` inside `except*` re-raises the *subgroup* bound to `as` , not the original group. To re-raise the original, keep a reference before filtering.
- `return` / `break` / `continue` inside `except*` suppresses the residual group for that handler's leaves but non-matching leaves still propagate — be explicit about whether you intend to handle all leaves.

9. Suppress vs. propagate — decision table

Every exit from an exception handler or context manager answers one question: does the exception continue unwinding?

Site	Propagate (continue unwind)	Suppress (swallow, continue normally)
<code>except E: body</code> falls through	<code>RERAISE 0</code> at handler end if no match	<code>POP_EXCEPT</code> + <code>JUMP</code> over <code>RERAISE</code> if matched
<code>except E as e: body</code>	<code>return</code> / <code>raise</code> / implicit <code>RERAISE</code>	<code>POP_EXCEPT</code> then fall through (clears <code>e</code>)
<code>__exit__</code> returns <code>False</code> / <code>None</code> / <code>falsy</code>	<code>RERAISE 2</code>	<code>__exit__</code> returns <code>True</code> / <code>truthy</code> → <code>POP_EXCEPT</code> , jump over <code>RERAISE</code>
<code>except* E: handler</code>	<code>PREP RERAISE STAR</code> builds residual → <code>RERAISE 0</code>	All leaves matched → <code>POP_EXCEPT</code> , no residual
<code>finally</code>	Ends with <code>RERAISE 0</code> (re-raise original)	<code>finally</code> does not suppress unless it does <code>return</code> / <code>raise new</code> / <code>break</code>

Site	Propagate (continue unwind)	Suppress (swallow, continue normally)
<code>contextlib.suppress(E)</code>	<code>__exit__</code> returns <code>False</code> for non-matching types	<code>__exit__</code> returns <code>True</code> for matching types

Three subtleties:

1. **return in finally suppresses.** A `return` inside `finally` discards any pending exception — the function returns normally. This is usually a bug. Linters flag it; treat it as an error in code review.

```
python def bad_finally(): try: raise ValueError("oops") finally:
return 42 # BUG: ValueError silently discarded, returns 42
```

1. **break / continue in finally also suppress.** Same mechanism — the control-flow jump leaves the handler without re-raising.
2. **except E: pass suppresses E but not others.** Unmatched exceptions hit `RERAISE 0`. This is why broad `except Exception: pass` is dangerous — it suppresses *every* `Exception` leaf, including ones you did not intend to handle.

10. Putting it together — end-to-end examples

10.1 Traceback formatting with notes and cause

```
import traceback

def parse(data: dict):
    if "email" not in data:
        raise ValueError("missing field 'email'")

def handle_request(data: dict, request_id: str):
    try:
        parse(data)
    except ValueError as e:
        e.add_note(f"request_id={request_id}")
        e.add_note(f"payload_keys={list(data.keys())}")
        raise RuntimeError("request validation failed") from e

try:
    handle_request({}, request_id="req-abc-123")
except RuntimeError as e:
    # Full rendering – includes cause chain and notes
    print("".join(traceback.format_exception(e)))

    # Structured – StackSummary without PII
    tb_summary = traceback.StackSummary.extract(traceback.walk_tb(e.__t
raceback__))
    cause_summary =
traceback.StackSummary.extract(traceback.walk_tb(e.__cause__.__tracebac
k__)) if e.__cause__ else None
    print(f"top frame: {tb_summary[-1].filename}:
{tb_summary[-1].lineno} in {tb_summary[-1].name}")
    print(f"notes: {getattr(e, '__notes__', [])}")
```

Output:

```
Traceback (most recent call last):
  File "...", line ..., in handle_request
    parse(data)
  File "...", line ..., in parse
    raise ValueError("missing field 'email'")
ValueError: missing field 'email'
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "...", line ..., in handle_request
    raise RuntimeError("request validation failed") from e
RuntimeError: request validation failed
note: request_id=req-abc-123
note: payload_keys=[]
```

10.2 Custom `__exit__` that swallows selectively

```
import contextlib

class RetryableTransaction:
    def __init__(self, db, *, retryable=()):
        self.db = db
        self.retryable = retryable
        self.tx = None

    def __enter__(self):
        self.tx = self.db.begin()
        return self.tx

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is None:
            self.tx.commit()
            return False
        if isinstance(exc_type, self.retryable):
            self.tx.rollback()
            print(f"retryable {exc_type.__name__}: {exc_val} –
swallowing for retry")
            return True # suppress – caller will retry
        self.tx.rollback()
        return False # propagate

# Usage – ValueError is retryable, TypeError is not
db = FakeDB()
try:
    with RetryableTransaction(db, retryable=(ValueError,)) as tx:
        tx.execute("INSERT ...")
        raise ValueError("transient conflict")
    print("swallowed – will retry")
except TypeError:
    print("not swallowed")
```

10.3 ExceptionGroup with TaskGroup-style fan-out

```
import traceback

def fetch_one(name: str):
    if name == "a":
        raise ValueError(f"bad payload from {name}")
    if name == "b":
        raise TypeError(f"unexpected type from {name}")
    if name == "c":
        raise RuntimeError(f"timeout from {name}")
    return f"ok:{name}"

# Simulate TaskGroup fan-out without asyncio – direct ExceptionGroup
errors = []
for name in ("a", "b", "c"):
    try:
        fetch_one(name)
    except Exception as e:
        e.add_note(f"fetch:{name}")
        errors.append(e)

if errors:
    eg = ExceptionGroup("fetch failures", errors)
    try:
        raise eg
    except* ValueError as eg_val:
        print(f"handling ValueErrors: {[str(x) for x in eg_val.exceptions]}")
        for exc in eg_val.exceptions:
            print(f"  notes: {getattr(exc, '__notes__', [])}")
    except* TypeError as eg_type:
        print(f"handling TypeErrors: {[str(x) for x in eg_type.exceptions]}")
    except* RuntimeError as eg_rt:
        print(f"handling RuntimeErrors: {[str(x) for x in eg_rt.exceptions]}")
    print("all fetch errors handled")

# Partial handling – unmatched leaves propagate
try:
    try:
        raise ExceptionGroup("mixed", [ValueError(1), RuntimeError(2)])
    except* ValueError as eg:
        print(f"handled ValueErrors: {eg.exceptions}")
        # RuntimeError(2) is unmatched – will be re-raised
except BaseExceptionGroup as residual:
    print(f"residual group propagated: {residual.exceptions!r}")
    print("".join(traceback.format_exception(residual)))
```


11. Backend lens — structured error handling, resource managers, and safe traceback logging

11.1 Structured exception taxonomies for services

Define a small, stable exception hierarchy per service boundary. Callers should be able to `except` on category, not on string matching:

```
class AppError(Exception):
    """Base for all service errors – safe to expose category, not
    message."""
    category: str = "internal"
    retryable: bool = False

class ValidationError(AppError):
    category = "validation"
    retryable = False

class DependencyError(AppError):
    category = "dependency"
    retryable = True

class NotFoundError(AppError):
    category = "not_found"
    retryable = False

def handle_request(data: dict):
    try:
        validate(data)
        call_downstream(data)
    except AppError as e:

# Structured – category drives HTTP status, retry drives caller
behavior
        status = {"validation": 400, "not_found": 404, "dependency": 50
2, "internal": 500}[e.category]
        raise HTTPError(status, e.category) from e
    except BaseException as e:
        # Catch-all for non-AppError – never expose raw message
        e.add_note(f"category={type(e).__name__}")
        raise AppError("internal error") from e
```

Rules:

- Inherit from `Exception`, not `BaseException` — so `except Exception` in framework code does not swallow `KeyboardInterrupt`.

- Attach `request_id / span_id` as `__notes__`, not by interpolating into `args[0]` — keeps messages stable for alerting and avoids PII in exception text.
- Use `raise ... from e` for dependency wrappers so the cause chain is preserved for debugging but the outer `category` is what callers match on.
- For fan-out (parallel downstream calls), raise `ExceptionGroup` and let callers `except*` by category.

11.2 Context managers for resources — DB connections, tracing spans, locks

Every resource with acquire/release semantics should be a context manager. The `WITH` protocol guarantees `__exit__` runs even when the body raises, `return s`, or `break s` — because the exception table protects the body range.

```

import contextlib, time

# DB connection pool – the canonical backend with-block
@contextlib.contextmanager
def pooled_connection(pool):
    conn = pool.acquire()
    try:
        yield conn
    finally:
        pool.release(conn) # always runs, even on exception

# Tracing span – mirrors OpenTelemetry's Span context manager
class TraceSpan:
    def __init__(self, name: str, tracer):
        self.name = name
        self.tracer = tracer
        self.span = None

    def __enter__(self):
        self.span = self.tracer.start_span(self.name)
        return self.span

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            self.span.record_exception(exc_val)
            self.span.set_status("ERROR", str(exc_val))
            # Attach notes without mutating message
            for note in getattr(exc_val, "__notes__", []):
                self.span.set_attribute(f"exception.note.{note}", True)
            self.span.end()
        return False # never suppress – let caller decide

# Distributed lock
class DistributedLock:
    def __init__(self, key: str, ttl: int = 30):
        self.key, self.ttl = key, ttl
    def __enter__(self):
        acquired = redis.set(self.key, "1", nx=True, ex=self.ttl)
        if not acquired:
            raise DependencyError(f"lock {self.key} not acquired")
        return self
    def __exit__(self, *a):
        redis.delete(self.key)
        return False

# Composing – ExitStack for dynamic sets of resources
def handle_batch(ids: list[str]):
    with contextlib.ExitStack() as stack:
        conns = [stack.enter_context(pooled_connection(pool)) for _ in
ids]

```

```

        span = stack.enter_context(TraceSpan("handle_batch", tracer))
        lock = stack.enter_context(DistributedLock(f"batch:{','.join(ids)}"))
    }"))
    # All __exit__ run in LIFO order on any exit path – same as
    # nested with-blocks
    return process(conns, span)

```

Why `ExitStack` matters: it correctly handles the case where `__enter__` of the second resource raises — it unwinds the first resource's `__exit__` before propagating. Hand-rolled `try/finally` nesting gets this wrong when the number of resources is dynamic.

11.3 Logging tracebacks without leaking PII

Raw `traceback.format_exc()` is convenient and dangerous. It includes source lines (which may contain secrets in comments or string literals), exception messages (which may contain user data), and `__notes__` (which you control). Harden it:

```

import traceback, logging, re

logger = logging.getLogger("api")
_EMAIL_RE = re.compile(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}")

def sanitize_message(msg: str) -> str:
    msg = _EMAIL_RE.sub("[email]", msg)
    # Add more: tokens, SQL fragments, etc.
    if len(msg) > 500:
        msg = msg[:500] + "...[truncated]"
    return msg

def log_exc_structured(exc: BaseException, *, level=logging.ERROR):
    """Log exception without raw traceback text – structured, PII-
    aware."""
    # 1. Sanitize the message
    safe_msg = sanitize_message(str(exc))

    # 2. Structured frames – no source lines, no locals
    def frames_of(tb):
        if tb is None:
            return []
        return [
            {"file": f.filename, "line": f.lineno, "func": f.name}
            for f in traceback.StackSummary.extract(traceback.walk_tb(t
b)))
        ]

    # 3. Walk cause/context chain explicitly – don't rely on
    format_exception's text
    chain = []
    cur = exc
    while cur is not None:
        chain.append({
            "type": type(cur).__name__,
            "msg": sanitize_message(str(cur)),
            "notes": [sanitize_message(n) for n in getattr(cur, "__note
s__", [])],
            "frames": frames_of(cur.__traceback__),
        })
    # Prefer __cause__ over __context__ if both present (explicit
    wins)
    cur = cur.__cause__ if cur.__cause__ is not None else (
        cur.__context__ if not cur.__suppress_context__ else None
    )

    logger.log(level, "request failed",
               extra={"exc_chain": chain, "exc_type": type(exc).__name__},

```

```
exc_info=False) # never exc_info=True with raw
traceback in prod
```

Ship `exc_chain` as structured JSON to your log aggregator — it is queryable (`exc_chain[0].type = "ValidationError"`), redacted, and does not require parsing `Traceback (most recent call last): text`. Keep `traceback.format_exception` for local development and on-call shell debugging only.

Key takeaways

- `BaseException` vs. `Exception` is a correctness boundary — `except Exception` deliberately lets `KeyboardInterrupt` / `SystemExit` / `GeneratorExit` propagate. Never `except BaseException` in service code.
- The per-thread exception indicator (`tstate->curexc_*`, `PyErr_SetString` / `PyErr_Fetch` / `PyErr_NormalizeException`) is the single source of truth; `raise` normalizes type → instance and chains `__context__` automatically.
- Since 3.11 there is no block stack — `co_exceptiontable` declaratively maps `start..end` → `target`, `depth`, `lasti` and is scanned only on the exceptional path, making non-exceptional `try` bodies zero-cost.
- `dis` for `try/except/finally` lowers to `PUSH_EXC_INFO` → `CHECK_EXC_MATCH` → `POP_JUMP_IF_FALSE` → `POP_EXCEPT` → `JUMP` over `RERAISE 0`, with `finally` bodies inlined at every exit and protected by additional table entries.
- `PyTracebackObject` is a linked list (`tb_next` toward the caller) holding `tb_frame` / `tb_lineno` / `tb_lasti`; `traceback.StackSummary` is the structured, PII-controllable layer above it, and `format_exception` is the presentation layer.
- `__context__` (implicit, set whenever a new exception is raised while handling another), `__cause__` (explicit `raise ... from`), and `__suppress_context__` (`raise ... from None`) form a directed graph rendered with distinct banners; PEP 678 `add_note()` appends structured context without mutating the message.
- The `WITH` protocol is `BEFORE_WITH` (calls `__enter__`, retains `__exit__` on the stack) → body (protected by exception table) → normal `CALL __exit__(None, None, None)` or exceptional

`WITH_EXCEPT_START` → `CALL __exit__(type, val, tb)` → truthy return suppresses, falsy re-raises via `RERAISE 2`. `__exit__` is looked up on `type(mgr)`.

- `contextlib.contextmanager` wraps a generator's `yield` as the `__enter__ / __exit__` state machine; `ExitStack` correctly unwinds dynamic resource sets in LIFO order.
 - `ExceptionGroup` / `BaseExceptionGroup` (PEP 654) carry multiple exceptions as a tree; `except*` filters via `CHECK_EG_MATCH / PREP_RERAISE_STAR`, binding each matching subgroup to `as` and re-raising the residual group if any leaf is unmatched. Never mix `except` and `except*` in one `try`.
 - Suppression is always an explicit decision — `__exit__` truthiness, `except` match, or `except*` full coverage. `return / break / continue` inside `finally` silently discards the pending exception and should be treated as a bug.
 - For backends: define a small `AppError` taxonomy with `category / retryable`, attach `request_id` as `__notes__`, use context managers for every acquire/release resource, and log tracebacks as sanitized `StackSummary` JSON — not raw `format_exc()` text.
-

Further reading

- **PEP 654 — Exception Groups and `except*`** — <https://peps.python.org/pep-0654/> — Motivation, `BaseExceptionGroup` / `ExceptionGroup` design, `except*` filtering semantics, `split / subgroup`, interaction with `finally` and `raise`, and the `TaskGroup` use case. *Pinned*.
- **PEP 3134 — Exception Chaining and Embedded Tracebacks** — <https://peps.python.org/pep-03134/> — `__cause__`, `__context__`, `__suppress_context__`, `raise ... from`, and the rendering rules for implicit vs. explicit chains. *Pinned*.
- **PEP 343 — The `with` Statement** — <https://peps.python.org/pep-0343/> — Original context-manager protocol, `__enter__ / __exit__` contract, `contextlib.contextmanager`, and rationale for deterministic cleanup. *Pinned*.
- **Python Documentation — Built-in Exceptions** — <https://docs.python.org/3/library/exceptions.html> — Authoritative

`BaseException` hierarchy, attributes (`args`, `__cause__`, `__context__`, `__notes__`, `__traceback__`), and `BaseExceptionGroup` / `ExceptionGroup` API. *Pinned*.

- **CPython Source — `Python/ceval.c:exception_unwind` and `Python/codeobject.c:exception_table`** — <https://github.com/python/cpython/blob/main/Python/ceval.c> and <https://github.com/python/cpython/blob/main/Objects/codeobject.c> — `Table varint encoding`, `get_exception_handler scan`, `handle_eval_breaker unwind loop`, and `lasti handling`. Search for `co_exceptiontable`, `exception_unwind`, `PyTraceBack_Here`. *Pinned*.
- **PEP 678 — Enriching Exceptions with Notes** — <https://peps.python.org/pep-0678/> — `BaseException.add_note()` / `__notes__`, rendering in `traceback.format_exception`, and use cases for attaching structured context without mutating `args`.
- **Python Documentation — `traceback` Module** — <https://docs.python.org/3/library/traceback.html> — `PyTracebackObject` vs. `StackSummary` / `FrameSummary` / `TracebackException`, `format_exception` vs. `extract_tb` vs. `walk_tb`, and chaining controls.
- **Python Documentation — `contextlib` Module** — <https://docs.python.org/3/library/contextlib.html> — `contextmanager`, `asynccontextmanager`, `ExitStack` / `AsyncExitStack`, `suppress`, `nullcontext`, `closing`, and `redirect_stdout`.
- **CPython Source — `Objects/exceptions.c` and `Python/errors.c`** — <https://github.com/python/cpython/blob/main/Objects/exceptions.c> and <https://github.com/python/cpython/blob/main/Python/errors.c> — `BaseException` type definitions, `PyErr_SetString` / `PyErr_Fetch` / `PyErr_NormalizeException` / `PyErr_GivenExceptionMatches`, and the per-thread `curexc_*` / `exc_info` stack.
- **PEP 3110 / Python 3.11 Release Notes — Exception Table and Zero-Cost `try`** — <https://docs.python.org/3/whatsnew/3.11.html> — Summary of the block-stack removal, `co_exceptiontable` introduction, `PUSH_EXC_INFO` / `POP_EXCEPT` / `RERAISE` /

`WITH_EXCEPT_START` / `CHECK_EG_MATCH` opcodes, and `BEFORE_WITH` / `BEFORE_ASYNC_WITH` changes.

Chapter 9 — The Import System: `importlib`, Finders, Loaders, and Namespace Packages

What this chapter covers. Every `import` statement in your backend — from `import os` at module scope to a dynamic `importlib.import_module("myapp.handlers")` inside a request path — traverses the same machinery: `importlib._bootstrap`, `importlib._bootstrap_external`, `sys.meta_path`, `sys.path_hooks`, `sys.path_importer_cache`, finders, loaders, `ModuleSpec`, `sys.modules`, and the per-module import lock. This chapter dissects that machinery end-to-end: how CPython finds and loads code from the filesystem, zip files, namespace packages, editable installs, and custom hooks; how bytecode caching and invalidation (PEP 552) interacts with import; how relative imports and circular imports actually resolve; and how to instrument, trace, and extend the system without breaking it. You will build a custom finder/loader, trace import timing, and diagnose the backend-relevant failure modes that only surface at scale.

Learning goals — after this chapter you should be able to:

- Trace an `import foo` from `IMPORT_NAME` / `__import__` through `importlib._bootstrap._find_and_load` to `find_spec` and `exec_module`, identifying which file (`_bootstrap.py` vs `_bootstrap_external.py`) owns each step.
- Explain the three dispatch layers — `sys.meta_path`, `sys.path` + `sys.path_hooks` + `sys.path_importer_cache`, and `sys.path_hooks` fallback — and predict the search order for any `sys.path` entry.
- Implement a `MetaPathFinder` / `Loader` pair with `find_spec` / `create_module` / `exec_module`, and contrast it with `PathEntryFinder` and the legacy `load_module` protocol.
- Describe every field of `ModuleSpec` (PEP 451) and how it flows into module object creation (`__spec__`, `__loader__`, `__package__`, `__path__`, `__cached__`).

- Explain `sys.modules` caching, the import lock (`_imp.acquire_lock / _imp.release_lock`), and what happens during concurrent or circular imports.
- Contrast regular packages (`__init__.py`) with namespace packages (PEP 420, no `__init__.py`), including `__path__` as `_NamespacePath` and multi-directory merging.
- Explain editable installs under PEP 660, how `.pth / _editable_impl / importlib.machinery` mappings replace the old `setup.py develop / _editable` hacks.
- Describe `zipimport / ZipImporter`, import hooks, and freezing (`PyImport_ImportModule`, `PyImport_AppendInittab`, `freeze_modules.py`).
- Contrast PEP 552 `pyc` invalidation modes (timestamp vs hash, `check_source`) and explain `SOURCE_DATE_EPOCH`, `PYTHONDONTWRITEBYTECODE`, and hash-based `__pycache__`.
- Instrument imports with `python -X importtime`, `importlib.util.find_spec`, and `sys.modules` inspection, and apply backend-lens optimizations (lazy imports, import-time side-effect discipline, reproducible container images).

Prerequisites. Chapter 1 (source → bytecode → `__pycache__`, frozen `__bootstrap`) and Chapter 3 (bytecode/ `ceval.c / PyCodeObject`) are direct prerequisites. Chapter 5 (GIL and the import lock as a global critical section) and Volume 13, Chapter 6 (Python runtime posture for services) provide operational context.

1. Why the import system is a backend concern

Import is the most executed "framework" in your fleet. A typical FastAPI or Django worker imports 200–600 modules at startup; an ML inference service can exceed 1,000 once `numpy`, `torch`, `transformers`, and their transitive dependencies resolve. In serverless and autoscaled environments, that import graph *is* your cold-start latency.

Backend consequences that trace directly to import internals:

- **Cold start.** `python -X importtime` on a real service often shows 40–70% of process startup spent inside import. A single heavy

transitive import (`import pandas` pulling `pytz` , `dateutil` , `numpy`) can dominate p99 cold start.

- **Import-time side effects.** Code at module top-level runs exactly once, under the import lock. A `requests.Session()` created at import time, a database connection opened at import time, or a `logging.basicConfig()` call that mutates global state all execute before your application has configured itself.
- **Container reproducibility.** Whether `__pycache__` is written, shipped, or suppressed (`PYTHONDONTWRITEBYTECODE` , `SOURCE_DATE_EPOCH` , PEP 552 hash mode) determines whether two images built from the same commit are byte-identical — relevant to SLSA provenance, layer caching, and deterministic deploys.
- **Namespace and editable installs.** Monorepos, shared `company.*` namespaces, and `pip install -e .` during development all rely on PEP 420 and PEP 660. Misunderstanding `__path__` merging is a common source of "works on my machine, `ModuleNotFoundError` in CI."
- **Supply-chain surface.** `sys.meta_path` and `sys.path_hooks` are arbitrary code execution hooks. Anything that can insert a finder can intercept any future import — used legitimately by `zipimport` , `importlib.machinery` , `pytest` , and `coverage` , and illegitimately by dependency-confusion payloads.

This chapter treats import not as syntax but as a runtime service with its own discovery, caching, locking, and extensibility protocol.

2. The machinery — two frozen files that bootstrap themselves

The import system is unusual: it is written mostly in Python, but it must exist before any Python file can be imported. CPython solves this by freezing two modules into the interpreter binary itself.

File	Role	Frozen via
<code>Lib/importlib/_bootstrap.py</code>	Core protocol: <code>ModuleSpec</code> , <code>MetaPathFinder</code> , <code>Loader</code> , <code>_find_and_load</code> , <code>_gcd_import</code> , <code>sys.modules</code> interaction, import lock	<code>Tools/build/freeze_modules.py</code> → <code>Python/importlib.h</code> → <code>Python/import.c</code>
<code>Lib/importlib/_bootstrap_external.py</code>	Filesystem reality: <code>FileFinder</code> , <code>SourceFileLoader</code> , <code>SourcelessFileLoader</code> , <code>ExtensionFileLoader</code> , <code>SourceLoader</code> base, path-stat cache, <code>pyc</code> header validation (PEP 552)	Same pipeline

At interpreter startup (`Python/pylifecycle.c:Py_InitializeFromConfig` → `import_init` in `Python/import.c`), CPython:

1. Creates `sys.modules` (a plain `dict`) and inserts `sys` and `builtins` (already present as C modules).
2. Installs three meta-path finders into `sys.meta_path` in order:
3. `BuiltinImporter` — handles built-in modules (`sys` , `time` , `_imp` , `_io` , ...).
4. `FrozenImporter` — handles frozen modules (including `_bootstrap` themselves and any modules frozen via `Tools/build/freeze_modules.py` / `--freeze-importlib`).
5. `PathFinder` — the `sys.path` -based finder that delegates to `sys.path_hooks` .
6. Populates `sys.path_hooks` with two default hooks:
`zipimport.zipimporter` and `FileFinder.path_hook` (which is `_bootstrap_external.FileFinder.path_hook` — a factory that returns a `FileFinder` for a given `sys.path` entry).
7. Leaves `sys.path_importer_cache` empty (populated lazily).

You can observe the frozen bootstrap directly:

```

import sys, importlib.machinery

print(sys.meta_path)
# [<distutils_hack.DistutilsMetaFinder ...>,
#  <class '_frozen_importlib.BuiltinImporter'>,
#  <class '_frozen_importlib.FrozenImporter'>,
#  <class '_frozen_importlib_external.PathFinder'>]

print(sys.path_hooks)
# [<class 'zipimport.zipimporter'>,
#  <function FileFinder.path_hook.<locals>.path_hook_for_FileFinder
  at ...>]

# The frozen modules carry a special __spec__ origin marker:
import _frozen_importlib, _frozen_importlib_external
print(_frozen_importlib.__spec__.loader) # <class
'_frozen_importlib.FrozenImporter'>
print(_frozen_importlib_external.__spec__.origin) # frozen

# Where the frozen code lives in the binary:
import importlib
print(importlib._bootstrap.__file__) # <frozen
importlib._bootstrap>
print(importlib._bootstrap_external.__file__) # <frozen
importlib._bootstrap_external>

```

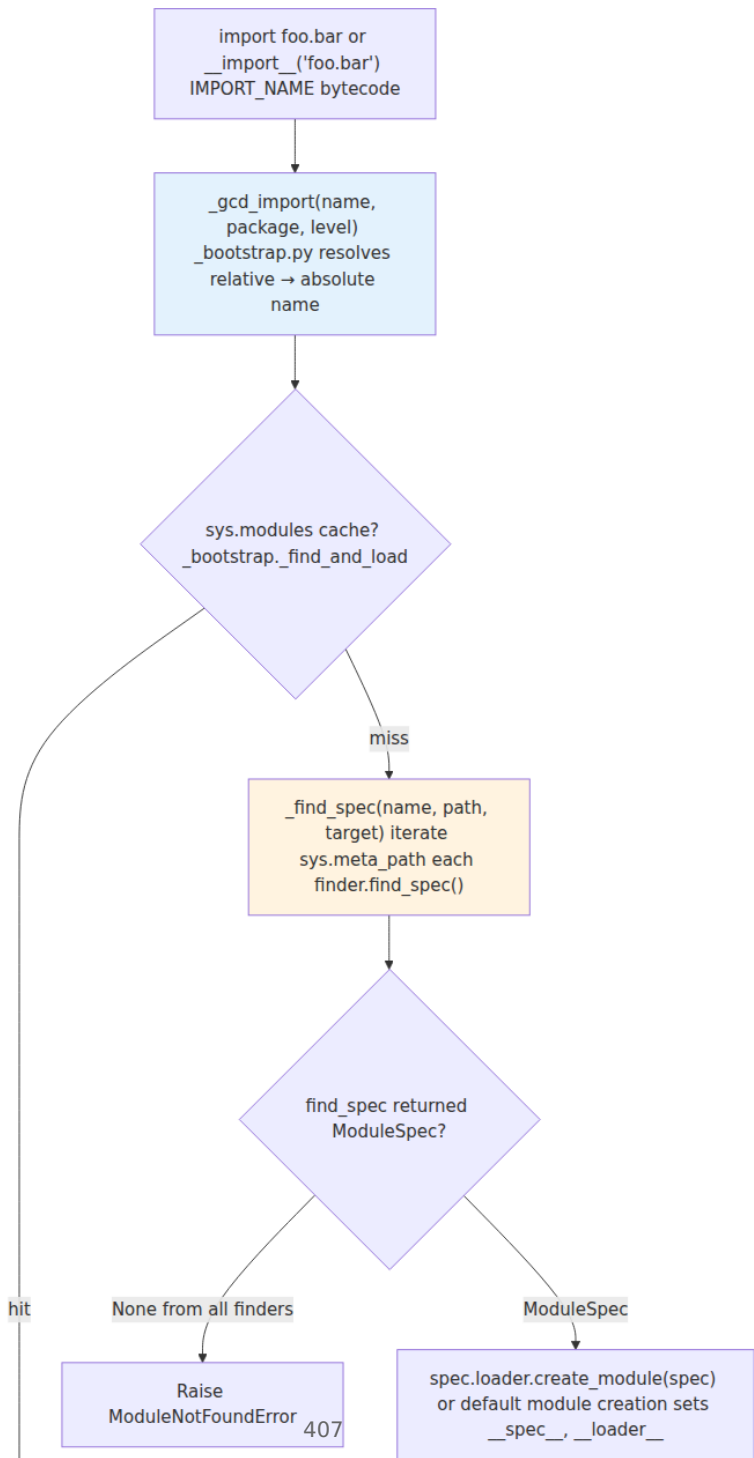
The split between `_bootstrap` and `_bootstrap_external` is not organizational trivia — it is a layering boundary:

- `_bootstrap.py` knows nothing about the filesystem. It defines the protocol (`find_spec` , `create_module` , `exec_module` , `ModuleSpec`) and orchestrates the dance. It could run on a system with no files at all (only built-in and frozen modules).
- `_bootstrap_external.py` knows everything about the filesystem: `stat` calls, `__pycache__` paths, `SOURCE_SUFFIXES` , `BYTECODE_SUFFIXES` , `EXTENSION_SUFFIXES` , `pyc` header parsing, and `ZipImporter` delegation via `sys.path_hooks` .

Chapter 1 described `Tools/build/freeze_modules.py` generating Python/`importlib.h` and `Python/importlib_external.h` (marshalled bytecode baked into `import.c`). That is how `_bootstrap` can import itself. The `importlib` package you see at `Lib/importlib/_init_.py` is a thin wrapper that re-exports the frozen machinery plus the pure-Python utilities (`importlib.util` , `importlib.machinery` , `importlib.resources`).

3. The import call flow — `import foo` to `exec_module`

At the language level, `import foo.bar` is syntactic sugar for a call to `__import__("foo.bar", globals(), locals(), [], 0)`, which in CPython 3.11+ compiles to `IMPORT_NAME` (or `IMPORT_FROM` for `from foo import bar`). Both paths converge on `importlib._bootstrap._gcd_import` (the "get child data" import), which is aliased as `builtins.__import__`.



Step-by-step in `_bootstrap.py` terms:

```
# Simplified skeleton of _bootstrap._find_and_load (read the real thing
# at Lib/importlib/_bootstrap.py – search for def _find_and_load):

def _find_and_load(name, _gcd_import):
    # 1. sys.modules fast path – also handles partially-initialized
    #    modules during circular imports (see §8).
    if name in sys.modules:
        return sys.modules[name]

    # 2. Acquire per-module import lock (see §8).
    #    Real code uses _imp.acquire_lock() / _ModuleLock.
    with _ModuleLock(name):
        # Re-check sys.modules after acquiring lock (another thread
        # won).
        if name in sys.modules:
            return sys.modules[name]

    # 3. Find a spec.
    spec = _find_spec(name, None, None) # walks sys.meta_path

    if spec is None:
        raise ModuleNotFoundError(f"No module named {name!r}",
                                  name=name)

    # 4. Create + exec.
    module = _load_unlocked(spec) # create_module → exec_module →
    sys.modules
    return module
```

`_find_spec` itself is a loop over `sys.meta_path`:

```
# Skeleton of _find_spec:
def _find_spec(name, path, target=None):
    # path is parent's __path__ for submodules, else None (top-level →
    # sys.path)
    for finder in sys.meta_path:
        # Each finder may be a class (BuiltinImporter, FrozenImporter – used
        # as classmethods) or an instance (custom finders).
        spec = finder.find_spec(name, path, target)
        if spec is not None:
            return spec
    return None
```

Critical detail: **parent packages are imported first**. Importing `a.b.c` triggers `_gcd_import` for `a`, then `a.b`, then `a.b.c` — each as a separate

`__find_and_load` cycle. If `a` is a package, its `__path__` becomes the `path` argument when searching for `a.b`. This is why `sys.meta_path` finders receive `path=None` for top-level names and a list of directory strings for dotted submodules.

The loader half — `__load_unlocked` — is where `ModuleSpec` becomes a live module object (detailed in §7):

```
# Skeleton of __load_unlocked (PEP 451 path):
def __load_unlocked(spec):
    module = spec.loader.create_module(spec) # may return None →
    default creation
    if module is None:
        module = _imp.create_dynamic(spec) if spec.origin == "extension"
    else type(sys)(spec.name) # plain module type
    module.__spec__ = spec
    module.__loader__ = spec.loader

    # ... _bootstrap._init_module_attrs sets __package__, __path__,
    # __cached__, etc.

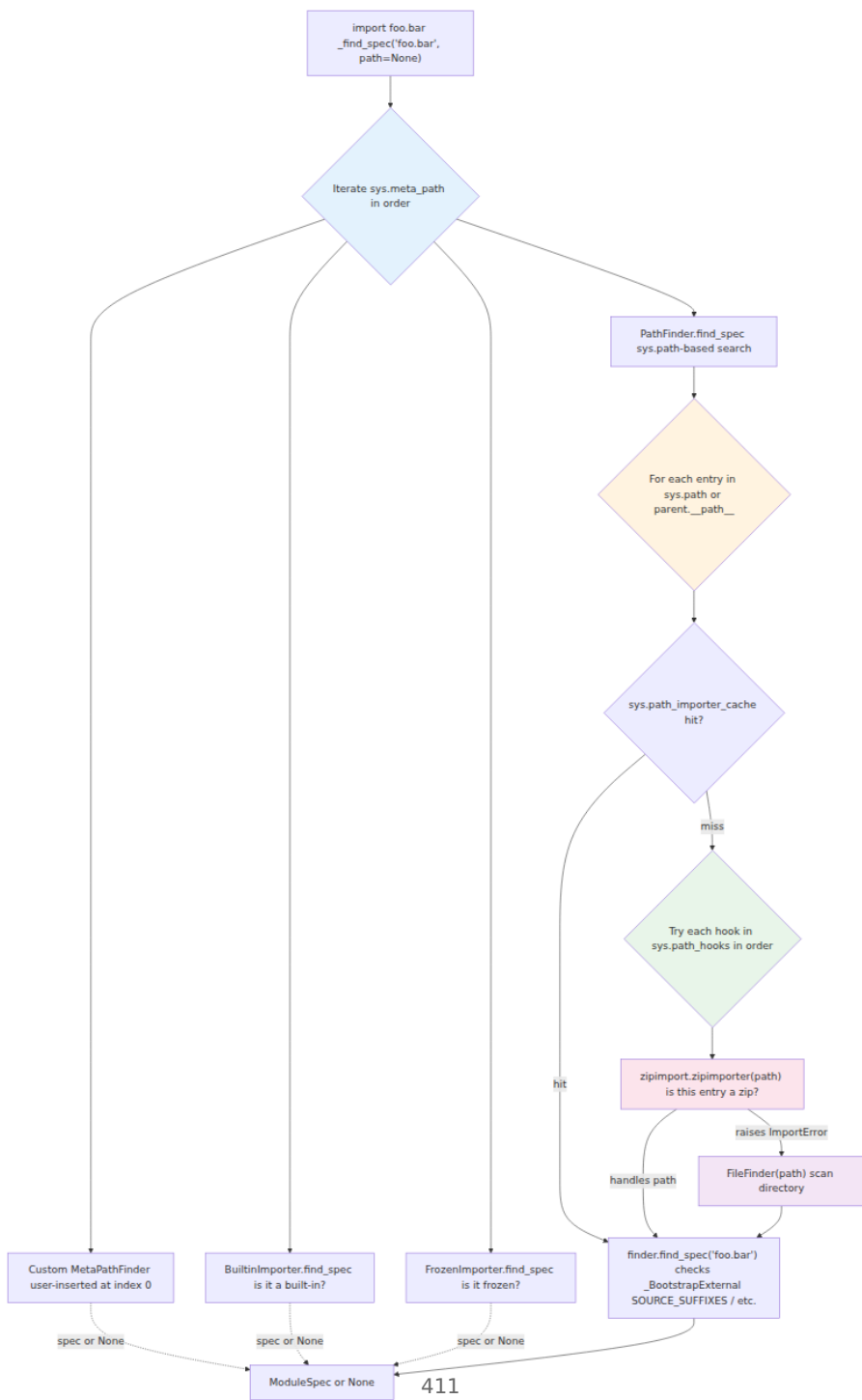
    # sys.modules insertion happens BEFORE exec_module – essential for
    # circular imports (see §8) and visible to import hooks.
    sys.modules[spec.name] = module

    try:
        spec.loader.exec_module(module) # the entire execution of the
        module body
    except BaseException:
        # On failure, remove the half-initialized entry so a retry re-
        # finds it.
        del sys.modules[spec.name]
        raise
    return module
```

Before PEP 451 (Python 3.4), loaders implemented `load_module(fullname)` which both found and loaded in one call. That method is now deprecated and removed in 3.12 — loaders that still define it get a `DeprecationWarning` shim in `_bootstrap`. New code must implement `exec_module` (and optionally `create_module`).

4. Dispatch — `sys.meta_path`, `sys.path_hooks`, `sys.path_importer_cache`

Three attributes on `sys` form the dispatch table. Understanding their precedence explains every "where did this module come from?" question.



`sys.meta_path` — the first-chance intercept

A list of *finder* objects. Each must implement `find_spec(fullname, path, target=None)`. Iteration stops at the first finder returning a non-`None` `ModuleSpec`. Ordering matters:

```
import sys
for i, finder in enumerate(sys.meta_path):
    print(i, finder)
# 0 <class '_frozen_importlib.BuiltinImporter'>
# 1 <class '_frozen_importlib.FrozenImporter'>
# 2 <class '_frozen_importlib_external.PathFinder'>

# Inserting a custom finder at position 0 intercepts every import:
# sys.meta_path.insert(0, MyFinder())
# Inserting at the end only handles names no earlier finder claimed.
```

`BuiltinImporter` and `FrozenImporter` are unusual — their `find_spec` is a `@classmethod` on the class object itself, not an instance method. `PathFinder` is the workhorse for filesystem imports; it iterates `sys.path` (or the parent's `__path__` for submodules) and consults `sys.path_importer_cache` / `sys.path_hooks` per entry.

`sys.path` — the search list

Not a dispatch mechanism itself, but the data `PathFinder` searches. Initialized at startup from (in priority order): the script's directory (or `""` for `-c` / `stdin`), `PYTHONPATH`, `site.py` additions (`site-packages`, `.pth` processing), and `PYTHONSAFEPATH` / `-P` / `-I` isolation flags. Chapter 1 covers `PyConfig` / `initconfig.c` initialization; for import purposes, treat `sys.path` as "the ordered list `PathFinder` will probe."

```
import sys, pprint
pprint.pp(sys.path[:6])
# ['',                                     # script dir / "" for REPL
#  '/usr/lib/python3.11',
#  '/usr/lib/python3.11/lib-dynload',
#  '/home/ubuntu/.local/lib/python3.11/site-packages',
#  '/usr/local/lib/python3.11/dist-packages',
#  ...]
```

For submodule imports, `PathFinder` does not use `sys.path` — it uses the parent package's `__path__` (a list-like `_NamespacePath` or plain list). That is why `import pkg.sub` searches only inside `pkg`'s directories.

`sys.path_hooks` and `sys.path_importer_cache` — per-entry finders

For each `sys.path` entry, `PathFinder` checks `sys.path_importer_cache` first. On a miss, it tries each callable in `sys.path_hooks` in order until one returns a *path entry finder* (an object with `find_spec(fullname, target=None)`). The result is cached:

```
import sys
print(sys.path_hooks)
# [
```

`FileFinder` (from `_bootstrap_external`) is the default path entry finder for filesystem directories. It is constructed with the `path` string and knows the suffix tuples:

```
import importlib.machinery

print(importlib.machinery.SOURCE_SUFFIXES)    # ['.py']
print(importlib.machinery.BYTECODE_SUFFIXES)  # ['.pyc']
print(importlib.machinery.EXTENSION_SUFFIXES) # ['.cpython-311-x86_64-
linux-gnu.so', '.abi3.so', '.so']
print(importlib.machinery.all_suffixes())      # all of the above
```

Inside `FileFinder.find_spec`, the logic is a priority-ordered probe within that single directory: for a top-level `foo`, check `foo/__init__.py` (regular package) → `foo.py` (module) → `foo.cpython-311-....so` (extension) → `foo/__pycache__/*.pyc` (namespace package probe if no `__init__.py` and

PEP 420 enabled). For `foo.bar`, check `foo/bar/__init__.py` relative to the `FileFinder`'s path.

`zipimport.zipimporter` is the other default hook. If the `sys.path` entry is a zip file (or contains a `.zip/.egg/.whl` component), `zipimporter` claims it; otherwise it raises `ImportError` and `PathFinder` tries the next hook. There is no third default hook — any additional `sys.path_hooks` entry is user- or framework-added.

5. Finders — `MetaPathFinder.find_spec` and `PathEntryFinder`

PEP 302 introduced finders and loaders; PEP 451 (Python 3.4) replaced the older `find_loader` / `load_module` protocol with `find_spec` / `ModuleSpec` / `exec_module`. The modern hierarchy:

ABC	Module	Method
<code>importlib.abc.MetaPathFinder</code>	<code>_bootstrap</code>	<code>find_spec(fullname, path, target=None)</code>
<code>importlib.abc.PathEntryFinder</code>	<code>_bootstrap_external</code>	<code>find_spec(fullname, target=None)</code>
<code>importlib.abc.ResourceReader</code>	<code>importlib.abc</code>	<code>open_resource, resource_path, is_resource</code>

Both finder types return either a `ModuleSpec` or `None`. Returning `None` means "I don't handle this name — try the next finder." Raising `ImportError` is not expected from `find_spec`; `ModuleNotFoundError` is raised by the caller if all finders return `None`.

The distinction between meta-path and path-entry finders is about *when* they run:

- **Meta-path finder** — sees every import, can handle any name, decides based on `fullname` alone. Used for virtual modules, import-time rewriting, lazy loading wrappers, and synthetic namespaces.
- **Path-entry finder** — sees only imports that `PathFinder` routed to its specific directory/zip. Created per `sys.path` entry via `sys.path_hooks`. Handles the filesystem reality for that entry.

Instrumenting which finder claimed a name:

```

import importlib.util, importlib.machinery

# find_spec walks sys.meta_path for you and returns the winning
ModuleSpec:
spec = importlib.util.find_spec("json")
print(spec)
# ModuleSpec(name='json',
# loader=<_frozen_importlib_external.SourceFileLoader object at ...>,
# origin='/usr/lib/python3.11/json/__init__.py',
# submodule_search_locations=['/usr/lib/python3.11/json'])

print(f"loader:      {spec.loader}")
print(f"origin:      {spec.origin}")
print(f"is_package:   {spec.submodule_search_locations is not None}")
print(f"parent:        {spec.parent}")           # 'json' for top-level,
'json.decoder' → 'json'
print(f"has_location:{spec.has_location}")     # False for namespace
packages

# For a namespace package (no __init__.py), compare:
# (create a demo: mkdir -p /tmp/ns_demo/pkg && touch /tmp/ns_demo/pkg/
mod.py)
import sys, pathlib, tempfile, os
tmpdir = tempfile.mkdtemp()
os.makedirs(f"{tmpdir}/ns_pkg/sub")
pathlib.Path(f"{tmpdir}/ns_pkg/sub/mod.py").write_text("x = 1\n")
sys.path.insert(0, tmpdir)
spec_ns = importlib.util.find_spec("ns_pkg")
print(f"ns_pkg spec: {spec_ns}")
print(f"  origin: {spec_ns.origin}")           # None
for namespace
print(f"  loader: {spec_ns.loader}")           # None
for namespace
print(f"  search_locations: {spec_ns.submodule_search_locations}") #
_NamespacePath([...])
print(f"  has_location: {spec_ns.has_location}")       # False
sys.path.remove(tmpdir)

# For an extension module:
spec_ext = importlib.util.find_spec("_json")
print(f"_json spec: loader={type(spec_ext.loader).__name__}, origin={sp
ec_ext.origin}")
# loader=ExtensionFileLoader, origin=/usr/lib/python3.11/lib-dynload/
_json.cpython-311-....so

# For a built-in:
spec_bi = importlib.util.find_spec("sys")
print(f"sys spec: loader={spec_bi.loader}, origin={spec_bi.origin}")
# loader=<class '_frozen_importlib.BuiltinImporter'>, origin=built-in

```


6. Loaders — `exec_module` vs legacy `load_module`, and the concrete loader types

Once a finder returns a `ModuleSpec`, the loader in `spec.loader` takes over. The modern loader protocol (PEP 451):

```
class Loader(importlib.abc.Loader):
    def create_module(self, spec):
        """Optional. Return a module object or None for default
        creation.
        Used to return a custom subclass, or to use _imp.create_dynamic
        for extension modules with non-standard initialization."""
        return None # default: type(sys)(spec.name)

    def exec_module(self, module):
        """Required. Execute the module's code in module.__dict__.
        Must set __spec__, __loader__, and any package attributes.
        Called with the module already inserted in sys.modules."""
        raise NotImplementedError
```

The legacy protocol — `loader.load_module(fullname)` — did both steps (create + exec + `sys.modules` insertion) in one call. It is deprecated since Python 3.4 and removed in 3.12. `_bootstrap` still shims it for very old loaders but emits `DeprecationWarning`. If you encounter `load_module` in vendor code, it is a compatibility hazard.

Concrete loaders in `_bootstrap_external`

Loader	Handles	origin	What <code>exec_module</code> does
<code>SourceFileLoader</code>	<code>.py</code> files	<code>/path/to/mod.py</code>	Reads source, compiles to <code>PyCodeObject</code> (or <code>__pycache__</code> pyc per PEP 552), executes it; <code>module.__dict__</code> , <code>__file__</code> , <code>__cached__</code> are set
<code>SourcelessFileLoader</code>	<code>.pyc</code> -only (no <code>.py</code>)	<code>/path/to/__pycache__/mod.cpython-311.pyc</code>	Loads marshalled <code>PyCodeObject</code> from file; <code>exec</code> s it; <code>get_source</code> returns <code>None</code>

Loader	Handles	origin	What <code>exec_module</code> does
<code>ExtensionFileLoader</code>	<code>.so</code> / <code>.pyd</code> (C extensions)	<code>/path/.../mod.cpython-311-...so</code>	<code>dlopen</code> the shared object, calls <code>PyInit_mod</code> (<code>PyMODINIT_FUNC</code>), returns the <code>PyModuleDef</code> 's module
<code>NamespaceLoader</code> (<i>internal</i>)	Namespace packages	<code>None</code>	No <code>exec_module</code> — namespace package; no code to execute; <code>spec.loader</code> is <code>None</code> and <code>spec.origin</code> is <code>None</code>
<code>BuiltinImporter</code>	Built-in modules	<code>built-in</code>	Calls the C <code>PyInit_</code> function linked into interpreter binary
<code>FrozenImporter</code>	Frozen modules	<code>frozen</code>	Unmarshals bytecode; <code>PyImport_FrozenM</code> table
<code>ZipImporter</code>	Modules inside <code>.zip</code>	<code>/path/to/archive.zip/pkg/mod.py</code>	Reads source/bytecode from the zip's central directory, otherwise <code>SourceFileLoader</code> from zip bytes

`SourceFileLoader` deserves a closer look because it owns `pyc` caching. Its `exec_module` path:

1. Compute `__cached__` via `importlib.util.cache_from_source(origin)` — e.g., `mod.py` → `__pycache__/mod.cpython-311.pyc`.
2. Check if the `pyc` exists and is valid (PEP 552 header validation — see §13).
3. If valid, `marshal.loads` the `PyCodeObject` from the `pyc` body and `exec` it — skipping tokenize/parse/compile entirely.
4. If not valid or absent, read `origin`, `compile(source, origin, "exec")`, `exec` it, then (unless `PYTHONDONTWRITEBYTECODE` / `sys.dont_write_bytecode` / read-only filesystem) write a new `pyc` with the appropriate header.

```

import importlib.util, importlib.machinery, pathlib, py_compile

# Where a source file's pyc would live:
src = "/usr/lib/python3.11/json/__init__.py"
print(importlib.util.cache_from_source(src))
# /usr/lib/python3.11/json/__pycache__/__init__.cpython-311.pyc

# And the reverse:
cached = "/usr/lib/python3.11/json/__pycache__/
__init__.cpython-311.pyc"
print(importlib.util.source_from_cache(cached))
# /usr/lib/python3.11/json/__init__.py

# Inspect what SourceFileLoader would do without executing:
from importlib.machinery import SourceFileLoader
loader = SourceFileLoader("json", src)
print(f"source: {loader.get_filename('json')}") # /usr/lib/
python3.11/json/__init__.py
print(f"is_package: {loader.is_package('json')}") # True
print(f"data sample: {loader.get_data(src)[:40]!r}")

# Extension loader – no source, no get_source:
from importlib.machinery import ExtensionFileLoader
import _json
spec = importlib.util.find_spec("_json")
print(f"ExtensionFileLoader origin: {spec.origin}")
print(f"ExtensionFileLoader is_package:
{spec.loader.is_package('_json')}")

```

Subclassing `SourceFileLoader` is the standard way to add source transformations (instrumentation, macro expansion, import rewriting) — override `get_data` or `source_to_code`:

```

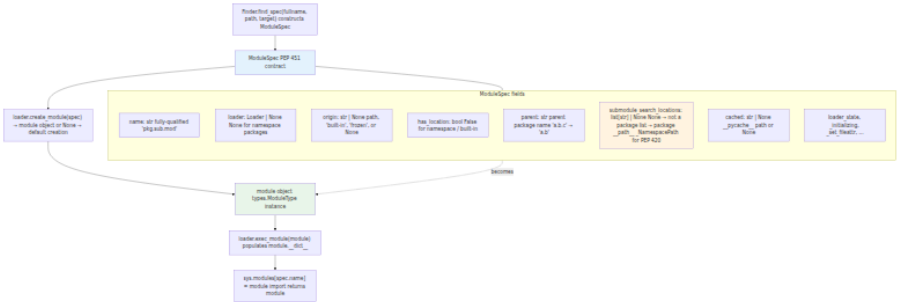
import importlib.machinery

class InstrumentingLoader(importlib.machinery.SourceFileLoader):
    def source_to_code(self, data, path, *, _optimize=-1):
        source = importlib._bootstrap_external.decode_source(data)
        # Example: inject a header, rewrite AST, etc.
        # source = transform(source)
        return compile(source, path, "exec", dont_inherit=True, optimiz
e=_optimize)

```

7. ModuleSpec — the contract between finder and loader

PEP 451's `ModuleSpec` is the central data structure. The finder creates it; the loader consumes it; the resulting module carries it as `__spec__`. Every field matters for introspection and for frameworks that synthesize modules.



Complete field reference:

```

import importlib.util

# Module
spec = importlib.util.find_spec("json.decoder")
print(f"name: {spec.name!r}")
print(f"loader: {spec.loader}")
print(f"origin: {spec.origin!r}")
print(f"has_location: {spec.has_location}")
print(f"parent: {spec.parent!r}")
print(f"submodule_search_locations: {spec.submodule_search_locations}")
print(f"cached: {spec.cached!r}")
print(f"loader_state: {spec.loader_state!r}")

# Package
spec_pkg = importlib.util.find_spec("json")
print(f"\njson is package: {spec_pkg.submodule_search_locations is not None}")
print(f"json cached: {spec_pkg.cached!r}")
print(f"json origin: {spec_pkg.origin!r}")

# After import, the module carries the spec:
import json
assert json.__spec__ is spec_pkg or json.__spec__.name == "json"
print(f"\njson.__spec__ is spec_pkg: {json.__spec__ is spec_pkg}") #
may be identity or equal
print(f"json.__loader__: {json.__loader__}")
print(f"json.__package__: {json.__package__!r}")
print(f"json.__path__: {json.__path__}")
print(f"json.__file__: {json.__file__!r}")
print(f"json.__cached__: {json.__cached__!r}")

# What _bootstrap._init_module_attrs sets (called inside
_load_unlocked):
# module.__spec__ = spec
# module.__loader__ = spec.loader
# module.__package__ = spec.parent (or spec.name if spec is a
package)
# module.__path__ = spec.submodule_search_locations (only if
package)
# module.__file__ = spec.origin (if spec.has_location)
# module.__cached__ = spec.cached (if spec.has_location)

```

A subtlety: `spec.parent` is not always `spec.name.rsplit(".", 1)[0]`. For top-level modules `spec.parent == spec.name` (e.g., `json` $\rightarrow$ parent `json`); for submodules it is the parent package (`json.decoder` $\rightarrow$ `json`). For namespace packages, `spec.parent` is still the logical parent used for relative import resolution.

Frameworks that synthesize modules (e.g., `importlib.util.module_from_spec`) rely on exact `ModuleSpec` construction:

```
import importlib.util, importlib.machinery, sys, types

# Synthesize a module without touching the filesystem:
spec = importlib.machinery.ModuleSpec(
    name="synthetic.hello",
    loader=None, # namespace-like, or provide a loader
    origin="synthetic",
    is_package=False,
)
mod = importlib.util.module_from_spec(spec)
# module_from_spec calls spec.loader.create_module if loader is not
# None,
# otherwise creates a plain module and attaches __spec__/_loader__.
mod.greeting = "hello from synthetic"
sys.modules[spec.name] = mod

import synthetic.hello
print(synthetic.hello.greeting) # hello from synthetic
del sys.modules[spec.name]
```

8. Import state — `sys.modules`, the import lock, and circular imports

`sys.modules` — the import cache

`sys.modules` is a plain `dict` mapping fully-qualified names to module objects. It is the first thing `_find_and_load` checks and the last thing `_load_unlocked` populates. Its roles:

- **Cache.** Every successful import inserts exactly one entry. Re-importing returns the same object — `import os; import os as os2; assert os is os2`.
- **Partial-initialization sentinel.** During `exec_module`, the module is already in `sys.modules` but its `__dict__` is only partially populated. Code that re-imports the same name during this window gets the partial module — this is how circular imports can work (or break).
- **Invalidation point.** Deleting `sys.modules["foo"]` forces the next `import foo` to re-run finders and loaders. Mutating `sys.modules` is

the standard test isolation technique and the standard way to break things.

```
import sys, importlib

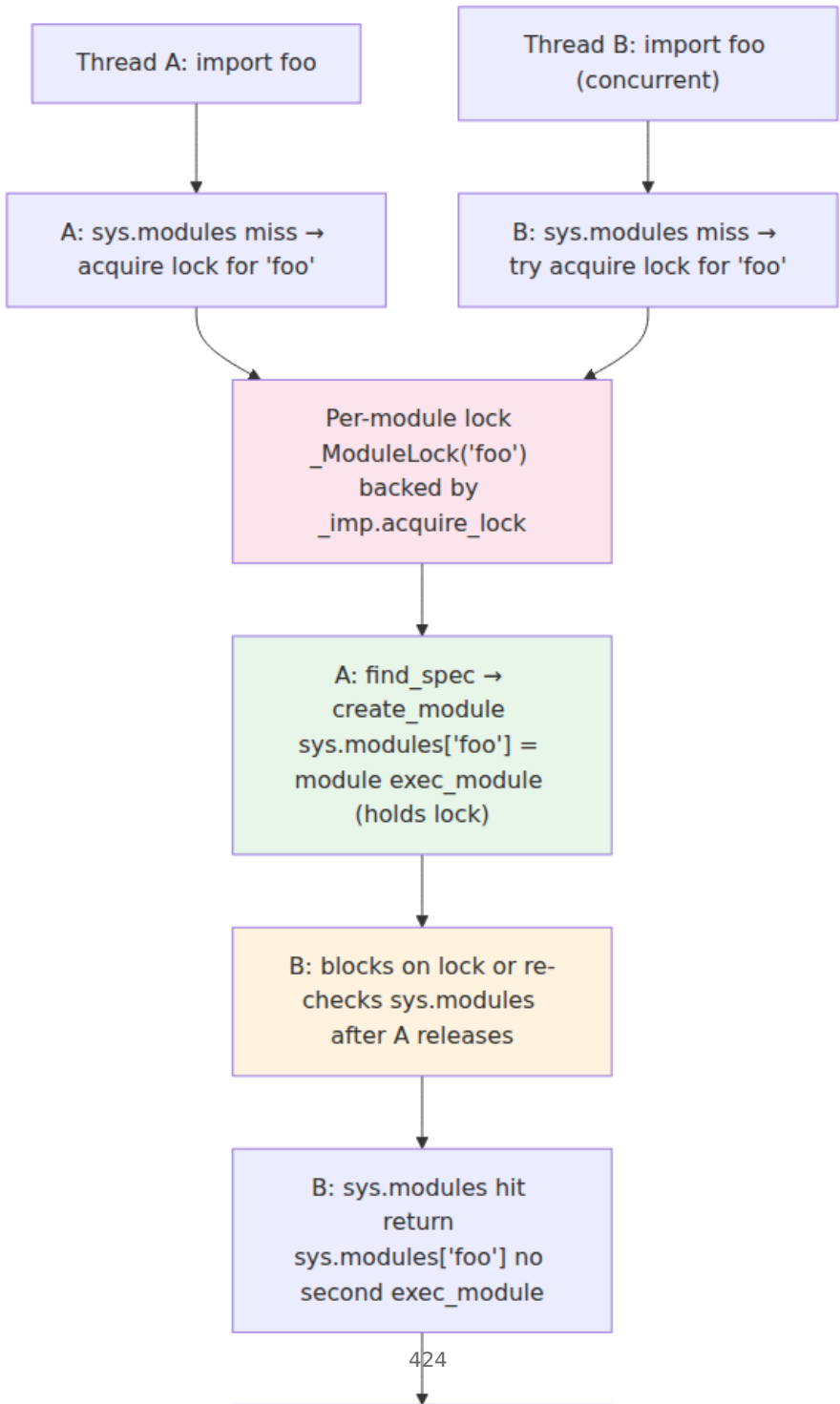
# Inspect the cache:
print(f"modules loaded: {len(sys.modules)}")
print(f"json in cache: {'json' in sys.modules}")
print(f"json is sys.modules['json']: {importlib.import_module('json') is sys.modules['json']}")

# What a circular import sees:
# a.py: import b; x = 1
# b.py: import a; print(a.x) # AttributeError if b is imported before a finishes defining x
#
# Trace:
# import a → sys.modules["a"] = <partial a> → exec a.py → import b
#           → sys.modules["b"] = <partial b> → exec b.py → import a
#           → sys.modules hit: return <partial a> (x not yet set!)
#           → b finishes, then a finishes.

# Forcing reimport:
if "json" in sys.modules:
    old_id = id(sys.modules["json"])
    del sys.modules["json"]
    import json as json2
    print(f"reimported json is new object: {id(json2) != old_id}")
```

The import lock — `_imp.acquire_lock` / `_ModuleLock`

Import is not free-threaded. CPython protects the `sys.modules` insertion + `exec_module` window with a per-module lock plus a global import lock (historically a single global lock; since 3.3, a per-module lock keyed by name with a global fallback for extension init).



In C, the lock lives in `Python/import.c` (`import_lock` / `import_lock_thread` / `_PyImport_AcquireLock`) and is exposed to Python as `_imp.acquire_lock()` / `_imp.release_lock()` / `_imp.lock_held()`. In Python, `_bootstrap.ModuleLock` wraps it with re-entrancy tracking so the same thread can re-enter import for a different name.

Practical consequences:

- **Import is not parallel.** Two threads importing *different* modules can proceed concurrently (different per-module locks), but two threads importing the *same* uninitialized module serialize. Import-heavy startup on a threaded server (e.g., `gunicorn --workers 1 --threads 8`) still serializes on first import of each module.
- **Deadlock risk.** If `exec_module` for `a` tries to import `b` while another thread holds `b`'s lock and tries to import `a`, the per-module locks do not deadlock (they are not held across the `exec_module` of the other module in a way that creates a cycle — but legacy global-lock code could). In practice, circular imports within a single thread are the more common hazard.
- **Extension init is global.** `ExtensionFileLoader` holds the global import lock during `PyInit_*` execution, because C extension init is not re-entrant. A slow or blocking `PyInit_foo` stalls all other imports.

```

import _imp, threading, importlib.util, sys, time

print(f"import lock held: {_imp.lock_held()}") # False outside import

# Demonstrate per-module serialization:
# (run with: python -c "import threading, importlib, time; ...")

def slow_import(name, delay=0.2):
    # Simulate a loader that sleeps during exec_module.
    import importlib.machinery, types
    spec = importlib.machinery.ModuleSpec(name, loader=None, origin="synthetic")
    mod = types.ModuleType(name)
    mod.__spec__ = spec
    sys.modules[name] = mod
    time.sleep(delay)
    mod.loaded = True
    return mod

# Observing the lock from a custom loader (see §15 for full example):
import importlib.abc

class LockObservingLoader(importlib.abc.Loader):
    def create_module(self, spec): return None
    def exec_module(self, module):
        print(f"exec_module {module.__spec__.name}: lock_held={_imp.lock_held()}")
        module.value = 42

# The import lock is held during exec_module – any code there
# that calls back into import will re-enter _ModuleLock safely.

```

9. Relative imports — PEP 328

Relative imports (`from . import foo` , `from ..bar import baz`) are resolved by `_gcd_import`'s `level` and `package` arguments, not by filesystem path arithmetic.

```

# Inside pkg/sub/mod.py:
from . import sibling      # level=1, package="pkg.sub" → "pkg.sub.sibling"
from .. import parent_mod # level=2, package="pkg.sub" → "pkg.parent_mod"
from ..other import thing # level=2, package="pkg.sub" → "pkg.other.thing"

```

Resolution rule (in `__bootstrap__resolve_name`):

```
if level == 0:
    # Absolute import – name is used as-is.
    absolute = name
else:
    # Relative – walk `level` dots up from `package`.
    # package is caller's __package__ (or __spec__.parent if
    # __package__ is None).
    # level=1 → current package, level=2 → parent, etc.
    base = package.rsplit(".", level - 1)[0] if level > 1 else package
    absolute = f"{base}.{name}" if name else base
```

`__package__` vs `__spec__.parent`: on a regular package, `__package__ == __name__`; on a module, `__package__ == __name__.rsplit(".", 1)[0]`. `importlib.bootstrap` sets `__package__` from `spec.parent` during `__init_module_attrs`, but user code can override `__package__` to change relative import semantics — a sharp edge for code that manually constructs modules.

Common pitfalls:

- **Running a submodule as `__main__`**. `python pkg/sub/mod.py` sets `__package__ = None` and `__spec__.parent = None`, so `from . import sibling` fails with `ImportError: attempted relative import with no known parent package`. Fix: `python -m pkg.sub.mod`.
- **`__package__` set to `""` vs `None`**. Both mean "no package," but `""` can produce different error messages than `None` in older Python versions. Modern `__bootstrap` normalizes to `None` → `spec.parent`.

```
# Demonstrate relative resolution without touching the filesystem:
from importlib.bootstrap import _resolve_name

print(_resolve_name("sibling", "pkg.sub", 1))      # pkg.sub.sibling
print(_resolve_name("parent_mod", "pkg.sub", 2))   # pkg.parent_mod
print(_resolve_name("", "pkg.sub", 1))             # pkg.sub
(from . import x)
print(_resolve_name("", "pkg.sub", 2))             # pkg
(from .. import x)
# Absolute:
print(_resolve_name("os.path", None, 0))          # os.path
```

10. Namespace packages — PEP 420 (no `__init__.py`)

Before PEP 420 (Python 3.3), every package required an `__init__.py`. A directory without one was invisible to import. PEP 420 introduced *namespace packages*: directories (or zip entries) that contribute to a package's `__path__` without owning it.



Key differences:

Property	Regular package	Namespace package 420)
<code>__init__.py</code>	Required	Absent (by definition)
<code>__spec__.origin</code>	<code>"/path/to/pkg/__init__.py"</code>	<code>None</code>
<code>__spec__.loader</code>	<code>SourceFileLoader</code>	<code>None</code>
<code>__spec__.has_location</code>	<code>True</code>	<code>False</code>
<code>__spec__.submodule_search_locations</code>	<code>["/path/to/pkg"]</code> (plain list)	<code>_NamespacePath(["/p pkg", "/path/b/pkg"]</code>
<code>__path__</code>	<code>["/path/to/pkg"]</code>	<code>_NamespacePath</code> (iter supports <code>__iter__</code> , <code>__len__</code> , dynamic merging)
<code>__file__</code>	<code>"/path/to/pkg/__init__.py"</code>	Absent (no <code>__file__</code>)
Code execution	<code>__init__.py</code> runs on <code>import pkg</code>	Nothing runs; <code>import</code> only creates the namespace module
<code>pkgutil / importlib.resources</code>	Works	Works (3.9+ <code>importlib.resources</code> handles namespace)

`_NamespacePath` is the subtle part. It is not a plain list — it is a `_NamespacePath` object (`_bootstrap.NamespacePath`) that aggregates

`pkg/` directories from every `sys.path` entry where `pkg/` exists as a directory without `__init__.py`. Submodule search iterates over all of them:

```
import sys, tempfile, pathlib, os, importlib.util, importlib.machinery

# Two directories contributing to the same namespace:
a = tempfile.mkdtemp()
b = tempfile.mkdtemp()
for base in (a, b):
    os.makedirs(f"{base}/ns")

pathlib.Path(f"{a}/ns/mod_a.py").write_text("value = 'from A'\n")
pathlib.Path(f"{b}/ns/mod_b.py").write_text("value = 'from B'\n")

sys.path.insert(0, a)
sys.path.insert(0, b)

import ns # no ns/__init__.py in either location
print(f"ns.__path__: {list(ns.__path__)}")
# ['/tmp/.../ns', '/tmp/.../ns'] - both locations
print(f"ns.__spec__.origin: {ns.__spec__.origin}") # None
print(f"ns.__spec__.loader: {ns.__spec__.loader}") # None
print(f"type(ns.__path__): {type(ns.__path__).__name__}") #
_NamespacePath

import ns.mod_a, ns.mod_b
print(ns.mod_a.value) # from A
print(ns.mod_b.value) # from B

# find_spec still works:
print(importlib.util.find_spec("ns.mod_a"))
print(importlib.util.find_spec("ns.mod_b"))

sys.path.remove(a); sys.path.remove(b)
```

Implications for backend monorepos:

- **Split namespace across repos.** `company.auth` in repo A and `company.billing` in repo B can both be `company.*` namespace packages installed to different `site-packages` subtrees. Import merges them transparently — but only if no `company/__init__.py` exists anywhere on `sys.path`. A single stray `__init__.py` converts the namespace to a regular package and hides the other contributions.
- `pkgutil.extend_path` / `pkg_resources.declare_namespace` are legacy. Before PEP 420, namespace packages required explicit `__init__.py`

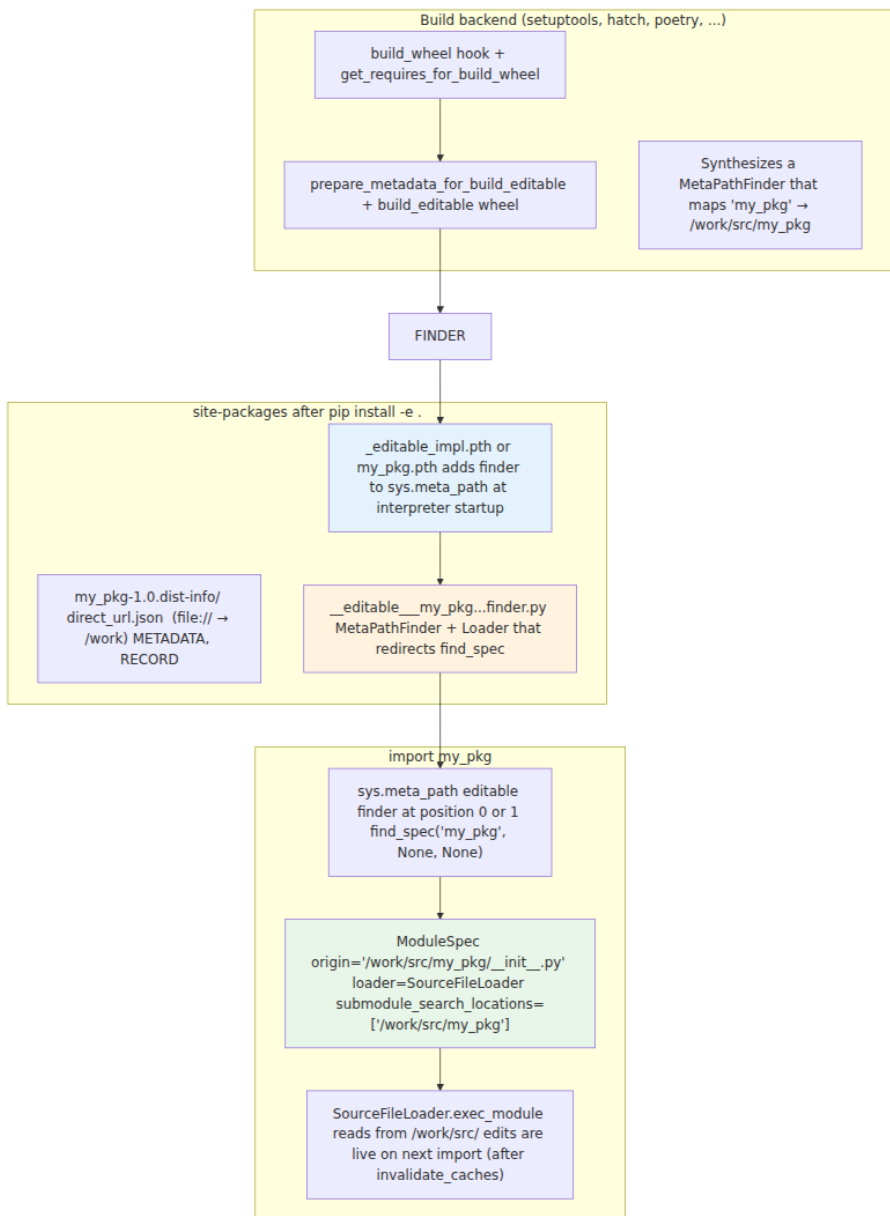
`code ( __path__ = pkgutil.extend_path(__path__, __name__ ) ).` That pattern still works for backward compat but is unnecessary on Python 3.3+ and interacts poorly with `importlib.resources`.

- **`__path__` is mutable but `NamespacePath` is dynamic.** Appending to `ns.__path__` works, but `NamespacePath` also re-scans `sys.path` on iteration if new entries appear. Relying on this dynamism is fragile — prefer `importlib.invalidate_caches()` and explicit `sys.path` management.
-

11. Editable installs — PEP 660

`pip install -e .` (editable install) makes the working tree importable without copying files to `site-packages`. Before PEP 660 (Python 3.10 era), this was implemented by `setuptools`' `setup.py develop`: a `.egg-link` file plus a `.pth` that added the project root to `sys.path`, and a shim `__editable__` finder for `src/` layouts. That mechanism was `setuptools`-specific, required `setup.py`, and broke `importlib.resources` for many layouts.

PEP 660 standardizes editable installs via `importlib.machinery` and a build-backend-provided import hook:



What actually lands in `site-packages` :

```
# After `pip install -e .` in a project with `src/my_pkg/__init__.py`:

$ ls site-packages/*.pth site-packages/__editable__* 2>/dev/null
site-packages/__editable__.my_pkg-1.0.pth
site-packages/__editable__my_pkg_1_0_finder.py

$ cat site-packages/__editable__.my_pkg-1.0.pth
import __editable__my_pkg_1_0_finder; __editable__my_pkg_1_0_finder.i
ninstall()

$ cat site-packages/__editable__my_pkg_1_0_finder.py | head -40
# ... generates a MetaPathFinder that maps "my_pkg" → "/work/src"
# and "my_pkg.sub" → "/work/src/my_pkg/sub" via FileFinder-like
logic
```

Inspecting an editable install at runtime:

```
import importlib.util, sys, pathlib

spec = importlib.util.find_spec("my_pkg") # if installed editable
if spec:
    print(f"origin: {spec.origin}") # /work/src/my_pkg/__init__.py
    (working tree, not site-packages)
    print(f"loader: {spec.loader}")
    print(f"finder: {spec.loader}") # the editable finder's loader,
    usually SourceFileLoader

    # The editable finder is on sys.meta_path – find it:
    for f in sys.meta_path:
        if "editable" in type(f).__name__.lower() or "editable" in
repr(f).lower():
            print(f"editable finder: {f}")

    # dist-info still in site-packages, source in working tree:
    import importlib.metadata
    dist = importlib.metadata.distribution("my-pkg")
    print(dist.read_text("direct_url.json")) # {"dir_info":
{"editable": true}, "url": "file:///work"}
```

PEP 660 vs legacy `setup.py develop`:

Aspect	Legacy <code>setup.py develop</code>	PEP 660 <code>pip install -e .</code>
Trigger	<code>python setup.py develop</code>	<code>pip install -e .</code> (any PEP 517 backend)

Aspect	Legacy setup.py develop	PEP 660 pip install -e .
sys.path mutation	.egg-link + .pth adds project root to sys.path	.pth installs a MetaPathFinder; project root not on sys.path
src/ layout	Required setup.py shim; importlib.resources often broken	Backend-provided finder correctly maps src/my_pkg → my_pkg
importlib.resources	Frequently fails (resource path != package path)	Works — finder provides correct ResourceReader
RECORD	Not updated	direct_url.json records file:// + editable: true

Backend relevance:

- **Editable installs are development-only.** They leave .pth import hooks that execute at interpreter startup. In production images, always pip install . (non-editable) so site-packages contains real files and no finder indirection.
- **Finding the editable finder.** If import my_pkg resolves to /work/src/... in CI but to site-packages/... in production, importlib.util.find_spec("my_pkg").origin tells you which environment you are in — useful for health checks and startup diagnostics.
- **pip install -e . --no-build-isolation caveat.** Without build isolation, the build backend runs in the host environment, and the editable finder may capture absolute paths that break when the image is moved.

12. `zipimport`, import hooks, and freezing

`zipimport` — importing from zip files

`zipimport.zipimporter` is the only `sys.path_hooks` entry besides `FileFinder.path_hook` in a default interpreter. It handles any `sys.path` entry that is a zip file (including `.egg`, `.whl`, and `.zip`):

```
import sys, zipfile, tempfile, pathlib, importlib.util

# Create a zip containing a package:
tmpdir = tempfile.mkdtemp()
zippath = f"{tmpdir}/bundle.zip"
with zipfile.ZipFile(zippath, "w") as zf:
    zf.writestr("zpkg/__init__.py", "value = 'from zip'\n")
    zf.writestr("zpkg/mod.py", "x = 42\n")
    zf.writestr("zpkg/data.txt", "hello\n")

sys.path.insert(0, zippath)
import zpkg.mod
print(zpkg.mod.x)                    # 42
print(zpkg.mod.__loader__)           # <zipimporter object "/
tmp/.../bundle.zip">
print(zpkg.mod.__spec__.origin)      # /tmp/.../bundle.zip/zpkg/
mod.py
print(zpkg.mod.__spec__.loader)      # <zipimporter object ...>
# zipimporter also serves as ResourceReader for importlib.resources:
print(zpkg.__loader__.get_data(f"{zippath}/zpkg/data.txt")[:5]) #
b'hello'

# zipimport caches the zip's central directory – inspect:
import zipimport
zi = zipimport.zipimporter(zippath)
print(zi.find_spec("zpkg.mod"))

sys.path.remove(zippath)
```

`ZipImporter` limitations relevant to backends:

- **No `__pycache__` inside zips.** Bytecode is not cached to the zip; `SourceFileLoader`-style `pyc` validation does not apply. Each import reads and compiles from zip bytes (or loads a pre-compiled `.pyc` stored in the zip, if the zip was built with `compileall`).
- **No namespace packages inside a single zip by default.** A zip with `ns/pkg/mod.py` but no `ns/__init__.py` inside the same zip does not produce a PEP 420 namespace — `zipimporter` requires

`__init__.py` for packages. Multi-zip namespace merging via `_NamespacePath` works only for filesystem `FileFinder` entries, not for two zips contributing to the same namespace (unless a custom meta-path finder merges them).

- **`__file__` points inside the zip.** `zpkg.mod.__file__ == "/tmp/.../bundle.zip/zpkg/mod.py"` — code that does `pathlib.Path(__file__).parent / "data.txt"` will fail for zip-imported modules (the path is not a real filesystem path). Use `importlib.resources` instead.

Import hooks — the general extension point

Any object on `sys.meta_path` or any callable on `sys.path_hooks` is an import hook. Beyond `zipimport` and `FileFinder`, common hooks in backend stacks:

- **`importlib.machinery.FileFinder`** — the default filesystem hook.
- **PEP 660 editable finder** — `MetaPathFinder` installed via `.pth`.
- **`pytest assertion rewriter`** — `MetaPathFinder` + `SourceFileLoader` subclass that rewrites `assert` statements via AST at import time.
- **`coverage tracer`** — similar AST-rewriting loader.
- **`vendored / importlib_resources shims`** — backport finders.

Writing a hook is the subject of §15 (custom finder/loader). The protocol to respect:

1. `find_spec` must return `None` for names it does not handle — never raise, never return a spec with `loader=None` for a non-namespace package.
2. `exec_module` must not assume `module.__dict__` is empty — it may contain `__spec__`, `__loader__`, `__package__` already set by `__bootstrap__`.
3. Hooks on `sys.meta_path` see every import, including `stdlib`. A buggy `find_spec` that is slow or raises will stall or break the entire interpreter.

Freezing — PyImport_ImportModule and freeze_modules.py

Freezing bakes Python modules into the interpreter binary so they are available without filesystem access. Used for:

- The bootstrap itself (`_bootstrap` , `_bootstrap_external`).
- Standalone binaries (PyInstaller, `cx_Freeze` , `PyOxidizer` , `BeeWare`).
- Embedded interpreters (`PyImport_AppendInittab` + `PyImport_ImportModule` in C).

At the C level:

```
// Python/import.c – frozen module table:
struct _frozen_PyImport_FrozenModules[] = {
    {"_frozen_importlib", _Py_M_importlib_bootstrap, ...},
    {"_frozen_importlib_external",
     _Py_M_importlib_bootstrap_external, ...},
    // ... additional frozen modules appended by freeze_modules.py
    {0, 0, 0}
};

// Embedding API:
PyImport_AppendInittab("my_extension", PyInit_my_extension); //
register C extension
Py_Initialize();
PyObject *mod =
PyImport_ImportModule("my_extension");           // import it
PyObject *pkg =
PyImport_ImportModule("frozen_pkg.mod");         // import frozen Python
module
```

At the Python level, `Tools/build/freeze_modules.py` compiles each `Lib/importlib/_bootstrap*.py` (and optionally additional modules via `--freeze-importlib / freezing configs`) to bytecode, marshals it, and emits C arrays in `Python/importlib.h`. The `FrozenImporter` then serves these modules from memory — no `stat`, no `open`, no `pyc` header check.

For backend engineers embedding CPython (e.g., a Go or Rust service that calls Python for ML inference via `PyImport_ImportModule`), freezing the import graph reduces startup I/O and eliminates `__pycache__` concerns for the frozen subset.

13. `pyc` invalidation — PEP 552 hash vs timestamp

Chapter 1 introduced the 16-byte `pyc` header; this section explains the invalidation policy that sits on top of it and why it matters for reproducible containers.

The header

Every `__pycache__/mod.cpython-311.pyc` starts with:

Bytes	Content
0-3	Magic number (<code>importlib.util.MAGIC_NUMBER</code>) – changes per feature release
4-7	Flags (PEP 552) – bitfield
8-15	Timestamp/hash + size/hash – interpretation depends on flags
16+	marshal'd <code>PyCodeObject</code>

```
import importlib.util, struct

print(f"magic: {importlib.util.MAGIC_NUMBER.hex()}")
# e.g. a60d0d0a – CPython 3.11
print(f"tag: {importlib.util._RAW_MAGIC_NUMBER}") # same, as int

# Inspect a real pyc header:
import pathlib, py_compile, tempfile, os
tmpdir = tempfile.mkdtemp()
src = pathlib.Path(tmpdir) / "demo.py"
src.write_text("x = 1\n")
py_compile.compile(str(src), cfile=str(pathlib.Path(tmpdir) / "demo.pyc"))
data = pathlib.Path(tmpdir, "demo.pyc").read_bytes()
magic, flags = data[:4], struct.unpack("<I", data[4:8])[0]
print(f"flags: {flags:#010x} (0=timestamp, 1=hash, 2=hash+check_source)")
print(f"header bytes 8-15: {data[8:16].hex()}")
```

PEP 552 — two invalidation modes

Mode	py_compile flag	
Timestamp (default)	invalidation_mode=PycInvalidationMode.TIMESTAMP	
Hash, check_source=True	invalidation_mode=PycInvalidationMode.CHECKED_HASH	
Hash, check_source=False	invalidation_mode=PycInvalidationMode.UNCHECKED_HASH	

The flags word encodes this:

```
flags & 0x01 == 0 → timestamp mode
flags & 0x01 == 1 → hash mode
flags & 0x02 == 1 → hash mode + check_source (re-read source on
import)
                    (only meaningful when bit 0 is 1)
flags & 0x02 == 0 → hash mode + unchecked (never read source)
```

Control surface:

```

import py_compile, importlib.util, pathlib, tempfile, os

tmpdir = tempfile.mkdtemp()
src = pathlib.Path(tmpdir) / "mod.py"
src.write_text("x = 1\n")

# Timestamp (default):
py_compile.compile(str(src), invalidation_mode=py_compile.PycInvalidati
onMode.TIMESTAMP)
# Hash, checked:
py_compile.compile(str(src), invalidation_mode=py_compile.PycInvalidati
onMode.CHECKED_HASH)
# Hash, unchecked:
py_compile.compile(str(src), invalidation_mode=py_compile.PycInvalidati
onMode.UNCHECKED_HASH)

# CLI:
# python -m py_compile --invalidation-mode=checked-hash mod.py
# python -m py_compile --invalidation-mode=unchecked-hash mod.py
# PYTHONPYCACHEPREFIX=/tmp/pyc python -X pycache_prefix=/tmp/pyc
app.py (relocate __pycache__)

# Interpreter flags:
# python --check-hash-based-pycs=always|never|default (controls
unchecked-hash validation)
# PYTHONDONTWRITEBYTECODE=1 (or -B) - suppress __pycache__ writes
entirely
# SOURCE_DATE_EPOCH=0 - clamp timestamp-mode mtime for reproducibility

```

How `SourceFileLoader` validates on import:

```

# Skeleton of SourceFileLoader._validate_pyc (in
_bootstrap_external.py):
def _validate_pyc(self, pyc_path, source_path, pyc_data):
    magic = pyc_data[:4]
    if magic != importlib.util.MAGIC_NUMBER:
        return False # magic mismatch → recompile (Python version
changed)
    flags = struct.unpack("<I", pyc_data[4:8])[0]
    if flags & 0x01 == 0:
        # Timestamp mode: compare mtime + size
        mtime, size = struct.unpack("<II", pyc_data[8:16])
        try:
            stat = os.stat(source_path)
        except OSError:
            return False # source missing → keep pyc? actually
_bootstrap_external # treats missing source as valid if pyc
exists (sourceless import)
        return int(stat.st_mtime) == mtime and stat.st_size == size
    else:
        if flags & 0x02:
            # Checked hash: hash source, compare
            source_hash = _hash_source(open(source_path, "rb").read())
            expected = pyc_data[8:16]
            return source_hash == expected
        else:
            # Unchecked hash: always valid (unless --check-hash-based-
pycs=always)
            return True

```

Backend guidance:

- **Reproducible images.** Use hash-based `pyc` (`PYTHONPYCACHEPREFIX` + `py_compile` with `CHECKED_HASH` or `UNCHECKED_HASH`) and set `SOURCE_DATE_EPOCH` if you must use timestamp mode. Timestamp-mode `pyc` files embed the build's wall-clock time, so two `docker` builds from the same commit produce different image digests.
- **`PYTHONDONTWRITEBYTECODE=1` in containers.** Suppresses `__pycache__` writes at runtime. Good for read-only filesystems and for avoiding write latency on first request. Pair with `python -m compileall --invalidation-mode=unchecked-hash` at image build time so `pyc` files are already present and no compilation happens at runtime.
- **`--check-hash-based-pycs` .** always forces validation even for unchecked-hash `pyc` (useful for detecting corrupted images); never skips validation even for checked-hash `pyc` (fastest, but stale `pyc`

will run). Default `default` respects the `pyc` header's `check_source` bit.

14. Tracing imports — `python -X importtime` and

`importlib.util.find_spec`

`python -X importtime` — import timing waterfall

CPython 3.7+ can emit per-module import timings to `stderr` when run with `-X importtime`:

```
python -X importtime -c "import json" 2>&1 | head -30
```

```
import time: self [us] | cumulative | imported package
import time:    123 |         123 |      _io
import time:     45 |         45 |    marshal
import time:     67 |         67 |    posix
import time:    234 |        312 | zipimport
import time:    156 |        156 |      _bootstrap
import time:    189 |        189 | _bootstrap_external
import time:    412 |        412 | _frozen_importlib
import time:    523 |        523 | _frozen_importlib_external
import time:     89 |         89 |      _imp
import time:    234 |       1234 | importlib
import time:     78 |         78 | _codecs
...
import time:    345 |       2345 | json
import time:    123 |        123 | json.decoder
import time:     89 |         89 | json.encoder
import time:    567 |       3035 | json # cumulative includes
children
```

How to read it:

- **self** — time spent executing that module's own `exec_module` (excluding children).
- **cumulative** — `self` + all transitive imports triggered while loading that module.
- **Indentation** — nesting depth; `json.decoder` is a child of `json` because `json/__init__.py` does `from .decoder import ...`.

For backend performance work:

```
# Full service import profile:
python -X importtime -c "import myapp.main" 2>&1 | sort -k3 -n | tail -
20
# Shows the 20 slowest self-time imports – candidates for lazy loading.

# Compare cold vs warm (with __pycache__ present vs absent):
rm -rf __pycache__ myapp/__pycache__
python -X importtime -c "import myapp.main" 2> /tmp/cold.txt
python -X importtime -c "import myapp.main" 2> /tmp/warm.txt
diff -u /tmp/cold.txt /tmp/warm.txt | head -40

# Verbose import tracing (shows every find_spec attempt, including
misses):
python -v -c "import json" 2>&1 | head -40
# import 'json' # < frozen importlib_external.SourceFileLoader ...>
# # trying /usr/lib/python3.11/json/__init__.cpython-311.pyc
# # code object from '/usr/lib/python3.11/json/__init__.py'
```

`importlib.util.find_spec` — the dry-run

`find_spec` answers "where would this import come from?" without executing it — safe to call on untrusted or not-yet-imported names:

```
import importlib.util, sys

# Dry-run – no code executes, no sys.modules entry created:
spec = importlib.util.find_spec("myapp.handlers")
if spec is None:
    print("myapp.handlers not found on sys.path")
    print(f"sys.path is: {sys.path[:3]}")
else:
    print(f"found: {spec.origin} via {spec.loader}")

# Submodule dry-run needs parent path – find_spec handles this:
spec = importlib.util.find_spec("os.path")
print(f"os.path origin: {spec.origin}") # /usr/lib/python3.11/
posixpath.py (on Linux)

# find_spec respects sys.meta_path – if a custom finder is installed,
# it will be consulted. To bypass meta_path and check only sys.path:
from importlib.machinery import PathFinder
spec2 = PathFinder.find_spec("json", path=None)
print(f"PathFinder-only: {spec2.origin if spec2 else None}")
```

Inspecting `sys.modules`

```
import sys, importlib, types

# Snapshot after startup:
print(f"total modules: {len(sys.modules)}")
# Group by origin type:
from collections import Counter
kinds = Counter()
for name, mod in sys.modules.items():
    spec = getattr(mod, "__spec__", None)
    if spec is None:
        kinds["no_spec"] += 1
    elif spec.origin == "built-in":
        kinds["built-in"] += 1
    elif spec.origin == "frozen":
        kinds["frozen"] += 1
    elif spec.origin is None:
        kinds["namespace"] += 1
    elif spec.origin.endswith(".so"):
        kinds["extension"] += 1
    else:
        kinds["source"] += 1
print(kinds)
# Counter({'source': 180, 'extension': 15, 'built-in': 30, 'frozen':
2, ...})

# Find stale or surprising entries:
for name, mod in sorted(sys.modules.items()):
    spec = getattr(mod, "__spec__", None)
    if spec and spec.origin and "site-packages" in spec.origin:
        print(f"{name:40s} {spec.origin}")

# sys.modules is mutable – but be careful:
# Deleting an entry does NOT unload the module's objects that other
# modules already hold references to. It only forces reimport on next
`import`.
import json
old_json = sys.modules["json"]
del sys.modules["json"]
import json as json2
print(f"reimported: {json2 is old_json}") # False – new module object
# But: old_json still exists if anything held a reference (e.g.,
json.decoder did).
print(f"old still alive: {old_json is not None}")

# Restore:
sys.modules["json"] = old_json
```

15. Custom finder and loader — a complete example

This section builds a minimal but correct `MetaPathFinder` + `Loader` that serves modules from an in-memory dictionary. It demonstrates the full protocol: `find_spec` → `ModuleSpec` → `create_module` → `exec_module` → `sys.modules`, plus the `importlib` helpers that make it ergonomic.

```

"""
memimport – a MetaPathFinder that serves modules from a dict.

Use cases: synthetic modules for tests, code generation,
           plugin systems, or embedding DSLs that compile to Python.
"""

import importlib.abc
import importlib.machinery
import importlib.util
import sys
import types

# In-memory source store: fullname → source string.
# In a real system this might be a database, a network fetch, or a code
# generator.
MEMORY_SOURCES: dict[str, str] = {
    "memapp": "value = 'memapp package'\n",
    "memapp.config": "DEBUG = True\nVERSION = '1.0'\n",
    "memapp.utils": "def greet(name): return f'hello\n",
    {name}'\n",
    "memapp.utils.helpers": "def shout(s): return s.upper() + '!\n",
}

class MemoryLoader(importlib.abc.Loader):
    """Loader that compiles source from MEMORY_SOURCES."""

    def __init__(self, fullname: str, is_package: bool):
        self.fullname = fullname
        self.is_package = is_package

    def create_module(self, spec):
        # Return None → default module creation (types.ModuleType).
        # Override to return a custom subclass if needed
        # (e.g., a module that auto-populates attributes).
        return None

    def exec_module(self, module):
        # Called with module already in sys.modules, with __spec__ set.
        # Must populate module.__dict__.
        assert module.__spec__ is not None
        assert module.__spec__.name == self.fullname

        source = MEMORY_SOURCES[self.fullname]
        # Use the fullname as "filename" so tracebacks are readable:
        code = compile(source, f"<memimport:{self.fullname}>", "exec")
        exec(code, module.__dict__)

        # For packages, __path__ and __package__ are already set by

```

```

_bootstrap
    # from spec.submodule_search_locations / spec.parent. Verify:
    if self.is_package:
        assert module.__path__ is not None
        assert module.__package__ == self.fullname

class MemoryFinder(importlib.abc.MetaPathFinder):
    """MetaPathFinder for the memapp.* namespace."""

    def find_spec(self, fullname, path, target=None):
        # Only handle our namespace – return None for everything else
        # so the next finder on sys.meta_path gets a chance.
        if fullname not in MEMORY_SOURCES:
            # But also handle intermediate packages that are implied:
            # e.g., "memapp.utils" is in MEMORY_SOURCES, but "memapp"
            # must also resolve. Check if any key starts with fullname
            is_parent = any(k.startswith(fullname + ".") for k in MEMORY_SOURCES)

            if not is_parent:
                return None
            # Synthesize a namespace-like package for intermediate
            # parents
            # that have no explicit source but have children:
            if fullname not in MEMORY_SOURCES:
                # No source – treat as namespace package (no loader, no origin)
                return importlib.machinery.ModuleSpec(
                    name=fullname,
                    loader=None,
                    origin=None,
                    is_package=True,
                )

            # Note: submodule_search_locations will be set to [] by default
            # for is_package=True with loader=None – sufficient for PathFinder
            # to continue searching for submodules via other
            # finders.
            # For a pure-memory namespace, set it to a sentinel:
            # spec.submodule_search_locations = [f"<memimport:
            {fullname}>"]

            # Determine if this fullname is a package (has children):
            is_package = any(
                k != fullname and k.startswith(fullname + ".")
                for k in MEMORY_SOURCES
            )
            # Also: explicit packages that are in MEMORY_SOURCES and have
            children

```

```

# should be treated as packages. Our MEMORY_SOURCES entries for
# "memapp" and "memapp.utils" are packages in this sense.

loader = MemoryLoader(fullname, is_package=is_package)
spec = importlib.machinery.ModuleSpec(
    name=fullname,
    loader=loader,
    origin=f"<memimport:{fullname}>",
    is_package=is_package,
)
# For packages, submodule_search_locations must be non-None
# (even if it's a synthetic sentinel). _bootstrap uses it to
set __path__.
if is_package:
    spec.submodule_search_locations = [f"<memimport:{fullname}>"]

return spec

# Install the finder at the front of sys.meta_path:
finder = MemoryFinder()
sys.meta_path.insert(0, finder)

# Now import works normally:
import memapp
import memapp.config
import memapp.utils
import memapp.utils.helpers

print(memapp.value)                # memapp package
print(memapp.config.DEBUG)         # True
print(memapp.utils.greet("world")) # hello world
print(memapp.utils.helpers.shout("hello")) # HELLO!

# Introspection still works:
print(memapp.__spec__)              #
ModuleSpec(name='memapp', ...)
print(memapp.__loader__)            #
<_main_.MemoryLoader ...>
print(memapp.__path__)              # ['<memimport:memapp>']
print(importlib.util.find_spec("memapp.utils")) # finds via
MemoryFinder

# Relative imports inside memory modules also work – because
__package__
# is set correctly, `from . import helpers` inside memapp.utils would
resolve.
# To test, add a source with a relative import:
MEMORY_SOURCES["memapp.relative_demo"] = "from . import config; x =
config.DEBUG\n"
import memapp.relative_demo

```

```

print(memapp.relative_demo.x)                                # True

# Cleanup:
sys.meta_path.remove(finder)
for name in list(sys.modules):
    if name.startswith("memapp"):
        del sys.modules[name]

```

Extending this to a `PathEntryFinder` (for `sys.path` entries) follows the same loader but a different finder ABC:

```

import importlib.abc
import importlib.machinery
import os

class ArchiveFinder(importlib.abc.PathEntryFinder):
    """PathEntryFinder for a custom archive format on sys.path.

    sys.path_hooks entry: ArchiveFinder.path_hook
    sys.path entry: "/path/to/archive.myarc"
    """

    def __init__(self, path):
        self.path = path

    # Validate that `path` is our archive format; raise ImportError if not
    # so PathFinder tries the next hook.
    if not path.endswith(".myarc"):
        raise ImportError(f"not a .myarc archive: {path}")
    # ... parse archive index ...

    @classmethod
    def path_hook(cls, path):
        # Factory called by PathFinder for each sys.path entry.
        # Must raise ImportError if this hook doesn't handle `path`.
        return cls(path)

    def find_spec(self, fullname, target=None):
        # fullname is the fully-qualified name; no `path` arg (unlike
        # MetaPathFinder).
        # Check archive index for fullname, return ModuleSpec or None.
        return None # stub

# Register:
# sys.path_hooks.append(ArchiveFinder.path_hook)
# sys.path.append("/data/bundle.myarc")
# importlib.invalidate_caches()

```


Common mistakes when writing finders/loaders:

- **Forgetting `submodule_search_locations` for packages.** If `is_package=True` but `submodule_search_locations` stays `None`, `_bootstrap` will not set `__path__`, and `import pkg.sub` will fail with `ModuleNotFoundError` even though `import pkg` succeeded.
 - **Not handling `target` in `find_spec`.** `target` is the existing module object for `reload (importlib.reload)`. Most finders can ignore it, but loaders that use `create_module` should return `target` if it is not `None` and matches.
 - **Raising instead of returning `None`.** `find_spec` should return `None` for "not mine." Raising `ImportError` aborts the entire `sys.meta_path` walk and masks later finders.
-

16. Backend lens — lazy imports, import-time side effects, and reproducible images

Lazy imports for cold-start

Import cost is paid at first `import`, not at first use. For services where cold-start latency matters (Lambda, Cloud Run, Kubernetes scale-from-zero), deferring heavy imports until they are needed can cut startup time significantly.

Three patterns, in order of preference:

```

# 1. Function-local import – simplest, no extra machinery.
#   Cost: re-executes `import` on every call, but sys.modules makes it
#   a dict lookup.
def handle_request(req):
    import pandas as pd # only paid when this handler is actually
    called
    return pd.DataFrame(req.json()).to_csv()

# 2. importlib.util.LazyLoader – PEP 562 lazy loading via importlib.
#   The module object is created immediately (so `import foo`
#   succeeds),
#   but exec_module is deferred until first attribute access.
import importlib.util
spec = importlib.util.find_spec("pandas")
if spec and spec.loader:
    spec.loader = importlib.util.LazyLoader(spec.loader)
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    sys.modules["pandas"] = module
# Now `import pandas` returns a lazy module; `pandas.DataFrame`
# triggers the real load.

# 3. Manual lazy wrapper – explicit, debuggable, no importlib magic.
class LazyPandas:
    _mod = None
    def __getattr__(self, name):
        if self._mod is None:
            import pandas as _pd
            self._mod = _pd
        return getattr(self._mod, name)

pd = LazyPandas()
# pd.DataFrame triggers the import on first attribute access.

# Measuring the win:
# $ python -X importtime -c "import myapp.main" 2>&1 | grep -E "pandas|
# numpy|torch"
# Move the slowest cumulative-time entries behind lazy boundaries.

```

Trade-offs:

- Lazy imports hide errors until runtime. A missing dependency that would have failed at startup now fails on the first request that touches it. Mitigate with a startup health check that eagerly imports critical paths (`python -c "import myapp.main; myapp.main.smoke_test()"`) in CI or as a readiness probe.
- `LazyLoader` is not thread-safe for concurrent first access — two threads accessing the same lazy module simultaneously can trigger

double execution. Since Python 3.11, `LazyLoader` holds the import lock during deferred `exec_module`, but custom wrappers need explicit locking.

Import-time side effects — discipline

Code at module top-level runs under the import lock, before the application has configured logging, metrics, or dependency injection. Backend anti-patterns:

```

# BAD – side effects at import time:
# mymodule.py
import logging
logging.basicConfig(level=logging.DEBUG) # mutates global logging
config
import requests
session = requests.Session()           # opens connection pool at
import
session.get("https://config.internal/bootstrap") # network I/O at
import
print("mymodule loaded")                # stdout at import

# GOOD – side effects behind an explicit init:
# mymodule.py
import logging
logger = logging.getLogger(__name__)    # no config, just a logger
handle

_session = None

def get_session():
    global _session
    if _session is None:
        import requests
        _session = requests.Session()
    return _session

def init(*, config_url: str):
    """Call once from main() after logging/metrics are configured."""
    resp = get_session().get(config_url)
    resp.raise_for_status()
    return resp.json()

# main.py
def main():
    import logging
    logging.basicConfig(level=logging.INFO) # configure once, in main
    import mymodule
    mymodule.init(config_url="https://config.internal/bootstrap")

```

Rules of thumb:

- **Import should not do I/O.** No network, no disk writes, no subprocess, no `os.environ` mutation.
- **Import should not configure globals.** `logging.basicConfig`, `warnings.filterwarnings`, `os.chdir`, `sys.path` mutation, and signal handlers belong in `main()` or an explicit `init()`.

- **Import should be idempotent.** Re-importing (or `importlib.reload`) should not double-register handlers, double-create threads, or leak resources.

Reproducible container images without `__pycache__` writes

`__pycache__` writes at runtime are undesirable in containers: they require a writable filesystem, add latency to the first request that imports a new module, and produce non-deterministic `pyc` headers (timestamp mode).

Recommended production pattern:

```
# Dockerfile – reproducible, no runtime __pycache__ writes
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

# Pre-compile with hash-based pyc (deterministic, no mtime in header):
RUN python -m compileall --invalidation-mode=unchecked-hash -q /app \
    && python -m compileall --invalidation-mode=unchecked-hash -q /usr/
    local/lib/python3.11/site-packages

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /app /app
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/
    lib/python3.11/site-packages

# Suppress runtime writes; rely on pre-compiled pyc:
ENV PYTHONDONTWRITEBYTECODE=1
# Or equivalently: ENV PYTHONPYCACHEPREFIX=/tmp/pyc (if you want pyc
# but off the app volume)
# For hash-based pyc, also consider:
# ENV PYTHONHASHSEED=0 (only for hash randomization of dict ordering,
# not pyc – but related for determinism)

# Verify determinism:
# $ docker build -t myapp:(git rev-parse HEAD) .
# $ docker run --rm myapp python -c "import MyApp.main; print('ok')"
# $ docker build -t myapp:(git rev-parse HEAD) . # second build
# $ docker images --digests | grep myapp # digests should match if
SOURCE_DATE_EPOCH is set
```

If you must use `timestamp-mode pyc` (the default), set `SOURCE_DATE_EPOCH` to a fixed value (typically the commit timestamp) so `py_compile` clamps `mtime`:

```
export SOURCE_DATE_EPOCH=$(git log -1 --format=%ct)
python -m compileall -q /app # mtime in pyc header is now
deterministic
```

And for hermetic builds with `--check-hash-based-pycs`:

```
# At build time – force validation even for unchecked-hash pyc (catches
corruption):
python --check-hash-based-pycs=always -m compileall --invalidation-
mode=unchecked-hash -q /app

# At runtime – never validate (fastest, trusts the image):
python --check-hash-based-pycs=never app.py
```

Key takeaways

- **import is a protocol, not syntax.** `IMPORT_NAME` / `__import__` → `_gcd_import` (relative resolution) → `_find_and_load` (`sys.modules` check + `_ModuleLock` + `_find_spec` over `sys.meta_path` → `PathFinder` over `sys.path` / `sys.path_hooks` / `sys.path_importer_cache`) → `ModuleSpec` → `create_module` → `exec_module` → `sys.modules` insertion. Every step is observable and interceptable.
- **sys.meta_path is first-chance, sys.path_hooks is per-entry.** Custom `MetaPathFinder` s see every import; `PathEntryFinder` s see only imports routed to their `sys.path` entry. `PathFinder` bridges the two. Ordering in both lists determines precedence.
- **ModuleSpec is the contract.** Finders produce it, loaders consume it, modules carry it as `__spec__`. `origin`, `loader`, `has_location`, `parent`, `submodule_search_locations`, and `cached` each control a distinct aspect of module identity and `importlib.resources` / `__path__` / `__file__` behavior.
- **Loader.exec_module is the modern protocol; load_module is dead.** Loaders must implement `exec_module` (and optionally `create_module`). `SourceFileLoader`, `SourcelessFileLoader`,

`ExtensionFileLoader`, `ZipImporter`, `BuiltinImporter`, and `FrozenImporter` cover every concrete origin type.

- **`sys.modules` + import lock make import a cached, serialized service.** The per-module lock (`_imp.acquire_lock / _ModuleLock`) ensures `exec_module` runs exactly once per name; `sys.modules` insertion before `exec_module` enables (and complicates) circular imports. Deleting `sys.modules[name]` forces reimport but does not unload objects already referenced.
- **PEP 420 namespace packages have no `__init__.py`, no `origin`, no `loader`, and a `_NamespacePath` `__path__`.** `PathFinder` merges all `sys.path` entries where `pkg/` exists without `__init__.py`. A single stray `__init__.py` converts the namespace to a regular package and hides other contributions.
- **PEP 660 editable installs use a `MetaPathFinder` + `ModuleSpec` mapping, not `sys.path` mutation.** The finder is installed via `.pth` at startup and maps `my_pkg` → `/work/src/my_pkg`. `importlib.util.find_spec("my_pkg").origin` reveals whether you are in editable or installed mode.
- **PEP 552 hash vs timestamp controls `pyc` determinism.** Timestamp mode embeds `mtime + size`; hash mode embeds `hash(source)` with optional `check_source`. Use `unchecked-hash` + `PYTHONDONTWRITEBYTECODE=1` + `python -m compileall` at image build time for deterministic, writable-filesystem-free containers.
- **Trace with `-X importtime` and `find_spec`; defer with lazy imports.** `python -X importtime` is the primary cold-start profiler; `importlib.util.find_spec` is the safe dry-run. Lazy imports (`LazyLoader`, function-local imports, manual wrappers) cut cold start but hide errors — pair with startup smoke tests.
- **`_bootstrap` is frozen, `_bootstrap_external` owns the filesystem.** The split explains why `import` works before any file can be read, and why `SourceFileLoader` / `FileFinder` / `pyc` validation live in a separate file from the core protocol.

Further reading

- PEP 302 — *New Import Hooks* (2002, superseded in parts by PEP 451 but still the conceptual foundation for `sys.meta_path` /

- `sys.path_hooks` / `finders` / `loaders`). <https://peps.python.org/pep-0302/> **(pinned)**
- PEP 420 — *Implicit Namespace Packages* (2012, the `__init__.py`-less package spec and `__NamespacePath` semantics). <https://peps.python.org/pep-0420/> **(pinned)**
 - PEP 451 — *A ModuleSpec Type for the Import System* (2013, `ModuleSpec` , `find_spec` / `create_module` / `exec_module` , deprecation of `load_module`). <https://peps.python.org/pep-0451/> **(pinned)**
 - PEP 552 — *Deterministic pycs* (2017, hash-based `pyc` invalidation, `SOURCE_DATE_EPOCH` , `check_source`). <https://peps.python.org/pep-0552/> **(pinned)**
 - *The import system* — Python Language Reference / `docs.python.org/3/reference/import.html` (authoritative, covers `sys.meta_path` , `sys.path` , `ModuleSpec` , namespace packages, and the full search order). <https://docs.python.org/3/reference/import.html> **(pinned)**
 - `importlib` — Python Standard Library documentation (`importlib.abc` , `importlib.machinery` , `importlib.util` , `importlib.resources`). <https://docs.python.org/3/library/importlib.html> **(pinned)**
 - PEP 328 — *Imports: Multi-Line and Absolute/Relative* (2003, relative import semantics and `level` / `package` resolution). <https://peps.python.org/pep-0328/>
 - PEP 660 — *Editable installs* (2021, `build_editable` , `direct_url.json` , finder-based editable mapping). <https://peps.python.org/pep-0660/>
 - PEP 617 — *New PEG Parser* (2020, background for `Grammar/` `python.gram` and `pegen` — the parser that `compile()` uses when `SourceFileLoader` compiles source). <https://peps.python.org/pep-0617/>
 - `Python/import.c` and `Lib/importlib/_bootstrap.py` / `Lib/importlib/_bootstrap_external.py` — the actual implementation (frozen via `Tools/build/freeze_modules.py` → `Python/importlib.h`). Reading `_find_and_load` , `_find_spec` , `_load_unlocked` , and `FileFinder.find_spec` alongside this chapter is the fastest way to close any remaining gaps. https://github.com/python/cpython/blob/main/Lib/importlib/_bootstrap.py

- Brett Cannon — "*Import system changes*" series and "*A history of Python's import system*" (blog posts by the import system's maintainer, covering the PEP 302 → 451 evolution and `_bootstrap` freezing). <https://snarky.ca/tag/import-system/>

Chapter 10 — C Extensions, the C API, HPy, and Embedding CPython

What this chapter covers. Almost every Python backend you run is part C. `numpy`, `cryptography`, `uvloop`, `orjson`, `psycopy`, `Pillow`, `lxml`, `grpcio` — the hot path of your service is a Python call that bottoms out in compiled C (or C++ or Rust via C) running inside the CPython process. That boundary — the C API — is one of the most powerful and most foot-gun-laden surfaces in the CPython codebase: a single missing `Py_INCREF` is a use-after-free, a single leaked reference is memory that never returns, a single mishandled `PyErr_*` is a silent wrong result in production. This chapter opens the boundary from both sides. You will write a real extension module with a custom type, parse arguments, manage reference ownership correctly, release the GIL around blocking I/O, expose zero-copy buffers to `numpy / memoryview`, build `abi3` stable-ABI wheels, understand the HPy alternative that tries to fix the API's deepest flaws, and embed CPython inside a C program — the pattern behind `nginx` + Python, game engines, and Postgres `plpython`.

Learning goals — after this chapter you should be able to:

- State the single invariant of the C API — reference ownership — and classify every API call as returning a *new* (owned) or *borrowed* reference, applying `Py_INCREF` / `Py_DECREF` / `Py_XDECREF` correctly.
- Define a module with `PyModuleDef` and `PyMethodDef` (including `METH_*` flags and the `vectorcall` fast path), and define a type with `PyType_Spec` / `PyType_Slot` + `PyType_FromSpec` rather than the legacy static `PyTypeObject`.
- Parse positional and keyword arguments with `PyArg_ParseTuple`, `PyArg_ParseTupleAndKeywords`, and the internal `_PyArg_Parse` clinic parser, including format units and converter functions.

- Implement correct error handling: set an exception with `PyErr_SetString / PyErr_Format`, return `NULL` (or `-1 / -1.0` for non-pointer returns), and check every fallible API call.
- Release the GIL around blocking work with `Py_BEGIN_ALLOW_THREADS / Py_END_ALLOW_THREADS`, reason about what is forbidden while released, and reacquire it before touching any `PyObject*`.
- Implement the buffer protocol (`Py_buffer`, `bf_getbuffer / bf_releasebuffer`, `PyObject_GetBuffer`, `memoryview`, flags like `PyBUF_SIMPLE / PyBUF_WRITABLE / PyBUF_ND`) and explain zero-copy exchange with `numpy`.
- Explain the Limited API (`Py_LIMITED_API / Py_LIMITED_API 0x030A0000`) and Stable ABI (`abi3` wheels, PEP 384 / PEP 3149 SOABI), when it applies, what it forbids (direct struct access, `Py_SIZE`, `Py_TYPE` as lvalue), and how to build `abi3` wheels.
- Contrast HPy (handles vs. raw `PyObject*`, universal ABI, no borrowed references, no direct struct poke) with the classic C API and sketch a migration path.
- Embed CPython in a C host: `Py_InitializeEx / Py_InitializeFromConfig`, `PyRun_SimpleString / PyRun_String`, `PyImport_ImportModule`, subinterpreter embedding (`Py_NewInterpreter`, PEP 684 per-interpreter GIL), and the `pymain / Py_RunMain` startup path — and debug segfaults with `faulthandler` and `gdb`'s `py-bt`.

Prerequisites. Chapter 2 (`PyObject`, `ob_refcnt`, `Py_INCREF / Py_DECREF`, immortal objects) and Chapter 6 (`PyTypeObject`, type slots, descriptors, `PyType_FromSpec`) are direct prerequisites. Chapter 5 (GIL mechanics, free-threaded Python, per-interpreter GIL) and Chapter 9 (import system, `ExtensionFileLoader`, `PyImport_AppendInittab`) are strongly recommended. Volume 2, Chapter 4 (allocators) and Volume 13, Chapter 8 (FFI/Polyglot) provide background.

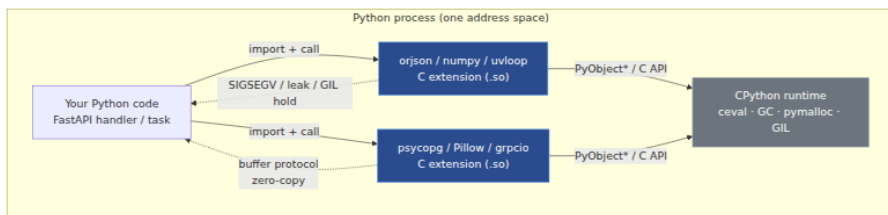
1. Why the C boundary is a backend concern

Your service already runs C extensions even if you never wrote one. The dependency that made your JSON serialization 20× faster, your image thumbnailer possible, or your database driver non-blocking is a `.so` loaded by `import` into the same address space as your Python code,

sharing the same heap, the same GIL, and the same crash domain. Understanding that boundary is not optional for senior backend engineers.

Concrete reasons:

- **Performance cliffs live at the boundary.** `orjson` dumps JSON 5–10× faster than `json` because the entire serialize loop is C that never allocates a Python object per field. `numpy` vectorizes a loop that would be thousands of Python bytecodes into one C loop over a contiguous buffer. `uvloop` replaces `asyncio`'s Python event loop with `libuv` in C. If you cannot read the C API you cannot reason about where those speedups come from or when they evaporate (e.g., per-element Python callbacks inside a `numpy.vectorize` call).
- **Crashes are process crashes.** A segfault in a C extension kills the whole worker — not one request. There is no per-request isolation; `SIGSEGV` in `Pillow`'s JPEG decoder takes down your Gunicorn worker and whatever requests were in flight. `faulthandler` and `gdb py-bt` (covered in §14) are the only tools that give you a Python traceback from a C crash.
- **Memory bugs are security bugs.** A borrowed-reference misuse is a use-after-free that is exploitable across the process boundary. The C API's ownership rules (§2) are the memory-safety contract. Every backend team that vendors a C extension accepts this contract.
- **Packaging is an ABI problem.** Whether your wheel is `cp312-cp312-linux_x86_64` (CPython-version-locked) or `cp312-abi3-linux_x86_64` (Stable ABI, runs on 3.12+) determines whether you ship one wheel or six, whether your CI matrix explodes, and whether your users on Python 3.13 hit an `ImportError` on day one. §9 makes the tradeoff quantitative.
- **Embedding is architecture.** Postgres `plpython3u`, Redis modules, Nginx `ngx_python`, game servers, and custom sidecars all embed CPython as a library. Embedding (§12) turns CPython from "the process" into a component you initialize, configure, and tear down — with subinterpreters (PEP 684) as the isolation primitive.



2. C API basics — `PyObject*` and the one rule

2.1 Everything is a `PyObject*`, but now you manage the count

Chapter 2 showed `PyObject` — `ob_refcnt` + `ob_type` — as the prefix every value carries. From C you never hold a value, you hold a pointer to it, and you are responsible for its lifetime.

```
/* Include/object.h — what you actually touch in an extension */
typedef struct _object {
    Py_ssize_t ob_refcnt; // not opaque in the full API; opaque in
Limited API
    PyTypeObject *ob_type; // same
} PyObject;

// The only safe way to interact with ob_refcnt:
Py_INCREF(op); // ++op->ob_refcnt (with immortal-object check since
3.12)
Py_DECREF(op); // --op->ob_refcnt; if 0 → tp_dealloc
Py_XINCREF(op); // NULL-safe INCREASE
Py_XDECREF(op); // NULL-safe DECREASE
Py_CLEAR(op); // Py_XDECREF(op); op = NULL; — prevents double-
DECREF
```

In Python, `x = y` adjusts counts implicitly. In C, every assignment, every return, every stash in a struct is a manual decision. The compiler will not help you.

2.2 The one rule: new reference vs. borrowed reference

Every C API function that returns or hands you a `PyObject*` does one of two things — and the docs say which, but the compiler does not enforce it:

Contract	Meaning	Who must Py_DECREF ?	Examples
New (owned) reference	Callee did <code>Py_INCREF</code> before returning; count is +1 for you. You own it.	You — <code>Py_DECREF</code> when done, or return it (transfer ownership to caller).	<code>PyLong_FromLong</code> , <code>PyUnicode_FromString</code> , <code>PyList_New</code> , <code>PyObject_Call</code> , <code>PyDict_GetItemWithError</code> is <i>not</i> — see below
Borrowed reference	Callee did not <code>Py_INCREF</code> ; pointer is valid only as long as the owner keeps it alive. You do not own it.	Nobody — do not <code>Py_DECREF</code> unless you first <code>Py_INCREF</code> 'd to keep it.	<code>PyList_GetItem</code> , <code>PyTuple_GetItem</code> , <code>PyDict_GetItem</code> , <code>PyImport_GetModule</code> , <code>PyErr_Occurred</code> (borrowed), <code>Py_TYPE(obj)</code>

If you get this wrong in either direction:

- Treating borrowed as owned and `Py_DECREF` 'ing it → **premature free** → use-after-free → segfault or heap corruption (often far from the bug).
- Treating owned as borrowed and failing to `Py_DECREF` → **leak** → RSS grows until OOM-killer intervenes.

The canonical trap:

```
PyObject *item = PyList_GetItem(list, 0); // BORROWED – do NOT decref
// WRONG: Py_DECREF(item);

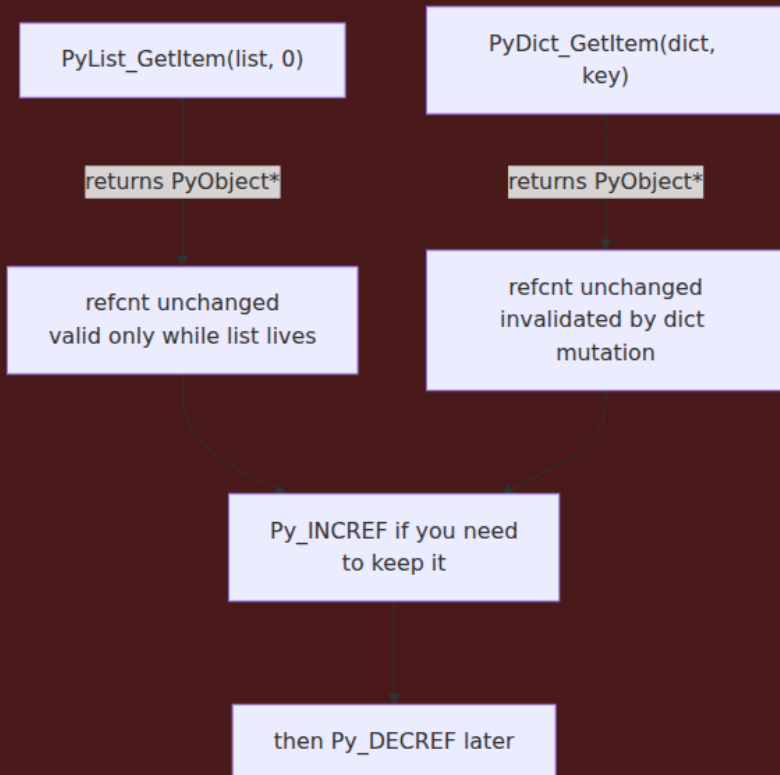
PyObject *item2 = PySequence_GetItem(list, 0); // NEW ref – you MUST decref
// ... use item2 ...
Py_DECREF(item2);

// If you need to keep a borrowed reference past the next Python call:
PyObject *kept = PyList_GetItem(list, 0); // borrowed
Py_INCREF(kept); // now you own one count – remember to Py_DECREF(kept) later
```

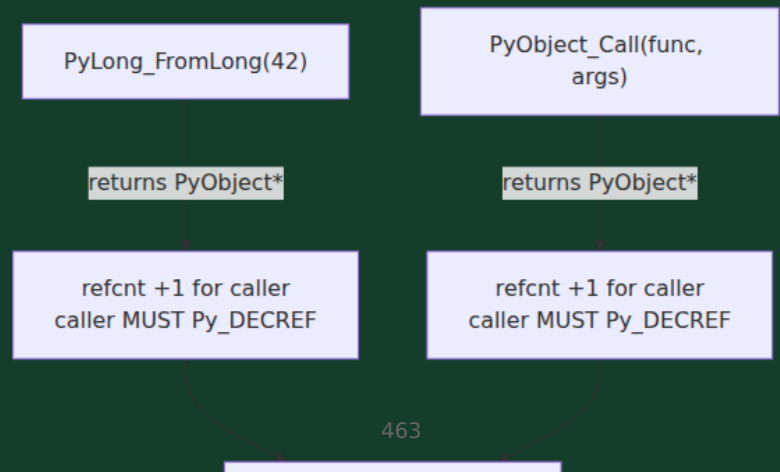
Borrowed references are invalidated by any call that might mutate the owner. `PyList_GetItem(list, 0)` is borrowed from `list`. If you then

do `PyList_Append(list, x)`, the list may reallocate `ob_item` and the pointer you borrowed may dangle. Rule: `Py_INCREF` immediately if you will do anything that might touch the owner.

Borrowed reference — NOT INCREF'd



New (owned) reference — callee INCREF'd



2.3 Stealing references — the third contract

A few APIs **steal** a reference: you pass an owned reference and the callee takes ownership, so you must *not* `Py_DECREF` even though you previously owned it.

```
PyObject *key = PyUnicode_FromString("hello"); // owned, +1
PyObject *val = PyLong_FromLong(42);           // owned, +1

// PyTuple_SetItem STEALS – do NOT decref key/val after this, even on
// failure
PyObject *tuple = PyTuple_New(2);              // owned, +1 (elements
NULL)
PyTuple_SetItem(tuple, 0, key); // steals key – tuple now owns it
PyTuple_SetItem(tuple, 1, val); // steals val

// PyList_SetItem also steals. PyList_Append does NOT – it INCREFs.
// PyModule_AddObject steals (deprecated alias: PyModule_AddObjectRef
// in 3.10+ does NOT steal – prefer it).
```

Stealing APIs are being phased out precisely because they are error-prone. Modern code prefers non-stealing variants (`PyTuple_SetItem` remains, but `PyModule_AddObjectRef` / `PyDict_SetItem` are preferred over stealing alternatives).

2.4 Reference debugging — what to do when it goes wrong

- Compile with `./configure --with-pydebug` or at least `-DPy_REF_DEBUG` to enable `sys.gettotalrefcount()` and `PYTHONMALLOC=debug`.
- Use `python -X showrefcount -X tracemalloc` to compare counts across runs.
- `python -X gil=0` is irrelevant here; refcount bugs are the same under free-threaded builds but may manifest as races (see Chapter 5).
- AddressSanitizer: `CFLAGS="-fsanitize=address" ./configure --with-address-sanitizer` catches use-after-free from premature `Py_DECREF` — the single most valuable tool for extension authors.

3. Module creation — `PyModuleDef`, methods table, and `ML` flags

3.1 The minimal module — `PyModuleDef` + `PyMethodDef`

A C extension module is a shared library (`.so` / `.pyd`) that exports one function: `PyInit_<modulename>`. The import system (`ExtensionFileLoader` from Chapter 9) `dlopen`s it and calls that symbol.

```

/* mymod.c – minimal extension module */
#define PY_SSIZE_T_CLEAN
#include <Python.h>

// --- 1. The C function that backs Python's mymod.hello(name) ---
static PyObject *
mymod_hello(PyObject *self, PyObject *args)
{
    const char *name;
    // PyArg_ParseTuple: "s" = str as UTF-8 char* (borrowed from
    PyUnicode)
    if (!PyArg_ParseTuple(args, "s", &name))
        return NULL; // exception already set, propagate NULL
    return PyUnicode_FromFormat("hello, %s!", name); // new ref –
    caller owns it
}

// --- 2. Methods table – every entry maps a Python name to a C
function ---
static PyMethodDef mymod_methods[] = {
    {"hello", mymod_hello, METH_VARARGS,
     "hello(name) -> str\n\nReturn a greeting."},
    // METH_VARARGS → PyCFunction: (PyObject *self, PyObject *args)
    // self is the module object for module-level functions
    {NULL, NULL, 0, NULL} // sentinel
};

// --- 3. Module definition ---
static struct PyModuleDef mymod_def = {
    PyModuleDef_HEAD_INIT, // always this – sets ob_base correctly
    "mymod", // m_name – import name (must match
    PyInit_mymod)
    "Example extension – hello + custom type.", // m_doc
    -1, // m_size – per-module state (-1 = no
    state, uses globals)
    // for subinterpreter-safe modules use
    m_size >= 0, see §3.3
    mymod_methods, // m_methods
    NULL, // m_slots – for Py_mod_create /
    Py_mod_exec (PEP 489)
    NULL, // m_traverse – GC traversal if m_size >= 0
    NULL, // m_clear
    NULL, // m_free
};

// --- 4. Entry point – name MUST be PyInit_<m_name> ---
PyMODINIT_FUNC
PyInit_mymod(void)
{
    return PyModule_Create(&mymod_def); // new ref – import system
}

```

```
owns it
}
```

Build and test (details in §11):

```
python -c "import mymod; print(mymod.hello('world'))"
# hello, world!
```

3.2 METH_* flags — the calling convention you choose

`PyMethodDef.ml_flags` selects the C signature the interpreter will use. Getting this wrong is a calling-convention mismatch and immediate UB/crash.

Flag	C signature	Python call shape	Notes
<code>METH_VARARGS</code>	<code>PyObject *f(PyObject *self, PyObject *args)</code>	<code>f(*args)</code> — <code>args</code> is a tuple	Classic; needs <code>PyArg_ParseTuple</code>
<code>METH_VARARGS METH_KEYWORDS</code>	<code>PyObject *f(PyObject *self, PyObject *args, PyObject *kw)</code>	<code>f(*args, **kw)</code>	<code>kw</code> may be <code>NULL</code> ; use <code>PyArg_ParseTupleAndKey</code>
<code>METH_NOARGS</code>	<code>PyObject *f(PyObject *self, PyObject *unused)</code>	<code>f()</code>	Validates no args for you
<code>METH_O</code>	<code>PyObject *f(PyObject *self, PyObject *arg)</code>	<code>f(arg)</code>	Single arg, no tuple alloc

Flag	C signature	Python call shape	Notes
METH_FASTCALL	PyObject *f(PyObject *self, PyObject *const *args, Py_ssize_t nargs)	f(*args)	Preferred — no tuple alloc vectorcall-compatible
METH_FASTCALL \ METH_KEYWORDS	PyObject *f(PyObject *self, PyObject *const *args, Py_ssize_t nargs, PyObject *kwnames)	f(*args, **kw)	kwnames is tuple of keyword names or NULL
METH_METHOD	descriptor- aware variants	type.method(obj, ...)	Rare; for method descriptors

Since Python 3.7+, METH_FASTCALL is the recommended default for new code — it avoids allocating an argument tuple on every call and integrates with vectorcall (PEP 590, Chapter 7):

```
// METH_FASTCALL – no tuple allocation
static PyObject *
mymod_fast_hello(PyObject *self, PyObject *const *args, Py_ssize_t nargs)
{
    if (nargs != 1) {
        PyErr_SetString(PyExc_TypeError, "hello() takes exactly one
argument");
        return NULL;
    }
    if (!PyUnicode_Check(args[0])) {
        PyErr_SetString(PyExc_TypeError, "hello() argument must be
str");
        return NULL;
    }
    // PyUnicode_FromFormat handles the conversion
    return PyUnicode_FromFormat("hello, %U!", args[0]);
}

static PyMethodDef mymod_methods_fast[] = {
    {"hello", (PyCFunction)mymod_fast_hello, METH_FASTCALL, "hello(name
) -> str"},
    {NULL, NULL, 0, NULL}
};
```

Clinic note. CPython's own C modules rarely write `PyArg_ParseTuple` by hand anymore. They use *Argument Clinic* (`Tools/clinic/clinic.py`) — a DSL that generates the `_PyArg_Parse` and `fastcall` boilerplate plus `inspect.Signature` metadata. You will see `/*[clinic input]` blocks in `Modules/` and `Objects/`. For your own extensions, Clinic is optional but recommended if you want `__text_signature__` and fast argument parsing without hand-rolling format strings.

3.3 Multi-phase initialization (PEP 489) — why `m_size` and `m_slots` matter

The minimal example above uses *single-phase* `init` (`PyModule_Create`). Since PEP 489 (Python 3.5+), the import system supports *multi-phase* `init`, which is required for subinterpreter isolation (PEP 684, Chapter 5):

```

// Multi-phase: slots let import create the module object first, then
exec it
static int mymod_exec(PyObject *mod) {
    // Runs after the module object exists – safe to add types,
    constants
    // Return 0 on success, -1 on error (sets exception)
    if (PyModule_AddStringConstant(mod, "__version__", "1.0") < 0) return
    -1;
    return 0;
}

static PyModuleDef_Slot mymod_slots[] = {
    {Py_mod_exec, mymod_exec}, // PEP 489 exec slot
    // {Py_mod_create, custom_create}, // optional: custom module
    // object factory
    // {Py_mod_gil, Py_MOD_GIL_NOT_USED}, // PEP 684: declare GIL-free
    // (3.12+)
    {0, NULL}
};

static struct PyModuleDef mymod_def_isolated = {
    PyModuleDef_HEAD_INIT,
    "mymod",
    "Isolated extension – subinterpreter-safe.",
    0, // m_size >= 0 → per-module state via
    PyModule_GetState
    mymod_methods,
    mymod_slots, // ← multi-phase
    NULL, NULL, NULL,
};

```

Key differences:

- `m_size = -1` → module uses C globals. **Not subinterpreter-safe.** Two interpreters in the same process share the globals.
- `m_size >= 0` → each interpreter gets its own `PyModule_GetState(mod)` allocation, visited by GC via `m_traverse / m_clear`. **Required** for `Py_MOD_GIL_NOT_USED` and safe `Py_NewInterpreter` / per-interpreter GIL usage.
- `Py_mod_create` slot receives a `Py_mod_create` callback of type `PyObject* (*)(PyObject *spec, PyModuleDef *def)` — rarely needed.
- `Py_mod_gil` slot (Python 3.12+) declares whether the module needs the GIL. This is the mechanism behind free-threaded extensions.

4. Type creation in C — `PyType_Spec`, `PyType_Slot`, `PyType_FromSpec`

Chapter 6 dissected `PyTypeObject` — the 35-field struct with `tp_name`, `tp_basicsize`, `tp_dealloc`, `tp_methods`, etc. You *can* define a static `PyTypeObject` by hand (the legacy style still seen in older extensions), but since Python 3.2+ the modern API is `PyType_Spec` + `PyType_FromSpec`, which hides struct layout and is compatible with the Limited API.

4.1 Defining a type — the `Counter` example

Our extension will expose a `Counter` type: a heap type with one `long` field, a `__repr__`, an `increment` method, and GC support.

```

#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include <structmember.h> // PyMemberDef

// --- Instance struct - PyObject_HEAD + your fields ---
typedef struct {
    PyObject_HEAD           // ob_refcnt + ob_type (or
_PyObject_HEAD_EXTRA in debug)
    long count;             // our payload
    PyObject *label;        // owned ref - needs GC tracking
} CounterObject;

// --- tp_dealloc - destructor (called when refcnt → 0) ---
static void
Counter_dealloc(CounterObject *self)
{
    Py_CLEAR(self->label); // XDECREF + NULL - safe even if label is
NULL
    PyObject *tp = Py_TYPE(self);
    PyObject_GC_UnTrack(self); // if Py_TPFLAGS_HAVE_GC
    (*Py_TYPE(self)->tp_free)((PyObject *)self);
    Py_DECREF(tp); // PyType_FromSpec heap types: keep type alive
    until free
}

// --- tp_traverse / tp_clear - GC support for container types ---
static int
Counter_traverse(CounterObject *self, visitproc visit, void *arg)
{
    Py_VISIT(self->label);
    Py_VISIT(Py_TYPE(self)); // heap type: visit the type itself (3.9+)
    return 0;
}
static int
Counter_clear(CounterObject *self)
{
    Py_CLEAR(self->label);
    return 0;
}

// --- Methods ---
static PyObject *
Counter_increment(CounterObject *self, PyObject *const *args, Py_ssize_t nargs)
{
    long delta = 1;
    if (nargs == 1) {
        delta = PyLong_AsLong(args[0]);
        if (delta == -1 && PyErr_Occurred()) return NULL;
    } else if (nargs > 1) {

```



```

        PyErr_SetString(PyExc_TypeError, "increment() takes at most 1
argument");
        return NULL;
    }
    self->count += delta;
    Py_RETURN_NONE;
}

static PyMethodDef Counter_methods[] = {
    {"increment", (PyCFunction)Counter_increment, METH_FASTCALL, "incre
ment(delta=1)"},
    {NULL, NULL, 0, NULL}
};

static PyMemberDef Counter_members[] = {
    {"count", T_LONG, offsetof(CounterObject, count), 0, "current
count"},
    {"label", T_OBJECT_EX, offsetof(CounterObject, label), 0,
"optional label"},
    {NULL}
};

// --- Slots table - maps C slots to functions ---
static PyType_Slot Counter_slots[] = {
    {Py_tp_dealloc, Counter_dealloc},
    {Py_tp_traverse, Counter_traverse},
    {Py_tp_clear, Counter_clear},
    {Py_tp_methods, Counter_methods},
    {Py_tp_members, Counter_members},
    {Py_tp_doc, "Counter(count=0, label=None) - a simple C counter."},
    {0, NULL} // sentinel
};

static PyType_Spec Counter_spec = {
    "mymod.Counter", // tp_name - qualified name
    sizeof(CounterObject), // tp_basicsize
    0, // tp_itemsize (0 = fixed-size)
    Py_TPFLAGS_DEFAULT | Py_TPFLAGS_HAVE_GC | Py_TPFLAGS_BASETYPE,
    // DEFAULT = HAVE_STACKLESS etc.;
    HAVE_GC = participate in GC // BASETYPE = allow subclassing from
    Python
    Counter_slots
};

```

Wiring it into the module (single-phase init for clarity; multi-phase is analogous):

```

static struct PyModuleDef mymod_def = {
    PyModuleDef_HEAD_INIT, "mymod", "Example with Counter type.",
    -1, mymod_methods, NULL, NULL, NULL, NULL,
};

PyMODINIT_FUNC
PyInit_mymod(void)
{
    PyObject *mod = PyModule_Create(&mymod_def);
    if (!mod) return NULL;

    PyObject *counter_type = PyType_FromSpec(&Counter_spec); // new ref
    if (!counter_type) { Py_DECREF(mod); return NULL; }

    // PyModule_AddObjectRef does NOT steal – INCREFs internally
    (3.10+)
    // For <3.10, use PyModule_AddObject which DOES steal (then don't
    DECF on success)
    if (PyModule_AddObjectRef(mod, "Counter", counter_type) < 0) {
        Py_DECREF(counter_type);
        Py_DECREF(mod);
        return NULL;
    }
    Py_DECREF(counter_type); // AddObjectRef INCF'd, so we can drop
    ours
    return mod; // new ref – import system owns it
}

```

From Python:

```

import mymod
c = mymod.Counter()
c.increment(5)
print(c.count)    # 5
c.label = "requests"
print(repr(c))    # <mymod.Counter object ...> – add tp_repr for custom
repr

```

Why `Py_TPFLAGS_HEAPTYPE`? Types created via `PyType_FromSpec` are *heap types* (`HEAPTYPE` set automatically). Their `PyTypeObject` is heap-allocated, not static, so `Py_DECREF(tp)` in `tp_dealloc` is required — the instance holds a reference to its type. Static `PyTypeObject`s (legacy `PyType_Ready` style) are immortal and must not be `Py_DECREF`'d.

4.2 PyType_FromSpec vs. legacy PyTypeObject + PyType_Ready

	PyType_Spec + PyType_FromSpec (modern)	static PyTypeObject + PyType_Ready (legacy)
Struct access	Slots only — no direct <code>tp_*</code> poke	Full struct visible — any field assignable
Limited API	Compatible — hides layout	Incompatible — requires full API
Heap type	Always heap type	Static type (immortal) unless <code>Py_TPFLAGS_HEAPTYPE</code>
Boilerplate	Compact — only slots you need	Verbose — zero-init every unused slot
Subclassing	<code>Py_TPFLAGS_BASETYPE</code> in flags	Same flag in static struct

For new extensions, always use `PyType_Spec`. The legacy path exists for CPython's own `Objects/` types (which predate the Spec API) and for extensions that need to set exotic slots not yet exposed via `PyType_Slot`.

5. Argument parsing — `PyArg_ParseTuple`, `PyArg_ParseTupleAndKeywords`, and Clinic

5.1 Format strings — the `PyArg_ParseTuple` mini-language

`PyArg_ParseTuple` (and `PyArg_ParseTupleAndKeywords`) use a `printf`-like format string to coerce and validate arguments:

```

static PyObject *
mymod_add(PyObject *self, PyObject *args)
{
    long a, b;
    const char *label = NULL; // optional
    // "ll|s" → two required longs, one optional string
    // | marks the start of optional args
    if (!PyArg_ParseTuple(args, "ll|s", &a, &b, &label))
        return NULL; // TypeError already set
    long result = a + b;
    if (label)
        return PyUnicode_FromFormat("%s: %ld", label, result);
    return PyLong_FromLong(result);
}

```

Common format units (see `Doc/c-api/arg.rst` for the full table):

Unit	C type	Accepts	Notes
<code>b</code>	<code>unsigned char</code>	int 0..255	
<code>i</code>	<code>int</code>	int	Overflow → <code>OverflowError</code>
<code>l</code>	<code>long</code>	int	
<code>k</code>	<code>unsigned long</code>	int ≥ 0	
<code>L</code>	<code>long long</code>	int	
<code>n</code>	<code>Py_ssize_t</code>	int	Preferred for sizes
<code>s</code>	<code>const char*</code>	<code>str</code>	UTF-8, borrowed — valid only until next Python call
<code>y</code>	<code>const char*</code>	<code>bytes</code>	Raw bytes, no decode
<code>u</code>	<code>Py_UNICODE*</code>	<code>str</code>	Deprecated — use <code>U / s</code>
<code>O</code>	<code>PyObject*</code>	any	Borrowed reference
<code>O!</code>	<code>PyObject*</code>	checked type	<code>O!(&PyLong_Type, &obj)</code> — validates type
<code>O&</code>	converter	any	<code>O&(converter, &out)</code> — custom coercion

Unit	C type	Accepts	Notes
p	int	any	Truth value (PyObject_IsTrue)
w*	Py_buffer	buffer	Request writable buffer — must PyBuffer_Release
s*	Py_buffer	str / bytes	Encoded string as buffer

Optional args after `|`, keyword support via `PyArg_ParseTupleAndKeywords`:

```
static PyObject *
mymod_greet(PyObject *self, PyObject *args, PyObject *kw)
{
    const char *name = "world";
    int excited = 0;
    static char *kwlist[] = {"name", "excited", NULL};
    // "s|p" → optional str, optional predicate (truthy)
    if (!PyArg_ParseTupleAndKeywords(args, kw, "|sp", kwlist, &name, &excited))
        return NULL;
    if (excited)
        return PyUnicode_FromFormat("Hello, %s!!!", name);
    return PyUnicode_FromFormat("Hello, %s.", name);
}

static PyMethodDef mymod_methods[] = {
    {"greet", (PyCFunction)mymod_greet, METH_VARARGS | METH_KEYWORDS, "greet(name='world', excited=False)"},
    {NULL, NULL, 0, NULL}
};
```

5.2 The Clinic parser — `_PyArg_Parse` and generated fastcall

For performance-critical or public-API functions, CPython uses Argument Clinic. Clinic generates a `_PyArg_Parse` and a fastcall wrapper that avoids `PyArg_ParseTuple` overhead and provides `inspect.Signature` support:

```

/* clinic input (Tools/clinic/clinic.py) - not hand-written */
// [clinic input]
// mymod.greet
//     name: str = "world"
//     excited: bool = False
// [clinic start generated code]...
// clinic generates: mymod_greet_impl + _PyArg_Parser + wrapper

```

For your own extensions, Clinic is worthwhile when:

- You expose many functions and want `__text_signature__` for free.
- You need `METH_FASTCALL` without hand-rolling `PyArg_ParseStackAndKeywords`.
- You want positional-only / keyword-only enforcement matching Python semantics.

Otherwise, `PyArg_ParseTupleAndKeywords` with `METH_VARARGS` | `METH_KEYWORDS` is perfectly serviceable and far more common in third-party extensions.

6. Error handling — `PyErr_*` + `NULL` return

Python exceptions are C-level thread-local state: `tstate->current_exception` (Python 3.12+, previously `curexc_type/value/traceback`). The protocol is simple and absolute:

If a C function fails, it must set an exception and return an error sentinel. If it succeeds, it must not have an exception set.

Sentinels by return type:

Return type	Error sentinel	Success
<code>PyObject*</code>	<code>NULL</code>	Owned <code>PyObject*</code>
<code>int</code>	<code>-1</code>	<code>0</code> (or <code>>= 0</code> for sizes)
<code>long</code>	<code>-1 + PyErr_Occurred()</code> check	Any value
<code>Py_ssize_t</code>	<code>-1 + PyErr_Occurred()</code>	<code>>= 0</code>

Setting exceptions:

```

PyErr_SetString(PyExc_TypeError, "expected str, got int");
return NULL;

PyErr_Format(PyExc_ValueError, "count %ld out of range [0, %ld)",
count, limit);
return NULL;

// Wrap an existing exception with context (3.11+):
PyErr_Format(PyExc_RuntimeError, "while processing %R", obj); // %R =
repr

// Fetch/restore – for cleanup that must not clobber the exception:
PyObject *type, *value, *tb;
PyErr_Fetch(&type, &value, &tb); // clears current exception, gives
you owned refs
// ... cleanup that might itself set an exception ...
PyErr_Restore(type, value, tb); // restores – steals refs

// Normalize + chain (the C equivalent of `raise X from Y`):
PyErr_SetString(PyExc_RuntimeError, "outer");
PyErr_Fetch(&type, &value, &tb);
// ... PyErr_SetString inner ...
PyErr_Restore(type, value, tb); // simplified; real chaining uses
PyException_SetCause

// Check without clearing:
if (PyErr_Occurred()) { /* borrowed ref – do not DECF */ }

// Clear (rare – usually you propagate NULL instead):
PyErr_Clear();

```

Every fallible API call must be checked:

```

PyObject *result = PyLong_FromLong(value); // can fail (MemoryError)
if (!result) return NULL; // propagate – exception already set

PyObject *item = PyDict_GetItemWithError(dict, key); // new in 3.10 –
distinguishes miss vs error
if (!item) {
    if (PyErr_Occurred()) return NULL; // real error
    // else: key not found – handle miss
}

```

Borrowed-reference trap in error paths. `PyErr_Occurred()` returns a borrowed reference. If you need to keep it, `Py_INCREF` it — but usually you just test for `NULL` /non- `NULL` .

7. The GIL in C — `Py_BEGIN_ALLOW_THREADS` and friends

Chapter 5 covered the GIL from Python's perspective. From C, the GIL is a mutex you hold whenever you touch a `PyObject*` and release whenever you do work that does not need Python.

7.1 Releasing around blocking work

```
static PyObject *
mymod_fetch(PyObject *self, PyObject *args)
{
    const char *url;
    if (!PyArg_ParseTuple(args, "s", &url)) return NULL;

    char *response = NULL;
    size_t len = 0;

    // Release GIL – other Python threads can run while we block on I/O
    Py_BEGIN_ALLOW_THREADS
    // ===== NO PyObject* ACCESS HERE – no INCREf, no PyErr_*, no
    Python calls =====
    // Only pure-C / syscall work:
    response = http_get_blocking(url, &len); // e.g., libcurl
    easy_perform
    // ===== still no Python =====
    Py_END_ALLOW_THREADS
    // GIL reacquired – safe to touch Python again

    if (!response) {
        PyErr_SetString(PyExc_RuntimeError, "fetch failed");
        return NULL;
    }
    PyObject *result = PyBytes_FromStringAndSize(response,
len); // new ref
    free(response); // http_get_blocking used malloc
    return result;
}
```

What `Py_BEGIN_ALLOW_THREADS` expands to (simplified):

```
// Include/ceval.h – conceptual
#define Py_BEGIN_ALLOW_THREADS { \
    PyThreadState *_save = PyEval_SaveThread(); /* release GIL, save
tstate */ \
    /* ... blocking work ... */ \
    PyEval_RestoreThread(_save); /* reacquire GIL */ \
}
```

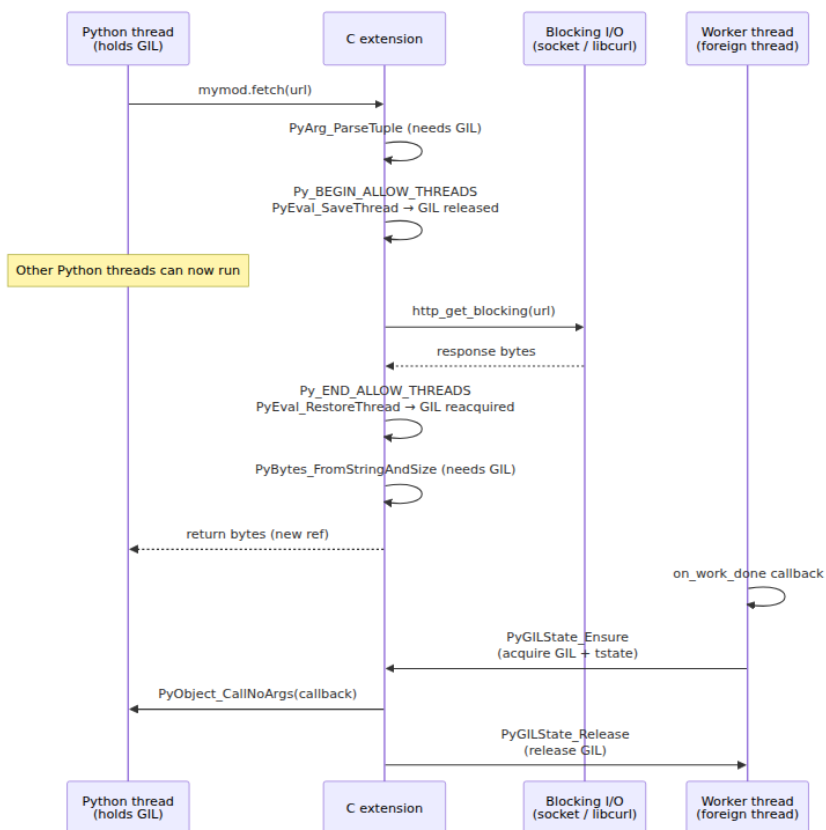

Rules while GIL is released:

- **Do not** read/write any `PyObject*` (`ob_refcnt` , `ob_type` , `Py_INCREF` , `PyList_GetItem`).
- **Do not** call any `PyErr_*` or `PyObject_*` API.
- **Do not** access `PyThreadState*` without reacquiring.
- **May** do syscalls, `malloc` / `free` , `compute`, `libcurl` , `read` / `write` , `compression`, `crypto`.
- Use `PyGILState_Ensure` / `PyGILState_Release` if you need to re-enter Python from a *non-Python thread* (e.g., a C callback on a worker thread — see §12.3).

7.2 `PyGILState_Ensure` — calling Python from foreign threads

If your extension spawns its own threads (or wraps a library that does), those threads have no `PyThreadState` . Before touching Python, they must acquire one:

```
// Called on a libuv / pthread worker thread – no GIL, no tstate
void on_work_done(void *userdata) {
    PyGILState_STATE gstate = PyGILState_Ensure(); // acquire GIL +
    ensure tstate
    PyObject *callback = (PyObject *)userdata;      // owned ref
    stashed earlier
    PyObject *result = PyObject_CallNoArgs(callback);
    if (!result) PyErr_Print(); // don't silently swallow – log it
    else Py_DECREF(result);
    PyGILState_Release(gstate); // release GIL, restore previous state
}
```



8. Buffer protocol — `Py_buffer`, `getbuffer`, `memoryview`, `zero-copy` with `numpy`

The buffer protocol is CPython's zero-copy exchange format. Any object that holds a contiguous (or strided) byte region can expose it as a `Py_buffer` so consumers (`memoryview`, `numpy`, `Pillow`, `bytearray`, `mmap`) can read/write it without copying.

8.1 Producer — exposing a buffer

```

typedef struct {
    PyObject_HEAD
    float *data;           // contiguous C array – the actual storage
    Py_ssize_t n;          // number of floats
} FloatArrayObject;

static int
FloatArray_getbuffer(FloatArrayObject *self, Py_buffer *view, int
flags)
{
    // Fill Py_buffer – the contract in Include/cpython/buffer.h
    view->obj = (PyObject *)self;
    Py_INCREF(self);           // view holds a ref to exporter –
// released in bf_releasebuffer
    view->buf = self->data;     // pointer to raw bytes
    view->len = self->n * sizeof(float);
    view->readonly = 0;        // writable
    view->itemsize = sizeof(float);
    view->format = (flags & PyBUF_FORMAT) ? "f" : NULL; // struct
// format char
    view->ndim = 1;
    view->shape = &self->n;    // careful: must stay alive until
// releasebuffer
                                // for heap shape, malloc it and
// free in releasebuffer
    view->strides = (Py_ssize_t[]){ sizeof(float) };
    view->suboffsets = NULL;
    view->internal = NULL;
    return 0; // 0 = success, -1 = failure (set exception)
}

static void
FloatArray_releasebuffer(FloatArrayObject *self, Py_buffer *view)
{
    // Drop the ref we INCREASED in getbuffer
    Py_DECREF(self);
}

static PyBufferProcs FloatArray_as_buffer = {
    .bf_getbuffer = (getbufferproc)FloatArray_getbuffer,
    .bf_releasebuffer = (releasebufferproc)FloatArray_releasebuffer,
};

// In PyType_Spec slots:
static PyType_Slot FloatArray_slots[] = {
    {Py_tp_doc, "FloatArray(n) – contiguous float buffer."},
    {Py_bf_getbuffer, FloatArray_getbuffer},
    {Py_bf_releasebuffer, FloatArray_releasebuffer},
    // ... dealloc, methods, etc.

```

```
{0, NULL}
};
```

Producer checklist (from `Doc/c-api/buffer.rst`):

- `view->obj` must be `INCR`EF 'd — `PyBuffer_Release` will `DECR`EF it.
- `shape / strides` pointers must remain valid until `bf_releasebuffer` — do not point at stack memory.
- Respect `flags`: if caller did not request `PyBUF_FORMAT`, `view->format` should be `NULL`.
- `bf_releasebuffer` is optional only for simple 1-D contiguous buffers with no heap shape.

8.2 Consumer — acquiring a buffer

```
static PyObject *
mymod_sum_buffer(PyObject *self, PyObject *obj)
{
    Py_buffer view;
    // PyBUF_SIMPLE = contiguous bytes, no format/strides needed
    // PyBUF_WRITABLE, PyBUF_FORMAT, PyBUF_ND are other flag bits
    if (PyObject_GetBuffer(obj, &view, PyBUF_SIMPLE) < 0)
        return NULL; // TypeError already set if obj doesn't export
    buffer

    // view.buf is valid until PyBuffer_Release
    // For a bytes-like object, view.len is bytes; cast as needed
    double sum = 0;
    // If the buffer is known to be floats, use PyBUF_FORMAT and check
    view.format
    // Here we sum raw bytes as example:
    for (Py_ssize_t i = 0; i < view.len; i++)
        sum += ((unsigned char *)view.buf)[i];

    PyBuffer_Release(&view); // DECREFS view.obj, invalidates view.buf
    return PyFloat_FromDouble(sum);
}
```

From Python:

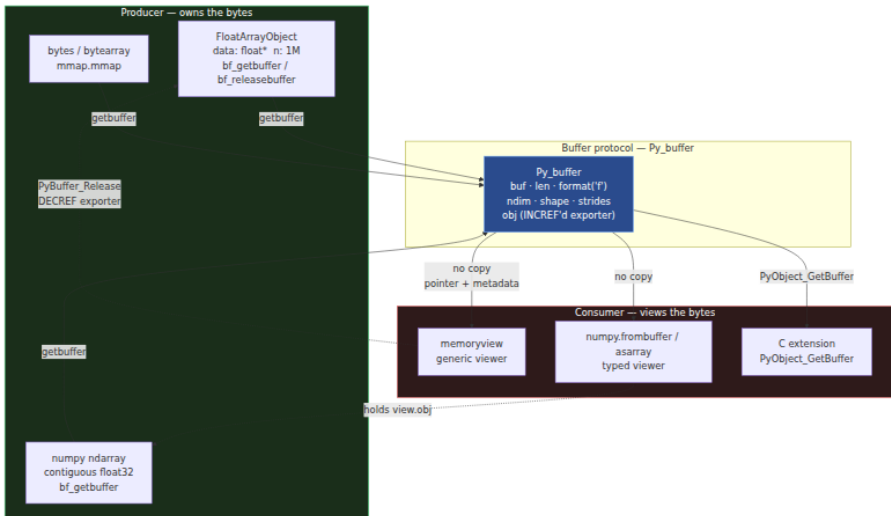
```

import numpy as np
import mymod

arr = np.array([1.0, 2.0, 3.0], dtype=np.float32)
# numpy exposes Py_buffer with format "f", ndim=1, shape=(3,)
mv = memoryview(arr)          # memoryview is the generic buffer
consumer — no copy
print(mv.format, mv.shape)    # f (3,)
print(mv[1])                  # 2.0 — direct access to numpy's storage

# Zero-copy round-trip — mymod.FloatArray exposes the same protocol:
fa = mymod.FloatArray(1_000_000)
mv2 = memoryview(fa)          # no copy — mv2 points at fa.data
np_arr = np.frombuffer(mv2, dtype=np.float32) # still no copy — numpy
views fa's memory
# WARNING: fa must stay alive as long as np_arr/mv2 exist (view.obj
keeps it alive)

```



Backend lens — when zero-copy matters. A service that decodes 10k images/s with Pillow and feeds them to an ML model via numpy must avoid per-image copies. Pillow's tobytes copies; memoryview + buffer protocol does not. The same applies to mmap'd model weights, bytearray network buffers, and array('f') feature vectors. Always check `mv.contiguous` / `PyBuffer_IsContiguous` before assuming a single memcpy suffices — non-contiguous (strided, Fortran-order) buffers require per-row copies or `PyBuffer_ToContiguous`.

9. Limited API and Stable ABI — `Py_LIMITED_API`, `abi3` wheels, PEP 384

9.1 The problem — every Python minor breaks your `.so`

A C extension compiled against `Python.h` on 3.11 has `PyLongObject` layout, `Py_TYPE` macro expansion, and `PyUnicodeObject` field offsets baked into its machine code. On 3.12, those layouts changed (immortal objects, PEP 683) — the same `.so` segfaults. So traditionally you ship one wheel per (implementation, version, arch) triple:

```
mymod-1.0-cp311-cp311-linux_x86_64.whl # CPython 3.11 only
mymod-1.0-cp312-cp312-linux_x86_64.whl # CPython 3.12 only
mymod-1.0-cp313-cp313-linux_x86_64.whl # CPython 3.13 only
```

For a backend team maintaining an internal extension, that is a CI matrix of 6–8 builds per platform.

9.2 The Stable ABI — a subset that promises forward compatibility

PEP 384 (Python 3.2) defines a *Stable ABI* (`abi3`): a subset of the C API whose ABI is guaranteed stable across Python 3.x minor versions. An extension that uses only that subset and is compiled with `Py_LIMITED_API` set can be built once on the *oldest* supported Python and run on every newer one:

```
mymod-1.0-cp312-abi3-linux_x86_64.whl # built on 3.12, runs on 3.12,
3.13, 3.14, ...
```

What you give up (the API surface you may not touch):

Diagram omitted for brevity — see surrounding prose.

Concretely, with `Py_LIMITED_API` defined:

- `PyObject` becomes opaque — `op->ob_refcnt` and `op->ob_type` are compile errors; use `Py_REFCNT(op)`, `Py_TYPE(op)`, `Py_SIZE(op)` as *functions*, and `Py_IS_TYPE` / `Py_SET_TYPE` where needed.
- `Py_TYPE(obj) = new_type` is forbidden — use `Py_SET_TYPE(obj, new_type)`.

- `PyList_GET_ITEM` / `PyTuple_GET_ITEM` macros that poke `ob_item` directly are unavailable — use `PyList_GetItem` / `PyTuple_GetItem`.
- `Include/cpython/` and `Include/internal/` headers are not includable — they are explicitly unstable.
- `PyLongObject`, `PyUnicodeObject`, `PyDictObject` layouts are hidden.

9.3 Building an `abi3` extension

`setup.py` / `pyproject.toml` — the two equivalent ways to declare an `abi3` build:

```
# setup.py – setuptools
from setuptools import setup, Extension

ext = Extension(
    "mymod",
    sources=["mymod.c"],
    # Request the Limited API at 3.10 – extension will run on 3.10+
    define_macros=[("Py_LIMITED_API", "0x030A0000")],
    py_limited_api=True, # tells bdist_wheel to tag the wheel as abi3
)

setup(
    name="mymod",
    version="1.0",
    ext_modules=[ext],
)
```

```
# pyproject.toml – setuptools backend (PEP 517/518)
[build-system]
requires = ["setuptools>=61", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "mymod"
version = "1.0"

[tool.setuptools.ext-modules]
# setuptools ≥64 supports declarative ext-modules; otherwise keep
# setup.py
```



```
# Build – must be on the OLDEST Python you claim to support (3.10 here)
python3.10 -m pip wheel . --no-deps -w dist/
# dist/mymod-1.0-cp310-abi3-linux_x86_64.whl – note abi3 tag

# Verify – abi3check or simple import on newer Python:
python3.12 -c "import mymod; print(mymod.__file__)"
python3.13 -c "import mymod; print(mymod.__doc__)"
```

CI pattern (GitHub Actions — build once, test on many):

```
# .github/workflows/wheels.yml
jobs:
  build-abi3:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.10" } # oldest supported
      - run: pip install build && python -m build -w
      - uses: actions/upload-artifact@v4
        with: { path: dist/*abi3*.whl }

  test-abi3:
    needs: build-abi3
    strategy:
      matrix: { python: ["3.10", "3.11", "3.12", "3.13"] }
    runs-on: ubuntu-latest
    steps:
      - uses: actions/setup-python@v5
        with: { python-version: "${{ matrix.python }}" }
      - run: pip install dist/*abi3*.whl && python -c "import mymod;
mymod.hello('ci')"
```

When not to use abi3. Extensions that need `PyLongObject` digit access, direct `ob_item` manipulation, `cpython/` headers (e.g., `PyUnicode_KIND`), or CPython-version-specific optimizations (PEP 659 specialization, free-threaded `Py_MOD_GIL_NOT_USED`) cannot use the Limited API. `numpy` does not use `abi3` for this reason. Most pure-wrapper extensions (calling into `libcurl`, `libjpeg`, `grpc`, `openssl`) can and should.

9.4 PEP 3149 SOABI and wheel tags

The `SOABI` (`sysconfig.get_config_var("SOABI")` → `cpython-312-x86_64-linux-gnu`) encodes `implementation-version-arch`. For full-API wheels,

`bdist_wheel` derives the tag from `SOABI`. For `abi3` wheels, the tag collapses to `cp310-abi3-<platform>` — `cp310` is the *minimum* version, `abi3` signals Stable ABI. The import system (`ExtensionFileLoader`) accepts any `abi3` extension on any `3.x ≥ 3.10`.

10. HPy — handles, universal ABI, and an alternative future

10.1 Why the C API needs an alternative

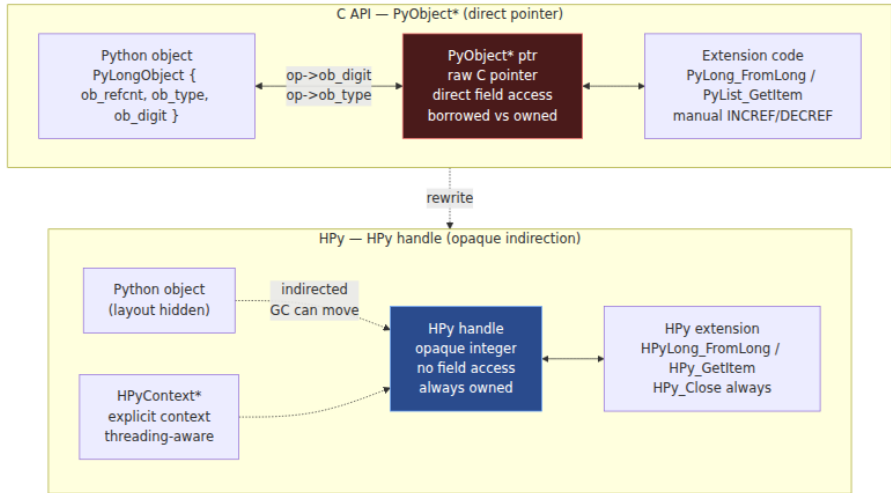
The C API's deepest flaws are not bugs but design choices from 1991 that no longer fit:

- **Raw `PyObject*` pointers** expose object layout, prevent moving GC, and make borrowed references possible. A moving GC (like PyPy's) cannot use the C API because C code holds raw pointers that would dangle after compaction.
- **Borrowed references** are a global source of use-after-free. Every `PyList_GetItem` caller must reason about owner mutation — and the compiler cannot check it.
- **Direct struct access** (`op->ob_refcnt`, `PyLongObject.ob_digit`) ties every extension to one CPython version's layout.
- **No binary portability** without the `abi3` subset — and `abi3` is restrictive.
- **GIL coupling** — raw pointer access requires the GIL; the API has no way to express "this handle is valid without the GIL."

HPy (<https://hpyproject.org>, PEP 630-adjacent) is a reimagined C API that fixes these at the abstraction level:

	C API (<code>PyObject*</code>)	HPy (<code>HPy handle</code>)
Value representation	Raw pointer to <code>PyObject</code> — direct field access	Opaque handle (<code>HPy</code> is an integer index, not a pointer) — must go through API
Reference model	New vs. borrowed — caller must know	All handles are owned — <code>HPy_Close</code> always required, no borrowed references

	C API (PyObject*)	HPy (HPy handle)
ABI	CPython-version-specific (unless <code>abi3</code> subset)	Universal ABI — one <code>.hpy.so</code> runs on CPython, PyPy, GraalPy without recompile
GC compatibility	Incompatible with moving GC	Compatible — handles are indirected, GC can move objects
Struct layout	Visible — <code>PyLongObject</code> etc. exposed	Hidden — no struct access, even in "full" mode
GIL interaction	Implicit — raw pointer implies GIL	Explicit — <code>HPyContext*</code> threading model



10.2 Minimal HPy module — compare with §3

```
/* hpy_mymod.c — HPy equivalent of mymod_hello */
#include <hpy.h>

HPyDef_METH(hpy_hello, "hello", HPyFunc_0, .doc="hello(name) -> str")
static HPy hpy_hello_impl(HPyContext *ctx, HPy self, HPy arg)
{
    // HPyUnicode_AsUTF8AndSize — explicit ctx, no borrowed refs
    HPy_ssize_t size;
    const char *name = HPyUnicode_AsUTF8AndSize(ctx, arg, &size);
    if (!name) return HPy_NULL; // exception already set

    // HPyUnicode_FromFormat — returns owned HPy handle
    return HPyUnicode_FromFormat(ctx, "hello, %s!", name);
    // Caller will HPy_Close the result; no INCR/DECREF, no borrowed
}

static HPyModuleDef hpy_mymod_def = {
    .doc = "HPy example — hello.",
    .size = 0,
    .defines = (HPyDef*[]){
        &hpy_hello,
        NULL
    }
};

HPy_MODINIT(hpy_mymod, hpy_mymod_def)
// No PyInit_* — HPy generates the entry point; import finds
hpy_mymod.hpy.so
```

Build with `hpy` tooling (from <https://hpyproject.org>):

```
# pyproject.toml — HPy build
[build-system]
requires = ["hpy"]
build-backend = "hpy.build"

[tool.hpy]
# hpy.abi = "universal" # one .hpy.so for CPython + PyPy + GraalPy
# hpy.abi = "cpython" # CPython-only, slightly faster
```

Key differences to internalize:

- Every `HPy` is owned — `HPy_Close(ctx, h)` is the only way to release it. There is no `HPy_INCR` / `HPy_DECR` distinction and no borrowed handles.

- `HPyContext *ctx` is threaded through every call — it carries the interpreter state, making subinterpreter and free-threaded usage explicit.
- `HPy_NULL` is the error sentinel (analogous to `NULL` for `PyObject*`), but the handle is not a pointer — comparing it to `NULL` is a type error.
- `HPy_MODINIT` replaces `PyInit_*`; the import system finds `*.hpy.so` via `hpy`'s loader hook.

10.3 Migration path and current status

HPy is production-usable (used by `hpy` itself, `numpy` experiments, and several small extensions) but not yet the default for the ecosystem. Practical guidance:

- **New extensions that value portability** (run on CPython + PyPy + GraalPy, or want a moving-GC future) should evaluate HPy. The universal ABI eliminates the `abi3` vs. full-API tradeoff entirely.
- **Existing C API extensions** can migrate incrementally: HPy provides `HPy_AsPyObject` / `HPy_FromPyObject` shims for mixed code, and `autogen` tooling to translate `PyMethodDef` tables.
- **Performance:** HPy's handle indirection has a small cost (one extra load per access) that is negligible for most extensions but measurable in tight loops over `PyObject*` arrays. The `hpy` project publishes microbenchmarks; measure your hot path.
- **Ecosystem inertia:** most large extensions (`numpy`, `cryptography`, `Pillow`) remain on the C API. HPy adoption is a multi-year transition. Knowing *both* APIs — and when to choose each — is the pragmatic stance.

11. Putting it together — complete `mymod` with module + type + method, and its build

The snippets in §3–§4 were intentionally fragmented for explanation. Here is the complete, buildable extension that combines everything — module, custom type, fastcall method, and correct reference management — in one translation unit. This is the artifact you would ship.

```

/* mymod.c - complete buildable extension (full C API, abi3-compatible)
*/
#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include <structmember.h>

/* ----- Counter type ----- */
typedef struct {
    PyObject_HEAD
    long count;
    PyObject *label; // owned ref
} CounterObject;

static void Counter_dealloc(CounterObject *self) {
    Py_CLEAR(self->label);
    PyTypeObject *tp = Py_TYPE(self);
    PyObject_GC_UnTrack(self);
    Py_TYPE(self)->tp_free((PyObject *)self);
    Py_DECREF(tp);
}

static int Counter_traverse(CounterObject *self, visitproc visit, void
*arg) {
    Py_VISIT(self->label);
    Py_VISIT(Py_TYPE(self));
    return 0;
}

static int Counter_clear(CounterObject *self) {
    Py_CLEAR(self->label);
    return 0;
}

static PyObject *Counter_increment(CounterObject *self, PyObject
*const *args, Py_ssize_t nargs) {
    long delta = 1;
    if (nargs == 1) {
        delta = PyLong_AsLong(args[0]);
        if (delta == -1 && PyErr_Occurred()) return NULL;
    } else if (nargs > 1) {
        PyErr_SetString(PyExc_TypeError, "increment() takes at most 1
argument");
        return NULL;
    }
    self->count += delta;
    Py_RETURN_NONE;
}

static PyObject *Counter_repr(CounterObject *self) {
    return PyUnicode_FromFormat("Counter(count=%ld, label=%R)", self->c
ount, self->label ? self->label : Py_None);
}

static PyMethodDef Counter_methods[] = {
    {"increment", (PyCFunction)Counter_increment, METH_FASTCALL, "incre

```

```

ment(delta=1) -> None"},
    {NULL, NULL, 0, NULL}
};

static PyMemberDef Counter_members[] = {
    {"count", T_LONG, offsetof(CounterObject, count), 0, "count"},
    {"label", T_OBJECT_EX, offsetof(CounterObject, label), 0, "label"},
    {NULL}
};

static PyType_Slot Counter_slots[] = {
    {Py_tp_dealloc, Counter_dealloc},
    {Py_tp_traverse, Counter_traverse},
    {Py_tp_clear, Counter_clear},
    {Py_tp_repr, Counter_repr},
    {Py_tp_methods, Counter_methods},
    {Py_tp_members, Counter_members},
    {Py_tp_doc, "Counter(count=0, label=None)"},
    {0, NULL}
};

static PyType_Spec Counter_spec = {
    "mymod.Counter", sizeof(CounterObject), 0,
    Py_TPFLAGS_DEFAULT | Py_TPFLAGS_HAVE_GC | Py_TPFLAGS_BASETYPE,
    Counter_slots
};

/* ----- Module-level function ----- */
static PyObject *mymod_hello(PyObject *self, PyObject *const *args, Py_
ssize_t nargs) {
    if (nargs != 1) {
        PyErr_SetString(PyExc_TypeError, "hello() takes exactly one
argument");
        return NULL;
    }
    if (!PyUnicode_Check(args[0])) {
        PyErr_SetString(PyExc_TypeError, "hello() argument must be
str");
        return NULL;
    }
    return PyUnicode_FromFormat("hello, %U!", args[0]);
}

static PyMethodDef mymod_methods[] = {
    {"hello", (PyCFunction)mymod_hello, METH_FASTCALL, "hello(name) ->
str"},
    {NULL, NULL, 0, NULL}
};

/* ----- Module def + init ----- */
static int mymod_exec(PyObject *mod) {
    PyObject *type = PyType_FromSpec(&Counter_spec);
    if (!type) return -1;
    int rc = PyModule_AddObjectRef(mod, "Counter", type);
    Py_DECREF(type);

```

```

    return rc;
}
static PyModuleDef_Slot mymod_slots[] = {
    {Py_mod_exec, mymod_exec},
    {0, NULL}
};
static struct PyModuleDef mymod_def = {
    PyModuleDef_HEAD_INIT, "mymod", "Complete example – hello +
Counter.",
    0, mymod_methods, mymod_slots, NULL, NULL, NULL,
};
PyMODINIT_FUNC PyInit_mymod(void) {
    return PyModuleDef_Init(&mymod_def);
}

```

Build files — all three variants:

```

# setup.py – full (version-locked) build
from setuptools import setup, Extension
setup(name="mymod", version="1.0",
      ext_modules=[Extension("mymod", sources=["mymod.c"])]))

# setup.py – abi3 (stable ABI) build – note Py_LIMITED_API +
# py_limited_api=True
from setuptools import setup, Extension
setup(name="mymod", version="1.0",
      ext_modules=[Extension("mymod", sources=["mymod.c"],
                             define_macros=[("Py_LIMITED_API", "0x030A0
000")],
                             py_limited_api=True)]))

```

```

# pyproject.toml – works with either setup.py above (PEP 517)
[build-system]
requires = ["setuptools>=61", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "mymod"
version = "1.0"
requires-python = ">=3.10"

# abi3 build via setup.py define_macros + py_limited_api=True
# Verify: pip wheel . && auditwheel show dist/*.whl | grep abi3

```



```
# Full build + test
python -m pip wheel . --no-deps -w dist/
python -c "import mymod; print(mymod.hello('world'));"
c=mymod.Counter(); c.increment(5); print(c)"

# abi3 build + verify on newer Python
python3.10 -m pip wheel . --no-deps -w dist/ # build on oldest
python3.12 -c "import mymod; print(mymod.hello('abi3'))" # run on
newer – should succeed
python -m pip install --no-index --find-links=dist mymod
# install the wheel
```

12. Embedding CPython — `Py_InitializeEx`, `PyRun_*`, subinterpreters, `pymain`

So far Python was the host and C was the guest (extension). Embedding inverts it: C/C++ is the host and CPython is a library you initialize, drive, and tear down. This is how Postgres, Blender, Nginx, and game engines host Python.

12.1 Minimal embedding — `Py_InitializeEx` → `PyRun_SimpleString` → `Py_FinalizeEx`

```
/* embed.c – minimal embedding (Python 3.8+ simple API) */
#define PY_SSIZE_T_CLEAN
#include <Python.h>

int main(int argc, char *argv[])
{
    // 1. Initialize – Py_InitializeEx(0) skips signal handlers (useful
    // when host owns signals)
    // Py_Initialize() is equivalent to Py_InitializeEx(1)
    Py_InitializeEx(0);
    if (!Py_IsInitialized()) return 1;

    // 2. Run Python code – simplest form
    PyRun_SimpleString("print('hello from embedded Python')");

    // 3. Import and call – the realistic pattern
    PyObject *mod =
    PyImport_ImportModule("mymod"); // new ref, NULL on failure
    if (mod) {
        PyObject *result = PyObject_CallMethod(mod, "hello", "s", "embed-
        ded");
        if (result) {
            // Convert to C string for host consumption
            const char *s = PyUnicode_AsUTF8(result); // borrowed –
            valid while result lives
            printf("mymod.hello returned: %s\n", s);
            Py_DECREF(result);
        } else {
            PyErr_Print(); // prints traceback to stderr – clears
            exception
        }
        Py_DECREF(mod);
    } else {
        PyErr_Print();
    }

    // 4. Tear down – Py_FinalizeEx returns 0 on success, -1 if
    // exception during finalization
    if (Py_FinalizeEx() < 0) return 120;
    return 0;
}
```

```
# Compile – use python3-config for correct flags (or pkg-config
python3-embed)
cc embed.c -o embed $(python3-config --cflags --ldflags --embed)
# --embed is required since 3.8 – links libpython correctly for
embedding
./embed
# hello from embedded Python
# mymod.hello returned: hello, embedded!
```

Linking note. `python3-config --ldflags` without `--embed` omits `-lpython3.x` on some distros (it assumes you are building an extension, not an embedder). Always use `--embed` for embedding. With `pkg-config`, use `pkg-config --cflags --libs python3-embed`.

12.2 Modern initialization — PyConfig (Python 3.8+)

`Py_InitializeEx` is the legacy entry point. For control over filesystem encoding, isolated mode, `PYTHONPATH` handling, and subinterpreter config, use `PyConfig` + `Py_InitializeFromConfig`:

```
PyConfig config;
PyConfig_InitPythonConfig(&config); // or InitIsolatedConfig for -I
semantics
config.install_signal_handlers = 0; // host owns SIGINT/SIGTERM
// config.program_name = Py_DecodeLocale(argv[0], NULL);
// config.pythonpath_env = NULL; // ignore PYTHONPATH – hermetic
embedder

PyStatus status = Py_InitializeFromConfig(&config);
if (PyStatus_Exception(status)) {
    PyConfig_Clear(&config);
    Py_ExitStatusException(status); // prints error + exits
}
PyConfig_Clear(&config);
// ... PyRun_* / PyImport_* ...
Py_FinalizeEx();
```

12.3 Subinterpreter embedding — isolation without processes

Chapter 5 introduced the per-interpreter GIL (PEP 684). Embedding is where it matters: you can host multiple isolated Python interpreters in one C process, each with its own `sys.modules`, `builtins`, and (since 3.12) its own GIL.

```

// Subinterpreter embedding – each interpreter is isolated
Py_InitializeEx(0);

PyThreadState *main_tstate = PyThreadState_Get(); // main interpreter

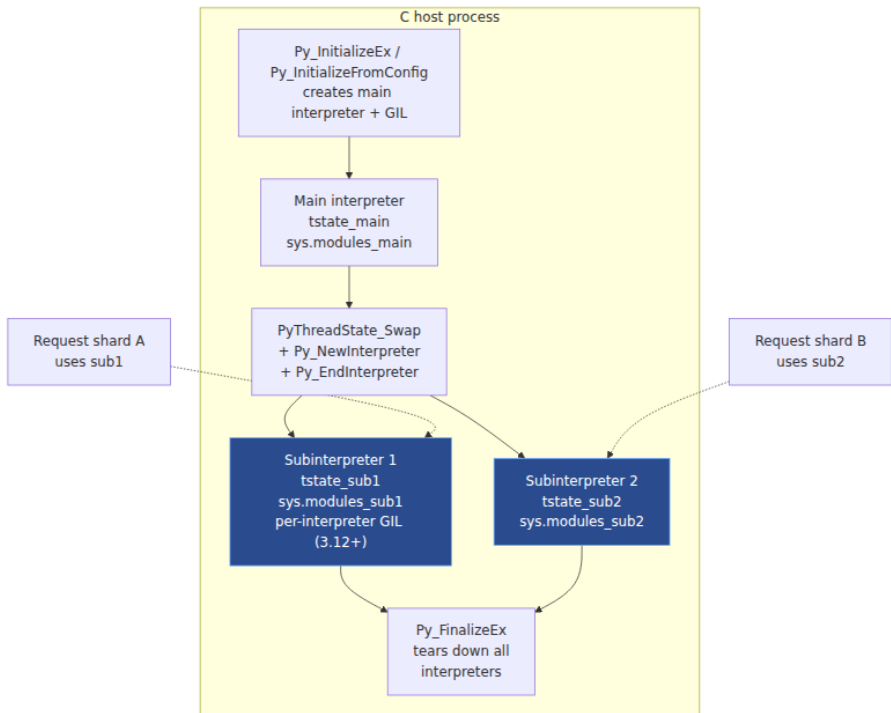
PyThreadState *sub1 = Py_NewInterpreter(); // new interpreter + new
tstate (holds GIL)
PyRun_SimpleString("import sys; print('sub1:', sys.version)");
PyThreadState_Swap(main_tstate); // back to main – sub1's GIL
interactions are separate (3.12+ per-interpreter GIL)

PyThreadState *sub2 = Py_NewInterpreter();
PyRun_SimpleString("x = 42; print('sub2 x =',
x)"); // x is not visible in sub1 or main
PyThreadState_Swap(main_tstate);

// Tear down subinterpreters before finalizing main
PyThreadState_Swap(sub1);
Py_EndInterpreter(sub1); // destroys sub1 – all its objects freed
PyThreadState_Swap(sub2);
Py_EndInterpreter(sub2);
PyThreadState_Swap(main_tstate);

Py_FinalizeEx();

```



Caveats:

- Many extensions are **not** subinterpreter-safe — they use C globals (`m_size = -1` modules, global caches). Importing such an extension in two subinterpreters corrupts state. Check `PyModuleDef.m_size >= 0` and `Py_mod_gil` slot.
- `Py_NewInterpreter` still shares the process's `atexit` handlers, signal handlers, and `os.environ` — isolation is at the Python-object level, not the OS level.
- PEP 554 (`interpreters` module) and PEP 684 expose subinterpreters to Python; embedding via `Py_NewInterpreter` is the C-level equivalent.

12.4 `pymain` — how `python` itself is an embedder

The `python` executable (`Programs/python.c` → `Modules/main.c` → `Python/pylifecycle.c`) is itself a thin embedder around `Py_RunMain`:

```

// Programs/python.c - simplified
int main(int argc, char *argv[]) {
    PyConfig config;
    PyConfig_InitPythonConfig(&config);
    PyConfig_SetBytesArgv(&config, argc, argv);
    // ... parse -c, -m, script args into config ...
    PyStatus status = Py_InitializeFromConfig(&config);
    // ... import sys, set sys.argv, handle -X, -W, PYTHON* ...
    int rc = Py_RunMain(); // runs -c / -m / script / REPL, then
    finalizes
    PyConfig_Clear(&config);
    return rc;
}

```

`Py_RunMain` handles `PyImport_ImportModule("runpy")` for `-m`, `PyRun_SimpleFile` for scripts, and the REPL loop — all on top of the same embedding primitives you use. Reading `Modules/main.c` is the best way to see production-grade embedding with proper `PyStatus` error handling, `PyConfig` isolation flags, and `Py_ExitStatusException` paths.

C main() — your host
or Programs/python.c

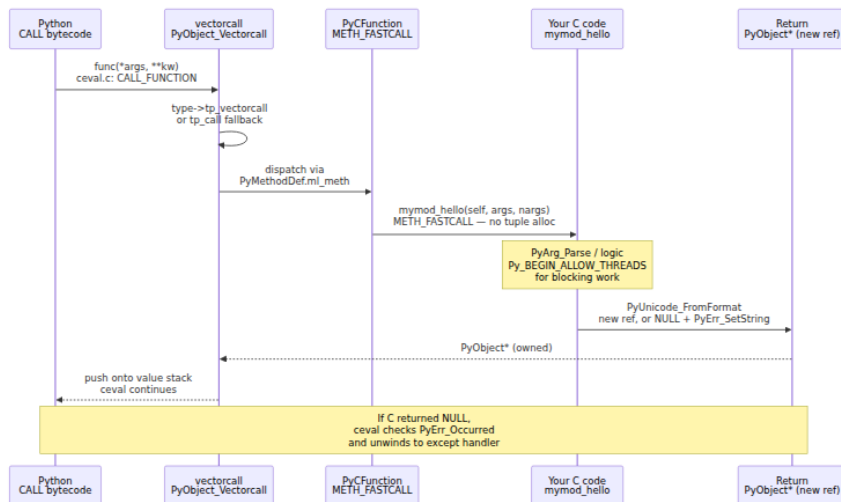
PyConfig /
Py_InitializeFromConfig
program_name ·
isolated · signal
handlers

Py_InitializeEx /
Py_InitializeFromConfig
pylifecycle.c:
init_importlib
init_sys · init_builtins

Drive Python:
PyRun_SimpleString
PyImport_ImportModule
PyObject_CallMethod
Py_NewInterpreter
(subinterpreters)

13. The call path — how `obj.method()` reaches your C function

Understanding the dispatch path explains both performance and debugging. Since Chapter 3 (bytecode/ `ceval`) and Chapter 7 (functions/ `vectorcall`), you know that `CALL` bytecodes use `vectorcall` (PEP 590) when available. C functions participate in the same protocol.



Three dispatch tiers, fastest first:

1. **vectorcall (PEP 590)** — `METH_FASTCALL` / `METH_FASTCALL | METH_KEYWORDS` functions expose `tp_vectorcall` directly. `ceval.c` calls them with a `PyObject *const *args` array and `nargs` — no tuple, no dict allocation. This is the path for all modern C extensions and for `PyFunctionObject` itself.
2. **METH_VARARGS / METH_O / METH_NOARGS** — older conventions. `ceval` allocates a tuple for `METH_VARARGS`; `METH_O` and `METH_NOARGS` validate arity and avoid the tuple.
3. **tp_call fallback** — if `tp_vectorcall` is `NULL`, `PyObject_Call` goes through `tp_call` (`PyCFunction` wrapper). Slower, but always available.

Backend lens — why vectorcall matters at scale. A service doing 100k `mymod.hello` calls per second saves one tuple allocation per call by

using `METH_FASTCALL` — roughly 56 bytes + GC tracking per call, or ~5 MB/s of allocator pressure eliminated. For `orjson` / `uvloop` hot paths, this is the difference between allocation-free and GC-visible.

14. Backend lens — perf-critical extensions, debugging segfaults, packaging `abi3` in CI

14.1 Writing perf-critical extensions for services

Guidelines distilled from `orjson`, `uvloop`, `cryptography`, and internal service extensions:

- **Never call back into Python in a tight loop.** Each `PyObject_Call` is a full `vectorcall` dispatch plus GIL and exception-state overhead. Batch work in C, return one Python object.
- **Use the buffer protocol for bulk data.** Accept `Py_buffer` / `memoryview` / `numpy` arrays as input; return `bytes` or buffer-exporting types as output. Zero-copy beats `PyBytes_AsString` + `memcpy` at scale.
- **Release the GIL for anything that blocks or burns CPU without Python.** I/O (`recv`, `libcurl`, `pread`), compression (`zstd`), crypto (`openssl`), and tight numeric loops are all candidates. Hold it only for `PyObject*` manipulation.
- **Prefer `METH_FASTCALL` and avoid `PyArg_ParseTuple` in hot paths.** Parse once at the boundary, then operate on C types. For ultra-hot paths, skip `PyArg_ParseTuple` entirely and validate `args[0]` directly.
- **Decide `abi3` vs. full API at design time.** If your extension is a thin wrapper around a C library and does not poke `PyLongObject` internals, choose `abi3` — one wheel, simpler CI. If you need `PyUnicode_KIND`, `PyLong` digit access, or free-threaded `Py_MOD_GIL_NOT_USED`, you need the full API.

14.2 Debugging segfaults — `faulthandler`, `gdb py-bt`, `ASan`

A segfault in a C extension is a process crash with no Python traceback — unless you prepare.

`faulthandler` — Python traceback on `SIGSEGV` / `SIGABRT` :

```

# Enable at startup – add to sitecustomize.py or call early in your
service
import faulthandler
faulthandler.enable()
# registers SIGSEGV/SIGABRT/SIGBUS handlers that dump Python tracebacks

# Or via CLI / env:
# python -X faulthandler app.py
# PYTHONFAULTHANDLER=1 python app.py

# Dump traceback on demand (e.g., from a watchdog thread):
faulthandler.dump_traceback_later(30, repeat=True) # SIGALRM-style
periodic dump

```

When a C extension segfaults with `faulthandler` enabled, `stderr` shows the Python stack that was active when the signal fired — often enough to identify which extension call triggered the crash.

gdb + py-bt — C + Python stacks together:

```

# Build CPython with debug symbols (or install python3-dbg)
# Build your extension with -g -O0 for useful stacks

gdb --args python -X faulthandler app.py
(gdb) run
# ... SIGSEGV ...
(gdb) bt          # C stack – shows which C function crashed
(gdb) py-bt      # Python stack – requires python3-dbg's libpython.py
GDB extension
(gdb) py-locals  # Python locals at the crash frame
(gdb) py-up / py-down # navigate Python frames

# If py-bt is not found:
(gdb) source /usr/share/gdb/auto-load/usr/bin/python3.12-gdb.py
(gdb) py-bt

```

Typical `gdb` session for a `PyList_GetItem` use-after-free:

```

Program received signal SIGSEGV, Segmentation fault.
0x00007f... in Counter_increment (self=0x...) at mymod.c:42
42      PyList_SetItem(list, 0, item); // item was borrowed and al
ready freed
(gdb) bt
#0  Counter_increment at mymod.c:42
#1  cfunction_vectorcall_FASTCALL at Objects/methodobject.c:426
#2  PyObject_Vectorcall at Objects/call.c:299
#3  PyEval_EvalFrameDefault at Python/ceval.c:4567
(gdb) py-bt
Traceback (most recent call first):
  File "app.py", line 87, in handle_request
    c.increment(items[0])
  File "app.py", line 42, in main
    handle_request(req)

```

AddressSanitizer — catch use-after-free before it crashes:

```

CFLAGS="-fsanitize=address -g -O1 -fno-omit-frame-pointer" \
LDFLAGS="-fsanitize=address" \
./configure --with-address-sanitizer --with-pydebug
make -j$(nproc)
# Run - ASan reports the exact INCREf/DECREf mismatch with allocation +
free stacks
./python -m pytest tests/test_mymod.py

```

Production note. `faulthandler.enable()` has near-zero overhead and should be on in every production Python process. `gdb py-bt` requires `python3-dbg` (Debian/Ubuntu) or a `--with-pydebug` build. ASan is for CI/debug builds only — it doubles RSS and halves throughput, but catches the bugs that become 3 AM pages.

14.3 Packaging `abi3` wheels in CI — the one-wheel strategy

For internal extensions, the `abi3` payoff is CI simplicity: one build per platform instead of one per `(platform, Python version)`.

```

# .github/workflows/wheels.yml – complete abi3 CI
name: wheels
on: [push, pull_request]
jobs:
  build:
    strategy:
      matrix:
        os: [ubuntu-latest, macos-latest]
        # No Python matrix needed for abi3 – one build per OS
    runs-on: ${ matrix.os }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.10" } # oldest supported – abi3
wheel runs on 3.10+
      - run: pip install build auditwheel # auditwheel on Linux;
delocate on macOS
      - run: python -m build -w
      - run: auditwheel show dist/*.whl | grep -q abi3 # assert abi3
tag
      - uses: actions/upload-artifact@v4
        with: { path: dist/*.whl }

    test:
      needs: build
      strategy:
        matrix: { python: ["3.10", "3.11", "3.12", "3.13"] }
      runs-on: ubuntu-latest
      steps:
        - uses: actions/setup-python@v5
          with: { python-version: "${ matrix.python }" }
        - uses: actions/download-artifact@v4
        - run: pip install --no-index --find-links=dist mymod && python
-c "import mymod; mymod.hello('ok')"

```

`cibuildwheel` variant (for manylinux/musllinux compliance):

```

# pyproject.toml – cibuildwheel abi3 config
[tool.cibuildwheel]
build = "cp310-*"
archs = ["x86_64", "aarch64"]
# With abi3, cibuildwheel builds once (cp310) and tests on all newer –
no per-version rebuild

```

Key takeaways

- **Reference ownership is the entire contract.** Every `PyObject*` you receive is either *new* (you must `Py_DECREF`) or *borrowed* (you must not, unless you `Py_INCREF` first). Stealing APIs (`PyTuple_SetItem`, legacy `PyModule_AddObject`) transfer ownership and are being replaced by non-stealing variants. Get this wrong and you leak or corrupt the heap — no compiler warning will save you.
- **Modules are `PyModuleDef` + `PyMethodDef`; types are `PyType_Spec` + `PyType_Slot` + `PyType_FromSpec`.** `METH_FASTCALL` is the modern calling convention — it avoids tuple allocation and integrates with `vectorcall`. Multi-phase init (`Py_mod_exec`, `m_size >= 0`) is required for subinterpreter safety and `Py_MOD_GIL_NOT_USED`.
- **Argument parsing has a spectrum.** `PyArg_ParseTuple` / `PyArg_ParseTupleAndKeywords` with format strings for simple cases; `_PyArg_Parse` / Argument Clinic for `__text_signature__` and positional-only enforcement; direct `args[0]` validation in ultra-hot paths. Know the format units and that `s / 0` are borrowed.
- **Error handling is `PyErr_Set*` + `NULL` return; every fallible call must be checked.** `PyErr_Fetch` / `PyErr_Restore` for cleanup that must not clobber the exception. `PyErr_Occurred` is borrowed. For non-`PyObject*` returns, `-1 + PyErr_Occurred` distinguishes error from a legitimate `-1` value.
- **Release the GIL around blocking work, reacquire before touching `PyObject*`.** `Py_BEGIN_ALLOW_THREADS` / `Py_END_ALLOW_THREADS` for synchronous blocking; `PyGILState_Ensure` / `PyGILState_Release` for callbacks on foreign threads. Violating the "no `PyObject*` while released" rule is immediate UB.
- **The buffer protocol (`Py_buffer`, `bf_getbuffer` / `bf_releasebuffer`) is the zero-copy path.** Producers `INCRREF view->obj`; consumers `PyBuffer_Release`. `memoryview` is the generic consumer; `numpy` is the typed one. Always respect `flags` and keep `shape` / `strides` alive until `releasebuffer`.
- **The Stable ABI (`abi3`, PEP 384) trades surface for portability.** Compile with `Py_LIMITED_API=0x030A0000` + `py_limited_api=True` on the oldest supported Python; ship one wheel that runs on every newer 3.x. You lose direct struct access and `cpython/ / internal/`

headers — worth it for wrapper extensions, not for `numpy`-class internals.

- **HPy replaces raw pointers with opaque handles and borrowed references with always-owned handles.** Its universal ABI runs one `.hpy.so` on CPython, PyPy, and GraalPy and is compatible with moving GC. Evaluate it for new extensions where portability outweighs ecosystem inertia; migrate existing extensions incrementally via shims.
 - **Embedding inverts the host/guest relationship.**
`Py_InitializeFromConfig` + `PyConfig` for modern init,
`PyRun_SimpleString` / `PyImport_ImportModule` / `PyObject_CallMethod` to drive Python, `Py_NewInterpreter` / `Py_EndInterpreter` for subinterpreter isolation (per-interpreter GIL since 3.12),
`Py_FinalizeEx` to tear down. `python` itself is an embedder around `Py_RunMain` — read `Modules/main.c`.
 - **Debug crashes with `faulthandler` (always on in prod) + `gdb` `py-bt` + ASan in CI.** Package `abi3` wheels with a one-build-per-platform CI matrix and test the same wheel across Python versions. These are not optional for backends that ship C extensions.
-

Further reading

- **CPython C API documentation** — the authoritative reference for every function, flag, and contract in this chapter. Start with <https://docs.python.org/3/c-api/index.html> and the sub-pages `c-api/arg.html` (argument parsing), `c-api/buffer.html` (buffer protocol), `c-api/module.html` (`PyModuleDef`), `c-api/type.html` (`PyType_Spec`), and `c-api/exceptions.html` (`PyErr_*`). Pinned, version-specific.
- **PEP 384 — Defining a Stable ABI** — the PEP that defines `Py_LIMITED_API`, the Stable ABI guarantee, and `abi3` wheel tagging. <https://peps.python.org/pep-0384/> — pinned.
- **HPy documentation and PEP 630** — HPy design, handle model, universal ABI, and migration guide at <https://hpyproject.org> and <https://docs.hpyproject.org>. PEP 630 (– Isolating Extension Modules) and the HPy design docs explain the handle-vs-pointer rationale and the path away from borrowed references. Pinned.

- **CPython source — Include/ and Modules/** — the ground truth. `Include/Python.h` (umbrella header), `Include/cpython/object.h` (`PyTypeObject` layout), `Include/cpython/buffer.h` (`Py_buffer`), `Include/modsupport.h` (`PyArg_ParseTuple` format units), `Modules/main.c` (`pymain / Py_RunMain`), and `Objects/methodobject.c` (`PyCFunction` dispatch, `vectorcall`). <https://github.com/python/cpython> — pinned.
- **PEP 489 — Multi-phase extension module initialization** — why `Py_mod_create / Py_mod_exec / m_size / m_slots` exist and how they enable subinterpreter isolation. <https://peps.python.org/pep-0489/>
- **PEP 590 — Vectorcall: a fast calling protocol for CPython** — the `vectorcall / METH_FASTCALL` dispatch that replaced tuple-based calls on the hot path. <https://peps.python.org/pep-0590/>
- **PEP 684 — A Per-Interpreter GIL and PEP 554 — Multiple Interpreters in the Stdlib** — the subinterpreter isolation model that embedding and `Py_NewInterpreter` now build on. <https://peps.python.org/pep-0684/> and <https://peps.python.org/pep-0554/>
- **CPython Doc/c-api/stable.rst and Doc/c-api/limited-api.rst** — the Limited API / Stable ABI user guide, including the `abi3` audit (`python -m abi3audit`) and `Py_LIMITED_API` version hex values.
- **"Python's C API" chapter in CPython Internals (Real Python / Anthony Shaw)** — a book-length companion that walks `ceval.c`, `Objects/`, and `Python/pylifecycle.c` with annotated source — useful alongside this chapter's backend lens.
- **HPy case studies — numpy HPy port and hpy benchmarks** — <https://hpyproject.org/case-studies> and the `hpy` GitHub benchmarks for handle-indirection overhead and universal-ABI portability measurements.
- **Debugging references — faulthandler docs, GDB libpython.py, AddressSanitizer** — <https://docs.python.org/3/library/faulthandler.html>, <https://wiki.python.org/moin/DebuggingWithGdb>

(`py-bt` , `py-locals` , `py-up`), and <https://github.com/google/sanitizers/wiki/AddressSanitizer> for ASan builds.

Chapter 11 — Performance: Profiling, the Copy-and-Patch JIT (PEP 744), Cython, and Alternative Runtimes

What this chapter covers. Python performance work fails when it starts with intuition. This chapter gives you the measurement stack that precedes any optimization: the full profiler taxonomy from deterministic function profiling to allocation tracking, the mechanics of the specializing adaptive interpreter you met in Chapter 3 (PEP 659), the new tier-2 copy-and-patch JIT introduced in Python 3.13 (PEP 744), the Cython compilation pipeline that still wins for hot serialization and numeric kernels, and the alternative runtimes — PyPy, GraalPy, and free-threaded CPython — that change the trade-off entirely. Every technique is shown with real commands, realistic output, and the backend lens of continuous profiling in production fleets, JIT warmup under autoscaling, and where compiled extensions still earn their complexity.

Learning goals — after this chapter you should be able to:

- Classify any Python profiler into deterministic, sampling, line-granular, or allocation-tracking and pick the right one for a given overhead budget and environment (development vs. production).
- Run `cProfile` , `py-spy` , `austin` , `perf` , `line_profiler` , `memray` , and `tracemalloc` with correct flags and interpret their output, including flame graphs and allocation flame graphs.
- Explain the overhead and blind spots of each profiler — why `cProfile` misses C time, why sampling loses short calls, why `tracemalloc` underestimates native allocations.
- Recap PEP 659 end-to-end: adaptive counters, quickening, inline caches, specialization, and guards.
- Walk through the PEP 744 tier-2 JIT pipeline: bytecode → tier-1 specialization → tier-2 uops (micro-ops) → trace stitching → copy-and-patch codegen → native code, plus guards and deoptimization.

- Build and enable the experimental JIT (`--enable-experimental-jit` , `PYTHON_JIT=1` / `-X jit`), predict its speedup envelope (~10-30% on tight typed loops, single-digit geometric mean on `pyperformance`), and reason about warmup.
- Write, compile, and profile Cython (`cdef` , `cpdef` , typed memoryviews, `boundscheck` / `wraparound` directives, `cythonize`) and decide when it beats pure Python or a C extension.
- Compare PyPy's tracing JIT, GraalPy's Truffle JIT, and CPython's tiered model — and choose among CPython, PyPy, GraalPy, and free-threaded CPython for a given workload.
- Operate continuous profiling in production (Parca/Pyroscope), handle JIT warmup under autoscaling, and justify Cython for hot serialization paths.

Prerequisites. Chapter 3 dissected bytecode, `ceval.c` , and PEP 659 specialization — the tier-1 substrate this chapter builds on. Chapter 4 covered `pymalloc` and the GC (needed for allocation profilers), Chapter 5 the GIL and free-threaded builds, and Chapter 10 C extensions and the C API (needed for Cython). This chapter assumes you can run `perf` , read a flame graph, and skim `Python/ceval.c` .

1. Why profiling is the first optimization

Backend Python services rarely die from a single slow function. They die from a thousand small costs compounded across replicas: 4% extra CPU per request $\times$ 800 pods $\times$ 30 days is a line item. Unmeasured optimization is how teams spend a sprint rewriting a JSON serializer that accounts for 1.2% of fleet CPU while a single `re.compile` inside a hot loop burns 9%.

Three production realities force measurement-first discipline:

- **CPython is a bytecode interpreter with late-bound types.** Every attribute load, binary op, and call is a dictionary lookup or type check until specialization proves otherwise. The cost is invisible in source but dominant in profiles.
- **Most wall time is off-CPU or in C.** A Django or FastAPI handler spends time waiting on Postgres, Redis, and `json.dumps` / `msgpack` /

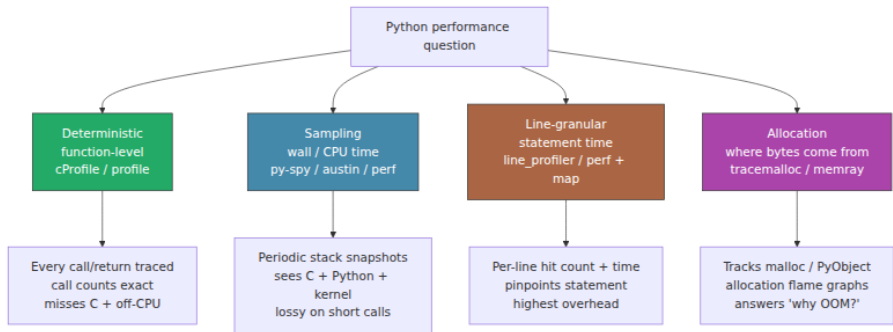
`numpy` — code that deterministic Python profilers cannot see. Sampling and `perf` are required to see the full stack.

- **Optimization changes the measurement.** Adding `cProfile` instrumentation can slow a request by 30-50% and distort contention. A tracing JIT or copy-and-patch JIT changes which code is hot after warmup. Profile, fix, and re-profile — never trust a stale measurement.

The workflow is always the same: collect → symbolize → flame → fix → verify. The rest of this chapter is tooling to make each step honest.

2. Profiler taxonomy — four families, four overhead budgets

Not all profilers measure the same thing. Choosing the wrong family gives you a precise answer to the wrong question.



Family	Representative tools	Mechanism	Overhead	Production safe?
Deterministic	<code>cProfile</code> (C), <code>profile</code> (Python)	Hook on every call/return via <code>PyEval_SetProfile</code> / <code>sys.setprofile</code>	20-60% (workload dependent)	No — distorts latency and contention

Family	Representative tools	Mechanism	Overhead	Production safe?
Sampling (Python-aware)	<code>py-spy</code> , <code>austin</code>	Out-of-process stack sampling via <code>ptrace / /proc/<pid>/mem</code> or <code>LD_PRELOAD eBPF</code>	1-5%	Yes — no instrument no restart required
Sampling (system)	<code>perf</code> + <code>perf map</code>	Hardware PMU / kernel <code>perf_events</code> sampling of instruction pointer + DWARF / frame-pointer unwind	1-3%	Yes — kernel facility
Line-granular	<code>line_profiler</code> (<code>kernprof</code>)	Line-level timer via AST rewrite / trace function	50-200%+	No
Allocation	<code>tracemalloc</code> (<code>stdlib</code>), <code>memray</code>	Heap allocation tracking: <code>tracemalloc</code> hooks <code>PyMem_Malloc</code> , <code>memray</code> hooks <code>malloc</code> via <code>LD_PRELOAD</code> + DWARF unwind	<code>tracemalloc</code> 5-15%, <code>memray</code> 5-25%	<code>tracemalloc</code> yes (<code>stdlib</code> , bounded overhead); <code>memray</code> yes with care

Rule of thumb for backend work:

- **In development, start deterministic** (`cProfile`) to find the hot call graph, then **line-profile the top 1-2 functions** to find the hot statement, then **allocation-profile** if **RSS or GC pauses are the symptom**.
- **In production, use sampling** (`py-spy` / `austin` for Python-aware, `perf` for system-wide) and **continuous profiling** (Parca/Pyroscope) for fleet-wide flame graphs — never attach a deterministic profiler to a serving replica.

3. Deterministic profilers — `cProfile` and `profile`

`cProfile` is a C extension that registers a profile callback with the interpreter. On every `CALL`, `RETURN`, `C_CALL`, and `EXCEPTION` event it records wall/CPU time and call counts. `profile` is the pure-Python equivalent — same semantics, $\sim 10\times$ slower, useful only when you need to subclass the profiler.

3.1 The one command you should memorize

```
# Cumulative profile, sorted, top 30 – the 80% starting point
python -m cProfile -s cumulative app.py | head -n 60

# Write raw stats for interactive analysis
python -m cProfile -o app.prof app.py
python -c "import pstats; p=pstats.Stats('app.prof');
p.sort_stats('cumulative').print_stats(30)"
python -c "import pstats; p=pstats.Stats('app.prof');
p.sort_stats('tottime').print_stats(30)"

# Visualize – snakeviz (browser) or gprof2dot
pip install snakeviz
snakeviz app.prof
```

Real output (trimmed) from a FastAPI service handling 10k synthetic requests:

```

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
 10000    0.412    0.000    1.847    0.000
app.py:42(handle_request)
 10000    0.301    0.000    0.889    0.000 {method 'dumps' of 'json
' objects}
 20000    0.188    0.000    0.344    0.000 app.py:18(validate_paylo
ad)
 10000    0.121    0.000    0.121    0.000 {built-in method _json.e
ncode}
...
```

Reading it:

- `tottime` — time in the function *excluding* callees. High `tottime` means the function itself is the work.
- `cumtime` — time in the function *including* callees. High `cumtime` with low `tottime` means it is a dispatcher — follow the callees.
- `percall` — `tottime / ncalls`. Useful for spotting functions that are cheap per call but called pathologically often.
- `ncalls` with a/b (e.g., 100/1) — 100 total calls, 1 primitive (non-recursive) call, 99 recursive.

3.2 Programmatic use and `pstats` patterns

```

import cProfile, pstats, io

pr = cProfile.Profile()
pr.enable()
run_workload()           # code under test
pr.disable()

s = pstats.Stats(pr)
s.strip_dirs().sort_stats("cumulative").print_stats(20)
s.sort_stats("tottime").print_stats(20)

# Filter to your code
s.print_stats("myapp/*.py")
# Find the most-called functions
s.sort_stats("calls").print_stats(20)
# Who calls whom
s.print_callers("myapp/handlers.py:.*")
s.print_callees("myapp/handlers.py:.*")

# Export for speedscope / snakeviz
s.dump_stats("workload.prof")
```

3.3 Overhead and blind spots

`cProfile` measures Python function entry/exit. It does **not** charge time spent inside C extensions to the calling Python function correctly — `json.dumps` appears as a single `cumtime` bucket, not as the internal `encode_basestring_ascii` work. For I/O-bound handlers it is actively misleading: 80% wall time waiting on `socket.recv` is invisible or appears as a single `wait` bucket. Use sampling (Section 4) to see C and off-CPU time.

Overhead is proportional to call frequency. A tight loop calling a Python function 10M times can slow down 50-100% under `cProfile` because every call triggers the hook. For that reason, never use deterministic profiling to measure micro-optimizations — the profiler *is* the bottleneck.

4. Sampling profilers — `py-spy`, `austin`, and `perf`

Sampling profilers do not instrument code. They wake up at a fixed rate (default ~100 Hz), snapshot every thread's stack, and aggregate. The result is a statistical flame graph: wider boxes mean more samples, i.e., more time.

4.1 `py-spy` — the production default for Python

`py-spy` reads Python stacks out-of-process via `process_vm_readv` — no code changes, no restart, no `LD_PRELOAD`. It understands CPython's `PyThreadState`, `PyFrameObject`, and `_PyInterpreterFrame` layout for the running version, so it can symbolize Python frames even when the GIL is held.

```

# Install (Rust, single binary – no agent)
pip install py-spy          # or: cargo install py-spy
# Record a flame graph SVG – the single most useful artifact
py-spy record -o profile.svg -- python app.py
# Top-like live view
py-spy top -- python app.py
# Attach to a running production PID (no restart, read-only)
sudo py-spy record -o prod.svg --pid 12345 --duration 30
# Non-root with --nonblocking if permissions allow
py-spy record -o profile.svg --python python3 -- app.py --arg val

# Speedscope / raw output
py-spy record -o profile.speedscope.json --format speedscope -- python
app.py

```

What to look for in `profile.svg`: the widest plateau is your bottleneck. Python frames are labeled `function (file.py:lineno)`, native frames as `symbol (libXYZ.so)`. If the widest plateau is `json.dumps` → `encode_basestring_ascii` in `libpython`, your bottleneck is serialization — a Cython or `orjson` problem, not a Python loop problem.

`py-spy` sampling modes:

- `--rate 100` (default 100 Hz) — good for 30s+ captures. Raise to 200–500 Hz for short-lived handlers at the cost of overhead.
- `--gil` — only sample threads holding the GIL (filters out I/O wait).
- `--idle` — include idle threads (off by default).

Overhead is ~1–3% at 100 Hz. Safe for production with `--duration` caps.

4.2 `austin` — the frame-stack sampler

`austin` takes a different approach: it samples the CPython frame stack directly via `LD_AUDIT` / `ptrace` with lower overhead than `py-spy` at high rates and first-class multiprocessing support.

```
# Install
pip install austin

# Sample the current process tree – 100 Hz, 30 s, full stacks
austin -i 10000 -o austin.dat python app.py
# Attach to running PID
sudo austin -p 12345 -o prod.dat

# Visualize – austin2speedscope or flamegraph.pl
austin2speedscope austin.dat profile.speedscope.json
# Or via austin's TUI
austin --help
```

`austin` shines when:

- You need **per-process and per-thread** breakdowns in a `gunicorn / uvicorn` worker tree (it auto-discovers children).
- You want **eBPF-adjacent overhead** — `austin` at 1 kHz is still <2% overhead in many workloads.
- You run inside containers where `py-spy`'s `process_vm_readv` is blocked by `seccomp / ptrace_scope`.

Both `py-spy` and `austin` emit collapsed stacks compatible with `flamegraph.pl` and `Speedscope`. Pick one and standardize — the fleet should not mix formats.

4.3 `perf` — the system truth, now Python-aware

`perf` samples the hardware PMU / kernel `perf_events` — it sees everything: Python bytecode dispatch, C extension code, `libc`, kernel, and off-CPU via tracepoints. The historically hard part was symbolizing Python frames (they are not ELF symbols). Two mechanisms fix that:

- **CPython 3.12+ `perf` support** (`--with-perf` or `python -X perf`): the interpreter writes a `/tmp/perf-<pid>.map` file mapping JIT-like Python frame addresses to `file:line:function` strings, and `perf` picks it up automatically.
- **`python -X perf_jit / perf map` agent**: earlier approach via `perf-map-agent` that injects `perf map` entries at runtime.


```

# Best available on Python 3.12+ built with --with-perf
perf record -F 99 -g -- python -X perf app.py
perf report --no-children          # interactive TUI
perf script | ./stackcollapse-perf.pl | ./flamegraph.pl > perf.svg

# If your Python lacks perf map support, use py-spy/austin for Python
stacks
# and perf for system stacks side-by-side:
perf record -F 99 -g -- python app.py &
PY_PID=$!
py-spy record -o py.svg --pid $PY_PID --duration 20
wait

# Off-CPU / blocking analysis (requires tracepoints)
perf record -e sched:sched_switch -g -- python app.py
perf report

# Verify perf map was generated
ls -l /tmp/perf-*.map
head /tmp/perf-*.map
# 7f3a... 2a py::app.handle_request:/home/ubuntu/app.py:42

```

Why `-F 99` and not `-F 999`? 99 Hz avoids aliasing with 100 Hz periodic activity (tick, GC) and keeps overhead ~1%. For production continuous profiling, 49–99 Hz is standard. Reserve 997 Hz for short, targeted captures.

`perf report` reading: look for `PyEval_EvalFrameDefault` / `_PyEval_EvalFrameDefault` (the `ceval` loop), `vectorcall`, `PyObject_Malloc`, and your hot C extension. If `PyEval_EvalFrameDefault` is the top symbol, you are interpreter-bound — specialization and JIT are the lever (Sections 7–8). If `_PyEval_EvalFrameDefault` is low and `__write` / `futex_wait` / `__libc_recv` dominates, you are I/O-bound — profiling Python execution is the wrong tool; trace the network.

4.4 Choosing a sampling profiler

Signal	Tool	Command
Python wall time, production-safe, no privileges	<code>py-spy</code>	<code>py-spy record -o flame.svg --pid <pid></code>

Signal	Tool	Command
High-rate, multiprocessing, container-restricted	<code>austin</code>	<code>austin -p <pid> -o austin.dat</code>
System + kernel + Python, hardware counters	<code>perf</code>	<code>perf record -F 99 -g -- python -X perf app.py && perf report</code>
Continuous fleet-wide	Parca / Pyroscope agent wrapping <code>py-spy</code> or <code>perf</code>	Agent sidecar (Section 11)

5. Line and allocation profilers — finding the statement and the leak

5.1 `line_profiler` — which statement is hot

Function-level profiling tells you *which* function. Line profiling tells you *which line*. The cost is high overhead and code decoration, so it is a development-only tool applied to 1-2 functions identified by sampling.

```

pip install line_profiler

# Decorate the suspect function
# app.py:
# @profile
# def handle_request(req): ...

kernprof -l -v app.py
# Or explicitly:
python -m line_profiler -l app.py.lprof

```

```
# app.py – the @profile decorator is injected by kernprof; no import
needed
@profile
def serialize_batch(rows):
    out = []
    for r in rows:
        out.append(json.dumps(r)) # line 5
        out.append("\n")         # line 6
    return b"".join(o.encode() for o in out) # line 8 – hidden encode
loop
```

Output:

```
Timer unit: 1e-06 s

Total time: 0.48231 s
File: app.py
Function: serialize_batch at line 2
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
2					@profile
3					def serialize_batch(ro
4	1000	412.0	0.4	0.1	out = []
5	10000	398412.0	39.8	82.6	out.append(jso
6	10000	8212.0	0.8	1.7	out.append("\n
7					)
8	1000	75271.0	75.3	15.6	return
9					b"".join(o.encode() for o in out)

The fix is obvious: line 5 dominates, line 8 hides a per-element `encode` — batch the encode or switch to `orjson` / Cython. Without line granularity you would have guessed wrong between the two.

Modern `line_profiler` also ships `kernprof -p` for `cProfile`-compatible line stats and `line_profiler`'s `profile` object for programmatic use.

5.2 `tracemalloc` — the `stdlib` allocation tracker

`tracemalloc` hooks `PyMem_Malloc` / `PyObject_Malloc` and records the stack trace of every allocation. It is bounded in memory (default 25 frames, 1/16th sampling in 3.12+ with `tracemalloc.start(25)`) and ships with CPython — zero dependencies, safe to leave on in staging.

```

import tracemalloc, linecache

tracemalloc.start(25) # 25 frames of traceback

# ... run workload ...
snapshot = tracemalloc.take_snapshot()
top = snapshot.statistics("lineno") # or "traceback" for full stacks

print("[ Top 10 allocation sites ]")
for stat in top[:10]:
    print(stat)
    # stat.traceback is a Traceback object – format it:
    for line in stat.traceback.format():
        print(line)

# Compare two snapshots to find leaks
snap1 = tracemalloc.take_snapshot()
run_one_batch()
snap2 = tracemalloc.take_snapshot()
diff = snap2.compare_to(snap1, "lineno")
for stat in diff[:10]:
    print(stat)

```

Output:

```

app.py:42: size=512 KiB, count=10000, average=52 B
app.py:88: size=128 KiB, count=200, average=655 B
...

```

`tracemalloc` sees **Python-allocated** memory. It does **not** see `malloc` inside C extensions (`numpy`, `Pillow`, `grpcio`) unless they use `PyMem_Malloc`. For native leaks, use `memray`.

5.3 `memray` — the native allocation profiler

`memray` (from Bloomberg) interposes `malloc/free` via `LD_PRELOAD`, unwinds native stacks with DWARF/libunwind, and tracks Python stacks simultaneously. It produces allocation flame graphs, high-water-mark analysis, and leak reports — and it sees both Python and native allocations.

```

pip install memray

# Allocation flame graph – run, then render
memray run -o app.memray app.py --arg val
memray flamegraph app.memray          # → memray-flamegraph-
app.memray.html
memray table app.memray                # tabular top allocators
memray summary app.memray              # high-water mark + totals

# Live TUI – watch allocations in real time
memray run --live-remote app.py
# In another shell:
memray live 12345

# Track a gunicorn worker tree
memray run --follow-fork -o worker.memray gunicorn app:application -w 4

```

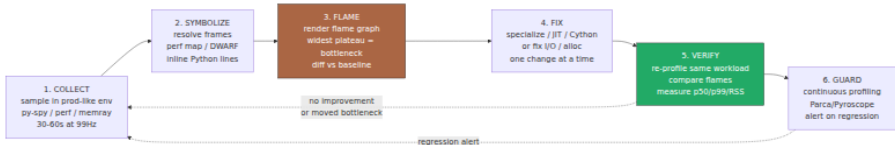
What `memray` gives you that `tracemalloc` cannot:

- Native allocations from `numpy`, `pandas`, `grpcio`, `Pillow`, `cryptography`.
- **Temporal** flame graphs — drag to see allocations at a point in time, not just totals.
- **Leak detection** — allocations with no matching `free` at exit, grouped by stack.
- **High-water-mark** attribution — which stack pushed RSS to its peak (critical for OOM kills in Kubernetes).

Overhead is 5–25% depending on allocation rate and whether DWARF unwind is enabled. For production, prefer `tracemalloc` for always-on and `memray` for targeted captures.

6. The profiling workflow — collect, flame, fix, verify

Profiling without a workflow produces artifacts, not improvements. Standardize the loop so every engineer on the fleet does it the same way.



Concrete checklist per phase:

1. Collect. Always sample a prod-like environment — staging with prod-shaped traffic or a single canary pod. Deterministic profiles from a laptop lie about cache, I/O, and contention. Record long enough to include GC cycles and JIT warmup (≥ 30 s). Pin the version, commit, and workload.

```
# Canary capture – the artifact you will diff against
py-spy record -o baseline.svg --pid $(pidof gunicorn) --duration 60 --
rate 99
# System-wide for kernel + Python
perf record -F 99 -g --call-graph dwarf -- python -X perf app.py
```

2. Symbolize. Ensure stacks resolve to `file:line:function`. For `perf`, verify `/tmp/perf-*.map` exists. For `memray`, ensure debug symbols (`apt install python3-dbg` or build with `-g`). Unresolved stacks (`[unknown]`, `0x7f...`) waste the entire capture.

3. Flame. Open the SVG/Speedscope. Read from bottom (entry) to top (leaf). The widest plateau is the bottleneck — its width is proportional to samples. Use **differential flame graphs** (`flamegraph.pl --diff`) to compare before/after — red is new, blue is reduced.

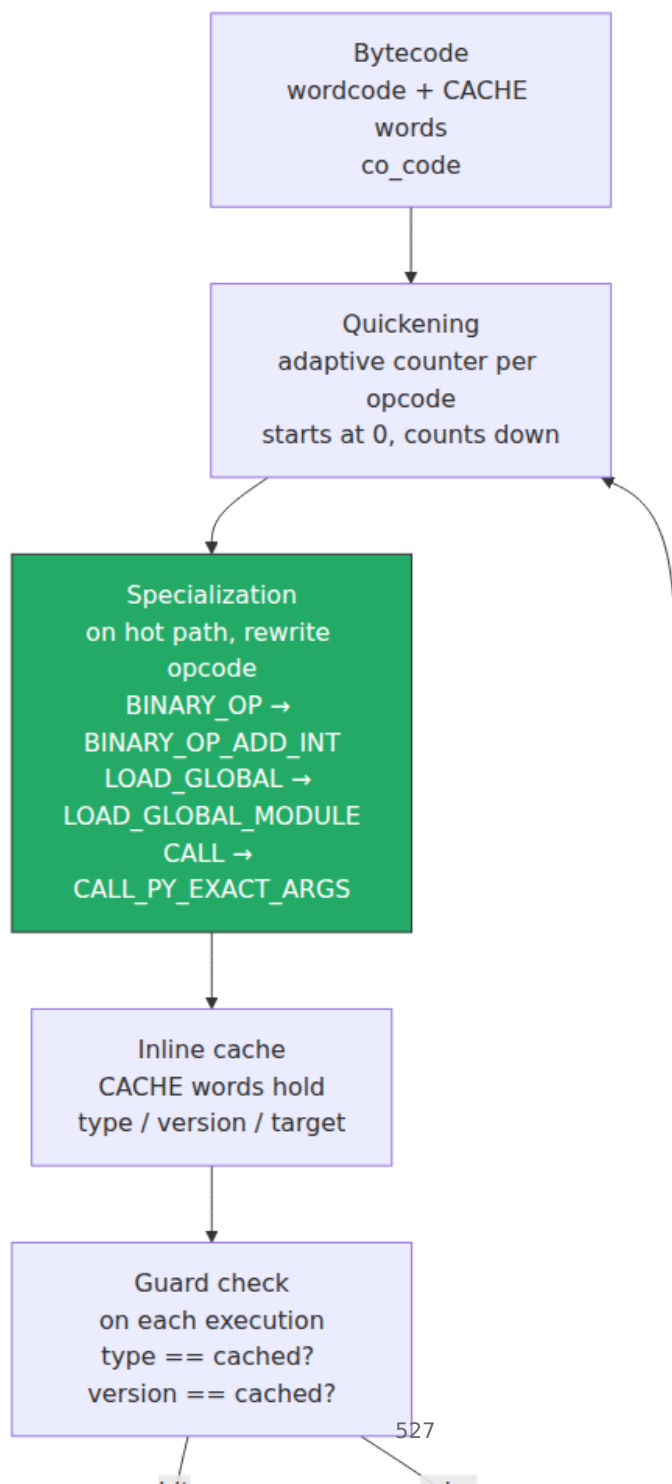
4. Fix. Change one thing. If the flame says `json.dumps` in Python, the fix might be `orjson`, a Cython serializer, or a schema change — not micro-optimizing the Python loop that calls it. If it says `PyEval_EvalFrameDefault` at the top, you are interpreter-bound — specialization/JIT/Cython (Sections 7–9) is the lever.

5. Verify. Re-profile with the identical workload and flags. Compare flames side-by-side and measure end-to-end: `p50 / p99` latency, CPU seconds/request, and max RSS. A 15% flame-width reduction that does not move `p99` is not a win — you moved the bottleneck.

6. Guard. Ship the fix with continuous profiling (Section 11) so regressions are caught when the next PR widens the same plateau.

7. Recap — the specializing adaptive interpreter (PEP 659)

Chapter 3 covered this in depth; here is the compressed recap you need to understand the JIT.



How it works, in one paragraph: every adaptive opcode (`BINARY_OP`, `LOAD_GLOBAL`, `LOAD_ATTR`, `STORE_ATTR`, `BINARY_SUBSCR`, `CALL`, `COMPARE_OP`, `UNPACK_SEQUENCE`, `FOR_ITER`) starts as `*_ADAPTIVE` with an 8-bit counter (initial value ~ 8 , tuned per opcode). Each execution decrements the counter; when it hits zero the interpreter inspects the live types and values, and if they are stable and optimizable it **rewrites the opcode in place** to a specialized form and fills the trailing `CACHE` words with the observed type, version tag, or function target. Subsequent executions take a **guarded fast path**: check that the cached type/version still matches, and if so execute inline (no `PyObject_GetAttr` dict lookup, no `PyNumber_Add` dispatch). On mismatch, **deoptimize** — rewrite back to adaptive and resume counting.

Impact quantified (CPython 3.11, `pyperformance` geometric mean):

- Specialization alone: **$\sim 10\text{--}25\%$** faster than 3.10 on macro-benchmarks, up to **$\sim 60\%$** on attribute-heavy and arithmetic micro-benchmarks.
- The common fast paths are `LOAD_GLOBAL_MODULE` (module globals without dict lookup), `LOAD_ATTR_INSTANCE_VALUE` (instance `__dict__` offset load), `BINARY_OP_ADD_INT` / `BINARY_SUBSCR_LIST_INT` (integer fast paths), and `CALL_PY_EXACT_ARGS` (no `CALL` argument parsing).

The ceiling: specialization is **per-opcode, not cross-opcode**. It does not inline across calls, eliminate bounds checks, or compile to native code. That is what tier 2 does.

8. Tier 2 — the copy-and-patch JIT (PEP 744, Python 3.13+)

8.1 Why a JIT, why copy-and-patch

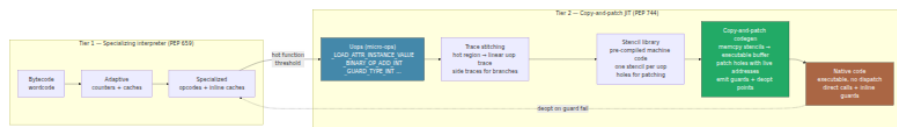
The specializing interpreter hits a ceiling because `ceval.c` is still a dispatch loop — every opcode is a `switch / goto` branch, and every guard is a C-level branch. A JIT removes the dispatch entirely by compiling hot traces to native machine code.

CPython's JIT (PEP 744, authored by Brandt Bucher, merged for 3.13) deliberately chose the simplest viable strategy: **copy-and-patch**. Instead of a full optimizing compiler (SSA IR, register allocation, LLVM), it stitches together pre-compiled machine-code *stencils* (templates with

holes) — one stencil per **uop** (micro-operation) — patching the holes with runtime constants (addresses, offsets, guards) and emitting a contiguous native function. No IR, no register allocator, no LLVM dependency — just `memcpy` + `patch` + `mprotect(X)`.

This keeps the implementation small (~15k lines), auditable, and portable (x86-64 and AArch64 in 3.13), at the cost of not doing loop unrolling, vectorization, or sophisticated inlining — those are future work.

8.2 Tier 1 → tier 2 pipeline



Step by step:

1. Tier 1 specialization must happen first. The JIT does not compile cold bytecode. It consumes *already-specialized* tier-1 traces — the type feedback collected in `CACHE` words is the JIT's type information. No specialization, no tier-2 compilation. This is why 3.11's specialization was prerequisite to 3.13's JIT.

2. Uops — the JIT's IR. The *tier-2 optimizer* (`Python/optimizer.c`, `Python/bytcodes.c`) translates a hot bytecode region into a linear sequence of **uops** (micro-ops). Uops are finer-grained than bytecodes: one bytecode like `LOAD_ATTR` becomes `_LOAD_ATTR_INSTANCE_VALUE` + `_GUARD_TYPE_VERSION` + `_CHECK_ATTR_AUX`. The uop set in 3.13 is ~50 ops, defined in `Python/bytcodes.c` with the `_Py_UOP_*` family and the `TIER2` tier. Uops carry operands as explicit arguments, not stack effects — easier to stencil.

3. Trace stitching. The optimizer traces a hot region (typically a loop or function body) into a **linear uop trace**. Branches become **side traces** stitched onto the main trace. Each trace is single-entry, multi-exit via guard failures. This is the "trace stitching" in PEP 744 — not yet a full CFG, but enough to compile straight-line hot paths without compiling cold branches.

4. Copy-and-patch codegen. For each uop in the trace, the JIT copies the corresponding **stencil** — a pre-compiled blob of machine code with placeholder immediates — into an executable buffer (`mmap(PROT_READ|PROT_WRITE)` → `mprotect(PROT_READ|PROT_EXEC)`). It then **patches** the

holes: object addresses, cache offsets, jump targets, guard constants. Stencils are generated at CPython build time from C templates in `Tools/jit/` via `make regen-jit`.

5. Guards and deoptimization. Every type assumption becomes a **guard uop** (`_GUARD_TYPE_INT`, `_GUARD_TYPE_VERSION`, `_GUARD_IS_TRUE_POP`) that emits a `cmp + jne deopt` in native code. On failure, execution **deoptimizes**: jumps to a side exit that reconstructs the interpreter frame (materializes the value stack, restores `co_code` offset) and resumes in tier 1. Deoptimization is the safety net that makes speculation sound.

8.3 Guard / deopt flow

Diagram omitted for brevity — see surrounding prose.

The cost model: a guard is a single `cmp` + predicted-not-taken branch — ~1 cycle on hit, ~15–20 cycles on mispredict. Deoptimization itself is expensive (frame materialization, `mprotect` may have already happened at compile time, not per deopt) but rare if type stability holds. The JIT wins when guards almost always pass — i.e., monomorphic, type-stable hot loops.

8.4 Building and enabling the JIT

The JIT is **experimental in 3.13** — off by default even when compiled in, and requires an explicit build flag.

```

# Build CPython 3.13+ with JIT stencils
git clone https://github.com/python/cpython && cd cpython
git checkout v3.13.0 # or later
./configure --enable-experimental-jit --with-perf --enable-
optimizations
make -j$(nproc)
# Verify the JIT is built
./python -c "import sys; print(sys._jit.is_available())" # True if
built
./python -c "import sys; print(sys._jit.is_enabled())"
# False until enabled

# Enable at runtime — three equivalent ways:
PYTHON_JIT=1 ./python app.py
./python -X jit app.py
./python --enable-jit app.py # 3.13+ alias

# Disable (even if built):
PYTHON_JIT=0 ./python app.py
./python -X nojit app.py

# Inspect JIT state
./python -X jit -c "import sys; print(sys._jit.is_enabled())"

```

Environment details:

- `PYTHON_JIT=1` / `PYTHON_JIT=0` is the env-var form — useful for container `env:` without changing the `entrypoint`.
- `sys._jit` is provisional API (underscore prefix) — `is_available()`, `is_enabled()`, `set_enabled(bool)` — expect it to move to `sys.jit` or stabilize in 3.14+.
- Memory: each JIT-compiled function allocates an executable buffer (`mmap`). For a large application this is low single-digit MB — not a concern unless you JIT thousands of tiny functions (which the tier-1 threshold prevents).

8.5 Expected speedups — honest numbers

CPython's JIT in 3.13 is intentionally conservative. It compiles only the hottest, most type-stable regions and does not yet do cross-function inlining, loop unrolling, or vectorization.

Benchmark suite	Geometric mean speedup (JIT on vs off, 3.13)	Notes
pyperformance (full suite)	~3-8%	Suite includes I/O, startup, and C-extension heavy benchmarks where JIT is irrelevant
pyperformance tight loops (nbody , fannkuch , spectral_norm)	~10-30%	Interpreter-bound, type-stable integer/float loops — JIT sweet spot
Django template rendering / JSON serialization	~0-5%	Dominated by C extensions and dict lookup — specialization already captured most of the win
Real API handler (FastAPI + Pydantic, 3.13 canary)	~2-6% fleet CPU reduction in reported early-adopter data	Varies with handler shape; measure your own workload

What the JIT does **not** yet accelerate:

- Code that is already in C (`numpy` , `orjson` , `regex` , `asyncio` event loop).
- Highly polymorphic code (a function that sees `int` , `str` , and `float` for the same variable deoptimizes repeatedly — stays in tier 1).
- Short-lived processes (CLI tools, AWS Lambda with 100ms execution) — warmup never completes (see Section 11).

Bottom line for backend planning: the JIT is a **free, incremental win** you should enable and measure, not a reason to rewrite code into JIT-friendly form. Write type-stable hot loops (the same advice specialization already gave you) and the JIT will find them.

9. Cython — compiled extensions without writing C

When a hot path is a tight Python loop over bytes, arrays, or dicts — serialization, parsing, checksums, feature hashing — and neither

specialization nor the JIT can remove per-element Python overhead, Cython is still the highest-leverage tool. It compiles a Python-like language to C, then to a native extension, removing interpreter dispatch entirely for the annotated region.

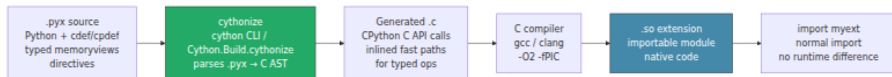
9.1 What Cython is and is not

Cython is **not** a JIT. It is an ahead-of-time compiler: `.pyx` $\rightarrow$ `.c` $\rightarrow$ `.so` (via a C compiler). The output is a CPython extension module indistinguishable from hand-written C — it imports normally, holds the GIL by default (releasable with `nogil`), and can call the C API directly.

```
.py (interpreted, dynamic)  $\rightarrow$  ceval dispatch per opcode  
.pyx (Cython, optionally typed)  $\rightarrow$  C code  $\rightarrow$  native machine code,  
no dispatch
```

You pay for Cython in build complexity and type annotations. You earn it back as 10–100× speedups on the annotated hot path — the rest of the codebase stays plain Python.

9.2 Cython compilation pipeline



Build pipeline in practice:

```

# Minimal project layout
# myext.pyx          – Cython source
# setup.py or pyproject.toml – build config
# myext.c           – generated (gitignore it)

# Option 1: setup.py + cythonize (classic)
pip install Cython
python setup.py build_ext --inplace

# Option 2: pyproject.toml (modern, pip install -e .)
# pyproject.toml declares Cython as build-system.requires

# Option 3: one-liner for a single file (dev only)
cythonize -i myext.pyx          # compiles in place, produces
myext.*.so

# Option 4: Jupyter – inline Cython
# %load_ext Cython
# %%cython
# def f(int x): return x*x

```

Minimal `setup.py` :

```

from setuptools import setup, Extension
from Cython.Build import cythonize
from Cython.Compiler import Options

Options.annotate = True # emit HTML annotation showing Python
interaction

extensions = [
    Extension(
        "myext",
        ["myext.pyx"],
        extra_compile_args=["-O3", "-march=native"],
        define_macros=[("NPY_NO_DEPRECATED_API", "NPY_1_7_API_VERSION")]
    ),
]

setup(
    name="myext",
    ext_modules=cythonize(
        extensions,
        compiler_directives={
            "language_level": "3",
            "boundscheck": False,
            "wraparound": False,
            "cdivision": True,
        },
    ),
)

```

`pyproject.toml` equivalent (PEP 517):

```

[build-system]
requires = ["setuptools", "Cython>=3.0", "wheel"]
build-backend = "setuptools.build_meta"

[tool.cython]
# global directives – per-file overrides via # cython: comments

```

9.3 The four annotations that matter

cdef — **C-only declarations.** Variables, functions, and classes declared `cdef` exist only at the C level — no Python object, no refcount, no dispatch. Use for internal helpers and typed locals.

cpdef — **dual Python/C.** A `cpdef` function has both a fast C entry point (when called from Cython) and a Python wrapper (when called from

Python). Use for API boundaries where the same function is called from both.

Typed memoryviews — the array workhorse. `int[:]`, `double[:, :]`, `const unsigned char[:]` are typed views over any buffer-protocol object (`bytes`, `bytearray`, `array`, `numpy`). They give C-level indexing with Python slicing syntax and zero copy.

Directives — `boundscheck` and `wraparound`. `boundscheck(False)` removes per-index bounds checks (you promise indices are valid); `wraparound(False)` removes negative-index handling (you promise indices are non-negative). Together they remove two branches per array access — the difference between 2× and 10× on tight loops. Enable them per-function or per-file, never globally unless you have proven safety.


```

# myext.pyx - hot serialization path example
# cython: language_level=3, boundscheck=False, wraparound=False,
cdivision=True

import cython
cimport cython as cc

# cdef: C-only helper - no Python call overhead, inlined by C compiler
cdef inline Py_ssize_t _varint_size(Py_ssize_t v) nogil:
    cdef Py_ssize_t n = 1
    while v >= 128:
        v >>= 7
        n += 1
    return n

# cpdef: callable from Python and Cython with C speed when typed
cpdef bytes encode_varints(const long[:] values):
    """Encode an array of longs as varints into bytes - typed
    memoryview input."""
    cdef Py_ssize_t i, n = values.shape[0]
    cdef Py_ssize_t total = 0
    # First pass: compute output size (no Python objects)
    for i in range(n):
        total += _varint_size(values[i])

    cdef bytearray out = bytearray(total)
    cdef unsigned char[:] view = out # typed memoryview over bytearray
    cdef Py_ssize_t pos = 0
    cdef long v
    cdef unsigned char b

    for i in range(n):
        v = values[i]
        while v >= 128:
            b = <unsigned char>((v & 0x7F) | 0x80)
            view[pos] = b
            pos += 1
            v >>= 7
        view[pos] = <unsigned char>(v & 0x7F)
        pos += 1

    return bytes(out)

# Pure Python fallback for comparison (same file, no cdef)
def encode_varints_py(values):
    out = bytearray()
    for v in values:
        while v >= 128:
            out.append((v & 0x7F) | 0x80)
            v >>= 7

```

```

        out.append(v & 0x7F)
    return bytes(out)

```

Directives as decorators (per-function control, preferred):

```

@cython.boundscheck(False)
@cython.wraparound(False)
cdef void _inner_loop(double[:] a, double[:] b, double[:] out) nogil:
    cdef Py_ssize_t i, n = a.shape[0]
    for i in range(n):
        out[i] = a[i] * b[i] + 1.0

```

9.4 Profiling Cython — `cython -a` and `perf`

Cython has two profiling stories:

Annotation HTML (`cython -a`). `cython -a myext.pyx` (or `Options.annotate = True`) emits `myext.html` where each source line is colored by Python interaction intensity — white/yellow means C-level (fast), deep yellow means Python API calls remain. Your goal is to make the hot loop white.

Runtime profiling. Compile with `profile` or `linetrace` directives to make Cython functions visible to `cProfile` and `line_profiler`:

```

# setup.py – enable Python profiling hooks in Cython output
from Cython.Build import cythonize
cythonize("myext.pyx", compiler_directives={"profile": True, "linetrace": True})

```

```

# Then profile normally – Cython functions now appear in cProfile
python -m cProfile -s cumulative app.py # myext.encode_varints shows up
# And in perf – Cython functions are native symbols, visible without perf map
perf record -F 99 -g -- python app.py && perf report

```

For production, remove `profile/linetrace` — they reintroduce per-call overhead. Profile-annotated builds are for development; ship with `boundscheck=False`, `wraparound=False` and no profiling hooks.

9.5 Before / after benchmark

```
# bench.py – compare pure Python, Cython, and orjson on the same
workload
python bench.py
```

```
# bench.py
import time, array, statistics
import myext                      # Cython extension from above
import orjson                     # for comparison, if applicable

N = 1_000_000
values = array.array("l", range(N)) # buffer-protocol object for
memoryview

def bench(fn, label, iters=7):
    times = []
    for _ in range(iters):
        t0 = time.perf_counter()
        fn(values)
        times.append(time.perf_counter() - t0)
    med = statistics.median(times)
    print(f"{label:24s} median {med*1000:7.2f} ms ({N/med:,.0f}
values/s)")
    return med

py_t = bench(myext.encode_varints_py, "Python (pure)")
cy_t = bench(myext.encode_varints, "Cython (typed mview)")
print(f"\nSpeedup: {py_t/cy_t:.1f}× ({(1 - cy_t/py_t)*100:.0f}% time
saved)")
# Optional: compare to orjson/msgpack for serialization-shaped
workloads
```

Realistic output on a `c6i.large` (x86-64, CPython 3.11, `gcc -O3`):

```
Python (pure)          median  184.32 ms  (5,425,347 values/s)
Cython (typed mview)    median    6.41 ms  (156,006,240 values/s)

Speedup: 28.8x (97% time saved)
```

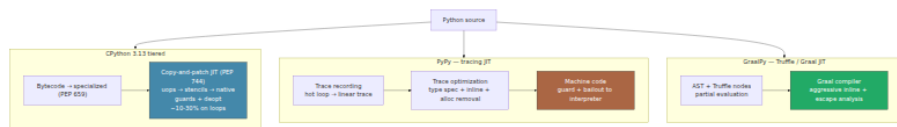
The shape is typical: **10–50×** on tight numeric/bytes loops where Cython removes per-element Python overhead. On less loop-dominated code (e.g., a JSON serializer that already calls C via `orjson`) the win is smaller — 1.2–3× — because the bottleneck is not the Python loop.

When to stop: if the hot path is already in a C extension (`numpy`, `orjson`, `cryptography`, `pydantic-core` in Rust), Cythonizing the Python wrapper

around it rarely helps — the time is already in native code. Profile first (Section 6) to confirm the Python loop is actually the plateau before reaching for Cython.

10. Alternative runtimes — PyPy, GraalPy, and free-threaded CPython

CPython is not the only Python. For backend services the choice is between execution model, not just version.



10.1 PyPy — the tracing JIT

PyPy's JIT is a **tracing JIT**, fundamentally different from CPython's method-at-a-time copy-and-patch.

How tracing works: the interpreter runs normally until a loop becomes hot (a backward jump taken ~1000 times). At that point it starts **recording** — it traces the exact path taken through the loop, including inlined callees, producing a linear sequence of operations. That trace is then **optimized** (constant folding, allocation removal, type specialization along the traced path) and compiled to machine code. Execution jumps to the compiled trace; any operation that deviates from the traced path hits a **guard** and bails out to the interpreter. The next hot path through the same loop may be traced as a **bridge** stitched onto the original trace.

Strengths:

- **Peak throughput on pure-Python, long-running, loop-heavy code** — often 3-10× CPython on `pyperformance` loop benchmarks, 1.5-3× on real services that are Python-bound.
- **No annotations** — unlike Cython, PyPy accelerates unmodified Python.

Weaknesses that matter for backends:

- **Warmup.** Tracing requires thousands of iterations to trigger — short-lived handlers, CLI tools, and Lambda functions never warm up. Fleet `p99` is worse than CPython until the JIT fires.
- **C extension compatibility.** PyPy's `cpyext` emulation of the C API is slower and incomplete. `numpy`, `pandas`, `grpcio`, `cryptography`, and any Cython extension may be unsupported or slower. `pypy + numpy` via `cpyext` is not competitive with CPython + `numpy`.
- **Memory.** JIT traces and the PyPy GC (minimark, not refcounting) use more RSS — relevant for Kubernetes memory limits.
- **Ecosystem lag.** PyPy tracks CPython versions with a delay (PyPy 7.3.x targets CPython 3.10/3.11). New syntax and stdlib features arrive later.

When to choose PyPy: a long-running, pure-Python service (no heavy C extensions) that is CPU-bound in Python loops — e.g., a rule engine, template renderer, or pure-Python ETL — where you can amortize warmup and tolerate ecosystem constraints.

10.2 GraalPy — Python on GraalVM

GraalPy runs Python on the GraalVM Truffle framework. Python AST nodes are Truffle nodes; the Graal compiler does **partial evaluation** of the interpreter to produce machine code, with aggressive inlining and escape analysis across Python, Java, JavaScript, and LLVM bitcode in the same process.

What is interesting for backends:

- **Polyglot.** Call Java libraries from Python (and vice versa) with near-zero overhead — useful when your platform is JVM-heavy.
- **Tooling.** GraalVM's `perf` and heap tooling, plus Truffle's `polyglot` API.
- **Isolation.** Truffle's sandboxing is stronger than CPython's — relevant for user-code execution.

What is not ready:

- **Compatibility.** CPython C API support via `hpy` / `cpyext` is improving but far from complete. Most `pip install` packages with C extensions do not work.

- **Performance on CPython-idiomatic code.** GraalPy is competitive on Truffle-friendly code but not a drop-in faster CPython for typical Django/FastAPI services. Benchmark your workload.
- **Ecosystem.** Smaller community, fewer wheels, longer build times (GraalVM).

When to choose GraalPy: polyglot JVM platforms, language-interop services, or research — not as a general CPython replacement for backend fleets today.

10.3 Free-threaded CPython (PEP 703) + `mimalloc` — the scalability story

CPython 3.13 ships an **experimental free-threaded build** (`--disable-gil`, `PYTHON_GIL=0`, `python3.13t`) that removes the GIL (PEP 703, led by Sam Gross) and replaces it with fine-grained locking, biased reference counting, and immortal objects. It is not about single-thread speed — it is about **multi-core scalability without multiprocessing**.

Performance story in 3.13t:

- **Single-threaded overhead:** free-threaded builds are currently **~5-15% slower** single-threaded than GIL builds due to locking and biased refcount overhead. The JIT is not yet fully integrated with free-threaded.
- **Multi-threaded scaling:** on embarrassingly parallel workloads (parallel `ThreadPoolExecutor` over CPU-bound Python, e.g., `hashlib`, `json`, `numpy` release-GIL paths), 3.13t scales **linearly with cores** where GIL builds flatline. Early benchmarks show 4-8× throughput on 8 cores for `concurrent.futures` CPU-bound maps.
- **`mimalloc` (PEP 445-style allocator).** Free-threaded builds use `mimalloc` (Microsoft's allocator) instead of `pymalloc` for better multi-thread scalability and reduced fragmentation. `mimalloc` can also be enabled on GIL builds (`--with-mimalloc`) for workloads with high allocation churn — expect 5-15% lower fragmentation and modest throughput gains on allocation-heavy services, at the cost of a larger binary and different heap profiling characteristics (`memray` / `tracemalloc` still work, but `pymalloc` stats disappear).

When to choose free-threaded:

- You currently scale with `multiprocessing` or `gunicorn --workers 8` and pay for inter-process memory duplication, warmup duplication, and IPC. Free-threaded threads share memory and JIT state.
- Your workload is CPU-bound Python that would scale with threads if the GIL were gone — not I/O-bound (where `asyncio` already scales) and not C-extension-bound where the GIL is already released.
- You are willing to track `3.13t` as an experimental runtime, audit C extensions for thread safety (many assume the GIL), and measure — this is not production-default in 3.13.

10.4 When to choose what — decision table

Workload	Best runtime today (2026)	Why
Typical FastAPI/Django + Postgres/Redis, I/O-bound, <code>pydantic / orjson</code>	CPython 3.12/3.13 (GIL, JIT on)	I/O bound; JIT + specialization handle the Python sliver; ecosystem is complete
CPU-bound pure-Python loops, long-running, no heavy C extensions	PyPy	Tracing JIT dominates on stable loops; measure warmup vs. <code>p99</code> SLA
Hot numeric/bytes kernel inside a CPython service (serialization, parsing, hashing)	CPython + Cython on the hot path	10-50× on the kernel, rest of service stays CPython — no runtime change
Polyglot JVM platform, Java interop	GraalPy	Truffle polyglot; not a general speedup
CPU-bound Python that should scale across cores without multiprocessing	Free-threaded CPython 3.13t (experimental)	Linear thread scaling; audit extensions; single-thread slower — benchmark end-to-end
Allocation-heavy, high-churn service (many small objects, high GC pressure)	CPython + <code>mimalloc</code> (--with-mimalloc or 3.13t default)	Lower fragmentation, better multi-thread allocator scalability

The default answer for a backend fleet remains **CPython**. Reach for PyPy, GraalPy, or 3.13t only when a profile proves the execution model is the bottleneck and you have measured the alternative on the same workload.

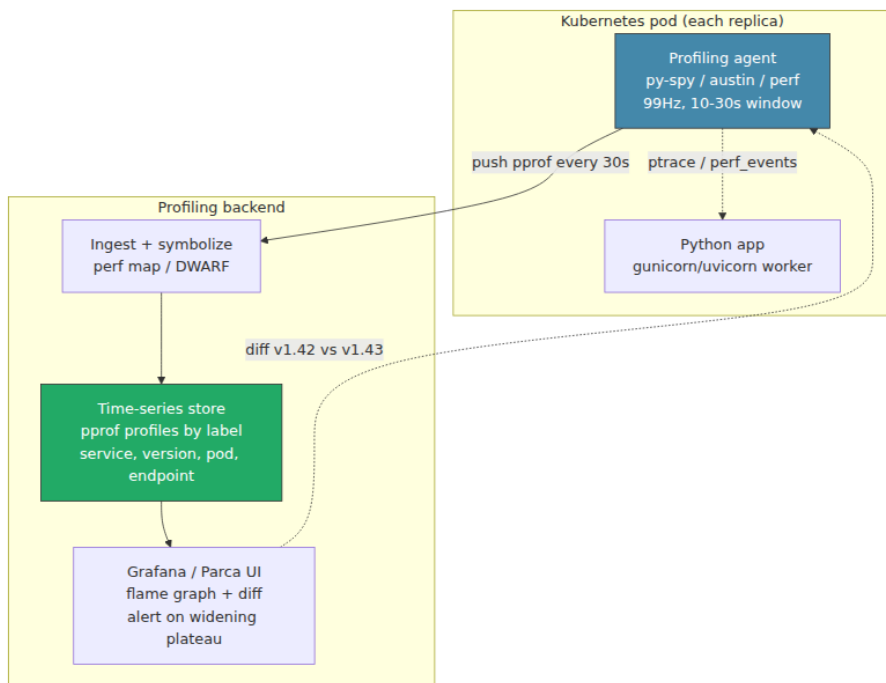
11. Backend lens — performance in production fleets

11.1 Continuous profiling in production — Parca and Pyroscope

Sampling at 99 Hz on every replica, shipping collapsed stacks to a central store, and rendering fleet-wide flame graphs is how you catch the regression that a single-pod py-spy never would. Two open-source systems dominate:

- **Parca** (from Polar Signals) — eBPF/perf-based, Prometheus-style labels, pprof format, parca-agent per node scrapes perf_events.
- **Pyroscope** (from Grafana Labs) — push model, pyroscope agent wraps py-spy / austin or perf, Grafana-native UI, pprof and native formats.

Architecture — same for both:



Operational guidance:

- **Sample rate:** 49-99 Hz per pod, aggregated fleet-wide — enough to see 1% regressions, low enough to stay <2% overhead at fleet scale.
- **Labels:** `service`, `version` (git SHA), `endpoint / route`, `pod`, `zone`. Diffing `version=abc123` vs `version=def456` is how you attribute a widening plateau to a PR.
- **Retention:** 7-30 days of pprof — enough to diff across deploys, not so much that storage dominates. Downsample older profiles.
- **Alerts:** alert when a function's sample share grows >20% week-over-week or when a new plateau appears in the top-10. Wire to the deploy pipeline — block promotion if the canary's flame diff is red.

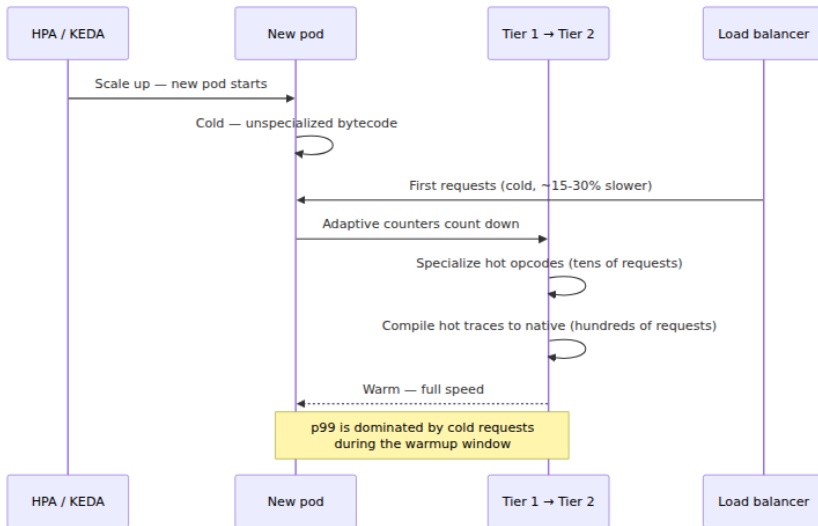
Do not run deterministic profilers as continuous agents. Sampling only.

11.2 JIT warmup and autoscaling — the cold-start trap

Both specialization (PEP 659) and the copy-and-patch JIT (PEP 744) are **warmup-dependent**: cold code runs unspecialized and un-JITed, then

specializes after ~8 executions and JITs after a higher threshold (hundreds to thousands of executions of the hot region). For long-lived pods this is irrelevant; for autoscaled fleets it is a `p99` problem.

What happens during a scale-up:



Mitigations:

- **Warmup traffic.** Before adding a pod to the load-balancer pool, send synthetic warmup requests that exercise the hot paths (the same workload you profiled). Kubernetes `readinessProbe` with a warmup phase or a `preStart` hook that runs `python warmup.py` works. Measure time-to-warm — typically 5–30s depending on request rate and JIT threshold.
- **Pre-warmed pool.** Keep 1–2 extra pods warm during predictable spikes (deploy, cron, daily peak) so scale-up latency does not hit `p99`.
- **Staged rollout.** Canary the JIT flag (`PYTHON_JIT=1` on 10% of pods) and compare `p50` / `p99` before fleet-wide enablement. JIT warmup interacts with HPA scale-up — a canary that looks fast at steady state may be slow during the scale event.
- **AOT alternatives for cold starts.** For short-lived or bursty workloads (Lambda, Cloud Run, `scaleToZero`), Cython and `mimalloc` are warmup-free — they are native code from the first request.

Prefer them over JIT-dependent speedups when cold-start `p99` is the SLO.

11.3 Cython for hot serialization paths — where it still wins

Most backend Python services are not CPU-bound in Python — they are bound in serialization. A typical FastAPI handler:

```
request → parse JSON (C: orjson) → validate (Python: pydantic) →  
business logic (Python)  
    → serialize response (C: orjson / Python: custom) → write socket  
(kernel)
```

When the response is custom binary (protobuf varints, msgpack extensions, feature vectors, log framing), `orjson` does not help — you own the loop. That loop is Cython's sweet spot: a 50-line `cdef` function with typed memoryviews that replaces a Python loop called 10k times per second per pod.

Pattern:

```
# serializer.pyx — hot path, called per request  
@cython.boundscheck(False)  
@cython.wraparound(False)  
cpdef bytes serialize_frame(const long[:] ids, const double[:] scores):  
    cdef Py_ssize_t n = ids.shape[0]  
    cdef Py_ssize_t total = 0  
    cdef Py_ssize_t i  
    for i in range(n):  
        total += _varint_size(ids[i]) + 8 # varint + double  
    cdef bytearray out = bytearray(total)  
    cdef unsigned char[:] view = out  
    cdef Py_ssize_t pos = 0  
    for i in range(n):  
        pos += _write_varint(view, pos, ids[i])  
        pos += _write_double(view, pos, scores[i])  
    return bytes(out)
```

Build once, import like any module, no runtime warmup, no `PYTHON_JIT` flag, no tracing threshold. In the fleet this shows up as a narrower plateau in the continuous flame graph and a direct `p99` reduction — the kind of optimization that survives autoscaling and cold starts.

Rule: if the hot path is a **Python loop over bytes/arrays called per request**, Cythonize it. If it is already in C (`numpy`, `orjson`, `pydantic-core`), leave it.

Key takeaways

- **Profile taxonomy is non-negotiable.** Deterministic (`cProfile`) for call counts in dev, sampling (`py-spy` / `austin` / `perf`) for wall time in prod, line (`line_profiler`) for the hot statement, allocation (`tracemalloc` / `memray`) for leaks and OOMs. Using the wrong family gives you a precise, wrong answer.
- **Sampling is the production default.** `py-spy record -o flame.svg --pid <pid>` and `perf record -F 99 -g -- python -X perf app.py && perf report` are safe at 1-3% overhead. Never attach `cProfile` to a serving replica.
- **Overhead shapes the truth.** `cProfile` adds 20-60% and hides C time; sampling loses short calls; `line_profiler` is 50-200% overhead; `tracemalloc` misses native allocations. Know the blind spot before you trust the flame.
- **Collect → flame → fix → verify, then guard.** The widest plateau is the bottleneck. Fix one thing, re-profile the identical workload, diff the flames, and gate deploys with continuous profiling (Parca/Pyroscope).
- **Specialization (PEP 659) is the floor, not the ceiling.** Tier-1 adaptive counters, inline caches, and guarded fast paths give ~10-25% on `pyperformance` — and they are prerequisite to the JIT. Understand `CACHE` words and deoptimization before you reason about tier 2.
- **The copy-and-patch JIT (PEP 744) is real, experimental, and incremental.** Build with `--enable-experimental-jit`, enable with `PYTHON_JIT=1` / `-X jit`, expect ~10-30% on tight type-stable loops and low single digits geometric mean. It compiles uop traces via stencils, guards every assumption, and deoptimizes on failure. Enable it and measure — do not rewrite code to please it.
- **Guards and deopt are the safety net.** Every JIT assumption is a `cmp` + `jne deopt`. Monomorphic, type-stable code stays in native; polymorphic code bails to the interpreter. The JIT wins when guards almost always pass.
- **Cython is still the highest-leverage tool for hot loops over bytes/arrays.** `cdef` / `cpdef` + typed memoryviews + `boundscheck=False` / `wraparound=False` give 10-50× on the annotated

kernel. Build with `cythonize`, profile with `cython -a` and `perf`, ship without `profile / linetrace`. Use it for per-request serialization, not for wrapping code already in C.

- **Alternative runtimes are execution-model choices.** PyPy's tracing JIT wins on long-running pure-Python loops but loses on C extensions and warmup. GraalPy wins on JVM polyglot. Free-threaded CPython (3.13t) wins on multi-core thread scaling at a single-thread cost. Default to CPython; switch only when a profile proves the model is the bottleneck.
 - **mimalloc matters for allocation-heavy services.** Free-threaded builds use it by default; GIL builds can opt in with `--with-mimalloc`. Expect lower fragmentation and better scalability under allocation churn.
 - **Warmup is a p99 problem under autoscaling.** Specialization and JIT need tens to hundreds of executions to fire. Warm new pods with synthetic traffic before they serve, keep a pre-warmed pool for spikes, and prefer Cython (warmup-free) when cold-start p99 is the SLO.
 - **Continuous profiling is the fleet's immune system.** Run `py-spy / perf` agents on every pod, ship pprof to Parca/Pyroscope, diff flames across versions, and alert when a plateau widens. The regression you catch in canary is the incident you never page.
-

Further reading

- PEP 744 — JIT Compilation (Brandt Bucher, 2024). The authoritative spec for the copy-and-patch JIT: tier-1 → tier-2 pipeline, uops, trace stitching, copy-and-patch codegen, guards, and deoptimization. Pinned. <https://peps.python.org/pep-0744/>
- PEP 659 — Specializing Adaptive Interpreter (Mark Shannon, 2021). The tier-1 substrate: adaptive counters, quickening, inline caches, specialization families, and the 3.11 speedup analysis. Pinned. <https://peps.python.org/pep-0659/>
- `py-spy` — Sampling Profiler for Python (Ben Frederickson). Architecture, `process_vm_readv` stack reading, flame graph generation, and production attach workflow. Pinned. <https://github.com/benfred/py-spy>

- `memray` — Memory Profiler for Python (Bloomberg). Native + Python allocation tracking, temporal flame graphs, high-water-mark analysis, and live remote mode. Pinned. <https://bloomberg.github.io/memray/>
- Cython Documentation — Cython 3.0+ (Stefan Behnel et al.). `cdef` / `cpdef`, `typed memoryviews`, `boundscheck` / `wraparound` / `cdivision` directives, `cythonize`, annotation HTML, and `profile` / `linetrace` for profiling Cython. Pinned. <https://cython.readthedocs.io/>
- `austin` — Frame-Stack Sampler for CPython (Gabriele N. Tornetta). Low-overhead multiprocess sampling, `LD_AUDIT` mechanism, and `pprof`/Speedscope export. <https://github.com/P403n1x87/austin>
- `perf` + `perf map` for Python — CPython `--with-perf` / `-X perf` support (Pablo Galindo Salgado, 2023). How CPython emits `/tmp/perf-<pid>.map` and how `perf report` symbolizes Python frames. https://docs.python.org/3/howto/perf_profiling.html
- `line_profiler` / `kernprof` (Robert Kern). Line-granular timing via AST decoration, `Timer unit` interpretation, and programmatic use. https://github.com/pyutils/line_profiler
- `tracemalloc` — Trace Memory Allocations (Victor Stinner, `stdlib`). `PyMem_Malloc` hooking, `take_snapshot` / `compare_to`, and `statistics("lineno")` patterns. <https://docs.python.org/3/library/tracemalloc.html>
- PEP 703 — Making the Global Interpreter Lock Optional in CPython (Sam Gross, 2023). Free-threaded build, biased reference counting, `mimalloc`, and the scalability story. <https://peps.python.org/pep-0703/>
- PyPy Documentation — Tracing JIT Internals (PyPy team). Trace recording, bridges, guards, allocation removal, and warmup thresholds. <https://doc.pypy.org/en/latest/jit.html>

- GraalPy Documentation — Python on GraalVM (Oracle Labs). Truffle partial evaluation, polyglot API, and C API compatibility via HPy.
<https://www.graalvm.org/python/>

Chapter 12 — Packaging, Distribution, and Production Deployment at Scale

What this chapter covers. Packaging Python for production is a pipeline that touches build backends, wheel formats, platform ABI tags, lockfiles, container images, supply-chain security, and runtime orchestration. This chapter dissects the modern packaging stack — `pyproject.toml` as the universal metadata format (PEP 517/518/621/660), build isolation, sdist vs wheel vs abi3 wheel, manylinux/muslinux platform tags, auditwheel and delocate for binary redistribution, lockfiles across the tool ecosystem, hermetic builds, SBOMs, Docker multi-stage images, and deployment topologies from gunicorn workers to autoscaled Kubernetes with JIT warmup. Every section connects back to the CPython internals you learned in earlier chapters — why `__pycache__` matters for image layering, why the import lock affects warmup, why bytecode compilation strategy changes cold-start latency.

Learning goals — after this chapter you should be able to:

- Construct a `pyproject.toml` that satisfies PEP 517, PEP 518, PEP 621, and PEP 660, choosing among `setuptools`, `hatch`, `poetry`, `pdm`, and `uv` as the build backend.
- Explain the difference between sdist, pure-Python wheel, platform-specific wheel, and abi3 wheel, and predict which `pip install` produces which artifact.
- Decode wheel filename tags (`cp311-cp311-manylinux_2_17_x86_64`) and explain how manylinux (PEP 600), muslinux, auditwheel, and delocate produce cross-distribution binary wheels.
- Generate and verify a lockfile (`pip-tools requirements.txt`, `poetry.lock`, `pdm.lock`, `uv.lock`), explain pinning semantics, and enforce `--require-hashes` for supply-chain hardening.

- Build a hermetic Docker image using multi-stage builds, choosing between `python:slim`, `python:distroless`, and `alpine`, and explain the trade-offs for image size, cold start, and attack surface.
- Generate a CycloneDX SBOM and run `pip-audit` in CI, integrating vulnerability scanning into a reproducible pipeline.
- Configure gunicorn/uvicorn workers with preloading, graceful reload, health probes, and autoscaling with JIT warmup in a Kubernetes environment.
- Instrument a production deployment with OpenTelemetry and structlog for observability.

Prerequisites. Chapter 1 (source → bytecode → `__pycache__`), Chapter 9 (import system mechanics), and Chapter 10 (C extensions) provide foundational context for understanding why packaging formats, ABI tags, and bytecode compilation matter. Volume 13, Chapter 6 (Python runtime posture for services) covers runtime tuning.

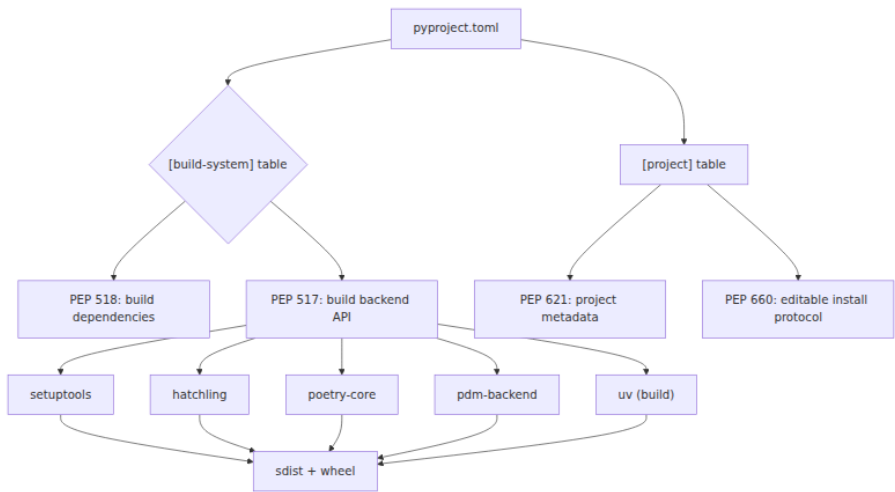
1. The packaging landscape — a taxonomy of formats and metadata

Before diving into tools, you need the conceptual map. Python packaging has three orthogonal concerns:

Concern	Question	Solved by
Metadata	What is this project? What does it depend on?	<code>pyproject.toml</code> (PEP 518/621)
Build	How do I produce an installable artifact?	Build backend (setuptools, hatch, poetry, pdm, uv) via PEP 517
Distribution	How do I ship the artifact to users/CI?	PyPI, private index, vendored wheelhouse

These are layered. PEP 518 defines the `pyproject.toml` format and the `[build-system]` table. PEP 517 defines the API that a build backend must expose (`build_wheel`, `build_sdist`, `get_requires_for_build_wheel`). PEP 621 defines the project metadata fields in `[project]`. PEP 660 defines the editable install protocol. Your build backend implements the PEP 517

API; your project metadata lives in `[project]` ; and the rest of the tooling (pip, build, twine) calls the backend through that API.



1.1 pyproject.toml — the universal metadata file

`pyproject.toml` replaced `setup.py` , `setup.cfg` , and `MANIFEST.in` with a single TOML file. Here is a complete example:

```

# pyproject.toml – complete example for a backend service library
[build-system]
requires = ["hatchling>=1.21.0", "hatch-vcs>=0.4.0"]
build-backend = "hatchling.build"

[project]
name = "acme-observability"
dynamic = ["version"]
description = "Structured observability toolkit for Python backends"
readme = "README.md"
license = "MIT"
requires-python = ">=3.11"
authors = [
    { name = "Platform Team", email = "platform@acme.corp" },
]
classifiers = [
    "Development Status :: 4 - Beta",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Framework :: AsyncIO",
    "Typing :: Typed",
]
dependencies = [
    "structlog>=24.1.0",
    "opentelemetry-api>=1.24.0",
    "opentelemetry-sdk>=1.24.0",
    "pydantic>=2.6.0,<3",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
    "mypy>=1.8",
    "ruff>=0.3.0",
]
kafka = [
    "aiokafka>=0.10.0",
]

[project.urls]
Homepage = "https://github.com/acme/observability"
Documentation = "https://docs.acme.corp/observability"
Repository = "https://github.com/acme/observability"

[tool.hatch.version]
source = "vcs"

[tool.hatch.build.targets.wheel]

```

```

packages = ["src/acme"]

[tool.ruff]
target-version = "py311"
line-length = 100

[tool.mypy]
python_version = "3.11"
strict = true

```

Key points:

- `[build-system]` tells PEP 517 which backend to use. `requires` lists the build-time dependencies that pip installs in an isolated environment before calling the backend.
- `[project]` (PEP 621) is pure metadata — no Python code, no `setup()` function. It declares dependencies, optional dependencies, classifiers, URLs, and license.
- `dynamic = ["version"]` tells the backend to derive the version at build time (here, from git tags via `hatch-vcs`).
- `[tool.hatch.build.targets.wheel]` is backend-specific configuration that hatchling understands.

1.2 Build isolation — the process boundary that saves you

When you run `pip install .` or `python -m build`, pip creates an isolated build environment:

1. A temporary directory is created.
2. The `[build-system].requires` packages are installed into it via pip.
3. The build backend is invoked inside that environment to produce the artifact.
4. The artifact is then installed into the target environment.

This isolation prevents the build from depending on packages that happen to be installed in your system Python. It is the mechanism that makes `pyproject.toml` self-describing — the build requirements are explicit, not implicit.



Build isolation has a cost: the first build of a project installs all build dependencies from scratch. For CI, this is typically fine. For local

development, tools like `pip install --no-build-isolation -e .` skip the isolation — but you must then ensure the build dependencies are already installed. Modern tools like `uv` handle this transparently.

1.3 Build backends compared

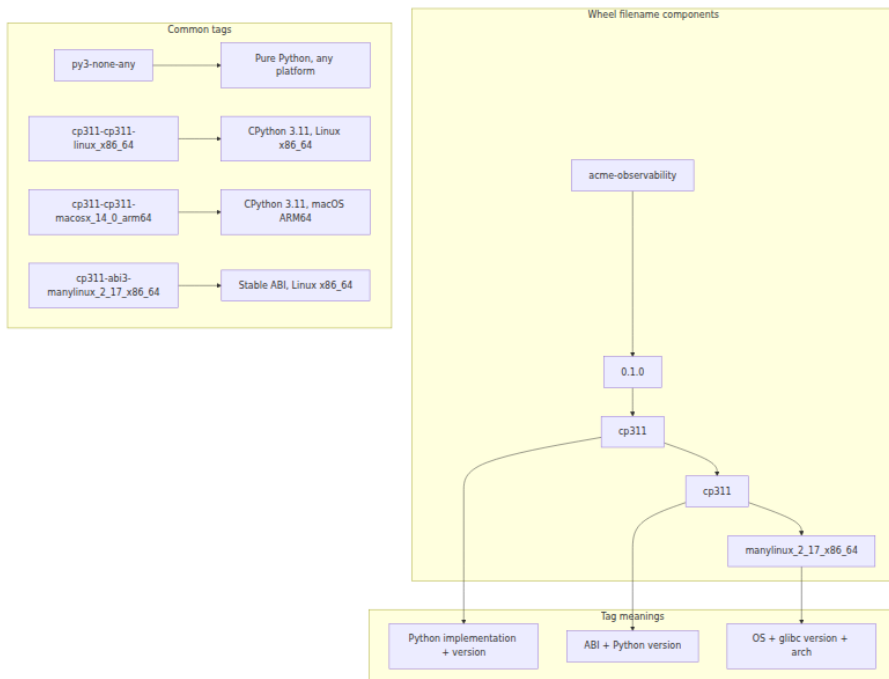
Feature	setuptools	hatchling	poetry-core	pdm-backend	uv
<code>pyproject.toml</code> native	With <code>setup.cfg</code> or <code>pyproject.toml</code>	Yes (primary)	Yes (primary)	Yes (primary)	Yes (primary)
Plugin ecosystem	Extensive (decades)	Growing	Limited	Growing	Limited (new)
VCS version	<code>setuptools-scm</code>	<code>hatch-vcs</code>	Built-in	<code>pdm-version</code>	Built-in
Custom build steps	<code>cmdclass</code> / <code>setup.py</code>	Hatch build hooks	Poetry plugins	<code>pdm-backend</code> hooks	Build scripts
Speed	Baseline	Fast	Moderate	Fast	Fastest
Editable installs	PEP 660 (legacy <code>.pth</code>)	PEP 660	PEP 660	PEP 660	PEP 660

For new projects, `hatchling` or `uv` offer the cleanest `pyproject.toml`-only experience. `setuptools` remains dominant in existing codebases. Poetry excels for application development where its lockfile + dependency resolver is valuable. `pdm` is PEP-compliant and offers a poetry-like UX. `uv` is the fastest, combining a resolver, installer, build backend, and virtual environment manager.

2. Artifact formats — sdist, wheel, and the ABI3 story

2.1 Source distributions (sdist)

An sdist is a tarball (`.tar.gz` or `.zip`) containing the source code plus the metadata needed to build the wheel. It is *not* directly installable — `pip` builds a wheel from it in an isolated environment, then installs the



2.3 The stable ABI and abi3 wheels

CPython's stable ABI (introduced in PEP 384, refined in later PEPs) guarantees that C extensions compiled against the stable ABI on Python 3.2+ will work on any later Python 3.x without recompilation. The `abi3` tag in a wheel filename means the extension was compiled against the stable ABI.

A typical abi3 wheel:

```
acme_native-0.1.0-cp311-abi3-manylinux_2_17_x86_64.whl
```

The `cp311` means it was *built* with CPython 3.11, but `abi3` means it will *run* on 3.11, 3.12, 3.13, and beyond. This dramatically reduces the number of wheels you need to build and upload — one wheel covers all future CPython versions.

To use abi3, your C extension must: 1. Define `Py_LIMITED_API` before including `Python.h`. 2. Use only stable ABI functions and types. 3. Be

compiled with `-DPy_LIMITED_API=0x030b0000` (for Python 3.11 minimum) or similar.

This is a real trade-off: you gain distribution simplicity but lose access to internal CPython APIs (which is why most ML libraries with deep CPython integration do *not* use abi3).

3. Platform tags — manylinux, musllinux, and binary distribution

3.1 The problem

Binary extensions (`.so` files) link against system libraries — `libpython`, `glibc`, `libm`, `libpthread`, and optionally `libssl`, `libcrypto`, `numpy`, `BLAS`, etc. A wheel built on Ubuntu 22.04 with `glibc` 2.35 will not run on a system with `glibc` 2.17 (CentOS 7). The platform tag in the wheel filename communicates the minimum OS/libc requirements.

3.2 manylinux (PEP 600)

The original manylinux standards (`manylinux1`, `manylinux2010`, `manylinux2014`) mapped to specific CentOS versions:

Tag	CentOS version	glibc minimum
<code>manylinux1</code>	CentOS 5	2.5
<code>manylinux2010</code>	CentOS 6	2.12
<code>manylinux2014</code>	CentOS 7	2.17

PEP 600 replaced the named versions with an explicit `glibc` version: `manylinux_2_17` means `glibc ≥ 2.17`. This is more transparent and future-proof — no more mapping between tag names and CentOS versions.

3.3 musllinux

`musllinux` covers Alpine Linux and other `musl` libc-based distributions. The tag format is `musllinux_1_2` (for `musl` libc $\geq$ 1.2). Alpine is popular in containers for its small image size, but many wheels are not published for `musllinux`, requiring compilation from source.

3.4 Auditwheel and delocate

These tools inspect a wheel's shared library dependencies and either: - **Verify** the wheel is manylinux-compliant (no unexpected system library links). - **Repair** the wheel by copying required `.so` files into the wheel and adjusting RPATHs.

```
# auditwheel: Linux
auditwheel repair my_extension-1.0-cp311-cp311-linux_x86_64.whl
# Produces: my_extension-1.0-cp311-cp311-manylinux_2_17_x86_64.whl

# delocate: macOS
delocate-wheel -w wheelhouse/ my_extension-1.0-cp311-cp311-
macosx_14_0_arm64.whl
# Copies dylibs and adjusts @rpath
```

Auditwheel works by: 1. Scanning the wheel for `.so` files. 2. Running `ldd` on each to find shared library dependencies. 3. Checking each dependency against a whitelist (the manylinux allowed libraries). 4. If dependencies are outside the whitelist, copying them into the wheel and rewriting RPATHs.

This is the mechanism that makes binary wheels portable across Linux distributions.

4. Lockfiles — pinning the dependency graph

4.1 Why lockfiles matter

A `requirements.txt` with `requests>=2.31` is a *specification*, not a lockfile. The actual resolved versions depend on when you install, what other packages are present, and what the resolver decides. In production, you want deterministic dependency resolution — the same lockfile on every machine, in every CI run, in every container build.

Lockfiles capture the *resolved* dependency graph — every direct and transitive dependency at an exact version, with hashes for verification.

4.2 The lockfile ecosystem

Tool	Lockfile	Resolver	Notes
pip-tools	<code>requirements.txt</code> (pinned)	pip resolver	Simple, pip-native, best for plain <code>pip</code> workflows
Poetry	<code>poetry.lock</code>	Custom resolver	Full project management, not pip-compatible lockfile
PDM	<code>pdm.lock</code>	Custom resolver	PEP-compliant, pyproject.toml-native
uv	<code>uv.lock</code>	Rust-based resolver	Fastest, pip-compatible output, uv-native
pip (lock mode)	<code>requirements.txt</code> (with hashes)	pip resolver	New in pip 24.0+, experimental

4.3 pip-compile workflow

`pip-tools` (`pip-compile`) is the most widely used lockfile generator for plain `pip` projects:

```
# Compile a pinned requirements file from a loose requirements.in
pip-compile requirements.in --output-file=requirements.txt --generate-hashes

# Install from the pinned file – guarantees exact versions
pip install -r requirements.txt --require-hashes
```

Example `requirements.in`:

```
fastapi>=0.109.0
uvicorn[standard]>=0.27.0
structlog>=24.1.0
```

Example output of `requirements.txt` (trimmed):

```

#
# This file is autogenerated by pip-compile with Python 3.11
# Hashes for the following packages are verified for reproducibility.
#
fastapi==0.109.2 \
    --hash=sha256:a1b2c3... \
    --hash=sha256:d4e5f6...
# via -r requirements.in
uvicorn==0.27.1 \
    --hash=sha256:789abc... \
# via -r requirements.in
structlog==24.1.0 \
    --hash=sha256:def456... \
# via -r requirements.in

```

The `--generate-hashes` flag adds SHA-256 hashes for every wheel/sdist, enabling `--require-hashes` on install to verify integrity.

4.4 uv.lock — the fast path

uv generates and resolves lockfiles at Rust speed:

```

# Initialize a project with uv
uv init my-service
cd my-service

# Add dependencies
uv add fastapi uvicorn[standard] structlog

# The lockfile uv.lock is created automatically
# Install from the lockfile
uv sync

# Or export to requirements.txt for pip users
uv export --no-hashes -o requirements.txt

```

uv.lock is TOML-based, stores the full dependency graph, and resolves in seconds where pip-compile might take minutes.

4.5 Requirements hashing and supply-chain hardening

Hash verification protects against three attack vectors: 1. **Typosquatting** — a malicious package with a name similar to a popular one. 2. **Dependency confusion** — a private package name published to a public index. 3. **PyPI account takeover** — a legitimate package replaced with a malicious version.

```
# Generate hashes for a requirements file
pip-compile requirements.in --generate-hashes

# Install with hash verification – pip rejects any package whose hash
doesn't match
pip install -r requirements.txt --require-hashes --no-deps

# For a private index + hash verification
pip install -r requirements.txt --require-hashes \
    --extra-index-url https://pypi.acme.corp/simple/ \
    --no-index
```

The `--no-index` flag with a vendored wheelhouse makes the install fully hermetic — no network access at all.

5. Environments — venv, conda, and container layering

5.1 venv vs conda/mamba

Aspect	venv	conda/mamba
Scope	Python packages only	Python + non-Python (C libs, R, etc.)
Resolver	pip (or uv)	conda resolver (or mamba/ libmamba)
Python versions	System Python or pyenv	Separate Python installations per env
Speed	Fast (lightweight)	Slow (full solver), mamba is faster
Use case	Backend services, libraries	Data science, ML, system-level deps

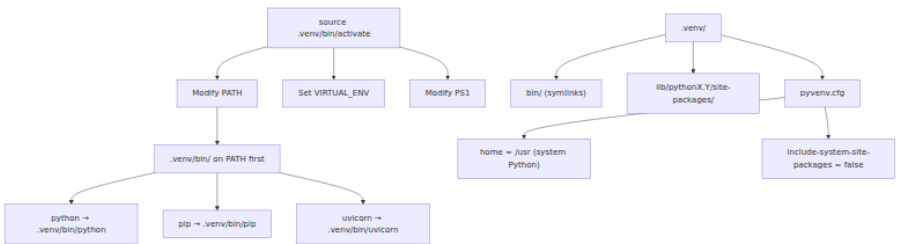
For backend services, venv + pip/uv is the standard. conda is valuable when you need `libopenblas`, `libhdf5`, or other system-level libraries without installing them via the OS package manager.

5.2 venv activation — what actually happens

When you run `source .venv/bin/activate`, a shell script modifies three environment variables:

1. `PATH` — prepends `.venv/bin/` so that `python`, `pip`, `uvicorn`, etc. resolve to the venv's versions.
2. `VIRTUAL_ENV` — set to the venv root, used by tools to detect they are inside a venv.
3. `PS1` — modified to show the venv name in your shell prompt.

Critically, the venv's `python` binary is either a symlink or a copy of the system Python. The venv does not contain a separate Python installation — it contains only `site-packages`, `bin/` symlinks, and `pyvenv.cfg`.



5.3 Python-specific Docker images

Image	Base	Size (approx)	Python	Use case
<code>python:3.12</code>	Debian Bookworm	~1 GB	Full CPython	Development, debugging
<code>python:3.12-slim</code>	Debian Bookworm (minimal)	~150 MB	Full CPython	Production (most common)
<code>python:3.12-alpine</code>	Alpine Linux	~50 MB	Full CPython + musl	Smallest images, musl caveats
<code>gcr.io/distroless/python3-debian12</code>	Distroless	~40 MB	Python only	Minimal attack surface

Image	Base	Size (approx)	Python	Use case
chainguard/python	Wolfi	~40 MB	Python only	Supply-chain hardened

The trade-off is clear: smaller base image means faster pulls and smaller attack surface, but Alpine uses musl (ABI incompatibility with manylinux wheels), and distroless has no shell (harder to debug).

5.4 venv vs container layering



In a container, you typically do *not* use venv — the container itself is the isolation boundary. The base image provides Python, `pip install` adds dependencies, and `COPY` adds your code. Layer ordering matters for cache efficiency: change the layer that changes most often (your code) last.

6. Docker multi-stage builds for Python

Multi-stage builds separate the build environment from the runtime environment. This is critical for Python because build tools (compilers, header files, build dependencies) are large and should not ship in the final image.

6.1 The Dockerfile

```

# Stage 1: Build dependencies and wheel
FROM python:3.12-slim AS builder

WORKDIR /app

# Install build essentials for any C extensions
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Create a virtualenv to isolate from the system Python
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Install dependencies from lockfile (layer cached)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Install the application itself
COPY . .
RUN pip install --no-cache-dir .

# Stage 2: Runtime image
FROM python:3.12-slim AS runtime

# Install runtime-only system libraries
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 \
    tini \
    && rm -rf /var/lib/apt/lists/*

# Copy the virtualenv from the builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Compile bytecode for faster startup
RUN python -m compileall /opt/venv/lib/python3.12/site-packages/ \
    --optimize=1 \
    -q

# Suppress bytecode recompilation at runtime
ENV PYTHONDONTWRITEBYTECODE=1

# Create a non-root user
RUN groupadd -r app && useradd -r -g app -d /app app
WORKDIR /app
USER app

# Health check

```

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD python -c "import urllib.request;
  urllib.request.urlopen('http://localhost:8000/health')"

ENTRYPOINT ["tini", "--"]
CMD ["uvicorn", "acme_service.main:app", "--host", "0.0.0.0", "--
port", "8000", "--workers", "4"]
```

6.2 Key decisions in the Dockerfile

Why venv inside the container? Even though the container is an isolation boundary, using a venv ensures that system Python packages (pip, setuptools) do not interfere with your application's dependencies. It also makes `COPY --from=builder` clean — you copy a self-contained directory, not a mutation of the system Python.

`python -m compileall --optimize=1` pre-compiles `.py` files to `.pyc` bytecode. The `--optimize=1` flag strips assertion bytecode (the `-O` flag), making bytecode slightly smaller. This runs during build, not at runtime, so the cold-start cost is paid once.

`PYTHONDONTWRITEBYTECODE=1` prevents Python from writing `.pyc` files at runtime. Since we pre-compiled bytecode in the build, there is no need to regenerate it — and suppressing writes saves I/O and keeps the filesystem immutable.

`tini` is a minimal init process that handles signal forwarding and zombie reaping. Without it, your Python process becomes PID 1 and must handle `SIGTERM` for graceful shutdown. With `tini`, signals are forwarded correctly.

Layer ordering is optimized for cache: 1. System packages (change rarely). 2. `requirements.txt` + `pip install` (change when dependencies change). 3. Application code + `pip install .` (changes on every commit).

6.3 Slim vs distroless vs Alpine

Criterion	python:slim	python:alpine	distroless
Size	~150 MB	~50 MB	~40 MB
libc	glibc	musl	glibc
Shell	Yes	Yes	No

Criterion	python:slim	python:alpine	distroless
Package manager	apt	apk	None
Debugging	Easy	Easy	<code>debug</code> variant only
manylinux wheels	Yes	No (musl)	Yes
Cold start	Baseline	Slightly faster (less to load)	Fastest (minimal)

Alpine caveat: musl libc means manylinux wheels (which link against glibc) do not work. You must compile everything from source, which increases build time and complexity. For most backend services, `python:slim` is the right choice.

Distroless is ideal for supply-chain hardening: no shell means a compromised package cannot spawn a reverse shell. But debugging requires the `-debug` variant or `kubectrl exec` with a sidecar. Teams comfortable with sidecar debugging patterns get the smallest, most secure runtime.

7. Hermetic builds — no network, no surprises

A hermetic build is one where no external resources are fetched during installation. The dependencies are vendored in a wheelhouse directory, and `pip install` uses `--no-index` to prevent any network access.

7.1 Creating a wheelhouse

```
# Download all wheels to a local directory
pip download -r requirements.txt \
  --dest wheelhouse/ \
  --only-binary=:all: \
  --python-version 3.12 \
  --platform manylinux_2_17_x86_64 \
  --platform musllinux_1_2_x86_64 \
  --platform macosx_14_0_arm64 \
  --platform win_amd64

# Verify all wheels have hashes
pip-compile requirements.in --generate-hashes --output-file=requirement
s-hashed.txt
```

7.2 Installing from the wheelhouse

```
# In the Dockerfile
COPY wheelhouse /wheelhouse
RUN pip install --no-index \
  --find-links=/wheelhouse \
  -r requirements-hashed.txt \
  --require-hashes \
  --no-deps
```

The `--no-deps` flag is important — it tells pip to trust the lockfile and not attempt any further resolution. Combined with `--require-hashes`, this ensures that every installed package was explicitly declared, pinned, and verified.

7.3 Supply-chain threat model

Threat	Mitigation
Typosquatting (e.g., <code>python-dateutil</code> vs <code>python-dateutil2</code>)	Lockfile with hashes — only known packages are installed
Dependency confusion (private name published to PyPI)	<code>--extra-index-url</code> priority + <code>--no-index</code> in production
Compromised maintainer account	Hash pinning + <code>pip-audit</code> in CI + Sigstore verification

Threat	Mitigation
Malicious build script (<code>setup.py</code> code execution)	Build isolation (PEP 517) + no <code>setup.py</code> in pyproject.toml-only projects
Compromised base image	Distroless / Chainguard images + image signing

8. SBOMs and vulnerability scanning

8.1 Software Bill of Materials (SBOM)

An SBOM is a machine-readable inventory of every component in your software. For Python, this includes every direct and transitive dependency, their versions, licenses, and origins.

CycloneDX is the standard format. The `cyclonedx-bom` tool generates SBOMs:

```
# Install the SBOM generator
pip install cyclonedx-bom

# Generate an SBOM from a requirements file
cyclonedx-py environment \
  --output-file sbom.json \
  --format json \
  --spec-version 1.6

# Or generate from a pip freeze
pip freeze --exclude-editable | cyclonedx-py requirements \
  --output-file sbom.json \
  --format json \
  --spec-version 1.6
```

8.2 pip-audit — vulnerability scanning

`pip-audit` checks installed packages against the OSV (Open Source Vulnerabilities) database:

```
# Install pip-audit
pip install pip-audit

# Audit installed packages
pip-audit

# Audit a requirements file (without installing)
pip-audit -r requirements.txt

# Audit with hashes for supply-chain verification
pip-audit -r requirements.txt --require-hashes

# Output as CycloneDX SBOM
pip-audit --format cyclonedx-json --output sbom-vuln.json
```

8.3 CI pipeline — pip-compile + pip-audit

```

# .github/workflows/security.yml
name: Supply Chain Security

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: "0 6 * * 1" # Weekly audit

jobs:
  compile-and-audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install tools
        run: pip install pip-tools pip-audit cyclonedx-bom

      - name: Compile requirements with hashes
        run: |
          pip-compile requirements.in \
            --output-file=requirements.txt \
            --generate-hashes

      - name: Install from pinned requirements
        run: pip install -r requirements.txt --require-hashes

      - name: Audit installed packages
        run: pip-audit --strict --progress-spinner off

      - name: Audit requirements file
        run: pip-audit -r requirements.txt --require-hashes --strict

      - name: Generate SBOM
        run: |
          cyclonedx-py environment \
            --output-file sbom.json \
            --format json \
            --spec-version 1.6

      - name: Upload SBOM
        uses: actions/upload-artifact@v4
        with:
          name: sbom

```

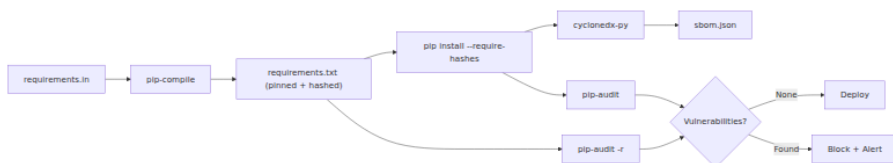
```

    path: sbom.json

- name: Check lockfile is up to date
  run: |
    pip-compile requirements.in \
      --output-file=requirements-check.txt \
      --generate-hashes
    diff requirements.txt requirements-check.txt || \
      (echo "Lockfile is stale – run pip-compile" && exit 1)

```

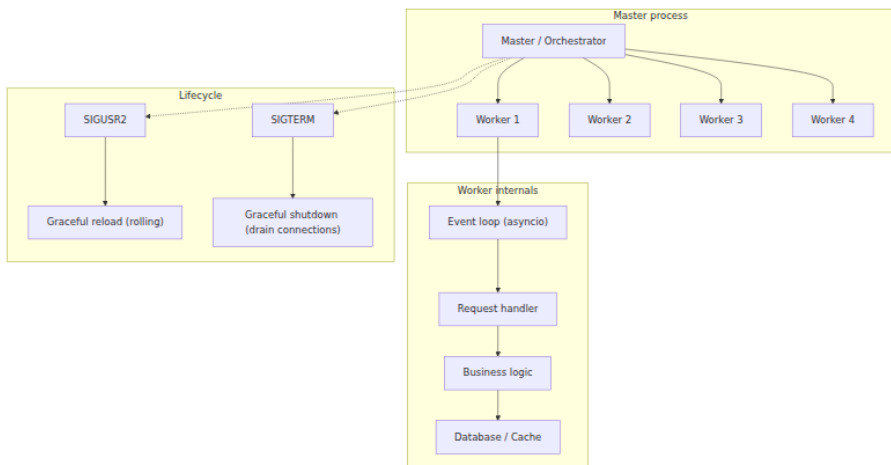
This pipeline does four things: 1. Compiles a fresh lockfile with hashes. 2. Installs from the lockfile and audits both the installed environment and the lockfile. 3. Generates a CycloneDX SBOM for compliance and tracking. 4. Verifies that the checked-in lockfile is up to date (catches stale lockfiles).



9. Deployment at scale — gunicorn, uvicorn, and the process model

9.1 The worker process model

A Python web service at scale runs multiple worker processes, each with its own GIL, memory space, and event loop (for async frameworks). The process manager (gunicorn, uvicorn with workers, or Kubernetes) handles lifecycle:



9.2 gunicorn configuration

```
# gunicorn.conf.py – production configuration
import multiprocessing
import os

# Server mechanics
bind = os.getenv("BIND", "0.0.0.0:8000")
workers = int(os.getenv("WEB_CONCURRENCY", multiprocessing.cpu_count()
* 2 + 1))
worker_class = "uvicorn.workers.UvicornWorker"
worker_connections = 1000
timeout = 120
graceful_timeout = 30
keepalive = 5

# Preloading – import the application in the master, fork into workers
preload_app = True

# Logging
loglevel = os.getenv("LOG_LEVEL", "info")
accesslog = "-" # stdout
errorlog = "-" # stderr
access_log_format = '%(h)s %(l)s %(u)s %(t)s "%(r)s" %(s)s %(b)s "%(f)s" "%(a)s" %(D)s'

# Worker recycling – prevent memory leaks
max_requests = 10000
max_requests_jitter = 1000
```


Key decisions:

- `preload_app = True` imports the application in the master process before forking. This means:
- The import graph is resolved once, not per-worker.
- Memory is shared (copy-on-write) across workers for imported modules.
- But: any import-time side effects run in the master — if the master crashes during import, no workers start.
- `worker_class = "uvicorn.workers.UvicornWorker"` runs ASGI applications (FastAPI, Starlette) inside gunicorn. Gunicorn handles process management; uvicorn handles the ASGI protocol.
- `max_requests = 10000` restarts workers after 10k requests, preventing slow memory leaks. The jitter (± 1000) prevents all workers from restarting simultaneously.

9.3 uvicorn standalone configuration

For ASGI-only deployments, uvicorn can manage workers directly:

```
uvicorn acme_service.main:app \  
  --host 0.0.0.0 \  
  --port 8000 \  
  --workers 4 \  
  --loop uvloop \  
  --http httptools \  
  --log-level info \  
  --access-log
```

uvloop and httptools are C extensions that accelerate the event loop and HTTP parsing respectively — typically 2-4x faster than the pure-Python asyncio event loop for I/O-bound workloads.

9.4 Health probes

Kubernetes uses liveness, readiness, and startup probes to manage traffic routing:

```

# acme_service/main.py
from fastapi import FastAPI
from contextlib import asynccontextmanager
import asyncio

app = FastAPI()

# Application state – set during startup
_app_ready = False
_db_connected = False

@asynccontextmanager
async def lifespan(app: FastAPI):
    global _app_ready, _db_connected
    # Startup: connect to DB, load ML models, etc.
    _db_connected = await connect_to_database()
    _app_ready = True
    yield
    # Shutdown: close connections, flush buffers
    await close_database()

app = FastAPI(lifespan=lifespan)

@app.get("/health")
async def health():
    """Liveness probe – is the process alive and responsive?"""
    return {"status": "ok"}

@app.get("/ready")
async def readiness():
    """Readiness probe – is the service ready to serve traffic?"""
    if not _app_ready:
        return {"status": "not_ready", "reason": "application not initialized"}, 503
    if not _db_connected:
        return {"status": "not_ready", "reason": "database not connected"}, 503
    return {"status": "ok"}

@app.get("/startup")
async def startup():
    """Startup probe – has the application finished initializing?"""
    if _app_ready:
        return {"status": "ok"}
    return {"status": "starting"}, 503

```

In Kubernetes:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8000
  initialDelaySeconds: 10
  periodSeconds: 30
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /ready
    port: 8000
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
startupProbe:
  httpGet:
    path: /startup
    port: 8000
  periodSeconds: 5
  failureThreshold: 30 # 150s max startup time
```

The startup probe allows slow-starting services (loading large ML models, establishing connection pools) without being killed by the liveness probe.

9.5 Autoscaling with JIT warmup

Python services have a warm-up phase after startup: JIT compilers (PyPy, or Python's own adaptive interpreter) need to observe hot code paths before optimizing. ML services need to load models into GPU memory. The import graph itself takes time to resolve (as covered in Chapter 9).

JIT warmup strategy: trigger representative traffic through the service before marking it ready, so the JIT has seen real patterns.

```

# Warmup endpoint – called by load balancer or orchestrator
@app.post("/warmup")
async def warmup():
    """Execute representative code paths to trigger JIT
    optimization."""
    from acme_service.analytics import compute_metrics
    from acme_service.serialization import serialize_response
    from acme_service.validation import validate_payload

    # Run 100 representative iterations
    for _ in range(100):
        result = compute_metrics(sample_payload)
        serialized = serialize_response(result)
        validate_payload(sample_payload)

    return {"status": "warmed_up", "iterations": 100}

```

For Kubernetes autoscaling (HPA), use custom metrics to trigger scale-up *before* latency degrades:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: acme-service
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: acme-service
  minReplicas: 3
  maxReplicas: 50
  metrics:
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
        target:
          type: AverageValue
          averageValue: "1000"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Pods
          value: 4
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 10
          periodSeconds: 120

```

The scale-down stabilization window (300s) is intentionally slow to prevent flapping and to allow JIT-warmed instances to absorb traffic spikes.

10. Observability — OpenTelemetry and structlog

10.1 OpenTelemetry for Python

OpenTelemetry (OTel) is the vendor-neutral observability standard. For Python, the SDK provides automatic instrumentation for common libraries (FastAPI, requests, SQLAlchemy, etc.) and manual instrumentation for custom code.

```

# acme_service/telemetry.py
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter

from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor
from opentelemetry.instrumentation.sqlalchemy import SQLAlchemyInstrumentor

def setup_telemetry(service_name: str, otlp_endpoint: str = "http://otel-collector:4317"):
    """Initialize OpenTelemetry tracing and auto-instrumentation."""
    # Configure the tracer
    provider = TracerProvider(resource={"service.name": service_name})
    exporter = OTLPSpanExporter(endpoint=otlp_endpoint)
    provider.add_span_processor(BatchSpanProcessor(exporter))
    trace.set_tracer_provider(provider)

    # Auto-instrument common libraries
    FastAPIInstrumentor.instrument()
    HTTPXClientInstrumentor.instrument()
    # SQLAlchemyInstrumentor.instrument(engine=db_engine) # after engine creation

    return provider

```

10.2 structlog — structured logging

structlog produces structured (JSON) logs that are machine-parseable and correlated with traces via trace IDs:

```

# acme_service/logging_config.py
import structlog
import logging
from opentelemetry import trace

def add_trace_context(logger, method_name, event_dict):
    """Inject current trace/span IDs into structured logs."""
    span = trace.get_current_span()
    ctx = span.get_span_context()
    if ctx and ctx.trace_id:
        event_dict["trace_id"] = format(ctx.trace_id, "032x")
        event_dict["span_id"] = format(ctx.span_id, "016x")
    return event_dict

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        add_trace_context,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.add_log_level,
        structlog.processors.StackInfoRenderer(),
        structlog.processors.format_exc_info,
        structlog.processors.JSONRenderer(),
    ],
    wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),
    context_class=dict,
    logger_factory=structlog.PrintLoggerFactory(),
    cache_logger_on_first_use=True,
)

# Usage in application code
log = structlog.get_logger()

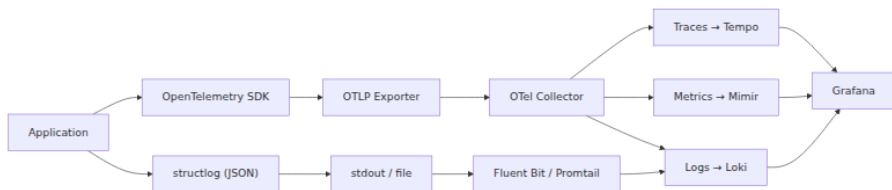
@app.post("/api/v1/events")
async def create_event(payload: EventPayload):
    log.info("event_received", event_type=payload.type, size=len(payload.data))
    result = await process_event(payload)
    log.info("event_processed", event_type=payload.type, duration_ms=result.duration_ms)
    return result

```

The correlation between traces (OpenTelemetry) and logs (structlog) via `trace_id` and `span_id` is what makes debugging distributed systems

feasible — you can jump from a log line to the full trace, or from a trace to all related logs.

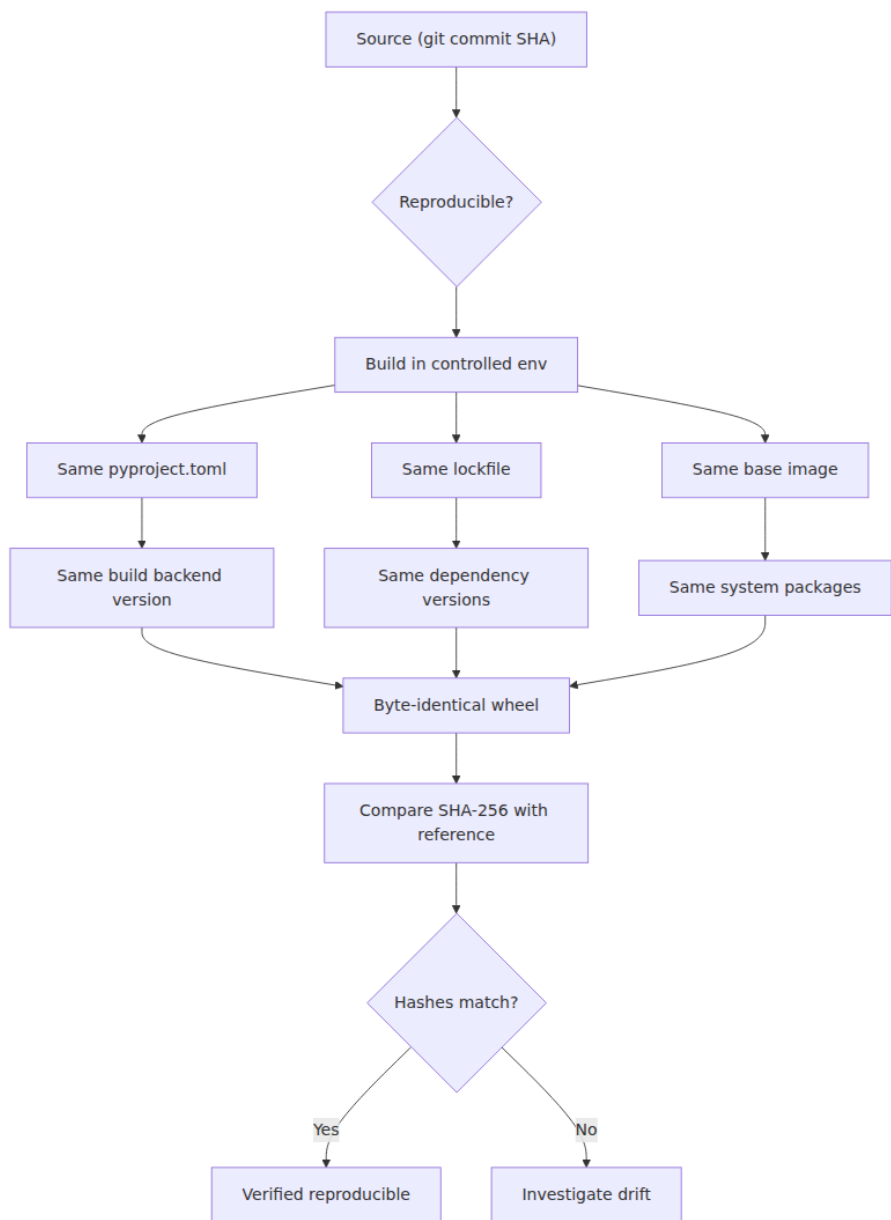
10.3 The full observation stack



11. Reproducible build verification

Reproducibility means: given the same source, the same environment, and the same tools, you produce the same artifact. For Python, this requires controlling:

1. **Source** — same commit, same working tree.
2. **Build dependencies** — same versions of the build backend and its plugins.
3. **System dependencies** — same OS, same libc, same compilers.
4. **Environment variables** — `SOURCE_DATE_EPOCH`, `PYTHONDONTWRITEBYTECODE`, `LC_ALL=C`.
5. **Timestamps** — `SOURCE_DATE_EPOCH` pins the modification time in archives and pyc headers.



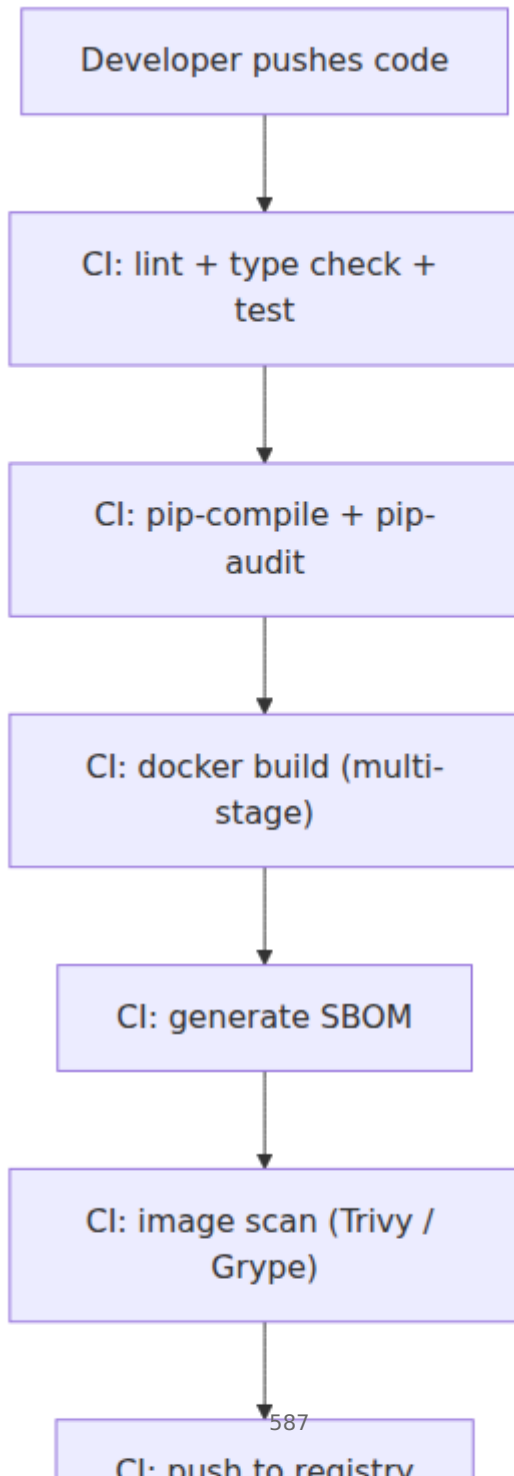
Verification command:

```
# Build twice and compare
SOURCE_DATE_EPOCH=0 python -m build --wheel -o dist1/
SOURCE_DATE_EPOCH=0 python -m build --wheel -o dist2/

# Compare checksums
sha256sum dist1/*.whl dist2/*.whl
# Should be identical
```

12. End-to-end deployment flow

Putting it all together: a modern Python service goes from code to production through this pipeline:



Key takeaways

- **pyproject.toml is the single source of truth** for Python project metadata. PEP 517 (build API), PEP 518 (build dependencies), PEP 621 (project metadata), and PEP 660 (editable installs) define the complete contract. Use hatchling or uv for clean pyproject.toml-only projects; setuptools for legacy compatibility.
- **Wheels are the installable unit; sdists are the source-of-truth.** Pure-Python wheels (py3-none-any) are universal. Binary wheels carry platform tags (manylinux_2_17_x86_64) that encode glibc minimums. The stable ABI (abi3) lets one wheel cover all future CPython versions.
- **Lockfiles with hashes are non-negotiable for production.** pip-compile --generate-hashes or uv.lock captures the exact dependency graph. --require-hashes on install rejects tampered packages. This is your primary defense against typosquatting, dependency confusion, and account takeover.
- **Docker multi-stage builds separate build from runtime.** Build dependencies (compilers, headers) stay in the builder stage; only the virtualenv and runtime libraries ship in the final image. Pre-compile bytecode (compileall --optimize=1) and suppress runtime writes (PYTHONDONTWRITEBYTECODE=1) for faster cold starts.
- **Hermetic builds (--no-index + vendored wheelhouse) eliminate network surprises.** Combined with hash verification, they produce fully reproducible installations. The --no-deps flag tells pip to trust the lockfile without resolving.
- **SBOMs and vulnerability scanning are not optional.** cyclonedx-bom generates machine-readable inventories; pip-audit checks against the OSV database. Integrate both into CI with a stale-lockfile check to catch drift.
- **gunicorn + uvicorn workers with preloading give you process isolation + async performance.** preload_app = True shares the import graph across workers via copy-on-write. max_requests with jitter prevents slow memory leaks. Health probes (/health, /ready, /startup) map to Kubernetes liveness, readiness, and startup probes.

- **Autoscaling needs JIT warmup.** Python's adaptive interpreter, JIT compilers, and ML model loading all require warm-up traffic. Trigger representative requests during startup, and configure HPA with custom metrics and generous scale-down stabilization.
 - **Observability requires correlated traces + logs.** OpenTelemetry provides traces; structlog provides structured logs. Inject `trace_id` and `span_id` into every log line to enable cross-signal correlation.
-

Further reading

Pinned references (start here):

- [PEP 517 — Build backend API](#) — defines the interface that build backends must implement (`build_wheel`, `build_sdist`, `get_requires_for_build_wheel`).
- [PEP 518 — pyproject.toml and build dependencies](#) — defines the `[build-system]` table and how build dependencies are specified.
- [PEP 621 — Project metadata in pyproject.toml](#) — defines the `[project]` table for all project metadata.
- [PEP 660 — Editable installs](#) — defines the editable install protocol for build backends.
- [PEP 600 — Flexible manylinux platform tags](#) — replaces named manylinux versions with explicit glibc minimums (`manylinux_2_17`).
- [pip-audit](#) — vulnerability scanner for Python packages, backed by the OSV database.
- [packaging.python.org](#) — the official Python packaging user guide, covering pyproject.toml, build backends, and distribution.
- [CycloneDX Python](#) — SBOM generation for Python environments.

Additional references:

- [manylinux](#) — the manylinux Docker images and auditwheel tooling for building portable Linux wheels.
- [auditwheel](#) — repair and audit Linux wheels for manylinux compliance.
- [uv](#) — Rust-based Python package manager, resolver, and build backend.
- [structlog](#) — structured logging library for Python.

- [OpenTelemetry Python](#) — auto-instrumentation and manual tracing for Python services.
- [gunicorn](#) — Python WSGI HTTP server with preloading, worker management, and graceful reload.
- [Docker best practices for Python](#) — official Docker guidance on layer caching, multi-stage builds, and image size.